

CS 583: Approximation Algorithms

Chandra Chekuri¹

May 4, 2026

¹Dept. of Computer Science, University of Illinois, Urbana, IL 61820. Email: chekuri@illinois.edu. Work on these notes partially supported by NSF grants over the years. Comments, suggestions, and corrections are welcome.

Contents

1	Introduction	7
1.1	Formal Aspects	10
1.1.1	NP Optimization Problems	10
1.1.2	Relative Approximation	11
1.1.3	Additive Approximation	11
1.1.4	Hardness of Approximation	12
1.2	Designing Approximation Algorithms	14
2	Covering Problems	16
2.1	Greedy for SET COVER and MAXIMUM COVERAGE	17
2.1.1	Greedy Algorithm	18
2.1.2	Analysis of GREEDY COVER	18
2.1.3	DOMINATING SET	21
2.2	VERTEX COVER	22
2.2.1	A 2-approximation for VERTEX COVER	22
2.2.2	SET COVER with small frequencies	23
2.3	Vertex Cover via LP	24
2.4	Set Cover via LP	26
2.4.1	Deterministic Rounding	27
2.4.2	Randomized Rounding	28
2.4.3	Dual-fitting	32
2.4.4	Greedy for implicit instances of SET COVER	34
2.5	Submodularity	34
2.5.1	SUBMODULAR SET COVER	35
2.5.2	SUBMODULAR MAXIMUM COVERAGE	36
2.6	Covering Integer Programs (CIPs)	36
3	Knapsack	38
3.1	The Knapsack Problem	38
3.1.1	A Greedy Algorithm	39

3.1.2	A Polynomial Time Approximation Scheme	40
3.1.3	Rounding and Scaling	42
3.2	Other Problems	43
4	Packing Problems	44
4.0.1	Rounding and scaling of weights	45
4.1	Maximum Independent Set Problem in Graphs	45
4.1.1	Elimination Orders and MIS and MWIS	49
4.1.2	MWIS in k -perfectly orientable graphs via random rounding plus alteration	54
4.2	The efficacy of the Greedy algorithm for a class of Independence Families	56
4.3	Randomized Rounding with Alteration for Packing Problems	57
4.4	Packing Integer Programs (PIPs)	60
4.4.1	Randomized Rounding with Alteration for PIPs	61
4.5	Bibliographic Notes	66
5	Load Balancing and Bin Packing	67
5.1	Load Balancing / MultiProcessor Scheduling	67
5.1.1	Problem Description	68
5.1.2	Greedy Algorithm	68
5.1.3	A PTAS for Multi-Processor Scheduling	70
5.1.4	Section Notes	72
5.2	Bin Packing	73
5.2.1	Problem Description	73
5.2.2	Greedy Approaches	73
5.2.3	(Asymptotic) PTAS for Bin Packing	74
5.2.4	Asymptotic PTAS for Bin Packing	76
5.2.5	Section Notes	77
6	Unrelated Machine Scheduling and Generalized Assignment	78
6.1	Scheduling on Unrelated Parallel Machines	78
6.2	Generalized Assignment Problem	82
6.2.1	Shmoys-Tardos Rounding	84
6.2.2	Iterative Rounding	88
6.3	Maximization version of GAP	92
6.4	Configuration LP: A Stronger LP Relaxation	93
6.5	Bibliographic Notes	96

7	Congestion Minimization in Networks	98
7.1	Congestion Minimization and VLSI Routing	98
7.2	Min-max Integer Programs	104
8	Introduction to Local Search	106
8.1	Local Search for MAX CUT	107
8.2	Local Search for Submodular Function Maximization	111
9	Clustering and Facility Location	114
9.1	k -Center	115
9.1.1	Gonzalez's algorithm and nets in metric spaces	116
9.1.2	Hochbaum-Shmoys bottleneck approach	117
9.1.3	Related Problems and Discussion	118
9.2	Uncapacitated Facility Location	119
9.2.1	LP Rounding	119
9.2.2	Primal-Dual	121
9.2.3	Local Search	131
9.3	k -Median	131
9.3.1	Local Search	132
9.4	k -Means	136
9.5	Lloyd's algorithm, D^2 -sampling and k -Means ++	137
9.6	Bibliographic Notes	139
10	Introduction to Network Design	140
10.1	The Steiner Tree Problem	141
10.1.1	The MST Algorithm	142
10.1.2	The Greedy/Online Algorithm	144
10.1.3	LP Relaxation	146
10.1.4	Other Results on Steiner Trees	146
10.2	The Traveling Salesperson Problem (TSP)	147
10.2.1	TSP in Undirected Graphs	147
10.2.2	LP Relaxation	151
10.2.3	TSP in Directed Graphs	152
10.2.4	LP Relaxation	154
11	Steiner Forest Problem	156
11.1	LP Relaxation	157
11.2	Warm-up: Primal-Dual for Steiner Tree	158
11.3	Primal-Dual Algorithm for Steiner Forest	160

12 Primal Dual for Constrained Forest Problems	167
12.1 Classes of Functions and Setup	168
12.2 A Primal-Dual Algorithm for Covering Uncrossable Functions . .	171
12.2.1 Proof of the main structural lemma	174
13 Survivable Network Design Problem	178
13.1 Augmentation approach	179
13.2 Iterated rounding based 2-approximation	182
13.2.1 Basic feasible solutions and laminar family of tight sets . .	184
13.2.2 Counting argument	186
14 Introduction to Cut and Partitioning Problems	192
14.1 s - t mincut via LP Rounding and Maxflow-Mincut	192
14.2 A Catalog of Cut and Partitioning Problems	195
15 Multiway Cut	198
15.1 Isolating Cut Heuristic	199
15.2 Distance based LP Relaxation	199
15.3 A Partitioning View and Geometric Relaxation	201
15.3.1 CKR Relaxation	203
15.3.2 Rounding Algorithm and Analysis	206
15.3.3 The CKR Rounding	209
15.4 Node-weighted Multiway Cut	210
15.5 Directed Multiway Cut	211
16 k-Cut and Steiner k-Cut	215
16.1 Gomory-Hu Trees	216
16.2 A Gomory-Hu Tree Based Algorithm for k -Cut	216
16.3 Greedy Splitting for Steiner k -Cut	218
17 Multicut	223
17.1 Upper Bound on the Integrality Gap	224
17.2 Lower Bound on the Integrality Gap	229
17.2.1 Expander Graphs	229
17.2.2 The Multicut Instance	232
18 Low Diameter Decomposition and Metric Tree Embeddings	234
18.1 Low Diameter Decompositions	234
18.2 CKR algorithm for weak diameter guarantee	235
18.2.1 A subtle modification in the algorithm and guarantee . . .	236
18.3 Randomized region growing for a strong diameter guarantee . .	237
18.4 Deterministic region growing and duality	238

18.4.1	Using LP Duality to Obtain an LDD	241
18.5	LDDs and Probabilistic Embedding of Metrics into Dominating Tree Metrics	242
18.5.1	Low stretch trees via LDDs	244
18.5.2	An optimal construction	247
19	Sparsest Cut	248
19.1	LP Relaxation and Maximum Concurrent Flow	250
19.2	Rounding the LP via Connection to Multicut	252
19.3	Rounding via ℓ_1 embeddings	254
19.3.1	A digression through trees	254
19.3.2	Cut metrics, line metrics, and ℓ_1 metrics	255
19.3.3	Brief introduction to metric embeddings	256
19.3.4	Utilizing the ℓ_1 embedding	257
19.3.5	Line embeddings, ℓ_1 embeddings and tree embeddings	259
19.4	A Proof Sketch of Bourgain's Theorem	261
19.5	SDP Relaxation	264
19.6	Spectral Relaxation for Conductance	265
19.7	Node capacities and Directed Graphs	265
20	Semidefinite Programming and Approximation	268
20.1	Convex Programming and Semidefinite Matrices	269
20.2	Semidefinite Programming (SDP)	272
20.2.1	SDP and Vector Programming	273
20.3	SDP for Max-Cut: The GW Algorithm	274
20.3.1	Rounding: The Random Hyperplane Algorithm	275
20.4	SDP for Constraint Satisfaction Problems	278
20.4.1	Warm-Up: MAX-2-SAT	278
20.4.2	A Generic SDP for MAX-CSPs	282
20.4.3	Raghavendra's Theorem	288
20.5	Graph Coloring via SDP	289
20.5.1	A Simple $O(\sqrt{n})$ -Coloring	289
20.5.2	Vector Chromatic Number	290
20.5.3	SDP-Based Coloring: $O(\Delta^a \log n)$ Colors	291
20.5.4	The Lovász Theta Function	292
20.6	Metric Embeddings via SDP	294
20.6.1	Finding the Optimal ℓ_2 Embedding	295
20.6.2	Embedding into Negative-type Metric	296
20.6.3	Application to Sparsest Cut	297
20.7	Sum-of-Squares and Polynomial Optimization	299
20.7.1	Checking SOS via SDP	301

20.7.2	Hilbert's 17th Problem	304
20.7.3	Polynomial Optimization with Constraints: The Lasserre Hierarchy	305
20.7.4	Dual Viewpoint: Moments and Pseudo-Distributions . . .	307
20.7.5	Sherali–Adams on the Boolean Hypercube	312
20.7.6	Boolean CSPs as a Common Framework	313
21	Hardness of Approximation	316
21.1	Direct Gap Reductions	317
21.2	Maximum Directed Edge-Disjoint Paths	318
21.2.1	Gap Amplification for Directed EDP	318
21.3	Gap Amplification for MAX-CLIQUE	322
21.3.1	Randomized graph products	324
21.4	APX-Hardness of MAX-3-SAT from PCP Theorem	326
21.4.1	From MAX-3-SAT to MAX-INDEPENDENT-SET	328
A	Basic Feasible Solutions to LPs and the Rank Lemma	354
A.0.1	Some Examples	355
A.0.2	Connection to Caratheodary's Theorem	356
B	Probabilistic Inequalities	357

Chapter 1

Introduction

These are lecture notes for a course on approximation algorithms.

Course Objectives

1. To appreciate that not all intractable problems are the same. **NP** optimization problems, identical in terms of exact solvability, can appear very different from the approximation point of view. This sheds light on why, in practice, some optimization problems (such as **KNAPSACK**) are easy, while others (like **CLIQUE**) are extremely difficult.
2. To learn techniques for design and analysis of approximation algorithms, via some fundamental problems.
3. To build a toolkit of broadly applicable algorithms/heuristics that can be used to solve a variety of problems.
4. To understand reductions between optimization problems, and to develop the ability to relate new problems to known ones.

The complexity class **P** contains the set of problems that can be solved in polynomial time. From a theoretical viewpoint, this describes the class of tractable problems, that is, problems that can be solved efficiently. The class **NP** is the set of problems that can be solved in *non-deterministic* polynomial time, or equivalently, problems for which a solution can be *verified* in polynomial time. **NP** contains many interesting problems that often arise in practice, but there is good reason to believe $\mathbf{P} \neq \mathbf{NP}$. That is, it is unlikely that there exist algorithms to solve **NP** optimization problems efficiently, and so we often resort to heuristic methods to solve these problems.

Heuristic approaches include backtrack search and its variants, mathematical programming methods, local search, genetic algorithms, tabu search, simulated

annealing etc. Some methods are guaranteed to find an optimal solution, though they may take exponential time; others are guaranteed to run in polynomial time, though they may not return a (optimal) solution. Approximation algorithms are (typically) polynomial time heuristics that do not always find an optimal solution but they are distinguished from general heuristics in providing guarantees on the *quality* of the solution they output.

Approximation Ratio: To give a guarantee on solution quality, one must first define what we mean by the quality of a solution. We discuss this more carefully later. For now, note that each *instance* of an optimization problem has a set of feasible solutions. The optimization problems we consider have an **objective function** which assigns a (real/rational) number/value to each feasible solution of each instance I . The goal is to find a feasible solution with minimum objective function value or maximum objective function value. The former problems are minimization problems and the latter are maximization problems.

For each instance I of a problem, let $\text{OPT}(I)$ denote the value of an optimal solution to instance I . We say that an algorithm $\mathcal{A}$ is an **α -approximation algorithm** for a problem if, for *every* instance I , the value of the feasible solution returned by $\mathcal{A}$ is within a (multiplicative) factor of α of $\text{OPT}(I)$. Equivalently, we say that $\mathcal{A}$ is an approximation algorithm with **approximation ratio α** . For a minimization problem we would have $\alpha \geq 1$ and for a maximization problem we would have $\alpha \leq 1$. However, it is not uncommon to find in the literature a different convention for maximization problems where one says that $\mathcal{A}$ is an α -approximation algorithm if the value of the feasible solution returned by $\mathcal{A}$ is at least $\frac{1}{\alpha} \cdot \text{OPT}(I)$; the reason for using this convention is so that approximation ratios for both minimization and maximization problems will be ≥ 1 . In this course we will for the most part use the convention that $\alpha \geq 1$ for minimization problems and $\alpha \leq 1$ for maximization problems.

Remarks:

1. The approximation ratio of an algorithm for a minimization problem is the *maximum* (or supremum), over all instances of the problem, of the ratio between the values of solution returned by the algorithm and the optimal solution. Thus, it is a bound on the *worst-case* performance of the algorithm.
2. The approximation ratio α can depend on the size of the instance I , so one should technically write $\alpha(|I|)$.
3. A natural question is whether the approximation ratio should be defined in an *additive* sense. For example, an algorithm has an α -approximation for a minimization problem if it outputs a feasible solution of value at most

$\text{OPT}(I) + \alpha$ for all I . This is a valid definition and is the more relevant one in some settings. However, for many **NP** problems it is easy to show that one cannot obtain any interesting additive approximation (unless of course $P = NP$) due to scaling issues. We will illustrate this via an example later.

Pros and cons of the approximation approach: Some advantages to the approximation approach include:

1. It explains why problems can vary considerably in difficulty.
2. The analysis of problems and problem instances distinguishes easy cases from difficult ones.
3. The worst-case ratio is *robust* in many ways. It allows *reductions* between problems.
4. Approximation algorithmic ideas/tools/relaxations are valuable in developing heuristics, including many that are practical and effective.
5. Quantification of performance via a concrete metric such as the approximation ratio allows for innovation in algorithm design and has led to many new ideas.

As a bonus, many of the ideas are beautiful and sophisticated, and involve connections to other areas of mathematics and computer science.

Disadvantages include:

1. The focus on *worst-case measures* risks ignoring algorithms or heuristics that are practical or perform well *on average*.
2. Unlike, for example, integer programming, there is often no incremental/continuous tradeoff between the running time and quality of solution.
3. Approximation algorithms are often limited to cleanly stated problems.
4. The framework does not (at least directly) apply to decision problems or those that are inapproximable.

Approximation as a broad lens

The use of approximation algorithms is not restricted solely to **NP**-Hard optimization problems. In general, ideas from approximation can be used to solve many problems where finding an exact solution would require too much of any resource.

A resource we are often concerned with is *time*. Solving NP-Hard problems exactly would (to the best of our knowledge) require exponential time, and so we may want to use approximation algorithms. However, for large data sets, even polynomial running time is sometimes unacceptable. As an example, the best exact algorithm known for the MATCHING problem in general graphs requires $O(m\sqrt{n})$ time; on large graphs, this may not be practical. In contrast, a simple greedy algorithm takes near-linear time and outputs a matching of cardinality at least $1/2$ that of the maximum matching; moreover there have been randomized sub-linear time algorithms as well.

Another often limited resource is *space*. In the area of data streams/streaming algorithms, we are often only allowed to read the input in a single pass, and given a small amount of additional storage space. Consider a network switch that wishes to compute statistics about the packets that pass through it. It is easy to exactly compute the average packet length, but one cannot compute the median length exactly. Surprisingly, though, many statistics can be approximately computed.

Other resources include programmer time (as for the MATCHING problem, the exact algorithm may be significantly more complex than one that returns an approximate solution), or communication requirements (for instance, if the computation is occurring across multiple locations).

1.1 Formal Aspects

1.1.1 NP Optimization Problems

In this section, we cover some formal definitions related to approximation algorithms. We start from the definition of optimization problems. A problem is simply an infinite collection of *instances*. Let Π be an optimization problem. Π can be either a minimization or maximization problem. Instances I of Π are a subset of Σ^* where Σ is a finite encoding alphabet. For each instance I there is a set of feasible solutions $\mathcal{S}(I)$. We restrict our attention to real/rational-valued optimization problems; in these problems each feasible solution $S \in \mathcal{S}(I)$ has a value $val(S, I)$. For a minimization problem Π the goal is, given I , find $\text{OPT}(I) = \min_{S \in \mathcal{S}(I)} val(S, I)$.

Now let us formally define NP optimization (NPO) which is the class of optimization problems corresponding to NP.

Definition 1.1. Π is in NPO if

- Given $x \in \Sigma^*$, there is a polynomial-time algorithm that decide if x is a valid instance of Π . That is, we can efficiently check if the input string is well-formed. This is a basic requirement that is often not spelled out.

- For each I , and $S \in \mathcal{S}(I)$, $|S| \leq \text{poly}(|I|)$. That is, the solution are of size polynomial in the input size.
- There exists a poly-time decision procedure that for each I and $S \in \Sigma^*$, decides if $S \in \mathcal{S}(I)$. This is the key property of NP; we should be able to verify solutions efficiently.
- $\text{val}(I, S)$ is a polynomial-time computable function.

We observe that for a minimization NPO problem Π , there is a associated natural decision problem $L(\Pi) = \{(I, B) : \text{OPT}(I) \leq B\}$ which is the following: given instance I of Π and a number B , is the optimal value on I at most B ? For maximization problem Π we reverse the inequality in the definition.

Lemma 1.1. $L(\Pi)$ is in NP if Π is in NPO.

1.1.2 Relative Approximation

When Π is a minimization problem, recall that we say an approximation algorithm $\mathcal{A}$ is said to have approximation ratio α iff

- $\mathcal{A}$ is a polynomial time algorithm
- for all instance I of Π , $\mathcal{A}$ produces a feasible solution $\mathcal{A}(I)$ s.t. $\text{val}(\mathcal{A}(I), I) \leq \alpha \text{val}(\text{OPT}(I), I)$. (Note that $\alpha \geq 1$.)

Approximation algorithms for maximization problems are defined similarly. An approximation algorithm $\mathcal{A}$ is said to have approximation ratio α iff

- $\mathcal{A}$ is a polynomial time algorithm
- for all instance I of Π , $\mathcal{A}$ produces a feasible solution $\mathcal{A}(I)$ s.t. $\text{val}(\mathcal{A}(I), I) \geq \alpha \text{val}(\text{OPT}(I), I)$. (Note that $\alpha \leq 1$.)

For maximization problems, it is also common to see use $1/\alpha$ (which must be ≥ 1) as approximation ratio.

1.1.3 Additive Approximation

Note that all the definitions above are about relative approximations; one could also define *additive* approximations. $\mathcal{A}$ is said to be an α -additive approximation algorithm, if for all I , $\text{val}(\mathcal{A}(I)) \leq \text{OPT}(I) + \alpha$. Most NPO problems, however, do not allow any additive approximation ratio because $\text{OPT}(I)$ has a scaling property.

To illustrate the scaling property, let us consider Metric-TSP. Given an instance I , let I_β denote the instance obtained by increasing all edge costs by a factor of β . It is easy to observe that for each $S \in \mathcal{S}(I) = \mathcal{S}(I_\beta)$, $val(S, I_\beta) = \beta val(S, I)$ and $OPT(I_\beta) = \beta OPT(I)$. Intuitively, scaling edge by a factor of β scales the value by the same factor β . Thus by choosing β sufficiently large, we can essentially make the additive approximation (or error) negligible.

Lemma 1.2. *Metric-TSP does not admit an α additive approximation algorithm for any polynomial-time computable α unless $P = NP$.*

Proof. For simplicity, suppose every edge has integer cost. For the sake of contradiction, suppose there exists an additive α approximation $\mathcal{A}$ for Metric-TSP. Given I , we run the algorithm on I_β and let S be the solution, where $\beta = 2\alpha$. We claim that S is the optimal solution for I . We have $val(S, I) = val(S, I_\beta)/\beta \leq OPT(I_\beta)/\beta + \alpha/\beta = OPT(I) + 1/2$, as $\mathcal{A}$ is α -additive approximation. Thus we conclude that $OPT(I) = val(S, I)$, since $OPT(I) \leq val(S, I)$, and $OPT(I), val(S, I)$ are integers. This is impossible unless $P = NP$. ■

Now let us consider two problems which allow additive approximations. In the Planar Graph Coloring, we are given a planar graph $G = (V, E)$. We are asked to color all vertices of the given graph G such that for any $vw \in E$, v and w have different colors. The goal is to minimize the number of different colors. It is known that to decide if a planar graph admits 3-coloring is NP-complete [239], while one can always color any planar graph G with using 4 colors (this is the famous 4-color theorem) [15, 246]. Further, one can efficiently check whether a graph is 2-colorable (that is, if it is bipartite). Thus, the following algorithm is a 1-additive approximation for Planar Graph Coloring: If the graph is bipartite, color it with 2 colors; otherwise, color with 4 colors.

As a second example, consider the Edge Coloring Problem, in which we are asked to color edges of a given graph G with the minimum number of different colors so that no two adjacent edges have the same color. By Vizing's theorem [253], we know that one can color edges with either $\Delta(G)$ or $\Delta(G) + 1$ different colors, where $\Delta(G)$ is the maximum degree of G . Since $\Delta(G)$ is a trivial lower bound on the minimum number, we can say that the Edge Coloring Problem allows a 1-additive approximation. Note that the problem of deciding whether a given graph can be edge colored with $\Delta(G)$ colors is NP-complete [150].

1.1.4 Hardness of Approximation

Now we move to hardness of approximation.

Definition 1.2 (Approximability Threshold). *Given a minimization optimization problem Π , it is said that Π has an approximation threshold $\alpha^*(\Pi)$, if for any $\epsilon > 0$, Π admits a $\alpha^*(\Pi) + \epsilon$ approximation but if it admits a $\alpha^*(\Pi) - \epsilon$ approximation then $P = NP$.*

If $\alpha^*(\Pi) = 1$, it implies that Π is solvable in polynomial time. Many NPO problems Π are known to have $\alpha^*(\Pi) > 1$ assuming that $P \neq NP$. We can say that approximation algorithms try to decrease the upper bound on $\alpha^*(\Pi)$, while hardness of approximation attempts to increase lower bounds on $\alpha^*(\Pi)$.

To prove hardness results on NPO problems in terms of approximation, there are largely two approaches; a direct way by reduction from NP-complete problems and an indirect way via gap reductions. Here let us take a quick look at an example using a reduction from an NP-complete problem.

In the (metric) k -center problem, we are given an undirected graph $G = (V, E)$ and an integer k . We are asked to choose a subset of k vertices from V called centers. The goal is to minimize the maximum distance to a center, i.e. $\min_{S \subseteq V, |S|=k} \max_{v \in V} \text{dist}_G(v, S)$, where $\text{dist}_G(v, S) = \min_{u \in S} \text{dist}_G(u, v)$.

The k -center problem has approximation threshold 2, since there are a few 2-approximation algorithms for k -center and there is no $2 - \epsilon$ approximation algorithm for any $\epsilon > 0$ unless $P = NP$. We can prove the inapproximability using a reduction from the decision version of Dominating Set: Given an undirected graph $G = (V, E)$ and an integer k , does G have a dominating set of size at most k ? A set $S \subseteq V$ is said to be a dominating set in G if for all $v \in V$, $v \in S$ or v is adjacent to some u in S . Dominating Set is known to be NP-complete.

Theorem 1.3 ([153]). *Unless $P = NP$, there is no $2 - \epsilon$ approximation for k -center for any fixed $\epsilon > 0$.*

Proof. Let I be an instance of Dominating Set Problem consisting of graph $G = (V, E)$ and integer k . We create an instance I' of k -center while keeping graph G and k the same. If I has a dominating set of size k then $\text{OPT}(I') = 1$, since every vertex can be reachable from the Dominating Set by at most one hop. Otherwise, we claim that $\text{OPT}(I') \geq 2$. This is because if $\text{OPT}(I') < 2$, then every vertex must be within distance 1, which implies the k -center that witnesses $\text{OPT}(I')$ is a dominating set of I . Therefore, the $(2 - \epsilon)$ approximation for k -center can be used to solve the Dominating Set Problem. This is impossible, unless $P = NP$. ■

1.2 Designing Approximation Algorithms

How does one design and more importantly analyze the performance of approximation algorithms? This is a non-trivial task and the main goal of the course is to expose you to basic and advanced techniques as well as central problems. The purpose of this section is to give some high-level insights. We start with how we design polynomial-time algorithms. Note that approximation makes sense mainly in the setting where one can find a feasible solution relatively easily but finding an *optimum* solution is hard. In some cases finding a feasible solution itself may involve some non-trivial algorithm, in which case it is useful to properly understand the structural properties that guarantee feasibility, and then build upon it.

Some of the standard techniques we learn in basic and advanced undergraduate algorithms courses are recursion based methods such as divide and conquer, dynamic programming, greedy, local search, combinatorial optimization via duality, and reductions to existing problems. How do we adapt these to the approximation setting? Note that intractability implies that there are no efficient characterizations of the optimum solution value.

Greedy and related techniques are often fairly natural for many problems and simple heuristic algorithms often suggest themselves for many problems. (Note that the algorithms may depend on being able to solve some existing problem efficiently. Thus, knowing a good collection of general poly-time solvable problems is often important.) The main difficulty is in analyzing their performance. The key challenge here is to identify appropriate *lower bounds* on the optimal value (assuming that the problem is a minimization problem) or *upper bounds* on the optimal value (assuming that the problem is a maximization problem). These bounds allow one to compare the output of the algorithm and prove an approximation bound. In designing poly-time algorithms we often prove that greedy algorithms do not work. We typically do this via examples. This skill is also useful in proving that some candidate algorithm does *not* give a good approximation. Often the bad examples lead one to a new algorithm.

How does one come up with lower or upper bounds on the optimum value? This depends on the problem at hand and knowing some background and related problems. However, one would like to find some automatic ways of obtaining bounds. This is often provided via linear programming relaxations and more advanced convex programming methods including semi-definite programming, lift-and-project hierarchies etc. The basic idea is quite simple. Since integer linear programming is NP-Complete one can formulate most discrete optimization problems easily and “naturally” as an integer program. Note that there may be many different ways of expressing a given problem as an integer program. Of course we cannot solve the integer program but we

can solve the linear-programming relaxation which is obtained by removing the integrality constraints on the variables. Thus, for each instance I of a given problem we can obtain an LP relaxation $LP(I)$ which we typically can solve in polynomial-time. This automatically gives a bound on the optimum value since it is a relaxation. How good is this bound? It depends on the problem, of course, and also the specific LP relaxation. How do we obtain a feasible solution that is close to the bound given by the LP relaxation. The main technique here is to *round* the fractional solution x to an integer feasible solution x' such that x' 's value is close to that of x . There are several non-trivial rounding techniques that have been developed over the years that we will explore in the course. We should note that in several cases one can analyze combinatorial algorithms via LP relaxations even though the LP relaxation does not play any direct role in the algorithm itself. Finally, there is the question of which LP relaxation to use. Often it is required to “strengthen” an LP relaxation via addition of constraints to provide better bounds. There are some automatic ways to strengthen any LP and often one also needs problem specific ideas.

Local search is another powerful technique and the analysis here is not obvious. One needs to relate the value of a local optimum to the value of a global optimum via various *exchange* properties which define the local search heuristic. For a formal analysis it is necessary to have a good understanding of the problem structure.

Finally, dynamic programming plays a key role in the following way. Its main use is in solving to optimality a *restricted* version of the given problem or a subroutine that is useful as a building block. How does one obtain a restricted version? This is often done by some clever preprocessing of a given instance.

Reductions play a very important role in both designing approximation algorithms and in proving inapproximability results. Often reductions serve as a starting point in developing a simple and crude heuristic that allows one to understand the structure of a problem which then can lead to further improvements.

Discrete optimization problems are brittle — changing the problem a little can lead to substantial changes in the complexity and approximability. Nevertheless it is useful to understand problems and their structure in broad categories so that existing results can be leveraged quickly and robustly. Thus, some of the emphasis in the course will be on classifying problems and how various parameters influence the approximability.

Chapter 2

Covering Problems

Part of these notes were scribed by Abul Hassan Samee and Lewis Tseng.

Packing and Covering problems together capture many important problems in combinatorial optimization. We will discuss several covering problems in this chapter. Two canonical problems are MINIMUM VERTEX COVER and its generalization MINIMUM SET COVER. (Typically we will omit the use of the qualifiers minimum and maximum since this is often clear from the definition of the problem and the context.) They play an important (historical) role in the study of approximation algorithms.

A *vertex cover* in an undirected graph $G = (V, E)$ is a set $S \subseteq V$ of vertices such that for each edge $e \in E$, at least one of its end points is in S . It is also called a node cover. In the VERTEX COVER problem, our goal is to find a smallest vertex cover of G . In the *weighted* version of the problem, a weight function $w : V \rightarrow \mathcal{R}^+$ is given, and our goal is to find a minimum weight vertex cover of G . The unweighted version of the problem is also known as CARDINALITY VERTEX COVER. Note that we are picking vertices to *cover* the edges. VERTEX COVER is NP-Hard and is on the list of problems in Karp's list.

In the SET COVER problem the input is a set $\mathcal{U}$ of n elements, and a collection $\mathcal{S} = \{S_1, S_2, \dots, S_m\}$ of m subsets of $\mathcal{U}$ such that $\bigcup_i S_i = \mathcal{U}$. Our goal in the SET COVER problem is to select as few sets as possible from $\mathcal{S}$ such that their union is $\mathcal{U}$. In the weighted version each set S_i has a non-negative weight w_i the goal is to find a set cover of minimum weight. It is easy to see VERTEX COVER is a special case of SET COVER. Closely related to the SET COVER problem is the MAXIMUM COVERAGE problem. In this problem the input is a set system given by $\mathcal{U}$ and $\mathcal{S}$ and in addition there is also an integer $k \leq m$. The goal is to select k sets from $\mathcal{S}$ such that the cardinality of their union is maximized. Note that SET COVER is a minimization problem while MAXIMUM COVERAGE is a maximization problem. SET COVER is equivalent to the HITTING SET problem. In HITTING SET the input is

$\mathcal{U}$ and $\mathcal{S}$ but the goal is to pick the smallest number of elements of $\mathcal{U}$ that cover the given sets in $\mathcal{S}$. In other words we are seeking a set cover in the dual set system¹.

SET COVER has many applications. In the standard formulation the set system is explicitly given. However, in many applications the set system is *implicitly* specified. SET COVER is an important problem because in discrete optimization. In the standard definition the set system is given *explicitly*. In many applications the set system is *implicit*, and often exponential in the explicit part of the input; nevertheless such set systems are ubiquitous and one can often obtain exact or approximation algorithms. As an example consider the well known MST problem in graphs. One way to phrase MST is the following: given an edge-weighted graph $G = (V, E)$ find a minimum cost subset of the edges that *cover* all the cuts of G ; by cover a cut $S \subseteq V$ we mean that at least one of the edges in $\delta(S)$ must be chosen. This may appear to be a strange way of looking at the MST problem but this view is useful as we will see later. Another implicit example is the following. Suppose we are given n rectangles in the plane and the goal is to choose a minimum number of points in the plane such that each input rectangle contains one of the chosen points. This is perhaps more natural to view as a special case of the HITTING SET problem. In principle the set of points that we can choose from is infinite but it can be seen that we can confine our attention to vertices in the arrangement of the given rectangles and it is easy to see that there are only $O(n^2)$ vertices — however, explicitly computing them may be expensive and one may want to treat the problem as an implicit one for the sake of efficiency.

Covering problems have the feature that a superset of a feasible solution is also a feasible solution. More abstractly one can cast covering problems as the following. We are given a finite ground set V (vertices in a graph or sets in a set system) and a family of feasible solutions $\mathcal{I} \subseteq 2^V$ where $\mathcal{I}$ is upward closed; by this we mean that if $A \in \mathcal{I}$ and $A \subset B$ then $B \in \mathcal{I}$. The goal is to find the smallest cardinality set A in $\mathcal{I}$. In the weighted case V has weights and the goal is to find a minimum weight set in $\mathcal{I}$. In some case one can also consider more complex non-additive objectives that assign a cost $c(S)$ for each $S \in \mathcal{I}$.

2.1 Greedy for SET COVER and MAXIMUM COVERAGE

In this section we consider the unweighted version of SET COVER.

¹A finite set system is also viewed as a bipartite graph with one side corresponding to sets and the other side corresponding to elements — there is an edge between a set vertex and an element vertex iff the element belongs to the set. The dual set system corresponds to flipping the role of the sets and elements in the same bipartite graph.

2.1.1 Greedy Algorithm

A natural greedy approximation algorithm for these problems is easy to come up with.

GREEDY COVER($\mathcal{U}, \mathcal{S}$)

1. **repeat**
 - A. pick the set that covers the maximum number of uncovered elements
 - B. mark elements in the chosen set as covered
2. **until** done

In case of SET COVER, the algorithm GREEDY COVER is *done* when all the elements in set $\mathcal{U}$ have been covered. And in case of MAXIMUM COVERAGE, the algorithm is *done* when exactly k subsets have been selected from $\mathcal{S}$.

We will prove the following theorem.

Theorem 2.1. GREEDY COVER is a $1 - (1 - 1/k)^k \geq (1 - \frac{1}{e}) \simeq 0.632$ approximation for MAXIMUM COVERAGE, and a $(\ln n + 1)$ approximation for SET COVER.

The following theorem implies that GREEDY COVER is essentially the best possible in terms of the approximation ratio that it guarantees.

Theorem 2.2 (Feige [103]). *Unless $NP \subseteq DTIME(n^{O(\log \log n)})$, there is no $(1 - o(1)) \ln n$ approximation for SET COVER. Unless $P=NP$, for any fixed $\epsilon > 0$, there is no $(1 - \frac{1}{e} - \epsilon)$ approximation for MAXIMUM COVERAGE.*

The preceding theorem has been strengthened so that the hardness holds under the more standard assumption that $P \neq NP$ [207].

2.1.2 Analysis of GREEDY COVER

We proceed towards the proof of Theorem 10.3 by providing analysis of GREEDY COVER separately for SET COVER and MAXIMUM COVERAGE.

Analysis for MAXIMUM COVERAGE

Let OPT denote the value of an optimal solution to the MAXIMUM COVERAGE problem; this is the maximum number of elements that are covered by k sets in the given set system. Let x_i denote the number of *new* elements covered by the i -th set chosen by GREEDY COVER. Also, let $y_i = \sum_{j=1}^i x_j$ be the total number of elements covered after i iterations, and $z_i = OPT - y_i$. Note that, according to

our notation, $y_0 = 0$ and y_k is the number of elements chosen by GREEDY COVER at the end of the algorithm, and $z_0 = \text{OPT}$. The key to the analysis is the following simple claim.

Claim 2.1.1. For $0 \leq i < k$, $x_{i+1} \geq \frac{z_i}{k}$.

Proof. Let $F^* \subseteq \mathcal{U}$ be the elements covered by some fixed optimum solution; we have $|F^*| = \text{OPT}$. Consider iteration $i + 1$. GREEDY COVER selects the subset S_j whose inclusion covers the maximum number of uncovered elements. Since y_i is the total number of elements covered up to iteration i , at least $z_i = \text{OPT} - y_i$ elements from F^* are uncovered. Let the set of uncovered elements from F^* at the end of iteration i be F_i^* . Since k sets together cover F^* , and hence F_i^* as well, there must be some set in that collection of k sets that covers at least $|F_i^*|/k$ elements. This is a candidate set that can be chosen in iteration $i + 1$. Since the algorithm picks the set that covers the maximum number of uncovered elements, the chosen set in iteration $i + 1$ covers at least $|F_i^*|/k = \frac{z_i}{k}$ uncovered elements. Hence, $x_{i+1} \geq \frac{z_i}{k}$. ■

Remark 2.1. It is tempting to make a stronger claim that $x_{i+1} \geq \frac{z_i}{k-i}$. This is however false, and it is worthwhile to come up with an example.

By definition we have $y_k = x_1 + x_2 + \dots + x_k$ is the total number of elements covered by GREEDY COVER. To analyze the worst-case we want to make this sum as small as possible given the preceding claim. Heuristically (which one can formalize), one can argue that choosing $x_{i+1} = z_i/k$ minimizes the sum. Using this one can argue that the sum is at least $(1 - (1 - 1/k)^k) \text{OPT}$. We give a formal argument now.

Claim 2.1.2. For $i \geq 0$, $z_i \leq (1 - \frac{1}{k})^i \cdot \text{OPT}$.

Proof. By induction on i . The claim is trivially true for $i = 0$ since $z_0 = \text{OPT}$. We assume inductively that $z_i \leq (1 - \frac{1}{k})^i \cdot \text{OPT}$. Then

$$\begin{aligned} z_{i+1} &= z_i - x_{i+1} \\ &\leq z_i \left(1 - \frac{1}{k}\right) \quad [\text{using Claim 2.1.1}] \\ &\leq \left(1 - \frac{1}{k}\right)^{i+1} \cdot \text{OPT}. \end{aligned}$$

■

The preceding claims yield the following lemma for algorithm GREEDY COVER when applied on MAXIMUM COVERAGE.

Lemma 2.1. GREEDY COVER is a $1 - (1 - 1/k)^k \geq 1 - \frac{1}{e}$ approximation for MAXIMUM COVERAGE.

Proof. It follows from Claim 2.1.2 that $z_k \leq (1 - \frac{1}{k})^k \cdot \text{OPT} \leq \frac{\text{OPT}}{e}$. Hence, $y_k = \text{OPT} - z_k \geq (1 - \frac{1}{e}) \cdot \text{OPT}$. ■

We note that $(1 - 1/e) \approx 0.632$.

Analysis for SET COVER

Let k^* denote the value of an optimal solution to the SET COVER problem. Then an optimal solution value to the MAXIMUM COVERAGE problem with the same system and $k = k^*$ would be $n = |\mathcal{U}|$ since it is possible to cover all the n elements in set $\mathcal{U}$ with k^* sets. From our previous analysis, $z_{k^*} \leq \frac{n}{e}$. Therefore, at most $\frac{n}{e}$ elements would remain uncovered after the first k^* steps of GREEDY COVER. Similarly, after $2 \cdot k^*$ steps of GREEDY COVER, at most $\frac{n}{e^2}$ elements would remain uncovered. This easy intuition convinces us that GREEDY COVER is a $(\ln n + 1)$ approximation for the SET COVER problem. A more formal proof is given below.

Lemma 2.2. GREEDY COVER is a $(\ln n + 1)$ approximation for SET COVER.

Proof. Since $z_i \leq (1 - \frac{1}{k^*})^i \cdot n$, after $t = \lceil k^* \ln \frac{n}{k^*} \rceil$ steps,

$$z_t \leq n(1 - 1/k^*)^{k^* \ln \frac{n}{k^*}} \leq ne^{-\ln \frac{n}{k^*}} \leq k^*.$$

Thus, after t steps, at most k^* elements are left to be covered. Since GREEDY COVER covers at least one element in each step, it covers all the elements after picking at most $\lceil k^* \ln \frac{n}{k^*} \rceil + k^* \leq k^*(\ln n + 1)$ sets. ■

A useful special case of SET COVER is when all sets are “small”. Does the approximation bound for Greedy improve? We can prove the following corollary via Lemma 2.2.

Corollary 2.3. If each set in the set system has at most d elements, then GREEDY COVER is a $(\ln d + 1)$ approximation for SET COVER.

Proof. If each set has at most d elements then we have that $k^* \geq \frac{n}{d}$ and hence $\ln \frac{n}{k^*} \leq \ln d$. Then the claim follows from Lemma 2.2. ■

Theorem 10.3 follows directly from Lemma 2.1 and 2.2.

A near-tight example for GREEDY COVER when applied on SET COVER : Let us consider a set $\mathcal{U}$ of n elements along with a collection $\mathcal{S}$ of $k + 2$ subsets $\{R_1, R_2, C_1, C_2, \dots, C_k\}$ of $\mathcal{U}$. Let us also assume that $|C_i| = 2^i$ and $|R_1 \cap C_i| = |R_2 \cap C_i| = 2^{i-1}$ ($1 \leq i \leq k$), as illustrated in Fig. 2.1.

Clearly, the optimal solution consists of only two sets, i.e., R_1 and R_2 . Hence, $\text{OPT} = 2$. However, GREEDY COVER will pick each of the remaining k sets, namely

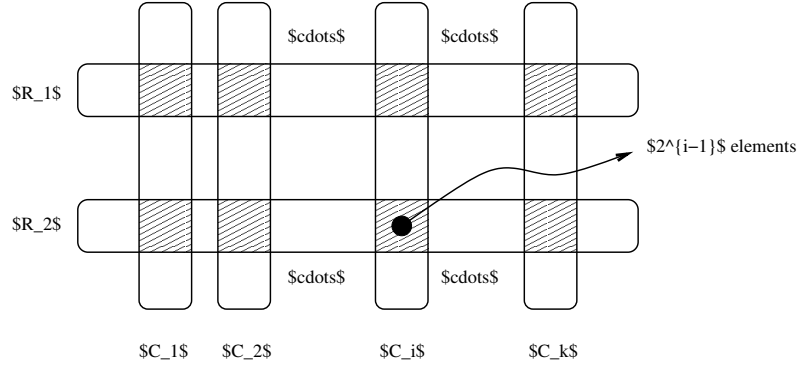


Figure 2.1: A near-tight example for GREEDY COVER when applied on SET COVER

$C_k, C_{k-1}, \dots, C_1$. Since $n = 2 \cdot \sum_{i=0}^{k-1} 2^i = 2 \cdot (2^k - 1)$, we get $k = \Omega(\log n)$. One can construct tighter examples with more involved analysis.

Remark 2.2. The preceding example is an instance of covering points by orthogonal rectangles and demonstrates the greedy algorithm's worst case behaviour holds even in such restricted instances. For covering points by rectangles one can obtain a constant factor approximation by other means. It should also be noted that each element in the example is contained in two sets and hence one can view it also as an instance of VERTEX COVER in multigraphs for which we will see a 2 approximation soon. Thus, although greedy is worst-case optimal for SET COVER in its full generality, in special cases of interest, other techniques can be used to obtain a better approximation.

Exercise 2.1. Consider the weighted version of the SET COVER problem where a weight function $w : \mathcal{S} \rightarrow \mathcal{R}^+$ is given, and we want to select a collection $\mathcal{S}'$ of subsets from $\mathcal{S}$ such that $\cup_{X \in \mathcal{S}'} X = \mathcal{U}$, and $\sum_{X \in \mathcal{S}'} w(X)$ is the minimum. One can generalize the greedy heuristic in the natural fashion where in each iteration the algorithm picks the set that maximizes the ratio of the number of elements to its weight. Adapt the unweighted analysis to prove that the greedy algorithm yields an $O(\ln n)$ approximation for the weighted version (you can be sloppy with the constant in front of $\ln n$).

2.1.3 DOMINATING SET

A *dominating set* in a graph $G = (V, E)$ is a set $S \subseteq V$ such that for each $u \in V$, either $u \in S$, or some neighbor v of u is in S . In other words S covers/dominates

all the nodes in V . In the DOMINATING SET problem, the input is a graph G and the goal is to find a smallest sized dominating set in G .

- Exercise 2.2.**
1. Show that DOMINATING SET is a special case of SET COVER.
 2. What is the greedy heuristic when applied to DOMINATING SET. Prove that it yields an $(\ln(\Delta + 1) + 1)$ approximation where Δ is the maximum degree in the graph.
 3. Show that SET COVER can be reduced in an approximation preserving fashion to DOMINATING SET. More formally, show that if DOMINATING SET has an $\alpha(n)$ -approximation where n is the number of vertices in the given instance then SET COVER has a $(1 - o(1))\alpha(n)$ -approximation.

2.2 VERTEX COVER

We have already seen that the VERTEX COVER problem is a special case of the SET COVER problem. The Greedy algorithm when specialized to VERTEX COVER picks a highest degree vertex, removes it and the covered edges from the graph, and recurses in the remaining graph. It follows that the Greedy algorithm gives an $O(\ln \Delta + 1)$ approximation for the unweighted versions of the VERTEX COVER problem. One can wonder whether the Greedy algorithm has a better worst-case for VERTEX COVER than the analysis suggests. Unfortunately the answer is negative and there are examples where the algorithm outputs a solution with $\Omega(\log n \cdot \text{OPT})$ vertices.

We sketch the construction. Consider a bipartite graph $G = (U, V, E)$ where $U = \{u_1, u_2, \dots, u_h\}$. V is partitioned into $S_1, S_2, \dots, S_h$ where S_i has $\lfloor h/i \rfloor$ vertices. Each vertex v in S_i is connected to exactly i distinct vertices of U ; thus, any vertex u_j is incident to at most one edge from S_i . It can be seen that the degree of each vertex $u_j \in U$ is roughly h . Clearly U is a vertex cover of G since the graph is bipartite. Convince yourself that the Greedy algorithm will pick all of V starting with the lone vertex in S_h (one may need to break ties to make this happen but the example can be easily perturbed to make this unnecessary). We have $n = \Theta(h \log h)$ and $\text{OPT} \leq h$ and Greedy outputs a solution of size $\Omega(h \log h)$.

2.2.1 A 2-approximation for VERTEX COVER

There is a very simple 2-approximation algorithm for the CARDINALITY VERTEX COVER problem.

MATCHING-VC(G)

1. $S \leftarrow \emptyset$
2. Compute a maximal matching M in G
3. **for** each edge $(u, v) \in M$ **do**
 - A. add both u and v to S
4. Output S

Theorem 2.4. *MATCHING-VC is a 2-approximation algorithm.*

The proof of Theorem 2.4 follows from two simple claims.

Claim 2.2.1. *Let OPT be the size of the vertex cover in an optimal solution. Then $\text{OPT} \geq |M|$.*

Proof. Any vertex cover must contain at least one end point of each edge in M since no two edges in M intersect. Hence $\text{OPT} \geq |M|$. ■

Claim 2.2.2. *Let $S(M) = \{u, v | (u, v) \in M\}$. Then $S(M)$ is a vertex cover.*

Proof. If $S(M)$ is not a vertex cover, then there must be an edge $e \in E$ such that neither of its endpoints are in M . But then e can be included in M , which contradicts the maximality of M . ■

We now finish the proof of Theorem 2.4. Since $S(M)$ is a vertex cover, Claim 2.2.1 implies that $|S(M)| = 2 \cdot |M| \leq 2 \cdot \text{OPT}$.

WEIGHTED VERTEX COVER: The matching based heuristic does not generalize in a straight forward fashion to the weighted case but 2-approximation algorithms for the WEIGHTED VERTEX COVER problem can be designed based on LP rounding.

2.2.2 SET COVER with small frequencies

VERTEX COVER is an instance of SET COVER where each element in $\mathcal{U}$ is in at most two sets (in fact, each element was in exactly two sets). This special case of the SET COVER problem admits a 2-approximation algorithm. What would be the case if every element is contained in at most three sets? More generally, given an instance of SET COVER, for each $e \in \mathcal{U}$, let $f(e)$ denote the number of sets containing e . Let $f = \max_e f(e)$, which we call the *maximum frequency*.

Exercise 2.3. Give an f -approximation for SET COVER, where f is the maximum frequency of an element. *Hint:* Follow the approach used for VERTEX COVER.

2.3 Vertex Cover via LP

Let $G = (V, E)$ be an undirected graph with arc weights $w : V \rightarrow R^+$. We can formulate VERTEX COVER as an integer linear programming problem as follows. For each vertex v we have a variable x_v . We interpret the variable as follows: $x_v = 1$ if v is chosen to be included in a vertex cover, otherwise $x_v = 0$. With this interpretation we can easily see that the minimum weight vertex cover can be formulated as the following integer linear program.

$$\begin{aligned} \min \quad & \sum_{v \in V} w_v x_v \\ \text{subject to} \quad & x_u + x_v \geq 1 \quad \forall e = (u, v) \in E \\ & x_v \in \{0, 1\} \quad \forall v \in V \end{aligned}$$

However, solving the preceding integer linear program is NP-Hard since it would solve VERTEX COVER exactly. Therefore we use Linear Programming (LP) to approximate the optimal solution, $\text{OPT}(I)$, for the integer program. First, we can relax the constraint $x_v \in \{0, 1\}$ to $x_v \in [0, 1]$. It can be further simplified to $x_v \geq 0, \forall v \in V$.

Thus, a linear programming formulation for Vertex Cover is:

$$\begin{aligned} \min \quad & \sum_{v \in V} w_v x_v \\ \text{subject to} \quad & x_u + x_v \geq 1 \quad \forall e = (u, v) \in E \\ & x_v \geq 0 \end{aligned}$$

We now use the following algorithm:

VERTEX COVER VIA LP

1. Solve LP to obtain an optimal fractional solution x^*
2. Let $S = \{v \mid x_v^* \geq \frac{1}{2}\}$
3. Output S

Claim 2.3.1. S is a vertex cover.

Proof. Consider any edge, $e = (u, v)$. By feasibility of x^* , $x_u^* + x_v^* \geq 1$, and thus $x_u^* \geq \frac{1}{2}$ or $x_v^* \geq \frac{1}{2}$. Therefore, at least one of u and v will be in S . ■

Claim 2.3.2. $w(S) \leq 2 \text{OPT}_{LP}(I)$.

Proof. $\text{OPT}_{LP}(I) = \sum_v w_v x_v^* \geq \frac{1}{2} \sum_{v \in S} w_v = \frac{1}{2} w(S)$. ■

Therefore, $\text{OPT}_{LP}(I) \geq \frac{\text{OPT}(I)}{2}$ for all instances I .

Remark 2.3. For minimization problems: $\text{OPT}_{LP}(I) \leq \text{OPT}(I)$, where $\text{OPT}_{LP}(I)$ is the optimal solution found by LP; for maximization problems, $\text{OPT}_{LP}(I) \geq \text{OPT}(I)$.

Integrality Gap: We introduce the notion of *integrality gap* to show the best approximation guarantee we can obtain if we only use the LP values as a lower bound.

Definition 2.5. For a minimization problem Π , the *integrality gap* for a linear programming relaxation/formulation LP for Π is $\sup_{I \in \Pi} \frac{\text{OPT}(I)}{\text{OPT}_{LP}(I)}$.

That is, the integrality gap is the worst case ratio, over all instances I of Π , of the integral optimal value and the fractional optimal value. Note that different linear programming formulations for the same problem may have different integrality gaps.

Claims 2.3.1 and 2.3.2 show that the integrality gap of the Vertex Cover LP formulation above is *at most* 2.

Question: Is this bound tight for the Vertex Cover LP?

Consider the following example: Take a complete graph, K_n , with n vertices, and each vertex has $w_v = 1$. It is clear that we have to choose $n - 1$ vertices to cover all the edges. Thus, $\text{OPT}(K_n) = n - 1$. However, $x_v = \frac{1}{2}$ for each v is a feasible solution to the LP, which has a total weight of $\frac{n}{2}$. So gap is $2 - \frac{1}{n}$, which tends to 2 as $n \rightarrow \infty$. One can also prove that the integrality gap is essentially 2 even in a class of sparse graphs.

Exercise 2.4. The vertex cover problem can be solved optimally in polynomial time in bipartite graphs. In fact the LP is integral. Prove this via the maxflow-mincut theorem and the integrality of flows when capacities are integral.

Other Results on Vertex Cover

1. The current best approximation ratio for Vertex Cover is $2 - \Theta(\frac{1}{\sqrt{\log n}})$ [166].
2. It is known that unless $P = NP$ there is α -approximation for VERTEX COVER for $\alpha < 1.36$ [93]. Under a stronger hypothesis called the Unique Games Conjecture it is known that there is no $2 - \epsilon$ approximation for any fixed $\epsilon > 0$ [176].

3. There is a polynomial time approximation scheme (PTAS), that is a $(1 + \epsilon)$ -approximation for any fixed $\epsilon > 0$, for planar graphs. This follows from a general approach due to Baker [31]. The theorem extends to more general classes of graphs.

2.4 Set Cover via LP

The input to the Set Cover problem consists of a finite set $U = \{1, 2, \dots, n\}$, and m subsets of U , $S_1, S_2, \dots, S_m$. Each set S_j has a non-negative weight w_j and the goal is to find the minimum weight collection of sets which cover all elements in U (in other words their union is U).

A linear programming relaxation for Set Cover is:

$$\begin{aligned}
 & \min \quad \sum_j w_j x_j \\
 & \text{subject to} \\
 & \quad \sum_{j: i \in S_j} x_j \geq 1 \quad \forall i \in \{1, 2, \dots, n\} \\
 & \quad x_j \geq 0 \quad 1 \leq j \leq m
 \end{aligned}$$

And its dual is:

$$\begin{aligned}
 & \max \quad \sum_{i=1}^n y_i \\
 & \text{subject to} \\
 & \quad \sum_{i \in S_j} y_i \leq w_j \quad \forall j \in \{1, 2, \dots, m\} \\
 & \quad y_i \geq 0 \quad \forall i \in 1, 2, \dots, n
 \end{aligned}$$

We give several algorithms for Set Cover based on this primal/dual pair LPs.

2.4.1 Deterministic Rounding

SET COVER VIA LP

1. Solve LP to obtain an optimal solution x^* , which contains fractional numbers.
2. Let $P = \{i \mid x_i^* > 0\}$
3. Output $\{S_j \mid j \in P\}$

Note that the above algorithm, even when specialized to VERTEX COVER is different from the one we saw earlier. It includes all sets which have a strictly positive value in an *optimum* solution to the LP.

Let x^* be an optimal solution to the primal LP, y^* be an optimum solution to the dual, and let $P = \{j \mid x_j^* > 0\}$. First, note that by strong duality, $\sum_j w_j x_j^* = \sum_i y_i^*$. Second, by complementary slackness if $x_j^* > 0$ then the corresponding dual constraint is tight, that is $\sum_{i \in S_j} y_i^* = w_j$.

Claim 2.4.1. *The output of the algorithm is a feasible set cover for the given instance.*

Proof. Exercise. ■

Claim 2.4.2. $\sum_{j \in P} w_j \leq f \sum_j w_j x_j^* = \text{OPT}_{LP}$.

Proof.

$$\sum_{j \in P} w_j = \sum_{j: x_j^* > 0} w_j = \sum_{j: x_j^* > 0} \left(\sum_{i \in S_j} y_i^* \right) = \sum_i y_i^* \left(\sum_{j: i \in S_j, x_j^* > 0} 1 \right) \leq f \sum_i y_i^* = f \text{OPT}_{LP}(I).$$

■

Notice that the second equality is due to complementary slackness conditions (if $x_j^* > 0$, the corresponding dual constraint is tight), the penultimate inequality uses the definition of f , and the last inequality follows from weak duality (a feasible solution for the dual problem is a lower bound on the optimal primal solution).

Therefore we have that the algorithm outputs a cover of weight at most $f \text{OPT}_{LP}$. We note that f can be as large as n in which case the bound given by the algorithm is quite weak. In fact, it is not hard to construct examples that demonstrate the tightness of the analysis.

Remark 2.4. The analysis crucially uses the fact that x^* is an optimal solution. On the other hand the algorithm for VERTEX COVER is more robust and works with any feasible solution x . It is easy to generalize the earlier rounding for VERTEX COVER to obtain an f -approximation. The point of the above rounding is to illustrate the utility of complementary slackness.

2.4.2 Randomized Rounding

Now we describe a different rounding that yields an approximation bound that does not depend on f .

SET COVER VIA RANDOMIZED ROUNDING

1. $A = \emptyset$, and let x^* be an optimal solution to the LP
2. for $k = 1$ to $2 \ln n$ do
 - A. pick each S_j independently with probability x_j^*
 - B. if S_j is picked, $A = A \cup \{j\}$
3. Output the sets with indices in A

Claim 2.4.3. $\mathbf{P}[i \text{ is not covered in an iteration}] = \prod_{j:i \in S_j} (1 - x_j^*) \leq \frac{1}{e}$.

Intuition: We know that $\sum_{j:i \in S_j} x_j^* \geq 1$. Subject to this constraint, if we want to minimize the probability that element i is covered, one can see that the minimum is achieved with $x_j^* = 1/\ell$ for each set S_j that covers i ; here ℓ is the number of sets that cover i . Then the probability is $(1 - \frac{1}{\ell})^\ell$.

Proof. We use the inequality $(1 - x) \leq e^{-x}$ for all $x \in [0, 1]$.

$$\mathbf{P}[i \text{ is not covered in an iteration}] = \prod_{j:i \in S_j} (1 - x_j^*) \leq \prod_{j:i \in S_j} e^{-x_j^*} \leq e^{-\sum_{j:i \in S_j} x_j^*} \leq \frac{1}{e}.$$

■

We then obtain the following corollaries:

Corollary 2.6. $\mathbf{P}[i \text{ is not covered at the end of the algorithm}] \leq e^{-2 \log n} \leq \frac{1}{n^2}$.

Corollary 2.7. $\mathbf{P}[\text{all elements are covered, after the algorithm stops}] \geq 1 - \frac{1}{n}$.

Proof. Via the union bound. The probability that i is not covered is at most $1/n^2$, hence the probability that there is some i that is not covered is at most $n \cdot 1/n^2 \leq 1/n$. ■

Now we bound the expected cost of the algorithm. Let C_t = cost of sets picked in iteration t , then $\mathbf{E}[C_t] = \sum_{j=1}^m w_j x_j^*$, where $\mathbf{E}[X]$ denotes the expectation of a random variable X . Then, let $C = \sum_{t=1}^{2 \ln n} C_t$; we have $\mathbf{E}[C] = \sum_{t=1}^{2 \ln n} \mathbf{E}[C_t] \leq 2 \ln n \text{OPT}_{LP}$. By Markov's inequality, $\mathbf{P}[C > 2 \mathbf{E}[C]] \leq \frac{1}{2}$, hence $\mathbf{P}[C \leq 4 \ln n \text{OPT}_{LP}] \geq \frac{1}{2}$. Therefore, $\mathbf{P}[C \leq 4 \ln n \text{OPT}_{LP} \text{ and all items are covered}] \geq \frac{1}{2} - \frac{1}{n}$. Thus, the randomized rounding algorithm, with probability close to $1/2$ succeeds in giving a feasible solution of cost $O(\log n) \text{OPT}_{LP}$. Note that we can check whether the solution satisfies the desired properties (feasibility and cost) and repeat the algorithm if it does not.

1. We can check if solution after rounding satisfies the desired properties, such as all elements are covered, or cost at most $2c \log n \text{OPT}_{LP}$. If not, repeat rounding. Expected number of iterations to succeed is a constant.
2. We can also use Chernoff bounds (large deviation bounds) to show that a single rounding succeeds with high probability (probability at least $1 - \frac{1}{\text{poly}(n)}$).
3. The algorithm can be *derandomized*. Derandomization is a technique of removing randomness or using as little randomness as possible. There are many derandomization techniques, such as the method of conditional expectation, discrepancy theory, and expander graphs.
4. After a few rounds, select the cheapest set that covers each uncovered element. This has low expected cost. This algorithm ensures feasibility but guarantees cost only in the expected sense. We will see a variant on the homework.

Randomized Rounding with Alteration: In the preceding analysis we had to worry about the probability of covering all the elements and the expected cost of the solution. Here we illustrate a simple yet powerful technique of *alteration* in randomized algorithms and analysis. Let d be the maximum set size.

SET COVER: RANDOMIZED ROUNDING WITH ALTERATION

1. $A = \emptyset$, and let x^* be an optimal solution to the LP
2. Add to A each S_j independently with probability $\min\{1, \ln d \cdot x_j^*\}$
3. Let $\mathcal{U}'$ be the elements uncovered by the chosen sets in A
4. For each uncovered element $i \in \mathcal{U}'$ do
 - A. Add to A the cheapest set that covers i
5. Output the sets with indices in A

The algorithm has two phases. A randomized phase and a fixing/altering phase. In the second phase we apply a naive algorithm that may have a high cost in the worst case but we will bound its expected cost appropriately. The algorithm deterministically guarantees that all elements will be covered, and hence we only need to focus on the expected cost of the chosen sets. Let C_1 be the random cost of the sets chosen in the first phase and let C_2 be the random cost of the sets chosen in the second phase. It is easy to see that $\mathbb{E}[C_1] = \ln d \sum_j w_j x_j^* = \ln d \text{OPT}_{LP}$. Let $\mathcal{E}_i$ be the event that element i is not covered after the first randomized phase.

Exercise 2.5. $\mathbb{P}[\mathcal{E}_i] \leq e^{-\ln d} \leq 1/d$.

The worst case second phase cost can be upper bounded via the next lemma.

Lemma 2.3. *Let β_i be the cost of the cheapest set covering i . Then $\sum_i \beta_i \leq d \text{OPT}_{LP}$.*

Proof. Consider an element i . We have the constraint that $\sum_{j:i \in S_j} x_j^* \geq 1$. Since each set covering i has cost at least β_i , we have $\sum_{j:i \in S_j} c_j x_j^* \geq \beta_i \sum_{j:i \in S_j} x_j^* \geq \beta_i$. Thus,

$$\sum_i \beta_i \leq \sum_i \sum_{j:i \in S_j} c_j x_j^* \leq \sum_j c_j x_j^* |S_j| \leq d \sum_j c_j x_j^* = d \text{OPT}_{LP}.$$

■

Now we bound the *expected* second phase cost.

Lemma 2.4. $\mathbb{E}[C_2] \leq \text{OPT}_{LP}$.

Proof. We pay for a set to cover element i in the second phase only if it is not covered in the first phase. Hence $C_2 = \sum_i \mathcal{E}_i \beta_i$. Note that the events $\mathcal{E}_i$ for different elements i are not necessarily independent, however, we can apply linearity of expectation.

$$\mathbb{E}[C_2] = \sum_i \mathbb{E}[\mathcal{E}_i] \beta_i = \sum_i \mathbb{P}[\mathcal{E}_i] \beta_i \leq 1/d \sum_i \beta_i \leq \text{OPT}_{LP}.$$

■

Combining the expected costs of the two phases we obtain the following theorem.

Theorem 2.8. *Randomized rounding with alteration outputs a feasible solution of expected cost $(1 + \ln d) \text{OPT}_{LP}$.*

Note that the simplicity of the algorithm and tightness of the bound.

Is expectation bound good enough? We proved that the preceding algorithm outputs a feasible solution but the cost was only in the expected sense. A natural question is whether this is adequate. First, the analysis shows that there *exists* a feasible integral solution whose cost is $(1 + \ln d) \text{OPT}_{LP}$ (why?). From a practical point of view we may want to not just settle for an expectation bound. We can use standard tricks to convert an expectation bound into a stronger guarantee. Via Markov's inequality we can see that the probability that the cost of the output is more than $(1 + \epsilon)\mu$, where μ is the expected value, is at most $1/(1 + \epsilon) \simeq \epsilon$. Hence, if we repeat the experiment many times and take the minimum cost solution, we can get arbitrarily close to the expected value. We will need Chernoff bounds for this but we can argue that if repeat the experiment $O(\frac{c}{\epsilon} \log n)$ -times, the probability that the minimum among the solutions is more than $(1 + \epsilon)\mu$ is at most $1/n^c$. Thus we can obtain a guarantee on the cost that is arbitrarily close to the expected value with high probability. Typically we are satisfied with this for algorithmic purposes. In some cases we may really want a deterministic guarantee. One can then try to derandomize the algorithm via known ideas or come up with an alternative algorithm. We refer the reader to books and resources on randomized algorithms an in-depth discussion.

Remark 2.5. If $d = 2$ the SET COVER problem becomes the EDGE COVER problem in a graph which is the following. Given an edge-weighted graph $G = (V, E)$, find the minimum weight subset of edges such that each vertex is covered. EDGE COVER admits a polynomial-time algorithm via a reduction to the minimum-cost matching problem in a general graph. However $d = 3$ for SET COVER is NP-Hard via a reduction from 3-D MATCHING.

2.4.3 Dual-fitting

In this section, we introduce the technique of dual-fitting for the analysis of approximation algorithms. At a high-level the approach is the following:

1. Consider an algorithm that one wants to analyze.
2. Construct a feasible solution to the dual LP based on the structure of the algorithm.
3. Show that the cost of the solution returned by the algorithm can be bounded in terms of the value of the dual solution.

Note that the algorithm itself need not be LP based. Here, we use SET COVER as an example. See the previous section for the primal and dual LP formulations for SET COVER.

We can interpret the dual as follows: Think of y_i as how much element i is willing to pay to be covered; the dual maximizes the total payment, subject to the constraint that for each set, the total payment of elements in that set is at most the cost of the set.

We rewrite the Greedy algorithm for WEIGHTED SET COVER.

GREEDY SET COVER

1. $Covered = \emptyset$
2. $A = \emptyset$;
3. While $Covered \neq U$ do
 - A. $j \leftarrow \arg \min_k \left(\frac{w_k}{|S_k \cap Uncovered|} \right)$;
 - B. $Covered = Covered \cup S_j$;
 - C. $A = A \cup \{j\}$.
4. end while;
5. Output sets in A as cover

Let $H_k = 1 + 1/2 + \dots + 1/k$ be the k th Harmonic number. It is well known that $H_k \leq 1 + \ln k$.

Theorem 2.9. GREEDY SET COVER picks a solution of cost $\leq H_d \cdot \text{OPT}_{LP}$, where d is the maximum set size, i.e., $d = \max_j |S_j|$.

To prove this, we augment the algorithm to keep track of some additional information.

AUGMENTED GREEDY ALGORITHM OF WEIGHTED SET COVER

1. $Covered = \emptyset$
2. While $Covered \neq U$ do
 - A. $j \leftarrow \arg \min_k \left(\frac{w_k}{|S_k \cap Uncovered|} \right)$
 - B. if i is uncovered and $i \in S_j$, set $p_i = \frac{w_j}{|S_j \cap Uncovered|}$;
 - C. $Covered = Covered \cup S_j$
 - D. $A = A \cup \{j\}$.
3. Output sets in A as cover

It is easy to see that the algorithm outputs a feasible cover.

Claim 2.4.4. $\sum_{j \in A} w_j = \sum_i p_i$.

Proof. Consider when j is added to A . Let $S'_j \subseteq S_j$ be the elements that are uncovered before j is added. For each $i \in S'_j$ the algorithm sets $p_i = w_j / |S'_j|$. Hence, $\sum_{i \in S'_j} p_i = w_j$. Moreover, it is easy to see that the sets $S'_j, j \in A$ are disjoint and together partition U . Therefore,

$$\sum_{j \in A} w_j = \sum_{j \in A} \sum_{i \in S'_j} p_i = \sum_{i \in U} p_i.$$

■

For each i , let $y'_i = \frac{1}{H_d} p_i$.

Claim 2.4.5. y' is a feasible solution for the dual LP.

Suppose the claim is true, then the cost of GREEDY SET COVER's solution = $\sum_i p_i = H_d \sum_i y'_i \leq H_d \text{OPT}_{LP}$. The last step is because any feasible solution for the dual problem is a lower bound on the value of the primal LP (weak duality).

Now, we prove the claim. Let S_j be an arbitrary set, and let $|S_j| = t \leq d$. Let $S_j = \{i_1, i_2, \dots, i_t\}$, where the elements are ordered such that i_1 is covered by Greedy no-later than i_2 , and i_2 is covered no later than i_3 and so on.

Claim 2.4.6. For $1 \leq h \leq t$, $p_{i_h} \leq \frac{w_j}{t-h+1}$.

Proof. Let $S_{j'}$ be the set that covers i_h in Greedy. When Greedy picked $S_{j'}$ the elements $i_h, i_{h+1}, \dots, i_t$ from S_j were uncovered and hence Greedy could have picked S_j as well. This implies that the density of $S_{j'}$ when it was picked was no more than $\frac{w_j}{t-h+1}$. Therefore p_{i_h} which is set to the density of $S_{j'}$ is at most $\frac{w_j}{t-h+1}$. ■

From the above claim, we have

$$\sum_{1 \leq h \leq t} p_{i_h} \leq \sum_{1 \leq h \leq t} \frac{w_j}{t-h+1} = w_j H_t \leq w_j H_d.$$

Thus, the setting of y'_i to be p_i scaled down by a factor of H_d gives a feasible solution.

2.4.4 Greedy for implicit instances of SET COVER

SET COVER and the Greedy heuristic are quite useful in applications because many instances are *implicit*, nevertheless, the algorithm and the analysis applies. That is, the universe $\mathcal{U}$ of elements and the collection $\mathcal{S}$ of subsets of $\mathcal{U}$ need not be restricted to be finite or explicitly enumerated in the SET COVER problem. For instance, a problem could require covering a finite set of points in the plane using disks of unit radius. There is an infinite set of such disks, but the Greedy approximation algorithm can still be applied. For such implicit instances, the Greedy algorithm can be used if we have access to an *oracle*, which, at each iteration, selects a set having the optimal density. However, an oracle may not always be capable of selecting an optimal set. In some cases it may have to make the selections *approximately*. We call an oracle an α -*approximate oracle* for some $\alpha \geq 1$ if, at each iteration, it selects a set S such that $\frac{w(S)}{|S|} \leq \alpha \min_{A \text{ in collection}} \frac{w(A)}{|A|}$.

Exercise 2.6. Prove that the approximation guarantee of Greedy with an α -approximate oracle would be $\alpha(\ln n + 1)$ for SET COVER, and $(1 - \frac{1}{e^\alpha})$ for MAXIMUM COVERAGE.

We will see several examples of implicit use of the greedy analysis in the course.

2.5 Submodularity

SET COVER turns out to be a special case of a more general problem called SUBMODULAR SET COVER. The Greedy algorithm and analysis applies in this more generality. Submodularity is a fundamental notion with many applications in

combinatorial optimization and elsewhere. Here we take the opportunity to provide some definitions and a few results.

Definition 2.10. Given a finite set E , a real-valued set function $f : 2^E \rightarrow \mathbb{R}$ is submodular iff

$$f(A) + f(B) \geq f(A \cup B) + f(A \cap B) \quad \forall A, B \subseteq E.$$

Alternatively, f is a submodular function iff

$$f(A \cup \{i\}) - f(A) \geq f(B \cup \{i\}) - f(B) \quad \forall A \subset B, i \in E \setminus B.$$

The second characterization shows that submodularity is based on *decreasing marginal utility* property in the discrete setting. Adding element i to a set A will help at least as much as adding it to a (larger) set $B \supset A$. It is common to use $A + i$ to denote $A \cup \{i\}$ and $A - i$ to denote $A \setminus \{i\}$.

Exercise 2.7. Prove that the two characterizations of submodular functions are equivalent.

Many application of submodular functions are when f is a non-negative function though there are several important applications when f can be negative. A submodular function $f(\cdot)$ is *monotone* if $f(A + i) \geq f(A)$ for all $i \in E$ and $A \subseteq E$. Typically one assumes that f is normalized by which we mean that $f(\emptyset) = 0$; this can always be done by shifting the function by $f(\emptyset)$. f is *symmetric* if $f(A) = f(E \setminus A)$ for all $A \subseteq E$. Submodular set functions arise in a large number of fields including combinatorial optimization, probability, and geometry. Examples include rank function of a matroid, the sizes of cutsets in a directed or undirected graph, the probability that a subset of events do not occur simultaneously, entropy of random variables, etc. In the following we show that the SET COVER and MAXIMUM COVERAGE problems can be easily formulated in terms of submodular set functions.

Exercise 2.8. Let $\mathcal{U}$ be a set and let $\mathcal{S} = \{S_1, S_2, \dots, S_m\}$ be a finite collection of subsets of $\mathcal{U}$. Let $N = \{1, 2, \dots, m\}$, and define $f : 2^N \rightarrow \mathbb{R}$ as: $f(A) = |\cup_{i \in A} S_i|$ for $A \subseteq N$. Show that f is a monotone non-negative submodular set function.

Exercise 2.9. Let $G = (V, E)$ be a directed graph and let $f : 2^V \rightarrow \mathbb{R}$ where $f(S) = |\delta^+(S)|$ is the number of arcs leaving S . Prove that f is submodular. Is the function monotone?

2.5.1 SUBMODULAR SET COVER

When formulated in terms of submodular set functions, the SET COVER problem is the following. Given a monotone submodular function f (whose value would

be computed by an oracle) on $N = \{1, 2, \dots, m\}$, find the smallest set $S \subseteq N$ such that $f(S) = f(N)$. Our previous greedy approximation can be applied to this formulation as follows.

GREEDY SUBMODULAR (f, N)

1. $S \leftarrow \emptyset$
2. **While** $f(S) \neq f(N)$ **do**
 - A. find i to maximize $f(S + i) - f(S)$
 - B. $S \leftarrow S \cup \{i\}$
3. Output S

Not so easy exercise.

- Exercise 2.10.** 1. Prove that the greedy algorithm is a $1 + \ln(f(N))$ approximation for SUBMODULAR SET COVER?
2. Prove that the greedy algorithm is a $1 + \ln(\max_i f(i))$ approximation for SUBMODULAR SET COVER.

The above results were first obtained by Wolsey [258].

2.5.2 SUBMODULAR MAXIMUM COVERAGE

By formulating the MAXIMUM COVERAGE problem in terms of submodular functions, we seek to maximize $f(S)$ such that $|S| \leq k$. We can apply algorithm GREEDY SUBMODULAR for this problem by changing the condition in line 2 to be: **while** $|S| \leq k$.

Exercise 2.11. Prove that greedy gives a $(1 - 1/e)$ -approximation for SUBMODULAR MAXIMUM COVERAGE problem when f is monotone and non-negative. *Hint:* Generalize the main claim that we used for MAXIMUM COVERAGE.

The above and many related results were shown in the influential papers of Fisher, Nemhauser and Wolsey [112, 213].

2.6 Covering Integer Programs (CIPs)

There are several extensions of SET COVER that are interesting and useful. SUBMODULAR SET COVER is a very general problem while there are intermediate problems

of interest such as SET MULTICOVER. We refer to the reader to the relevant chapters in the two reference books. Here we refer to a general problem called COVERING INTEGER PROGRAMS (CIPs for short). The goal is to solve the following *integer* program where $A \in \mathbb{R}_+^{n \times m}$ is a *non-negative* matrix. We can assume without loss of generality that w and b are also non-negative.

$$\begin{aligned}
 & \min \quad \sum_{j=1}^n w_j x_j \\
 & \text{subject to} \\
 & \quad Ax \geq b \\
 & \quad x_j \leq d_j \quad 1 \leq j \leq m \\
 & \quad x_j \geq 0 \quad 1 \leq j \leq m \\
 & \quad x_j \in \mathbb{Z} \quad 1 \leq j \leq m
 \end{aligned}$$

$Ax \geq b$ model covering constraints and $x_j \leq d_j$ models multiplicity constraints. Note that SET COVER is a special case where A is simply the incidence matrix of the sets and elements (the columns correspond to sets and the rows to elements) and $d_j = 1$ for all j . What are CIPs modeling? It is a generalization of SET COVER. To see this, assume, without loss of generality, that A, b are integer matrices. For each element corresponding to row i the quantity b_i corresponds to the requirement of how many times i needs to be covered. A_{ij} corresponds to the number of times set S_j covers element i . d_j is an upper bound on the number of copies of set S_j that are allowed to be picked.

Exercise 2.12. Prove that CIPs are a special case of Submodular Set Cover.

One can apply the Greedy algorithm to the above problem and the standard analysis shows that the approximation ratio obtained is $O(\log B)$ where $B = \sum_i b_i$ (assuming that they are integers). Even though this is reasonable we would prefer a strongly polynomial bound. In fact there are instances where B is exponential in n and the worst-case approximation ratio can be poor. The natural LP relaxation of the above integer program has a large integrality gap in contrast to the case of SET COVER. One needs to *strengthen* the LP relaxation via what are known as *knapsack cover inequalities*. We refer the reader to the paper of Kolliopoulos and Young [180] and recent one by Chekuri and Quanrud [79] for more on this problem.

Chapter 3

Knapsack

In this lecture we explore the KNAPSACK problem. This problem provides a good basis for learning some important procedures used for approximation algorithms that give better solutions at the cost of higher running time.

3.1 The Knapsack Problem

In the KNAPSACK problem we are given a number (knapsack capacity) $B \geq 0$, and a set N of n items; each item i has a given size $s_i \geq 0$ and a profit $p_i \geq 0$. We will assume that all the input numbers are integers (or more generally rationals). Given a subset of the items $A \subseteq N$, we define two functions, $s(A) = \sum_{i \in A} s_i$ and $p(A) = \sum_{i \in A} p_i$, representing the total size and profit of the group of items respectively. The goal is to choose a subset of the items, A , such that $s(A) \leq B$ and $p(A)$ is maximized. We will assume, without loss of generality, that $s_i \leq B$ for all i ; we can discard items that do not satisfy this constraint.

It is not difficult to see that if all the profits are identical (say 1), the natural greedy algorithm that inserts items in the order of non-increasing sizes yields. Assuming the profits and sizes are integral, we can still find an optimal solution to the problem using dynamic programming in either $O(nB)$ or $O(nP)$ time, where $P = \sum_{i=1}^n p_i$. These are standard exercises. While these algorithms appear to run in polynomial time, it should be noted that B and P can be exponential in the size of the input written in binary. We call such algorithms *pseudo-polynomial time algorithms* as their running times are polynomial when numbers in the input are given in unary. KNAPSACK is a classical NP-Hard problem and these results (and the proof of NP-Hardness) show that the hardness manifests itself when the numbers are large (exponential in n which means that the number of bits in the size or profit are polynomial in n).

3.1.1 A Greedy Algorithm

Consider the following greedy algorithm for the KNAPSACK problem which we will refer to as GREEDYKNAPSACK. We sort all the items by the ratio of their profits to their sizes so that $\frac{p_1}{s_1} \geq \frac{p_2}{s_2} \geq \dots \geq \frac{p_n}{s_n}$. Afterward, we greedily take items in this order as long as adding an item to our collection does not exceed the capacity of the knapsack. It turns out that this algorithm can be arbitrarily bad. Suppose we only have two items in N . Let $s_1 = 1$, $p_1 = 2$, $s_2 = B$, and $p_2 = B$. GREEDYKNAPSACK will take only item 1, but taking only item 2 would be a better solution and the ratio of the profits in the two cases is $2/B$ which can be made arbitrarily small. As it turns out, we can easily modify this algorithm to provide a 2-approximation by simply taking the best of GREEDYKNAPSACK's solution or the most profitable item. We will call this new algorithm MODIFIEDGREEDY.

Theorem 3.1. *MODIFIEDGREEDY has an approximation ratio of $1/2$ for the KNAPSACK problem.*

Proof. Let k be the index of the first item that is not accepted by GREEDYKNAPSACK. Consider the following claim:

Claim 3.1.1. $p_1 + p_2 + \dots + p_k \geq \text{OPT}$. In fact, $p_1 + p_2 + \dots + \alpha p_k \geq \text{OPT}$ where $\alpha = \frac{B - (s_1 + s_2 + \dots + s_{k-1})}{s_k}$ is the fraction of item k that can still fit in the knapsack after packing the first $k - 1$ items.

The proof of Theorem 3.1 follows immediately from the claim. In particular, either $p_1 + p_2 + \dots + p_{k-1}$ or p_k must be at least $\text{OPT}/2$. We now only have to prove Claim 3.1.1. We give an LP relaxation of the KNAPSACK problem as follows: Here, $x_i \in [0, 1]$ denotes the fraction of item i packed in the knapsack.

$$\begin{aligned} & \text{maximize} && \sum_{i=1}^n p_i x_i \\ & \text{subject to} && \sum_{i=1}^n s_i x_i \leq B \\ & && x_i \leq 1 \text{ for all } i \text{ in } \{1 \dots n\} \\ & && x_i \geq 0 \text{ for all } i \text{ in } \{1 \dots n\} \end{aligned}$$

Let OPT_{LP} be the optimal value of the objective function in this linear programming instance. Any solution to KNAPSACK is a feasible solution to the LP and both problems share the same objective function, so $\text{OPT}_{LP} \geq \text{OPT}$. Now set $x_1 = x_2 = \dots = x_{k-1} = 1$, $x_k = \alpha$, and $x_i = 0$ for all $i > k$. This is a feasible solution to the LP. We leave it as an exercise to the reader to argue that it is an optimum solution. Therefore, $p_1 + p_2 + \dots + \alpha p_k = \text{OPT}' \geq \text{OPT}$. The first statement of the lemma follows from the second as $\alpha \leq 1$. ■

3.1.2 A Polynomial Time Approximation Scheme

Using the results from the last section, we make a few simple observations. Some of these lead to a better approximation.

Observation 3.2. *If for all i , $p_i \leq \epsilon \text{OPT}$, GREEDYKNAPSACK gives a $(1 - \epsilon)$ approximation.*

Proof. Follows easily from Claim 3.1.1. ■

Observation 3.3. *There are at most $\lceil \frac{1}{\epsilon} \rceil$ items with profit at least ϵOPT in any optimal solution.*

The next claim is perhaps more interesting and captures the intuition that the bad case for greedy happens only when there are “big” items.

Claim 3.1.2. *If for all i , $s_i \leq \epsilon B$, GREEDYKNAPSACK gives a $(1 - \epsilon)$ approximation.*

Proof. We give a proof sketch via the LP relaxation. Recall that k is the first item that did not fit in the knapsack. We make the following observation. Recall that OPT_{LP} is the optimum value of LP relaxation. Suppose we reduce the knapsack capacity to $B' = s_1 + s_2 + \dots + s_{k-1}$ while keeping all the items the same. Let OPT'_{LP} be the value for the new size. We claim that $\text{OPT}'_{LP} \geq \frac{B'}{B} \text{OPT}_{LP}$ — this is because we can take any feasible solution to the original LP and scale each variable by B'/B to obtain a feasible solution with the new capacity. What is OPT'_{LP} ? We note that Greedy will fill B' to capacity with the first $k - 1$ items and hence, $\text{OPT}'_{LP} = p_1 + \dots + p_{k-1}$. Combining, we obtain that

$$p_1 + \dots + p_{k-1} \geq \frac{B'}{B} \text{OPT}_{LP} \geq \frac{B'}{B} \text{OPT}.$$

We note that $B' + s_k \geq B$ since item k did not fit, and hence $B' \geq B - s_k \geq B - \epsilon B \geq (1 - \epsilon)B$. Therefore $B'/B \geq (1 - \epsilon)$ and this finishes the proof. ■

We may now describe the following algorithm. Let $\epsilon \in (0, 1)$ be a fixed constant and let $h = \lceil \frac{1}{\epsilon} \rceil$. We will try to guess the h most profitable items in an optimal solution and pack the rest greedily.

GUESS h + GREEDY(N, B)

1. For each $S \subseteq N$ such that $|S| \leq h$ and $s(S) \leq B$ do
 - A. Pack S in knapsack of size at most B
 - B. Let i be the least profitable item in S . Remove all items $j \in N - S$ where $p_j > p_i$.
 - C. Run GREEDYKNAPSACK on remaining items with remaining capacity $B - \sum_{i \in S} s_i$
2. Output best solution from above

Theorem 3.4. *GUESS h + GREEDY gives a $(1 - \epsilon)$ approximation and runs in $O(n^{\lceil 1/\epsilon \rceil + 1})$ time.*

Proof. For the running time, observe that there are $O(n^h)$ subsets of N . For each subset, we spend linear time greedily packing the remaining items. The time initially spent sorting the items can be ignored thanks to the rest of the running time.

For the approximation ratio, consider a run of the loop where S actually is the h most profitable items in an optimal solution and the algorithm's greedy stage packs the set of items $A' \subseteq (N - S)$. Let OPT' be the optimal way to pack the smaller items in $N - S$ so that $\text{OPT} = p(S) + \text{OPT}'$. Let item k be the first item rejected by the greedy packing of $N - S$. We know $p_k \leq \epsilon \text{OPT}$ so by Claim 3.1.1 $p(A') \geq \text{OPT}' - \epsilon \text{OPT}$. This means the total profit found in that run of the loop is $p(S) + p(A') \geq (1 - \epsilon) \text{OPT}$. ■

Note that for any fixed choice of $\epsilon > 0$, the preceding algorithm runs in polynomial time. This type of algorithm is known as a *polynomial time approximation scheme* or PTAS. The term “scheme” refers to the fact that the algorithm varies with ϵ . We say a maximization problem Π has a PTAS if for all $\epsilon > 0$, there exists a polynomial time algorithm that gives a $(1 - \epsilon)$ approximation ($(1 + \epsilon)$ for minimization problems). In general, one can often find a PTAS for a problem by greedily filling in a solution after first searching for a good basis on which to work. As described below, KNAPSACK actually has something stronger known as a *fully polynomial time approximation scheme* or FPTAS. A maximization problem Π has a FPTAS if for all $\epsilon > 0$, there exists an algorithm that gives a $(1 - \epsilon)$ approximation ($(1 + \epsilon)$ for minimization problems) and runs in time polynomial in both the input size and $1/\epsilon$.

3.1.3 Rounding and Scaling

Earlier we mentioned exact algorithms based on dynamic programming that run in $O(nB)$ and $O(nP)$ time but noted that B and P may be prohibitively large. If we could somehow decrease one of those to be polynomial in n without losing too much information, we might be able to find an approximation based on one of these algorithms. Let $p_{\max} = \max_i p_i$ and note the following.

Observation 3.5. $p_{\max} \leq \text{OPT} \leq np_{\max}$

Now, fix some $\epsilon \in (0, 1)$. We want to scale the profits and round them to be integers so we may use the $O(nP)$ algorithm efficiently while still keeping enough information in the numbers to allow for an accurate approximation. For each i , let $p'_i = \lfloor \frac{n}{\epsilon} \frac{1}{p_{\max}} p_i \rfloor$. Observe that $p'_i \leq \frac{n}{\epsilon}$ so now the sum of the profits P' is at most $\frac{n^2}{\epsilon}$. Also, note that we lost at most n profit from the scaled optimal solution during the rounding, but the scaled down OPT is still at least $\frac{n}{\epsilon}$. We have only lost an ϵ fraction of the solution. This process of rounding and scaling values for use in exact algorithms has use in a large number of other maximization problems. We now formally state the algorithm **ROUND&SCALE** and prove its correctness and running time.

ROUND&SCALE(N, B)

1. For each i set $p'_i = \lfloor \frac{n}{\epsilon} \frac{1}{p_{\max}} p_i \rfloor$
2. Run exact algorithm with run time $O(nP')$ to obtain A
3. Output A

Theorem 3.6. **ROUND&SCALE** gives a $(1 - \epsilon)$ approximation and runs in $O(\frac{n^3}{\epsilon})$ time.

Proof. The rounding can be done in linear time and as $P' = O(\frac{n^2}{\epsilon})$, the dynamic programming portion of the algorithm runs in $O(\frac{n^3}{\epsilon})$ time. To show the approximation ratio, let $\alpha = \frac{n}{\epsilon} \frac{1}{p_{\max}}$ and let A be the solution returned by the algorithm and A^* be the optimal solution. Observe that for all $X \subseteq N$, $\alpha p(X) - |X| \leq p'(X) \leq \alpha p(X)$ as the rounding lowers each scaled profit by at most 1. The algorithm returns the best choice for A given the scaled and rounded values, so we know $p'(A) \geq p'(A^*)$.

$$p(A) \geq \frac{1}{\alpha} p'(A) \geq \frac{1}{\alpha} p'(A^*) \geq p(A^*) - \frac{n}{\alpha} = \text{OPT} - \epsilon p_{\max} \geq (1 - \epsilon) \text{OPT}$$

■

It should be noted that this is not the best FPTAS known for KNAPSACK. In particular, [187] shows a FPTAS that runs in $O(n \log(1/\epsilon) + 1/\epsilon^4)$ time. There have been several improvements and we refer the reader to Chan's paper for the latest [59].

3.2 Other Problems

There are many variants of Knapsack and it is a fundamental problem of interest in integer programming as well in several other areas. One can find a book length treatment in [173]. We close with an interesting variant.

Multiple Knapsack: The input now consists of m knapsacks with capacities $B_1, B_2, \dots, B_m$ and n items with sizes and profits as in KNAPSACK. We again wish to pack items to obtain as large a profit as possible, except now we have more than one knapsack with which to do so. An interesting special case is when all the knapsack capacities are the same quantity B which is related to the well known Bin Packing problem.

Chapter 4

Packing Problems

In the previous lecture we discussed the KNAPSACK problem. In this lecture we discuss other *packing* and *independent set* problems. We first discuss an abstract model of packing problems. Let N be a finite ground set. A collection of $\mathcal{I} \subset 2^N$ of subsets of N is said to be *down closed* if the following property is true: $A \in \mathcal{I}$ implies that for all $B \subset A$, $B \in \mathcal{I}$. A down closed collection is also often called an *independence system*. The sets in $\mathcal{I}$ are called independent sets. Given an independence family $(N, \mathcal{I})$ and a non-negative weight function $w : N \rightarrow \mathbb{R}^+$ the maximum weight independent set problem is to find $\max_{S \in \mathcal{I}} w(S)$. That is, find an independent set in $\mathcal{I}$ of maximum weight. Often we may be interested in the setting where all weights are 1 in which case we wish to find the maximum cardinality independent set. We will be assuming that we have an oracle or an efficient algorithm for the following problem: given $S \subseteq N$ check whether $S \in \mathcal{I}$. If we do not have such an oracle it does not make much sense to discuss approximation for the maximum weight independent set. We discuss some canonical examples.

Example 4.1. Independent sets in graphs: Given a graph $G = (V, E)$ $\mathcal{I} = \{S \subseteq V \mid \text{there are no edges between nodes in } S\}$. Here the ground set is V . There are many interesting special cases of the graph problem. For instance problems arising from geometric objects such as intervals, rectangles, disks and others.

Example 4.2. Matchings in graphs: Given a graph $G = (V, E)$ let $\mathcal{I} = \{M \subseteq E \mid M \text{ is a matching in } G\}$. Here the ground set is E .

Example 4.3. Matroids: A matroid $\mathcal{M} = (N, \mathcal{I})$ is defined as a system where $\mathcal{I}$ is down closed and in addition satisfies the following key property: if $A, B \in \mathcal{I}$ and $|B| > |A|$ then there is an element $e \in B \setminus A$ such that $A \cup \{e\} \in \mathcal{I}$. There are many examples of matroids. We will not go into details here.

Example 4.4. Intersections of independence systems: given some k independence systems on the same ground set $(N, \mathcal{I}_1), (N, \mathcal{I}_2), \dots, (N, \mathcal{I}_k)$ the system defined by $(N, \mathcal{I}_1 \cap \mathcal{I}_2 \dots \cap \mathcal{I}_k)$ is also an independence system. Well-known examples include intersections of matroids.

4.0.1 Rounding and scaling of weights

This subsection can be ignored on a first reading.

In the chapter on KNAPSACK we saw a technique to round and scale the profits. Given any fixed $\epsilon \in (0, 1)$, the technique uses guessing/enumeration to reduce profits to non-negative integers between 1 and $\lceil n/\epsilon \rceil$. We observe that this idea can be ported over to any packing problem where we reduce the weights to be integer in the same range.

Geometric grouping of weights: We discuss a second idea that is often useful in packing problem. Once the weights are integers in $[1, n/\epsilon]$ we can further simplify them by considering geometrically increasing intervals. Consider the intervals $[1, (1 + \epsilon)), [(1 + \epsilon), (1 + \epsilon)^2), \dots, [(1 + \epsilon)^{k-1}, (1 + \epsilon)^k]$ where k is the smallest integer such that $(1 + \epsilon)^k > \max_i w_i$. Suppose we round down each weight in the interval $[(1 + \epsilon)^i, (1 + \epsilon)^{i+1})$ to $(1 + \epsilon)^i$. We lose at most a $\frac{1}{(1 + \epsilon)} \simeq (1 - \epsilon)$ factor in the approximation. With this rounding-down we have reduced the number of *distinct* weights in the instance to $O(\frac{\log n}{\epsilon})$. This can often be leveraged in various ways depending on the application and we will see some examples later on. One easy and crude use of this idea is the following. Suppose we have a black-box algorithm for a packing problem that only works for unweighted instances, that is, maximum cardinality instances. Can we use it for weighted instances in some fashion? Via the weight rounding idea, if we have an α -approximation for the unweighted problem we can obtain an $\frac{\alpha}{c \log n}$ -approximation for the weighted problem for some fixed constant c . Do you see how? This crude result may not seem interesting but for some problems this is the only approach known to handle the weighted case even though we know a clever and non-trivial algorithm for the unweighted case; an example is the problem considered in [74].

4.1 Maximum Independent Set Problem in Graphs

A basic graph optimization problem with many applications is the *maximum (weighted) independent set problem* (MIS) in graphs.

Definition 4.1. Given an undirected graph $G = (V, E)$ a subset of nodes $S \subseteq V$ is an independent set (stable set) iff there is no edge in E between any two nodes in S . A subset of nodes S is a clique if every pair of nodes in S have an edge between them in G .

The MIS problem is the following: given a graph $G = (V, E)$ find an independent set in G of maximum cardinality. In the weighted case, each node $v \in V$ has an associated non-negative weight $w(v)$ and the goal is to find a maximum weight independent set. This problem is NP-Hard and it is natural to ask for approximation algorithms. Unfortunately, as the famous theorem below shows, the problem is extremely hard to approximate.

Theorem 4.2 (Håstad [144]). Unless $P = NP$ there is no $\frac{1}{n^{1-\epsilon}}$ -approximation for MIS for any fixed $\epsilon > 0$ where n is the number of nodes in the given graph.

Remark 4.1. The maximum clique problem is to find the maximum cardinality clique in a given graph. It is approximation-equivalent to the MIS problem; simply complement the graph.

The theorem basically says the following: there are a class of graphs in which the maximum independent set size is either less than n^δ or greater than $n^{1-\delta}$ and it is NP-Complete to decide whether a given graph falls into the former category or the latter.

The lower bound result suggests that one should focus on special cases, and several interesting positive results are known. First, we consider a simple greedy algorithm for the unweighted problem.

GREEDY(G)

1. $S \leftarrow \emptyset$
2. While G is not empty do
 - A. Let v be a node of minimum degree in G
 - B. $S \leftarrow S \cup \{v\}$
 - C. Remove v and its neighbors from G
3. Output S

Theorem 4.3. Greedy outputs an independent set S such that $|S| \geq n/(\Delta + 1)$ where Δ is the maximum degree of any node in the graph. Moreover $|S| \geq \alpha(G)/\Delta$ where $\alpha(G)$ is the cardinality of the largest independent set. Thus Greedy is a $1/\Delta$ approximation.

Proof. We upper bound the number of nodes in $V \setminus S$ as follows. A node u is in $V \setminus S$ because it is removed as a neighbor of some node $v \in S$ when Greedy

added v to S . Charge u to v . A node $v \in S$ can be charged at most Δ times since it has at most Δ neighbors. Hence we have that $|V \setminus S| \leq \Delta|S|$. Since every node is either in S or $V \setminus S$ we have $|S| + |V \setminus S| = n$ and therefore $(\Delta + 1)|S| \geq n$ which implies that $|S| \geq n/(\Delta + 1)$.

We now argue that $|S| \geq \alpha(G)/\Delta$. Let S^* be a largest independent set in G . As in the above proof we can charge each node v in $S^* \setminus S$ to a node $u \in S \setminus S^*$ which is a neighbor of v . The number of nodes charged to a node $u \in S \setminus S^*$ is at most Δ . Thus $|S^* \setminus S| \leq \Delta|S \setminus S^*|$. This implies that $|S^*| = |S^* \cap S| + |S^* \setminus S| \leq |S^* \cap S| + \Delta|S \setminus S^*| \leq \Delta|S|$. This implies that $|S| \geq |S^*|/\Delta$. ■

Exercise 4.1. Show that Greedy outputs an independent set of size at least $\frac{n}{2(d+1)}$ where d is the *average* degree of G .

Remark 4.2. The well-known Turan's theorem shows via a clever argument that there is always an independent set of size $\frac{n}{(d+1)}$ where d is the average degree of G .

Remark 4.3. For the case of unweighted graphs one can obtain an approximation ratio of $\Omega(\frac{\log d}{d \log \log d})$ where d is the average degree. Surprisingly, under a complexity theory conjecture called the Unique-Games conjecture it is known to be NP-Hard to approximate MIS to within a factor of $O(\frac{\log^2 \Delta}{\Delta})$ in graphs with maximum degree Δ when Δ is sufficiently large.

Exercise 4.2. Consider the weighted MIS problem on graphs of maximum degree Δ . Alter Greedy to sort the nodes in non-increasing order of the weight and show that it gives a $\frac{1}{\Delta}$ -approximation. Can one obtain an $\Omega(1/d)$ -approximation for the weighted case where d is the average degree?

LP Relaxation: One can formulate a simple linear-programming relaxation for the (weighted) MIS problem where we have a variable $x(v)$ for each node $v \in V$ indicating whether v is chosen in the independent set or not. We have constraints which state that for each edge (u, v) only one of u or v can be chosen.

$$\begin{aligned} & \text{maximize } \sum_{v \in V} w(v)x(v) \\ & \text{subject to } x(u) + x(v) \leq 1 \quad (u, v) \in E \\ & \quad \quad \quad x(v) \in [0, 1] \quad v \in V \end{aligned}$$

Although the above is a valid integer programming relaxation of MIS when the variables are constrained to be in $\{0, 1\}$, it is not a particularly useful formulation for the following simple reason.

Claim 4.1.1. *For any graph the optimum value of the above LP relaxation is at least $w(V)/2$. In particular, for the unweighted case it is at least $n/2$.*

Simply set each $x(v)$ to $1/2$!

One can obtain a *strengthened* formulation below by observing that if S is clique in G then any independent set can pick at most one node from S .

$$\begin{aligned} & \text{maximize } \sum_{v \in V} w(v)x(v) \\ & \text{subject to } \sum_{v \in S} x(v) \leq 1 \quad S \text{ is a clique in } G \\ & \quad x(v) \in [0, 1] \quad v \in V \end{aligned}$$

The above linear program has an exponential number of constraints, and it cannot be solved in polynomial time in general, but for some special cases of interest the above linear program can indeed be solved (or approximately solved) in polynomial time and leads to either exact algorithms or good approximation bounds. Another set of inequalities are the so called odd-cycle inequalities. Suppose C is an odd-length cycle in G . It is easy to see that at most $(|C| - 1)/2$ vertices from C can be in any independent set. Hence the following is a set of valid inequalities.

$$\sum_{v \in C} x_v \leq (|C| - 1)/2 \quad C \text{ is an odd cycle in } G$$

There can be an exponential number of such inequalities, however, one can show that there is an efficient separation oracle for them.

Approximability of VERTEX COVER and MIS: The following is a basic fact and is easy to prove.

Fact 4.1. *In any graph $G = (V, E)$, S is a vertex cover in G if and only if $V \setminus S$ is an independent set in G . Thus $\alpha(G) + \beta(G) = |V|$ where $\alpha(G)$ is the size of a maximum independent set in G and $\beta(G)$ is the size of a minimum vertex cover in G .*

The above shows that if one of VERTEX COVER or MIS is NP-Hard then the other is as well. We have seen that VERTEX COVER admits a 2-approximation while MIS admits no constant factor approximation. It is useful to see why a 2-approximation for VERTEX COVER does not give any useful information for MIS even though $\alpha(G) + \beta(G) = |V|$. Suppose S^* is an optimal vertex cover and has size $\geq |V|/2$. Then a 2-approximation algorithm is only guaranteed to give a vertex cover of size $|V|$! Hence one does not obtain a non-trivial independent set by complementing the approximate vertex cover.

Some special cases of MIS: We mention some special cases of MIS that have been considered in the literature, this is by no means an exhaustive list.

- Interval graphs; these are intersection graphs of intervals on a line. An exact algorithm can be obtained via dynamic programming and one can solve more general versions via linear programming methods. Interval graphs are a special case of chordal graphs which also admit a polynomial-time algorithm for MWIS.
- Note that a maximum (weight) matching in a graph G can be viewed as a maximum (weight) independent set in the line-graph of G and can be solved exactly in polynomial time. This has been extended to what are known as claw-free graphs.
- Perfect graphs. A fundamental and celebrated result in combinatorial optimization is an efficient algorithm for MWIS in perfect graphs. These graphs are well-known in graph theory and capture several other classes including chordal graphs. One characterization is that a graph $G = (V, E)$ is perfect if for each vertex-induced subgraph $G[S]$ of G , $\chi(G[S]) = \omega(G[S])$, that is the clique number is equal to the coloring number. Another characterization, called the strong perfect graph theorem, is that G is perfect iff G does not contain an odd hole (an induced subgraph that is an odd cycle of length ≥ 5) or an odd antihole (a complement of an odd hole). The MWIS algorithm for perfect graphs was first shown by Lovász via semi-definite programming.
- Planar graphs and generalizations to bounded-genus graphs, and graphs that exclude a fixed minor. For such graphs one can obtain a PTAS due to ideas originally from Brenda Baker.
- Geometric intersection graphs. For example, given n disks on the plane find a maximum number of disks that do not overlap. One could consider other (convex) shapes such as axis parallel rectangles, line segments, pseudo-disks etc. A number of results are known. For example a PTAS is known for disks in the plane. An $\Omega(\frac{1}{\log n})$ -approximation for axis-parallel rectangles in the plane when the rectangles are weighted and a recent $\frac{1}{2+\epsilon}$ -approximation for the unweighted case.

4.1.1 Elimination Orders and MIS and MWIS

We have seen that a simple Greedy algorithm gives a Δ -approximation for MIS in graphs with max degree Δ . One can also get a Δ approximation for a larger class of Δ -degenerate graphs. To motivate degenerate graphs consider the class

of planar graphs. The maximum degree of a planar graph need not be small. Nevertheless, via Euler's theorem, we know that every planar graph has a vertex of degree at most 5 since the maximum number of edges in a planar graph is at most $3n - 6$. Moreover, every subgraph of a planar graph is planar, and hence the Greedy algorithm will repeatedly find a vertex of degree at most 5 in each iteration. From this one can show that Greedy gives a $1/5$ -approximation for MIS in planar graphs. Now consider the intersection graph of a collection of intervals on the real line. That is, we are given n intervals $I_1, I_2, \dots, I_n$ where each $I_i = [a_i, b_i]$ for real numbers $a_i \leq b_i$. The goal is to find a maximum number of the intervals in the given set of intervals which do not overlap. This is the same as finding MIS in the *intersection graph* of the intervals - the graph is obtained by creating a vertex v_i for each I_i , and by adding edges $v_i v_j$ if I_i and I_j overlap. It is well-known that greedily picking intervals in earliest finish time order (ordering them according to b_i values) is optimal; the reader should try to prove this. Can one understand the analysis of all these examples in a unified fashion? Yes. For this purpose we consider the class of inductive k -independent graphs considered by Akcoglu et al. [11] and later again by Ye and Borodin [259].

For a vertex v in a graph we use $N(v)$ to denote the neighbors of v (not including v itself). For a graph $G = (V, E)$ and $S \subset V$ we use $G[S]$ to denote the subgraph of G induced by S .

Definition 4.4 ([11, 259]). An undirected graph $G = (V, E)$ is *inductively k -independent* if there is an ordering of the vertices $v_1, v_2, \dots, v_n$ such that for $1 \leq i \leq n$, $\alpha(G[N(v_i) \cap \{v_{i+1}, \dots, v_n\}]) \leq k$.

Typically one establishes that a graph is inductively k -independent by showing that it belongs to the following class.

Definition 4.5 ([165]). An undirected graph $G = (V, E)$ is *k -simplicial* if there is an ordering of the vertices $v_1, v_2, \dots, v_n$ such that for $1 \leq i \leq n$, $G[N(v_i) \cap \{v_{i+1}, \dots, v_n\}]$ can be covered by k cliques.

Graphs which are inductively 1-independent have a *perfect* elimination ordering and are called chordal graphs because they have an alternate characterization. A graph is chordal iff each cycle C in G has a chord (an edge connecting two nodes of C which is not an edge of C), or in other words there is no induced cycle of length more than 3.

Exercise 4.3. Prove that the intersection graph of intervals is chordal.

Exercise 4.4. Prove that if $\Delta(G) \leq k$ then G is inductively k -independent. Prove that if G is k -degenerate then G is inductively k -independent.

The preceding shows that planar graphs are inductively 5-independent. In fact, one can show something stronger, that they are inductively 3-independent. Given a graph G one can ask whether there is an algorithm that checks whether G is inductively k -independent. There is such an algorithm that runs in time $O(k^2 n^{k+2})$ [259]. A classical result shows how to recognize chordal graphs ($k = 1$) in linear time. However, most of the useful applications arise by showing that a certain class of graphs are inductively k -independent for some small value of k . See [259] for several examples.

Exercise 4.5. Prove that the Greedy algorithm that considers the vertices in the inductive k -independent order gives a $\frac{1}{k}$ -approximation for MIS.

Interestingly one can obtain a $\frac{1}{k}$ -approximation for the maximum weight independent set problem in inductively k -independent graphs. The algorithm is simple and runs in linear time but is not obvious. To see this consider the weighted problem for intervals. The standard algorithm to solve this is via dynamic programming. However, one can obtain an optimum solution for all chordal graphs (given the ordering). We refer the reader to [259] for the algorithm and proof (originally from [11]). One can interpret the algorithm as a primal-dual algorithm with respect to an LP relaxation that we write below. We note that this relaxation is valid only for this specific class of graphs and when given a valid ordering.

LP Relaxation: Let $G = (V, E)$ be an inductively k -independent graph and let $v_1, v_2, \dots, v_n$ be the corresponding order. Let $\mathcal{I}$ denote the set of independent sets of G . We consider the following simple polyhedral relaxation for $\mathcal{I}$ where there is a variable x_i for each vertex v_i . For notational simplicity we let A_i denote the set $N(v_i) \cap \{v_{i+1}, \dots, v_n\}$ which is the set of neighbors of v_i that come after v_i in the ordering.

$$\begin{aligned} \max \quad & \sum_{i=1}^n w_i x_i \\ \text{s.t.} \quad & x_i + \sum_{v_j \in A_i} x_j \leq k \quad \text{for all } i \in [n] \\ & x_i \in [0, 1] \quad \text{for all } i \in [n] \end{aligned}$$

A stack based algorithm and its primal-dual interpretation: We show that a stack based algorithm in [259] can be interpreted as a primal-dual approximation algorithm via the LP relaxation Q_G that we saw previously. We state the primal and dual LPs below. The primal is the MWIS LP with x_i denoting whether v_i is chosen in the independent set. The dual can be seen as a covering LP. Recall that

A_i is the set of neighbors of v_i that come after it in the ordering. Similarly we let $B_i = N(v_i) \cap \{v_1, \dots, v_{i-1}\}$ denote the set of neighbors of v_i that come before v_i in the ordering.

$$\begin{array}{c|c}
 \max \sum_{i=1}^n w_i x_i & \min k \sum_{i=1}^n y_i \\
 x_i + \sum_{v_j \in A_i} x_j \leq k \quad i \in [n] & y_i + \sum_{v_j \in B_i} y_j \geq w_i \quad i \in [n] \\
 x_i \geq 0 \quad i \in [n] & y_i \geq 0 \quad i \in [n]
 \end{array}$$

Figure 4.1: Primal and Dual LPs for MWIS in an inductively k -independent graph G with a given ordering $\{v_1, v_2, \dots, v_n\}$.

Remark 4.4. We observe that the primal LP relaxation does not enforce the condition that $x_i \leq 1$. Thus the relaxation allows up to k copies of a vertex to be chosen. The primal-dual algorithm chooses at most one copy of a vertex. The analysis shows that the integrality gap is at most $1/k$ even with the relaxation. The advantage of dropping the $x_i \leq 1$ constraints is a simpler dual.

The primal-dual algorithm is described below. It has two phases, a growing phase in which a set $S \subseteq V$ is created. This is guided by a dual solution y which processes vertices in the inductive k -independent order. In the second phase the vertices in S are processed in the reverse order to create a maximal independent set $\hat{S}$ from S .

STACK($G = (V, E), w : V \rightarrow \mathbb{R}_+, \{v_1, v_2, \dots, v_n\}, k$)

1. Initialize an empty stack $S \leftarrow \emptyset$. Initialize primal and dual solutions $x \leftarrow \mathbb{0}$ and $y \leftarrow \mathbb{0}$.
2. for $i = 1, \dots, n$:
 - Let $y_i = \max\{0, w_i - \sum_{j < i, v_i \in A_j} y_j\}$
 - If $y_i > 0$ then call $S.\text{push}(v_i)$ and set $x_i = 1$.
3. Let $\hat{S} \leftarrow \emptyset$.
4. While S is not empty:
 - $v \leftarrow S.\text{pop}()$
 - If $\hat{S} \cap N(v) = \emptyset$ then set $\hat{S} \leftarrow \hat{S} + v$
5. Output $\hat{S}$

We now analyze the algorithm. Let S_{end} be the set of vertices in the stack S at the end of the first phase, and let $\hat{S} \subseteq S_{\text{end}}$ be the final output of the algorithm. The following observations are easy to verify and we omit a formal proof.

- The algorithm constructs a feasible dual solution y .
- A vertex $v_i \in S_{\text{end}}$ (that is $x_i = 1$) during the first phase iff $y_i > 0$. Further, if $v_i \in S_{\text{end}}$ then the dual constraint for v_i is tight: $y_i + \sum_{j < i, v_i \in A_j} y_j = w_i$. In other words the algorithm maintains primal complementary slackness condition for x, y .
- $\hat{S}$ is a maximal independent set in $G[S_{\text{end}}]$.

The key claim is the following:

Lemma 4.1. *Let $\hat{S}$ be the set of vertices output by the algorithm. Then $w(\hat{S}) \geq \sum_i y_i$ where y is the dual constructed by the algorithm.*

Proof. We observed that $x_i = 1$ implies that $y_i + \sum_{j < i, v_i \in A_j} y_j = w_i$. Hence, $w(\hat{S}) = \sum_{v_i \in \hat{S}} (y_i + \sum_{j < i, v_i \in A_j} y_j)$. Suppose $v_{i'} \in S_{\text{end}} \setminus \hat{S}$, then $v_{i'}$ was considered in the second phase but was not included since there was some $i > i'$ such that $v_i \in \hat{S}$ and $v_i \in A_{i'}$. This implies that $y_{i'}$ is counted in the term $\sum_{j < i, v_i \in A_j} y_j$.

Thus, for every $v_j \in S_{\text{end}}$, the dual variable y_j is included in the sum, and hence,

$$w(\hat{S}) = \sum_{v_i \in \hat{S}} (y_i + \sum_{j < i, v_i \in A_j} y_j) \geq \sum_{i \in S_{\text{end}}} y_i = \sum_i y_i$$

where the last equality follows from the fact that $v_i \in S_{\text{end}}$ iff $y_i > 0$. ■

Thus the algorithm outputs a feasible independent set $\hat{S}$ such that $w(\hat{S}) \geq \sum_i y_i$. But note that dual value $k \sum_i y_i$ is an upper bound on the optimum LP value OPT_{LP} since y is feasible. Thus $w(\hat{S}) \geq \frac{1}{k} \text{OPT}_{LP}$ which proves that the algorithm yields a $\frac{1}{k}$ approximation with respect to the LP relaxation Q_G (in fact the relaxation that drops the constraints $x_i \leq 1, i \in [n]$).

4.1.2 MWIS in k -perfectly orientable graphs via random rounding plus alteration

We now consider another class of graphs that have significantly more modeling power. Given an undirected graph $G = (V, E)$ we call a directed graph $H = (V, A)$ an *orientation* of G if H is obtained by taking each edge $e = uv$ of G and replacing it by an arc (u, v) or by an arc (v, u) .

Definition 4.6 ([165]). *For a fixed integer $k \geq 1$, $G = (V, E)$ is k -perfectly orientable if there is an orientation $H = (V, A)$ of G such that for each vertex $v \in V$, $\alpha(G[S_v]) \leq k$ where $S_v = N_H^+(v)$ is the out-neighborhood of v in H .*

Exercise 4.6. Argue that an odd cycle C is 2-inductively independent and 1-perfectly orientable.

Exercise 4.7. Show that if G is inductively k -independent then it is k -perfectly orientable.

There are graphs that are 2-perfectly orientable graphs on n vertices but they are not $\sqrt{n}$ -independent. Thus orientations give substantial power. The following theorem is quite neat but not that easy.

Theorem 4.7. *There exists a $\frac{1}{2k}$ -approximation for MWIS in k -perfectly-orientable graphs if the ordering is given.*

We will instead prove a slightly weaker result and also use the opportunity to showcase a randomized rounding plus alteration technique for packing problems that is quite useful.

LP Relaxation: Let $G = (V, E)$ be a k -perfectly orientable graph. Let $H = (V, A)$ be an orientation of G . For a given $v \in V$ we let $N_H^+(v) = \{u \in V \mid (v, u) \in A\}$ denote the out-neighbors of v in H . We can write a simple polyhedral relaxation for independent sets in G where we have a variable x_v for each $v \in V$ as follows:

$$\begin{aligned} \max \quad & \sum_{v \in V} w_v x_v \\ \text{s.t.} \quad & x_v + \sum_{u \in N_H^+(v)} x_u \leq k \quad \text{for all } v \in V \\ & x_v \in [0, 1] \quad \text{for all } v \in V \end{aligned}$$

We will use Q_G to refer to the preceding relaxation for a k -perfectly orientable graph G . Let x^* be an optimum or feasible solution to the relaxation. We have $\sum_v w_v x_v^* \geq \text{OPT}$. We consider two rounding schemes.

ROUND-AND-ALTER(G)

1. Let x^* be an optimum solution to the LP relaxation
2. Pick each v independently with probability $\frac{b}{k} x_v^*$. Let R be this random set of vertices
3. Let $S = \{v \in R \mid N_H^+(v) \cap R = \emptyset\}$. In other words, discard any vertex $v \in R$ if it has any out neighbor from H in R .
4. Output S .

Lemma 4.2. *The output of the algorithm is an independent set of G .*

Proof. We note that $S \subseteq R$. We claim that if both u and v are vertices in S then uv cannot be an edge of G . To see this, we note that H contains an orientation of uv . Suppose (u, v) is the arc in H corresponding to uv . Then $u, v \in R$ but then we would not have included u in S since $v \in N_H^+(u)$. Similarly if (v, u) was the arc in H we would not include v in S . ■

Lemma 4.3. *For any $b \in (0, 1)$ the probability that a vertex v is in the output set S is at least $b(1 - b)x_v^*/k$.*

Proof. Let $\mathcal{E}_1$ be the event that $v \in R$ and let $\mathcal{E}_2$ be the event that $v \in S$. We see that $\mathbf{P}[v \in R] = bx_v^*/k$. We want to estimate $\mathbf{P}[\mathcal{E}_2 \mid \mathcal{E}_1]$. Instead we upper bound $\mathbf{P}[\bar{\mathcal{E}}_2 \mid \mathcal{E}_1]$ — this is the event that v is chosen in R but discarded because one of its neighbors is also in R . Since $\sum_{u \in N_H^+(v)} x_u^* \leq k$, we see that $\mathbf{P}[\bar{\mathcal{E}}_2 \mid \mathcal{E}_1] \leq b$. Thus $\mathbf{P}[\mathcal{E}_2 \mid \mathcal{E}_1] \geq (1 - b)$. Therefore, $\mathbf{P}[\mathcal{E}_2] = \mathbf{P}[\mathcal{E}_1] \cdot \mathbf{P}[\mathcal{E}_2 \mid \mathcal{E}_1] \geq (1 - b)bx_v^*/k$. ■

Via linearity of expectation we obtain the following.

Theorem 4.8. *The expected weight of the set S is at least $\frac{b(1-b)}{k} \sum_v w_v x_v^*$. Thus, for $b = 1/2$ the algorithm is a randomized $\frac{1}{4k}$ -approximation.*

4.2 The efficacy of the Greedy algorithm for a class of Independence Families

The Greedy algorithm can be defined easily for an arbitrary independence system. It iteratively adds the best element to the current independent set while maintaining feasibility. Note that the implementation of the algorithm requires having an oracle to find the best element to add to a current independent set S .

GREEDY($N, \mathcal{I}$)

1. $S \leftarrow \emptyset$
2. While (TRUE)
 - A. Let $A \leftarrow \{e \in N \setminus S \mid S + e \in \mathcal{I}\}$
 - B. If $A = \emptyset$ break
 - C. $e \leftarrow \operatorname{argmax}_{e \in A} w(e)$
 - D. $S \leftarrow S \cup \{e\}$
3. Output S

Exercise 4.8. Prove that the Greedy algorithm gives a $1/2$ -approximation for the maximum weight matching problem in a general graph. Also prove that this bound is tight even in bipartite graphs. Note that max weight matching can be solved exactly in polynomial time.

Remark 4.5. It is well-known that the Greedy algorithm gives an optimum solution when $(N, \mathcal{I})$ is a matroid. Kruskal's algorithm for min/max weight spanning tree is a special case of this fact.

It is easy to see that Greedy does poorly for MIS problem in general graphs. A natural question is what properties of $\mathcal{I}$ enable some reasonable performance guarantee for Greedy. A very general result in this context has been established due to Jenkyn's generalizing several previous results. In order to state the result we set up some notation. Given an independence system $(N, \mathcal{I})$ we say that a set $A \in \mathcal{I}$ is a *base* if it is a *maximal* independent set. It is well-known that in

a matroid $\mathcal{M}$ all bases have the same cardinality. However this is not true in general independence system.

Definition 4.9. An independence system $(N, \mathcal{I})$ is a k -system if for any two bases $A, B \in \mathcal{I}$, $|A| \leq k|B|$. That is, the ratio of the cardinality of a maximum base and the cardinality of a minimum base is at most k .

The following theorem is not too difficult but not so obvious either.

Theorem 4.10. Greedy gives a $1/k$ -approximation for the maximum weight independent set problem in a k -system.

The above theorem generalizes and unifies several examples that we have seen so far including MIS in bounded degree graphs, matchings, matroids etc. How does one see that a given independence system is indeed a k -system for some parameter k ? For instance matchings in graphs form a 2-system. The following simple lemma gives an easy way to argue that a given system is a k -system.

Lemma 4.4. Suppose $(N, \mathcal{I})$ is an independence system with the following property: for any $A \in \mathcal{I}$ and $e \in N \setminus A$ there is a set $Y \subset A$ such that $|Y| \leq k$ and $(A \setminus Y) \cup \{e\} \in \mathcal{I}$. Then $\mathcal{I}$ is a k -system.

We leave the proof of the above as an exercise.

We refer the reader to [109, 202] for analysis of Greedy in k -systems and other special cases.

4.3 Randomized Rounding with Alteration for Packing Problems

The purpose of this section is to highlight a technique for rounding LP relaxations for packing problems. We will consider a simple example, namely the maximum weight independent set problem in interval graphs. Recall that we are given n intervals $I_1, I_2, \dots, I_n$ with non-negative weights $w_1, \dots, w_n$ and the goal is to find a maximum weight subset of them which do not overlap. Let $I_i = [a_i, b_i]$ and let $p_1, p_2, \dots, p_m$ be the collection of end points of the intervals. We can write a simple LP relaxation for this problem. For each interval i we have a variable $x_i \in [0, 1]$ to indicate whether I_i is chosen or not. For each point p_j , among all intervals that contain it, at most one can be chosen. These are clique constraints in the underlying interval graph.

$$\begin{aligned}
& \text{maximize} && \sum_{i=1}^n w_i x_i \\
& \text{subject to} && \sum_{i: p_j \in I_i} x_i \leq 1 \quad 1 \leq j \leq m \\
& && x_i \in [0, 1] \quad 1 \leq i \leq n
\end{aligned}$$

Note that it is important to retain the constraint that $x_i \leq 1$. Interestingly it is known that the LP relaxation defines an integer polytope and hence one can solve the integer program by solving the LP relaxation! This is because the incidence matrix defining the LP is totally unimodular (TUM). We refer the reader to books on combinatorial optimization for further background on this topic. Here we assume that we do not know the integer properties of the LP. We will round it via a technique that is powerful and generalizes to NP-Hard variants of the interval scheduling problem among many others.

Suppose we solve the LP and obtain an optimum fraction solution x^* . We have $\sum_i w_i x_i^* \geq OPT$. How do we round to obtain an integer solution whose value is close to that of OPT ? Suppose we randomly choose I_i with probability cx_i^* for some $c \leq 1$. Let R be the random set of chosen intervals. Then the expected weight of R , by linearity of expectation, is $c \sum_i w_i x_i^* \geq c \cdot OPT$. However, it is highly likely that the random solution R is not going to be feasible. Some constraint will be violated. The question is how we can *fix* or *alter* R to find a subset $R' \subseteq R$ such that R' is a feasible solution *and* the expected value of R' is not too much smaller than that of R . This depends on the independence structure.

Here we illustrate this via the interval problem. Without loss of generality we assume that $I_1, \dots, I_n$ are sorted by their right end point. In other words the order is a perfect elimination order for the underlying interval graph.

ROUNDING-WITH-ALTERATION

1. Let x be an optimum fractional solution
2. Round each i to 1 independently with probability $x_i/2$. Let x' be rounded solution.
3. $R \leftarrow \{i \mid x'_i = 1\}$
4. $S \leftarrow \emptyset$
5. For $i = n$ **down to** 1 do
 - A. If $(i \in R)$ and $(S \cup \{i\}$ is feasible) then $S \leftarrow S \cup \{i\}$
6. Output feasible solution S

The algorithm consists of two phases. The first phase is a simple selection phase via independent randomized rounding. The second phase is deterministic and is a greedy pruning step in the *reverse* elimination order. To analyze the expected value of S we consider two binary random variables for each i , Y_i and Z_i . Y_i is 1 if $i \in R$ and 0 otherwise. Z_i is 1 if $i \in S$ and 0 otherwise.

By linearity of expectation,

Claim 4.3.1. $\mathbb{E}[w(S)] = \sum_i w_i \mathbb{E}[Z_i] = \sum_i w_i \mathbf{P}[Z_i = 1]$.

Via the independent randomized rounding in the algorithm.

Claim 4.3.2. $\mathbf{P}[Y_i = 1] = x_i/2$.

How do we analyze $\mathbf{P}[Z_i = 1]$? The random variables $Z_1, \dots, Z_n$ are not independent and could be highly correlated even though $Y_1, \dots, Y_n$ are independent. For this purpose we try to understand $\mathbf{P}[Z_i = 0 \mid Y_i = 1]$ which is the conditional probability that an interval I_i that is chosen in the first step is rejected in the pruning phase. We often would not be able to get an exact estimate of this quantity but we can upper bound it as follows. Here the ordering plays a crucial role. Why would I_i be rejected in the pruning phase? Note that when I_i is considered in the pruning phase, the only intervals that have been considered have their right end points after the right end point of I_i . Let $A_i = \{j \mid j > i \text{ and } I_j \text{ and } I_i \text{ intersect at } b_i\}$ be the potential set of intervals that can cause i to be rejected. Recall that the LP implies the following constraint:

$$x_i + \sum_{j \in A_i} x_j \leq 1$$

at the point b_j . Let $\mathcal{E}_1$ be the event that I_i is rejected in the pruning phase. Let $\mathcal{E}_2$ be the event that at least one of the intervals in A is selected in the *first phase*. Note that $\mathcal{E}_1$ can happen only if $\mathcal{E}_2$ happens. Thus $\mathbf{P}[\mathcal{E}_1] \leq \mathbf{P}[\mathcal{E}_2]$. In general we try to upper bound $\mathbf{P}[\mathcal{E}_2]$. In this simple case we have an exact formula for it.

$$\mathbf{P}[\mathcal{E}_2] = 1 - \prod_{j \in A} \mathbf{P}[Y_j = 0] = 1 - \prod_{j \in A} (1 - x_j/2).$$

We claim that $\mathbf{P}[\mathcal{E}_2] \leq \sum_{j \in A} x_j/2 \leq 1/2$. One can derive this by showing that $\prod_{j \in A} (1 - x_j/2)$ subject to $\sum_{j \in A} x_j/2 \leq 1/2$ is at least $1/2$. Another way of doing this is via Markov's inequality. Let $T = \sum_{j \in A} Y_j$ be the number of intervals from A selected in the first phase. $E[T] \leq \sum_{j \in A} x_j/2 < 1/2$. By Markov's inequality $\mathbf{P}[T \geq 2E[T]] \leq 1/2$. $\mathcal{E}_2$ is the event that $\mathbf{P}[T \geq 1]$.

Using the claim,

$$\mathbf{P}[Z_i = 1 \mid Y_i = 1] = 1 - \mathbf{P}[Z_i = 0 \mid Y_i = 1] \geq 1/2.$$

This allows us to lower bound the expected weight of the solution output by the algorithm, and yields a randomized $1/4$ approximation.

Claim 4.3.3. $\mathbb{E}[w(S)] \geq \sum_i w_i x_i/4$.

Proof. We have

$$\mathbb{E}[w(S)] = \sum_i w_i \mathbf{P}[Z_i = 1] = \sum_i w_i \mathbf{P}[Y_i = 1] \mathbf{P}[Z_i = 1 \mid Y_i = 1] \geq \sum_i w_i \left(\frac{x_i}{4} \cdot \frac{1}{2}\right) \geq \sum_i w_i x_i/4. \quad \blacksquare$$

This type of rounding has applications to a variety of settings - see [81] for applications and the general framework called *contention resolution schemes*.

4.4 Packing Integer Programs (PIPs)

We can express the KNAPSACK problem as the following integer program. We scaled the knapsack capacity to 1 without loss of generality.

$$\begin{aligned} & \text{maximize} && \sum_{i=1}^n p_i x_i \\ & \text{subject to} && \sum_i s_i x_i \leq 1 \\ & && x_i \in \{0, 1\} \quad 1 \leq i \leq n \end{aligned}$$

More generally if we have multiple linear constraints on the “items” we obtain the following integer program.

Definition 4.11. A *packing integer program (PIP)* is an integer program of the form $\max\{wx \mid Ax \leq 1, x \in \{0, 1\}^n\}$ where w is a $1 \times n$ non-negative vector and A is a $m \times n$ matrix with entries in $[0, 1]$. We call it a $\{0, 1\}$ -PIP if all entries are in $\{0, 1\}$.

In some cases it is useful/natural to define the problem as $\max\{wx \mid Ax \leq b, x \in \{0, 1\}^n\}$ where entries in A and b are required to rational/integer valued. We can convert it into the above form by dividing each row of A by b_i .

When m the number of rows of A (equivalently the constraints) is small the problem is tractable. It is some times called the m -dimensional knapsack and one can obtain a PTAS for any fixed constant m . However, when m is large we observe that MIS can be cast as a special case of $\{0, 1\}$ -PIP. It corresponds exactly to the simple integer/linear program that we saw in the previous section. Therefore the problem is at least as hard to approximate as MIS. Here we show via a clever LP-rounding idea that one can generalize the notion of bounded-degree to *column-sparsity* in PIPs and obtain a related approximation. We will then introduce the notion of *width* of the constraints and show how it allows for improved bounds.

Definition 4.12. A PIP is k -column-sparse if the number of non-zero entries in each column of A is at most k . A PIP has width W if $\max_{i,j} A_{ij}/b_i \leq 1/W$.

4.4.1 Randomized Rounding with Alteration for PIPs

We saw that randomized rounding gave an $O(\log n)$ approximation algorithm for the SET COVER problem which is a canonical covering problem. Here we will consider the use of randomized rounding for packing problems. Let x be an optimum fractional solution to the natural LP relaxation of a PIP where we replace the constraint $x \in \{0, 1\}^n$ by $x \in [0, 1]^n$. Suppose we apply independent randomized rounding where we set x'_i to 1 with probability x_i . Let x' be the resulting integer solution. The expected weight of this solution is exactly $\sum_i w_i x_i$ which is the LP solution value. However, x' may not satisfy the constraints given by $Ax \leq b$. A natural strategy to try to satisfy the constraints is to set x'_1 to 1 with probability cx_i where $c < 1$ is some scaling constant. This may help in satisfying the constraints because the scaling creates some room in the constraints; we now have that the expected solution value is $c \sum_i w_i x_i$, a loss of a factor of c . Scaling by itself does not allow us to claim that all constraints are satisfied with good probability. A very useful technique in this context is the technique of *alteration*; we judiciously fix/alter the rounded solution x' to force it to satisfy the constraints by setting some of the variables that are 1 in x' to 0. The trick is to do this in such a way as to have a handle on the final probability that a variable is set to 1. We will illustrate this for the KNAPSACK problem and then

generalize the idea to k -sparse PIPs. The algorithms we present are from [34]. See [81] for further applications and related problems.

Rounding for Knapsack: Consider the KNAPSACK problem. It is convenient to think of this in the context of PIPs. So we have $ax \leq 1$ where a_i now represents the size of item i and the knapsack capacity is 1; w_i is the weight of item. Suppose x is a fractional solution. Call an item i “big” if $a_i > 1/2$ and otherwise it is “small”. Let S be the indices of small items and B the indices of the big items. Consider the following rounding algorithm.

ROUNDING-WITH-ALTERATION FOR KNAPSACK

1. Let x be an optimum fractional solution
2. Round each i to 1 independently with probability $x_i/8$. Let x' be rounded solution.
3. $x'' = x'$
4. If ($x'_i = 1$ for exactly one big item i)
 - A. For each $j \neq i$ set $x''_j = 0$
5. Else If ($\sum_{i \in S} s_i x'_i > 1$ or two or more big items are chosen in x')
 - A. For each j set $x''_j = 0$
6. Output feasible solution x''

In words, the algorithm alters the rounded solution x' as follows. If exactly one big item is chosen in x' then the algorithm retains that item and rejects all the other small items. Otherwise, the algorithm rejects all items if two or more big items are chosen in x' or if the total size of all small items chosen in x' exceeds the capacity.

The following claim is easy to verify.

Claim 4.4.1. *The integer solution x'' is feasible.*

Now let us analyze the probability of an item i being present in the final solution. Let $\mathcal{E}_1$ be the event that $\sum_{i \in S} a_i x'_i > 1/2$, that is the sum of the sizes of the small items chosen in x' exceeds $1/2$ the capacity. Why do we care about $\mathcal{E}_1$? Suppose we only have small items. We want to understand the probability that the sum of the sizes of the sampled small items prevents a specific small item from being included. Since a small item can be of size at most $1/2$, it can only be

rejected if the sum of other small items is bigger than $1/2$. Let $\mathcal{E}_2$ be the event that at least one big item is chosen in x' .

Claim 4.4.2. $\mathbf{P}[\mathcal{E}_1] \leq 1/4$.

Proof. Let $X_s = \sum_{i \in S} a_i x'_i$ be the random variable that measures the sum of the sizes of the small items chosen. We have, by linearity of expectation, that

$$\mathbb{E}[X_s] = \sum_{i \in S} a_i \mathbb{E}[x'_i] = \sum_{i \in S} a_i x_i / 8 \leq 1/8.$$

By Markov's inequality, $\mathbf{P}[X_s > 1/2] \leq \mathbb{E}[X_s] / (1/2) \leq 1/4$. ■

Claim 4.4.3. $\mathbf{P}[\mathcal{E}_2] \leq 1/4$.

Proof. Since the size of each big item in B is at least $1/2$, we have $1 \geq \sum_{i \in B} a_i x_i \geq \sum_{i \in B} x_i / 2$. Therefore $\sum_{i \in B} x_i / 8 \leq 1/4$. Event $\mathcal{E}_2$ happens if some item $i \in B$ is chosen in the random selection. Since i is chosen with probability $x_i / 8$, by the union bound, $\mathbf{P}[\mathcal{E}_2] \leq \sum_{i \in B} x_i / 8 \leq 1/4$. ■

Lemma 4.5. Let Z_i be the indicator random variable that is 1 if $x''_i = 1$ and 0 otherwise. Then $\mathbb{E}[Z_i] = \mathbf{P}[Z_i = 1] \geq x_i / 16$.

Proof. We consider the binary random variable X_i which is 1 if $x'_i = 1$. We have $\mathbb{E}[X_i] = \mathbf{P}[X_i = 1] = x_i / 8$. We write

$$\mathbf{P}[Z_i = 1] = \mathbf{P}[X_i = 1] \cdot \mathbf{P}[Z_i = 1 \mid X_i = 1] = \frac{x_i}{8} \mathbf{P}[Z_i = 1 \mid X_i = 1].$$

To lower bound $\mathbf{P}[Z_i = 1 \mid X_i = 1]$ we upper bound the probability $\mathbf{P}[Z_i = 0 \mid X_i = 1]$, that is, the probability that we reject i conditioned on the fact that it is chosen in the random solution x' .

First consider a big item i that is chosen in x' . Then i is rejected iff *another* big item is chosen in x' ; the probability of this can be upper bounded by $\mathbf{P}[\mathcal{E}_2]$. If item i is small then it is rejected only if a big item is chosen (event $\mathcal{E}_2$) or if the total sum of the sizes of the small items exceeds $1 - s_i$; the latter event would imply that $\mathcal{E}_1$ happened. Thus, we can upper bound the conditional reject probability as follows:

$$\mathbf{P}[Z_i = 0 \mid X_i = 1] \leq \mathbf{P}[\mathcal{E}_1] + \mathbf{P}[\mathcal{E}_2] \leq 1/4 + 1/4 = 1/2.$$

Thus,

$$\mathbf{P}[Z_i = 1] = \mathbf{P}[X_i = 1] \cdot \mathbf{P}[Z_i = 1 \mid X_i = 1] = \frac{x_i}{8} (1 - \mathbf{P}[Z_i = 0 \mid X_i = 1]) \geq \frac{x_i}{16}.$$
■

One can improve the above analysis to show that $\mathbf{P}[Z_i = 1] \geq x_i/8$.

Theorem 4.13. *The randomized algorithm outputs a feasible solution of expected weight at least $\sum_{i=1}^n w_i x_i / 16$.*

Proof. The expected weight of the output is

$$\mathbb{E}[\sum_i w_i x_i''] = \sum_i w_i \mathbb{E}[Z_i] \geq \sum_i w_i x_i / 16$$

where we used the previous lemma to lower bound $\mathbb{E}[Z_i]$. ■

Rounding for k -sparse PIPs: We now extend the rounding algorithm and analysis above to k -sparse PIPs. Let x be a feasible fractional solution to $\max\{wx \mid Ax \leq 1, x \in [0, 1]^n\}$. For a column index i we let $N(i) = \{j \mid A_{j,i} > 0\}$ be the indices of the rows in which i has a non-zero entry. Since A is k -column-sparse we have that $|N(i)| \leq k$ for $1 \leq i \leq n$. When we have more than one constraint we cannot classify an item/index i as big or small since it may be big for some constraints and small for others. We say that i is small for constraint $j \in N(i)$ if $A_{j,i} \leq 1/2$ otherwise i is big for constraint j . Let $S_j = \{i \mid j \in N(i), \text{ and } i \text{ small for } j\}$ be the set of all small columns for j and $B_j = \{i \mid j \in N(i), \text{ and } i \text{ big for } j\}$ be the set of all big columns for j . Note that $S_j \cap B_j$ is the set of all i with $A_{j,i} > 0$.

ROUNDING-WITH-ALTERATION FOR k -SPARSE PIPs

1. Let x be an optimum fractional solution
2. Round each i to 1 independently with probability $x_i/(8k)$. Let x' be rounded solution.
3. $x'' = x'$
4. For $j = 1$ to m do
 - A. If $(x'_i = 1 \text{ for exactly one } i \in B_j)$
 1. For each $h \in S_j \cup B_j$ and $h \neq i$ set $x''_h = 0$
 - B. Else If $(\sum_{i \in S_j} A_{j,i} x'_i > 1 \text{ or two or more items from } B_j \text{ are chosen in } x')$
 1. For each $h \in S_j \cup B_j$ set $x''_h = 0$
5. Output feasible solution x''

The algorithm, after picking the random solution x' , alters it as follows: it applies the previous algorithm's strategy to each constraint j separately. Thus an element i can be rejected at different constraints $j \in N(i)$. We need to bound the total probability of rejection. As before, the following claim is easy to verify.

Claim 4.4.4. *The integer solution x'' is feasible.*

Now let us analyze the probability of an item i being present in the final solution. Let $\mathcal{E}_1(j)$ be the event that $\sum_{i \in S_j} A_{j,i} x'_i > 1/2$, that is the sum of the sizes of the items that are small for j in x' exceed half the capacity. Let $\mathcal{E}_2(j)$ be the event that at least one big item for j is chosen in x' . The following claims follow from the same reasoning as the ones before with the only change being the scaling factor.

Claim 4.4.5. $\mathbf{P}[\mathcal{E}_1(j)] \leq 1/(4k)$.

Claim 4.4.6. $\mathbf{P}[\mathcal{E}_2(j)] \leq 1/(4k)$.

Lemma 4.6. *Let Z_i be the indicator random variable that is 1 if $x''_i = 1$ and 0 otherwise. Then $\mathbb{E}[Z_i] = \mathbf{P}[Z_i = 1] \geq x_i/(16k)$.*

Proof. We consider the binary random variable X_i which is 1 if $x'_i = 1$ after the randomized rounding. We have $\mathbb{E}[X_i] = \mathbf{P}[X_i = 1] = x_i/(8k)$. We write

$$\mathbf{P}[Z_i = 1] = \mathbf{P}[X_i = 1] \cdot \mathbf{P}[Z_i = 1 \mid X_i = 1] = \frac{x_i}{8k} \mathbf{P}[Z_i = 1 \mid X_i = 1].$$

We upper bound the probability $\mathbf{P}[Z_i = 0 \mid X_i = 1]$, that is, the probability that we reject i conditioned on the fact that it is chosen in the random solution x' . We observe that

$$\mathbf{P}[Z_i = 0 \mid X_i = 1] \leq \sum_{j \in N(i)} (\mathbf{P}[\mathcal{E}_1(j)] + \mathbf{P}[\mathcal{E}_2(j)]) \leq k(1/(4k) + 1/(4k)) \leq 1/2.$$

We used the fact that $N(i) \leq k$ and the claims above. Therefore,

$$\mathbf{P}[Z_i = 1] = \mathbf{P}[X_i = 1] \cdot \mathbf{P}[Z_i = 1 \mid X_i = 1] = \frac{x_i}{8k} (1 - \mathbf{P}[Z_i = 0 \mid X_i = 1]) \geq \frac{x_i}{16k}.$$

■

The theorem below follows by using the above lemma and linearity of expectation to compare the expected weight of the output of the randomized algorithm with that of the fractional solution.

Theorem 4.14. *The randomized algorithm outputs a feasible solution of expected weight at least $\sum_{i=1}^n w_i x_i / (16k)$. There is $1/(16k)$ -approximation for k -sparse PIPs.*

Larger width helps: We saw during the discussion on the KNAPSACK problem that if all items are small with respect to the capacity constraint then one can obtain better approximations. For PIPs we defined the *width* of a given instance as W if $\max_{i,j} A_{ij}/b_i \leq 1/W$; in other words no single item is more than $1/W$ times the capacity of any constraint. One can show using a very similar algorithm and analysis as above that the approximation bound improves to $\Omega(1/k^{\lceil W \rceil})$ for instance with width W . Thus if $W = 2$ we get a $\Omega(1/\sqrt{k})$ approximation instead of $\Omega(1/k)$ -approximation. More generally when $W \geq c \log k/\epsilon$ for some sufficiently large constant c we can get a $(1 - \epsilon)$ -approximation. Thus, in the setting with multiple knapsack constraints, the notion of small with respect to capacities is that in each constraint the size of the item is $\leq \frac{c\epsilon}{\log k}$ times the capacity of that constraint.

4.5 Bibliographic Notes

We have taken a broad view of packing problems. Since MIS and MWIS are very hard to approximate there is lot of work on special cases including work on geometric intersection graphs, special classes of graphs, etc. Typical approximation algorithms books/courses ignore these since there is no clean story to convey. We used elimination graphs and related graph classes from [165, 259] to illustrate some nice results as well as techniques. We also refer the reader to [139] about computing the orderings. Some of the material in these notes on these graph classes is from the instructor's paper [78]. For those interested in geometric instances we refer the reader to work of Mitchell on rectangle independent set [203] and follow up work [115], and the work on quasi-polynomial time approximation schemes which is very general [4].

Contention resolution schemes ([81] formalized this notion though such algorithms were known implicitly) are a powerful technique that allow us to *combine* constraints that are quite disparate. They also have applications when the objective is not linear but submodular (this was the original motivation in [81]). They also play an important role in stochastic optimization. For these reasons they have become quite popular and useful.

Chapter 5

Load Balancing and Bin Packing

This chapter is based on notes first scribed by Rachit Agarwal.

In the last lecture, we studied the KNAPSACK problem. The KNAPSACK problem is an NP-hard problem but does admit a *pseudo-polynomial time* algorithm and can be solved efficiently if the input size is small. We used this pseudo-polynomial time algorithm to obtain an FPTAS for KNAPSACK. In this lecture, we study another class of problems, known as *strongly NP-hard* problems.

Definition 5.1 (Strongly NP-hard Problems). *An NPO problem π is said to be strongly NP-hard if it is NP-hard even if the inputs are polynomially bounded in the combinatorial size of the problem ¹.*

Many NP-hard problems are in fact strongly NP-hard. If a problem Π is strongly NP-hard, then Π does not admit a pseudo-polynomial time algorithm. We study two such problems in this lecture, MULTIPROCESSOR SCHEDULING and BIN PACKING.

5.1 Load Balancing / MultiProcessor Scheduling

A central problem in scheduling theory is to design a schedule such that the finishing time of the last job (also called *makespan*) is minimized. This problem is often referred to as the LOAD BALANCING, the MINIMUM MAKESPAN SCHEDULING or MULTIPROCESSOR SCHEDULING problem.

¹An alternative definition: A problem π is strongly NP-hard if every problem in NP can be polynomially reduced to π in such a way that numbers in the reduced instance are always written in unary

5.1.1 Problem Description

In the MULTIPROCESSOR SCHEDULING problem, we are given m identical machines $M_1, \dots, M_m$ and n jobs $J_1, J_2, \dots, J_n$. Job J_i has a processing time $p_i \geq 0$ and the goal is to assign jobs to the machines so as to minimize the maximum load².

5.1.2 Greedy Algorithm

Consider the following greedy algorithm for the MULTIPROCESSOR SCHEDULING problem which we will call GREEDY MULTIPROCESSOR SCHEDULING.

GREEDY MULTIPROCESSOR SCHEDULING:
 Order (list) the jobs arbitrarily
 For $i = 1$ to n do
 Assign Job J_i to the machine with *least current load*
 Update load of the machine that receives job J_i

This algorithm is also called a *list scheduling* algorithm following Graham's terminology from his paper from 1966 [127]. The list is the order in which the jobs are processed and changing it creates different schedules. We prove that the GREEDY MULTIPROCESSOR SCHEDULING algorithm gives a $(2 - 1/m)$ -approximation.

Theorem 5.2. GREEDY MULTIPROCESSOR SCHEDULING algorithm gives a $(2 - \frac{1}{m})$ -approximation for any list.

To prove the theorem, we will make use of two lower bounds on the length of the optimal schedule which we denote by OPT.

Observation 5.3. $\text{OPT} \geq \frac{\sum_i p_i}{m}$.

Observation 5.4. $\text{OPT} \geq \max_i p_i$.

We leave the proofs of the observations as easy exercises.

Proof of Theorem 5.2. Fix the list and let L denote the makespan of the GREEDY MULTIPROCESSOR SCHEDULING algorithm. Let L_i denote the load of machine M_i and let M_{i^*} be the most heavily loaded machine in the schedule by GREEDY MULTIPROCESSOR SCHEDULING algorithm.

Let J_k be the last job assigned to M_{i^*} . Since GREEDY MULTIPROCESSOR SCHEDULING algorithm assigns a job to the machine that is least loaded, all machines must be loaded at least $L - p_k$ at the time of assigning J_k . Hence, we have:

$$\left(\sum_{i=1}^n p_i \right) - p_k \geq m (L - p_k) \quad (5.1)$$

²The load of a machine is defined as the sum of the processing times of jobs that are assigned to that machine.

which implies:

$$\begin{aligned}
 L - p_k &\leq \frac{(\sum_{i=1}^n p_i) - p_k}{m} \\
 \text{hence} \\
 L &\leq \frac{(\sum_{i=1}^n p_i)}{m} + p_k \left(1 - \frac{1}{m}\right) \\
 &\leq \text{OPT} + \text{OPT} \left(1 - \frac{1}{m}\right) \\
 &= \text{OPT} \left(2 - \frac{1}{m}\right)
 \end{aligned}$$

where the third step follows from the two lower bounds on OPT. ■

The above analysis is tight, *i.e.*, there exist instances where the greedy algorithm produces a schedule which has a makespan $(2 - 1/m)$ times the optimal. Consider the following instance: $m(m - 1)$ jobs with unit processing time and a single job with processing time m . Suppose the greedy algorithm schedules all the short jobs before the long job, then the makespan of the schedule obtained is $(2m - 1)$ while the optimal makespan is m . Hence the algorithm gives a schedule which has makespan $2 - 1/m$ times the optimal.

It may seem from the tight example above that an approximation ratio $\alpha < (2 - 1/m)$ could be achieved if the jobs are sorted before processing, which indeed is the case. The following algorithm, due to [128], sorts the jobs in decreasing order of processing time prior to running GREEDY MULTIPROCESSOR SCHEDULING algorithm.

MODIFIED GREEDY MULTIPROCESSOR SCHEDULING:
 Sort the jobs in decreasing order of processing times
 For $i = 1$ to n do
 Assign Job J_i to the machine with *least current load*
 Update load of the machine that receives job J_i

Graham [128] proved the following tight bound.

Theorem 5.5. MODIFIED GREEDY MULTIPROCESSOR SCHEDULING algorithm gives a $(4/3 - 1/3m)$ -approximation for the MULTIPROCESSOR SCHEDULING problem.

We will not prove the preceding theorem which requires some careful case analysis. Instead we will show how one can obtain an easier bound of $3/2$ via the following claim.

Claim 5.1.1. Suppose $p_i \geq p_j$ for all $1 \leq i < j \leq n$, and suppose $n > m$. Then, $\text{OPT} \geq p_m + p_{m+1}$.

Proof. Since $n > m$ and the processing times are sorted in decreasing order, some two of the $(m + 1)$ largest jobs must be scheduled on the same machine due to the pigeon hole principle. The load of this machine is at least $p_m + p_{m+1}$ due to the sorting of the processing times. ■

Exercise 5.1. Prove that MODIFIED GREEDY MULTIPROCESSOR SCHEDULING gives a $(3/2 - 1/2m)$ -approximation using the preceding claim and the other two lower bounds on OPT that we have seen already.

Before going to the description of a PTAS for MULTIPROCESSOR SCHEDULING problem, we discuss the case when the processing times of the jobs are bounded from above.

Claim 5.1.2. *If $p_i \leq \epsilon \cdot \text{OPT}$, $\forall i$, then MODIFIED GREEDY MULTIPROCESSOR SCHEDULING gives a $(1 + \epsilon)$ -approximation.*

5.1.3 A PTAS for Multi-Processor Scheduling

We will now give a PTAS for the problem of scheduling jobs on identical machines. We would like to use the same set of ideas that were used for the KNAPSACK problem (see Lecture 4): that is, given an explicit time T we would like to round the job lengths and use dynamic programming to see if they will fit within time T . Then the unrounded job lengths should fit within time $T(1 + \epsilon)$.

Big Jobs, Small Jobs and Rounding Big Jobs: For the discussion that follows, we assume that all the processing times have been scaled so that $\text{OPT} = 1$ and hence, $p_{\max} \leq 1$.

Given all the jobs, we partition the jobs into two sets: *Big jobs* and *Small jobs*. We call a job J_i “big” if $p_i \geq \epsilon$. Let $\mathcal{B}$ and $\mathcal{S}$ denote the set of big jobs and small jobs respectively, i.e., $\mathcal{B} = \{J_i : p_i \geq \epsilon\}$ and $\mathcal{S} = \{J_i : p_i < \epsilon\}$. The significance of such a partition is that once we pack the jobs in set $\mathcal{B}$, the jobs in set $\mathcal{S}$ can be greedily packed using list scheduling.

Claim 5.1.3. *If there is an assignment of jobs in $\mathcal{B}$ to the machines with load L , then greedily scheduling jobs of $\mathcal{S}$ on top gives a schedule of value no greater than $\max\{L, (1 + \epsilon) \text{OPT}\}$.*

Proof. Consider scheduling the jobs in $\mathcal{S}$ after all the jobs in $\mathcal{B}$ have been scheduled (with load L). If all of these jobs in $\mathcal{S}$ finish processing by time L , the total load is clearly no greater than L .

If the jobs in $\mathcal{S}$ can *not* be scheduled within time L , consider the last job to finish (after scheduling the small jobs). Suppose this job starts at time T' . All the machines must have been fully loaded up to T' , which gives $\text{OPT} \geq T'$. Since, for all jobs in $\mathcal{S}$, we have $p_i \leq \epsilon \cdot \text{OPT}$, this job finishes at $T' + \epsilon \cdot \text{OPT}$. Hence, the schedule can be no more than $T' + \epsilon \cdot \text{OPT} \leq (1 + \epsilon) \text{OPT}$, settling the claim. ■

Scheduling Big Jobs: We concentrate on scheduling the jobs in $\mathcal{B}$. We round the sizes of all jobs in $\mathcal{B}$ using geometrically increasing interval sizes using the following procedure:

ROUNDING JOBS:
 For each big job i do
 If $p_i \in (\epsilon(1 + \epsilon)^j, \epsilon(1 + \epsilon)^{j+1}]$
 Set $p_i = \epsilon(1 + \epsilon)^{j+1}$

Let $\mathcal{B}'$ be the set of new jobs.

Claim 5.1.4. *If jobs in $\mathcal{B}$ can be scheduled with load 1, then the rounded jobs in $\mathcal{B}'$ can be scheduled with load $(1 + \epsilon)$.*

Claim 5.1.5. *The number of distinct big job sizes after rounding is $O(\frac{\ln(1/\epsilon)}{\epsilon})$.*

Proof. Notice that due to scaling, we have $p_i \leq 1$ for all jobs J_i . Since the job sizes are between ϵ and 1 the number of geometric powers of $(1 + \epsilon)$ required is k where

$$\begin{aligned} \epsilon(1 + \epsilon)^k &\leq 1 \\ \Rightarrow k &\leq \ln_{(1+\epsilon)} \frac{1}{\epsilon} = O\left(\frac{\ln(1/\epsilon)}{\epsilon}\right). \end{aligned}$$

■

Lemma 5.1. *If the number of distinct job sizes is k , then there is an exact algorithm that returns the schedule (if there is one) and runs in time $O(n^{2k})$.*

Proof. Use Dynamic Programming. ■

Corollary 5.6. *Big Jobs can be scheduled (if possible) with load $(1 + \epsilon)$ in time $n^{O(\frac{\ln(1/\epsilon)}{\epsilon})}$.*

Once we have scheduled the jobs in $\mathcal{B}$, using Claim 5.1.3, we can pack small items using greedy list scheduling on top of them. The overall algorithm is then given as:

PTAS MULTIPROCESSOR SCHEDULING:
 1. Guess OPT
 2. Define $\mathcal{B}$ and $\mathcal{S}$
 3. Round $\mathcal{B}$ to $\mathcal{B}'$
 4. If jobs in $\mathcal{B}'$ can be scheduled in $(1 + \epsilon)$ OPT
 Greedy pack $\mathcal{S}$ on top
 Else
 Modify the guess and Repeat.

In the following subsection, we comment on the guessing process.

Guessing: We define a $(1 + \epsilon)$ -relaxed decision procedure:

Definition 5.7. Given $\epsilon > 0$ and a time T , a $(1 + \epsilon)$ -relaxed decision procedure returns:

- Output a schedule (if there is one) with makespan load $(1 + \epsilon) \cdot T$ or
- Output correctly that there is no schedule with makespan T .

Define

$$L = \max \left\{ \max_j p_j, \frac{1}{m} \sum_j p_j \right\}$$

L is a lower bound on OPT as we saw earlier. Furthermore, an upper bound on OPT is given by the GREEDY MULTIPROCESSOR SCHEDULING algorithm, which is $2L$. Consider running the decision procedure with guess $L + i\epsilon L$ for each integer $i \in \lceil [2/\epsilon] \rceil$. We will choose the schedule with the best makespan among all the successful runs. If L^* is the optimum load then the algorithm will try the decision procedure with $L^* + \epsilon L \leq (1 + \epsilon)L^*$. For this guess we are guaranteed a solution and the decision procedure will succeed in outputting a schedule with load $(1 + \epsilon)(1 + \epsilon)L^* \leq (1 + 3\epsilon)L^*$ for sufficiently small ϵ . We run the decision procedure $O(1/\epsilon)$ times. This gives us the desired PTAS.

Remark 5.1. A PTAS indicates that the problem can be approximated arbitrarily well in polynomial time. However, a running time of the form $n^{f(\epsilon)}$ is typically not very interesting. We have seen that an FPTAS is ruled out for the makespan minimization problem. However, it does admit what is now called an *Efficient* PTAS (EPTAS) whose running time is $2^{O(1/\epsilon^2 \cdot (\log(1/\epsilon))^3)} + \text{poly}(n)$. See [158].

5.1.4 Section Notes

MULTIPROCESSOR SCHEDULING is NP-hard as we can reduce 2-PARTITION to MULTIPROCESSOR SCHEDULING on two machines. Note that this reduction only proves that MULTIPROCESSOR SCHEDULING is weakly NP-hard. When m is a fixed constant Horowitz and Sahni [152] give an FPTAS. However MULTIPROCESSOR SCHEDULING problem is strongly NP-hard when m is part of the input (by a reduction from 3-PARTITION [116]). Thus, there cannot exist an FPTAS for the MULTIPROCESSOR SCHEDULING problem in general, unless $P = NP$. However, Hochbaum and Shmoys [148] gave a PTAS which is the one we described. EPTASes have been developed for several problems and a key technique is the use of integer linear programming solvers with a small number of variables.

5.2 Bin Packing

5.2.1 Problem Description

In the BIN PACKING problem, we are given a set of n items $\{1, 2, \dots, n\}$. Item i has size $s_i \in (0, 1]$. The goal is to find a minimum number of bins of capacity 1 into which all the items can be packed.

One could also formulate the problem as partitioning $\{1, 2, \dots, n\}$ into k sets $\mathcal{B}_1, \mathcal{B}_2, \dots, \mathcal{B}_k$ such that $\sum_{i \in \mathcal{B}_j} s_i \leq 1$ and k is minimum.

5.2.2 Greedy Approaches

Consider the following greedy algorithm for bin packing:

GREEDY BIN PACKING:
 Order items in some way
 For $i = 1$ to n
 If item i can be packed in some *open* bin
 Pack it
 Else
 Open a new bin and pack i in the new bin

In GREEDY BIN PACKING algorithm, a new bin is opened only if the item can not be packed in any of the already opened bins. However, there might be several opened bins in which the item i could be packed. Several rules could be formulated in such a scenario:

- *First Fit*: Pack item in the earliest opened bin
- *Last Fit*: Pack item in the last opened bin
- *Best Fit*: Pack item in the bin that would have least amount of space left after packing the item
- *Worst Fit*: Pack item in the bin that would have most amount of space left after packing the item

Irrespective of what strategy is chosen to pack an item in the opened bins, one could get the following result:

Theorem 5.8. *Any greedy rule yields a 2-approximation.*

Observation 5.9. $\text{OPT} \geq \sum_i s_i$.

We call a bin α -full if items occupy space at most α .

Claim 5.2.1. *Greedy has at most 1 bin that is $\frac{1}{2}$ -full.*

Proof. For the sake of contradiction, assume that there are two bins B_i and B_j that are $\frac{1}{2}$ -full. WLOG, assume that GREEDY BIN PACKING algorithm opened bin B_i before B_j . Then, the first item that the algorithm packed into B_j must be of size at most $\frac{1}{2}$. However, this item could have been packed into B_i since B_i is $\frac{1}{2}$ -full. This is a contradiction to the fact that GREEDY BIN PACKING algorithm opens a new bin if and only if the item can not be packed in any of the opened bins. ■

Proof of Theorem 5.8. Let m be the number of bins opened by GREEDY BIN PACKING algorithm. From Claim 5.2.1, we have:

$$\sum_i s_i > \frac{m-1}{2}$$

Using the observation that $\text{OPT} \geq \sum_i s_i$, we get:

$$\text{OPT} > \frac{m-1}{2}$$

which gives us:

$$\begin{aligned} m &< 2 \cdot \text{OPT} + 1 \\ \Rightarrow m &\leq 2 \cdot \text{OPT} \end{aligned}$$

■

5.2.3 (Asymptotic) PTAS for Bin Packing

A natural question follows the discussion above: Can BIN PACKING have a PTAS? In this subsection, we settle this question in the negative. In particular, we give a reduction from an NP-complete problem to the BIN PACKING problem and show that a PTAS for the BIN PACKING problem will give us an exact solution for the NP-complete problem in polynomial time. We consider the PARTITION problem:

In the PARTITION problem, we are given a set of items $\{1, 2, \dots, n\}$. Item i has a size s_i . The goal is to partition the $\{1, 2, \dots, n\}$ into two sets $\mathcal{A}$ and $\mathcal{B}$ such that $\sum_{i \in \mathcal{A}} s_i = \sum_{j \in \mathcal{B}} s_j$.

Claim 5.2.2. *If BIN PACKING has a $(\frac{3}{2} - \epsilon)$ -approximation for any $\epsilon > 0$, the PARTITION problem can be solved exactly in polynomial time.*

Proof. Given an instance I of the PARTITION problem, we construct an instance of the BIN PACKING problem as follows: Scale the size of the items such that $\sum_i s_i = 2$. Consider the scaled sizes of the items as an instance I' of the BIN PACKING problem. If all items of I' can be packed in 2 bins, then we have an “yes” answer to I . Otherwise, the items of I' need 3 bins and the answer to I is “no”.

OPT for I' is 2 or 3. Hence, if there is a $(\frac{3}{2} - \epsilon)$ -approximation algorithm for the BIN PACKING problem, we can determine the value of OPT which in turn implies that we can solve I . Thus, there cannot exist a $(\frac{3}{2} - \epsilon)$ -approximation algorithm for the BIN PACKING problem, unless $P = NP$. ■

Recall the scaling property where we discussed why many optimization problems do not admit additive approximations. We notice that the BIN PACKING problem does not have the scaling property. Hence it may be possible to find an additive approximation algorithm. We state some of the results in this context:

Theorem 5.10 (Johnson '74 [162]). *There exists a polynomial time algorithm $\mathcal{A}_J$ such that:*

$$\mathcal{A}_J(I) \leq \frac{11}{9} \text{OPT}(I) + 4$$

for all instances I of the BIN PACKING problem.

Theorem 5.11 (de la Vega, Lueker '81 [91]). *For any fixed $\epsilon > 0$ there exists a polynomial time algorithm $\mathcal{A}_{FL}$ such that:*

$$\mathcal{A}_{FL}(I) \leq (1 + \epsilon) \text{OPT}(I) + 1$$

for all instances I of the BIN PACKING problem.

Theorem 5.12 (Karmarkar, Karp '82 [170]). *There exists a polynomial time algorithm $\mathcal{A}_{KK}$ such that:*

$$\mathcal{A}_{KK}(I) \leq \text{OPT}(I) + O(\log^2(\text{OPT}(I)))$$

for all instances I of the BIN PACKING problem.

This has been improved recently.

Theorem 5.13 (Hoberg and Rothvoss 2017 [147]). *There exists a polynomial time algorithm $\mathcal{A}_{HT}$ such that:*

$$\mathcal{A}_{HT}(I) \leq \text{OPT}(I) + O(\log(\text{OPT}(I)))$$

for all instances I of the BIN PACKING problem.

A major open problem is the following.

Open Question 5.14. *Is there a polynomial-time algorithm $\mathcal{A}$ such that $\mathcal{A}(I) \leq \text{OPT}(I) + c$, for some fixed constant c ? In particular is $c = 1$?*

Exercise 5.2. Show that First Fit greedy rule yields a $\frac{3}{2} \text{OPT} + 1$ -approximation.

5.2.4 Asymptotic PTAS for Bin Packing

A recurring theme in last two lectures has been the rounding of jobs/tasks/items. To construct an asymptotic PTAS for BIN PACKING problem, we use the same set of ideas with simple in retrospect but non-obvious modifications. In particular, we divide the set of items into *big* and *small* items and concentrate on packing the big items first. We show that such a technique results in an asymptotic PTAS for the BIN PACKING problem.

Consider the set of items, $s_1, s_2, \dots, s_n$. We divide the items into two sets, $\mathcal{B} = \{i : s_i \geq \epsilon\}$ and $\mathcal{S} = \{j : s_j < \epsilon\}$. Similar to the MULTIPROCESSOR SCHEDULING problem, where we rounded up the processing times of the jobs, we round up the sizes of the items in the BIN PACKING problem. Again, we concentrate only on the items in $\mathcal{B}$. Let $n' = |\mathcal{B}|$ be the number of big items.

We observed earlier that $\text{OPT} \geq \sum_i s_i$ and hence $\text{OPT} \geq \epsilon \cdot n'$ by just considering the big items.

Claim 5.2.3. Suppose $n' > 4/\epsilon^2$. Then $\text{OPT} \geq 4/\epsilon$.

If there are very few big items one can solve the problem by brute force.

Claim 5.2.4. Suppose $n' < 4/\epsilon^2$. An optimal solution for the BIN PACKING problem can be computed in $2^{O(1/\epsilon^4)}$ time.

Proof Sketch. If the number of big items is small, one can find the optimal solution using brute force search. ■

The following gives a procedure to round up the items in $\mathcal{B}$:

ROUNDING ITEM SIZES:

Sort the items such that $s_1 \geq s_2 \geq \dots \geq s_{n'}$

Group items in $k = 2/\epsilon^2$ groups $\mathcal{B}_1, \dots, \mathcal{B}_k$ such that each group has $\lfloor n'/k \rfloor$ items

Round the size of all the items in group $\mathcal{B}_i$ to the size of the smallest item in $\mathcal{B}_{i-1}$

Lemma 5.2. Consider the restriction of the bin packing problem to instances in which the number of distinct item sizes is k . There is an $n^{O(k)}$ -time algorithm that outputs the optimum solution.

Proof Sketch. Use Dynamic Programming. ■

Claim 5.2.5. The items in $\mathcal{B}$ can be packed in $\text{OPT} + |\mathcal{B}_1|$ bins in time $n^{O(1/\epsilon^2)}$.

Proof. Using ROUNDING ITEM SIZES, we have restricted all items but those in $\mathcal{B}_1$ to have one of the $k - 1$ distinct sizes. Using lemma 5.2, these items can be packed efficiently in OPT . Furthermore, the items in $\mathcal{B}_1$ can always be packed in $|\mathcal{B}_1|$ bins (one per bin). Hence, the total number of bins is $\text{OPT} + |\mathcal{B}_1|$.

The running time of the algorithm follows since $k = O(1/\epsilon^2)$. ■

Lemma 5.3. *Let $\epsilon > 0$ be fixed. Consider the restriction of the bin packing problem to instances in which each item is of size at least ϵ . There is a polynomial time algorithm that solves this restricted problem within a factor of $(1 + \epsilon)$.*

Proof. Using Claim 5.2.5, we can pack $\mathcal{B}$ in $\text{OPT} + |\mathcal{B}_1|$ bins. Recall that $|\mathcal{B}_1| = \lfloor n'/k \rfloor \leq \epsilon^2 \cdot n'/2 \leq \epsilon \cdot \text{OPT}/8$ where, we have used Claim 5.2.3 to reach the final expression. ■

Theorem 5.15. *For any ϵ , $0 < \epsilon < 1/2$, there is an algorithm $\mathcal{A}_\epsilon$ that runs in time polynomial in n and finds a packing using at most $(1 + 2\epsilon)\text{OPT} + 1$ bins.*

Proof. Assume that the number of bins used to pack items in $\mathcal{B}$ is m and the total number of bins used after packing items in $\mathcal{S}$ is m' . Clearly

$$m' \leq \max \left\{ m, \left\lceil \frac{(\sum_i s_i)}{(1 - \epsilon)} \right\rceil \right\}$$

since at most one bin must be $(1 - \epsilon)$ full using an argument in GREEDY BIN PACKING. Furthermore,

$$\left\lceil \frac{(\sum_i s_i)}{(1 - \epsilon)} \right\rceil \leq \left(\sum_i s_i \right) (1 + 2\epsilon) + 1$$

for $\epsilon < 1/2$. This gives the required expression. ■

The algorithm is summarized below:

ASYMPTOTIC PTAS BIN PACKING:

Split the items in $\mathcal{B}$ (big items) and $\mathcal{S}$ (small items)
 Round the sizes of the items in $\mathcal{B}$ to obtain constant number of item sizes
 Find optimal packing for items with rounded sizes
 Use this packing for original items in $\mathcal{B}$
 Pack items in $\mathcal{S}$ using GREEDY BIN PACKING.

5.2.5 Section Notes

An excellent but perhaps somewhat dated survey on approximation algorithms for BIN PACKING problem is [164]. See [147] for some pointers to more recent work. There has also been substantial recent work on various generalizations to multiple dimensions.

Chapter 6

Unrelated Machine Scheduling and Generalized Assignment

This chapter is based on notes first scribed by Alina Ene.

6.1 Scheduling on Unrelated Parallel Machines

We have a set J of n jobs, and a set M of m machines. The processing time of job i is p_{ij} on machine j . Let $f : J \rightarrow M$ be a function that assigns each job to exactly one machine. The *makespan* of f is $\max_{1 \leq j \leq m} \sum_{i: f(i)=j} p_{ij}$, where $\sum_{i: f(i)=j} p_{ij}$ is the total processing time of the jobs that are assigned to machine j . In the SCHEDULING ON UNRELATED PARALLEL MACHINES problem, the goal is to find an assignment of jobs to machines of minimum makespan.

We can write an LP for the problem that is very similar to the routing LP from the previous lecture. For each job i and each machine j , we have a variable x_{ij} that denotes whether job i is assigned to machine j . We also have a variable λ for the makespan. We have a constraint for each job that ensures that the job is assigned to some machine, and we have a constraint for each machine that ensures that the total processing time of jobs assigned to the machines is at most the makespan λ .

$$\begin{array}{ll}
\text{minimize} & \lambda \\
\text{subject to} & \sum_{j \in M} x_{ij} = 1 \quad \forall i \in J \\
& \sum_{i \in J} x_{ij} p_{ij} \leq \lambda \quad \forall j \in M \\
& x_{ij} \geq 0 \quad \forall i \in J, j \in M
\end{array}$$

The above LP is very natural, but unfortunately it has unbounded integrality gap. Suppose that we have a single job that has processing time T on each of the machines. Clearly, the optimal schedule has makespan T . However, the LP can schedule the job to the extent of $1/m$ on each of the machines, i.e., it can set $x_{1j} = 1/m$ for all j , and the makespan of the resulting fractional schedule is only T/m .

To overcome this difficulty, we modify the LP slightly. Suppose we knew that the makespan of the optimal solution is equal to λ , where λ is some fixed number. If the processing time p_{ij} of job i on machine j is greater than λ , job i is *not* scheduled on machine j , and we can strengthen the LP by setting x_{ij} to 0 or equivalently, by removing the variable. More precisely, let $\mathcal{S}_\lambda = \{(i, j) \mid i \in J, j \in M, p_{ij} \leq \lambda\}$. Given a value λ , we can write the following LP for the problem.

$$\begin{array}{ll}
\text{LP}(\lambda) & \\
\sum_{j: (i,j) \in \mathcal{S}_\lambda} x_{ij} = 1 & \forall i \in J \\
\sum_{i: (i,j) \in \mathcal{S}_\lambda} x_{ij} p_{ij} \leq \lambda & \forall j \in M \\
x_{ij} \geq 0 & \forall (i, j) \in \mathcal{S}_\lambda
\end{array}$$

Note that the LP above does *not* have an objective function. In the following, we are only interested in whether the LP is feasible, i.e., whether there is an assignment that satisfies all the constraints. Also, we can think of λ as a parameter and $\text{LP}(\lambda)$ as a family of LPs, one for each value of the parameter. A useful observation is that, if λ is a lower bound on the makespan of the optimal schedule, $\text{LP}(\lambda)$ is feasible and it is a valid relaxation for the SCHEDULING ON UNRELATED PARALLEL MACHINES problem.

Lemma 6.1. *Let λ^* be the minimum value of the parameter λ such that $\text{LP}(\lambda)$ is feasible. We can find λ^* in polynomial time.*

Proof. For any fixed value of λ , we can check whether $\mathbf{LP}(\lambda)$ is feasible using a polynomial-time algorithm for solving LPs. Thus we can find λ^* using binary search starting with the interval $[0, \sum_{i,j} p_{ij}]$. ■

In the following, we will show how to round a solution to $\mathbf{LP}(\lambda^*)$ in order to get a schedule with makespan at most $2\lambda^*$. As we will see shortly, it will help to round a solution to $\mathbf{LP}(\lambda^*)$ that is a *vertex* solution.

Let x be a vertex solution to $\mathbf{LP}(\lambda^*)$. Let G be a bipartite graph on the vertex set $J \cup M$ that has an edge ij for each variable $x_{ij} \neq 0$. We say that job i is fractionally set if $x_{ij} \in (0, 1)$ for some j . Let F be the set of all jobs that are fractionally set, and let H be a bipartite graph on the vertex set $F \cup M$ that has an edge ij for each variable $x_{ij} \in (0, 1)$; note that H is the induced subgraph of G on $F \cup M$. As shown in Lemma 6.2, the graph H has a matching that matches every job in F to a machine, and we will use it in the rounding algorithm.

Lemma 6.2. *The graph G has a matching that matches every job in F to a machine.*

We are now ready to give the rounding algorithm.

SUPM-Rounding
Find λ^*
Find a vertex solution x to $\mathbf{LP}(\lambda^*)$
For each i and j such that $x_{ij} = 1$, assign job i to machine j
Construct the graph H
Find a maximum matching $\mathcal{M}$ in H
Assign the fractionally set jobs according to the matching $\mathcal{M}$

Theorem 6.1. *Consider the assignment constructed by SUPM-Rounding. Each job is assigned to a machine, and the makespan of the schedule is at most $2\lambda^*$.*

Proof. By Lemma 6.2, the matching $\mathcal{M}$ matches every fractionally set job to a machine and therefore all of the jobs are assigned. After assigning all of the integrally set jobs, the makespan (of the partial schedule) is at most λ^* . Since $\mathcal{M}$ is a matching, each machine receives at most one additional job. Let i be a fractionally set job, and suppose that i is matched (in $\mathcal{M}$) to machine j . Since the pair (i, j) is in $\mathcal{S}_{\lambda^*}$, the processing time p_{ij} is at most λ^* , and therefore the total processing time of machine j increases by at most λ after assigning the fractionally set jobs. Therefore the makespan of the final schedule is at most $2\lambda^*$. ■

Exercise 6.1. Give an example that shows that Theorem 6.1 is tight. That is, give an instance and a vertex solution such that the makespan of the schedule SUPM-Rounding is at least $(2 - o(1))\lambda^*$. Also show that the integrality gap of the relaxation is 2.

Since λ^* is a lower bound on the makespan of the optimal schedule, we get the following corollary.

Corollary 6.2. *SUPM-Rounding achieves a 2-approximation.*

Now we turn our attention to Lemma 6.2 and some other properties of vertex solutions to $\mathbf{LP}(\lambda)$. The following can be derived from the rank lemma which is described in Chapter A. Here we give a self-contained proof.

Lemma 6.3. *If $\mathbf{LP}(\lambda)$ is feasible, any vertex solution has at most $m + n$ non-zero variables and it sets at least $n - m$ of the jobs integrally.*

Proof. Let x be a vertex solution to $\mathbf{LP}(\lambda)$. Let r denote the number of pairs in $\mathcal{S}_\lambda$. Note that $\mathbf{LP}(\lambda)$ has r variables, one for each pair $(i, j) \in \mathcal{S}_\lambda$. If x is a vertex solution, it satisfies r of the constraints of $\mathbf{LP}(\lambda)$ with equality. The first set of constraints consists of n constraints, and the second set of constraints consists of m constraints. Therefore at least $r - (m + n)$ of the tight constraints are from the third set of constraints, i.e., at least $r - (m + n)$ of the variables are set to zero.

We say that job i is set fractionally if $x_{ij} \in (0, 1)$ for some j ; job i is set integrally if $x_{ij} \in \{0, 1\}$ for all j . Let I and F be the set of jobs that are set integrally and fractionally (respectively). Clearly, $|I| + |F| = n$. Any job i that is fractionally set is assigned (fractionally) to at least two machines, i.e., there exist $j \neq \ell$ such that $x_{ij} \in (0, 1)$ and $x_{i\ell} \in (0, 1)$. Therefore there are at least $2|F|$ distinct non-zero variables corresponding to jobs that are fractionally set. Additionally, for each job i that is integrally set, there is a variable x_{ij} that is non-zero. Thus the number of non-zero variables is at least $|I| + 2|F|$. Hence $|I| + |F| = n$ and $|I| + 2|F| \leq m + n$, which give us that $|I|$ is at least $n - m$. ■

Definition 6.3. *A connected graph is a **pseudo-tree** if the number of edges is at most the number of vertices. A graph is a **pseudo-forest** if each of its connected components is a pseudo-tree.*

Lemma 6.4. *The graph G is a pseudo-forest.*

Proof. Let C be a connected component of G . We restrict $\mathbf{LP}(\lambda)$ and x to the jobs and machines in C to get $\mathbf{LP}'(\lambda)$ and x' . Note that x' is a feasible solution to $\mathbf{LP}'(\lambda)$. Additionally, x' is a vertex solution to $\mathbf{LP}'(\lambda)$. If not, x' is a convex combination of two feasible solutions x'_1 and x'_2 to $\mathbf{LP}'(\lambda)$. We can extend x'_1 and x'_2 to two solutions x_1 and x_2 to $\mathbf{LP}(\lambda)$ using the entries of x that are not in x' . By construction, x_1 and x_2 are feasible solutions to $\mathbf{LP}(\lambda)$. Additionally, x is a convex combination of x_1 and x_2 , which contradicts the fact that x is a vertex solution. Thus x' is a vertex solution to $\mathbf{LP}'(\lambda)$ and, by Lemma 6.3, x' has at most $n' + m'$ non-zero variables, where n' and m' are the number of jobs and machines in C . Thus C has $n' + m'$ vertices and at most $n' + m'$ edges, and therefore it is a pseudo-tree. ■

Proof. of Lemma 6.2 Note that each job that is integrally set has degree one in G . We remove each integrally set job from G ; note that the resulting graph is H . Since we removed an equal number of vertices and edges from G , it follows that H is a pseudo-forest as well. Now we construct a matching M as follows.

Note that every job vertex has degree at least 2, since the job is fractionally assigned to at least two machines. Thus all of the leaves (degree-one vertices) of H are machines. While H has at least one leaf, we add the edge incident to the leaf to the matching and we remove both of its endpoints from the graph. If H does not have any leaves, H is a collection of vertex-disjoint cycles, since it is a pseudo-forest. Moreover, each cycle has even length, since H is bipartite. We construct a perfect matching for each cycle (by taking alternate edges), and we add it to our matching. ■

Exercise 6.2. (Exercise 17.1 in [250]) Give a proof of Lemma 6.2 using Hall's theorem.

6.2 Generalized Assignment Problem

The GENERALIZED ASSIGNMENT problem is a generalization of the SCHEDULING ON UNRELATED PARALLEL MACHINES problem in which there are costs associated with each job-machine pair, in addition to a processing time. More precisely, we have a set J of n jobs, a set M of m machines, and a target λ . The processing time of job i is p_{ij} on machine j , and the cost of assigning job i to machine j is c_{ij} . Let $f : J \rightarrow M$ be a function that assigns each job to exactly one machine. The assignment f is *feasible* if its makespan is at most λ (recall that λ is part of the input), and its cost is $\sum_i c_{if(i)}$. In the GENERALIZED ASSIGNMENT problem, the goal is to construct a minimum cost assignment f that is *feasible*, provided that there is a feasible assignment.

Remark 6.1. We could allow each machine M_j to have a different capacity c_j which is more natural in certain settings. However, since p_{ij} values are allowed to depend on j we can scale them to ensure that $c_j = \lambda$ for every j without loss of generality.

In the following, we will show that, if there is an assignment of cost C and makespan at most λ , then we can construct a schedule of cost at most C and makespan at most 2λ . In fact the assignment will have a stronger property that the load on a machine exceeds λ due to at most one job.

As before, we let S_λ denote the set of all pairs (i, j) such that $p_{ij} \leq \lambda$. We can generalize the relaxation $\text{LP}(\lambda)$ from the previous section to the following LP.

GAP-LP

$$\begin{array}{ll}
 \min & \sum_{(i,j) \in \mathcal{S}_\lambda} x_{ij} c_{ij} \\
 \text{subject to} & \sum_{j: (i,j) \in \mathcal{S}_\lambda} x_{ij} = 1 \quad \forall i \in J \\
 & \sum_{i: (i,j) \in \mathcal{S}_\lambda} x_{ij} p_{ij} \leq \lambda \quad \forall j \in M \\
 & x_{ij} \geq 0 \quad \forall (i,j) \in \mathcal{S}_\lambda
 \end{array}$$

Since we also need to preserve the costs, we can no longer use the previous rounding; in fact, it is easy to see that the previous rounding is arbitrarily bad for the GENERALIZED ASSIGNMENT problem. However, we will still look for a matching, but in a slightly different graph.

But before we give the rounding algorithm for the GENERALIZED ASSIGNMENT problem, we take a small detour into the problem of finding a minimum-cost matching in a bipartite graph. In the MINIMUM COST BIPARTITE MATCHING problem, we are given a bipartite graph $B = (V_1 \cup V_2, E)$ with costs c_e on the edges, and we want to construct a minimum cost matching $\mathcal{M}$ that matches every vertex in V_1 , if there is such a matching. For each vertex v , let $\delta(v)$ be the set of all edges incident to v . We can write the following LP for the problem.

BipartiteMatching(B)

$$\begin{array}{ll}
 \min & \sum_{e \in E(B)} c_e y_e \\
 \text{subject to} & \sum_{e \in \delta(v)} y_e = 1 \quad \forall v \in V_1 \\
 & \sum_{e \in \delta(v)} y_e \leq 1 \quad \forall v \in V_2 \\
 & y_e \geq 0 \quad \forall e \in E(B)
 \end{array}$$

The following is well-known in combinatorial optimization [230].

Theorem 6.4. *For any bipartite graph B , any vertex solution to **BipartiteMatching**(B) is an integer solution. Moreover, given a feasible fractional solution y , we can find in polynomial time a feasible solution z such that z is integral and*

$$\sum_{e \in E(B)} c_e z_e \leq \sum_{e \in E(B)} c_e y_e.$$

In the rest of the section we give two different proofs that establish our claimed result. One is based on the first work that gave this result [236], and the other is based on iterative rounding [185].

6.2.1 Shmoys-Tardos Rounding

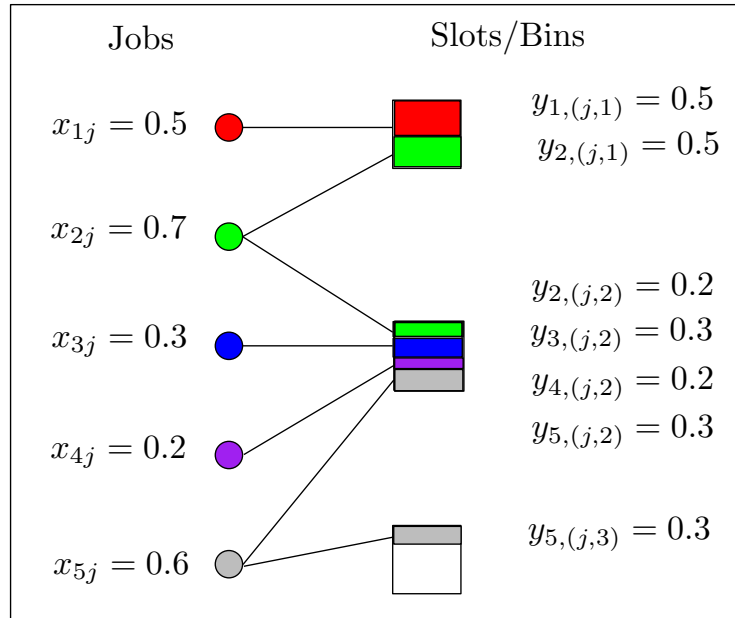
Let x be an optimal vertex solution to **GAP-LP**. As before, we want to construct a graph G that has a matching $\mathcal{M}$ that matches all jobs. The graph G will now have costs on its edges and we want a matching of cost at most C . Recall that for **SCHEDULING ON UNRELATED PARALLEL MACHINES** we defined a bipartite graph on the vertex set $J \cup M$ that has an edge ij for every variable x_{ij} that is non-zero. We can construct the same graph for **GENERALIZED ASSIGNMENT**, and we can assign a cost c_{ij} to each edge ij . If the solution x was actually a fractional matching — that is, if x was a feasible solution to **BipartiteMatching**(G) — Theorem 6.4 would give us the desired matching. The solution x satisfies the constraints corresponding to vertices $v \in J$, but it does not necessarily satisfy the constraints corresponding vertices $v \in M$, since a machine can be assigned more than one job. To get around this difficulty, we will introduce several nodes representing the same machine, and we will use x to construct a fractional matching for the resulting graph.

The fractional solution x assigns $\sum_{i \in J} x_{ij}$ jobs to machine j ; let $k_j = \lceil \sum_{i \in J} x_{ij} \rceil$. We construct a bipartite graph G as follows. For each job i , we have a node i . For each machine j , we have k_j nodes $(j, 1), \dots, (j, k_j)$. We can think of the nodes $(j, 1), \dots, (j, k_j)$ as *slots* on machine j . Since now we have multiple slots on each of the machines, we need a fractional assignment y that assigns a job to slots on the machines. More precisely, y has an entry $y_{i,(j,s)}$ for each job i and each slot (j, s) that represents the fraction of job i that is assigned to the slot. We give the algorithm that constructs y from x below. Once we have the solution y , we add an edge between any job i and any machine slot (j, s) such that $y_{i,(j,s)}$ is non-zero. Additionally, we assign a cost $c_{i,(j,s)}$ to each edge $(i, (j, s))$ of G that is equal to c_{ij} .

```

GREEDYPACKING( $x$ )
 $y = 0$  «initialize  $y$  to 0»
 $s = 1$  « $s$  is the current bin»
 $R = 1$  « $R$  is the space available on bin  $s$ »
for  $i = 1$  to  $h$ 
  «pack  $x_{ij}$  into the bins»
  if  $x_{ij} \leq R$ 
     $y_{i,(j,s)} = x_{ij}$ 
     $R = R - x_{ij}$ 
    if  $R = 0$ 
       $s = s + 1$ 
       $R = 1$ 
  else
     $y_{i,(j,s)} = R$ 
     $y_{i,(j,s+1)} = x_{ij} - R$  «pack  $x_{ij} - R$  in the next bin»
     $R = 1 - y_{i,(j,s+1)}$ 
     $s = s + 1$ 
return  $y$ 

```

Figure 6.1: Constructing y from x .

When we construct y , we consider each machine in turn. Let j be the current machine. Recall that we want to ensure that y assigns at most one job to each slot; as such, we will think of each slot on machine j as a bin with capacity 1. We

“pack” jobs into the bins greedily. We only consider jobs i such that p_{ij} is at most λ ; let h denote the number of such jobs. We assume without loss of generality that these are labeled as $1, 2, \dots, h$, and $p_{1j} \geq p_{2j} \geq \dots \geq p_{hj}$. Informally, when we construct y , we consider the jobs $1, 2, \dots, h$ in this order. Additionally, we keep track of the bin that has not been filled and the amount of space s available on that bin. When we consider job i , we try to pack x_{ij} into the current bin: if there is at least x_{ij} space available, i.e., $x_{ij} \leq s$, we pack the entire amount into the current bin; otherwise, we pack as much as we can into the current bin, and we pack the rest into the next bin. (See Figure 1 for an example.)

Lemma 6.5. *The solution y constructed by GREEDYPACKING is a feasible solution to BIPARTITEMATCHING(G). Moreover,*

$$\sum_{(i,(j,s)) \in E(G)} y_{i,(j,s)} c_{i,(j,s)} = \sum_{(i,j) \in S_\lambda} x_{ij} c_{ij}.$$

Proof. Note that, by construction, $x_{ij} = \sum_{s=1}^{k_j} y_{i,(j,s)}$. Therefore, for any job i , we have

$$\sum_{(i,(j,s)) \in \delta(i)} y_{i,(j,s)} = \sum_{j: (i,j) \in S_\lambda} \sum_{s=1}^{k_j} y_{i,(j,s)} = \sum_{j: (i,j) \in S_\lambda} x_{ij} = 1$$

Additionally, since we imposed a capacity of 1 on the bins associated with each slot, it follows that, for any slot (j, s) ,

$$\sum_{(i,(j,s)) \in \delta((j,s))} y_{i,(j,s)} \leq 1$$

Therefore y is a feasible solution to BIPARTITEMATCHING(G). Finally,

$$\sum_{(i,(j,s)) \in E(G)} y_{i,(j,s)} c_{i,(j,s)} = \sum_{i=1}^n \sum_{j: (i,j) \in S_\lambda} \sum_{s=1}^{k_j} y_{i,(j,s)} c_{ij} = \sum_{(i,j) \in S_\lambda} x_{ij} c_{ij}$$

■

Theorem 6.4 gives us the following corollary.

Corollary 6.5. *The graph G has a matching $\mathcal{M}$ that matches every job and it has cost at most $\sum_{(i,j) \in S_\lambda} x_{ij} c_{ij}$. Moreover, we can find such a matching in polynomial time.*

GAP-Rounding

let x be an optimal solution to **GAP-LP**
 $y = \text{GREEDYPACKING}(x)$
 construct the graph G
 construct a matching $\mathcal{M}$ in G such that $\mathcal{M}$ matches every job
 and the cost of $\mathcal{M}$ is at most $\sum_{(i,j) \in S_\lambda} x_{ij} c_{ij}$
 for each edge $(i, (j, s)) \in \mathcal{M}$
 assign job i to machine j

Theorem 6.6. *Let $C = \sum_{(i,j) \in S_\lambda} x_{ij} c_{ij}$. The schedule returned by **GAP-Rounding** has cost at most C and makespan at most 2λ .*

Proof. By Corollary 6.5, the cost of the schedule is at most C . Therefore we only need to upper bound the makespan of the schedule.

Consider a machine j . For any slot (j, s) on machine j , let

$$q_{js} = \max_{i: y_{i,(j,s)} > 0} p_{ij}$$

That is, q_{js} is the maximum processing time of any pair ij such that job i is assigned (in y) to the slot (j, s) . It follows that the total processing time of the jobs that $\mathcal{M}$ assigns to machine j is at most $\sum_{s=1}^{k_j} q_{js}$.

Since **GAP-LP** has a variable x_{ij} only for pairs (i, j) such that p_{ij} is at most λ , it follows that q_{j1} is at most λ . We restrict attention to the case when at least two slots are assigned to j , for otherwise it is easy to see that the load is at most λ . Therefore we only need to show that $\sum_{s=2}^{k_j} q_{js}$ is at most λ as well. Consider a slot s on machine j such that $s > 1$. Recall that we labeled the jobs that are relevant to machine j — that is, jobs i such that p_{ij} is at most λ — as $1, 2, \dots, h$ such that $p_{1j} \geq p_{2j} \geq \dots \geq p_{hj}$. Consider a job ℓ that is assigned to slot s . Since **GREEDYPACKING** considers jobs in non-increasing order according to their processing times, the processing time $p_{\ell j}$ of job ℓ is at most the processing time of any job assigned to the slot $s - 1$. Therefore $p_{\ell j}$ is upper bounded by any convex combination of the processing times of the jobs that are assigned to the slot $s - 1$. Since the slot $s - 1$ is full, $\sum_i y_{i,(j,s-1)} = 1$ and thus $p_{\ell j}$ is at most $\sum_i y_{i,(j,s-1)} p_{ij}$. It follows that

$$\sum_{s=2}^{k_j} q_{js} \leq \sum_{s=2}^{k_j} \sum_i y_{i,(j,s-1)} p_{ij} \leq \sum_{s=1}^{k_j} \sum_i y_{i,(j,s)} p_{ij}$$

By construction, $\sum_s y_{i,(j,s)} = x_{ij}$, and therefore

$$\sum_{s=1}^{k_j} \sum_i y_{i,(j,s)} p_{ij} = \sum_i p_{ij} \sum_{s=1}^s y_{i,(j,s)} = \sum_i p_{ij} x_{ij}$$

Since x is a feasible solution to the **GAP-LP**,

$$\sum_{s=2}^{k_j} q_{js} \leq \sum_i p_{ij} x_{ij} \leq \lambda$$

which completes the proof. ■

6.2.2 Iterative Rounding

Here we describe an alternative proof/algorithm that illustrates the iterative rounding framework that initially came out of Jain's seminal work [155]. Since then it has become a powerful technique in exact and approximation algorithms. We need some additional formalism to describe the algorithm. We will consider the input instance as being specified by a graph $G = (J \cup M, E)$ where an edge $ij \in E$ implies that i is allowed to be scheduled on j and has size p_{ij} . It is also important to consider non-uniform capacities on the machines to allow for a recursive (or iterative) algorithm. Thus we will assume that each machine M_j has a capacity b_j . We will assume that $p_{ij} \leq b_j$ for all $ij \in E$; in fact this assumption will not be needed until the very end when we analyze the approximation ratio. It is easy to generalize the LP relaxation for **GAP-LP** to handle non-uniform capacities and to handle the constraints specified by G . We will use the notation $\delta(i)$ and $\delta(j)$ to denote the edges incident to job i and machine j respectively.

$$\begin{array}{ll}
 \textbf{GAP-LP} & \\
 \min & \sum_{(i,j) \in E} c_{ij} x_{ij} \\
 \text{subject to} & \sum_{j: (i,j) \in \delta(i)} x_{ij} = 1 \quad \forall i \in J \\
 & \sum_{i: (i,j) \in E} p_{ij} x_{ij} \leq b_j \quad \forall j \in M \\
 & x_{ij} \geq 0 \quad \forall (i,j) \in E
 \end{array}$$

To explain the underlying intuition for iterated rounding approach in the specific context of **GAP**, consider the situation where each machine j has infinite capacity. In this case it is easy to find the minimum cost assignment. We simply assign each job i to the machine $j = \arg \min_{j' \in \delta(i)} c_{ij'}$ which is the cheapest one that it is allowed to be assigned to. We also observe that if we drop all the capacity constraints from **GAP-LP**, and only leave the assignment constraints ($\sum_j x_{ij} = 1$ for each i), then the optimum solution of this LP is the same as the

one obtained by assigning each job to its cheapest allowed machine (one can also argue that the LP is an integer polytope). Now consider another scenario. Suppose each machine j has in-degree at most k in G — that is, there are only k jobs that can ever be assigned to any machine j . Now suppose we assign each job to its cheapest allowed machine. Clearly the cost is at most the optimum cost of any feasible solution. But what about the load? Since each machine had in-degree at most k we will load a machine j to at most kb_j . Thus, if $k = 2$ we will only violate the machine's load by a factor of 2. However, this seems to be very restrictive assumption. Now consider a less restrictive scenario where there is one machine j such that its in-degree is at most 2. Then, in the LP relaxation, we can omit the constraint that limits its load since we are guaranteed that at most 2 jobs can be assigned to it (note that we still have the job assignment constraints which only allow a job to be assigned to machines according to the edges of G). Omitting constraints in an iterative fashion by taking advantage of sparsity in the basic feasible solution is the key idea.

To allow dropping of constraints we need some notation. Given an instance of GAP specified by $G = (J \cup M, E)$ and $M' \subseteq M$, we let $GAPLP(G, M')$ denote the LP relaxation for GAP where we only impose the load constraints for machines in M' . In other words we drop the load constraints for $M \setminus M'$. Note that jobs are still allowed to be assigned to machines in $M \setminus M'$.

The key structural lemma that allows for iterated rounding is the following.

Lemma 6.6. *Let y be a basic feasible solution to $GAPLP(G, M')$. Then one of the following properties holds:*

1. *There is some $ij \in E$ such $y_{ij} = 0$ or $y_{ij} = 1$.*
2. *There is a machine $j \in M'$ such that $d(j) \leq 1$.*
3. *There is a machine $j \in M'$ such that $\deg(j) = 2$ and $\sum_{ij} y_{ij} \geq 1$.*

Proof. Let y be a basic feasible solution. If $y_{ij} = 0$ or $y_{ij} = 1$ for some edge $ij \in E$ we are done. Similarly if there is a machine $j \in M'$ with $d(j) \leq 1$ we are done. Thus we restrict our attention to the case when y_{ij} is strictly fractional for *every* edge $ij \in E$ and $d(j) > 1$ for each machine $j \in M'$. Note that $\deg(i) \geq 2$ for each job i ; otherwise $\deg(i) = 1$ and in that case $y_{ij} = 1$ for the lone edge ij incident to i . We will prove that the third property holds.

$GAPLP(G, M')$ has $n + m'$ non-trivial constraints where $n = |J|$ and $m' = |M'|$. Since y is a basic feasible solution, via the rank lemma, this implies that $|E| \leq n + m'$. But degree of every i and every $j \in M'$ is at least 2. This implies that $|E| \geq (2n + 2m')/2 \geq n + m'$. Hence we have $|E| = n + m'$ and $\deg(i) = 2$ for every $i \in J$ and $\deg(j) = 2$ for every $j \in M'$ and $\deg(j) = 0$ for every $j \in M \setminus M'$. Thus G consists of a collection of disjoint even cycles. Let S be any such cycle.

For every job $i \in S$ we have $\sum_j y_{ij} = 1$; hence $\sum_{ij \in S} y_{ij} = |S|/2$. Hence there is some machine $j \in S$ such that $\sum_{ij} y_{ij} \geq 1$ and moreover its degree is exactly two as we argued. ■

GAP-ITER-ROUNDING(G)

1. $F = \emptyset$, $M' = M$
2. While $(|F| < n)$ do
 - A. Obtain an optimum basic feasible solution y to $GAPLP(G, M')$
 - B. If there is $ij \in E$ such that $y_{ij} = 0$ then $G = G - ij$
 - C. Else If there is $ij \in E$ such that $y_{ij} = 1$ then
 $F = F \cup \{(ij)\}$, $G = G - i$, $b_j = b_j - p_{ij}$
 - D. Else If there $j \in M'$ such that $d(j) = 1$ or $d(j) = 2$ and $\sum_i y_{ij} \geq 1$ then
 $M' = M' - j$.
3. Output assignment F

Theorem 6.7. *Given an instance of GAP that is feasible and has optimum cost C , the algorithm GAP-ITER-ROUNDING outputs an assignment whose cost is at most C and such that each machine j has load at most $2b_j$.*

The proof is by induction on the number of iterations. Alternatively, it is useful to view the algorithm recursively. We will sketch the proof and leave some of the formal details to the reader (who can also consult [185]). We observe that the algorithm makes progress in each iteration via Lemma 6.6. The analysis will consider the four cases that can happen in each iteration: (i) $y_{ij} = 0$ for some $ij \in E$ (ii) $y_{ij} = 1$ for some $ij \in E$ (iii) $d(j) \leq 1$ for some $j \in M'$ and (iv) $d(j) = 2$ and $\sum_{ij} y_{ij} \geq 1$ for some $j \in M'$.

Thus the algorithm terminates in polynomial number of iterations. It is also not hard to see that F corresponds to an assignment of jobs to machines.

Observation 6.8. *The algorithm terminates and outputs an assignment of jobs to machines, and job i is assigned to j implies $ij \in E$.*

Now we prove that the assignment has good properties in terms of the cost and loads.

Lemma 6.7. *The cost of the LP solution at the start of each iteration is at most $C - \sum_{ij \in F} c_{ij}$. Hence, at the end of the algorithm the cost of the assignment F is at most C .*

Proof. This is true in the first iteration since $F = \emptyset$ and the LP cost is less than that of an optimum integer feasible solution. Now consider an iteration assuming that the precondition holds.

If $y_{ij} = 0$ we remove ij from E and we note that the cost of the LP for the next iteration does not increase since y itself is feasible for the residual instance.

If $y_{ij} = 1$ and we add ij to F , we can charge the cost of ij to what the LP has already paid on the edge ij , and the solution y with ij removed is feasible to the residual instance obtained by removing job i and reducing the capacity of j to $b_j - p_{ij}$.

In the other cases we do not change F but drop constraints so the LP cost can only decrease in the subsequent iteration. ■

Now we upper bound the load on each machine j .

Lemma 6.8. *For each machine j , $\sum_{ij \in F} p_{ij} \leq 2b_j$. In fact, a stronger property holds: for each j , its load at the end of the algorithm is at most b_j or there is a single job assigned to j such that removing it reduces the load of j to at most b_j .*

Proof. The proof is by induction on iterations. We will sketch it. Consider a machine j . If $y_{ij} = 0$ in some iteration we remove ij and the load on any machine does not change. If $y_{ij} = 1$ we add ij to F but for subsequent iterations we reduce b_j by p_{ij} hence we account for the increase in load of j .

Thus, the only reason why the load of j may exceed b_j is because we drop the load constraint for j in some iteration. If we drop it when $d(j) = 1$, then at most one more job can be assigned to j and hence its final load can be at most $b_j + p_{ij}$ for some $ij \in E$. Thus, if $p_{ij} \leq b_j$ for all i the load is at most $2b_j$. We can also drop the constraint for j when $d(j) = 2$. However, in this case we have the property that $y_{i_a,j} + y_{i_b,j} \geq 1$ for some two jobs i_a and i_b which are the only edges incident to j in that iteration. Since y was feasible, we also had the constraint that $p_{i_a,j}y_{i_a,j} + p_{i_b,j}y_{i_b,j} \leq b'_j$ where b'_j was the residual capacity of j in that iteration. Assume without loss of generality that $p_{i_a,j} \leq p_{i_b,j}$; then it follows that $b'_j \geq p_{i_a,j}$. Thus the final load of j is at most $b_j - b'_j + p_{i_a,j} + p_{i_b,j}$ since both i_a and i_b can be assigned to j . But this load is at most $b_j + p_{i_b,j} \leq 2b_j$. We leave it to the reader to verify the refined property regarding the load claimed in the lemma. ■

Running time: The algorithm runs in polynomial number of iterations, and in each iteration it requires an optimum basic feasible solution to the $GAPLP(G, M')$. This can be done in polynomial time. We remark that the algorithm is deliberately described in a simple iterative fashion to make the proof easier. One can speed up the algorithm by considering all the cases together in each iteration. Although iterated rounding is a powerful technique, the running time is typically expensive.

Finding faster algorithms that achieve similar guarantees is an open area of research.

6.3 Maximization version of GAP

We consider the maximization version which we refer to **Max-GAP**. We have n items and m bins (instead of jobs and machines) where p_{ij} is the size of item i in bin j . Each bin j has a capacity b_j and assigning item i to bin j yields a profit/weight w_{ij} . The goal is to assign items to bins to maximize the weight while not violating any capacities. When $m = 1$ we obtain the **Knapsack** problem.

Multiple Knapsack Problem (MKP): MKP is a special case of **Max-GAP** in which $w_{ij} = w_i$ for all i, j and $p_{ij} = p_i$ for all i, j . In other words the item characteristics do not depend on where it is assigned to.

Exercise 6.3. Prove that MKP does not admit an FPTAS even for $m = 2$.

MKP admits a PTAS [73] and even an EPTAS [159]. Simply greedy algorithms that pack bins one by one using an algorithm for **Knapsack** as a black box yield a $(1 - 1/e - \epsilon)$ and $1/2 - \epsilon$ approximation for MKP when the bins are identical and when the bins are arbitrary [73].

In contrast to MKP, **Max-GAP** does not admit a PTAS. There is an absolute constant $\alpha > 1$ such that a $\alpha - \epsilon$ approximation implies $P = NP$ [73]. However, the following is known.

Theorem 6.9. *For every fixed m there is a PTAS for **Max-GAP**.*

The preceding theorem can be shown by generalizing the ideas behind the PTAS for **Knapsack** we discussed in an earlier chapter. An interested reader may try to prove this by considering the case of $m = 2$.

A $\frac{1}{2}$ -approximation: There is a simple yet clever way to achieve a $\frac{1}{2}$ -approximation for **Max-GAP** via the 2-approximation for the min-cost version that we already saw. Recall that for the min-cost version the algorithm outputs a solution with cost no more than the optimum cost while violating the capacity of each bin by at most one item. We outline how one can use this to obtain a 2-approximation for **Max-GAP** and leave the details as an exercise to the reader.

- Reduce the exact maximization version to the exact min-cost version in which all items have to be assigned by adding an extra dummy bin.
- Use the result for min-cost version to obtain an assignment with weight at least that of the optimum while violating each bin's capacity by at most one item.

- Use the preceding assignment to find a feasible packing of items that has profit at least $\text{OPT}/2$.

6.4 Configuration LP: A Stronger LP Relaxation

The LP relaxation for unrelated machine scheduling and **Max-GAP** have integrality gap of 2 and $1/2$ respectively. The examples are simple. For scheduling: consider an instance with n machines and $n + 1$ jobs each of which can be scheduled on any machine with size 1. Then optimum schedule value is 2 while the LP allows a value of $1 + 1/n$. For **Max-GAP**, consider n bins with capacities 1 and $2n$ items of size $1/2 + \epsilon$ (same size on each bin) then we can only pack n items, while the fractional solution will essentially pack all the jobs. As one can see, these bad examples hold for the simpler setting of identical machines where we can obtain PTASes. Can we do better than 2 and $1/2$ in the general setting? Another natural question is whether we can *strengthen* the LP relaxation. Indeed we can, but it requires some sophisticated ideas. We will give a sketch in this section.

Unrelated machine scheduling: We will write a big *implicit* LP as follows. We have n jobs and m machines. For each machine j let $\mathcal{S}_j = \{S \subseteq J \mid p_j(S) \leq b_j\}$ be the set of all subsets of jobs that can fit feasibly in machine j . Note that given $S \subseteq J$ we can check if $S \in \mathcal{S}_j$ efficiently (and easily). For each $S \in \mathcal{S}_j$ and j we have a variable $x(S, j)$ in our LP. We want to assign to each machine j exactly one set S from $\mathcal{S}_j$. Moreover we want to make sure that every job is assigned to some machine. This motivates us to write the following feasibility LP.

GAP-Configuration-LP

$$\begin{aligned} \sum_{S \in \mathcal{S}_j} x(S, j) &= 1 & \forall j \in M \\ \sum_{j=1}^m \sum_{S \in \mathcal{S}_j: i \in S} x(S, j) &= 1 & \forall i \in J \\ x(S, j) &\geq 0 & \forall j \in M, S \in \mathcal{S}_j \end{aligned}$$

Another way to see the above LP is as follows. For each job i and machine j define $x_{i,j} = \sum_{S \in \mathcal{S}_j: i \in S} x(S, j)$. Then $x_{i,j}$ defines the fraction of i assigned to j as before. However we are insisting that these fractional assignments should satisfy more than the simple volume constraint for machine j of the form $\sum_i p_{i,j} x_{i,j} \leq b_j$.

They need to be supported by a distribution over sets that are feasible in bin j . For example, if we have a bin j of capacity 1 and two items of size $1/2 + \epsilon$ each. Only one of them can fit in the bin so there are only three feasible sets including the emptyset. While the simple volume constraint allows both items to be simultaneously assigned up to $(1 - \epsilon)$ each, the configuration LP will force their total assignment to at most 1. Do you see why?

It is not obvious that we can solve the preceding LP efficiently but it turns out that one can solve it to within a $(1 - \epsilon)$ -approximation. Is this LP any better? It is indeed a strengthening. It turns out that the integrality gap is still 2 for the general unrelated machine scheduling problem, however it does improve to strictly less than 2 for the *Restricted Assignment Problem*. In this special case p_{ij} is p_i or ∞ for each job i ; in other words each job specifies the machines it can be assigned to but its processing time does not depend on the machine. Even for this restricted case, unless $P = NP$, one cannot obtain better than $3/2$ -approximation. However, it is known that the integrality gap of the configuration LP for this special case is at most 1.8333 approximation via the configuration LP. Hence the configuration LP is in fact strictly stronger than the vanilla LP.

Max-GAP: We now consider the configuration LP for **Max-GAP** and how to obtain a $(1 - 1/e)$ -approximation. We write the relaxation below. It is similar to the one for scheduling that now a job/item need not be assigned fully so we change the equality to an inequality for each job i . Further we have an objective. We use w_{ij} to denote the weight/value of assigning i to j and $w_j(S) = \sum_{i \in S} w_{ij}$.

Max-GAP-Configuration-LP

$$\begin{aligned}
 \max \quad & \sum_{j=1}^m \sum_{S \in \mathcal{S}_j} w_j(S) x(S, j) \\
 \sum_{S \in \mathcal{S}_j} x(S, j) &= 1 \quad \forall j \in M \\
 \sum_{j=1}^m \sum_{S \in \mathcal{S}_j: i \in S} x(S, j) &\leq 1 \quad \forall i \in J \\
 x(S, j) &\geq 0 \quad \forall j \in M, S \in \mathcal{S}_j
 \end{aligned}$$

Suppose we can solve the above relaxation and obtain a feasible solution y . We describe a simple rounding scheme. For each machine j we interpret $\sum_{S \in \mathcal{S}_j} y(S, j) = 1$ as specifying a probability distribution over sets that have been assigned to machine j . We assign a single set to j according to this distribution:

in other words $S \in \mathcal{S}_i$ is assigned to j with probability precisely equal to $y(S, j)$. We do the assignment to each machine independently. Let A_j be the set of items assigned to machine j . Note that we are not guaranteed that these sets will be disjoint. An item i may belong to multiple sets! How do we resolve the conflicts? For each item i , if it is assigned to multiple sets, we give it to the machine/bin with the largest weight for item i . This gives us a new assignment A'_j for each machine j .

It is easy to see that $A'_j \subseteq A_j$ for each j and also that $A'_j \in \mathcal{S}_j$ and hence feasible in machine j .

Lemma 6.9. *Let $\text{OPT}_{LP} = \sum_j \sum_{S \in \mathcal{S}_j} w_j(S) y(S, j)$. Then the $E[\sum_j w_j(A'_j)] \geq (1 - 1/e) \text{OPT}_{LP}$.*

We sketch a proof of the above lemma. We analyze the process from the perspective of the items. Note that $\text{OPT}_{LP} = \sum_{i,j} w_{i,j} x_{i,j}$ where $x_{i,j}$ is the fraction of item i that is assigned to machine j . Fix item i . Let Z_i be the random variable which indicates the weight of i in the bin that i is assigned to by the algorithm — note that i may not be assigned in which case Z_i is 0. By renumbering, we can assume without loss of generality, that $w_{i,1} \geq w_{i,2} \dots \geq w_{i,m}$. From the algorithm description, i will be in A_j independently with probability $x_{i,j}$. Since we prefer to assign i in order of the bins after sorting, we can then see that

$$E[Z_i] = w_{i,1}x_{i,1} + w_{i,2}(1 - x_{i,1})x_{i,2} + \dots + w_{i,m}(1 - x_{i,1})(1 - x_{i,2}) \dots (1 - x_{i,m-1}).$$

It is not hard to prove that the above quantity is at least $(1 - 1/e) \sum_{j=1}^m w_{i,j} x_{i,j}$. The worst case is when all the weights are the same and $x_{i,j} = 1/m$ for all j and we are then focusing on whether item i is allocated at all. We leave the formal details as an exercise to the reader. Once we have this for a single item, by linearity of expectation, we obtain the lemma. This finishes the sketch of the analysis.

Solving the LP via the dual and separation oracle: We sketch how the LP can be solved. Since the primal LP has an exponential number of variables we cannot even write down the variables. However the LP only has a polynomial number of non-trivial constraints so we consider the dual. We introduce a dual variable α_j for each machine j and a dual variable β_i for each item i . To simplify the dual we will assume that in the primal we can change the first constraint to be an inequality instead of an equality. This does not change the objective value but has the advantage of making the primal LP a pure packing LP. The dual becomes a pure covering LP.

Max-GAP-Configuration-Dual-LP

$$\begin{aligned}
 & \min \sum_{j=1}^m \alpha_j + \sum_{i=1}^n \beta_i \\
 & \alpha_j + \sum_{i \in S} \beta_i \geq w_j(S) \quad \forall j, S \in \mathcal{S}_j \\
 & \alpha_j, \beta_i \geq 0
 \end{aligned}$$

The dual has a $m + n$ variables but an exponential number of constraints. The Ellipsoid method allows one to solve such LPs in polynomial-time if there is an efficient *separation oracle* for the LP. A separation oracle for such an LP is an algorithm that does the following: given a point $x \in \mathbb{R}^d$ (here we are assuming that the LP is over d variables) it should answer if x satisfies all the constraints of the LP. The oracle has to correctly answer this question, and if the answer is no for a given x , it has to provide a constraint in the LP that is violated by x .

Exercise 6.4. Argue that an algorithm for the Knapsack problem would suffice as a separation oracle for the preceding dual LP.

As we know, Knapsack is NP-Hard so we do not have an “efficient” separation oracle. However we can approximate Knapsack arbitrarily well — we have an FPTAS as we already saw. It is not obvious whether such an approximate separation oracle will be useful in the Ellipsoid method. It turns out that this is indeed the case since we have a pure packing/covering pair of LPs. Thus a $(1 - \epsilon)$ -approximation for Knapsack can be used to obtain a $(1 - \epsilon)$ -approximation to the primal LP. Note that we will obtain an approximation to the primal LP in terms of value and will be guaranteed to find a feasible solution.

6.5 Bibliographic Notes

The 2-approximation for unrelated machine scheduling is by Lenstra, Shmoys and Tardos [191]. The same paper showed that unless $P = NP$ there is no $3/2 - \epsilon$ -approximation for unrelated machine scheduling, even for the restricted assignment problem that we mentioned above. Bridging this gap has been a major open problem in scheduling. Svensson [241] showed that the configuration LP has an integrality gap better than 2 for restricted assignment problem but so far there is no polynomial time algorithm to actually output an assignment! His bound was improved to 1.8333 [161]. The best algorithm that beats 2 for this case

runs in quasi-polynomial time [160]. Thus, although we are able to estimate the optimum value to within a factor of 1.8333 of the optimum in polynomial-time, we do not have an algorithm to actually compute a schedule achieving that guarantee.

As we mentioned Shmoys and Tardos obtained the 2-approximation for **GAP**. The iterated rounding proof is from [185].

The approximability of **Max-GAP** when m is not a fixed constant was studied in [73] although a PTAS for fixed m was known quite early [60]. [73] described a $1/2$ -approximation. Fleischer et al. [113] used the configuration LP to obtain a $(1 - 1/e)$ -approximation that we described. Feige and Vondrák [108] showed that $(1 - 1/e)$ is not tight by obtaining a $(1 - 1/e + \delta)$ -approximation for a tiny but fixed δ via the same LP approximation ratio is via a configuration LP. The precise integrality gap of the configuration LP is an interesting open problem. There are interesting connections between **Max-GAP** and submodular function maximization. See [52]. This connection allows one to obtain a very simple $1/2 - \epsilon$ greedy approximation algorithm that is not obvious.

Chapter 7

Congestion Minimization in Networks

In Chapter 6 we saw the SCHEDULING ON UNRELATED PARALLEL MACHINES problem. Here we consider two problems that also consider allocation with the objective of minimizing load/congestion. We will first consider the CONGESTION MINIMIZATION problem in graphs and then the abstract problem of MIN-MAX-INTEGER-PROGRAMS.

7.1 Congestion Minimization and VLSI Routing

A classical routing problem that was partly inspired by VLSI (very large scale integrated) circuit design is the following. Let $G = (V, E)$ be a directed graph and let $(s_1, t_1), \dots, (s_k, t_k)$ be k source-sink pairs. We want to connect each pair (s_i, t_i) by a path P_i such that the paths do not share edges (or nodes). Alternatively we would like to minimize the maximum number of paths that use any edge — this is called the CONGESTION MINIMIZATION problem. A special case is EDP which is to decide if there are paths for the pairs that are edge-disjoint (NDP is the version where the paths are required to be node-disjoint). EDP and NDP are classical NP-Complete problems and have many important connections to multicommodity flows, routing, cuts, and graph theory. Thus CONGESTION MINIMIZATION is an NP-Hard optimization problem. Here we will consider two variants.

Choosing one path from a given collection: We consider a conceptually simpler variant where we are given a finite path collection $\mathcal{P}_i$ for each pair (s_i, t_i) where each path $P \in \mathcal{P}_i$ connects s_i to t_i . The goal is to choose, for each pair (s_i, t_i) , one path from $\mathcal{P}_i$ so as to minimize the maximum congestion on any edge. We can develop an integer programming formulation as follows. For each i and each $P \in \mathcal{P}_i$ we have a variable $x_{i,P}$ which indicates whether we choose P

to route pair i . The constraints express that exactly one path is chosen for each pair i . To minimize the maximum number of paths using any edge we introduce a variable λ and minimize it subject to a natural packing constraint.

$$\begin{aligned}
& \text{minimize} && \lambda \\
& \text{subject to} && \sum_{P \in \mathcal{P}_i} x_{i,P} = 1 && 1 \leq i \leq k \\
& && \sum_{i=1}^k \sum_{P \in \mathcal{P}_i, P \ni e} x_{i,P} \leq \lambda && \forall e \in E \\
& && x_{i,P} \in \{0, 1\} && 1 \leq i \leq k, P \in \mathcal{P}_i
\end{aligned}$$

As usual we relax the integer constraints to obtain an LP relaxation where we replace $x_{i,P} \in \{0, 1\}$ with $x_{i,P} \in [0, 1]$ (in this particular case we can simply use $x_{i,P} \geq 0$ due to the other constraints). Note the similarities with the IP/LP for SCHEDULING ON UNRELATED PARALLEL MACHINES. The LP relaxation is of size polynomial in the input size since the path collection $\mathcal{P}_i$ is explicitly given for each i as part of the input.

Let λ^* be the optimum LP solution value. There is a technicality that arises just as we saw with SCHEDULING ON UNRELATED PARALLEL MACHINES. It may happen that $\lambda^* < 1$ while we know that the optimum congestion is at least 1. Technically we should find the smallest integer λ^* such that the preceding LP is feasible. We will assume henceforth that λ^* is an integer.

How do we round? A simple strategy is randomized rounding. In fact the technique of randomized rounding and analysis via Chernoff bounds was developed in the influential paper of Raghavan and Thompson [221] precisely for this problem!

RANDOMIZED-ROUNDING

1. Solve LP relaxation and find optimum solution x^*, λ^* .
2. For $i = 1$ to k do
 - A. Pick exactly one path $Q_i \in \mathcal{P}_i$ randomly where the probability of picking P is exactly $x_{i,P}^*$.
3. Output $Q_1, Q_2, \dots, Q_k$.

Note that the choices for the pairs are done with independent randomness. The analysis requires the use of Chernoff-Hoeffding bounds. See Chapter B.

Theorem 7.1. *Randomized rounding outputs one path per pair and with probability at least $(1 - 1/m^2)$ no edge is contained in more than $c \frac{\log m}{\log \log m} \cdot \lambda^*$ paths where c is an absolute constant. Here m are the number of edges in the graph G . One can also show that for any fixed $\epsilon > 0$ the congestion is at most $(1 + \epsilon)\lambda^* + c \frac{\log m}{\epsilon^2}$ with high probability.*

Proof. The proof is a simple application of Chernoff-bounds and the union bound. Fix an edge $e \in E$. Let Y_e be the random variable which is the total number of paths in $Q_1, Q_2, \dots, Q_k$ that use e . Let $Y_{e,i}$ be the binary random variable that indicates whether $e \in Q_i$. Note that $Y_e = \sum_{i=1}^k Y_{e,i}$.

The first observation is that the variables $Y_{e,1}, Y_{e,2}, \dots, Y_{e,k}$ are independent since we used independent randomness for the pairs. Second we claim that $\mathbf{E}[Y_{e,i}] = \mathbf{P}[Y_{e,i} = 1] = \sum_{P \in \mathcal{P}_i, e \in P} x_{i,P}^*$. Do you see why? Thus, by linearity of expectation,

$$\mathbf{E}[Y_e] = \sum_i \mathbf{E}[Y_{e,i}] = \sum_i \sum_{P \in \mathcal{P}_i, e \in P} x_{i,P}^* \leq \lambda^*$$

where the last inequality follows from the LP constraint.

Since Y_e is the sum of independent binary valued random variables and $\mathbf{E}[Y_e] \leq \lambda^*$ we can apply Chernoff-bounds to estimate $\mathbf{P}[Y_e \geq c \frac{\log m}{\log \log m} \lambda^*]$. Applying Corollary B.5 we conclude that we can choose c such that this probability is at most $1/m^3$. Now we apply the union bound over all edges and conclude that

$$\mathbf{P}[\exists e \in E, Y_e \geq c \frac{\log m}{\log \log m} \lambda^*] \leq m/m^3 \leq 1/m^2.$$

Thus, with probability $\geq 1 - 1/m^2$ no edge is loaded to more than $c \frac{\log m}{\log \log m} \lambda^*$.

The second bound can be derived in the same way by using the second Chernoff-bound in Corollary B.5. $\blacksquare$

Remark 7.1. We chose the bound $(1 - 1/m^2)$ for concreteness. A success probability of the form $(1 - 1/\text{poly}(n))$ where n is the input size is typically called “with high probability”.

Remark 7.2. The bound $(1 + \epsilon)\lambda^* + c \log m / \epsilon^2$ implies that when $\lambda^* \geq c \log m$ we obtain a constant factor approximation.

Remark 7.3. In a graph $m = O(n^2)$ and hence one often sees the bounds expressed in terms of n rather than m . We chose to write in terms of m rather than n to highlight the fact that the bound depends on the number of constraints via the union bound. We will discuss later how column sparsity based bounds can give refined results that avoid the union bound.

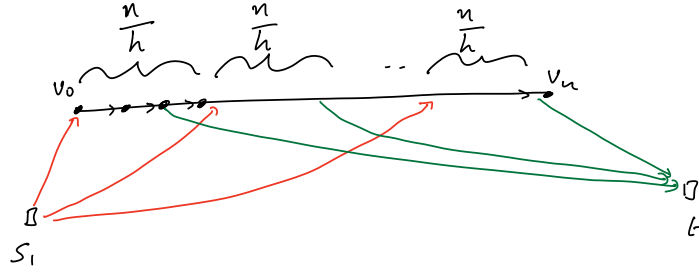
Implicitly given path collections: In the traditional version of CONGESTION MINIMIZATION we are only given G and the pairs, and the goal is to choose one path P for each (s_i, t_i) pair from the set of all paths between s_i and t_i . In other words $\mathcal{P}_i = \{P \mid P \text{ is an } s_i\text{-}t_i \text{ path in } G\}$. $\mathcal{P}_i$ is implicitly defined and its size can be exponential in the graph size. It is not obvious that we can solve the LP relaxation that we saw above. However, one can indeed solve it in polynomial-time via the Ellipsoid method. First, we observe that the LP could have an exponential number of variables but only a polynomial number of non-trivial constraints: k for the pairs and m for the edges. Thus, one is guaranteed, by the rank lemma that there is an optimum solution that has only $k + m$ non-zero variables. To see that one can indeed find it efficiently, we need to look at the dual and notice that the separation oracle for the dual is the shortest path problem.

Another way to see that the LP can be solved is by writing a compact formulation via the well-known multicommodity flow. We want to send one unit of flow from s_i to t_i so that the total flow on any edge is at most λ . We use variables $f(e, i)$ to denote a flow for pair i on edge e .

$$\begin{aligned}
& \text{minimize} && \lambda \\
& \text{subject to} && \sum_{e \in \delta^+(s_i)} f(e, i) - \sum_{e \in \delta^-(s_i)} f(e, i) = 1 && 1 \leq i \leq k \\
& && \sum_{e \in \delta^+(v)} f(e, i) - \sum_{e \in \delta^-(v)} f(e, i) = 0 && 1 \leq i \leq k, v \in V - \{s_i, t_i\} \\
& && \sum_{i=1}^k f(e, i) \leq \lambda && \forall e \in E \\
& && f(e, i) \geq 0 && 1 \leq i \leq k, e \in E
\end{aligned}$$

The preceding multicommodity flow LP has a polynomial number of variables and can be solved in polynomial-time. Given a flow, for each commodity/pair i we can take the one unit of $s_i\text{-}t_i$ flow and use standard flow-decomposition to obtain a path-flow with at most m paths in the collection. We can then apply the rounding that we saw above with an given explicit path collection in exactly the same way.

Remark 7.4. The Ellipsoid based algorithm may seem impractical. However, one can approximately solve the implicit path based LP via multiplicative weight update methods efficiently. The implicit formulation and Ellipsoid method is also useful when one may want to restrict $\mathcal{P}_i$ in some fashion. For instance we can set $\mathcal{P}_i$ to be the set of all $s_i\text{-}t_i$ paths in G with at most d edges for some given



parameter d . This will ensure that we choose only “short” paths for each pair. It is not hard to see that the separation oracle for the dual is another shortest path type problem that can be solved efficiently (via Bellman-Ford type algorithm). This is not easy to capture/see via the compact flow based formulation.

Derandomization: Is there a deterministic algorithm with the roughly the same approximation guarantee? The algorithm can be derandomized via the notion of pessimistic estimators. CONGESTION MINIMIZATION was one of the first instances with a sophisticated use of this technique [220].

Integrality gap and Hardness of Approximation: There is simple yet clever example demonstrating that the integrality gap of the flow relaxation in directed graphs is $\Omega(\log m / \log \log m)$ [190]. In a remarkable result, [86] showed that $O(\log m / \log \log m)$ is the hardness factor. The complexity of CONGESTION MINIMIZATION is less clear in *undirected* graphs. It is known that the LP integrality gap and hardness of approximation are $\Omega(\log \log n / \log \log \log n)$ [14]. Closing the gap between the upper and lower bounds is a major open problem.

Here we outline the integrality gap example for directed graphs from [190]. The graph G and the pairs are constructed in a recursive fashion. Let h be a parameter that we will fix later. We start with a directed path $v_0, v_1, \dots, v_n$. We add a demand pair (s_1, t_1) which connects to the path as follows. We partition the path into n/h paths of equal length: add an arc from s to the start of each sub-path and an arc from the end of each sub-path to t . See figure.

One can see from the figure that the pair (s, t) can split its flow along h paths. Now we consider each of the h sub-paths and recursively create an instance on the path with length $n/h - 1$ (while keeping parameter h the same). Note that in the second level of the recursion we add h new source-sink pairs, one for each sub-path. We stop the recursion when the size of the sub-path is $\Theta(h)$. Let d be the depth of the recursion.

We claim that there is a fractional routing of all demand pairs where the congestion is at most d/h . This follows by splitting the flow of the pairs h ways.

The next claim is that some edge has congestion d in any integral routing. This can be seen inductively. The top level pair (s, t) has to choose one amongst the h sub-paths — all edges in that sub-path will be used by the route for (s, t) . Inductively there is some edge in that sub-path with congestion $d - 1$ and hence the congestion of that edge will be d when we add the path for (s, t) .

It now remains to set the parameters. If we choose $h = \log^2 n$ say then $d = \Theta(\log n / \log \log n)$. The fractional congestion is ≤ 1 and integral congestion is $\Theta(\log n / \log \log n)$.

Short paths and improved congestion via Lovász-Local-Lemma: We consider the congestion minimization problem when the path for each pair is required to be “short”. By this we mean that we are required to route on a path with at most d edges where d is some given parameter. One can imagine that in many applications d is small and is a fixed constant, say 10. The question is whether the approximation ratio can be improved. Indeed one can show that the LP integrality gap is $O(\log d / \log \log d)$. Thus, when $d \ll n$ we get a substantial improvement. However, proving this and obtaining a polynomial time algorithm are quite non-trivial. One requires the use of the subtle Lovász-Local-Lemma (LLL), a powerful tool in probabilistic combinatorics. Typically LLL only gives a proof of existence and there was substantial work in making LLL constructive/efficient. Srinivasan obtained an algorithm via derandomization of LLL in this context with a lot of technical work [238]. There was a breakthrough work of Moser and Tardos [206] that gave an extremely simple way to make LLL constructive and this has been refined and developed over the last decade. For the congestion minimization problem we refer the reader to [138] which builds upon [206] and describes an efficient randomized algorithm that outputs a solution with congestion $O(\log d / \log \log d)$. In fact the application is given in the context of a more abstract problem that we discuss in the next section.

Integer flows and Unsplittable flows: We worked with the simple setting where each pair (s_i, t_i) wishes to send one unit of flow. One can imagine a situation where one wants to send d_i units of flow for pair i where d_i is some (integer) demand value. There are two interesting variants. The first one requires integer valued flow for each pair which means that we want to find d_i paths for (s_i, t_i) that each carry one unit of flow (the paths can overlap). This variant can be essentially reduced to the unit demand flow by creating d_i copies of (s_i, t_i) — we leave this as a simple exercise for the reader. The second variant is that we want each pair’s flow of d_i units to be sent along a single path — this is called *unsplittable* flow. When discussing unsplittable flow it is also natural to consider capacities on the edges. Thus, each edge has a capacity u_e and one wants to minimize congestion relative to u_e . The techniques we discussed can be generalized relatively easily to this version as well to obtain the same kind

of bounds. The unsplittable flow problem is interesting even in the setting where there is a single source/sink or when the graph is a simple ring or a path. Interesting results are known here and we refer the reader to [3, 55, 92, 129, 205, 237] for further pointers.

7.2 Min-max Integer Programs

If one looks at the rounding and analysis for CONGESTION MINIMIZATION we notice that the algorithm uses very little about the structure of the graph. This can be thought about in two ways. One is that perhaps we can do better by exploiting graph structure. Two, we can abstract the problem into a more general class where the same technique applies. As we mentioned, in directed graphs the bound of $O(\log n / \log \log n)$ is tight but the bound may not be tight in undirected graphs which admit more structure.

Here we consider the second point and develop a resource allocation view point while making an analogy to CONGESTION MINIMIZATION so that the abstract problem can be more easily understood. Suppose we have m resources $e_1, e_2, \dots, e_m$. We have k requests $r_1, r_2, \dots, r_k$. Each request i can be satisfied in ℓ_i ways — let $\mathcal{P}_i$ denote a collection of ℓ_i vectors $v_{i,1}, v_{i,2}, \dots, v_{i,\ell_i}$. Each vector $v_{i,j} \in \mathcal{P}_i$ is an m -dimensional vector: for each $k \in [m]$, $v_{i,j,k}$ is a scalar that represents the load it induces on resource e_k . The goal is to choose, for each i , exactly one $j \in [\ell_i]$ so as to minimize the maximum load on any resource. One can write this conveniently as the following integer program where we have variables $x_{i,j}$ for $1 \leq i \leq k$ and $1 \leq j \leq \ell_i$ which indicates whether i chooses j .

$$\begin{array}{ll}
 \text{minimize} & \lambda \\
 \text{subject to} & \sum_{1 \leq j \leq \ell_i} x_{i,j} = 1 \quad 1 \leq i \leq k \\
 & \sum_{i=1}^k \sum_{1 \leq j \leq \ell_i} v_{i,j,k} x_{i,j} \leq \lambda \quad \forall e_k \\
 & x_{i,j} \in \{0, 1\} \quad 1 \leq i \leq k, 1 \leq j \leq \ell_i
 \end{array}$$

One can view the above integer program compactly as

$$\begin{array}{ll}
\text{minimize} & \lambda \\
\text{subject to} & \sum_{1 \leq j \leq \ell_i} x_{i,j} = 1 \quad 1 \leq i \leq k \\
& Ax \leq \lambda \mathbf{1} \\
& x_{i,j} \in \{0, 1\} \quad 1 \leq i \leq k, 1 \leq j \leq \ell_i
\end{array}$$

where A is a non-negative matrix with m rows. As with SCHEDULING ON UNRELATED PARALLEL MACHINES we need to be careful when relaxing the IP to an LP since the optimum solution to the LP can be a poor lower bound unless we ensure that $x_{i,j} = 0$ if $v_{i,j,k} > \lambda$. We will assume that we have indeed done this.

One can do randomized rounding exactly as we did for CONGESTION MINIMIZATION and obtain an $O(\log m / \log \log m)$ approximation. We say that A is d -column-sparse if the maximum number of non-zeroes in any column of A is at most d . This corresponds to paths in CONGESTION MINIMIZATION being allowed to have only d edges. One can obtain an $O(\log d / \log \log d)$ -approximation in this more general setting as well [138].

Chapter 8

Introduction to Local Search

Local search is a powerful and widely used heuristic method (with various extensions). In this lecture we introduce this technique in the context of approximation algorithms. The basic outline of local search is as follows. For an instance I of a given problem let $\mathcal{S}(I)$ denote the set of feasible solutions for I . For a solution S we use the term (*local*) *neighborhood* of S to be the set of all solutions S' such that S' can be obtained from S via some *local moves*. We let $N(S)$ denote the neighborhood of S .

```
LOCALSEARCH:  
Find a "good" initial solution  $S_0 \in \mathcal{S}(I)$   
 $S \leftarrow S_0$   
repeat  
  If  $(\exists S' \in N(S)$  such that  $\text{val}(S')$  is strictly better than  $\text{val}(S)$ )  
     $S \leftarrow S'$   
  Else  
     $S$  is a local optimum  
    return  $S$   
  EndIf  
Until (True)
```

For minimization problems S' is strictly better than S if $\text{val}(S') < \text{val}(S)$ whereas for maximization problems it is the case if $\text{val}(S') > \text{val}(S)$.

The running time of the generic local search algorithm depends on several factors. First, we need an algorithm that given a solution S either declares that S is a local optimum or finds a solution $S' \in N(S)$ such that $\text{val}(S')$ is strictly better than $\text{val}(S)$. A standard and easy approach for this is to ensure that the local moves are defined in such a way that $|N(S)|$ is polynomial in the input size $|I|$ and $N(S)$ can be enumerated efficiently; thus one can check each $S' \in N(S)$ to

see if any of them is an improvement over S . However, in some more advanced settings, $N(S)$ may be exponential in the input size but one may be able to find a solution in $S' \in N(S)$ that improves on S in polynomial time. Second, the running time of the algorithm depends also on the number of iterations it takes to go from S_0 to a local optimum. In the worst case the number of iterations could be $|\text{OPT} - \text{val}(S_0)|$ which can be exponential in the input size. One can often use a standard scaling trick to overcome this issue; we stop the algorithm unless the improvement obtained over the current S is a significant fraction of $\text{val}(S)$. Finally, the quality of the initial solution S_0 also factors into the running time.

Remark 8.1. There is a distinction made sometimes in the literature between *oblivious* and *non-oblivious* local search. In oblivious local search the algorithm uses f as a black box when comparing S with a candidate solution S' ; the analysis and the definition of local moves are typically based on properties of f . In non-oblivious local search one may use an auxiliary function g derived from f in comparing S with S' . Typically the function g is some kind of *potential function* that aids the analysis. This allows one to move to a solution S' even though $f(S')$ may not be an improvement over $f(S)$.

Remark 8.2. There are several heuristics that are (loosely) connected to local search. These include simulated annealing, tabu search, genetic algorithms and others. These fall under the broad terminology of metaheuristics.

8.1 Local Search for MAX CUT

We illustrate local search for the well-known MAX CUT problem. In MAX CUT we are given an undirected graph $G = (V, E)$ and the goal is to partition V into $(S, V \setminus S)$ so as to maximize the number of edges crossing S , that is, $|\delta_G(S)|$; we will use $\delta(S)$ when G is clear. For a vertex v we use $\delta(v)$ instead of $\delta(\{v\})$ to simplify notation. In the weighted version each edge e has a non-negative weight $w(e)$ and the goal is to maximize the weight of the edges crossing S , that is, $w(\delta(S))$; here $w(A)$ for a set A denotes the quantity $\sum_{e \in A} w(e)$.

We consider a simple local search algorithm for MAX CUT that starts with an arbitrary set $S \subseteq V$ and in each iteration either adds a vertex to S or removes a vertex from S as long as it improves the cut value $\delta(S)$.

LOCALSEARCH FOR MAX CUT:
 $S \leftarrow \emptyset$
 repeat
 If $(\exists v \in V \setminus S \text{ such that } w(\delta(S + v)) > w(\delta(S)))$
 $S \leftarrow S + v$
 Else If $(\exists v \in S \text{ such that } w(\delta(S - v)) > w(\delta(S)))$
 $S \leftarrow S - v$
 Else
 S is a *local optimum*
 return S
 EndIf
 Until (True)

We will first focus on the quality of solution output by the local search algorithm.

Lemma 8.1. *Let S be the output of the local search algorithm. Then for each vertex v , $w(\delta(S) \cap \delta(v)) \geq w(\delta(v))/2$.*

Proof. Let $\alpha_v = w(\delta(S) \cap \delta(v))$ be the weight of edges among those incident to v that cross the cut S . Let $\beta_v = w(\delta(v)) - \alpha_v$.

We claim that $\alpha_v \geq \beta_v$ for each v . If $v \in V \setminus S$ and $\alpha_v < \beta_v$ then moving v to S will strictly increase $w(\delta(S))$ and S cannot be a local optimum. Similarly if $v \in S$ and $\alpha_v < \beta_v$, we would have $w(\delta(S - v)) > w(\delta(S))$ and S would not be a local optimum. ■

Corollary 8.1. *If S is a local optimum then $w(\delta(S)) \geq w(E)/2 \geq \text{OPT}/2$.*

Proof. Since each edge is incident to exactly two vertices we have $w(\delta(S)) = \frac{1}{2} \sum_{v \in V} w(\delta(S) \cap \delta(v))$. Apply the above lemma,

$$\begin{aligned}
 w(\delta(S)) &= \frac{1}{2} \sum_{v \in V} w(\delta(S) \cap \delta(v)) \\
 &\geq \frac{1}{2} \sum_{v \in V} w(\delta(v))/2 \\
 &\geq \frac{1}{2} w(E) \\
 &\geq \frac{1}{2} \text{OPT},
 \end{aligned}$$

since $\text{OPT} \leq w(E)$. ■

The running time of the local search algorithm depends on the number of local improvement iterations; checking whether there is a local move that results

in an improvement can be done by trying all possible vertices. If the graph is unweighted then the algorithm terminates in at most $|E|$ iterations. However, in the weighted case, it is known that the algorithm can take an exponential time in $|V|$ when the weights are large. A very interesting problem is whether one can find a local optimum efficiently — note that we do not need to find a local optimum by doing local search! It is believed that finding a local optimum for MAX CUT in the weighted case is hard. There is a complexity class called PLS that was defined by Johnson, Papadimitriou and Yannakakis for which MAX CUT is known to be a complete problem. PLS plays an important role in algorithmic game theory in recent times. We refer the reader to the Wikipedia page on PLS for many pointers.

Many local search algorithms can be modified slightly to terminate with an approximate local optimum such that (i) the running time of the modified algorithm is strongly polynomial in the input size and (ii) the quality of the solution is very similar to that given by the original local search. We illustrate these ideas for MAX CUT. Consider the following algorithm where $\epsilon > 0$ is a parameter that can be chosen. Let n be the number of nodes in G .

```

MODIFIED LOCALSEARCH FOR MAX CUT( $\epsilon$ ):
 $S \leftarrow \{v^*\}$  where  $v^* = \arg \max_{v \in V} w(\delta(v))$ 
repeat
  If ( $\exists v \in V \setminus S$  such that  $w(\delta(S + v)) > (1 + \frac{\epsilon}{n})w(\delta(S))$ )
     $S \leftarrow S + v$ 
  Else If ( $\exists v \in S$  such that  $w(\delta(S - v)) > (1 + \frac{\epsilon}{n})w(\delta(S))$ )
     $S \leftarrow S - v$ 
  Else
    return  $S$ 
  EndIf
Until (True)

```

The above algorithm terminates unless the improvement is a relative factor of $(1 + \frac{\epsilon}{n})$ over the current solution's value. Thus the final output S is an *approximate* local optimum.

Remark 8.3. An alert reader may wonder why the improvement is measured with respect to the global value $w(\delta(S))$ rather than with respect to $w(\delta(v))$. One reason is to illustrate the general idea when one may not have fine grained information about the function like we do here in the specific case of MAX CUT. The global analysis will also play a role in the running time analysis as we will see shortly.

Lemma 8.2. *Let S be the output of the modified local search algorithm for MAX CUT. Then $w(\delta(S)) \geq \frac{1}{2(1+\epsilon/4)}w(E)$.*

Proof. As before let $\alpha_v = w(\delta(S) \cap \delta(v))$ and $\beta_v = w(\delta(v)) - \alpha_v$. Since S is an approximately local optimum we claim that for each v

$$\beta_v - \alpha_v \leq \frac{\epsilon}{n} w(\delta(S)).$$

Otherwise a local move using v would improve S by more than $(1 + \epsilon/n)$ factor. (The formal proof is left as an exercise to the reader).

We have,

$$\begin{aligned} w(\delta(S)) &= \frac{1}{2} \sum_{v \in V} \alpha_v \\ &= \frac{1}{2} \sum_{v \in V} ((\alpha_v + \beta_v) - (\beta_v - \alpha_v))/2 \\ &\geq \frac{1}{4} \sum_{v \in V} (w(\delta(v)) - \frac{\epsilon}{n} w(S)) \\ &\geq \frac{1}{2} w(E) - \frac{1}{4} \sum_{v \in V} \frac{\epsilon}{n} w(S) \\ &\geq \frac{1}{2} w(E) - \frac{1}{4} \epsilon \cdot w(S). \end{aligned}$$

Therefore $w(S)(1 + \epsilon/4) \geq w(E)/2$ and the lemma follows. $\blacksquare$

Now we argue about the number of iterations of the algorithm.

Lemma 8.3. *The modified local search algorithm terminates in $O(\frac{1}{\epsilon} n \log n)$ iterations of the improvement step.*

Proof. We observe that $w(S_0) = w(\delta(v^*)) \geq \frac{2}{n} w(E)$ (why?). Each local improvement iteration improves $w(\delta(S))$ by a multiplicative factor of $(1 + \epsilon/n)$. Therefore if k is the number of iterations that the algorithm, then $(1 + \epsilon/n)^k w(S_0) \leq w(\delta(S))$ where S is the final output. However, $w(\delta(S)) \leq w(E)$. Hence

$$(1 + \epsilon/n)^k 2w(E)/n \leq w(E)$$

which implies that $k = O(\frac{1}{\epsilon} n \log n)$. $\blacksquare$

A tight example for local optimum: Does the local search algorithm do better than $1/2$? Here we show that a local optimum is no better than a $1/2$ -approximation. Consider a complete bipartite graph $K_{2n,2n}$ with $2n$ vertices in each part. If L and R are the parts of a set S where $|S \cap L| = n = |S \cap R|$ is a local optimum with $|\delta(S)| = |E|/2$. The optimum solution for this instance is $|E|$.

Max Directed Cut: A problem related to MAX CUT is MAX DIRECTED CUT in which we are given a directed edge-weighted graph $G = (V, E)$ and the goal is to find a set $S \subseteq V$ that maximizes $w(\delta_G^+(S))$; that is, the weight of the directed edges leaving S . One can apply a similar local search as the one for MAX CUT. However, the following example shows that the output S can be arbitrarily bad. Let $G = (V, E)$ be a directed in-star with center v and arcs connecting each of $v_1, \dots, v_n$ to v . Then $S = \{v\}$ is a local optimum with $\delta^+(S) = \emptyset$ while $\text{OPT} = n$. However, a minor tweak to the algorithm gives a $1/3$ -approximation! Instead of returning the local optimum S return the better of S and $V \setminus S$. This step is needed because the directed cuts are not symmetric.

8.2 Local Search for Submodular Function Maximization

In this section we consider the utility of local search for maximizing non-negative submodular functions. Let $f : 2^V \rightarrow \mathbb{R}_+$ be a *non-negative* submodular set function on a ground set V . Recall that f is submodular if $f(A) + f(B) \geq f(A \cup B) + f(A \cap B)$ for all $A, B \subseteq V$. Equivalently f is submodular if $f(A + v) - f(A) \geq f(B + v) - f(B)$ for all $A \subset B$ and $v \notin B$. f is *monotone* if $f(A) \leq f(B)$ for all $A \subseteq B$. f is *symmetric* if $f(A) = f(V \setminus A)$ for all $A \subseteq V$. Submodular functions arise in a number of settings in combinatorial optimization. Two important examples are the following.

Example 8.1. Coverage in set systems. Let $S_1, S_2, \dots, S_n$ be subsets of a set $\mathcal{U}$. Let $V = \{1, 2, \dots, n\}$ and define $f : 2^V \rightarrow \mathbb{R}_+$ where $f(A) = |\cup_{i \in A} S_i|$. f is a monotone submodular function. One can also associate weights to elements of $\mathcal{U}$ via a function $w : \mathcal{U} \rightarrow \mathbb{R}_+$; the function f defined as $f(A) = w(\cup_{i \in A} S_i)$ is also monotone submodular.

Example 8.2. Cut functions in graphs. Let $G = (V, E)$ be an undirected graph with non-negative edge weights $w : E \rightarrow \mathbb{R}_+$. The cut function $f : 2^V \rightarrow \mathbb{R}_+$ defined as $f(S) = \sum_{e \in \delta_G(S)} w(e)$ is a symmetric submodular function; it is not monotone unless the graph is trivial. If G is directed and we define f as $f(S) = \sum_{e \in \delta_G^+(S)} w(e)$ then f is submodular but is not necessarily symmetric.

The following problem generalizes MAX CUT and MAX DIRECTED CUT that we have already seen.

Problem 8.2. MAX SUBMOD FUNC. *Given a non-negative submodular set function f on a ground set V via a value oracle¹ find $\max_{S \subseteq V} f(S)$.*

¹A value oracle for a set function $f : 2^V \rightarrow \mathbb{R}$ provides access to the function by giving the value $f(A)$ when presented with the set A .

Note that if f is monotone then the problem is trivial since V is the optimum solution. Therefore, the problem is interesting (and NP-Hard) only when f is not necessarily monotone. We consider a simple local search algorithm for MAX SUBMOD FUNC and show that it gives a $1/3$ -approximation and a $1/2$ -approximation when f is symmetric. This was shown in [106].

Remark 8.4. Given a graph $G = (V, E)$ consider the submodular function $f : 2^V \rightarrow \mathbb{R}$ where $f(S) = |\delta(S)| - B$ where B is a fixed number. Is there a polynomial time algorithm to decide whether there is a set S such that $f(S) \geq 0$?

LOCALSEARCH FOR MAX SUBMOD FUNC:
 $S \leftarrow \emptyset$
 repeat
 If $(\exists v \in V \setminus S \text{ such that } f(S + v) > f(S))$
 $S \leftarrow S + v$
 Else If $(\exists v \in S \text{ such that } f(S - v) > f(S))$
 $S \leftarrow S - v$
 Else
 S is a *local optimum*
 return the better of S and $V \setminus S$
 EndIf
 Until (True)

We start the analysis of the algorithm with a basic lemma on submodularity.

Lemma 8.4. Let $f : 2^V \rightarrow \mathbb{R}_+$ be a submodular set function on V . Let $A \subset B \subseteq V$. Then

- If $f(B) > f(A)$ then there is an element $v \in B \setminus A$ such that $f(A + v) - f(A) > 0$.
 More generally there is an element $v \in B \setminus A$ such that $f(A + v) - f(A) \geq \frac{1}{|B \setminus A|}(f(B) - f(A))$.
- If $f(A) > f(B)$ then there is an element $v \in B \setminus A$ such that $f(B - v) - f(B) > 0$.
 More generally there is an element $v \in B \setminus A$ such that $f(B - v) - f(B) \geq \frac{1}{|B \setminus A|}(f(A) - f(B))$.

Exercise 8.1. Prove the preceding lemma.

We obtain the following corollary.

Corollary 8.3. Let S be a local optimum for the local search algorithm and let S^* be an optimum solution. Then $f(S) \geq f(S \cap S^*)$ and $f(S) \geq f(S \cup S^*)$.

Theorem 8.4. The local search algorithm is a $1/3$ -approximation and is a $1/2$ -approximation if f is symmetric.

Proof. Let S be the local optimum and S^* be a global optimum for the given instance. From the previous corollary we have that $f(S) \geq f(S \cap S^*)$ and $f(S) \geq f(S \cup S^*)$. Note that the algorithm outputs the better of S and $V \setminus S$. By submodularity, we have,

$$f(V \setminus S) + f(S \cup S^*) \geq f(S^* \setminus S) + f(V) \geq f(S^* \setminus S)$$

where we used the non-negativity of f in the second inequality. Putting together the inequalities,

$$\begin{aligned} 2f(S) + f(V \setminus S) &= f(S) + f(S) + f(V \setminus S) \\ &\geq f(S \cap S^*) + f(S^* \setminus S) \\ &\geq f(S^*) + f(\emptyset) \\ &\geq f(S^*) = \text{OPT}. \end{aligned}$$

Thus $2f(S) + f(V \setminus S) \geq \text{OPT}$ and hence $\max\{f(S), f(V \setminus S)\} \geq \text{OPT}/3$.

If f is symmetric we argue as follows. Using Lemma 8.4 we claim that $f(S) \geq f(S \cap S^*)$ as before but also that $f(S) \geq f(S \cup \bar{S}^*)$ where $\bar{A}$ is shorthand notation for the complement $V \setminus A$. Since f is symmetric $f(S \cup \bar{S}^*) = f(V \setminus (S \cup \bar{S}^*)) = f(\bar{S} \cap S^*) = f(S^* \setminus S)$. Thus,

$$\begin{aligned} 2f(S) &\geq f(S \cap S^*) + f(S \cup \bar{S}^*) \\ &\geq f(S \cap S^*) + f(S^* \setminus S) \\ &\geq f(S^*) + f(\emptyset) \\ &\geq f(S^*) = \text{OPT}. \end{aligned}$$

Therefore $f(S) \geq \text{OPT}/2$. ■

The running time of the local search algorithm may not be polynomial but one can modify the algorithm as we did for MAX CUT to obtain a strongly polynomial time algorithm that gives a $(1/3 - o(1))$ -approximation $((1/2 - o(1))$ for symmetric). See [106] for more details. There has been much work on submodular function maximization including work on variants with additional constraints. Local search has been a powerful tool for these problems. See [47, 110, 188] for some of the results on local search based methods, and [48] for a survey on submodular function maximization.

Chapter 9

Clustering and Facility Location

Clustering and Facility Location are two widely studied topics with a vast literature. Facility location problems have been well-studied in Operations Research and logistics. Clustering is ubiquitous with many applications in data analysis and machine learning. We confine attention to a few central problems and provide some pointers as needed to other topics. These problems have also played an important role in approximation algorithms and their study has led to a variety of interesting techniques. Research on these topics is still quite active.

For both classes of problems a key assumption that we will make is that we are working with points in some underlying metric space. Recall that a space (V, d) where $d : V \times V \rightarrow \mathbb{R}_+$ is a metric space if the distance function d satisfies metric properties: (i) $d(u, v) = 0$ iff $u = v$ (reflexivity) (ii) $d(u, v) = d(v, u)$ for all $u, v \in V$ (symmetry) and (iii) $d(u, v) + d(v, w) \geq d(u, w)$ for all $u, v, w \in V$ (triangle inequality). We will abuse the notation and use $d(A, B)$ for two sets $A, B \subseteq V$ to denote the quantity $\min_{p \in A, q \in B} d(p, q)$. Similarly $d(p, A)$ for $p \in V$ and $A \subseteq V$ will denote $\min_{q \in A} d(p, q)$.

Center based clustering: In center based clustering we are given n points $P = \{p_1, p_2, \dots, p_n\}$ in a metric space (V, d) , and an integer k . The goal is to cluster/partition P into k clusters $C_1, C_2, \dots, C_k$ which are induced by choosing k centers $c_1, c_2, \dots, c_k$ from V . Each point p_i is assigned to its nearest center from $c_1, c_2, \dots, c_k$ and this induces a clustering. The nature of the clustering is controlled by an objective function that measures the quality of the clusters. Typically we phrase the problem as choosing $c_1, c_2, \dots, c_k$ to minimize the clustering objective $\sum_{i=1}^n d(p_i, \{c_1, \dots, c_k\})^q$ for some q . The three most well-studied problems are special cases obtained by choosing an appropriate q .

- k -Center is the problem when $q = \infty$ which can be equivalently phrased as $\min_{c_1, c_2, \dots, c_k \in V} \max_{i=1}^n d(p_i, \{c_1, \dots, c_k\})$. In other words we want to minimize the maximum distance of the input points to the cluster centers.

- k -Median is the problem when $q = 1$. $\min_{c_1, c_2, \dots, c_k \in V} \sum_{i=1}^n d(p_i, \{c_1, \dots, c_k\})$.
- k -Means is the problem when $q = 2$. $\min_{c_1, c_2, \dots, c_k \in V} \sum_{i=1}^n d(p_i, \{c_1, \dots, c_k\})^2$.

We will mainly focus on the discrete versions of the problems where $V = \{p_1, p_2, \dots, p_n\}$ which means that the centers are to be chosen from the input points themselves. However, in many data analysis applications the points lie in $\mathbb{R}^d$ for some d and the centers can be chosen anywhere in the ambient space. In fact this makes the problems more difficult in a certain sense since the center locations now come from an infinite set. One can argue that limiting centers to the input points does not lose more than a constant factor in the approximation and this may be reasonable from a first-cut point of view but perhaps not ideal from a practical point of view. In some settings there may be a requirement or advantage in choosing the cluster centers from the input set.

Facility Location: In facility location we typically have two finite sets $\mathcal{F}$ and $\mathcal{D}$ where $\mathcal{F}$ represents a set of locations where *facilities* can be located and $\mathcal{D}$ is a set of *client/demand* locations. We will assume that $V = \mathcal{F} \cup \mathcal{D}$ and that there is a metric d over V . There are several variants but one of the simplest one is the UNCAPACITATED FACILITY LOCATION (UCFL) problem. In UCFL we are given $(\mathcal{F} \cup \mathcal{D}, d)$ as well as auxiliary information which specifies the cost f_i of opening a facility at location $i \in \mathcal{F}$. The goal is to open a subset of facilities in $\mathcal{F}$ to minimize the sum of the cost of the opened facilities and the total distance traveled by the clients to their nearest open facility. In other words we want to solve $\min_{\mathcal{F}' \subseteq \mathcal{F}} (\sum_{i \in \mathcal{F}'} f_i + \sum_{j \in \mathcal{D}} d(j, \mathcal{F}'))$. The problem has close connections to k -Median problem. The term “uncapacitated” refers to the fact that we do not limit the number of clients that can be assigned to an open facility. In several OR applications that motivate facility location (opening warehouses or distribution facilities), capacity constraints are likely to be important. For these reasons there are several capacitated versions.

9.1 k -Center

Recall that in k -Center we are given n points $p_1, \dots, p_n$ in a metric space and an integer k and we need to choose k cluster centers $C = \{c_1, c_2, \dots, c_k\}$ such that we minimize $\max_i d(p_i, C)$. An alternative view is that we wish to find the smallest radius R such that there are k balls of radius R that together cover all the input points. Given a fixed R this can be seen as a SET COVER problem. In fact there is an easy reduction from DOMINATING SET to k -Center establishing the NP-Hardness. Moreover, as we saw already in Chapter 1, k -Center has no $2 - \epsilon$ -approximation unless $P = NP$ via a reduction from DOMINATING SET. Here we will see two 2-approximation algorithms that are quite different and have

their own advantages. The key lemma for their analysis is common and is stated below.

Lemma 9.1. *Suppose there are $k + 1$ points $q_1, q_2, \dots, q_{k+1} \in P$ such that $d(q_i, q_j) > 2R$ for all $i \neq j$. Then $\text{OPT} > R$.*

Proof. Suppose $\text{OPT} \leq R$. Then there are k centers $C = \{c_1, c_2, \dots, c_k\}$ which induces k clusters $C_1, \dots, C_k$ such that for each cluster C_h and each $p \in C_h$ we have $d(p, c_h) \leq R$. By the pigeon hole principle some $q_i, q_j, i \neq j$ are in the same cluster C_h but this implies that $d(q_i, q_j) \leq d(q_i, c_h) + d(q_j, c_h) \leq 2R$ which contradicts the assumption of the lemma. ■

Note that the lemma holds even if the centers can be chosen from outside the given point set P .

9.1.1 Gonzalez's algorithm and nets in metric spaces

The algorithm starts with an empty set of centers, and in each iteration picks a new center which is the *farthest* point from the set of centers chosen so far.

GONZALEZ- k -CENTER(P, k)

1. $C \leftarrow \emptyset$
2. For $i = 1$ to k do
 - A. Let $c_i = \arg \max_{c \in P} d(c, C)$
 - B. $C \leftarrow C \cup \{c_i\}$.
3. Output C

Note that c_1 is chosen arbitrarily.

Theorem 9.1. *Let $R = \max_{p \in P} d(p, C)$ where C is the set of centers chosen by Gonzalez's algorithm. Then $R \leq 2R^*$ where R^* is the optimum k -Center radius for P .*

Proof. Suppose not. There is a point $p \in P$ such that $d(p, C) > 2R^*$ which implies that $p \notin C$. Since the algorithm chose the farthest point in each iteration and could have chosen p in each of the k iteration but did not, we have the property that $d(c_i, \{c_1, \dots, c_{i-1}\}) > 2R^*$ for $i = 2$ to k . This implies that the distance between each pair of points in the set $\{c_1, c_2, \dots, c_k, p\}$ is more than $2R^*$. By Lemma 9.1, the optimum radius must be larger than R^* , a contradiction. ■

Exercise 9.1. Construct an instance to demonstrate that the algorithm's worst-case performance is 2.

Remark 9.1. Gonzalez's algorithm can be extended in a simple way to create a permutation of the points P ; we simply run the algorithm with $k = n$. It is easy to see from the proof above that for any $k \in [n]$, the prefix of the permutation consisting of the first k points provides a 2-approximation for that choice of k . Thus, one can compute the permutation once and reuse it for all k .

9.1.2 Hochbaum-Shmoys bottleneck approach

A second algorithmic approach for k -Center is due to Hochbaum and Shmoys and has played an influential role in variants of this problem.

For a point v and radius r let $B(v, r) = \{u \mid d(u, v) \leq r\}$ denote the ball of radius r around v .

HS- k -CENTER(P, k)

1. Guess R^* the optimum radius
2. $C \leftarrow \emptyset$, $S \leftarrow P$
3. While ($S \neq \emptyset$) do
 - A. Let c be an arbitrary point in S
 - B. $C \leftarrow C \cup \{c\}$
 - C. $S \leftarrow S \setminus B(c, 2R^*)$
4. Output C

Theorem 9.2. Let C be the output of the HS algorithm for a guess R . Then for all $p \in P$, $d(p, C) \leq 2R$ and moreover if $R \geq R^*$ then $|C| \leq k$.

Proof. The first property is easy to see since we only remove a point p from S if we add a center c to C such that $p \in B(c, 2R)$. Let $c_1, c_2, \dots, c_h$ be the centers chosen by the algorithm. We observe that $d(c_i, \{c_1, \dots, c_{i-1}\}) > 2R$. Thus, if the algorithm outputs h points then the pairwise distance between any two of them is more than $2R$. By Lemma 9.1, if $h \geq k + 1$ the optimum radius is $> R$. Hence, if the guess $R \geq R^*$ the algorithm outputs at most k centers. ■

The guessing of R^* can be implemented by binary search in various ways. We omit these routine details.

Exercise 9.2. Describe an example where the algorithm uses exactly k centers even with guess R^* . Describe an example where the algorithm outputs less than k centers with a guess of R^* .

9.1.3 Related Problems and Discussion

The k -Center problem is natural in geometric settings and one can see from the proof that the 2-approximation for the two algorithms holds even when allowing for centers to be chosen outside. A surprising result of Feder and Greene [102] shows that even in two dimensions (the Euclidean plane) one cannot improve the factor of 2 unless $P = NP$.

The k -Supplier problem is closely related to k -Center and is motivated by facility location considerations. Here we are given $\mathcal{F} \cup \mathcal{D}$ which are in a metric space. We need to choose k centers $C \subseteq \mathcal{F}$ to minimize $\max_{p \in \mathcal{D}} d(p, C)$. Note that we don't have to cover the facilities. Hochbaum and Shmoys gave a variant of their algorithm that obtains a 3-approximation for k -Supplier and moreover showed that unless $P = NP$ one cannot improve 3 [149]. Interestingly in Euclidean spaces 3 is not tight [211]. Several generalizations of k -Center which constrain the choice of centers have been considered — see [57] for a general framework that also considers outliers.

One can consider weighted version of k -Center or relabeled as priority version in subsequent work. We refer to the work of Plesnik [218] and a recent one [30] on this variant which has found applications in fair clustering.

We finally mention that k -Center clustering has nice connections to the notion of r -nets in metric spaces. Given a set of points P in a metric space and a radius r , an r -net is a set of centers C such that (i) for all $p \in P$ we have $d(p, C) \leq r$ (that is the points are covered by balls of radius r) and (ii) for any distinct $c, c' \in C$ we have $B(c, r/2)$ and $B(c', r/2)$ are disjoint (packing property or the property that no two centers are too close). r -nets provide a concise summary of distances in a metric space at scale r . One can use r -nets to obtain nearest-neighbor data structures and other applications, especially in low-dimensional settings. We refer the reader to [141, 142].

LP Relaxation: The two k -Center algorithms we described are combinatorial. One can also consider an LP relaxation. Since it is a bottleneck problem, writing an LP relaxation involves issues similar to what we saw with unrelated machine scheduling. Given a guess R we can write an LP to check whether a radius R is feasible and then find the smallest R for which it is feasible. The feasibility LP can be written as follows. Let x_i be an indicator for whether we open a center at point p_i .

$$\begin{aligned}
\sum_{i=1}^n x_i &= k \\
\sum_{p_i \in B(p_j, R)} x_i &\geq 1 \quad \forall p_j \in P \\
x_i &\geq 0 \quad \forall p_i \in P
\end{aligned}$$

Exercise 9.3. Prove that if R is feasible for the preceding LP then one can obtain a solution with k centers with max radius $2R$.

Exercise 9.4. Generalize the LP for the k -Supplier problem and prove that one can obtain a 3-approximation with respect to lower bound provided via the LP approach.

9.2 Uncapacitated Facility Location

We now discuss UCFL. One can obtain a constant factor for UCFL via several techniques: LP rounding, primal-dual, local search and greedy. The best known approximation bound is 1.488 due to Li [193] while it is known that one cannot obtain a ratio better than 1.463 [131]. We will describe the complicated algorithms and focus on the high-level approaches that yield some constant factor approximation.

It is common to use i to denote a facility in $\mathcal{F}$ and j to denote a demand/client.

9.2.1 LP Rounding

The first constant factor approximation for UCFL was via LP rounding by Aardal, Shmoys and Tardos using a filtering technique of Lin and Vitter. We start with the LP relaxation. We use a variable y_i for $i \in \mathcal{F}$ to indicate whether i is opened or not. We use a variable $x_{i,j}$ to indicate whether j is connected to i (or assigned to i). One set of constraints are natural here: each client has to be assigned/connected to a facility. The other constraint requires that j is assigned to i only if i is open.

$$\begin{aligned}
\min \quad & \sum_{i \in \mathcal{F}} f_i y_i + \sum_{j \in \mathcal{D}} \sum_{i \in \mathcal{F}} d(i, j) x_{i,j} \\
& \sum_i x_{i,j} = 1 \quad \forall j \in \mathcal{D} \\
& x_{i,j} \leq y_i \quad i \in \mathcal{F}, j \in \mathcal{D} \\
& x, y \geq 0
\end{aligned}$$

Given a feasible solution x, y to the LP the question is how to round. We note that the LP relaxation does not “know” whether d is a metric or not. In fact when d is arbitrary (but non-negative) we obtain the non-metric facility location problem which is as hard as the SET COVER problem but not much harder — one can obtain an $O(\log n)$ -approximation. However, we can obtain a constant factor when d is a metric and the rounding will exploit this property.

Given the fractional solution x, y for each j we define α_j to be the distance cost paid for by the LP: therefore $\alpha_j = \sum_{i \in \mathcal{F}} d(i, j)x_{i,j}$. Note that the LP cost is $\sum_i f_i y_i + \sum_j \alpha_j$.

Lemma 9.2. *For each j and each $\delta \in (0, 1)$ there is a total facility value of at least $(1 - \delta)$ in $B(j, \alpha_j/\delta)$. That is, $\sum_{i \in B(j, \alpha_j/\delta)} y_i \geq 1 - \delta$. In particular $\sum_{i \in B(j, 2\alpha_j)} y_i \geq 1/2$.*

Proof. This essentially follows from Markov’s inequality or averaging. Note that $\alpha_j = \sum_i d(i, j)x_{i,j}$ and $\sum_i x_{i,j} = 1$. Suppose $\sum_{i \in B(j, \alpha_j/\delta)} y_i < 1 - \delta$. Since $x_{i,j} \leq y_i$ for all i, j , we will have $\alpha_j > \delta \alpha_j / \delta$ which is impossible. ■

We say that two clients j and j' intersect if there is some $i \in \mathcal{F}$ such that $i \in B(j, 2\alpha_j) \cap B(j', 2\alpha_{j'})$. The rounding algorithm is described below.

UCFL-PRIMAL-ROUNDING

1. Solve LP and let (x, y) be a feasible LP solution
2. For each j compute $\alpha_j = \sum_i d(i, j)x_{i,j}$
3. Renumber clients such that $\alpha_1 \leq \alpha_2 \leq \dots \leq \alpha_h$ where h is number of clients
4. For $j = 1$ to h do
 - A. If j already assigned continue
 - B. Open cheapest facility i in $B(j, 2\alpha_j)$ and assign j to i
 - C. For each remaining client $j' > j$ that intersects with j , assign j' to i
5. Output the list of open facilities and the client assignment

It is not hard to see that every client is assigned to an open facility. The main issue is to bound the total cost. Let F be the total facility opening cost, and let C be the total connection cost. We will bound these separately.

Lemma 9.3. $F \leq 2 \sum_i f_i y_i$.

Proof. Note that a client j opens a new facility only if it has not been assigned when it is considered by the algorithm. Let $j_1, j_2, \dots, j_k$ be the clients that open

facilities. Let $A_j = \mathcal{F} \cap B(j, 2\alpha_j)$ be the set of facilities in the ball of radius $2\alpha_j$ around j . From the algorithm and the definition of intersection of clients, we see that the sets $A_{j_1}, A_{j_2}, \dots, A_{j_k}$ are pairwise disjoint. The algorithm opens the cheapest facility in A_{j_ℓ} for $1 \leq \ell \leq k$. Note that $\sum_{i \in A_{j_\ell}} y_i \geq 1/2$ by Lemma 9.2. Hence the cost of the cheapest facility in A_{j_ℓ} is at most $2 \sum_{i \in A_{j_\ell}} f_i y_i$ (why?). By the disjointness of the sets $A_{j_1}, \dots, A_{j_k}$ we see that the total cost of the facilities opened is at most $2 \sum_i f_i y_i$. ■

Lemma 9.4. $C \leq 6 \sum_j \alpha_j$.

Proof. Consider a client j that opens a facility i in $B(j, 2\alpha_j)$. In this case j is assigned to i and $d(i, j) \leq 2\alpha_j$. Now consider a client j that does not open a facility. This implies that there was a client $j' < j$ that opened a facility i' and j' and j intersect, and j was assigned to i' . What is $d(i', j)$? We claim that $d(i', j) \leq 6\alpha_j$. To see this we note that j' and j intersect because there is some facility $i \in B(j, 2\alpha_{j'}) \cap B(j, 2\alpha_j)$. By considering the path $j \rightarrow i \rightarrow j' \rightarrow i'$, via triangle inequality, $d(i', j) \leq 2\alpha_j + 2\alpha_{j'} + 2\alpha_{j'} \leq 6\alpha_j$ since $\alpha_{j'} \leq \alpha_j$. Thus the distance traveled by each client j to its assigned facility is at most $6\alpha_j$. The lemma follows. ■

The two preceding lemmas give us the following which implies that the algorithm yields a 6-approximation.

Theorem 9.3. $C + F \leq 6 \text{OPT}_{LP}$.

It should be clear to the reader that the algorithm and analysis are not optimized for the approximation ratio. The goal here was to simply outline the basic scheme that led to the first constant factor approximation.

9.2.2 Primal-Dual

Jain and Vazirani developed an elegant and influential primal-dual algorithm for UCFL [156]. It was influential since it allowed new algorithms for k -median and several generalizations of UCFL in a clean way. Moreover the primal-dual algorithm is simple and efficient to implement. On the other hand we should mention that one advantage of having an LP solution is that it gives an explicit lower bound on the optimum value while a primal-dual method yields a lower bound via a feasible dual which may not be optimal. We need some background to describe the primal-dual method in approximation.

Complementary slackness: To understand primal-dual we need some basic background in complementary slackness. Suppose we have a primal LP (P) of the form $\min cx$ s.t $Ax \geq b, x \geq 0$ which we intentionally wrote in this standard

form as a covering LP. Its dual (D) is a packing LP $\max b^t y$ s.t. $yA^t \leq c, y \geq 0$. We will assume that both primal and dual are feasible and hence the optimum values are finite, and via strong duality we know that the optimum values are the same.

Definition 9.4. A feasible solution x to (P) and a feasible solution y to (D) satisfy the primal complementary slackness condition with respect to each other if the following is true: for each i , $x_i = 0$ or the corresponding dual constraint is tight, that is $\sum_j A_{i,j} y_j = c_i$. They satisfy the dual complementary slackness condition if the following is true: for each j , $y_j = 0$ or the corresponding primal constraint is tight, that is $\sum_i A_{j,i} x_i = b_j$.

One of the consequences of the duality theory of LP is the following.

Theorem 9.5. Suppose (P) and (D) are a primal and dual pair of LPs that both have finite optima. A feasible solution x to (P) and a feasible solution y to (D) satisfy the primal-dual complementary slackness properties with respect to each other if and only if they are both optimum solutions to the respective LPs.

We illustrate the use of complementary slackness in the context of approximation via VERTEX COVER. Recall the primal covering LP and we also write the dual.

$$\begin{aligned} \min \sum_{v \in V} w_v x_v \\ x_u + x_v &\geq 1 & \forall e = (u, v) \in E \\ x_v &\geq 0 & \forall v \in V \end{aligned}$$

$$\begin{aligned} \max \sum_{e \in E} y_e \\ \sum_{e \in \delta(v)} y_e &\leq w_v & \forall v \in V \\ y_e &\geq 0, & \forall e \in E \end{aligned}$$

Recall that we described a simple rounding algorithm. Given a feasible primal solution x . We let $S = \{v \mid x_v \geq 1/2\}$ and showed that (i) S is a vertex cover for the given graph G and (ii) $w(S) \leq 2 \sum_v w_v x_v$. Now suppose we have an optimum solution x^* to the primal rather than an arbitrary feasible solution. We can prove an interesting and stronger statement via complementary slackness.

Lemma 9.5. *Let x^* be an optimum solution to the primal covering LP. Then $S = \{v \mid x_v^* > 0\}$ is a feasible vertex cover for G and moreover $w(S) \leq 2 \sum_v w_v x_v^*$.*

Proof. It is easy to see that S is a vertex cover via the same argument that we have seen before. How do we bound the cost now that we may be rounding x_v^* to 1 even though x_v^* may be tiny? Let y^* be any optimum solution to the dual; one exists (why?). Via strong duality we have $\sum_v w_v x_v^* = \sum_e y_e^*$. Via primal-complementary slackness we have the property that if $x_v^* > 0$ then $\sum_{e \in \delta(v)} y_e^* = w_v$. Hence

$$w(S) = \sum_{v: x_v^* > 0} w_v = \sum_{v: x_v^* > 0} \sum_{e \in \delta(v)} y_e^*.$$

Interchanging the order of summation we obtain that

$$w(S) = \sum_{v: x_v^* > 0} \sum_{e \in \delta(v)} y_e^* \leq 2 \sum_{e \in E} y_e^* = 2 \sum_v w_v x_v^*$$

where we use the fact that an edge e has only two end points in the inequality. ■

Primal-dual for approximating Vertex Cover: We will first study a primal-dual approximation algorithm for the VERTEX COVER problem — this algorithm due to Bar-Yehuda and Even [35] can perhaps be considered the first primal-dual algorithm in approximation. Primal-dual is a classical method in optimization and is applicable in both continuous and discrete settings. The basic idea, in the context of LPs (the method applies more generally), is to obtain a solution x to a primal LP and a solution y to the dual LP *together* and certify the optimality of each solution via complementary slackness. It is beyond the scope of this notes to give a proper treatment. One could argue that understanding the setting of approximation is easier than in the classical setting where one seeks exact algorithms, since our goals are more modest.

Typically one starts with one of x, y being feasible and the other infeasible, and evolve them over time. In discrete optimization, this method is successfully applied to LPs that are known to be integer polytopes. Examples include shortest paths, matchings, and others. In approximation the LP relaxations are typically not integral. In such a setting the goal is to produce a primal and dual pair x, y where x is an integral feasible solution, and y is fractional feasible solution. The goal is to approximately bound the value of x via the dual y , and for this purpose we will enforce only the primal complementary slackness condition for the pair (x, y) . To make the algorithm and analysis manageable, the primal-dual algorithm is typically done in a simple but clever fashion — there have been several surprisingly strong and powerful approximation results via this approach.

We illustrate this in the context of VERTEX COVER first. It is useful to *interpret* the dual as we did already in the context of the dual-fitting technique for SET COVER. We think of the edges $e \in E$ as agents that wish to be covered by the vertices in G at minimum cost. The dual variable y_e can be thought of as the amount that edge e is willing to pay to be covered. The dual packing constraint $\sum_{e \in \delta(v)} y_e \leq w_v$ says that for any vertex v , the total payments of all edges incident to v cannot exceed its weight. This can be understood game theoretically as follows. The set of edges $\delta(v)$ can together pay w_v and get covered, and hence we cannot charge them more. The dual objective is to maximize the total payment that can be extracted from the edges subject to these natural and simple constraints. With this interpretation in mind we wish to produce a feasible dual (payments) and a corresponding feasible integral primal (vertex cover). The basic scheme is to start with an infeasible primal $x = 0$ and a feasible dual $y = 0$ and increase y while maintaining feasibility; during the process we will maintain primal complementary slackness which means that if a dual constraint for a vertex v becomes tight we set $x_v = 1$. Note that we are producing an integer primal solution in this process. How should we increase y values? We will do it in the naive fashion which is to *uniformly* increase y_e for all e that are not already part of a tight constraint (and hence not covered yet).

VC-PRIMAL-DUAL($G = (V, E), w : V \rightarrow \mathbb{R}_+$)

1. $x \leftarrow 0, y \leftarrow 0$ *// initialization: primal infeasible, dual feasible*
2. $U \leftarrow E$ *// uncovered edges that are active*
3. While ($U \neq \emptyset$) do
 - A. Increase y_e uniformly for each $e \in U$ until constraint $\sum_{e \in \delta(v)} y_e = w_v$ for some vertex a
 - B. Set $x_a = 1$ *// Maintain primal complementary slackness*
 - C. $U \leftarrow U \setminus \delta(a)$ *// Remove all edges covered by a*
4. Output integer solution x and dual certificate y

Remark 9.2. Note that when checking whether a vertex v is tight we count the payments from $e \in \delta(v)$ even though some of them are no longer active.

Lemma 9.6. *At the end of the algorithm x is a feasible vertex cover for G and $\sum_v w_v x_v \leq 2 \text{OPT}$.*

Proof. By induction on the iterations one can prove that (i) y remains dual

feasible throughout (ii) $ab \in U$ at the start of an iteration iff $e = ab$ is not covered yet (iii) each iteration adds at least one more vertex and hence the algorithm terminates in $\leq n$ iterations and outputs a feasible vertex cover. The main issue is the cost of x .

For this we note that the algorithm maintains primal complementary slackness. That is, $x_v = 0$ or if $x_v = 1$ then $\sum_{e \in \delta(v)} y_e = w_v$. Thus, we have

$$\sum_v w_v x_v = \sum_{v: x_v > 0} \sum_{e \in \delta(v)} y_e \leq 2 \sum_e y_e \leq 2 \text{OPT}_{LP}.$$

We used the fact that e has at most two end points in the first inequality and the fact that y is dual feasible in the second inequality. In terms of payment what this says is that edge uv 's payment of y_{uv} can be used to pay for opening u and v while the dual pays only once. ■

As the reader can see, the algorithm is very simple to implement.

Exercise 9.5. Describe an example to show that the primal-dual algorithm's worst case performance is 2. Describe an example to show that the dual value constructed by the algorithm is $\simeq \text{OPT}/2$. Are these two parts the same?

Remark 9.3. The algorithm generalizes to give an f -approximation for SET COVER where f is the maximum frequency of any element. There are examples showing that the performance of this algorithm in the worst-case can indeed be a factor of f . We saw earlier that the integrality gap of the LP is at most $1 + \ln d$ where d is the maximum set size. There is no contradiction here since the specific primal-dual algorithm that we developed need not achieve the tight integrality gap.

Remark 9.4. In the algorithm the dual variables for uncovered/active edges were raised *uniformly* at the same rate. Is this important? In the case of Vertex Cover problem this is not important. In fact one can raise the dual variables in an arbitrary fashion as long as we maintain the primal complementary slackness and the approximation guarantee of 2 will still hold. However, the uniform growth strategy is needed in some applications of primal-dual, as we will see later.

Primal-Dual for UCFL: Now we consider a primal-dual algorithm for UCFL. Recall the primal LP that we discussed previously. Now we describe the dual LP written below. The dual has two types of variables. For each client j there is a variable α_j corresponding to the primal constraint that each client j needs to be connected to a facility. For each facility i and client j there is a variable $\beta_{i,j}$ corresponding to the constraint that $y_i \geq x_{i,j}$.

$$\begin{aligned}
& \max \sum_{j \in \mathcal{D}} \alpha_j \\
& \sum_i \beta_{i,j} \leq f_i \quad \forall i \in \mathcal{F} \\
& \alpha_j - \beta_{i,j} \leq d(i, j) \quad i \in \mathcal{F}, j \in \mathcal{D} \\
& \alpha, \beta \geq 0
\end{aligned}$$

It is important to interpret the dual variables. There is a similarity to SET COVER since Non-Metric Facility Location is a generalization, and the LP formulation does not distinguish between metric and non-metric facility location — it is only in the rounding that we can take advantage of metric properties. The variable α_j can be interpreted as the amount of payment client j is willing to make. This comes in two parts — the payment to travel to a facility which it cannot share with any other clients, and the payment it is willing to make to open a facility which it can share with other clients. The variable $\beta_{i,j}$ corresponds to the amount client j is willing to pay to facility i to open it. (In SET COVER there is no need to distinguish between α_j and $\beta_{i,j}$ since there are no distances (or they can be assumed to be 0 or ∞).) The first set of constraints in the dual say that for any facility i , the total payments from all the clients ($\sum_j \beta_{i,j}$) cannot exceed cost f_i . The second set of constraints specify that $\alpha_j - \beta_{i,j}$ is at most $d(i, j)$. One way to understand this is that if $\alpha_j < d(i, j)$ then client j will not even be able to reach i and hence will not contribute to opening i . Via this interpretation it is convenient to assume that $\beta_{i,j} = \max\{0, \alpha_j - d(i, j)\}$.

The primal-dual algorithm for UCFL will have a growth stage that is similar to what we saw for VERTEX COVER. We increase α_j for each “uncovered” client j uniformly. A facility i will receive payment $\beta_{i,j} = \max\{0, \alpha_j - d(i, j)\}$ from j . To maintain dual feasibility, as soon as the constraint for facility i becomes tight we open facility i (in the primal we set $y_i = 1$); note that any client j such that $\beta_{i,j} > 0$ cannot increase its α_j any further and hence will stop growing since it is connected to an open facility. This process is very similar to that in SET COVER. The main issue is that we will get a weak approximation (a factor of $|\mathcal{F}|$) since a client can contribute payments to a large number of facilities. Note that the process so far has not taken advantage of the fact that we have a metric facility location problem. Therefore, in the second phase we will close some facilities which means that a client may need to get connected to a facility that it did not contribute to — however we will use the metric property to show that a client does not need to travel too far to reach a facility that remains open.

With the above discussion in place, we describe the two phase primal-dual algorithm below. The algorithm also creates a bipartite graph G with vertex set

$\mathcal{F} \cup \mathcal{D}$ and initially it has no edges.

JV-PRIMAL-DUAL($(\mathcal{F} \cup \mathcal{D}, d), f_i, i \in \mathcal{F}$)

1. $\alpha_j = 0$ for all $j \in \mathcal{D}$, $\beta_{i,j} \leftarrow 0$ for all i, j *// initialize dual values to 0*
2. $A \leftarrow \mathcal{D}$ *// active clients that are unconnected*
3. $O \leftarrow \emptyset$ *// temporarily opened facilities*
4. While $(A \neq \emptyset)$ do *// Growth phase*
 - A. Increase α_j uniformly for each $j \in A$ while maintaining the invariant $\max\{0, \alpha_j - d(i, j)\} = \beta_{i,j}$ until one of following conditions hold: (i) $\alpha_j = d(i, j)$ for some $j \in A$ and $i \in O$ or (ii) $\sum_j \beta_{ij} = f_i$ for some $i \in \mathcal{F} \setminus O$
 - B. If (i) happens then add edge (i, j) to G and $A \leftarrow A - \{j\}$ *// j is connected to already open facility i ∈ O*
 - C. Else If (ii) happens then
 1. $O \leftarrow O \cup \{i\}$ *// temporarily open i that became tight*
 2. for each $j \in \mathcal{D}$ such that $\beta_{i,j} > 0$ add edge (i, j) to G *// note: clients not in A may also get edges to i*
 3. $A \leftarrow A \setminus \{j : \beta(i, j) > 0\}$ *// make active clients connected to i inactive*
5. Create graph H with vertex set O and edge set Q where $(i, i') \in Q$ iff there exists client j such that $\beta(i, j) > 0$ and $\beta(i', j) > 0$
6. O' is **any maximal** independent set in H
7. Close facilities in $O \setminus O'$ *// Pruning phase*
8. Assign each client j to nearest facility in O'

Example: The example in Fig ?? illustrates the need for the pruning phase. There are $2n$ clients and n facilities and the opening costs of the facilities are $n + 2$ except for the first one which has an opening cost of $n + 1$. The first group of clients shown at the top of the figure have a connection cost of 1 to each facility. The second group of clients have the following property: $d(i_h, j'_h) = 1$ and $d(i_\ell, j'_h) = 3$ when $\ell \neq j$. The rest of the distances are induced by these. One can verify that in the growth phase all the facilities will be opened. However the total dual value after the growth phase is $5n - 1$ while the total cost of

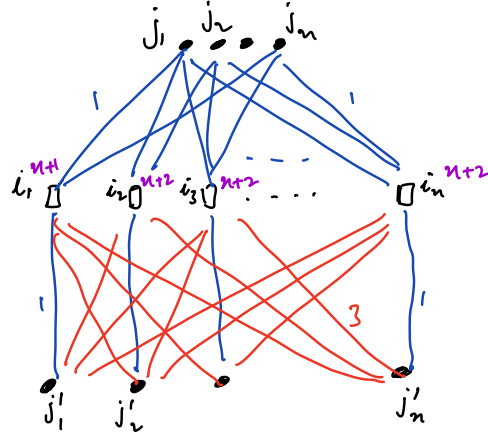


Figure 9.1: Example to illustrate the need for the pruning phase.

the opened facilities is $\Theta(n^2)$ and hence pruning is necessary to obtain a good approximation.

We will do a high-level analysis of the algorithm skipping a few fine details. A formal treatment with full details can be found in [250].

The algorithm maintains the property that α_j and $\beta_{i,j}$ variables are dual feasible. Consider the graph G created by the algorithm. We call an edge $(i, j) \in G$ a witness edge for j when j is removed from A (it happens exactly once for each j). We call an edge $(i, j) \in G$ special if $\beta_{i,j} > 0$ which means that j paid to temporarily open i . We remark that a client j may have a special edge to i even though (i, j) is not its witness edge — this can happen if i is temporarily opened after j was already removed from A (due to other clients contributing to opening i later). One can associate a notion of time with the progression of the algorithm since dual variables are monotonically increased. This can be used to order events.

A basic property maintained by the algorithm is the following.

Claim 9.2.1. *If (i, j) is an edge in G then $\alpha_j - \beta_{i,j} = d(i, j)$ and hence $\alpha_j \geq d(i, j)$.*

Proof. The algorithm adds an edge (i, j) only if (i, j) is a witness edge for j or if $\beta_{i,j} > 0$ (in which case (i, j) is special). The algorithm maintain the invariant $\beta_{i,j} = \max\{0, \alpha_j - d(i, j)\}$ and hence if $\beta_{i,j} > 0$ the claim is clear. If $\beta_{i,j} = 0$ then (i, j) is a witness edge for j and case (i) happens when j is removed from A and in this case $\alpha_j = d(i, j)$. ■

Analysis: We upper bound the cost of opening the facilities in O' and the connection cost of the clients.

We leave the proof of the following lemma as an exercise.

Lemma 9.7. *For each facility $i \in O$ we have $\sum_{(i,j) \text{ is special}} \beta_{i,j} = f_i$.*

Since a client j can pay for multiple facilities which we cannot afford, the pruning phase removes facilities such that no client j is connected two facilities in O' with special edges (otherwise those two facilities will have an edge in H). We say that a client j is directly connected to a facility $i \in O'$ if (i, j) is a special edge. We call all such clients directly connected clients and the rest of the clients are called indirectly connected. For $i \in O'$ let Z_i be the directly connected clients. We let $\mathcal{D}_1 = \cup_{i \in O'} Z_i$ be the set of all directly connected clients and let $\mathcal{D}_2$ be the set of all indirectly connected clients.

By the pruning rule we have the property that a client j is directly connected to at most one facility in O' . We show that each facility in O' can be paid for by its directly connected clients.

Lemma 9.8. *For each $i \in O'$, $\sum_{j \in Z_i} \beta_{i,j} = f_i$.*

Proof. From Lemma 9.7 and the fact that if $i \in O'$ then every client j with special edge (i, j) must be directly connected to i . ■

From Claim 9.2.1 we see that if j is directly connected to i then $\alpha_j - \beta_{i,j} = d(i, j)$, and hence j can pay its connection cost to i and its contribution to opening i . What about indirectly connected clients? The next lemma bounds their connection cost.

Lemma 9.9. *Suppose $j \in \mathcal{D}_2$, that is, it is an indirectly connected client. Let i be its closest facility in O' then $d(i, j) \leq 3\alpha_j$.*

Proof. Let (i, j) be the witness edge for j . Note that $i \in O$. Since j is an indirectly connected client there is no facility $i' \in O'$ such that (i', j) is a special edge. Since $i \notin O'$ it must be because i was closed in the pruning phase and hence there must be a facility $i' \in O'$ such that (i, i') is an edge in H (otherwise O' would not be a maximal independent set). Therefore there is some client $j' \neq j$ such that (i', j') and (i, j') are both special edges. We claim that $\alpha_j \geq \alpha_{j'}$. Assuming this claim we see via triangle inequality and Claim 9.2.1 that,

$$d(i', j) \leq d(i, j) + d(i, j') + d(i', j') \leq \alpha_j + 2\alpha_{j'} \leq 3\alpha_j.$$

Since $i' \in O'$ the nearest client to j is within distance $\leq 3\alpha_j$.

We now prove the claim that $\alpha_j \geq \alpha_{j'}$. Let $t = \alpha_j$ be the time when j connects to facility i as its witness. Consider two cases. In the first case $d(i, j') \leq t$ which

means that j' had already reached i at or before t ; in this case $\alpha_{j'} \leq t$ since i was open at t . In the second case $d(i, j') > t$; this means that j' reached i strictly after t . Since i was already open at t , j' would not pay to open i which implies that $\beta(i, j') = 0$ but then (i, j') would not be a special edge and hence this case cannot arise. This finishes the proof of the claim. ■

With the preceding two lemmas in place we can bound the total cost of opening facilities in O' and connecting clients to them. We will provide a refined statement that turns out to be useful in some applications.

Theorem 9.6.

$$\sum_{j \in \mathcal{D}} d(O', j) + 3 \sum_{i \in O'} f_i \leq 3 \sum_{j \in \mathcal{D}} \alpha_j \leq 3 \text{OPT}_{LP}.$$

In particular the algorithm yields a 3-approximation.

Proof. Consider directly connected clients $\mathcal{D}_1$. We have $\mathcal{D}_1 = \cup_{i \in O'} Z_i$ where Z_i are directly connected to i . Via Lemma 9.8 and Claim 9.2.1

$$\begin{aligned} \sum_{j \in \mathcal{D}_1} \alpha_j &= \sum_{i \in O'} \sum_{j \in Z_i} \alpha_j \\ &= \sum_{i \in O'} \sum_{j \in Z_i} (d(i, j) + \beta_{i,j}) \\ &= \sum_{i \in O'} \left(f_i + \sum_{j \in Z_i} d(i, j) \right) \\ &\geq \sum_{i \in O'} f_i + \sum_{j \in \mathcal{D}_1} d(O', j). \end{aligned}$$

For indirectly connected clients, via Lemma 9.9, we have $3 \sum_{j \in \mathcal{D}_2} \alpha_j \geq \sum_{j \in \mathcal{D}_2} d(O', j)$. Thus

$$\begin{aligned} 3 \sum_{j \in \mathcal{D}} \alpha_j &= 3 \sum_{j \in \mathcal{D}_1} \alpha_j + 3 \sum_{j \in \mathcal{D}_2} \alpha_j \\ &\geq 3 \sum_{i \in O'} f_i + 3 \sum_{j \in \mathcal{D}_1} d(O', j) + \sum_{j \in \mathcal{D}_2} d(O', j) \\ &\geq 3 \sum_{i \in O'} f_i + \sum_{j \in \mathcal{D}} d(O', j). \end{aligned}$$

■

The algorithm is easy and efficient to implement. One of the main advantages of the stronger property that we saw in the theorem is that it leads to a nice algorithm for the k -median problem; we refer the reader to Chapter 25 in [250] for a detailed description. In addition the flexibility of the primal-dual algorithm has led to algorithms for several other variants; see [39] for one such example.

9.2.3 Local Search

Local search has been shown to be a very effective heuristic algorithm for facility location and clustering problems and there is extensive literature on this topic. The first paper that proved constant factor approximation bounds for UCFL is by Korupolu, Plaxton and Rajaraman [182] and it provided a useful template for many future papers. We refer the reader to Chapter 9 in [255] for local search analysis for UCFL.

9.3 k -Median

k -Median has been extensively studied in approximation algorithms due to its simplicity and connection to UCFL. The first constant factor approximation was obtained in [65] via LP rounding. We will consider a slight generalization of k -Median where the medians are to be selected from the facility set $\mathcal{F}$. We describe the LP which is closely related to that for UCFL which we have already seen. The variables are the same: y_i indicates whether a center is opened at location $i \in \mathcal{F}$ and $x_{i,j}$ indicates whether client j is connected to facility i . The objective and constraints change since the problem requires one to open at most k facilities but there is no cost to opening them.

$$\begin{aligned}
 \min \quad & \sum_{j \in \mathcal{D}} \sum_{i \in \mathcal{F}} d(i, j) x_{i,j} \\
 \sum_i x_{i,j} &= 1 \quad \forall j \in \mathcal{D} \\
 x_{i,j} &\leq y_i \quad i \in \mathcal{F}, j \in \mathcal{D} \\
 \sum_i y_i &\leq k \\
 x, y &\geq 0
 \end{aligned}$$

Rounding of the LP for k -Median is not as simple as it is for UCFL. This is mainly because one needs a global argument. To understand this consider the following example. Suppose we have $k + 1$ points where the distance between

each pair is 1 (uniform metric space). Then the optimum integer solution has cost 1 since we can place k medians at k points and the remaining point has to travel a distance of 1. Now consider a fractional solution where $y_i = (1 - 1/(k + 1))$ for each point. The cost of this fractional solution is also 1, however, each client now pays a fractional cost of $1/(k + 1)$. Any rounding algorithm will make at least one of the clients pay a cost of 1 which is much larger than its fractional cost; thus the analysis cannot be based on preserving the cost of each client within a constant factor of its fractional cost.

There are several LP rounding algorithms known. An advantage of LP based approach is that it leads to a constant factor approximation for the MATROID MEDIAN problem which is a nice and powerful generalization of the k -Median problem; here there is a matroid defined over the facilities and the constraint is that the set of facilities chosen must be independent in the given matroid. One can write a natural LP relaxation for this problem and prove that the LP has a constant integrality gap by appealing to matroid intersection! It showcases the power of classical result in combinatorial optimization. We refer the reader to [183, 243].

9.3.1 Local Search

Local search for center based clustering problems is perhaps one of the most natural heuristics. In particular we will consider the p -swap heuristic. Given the current set of k centers S , the p -swap heuristic will consider swapping out up to p centers from S with p new centers. It is easy to see that this local search algorithm can be implemented in $n^{O(p)}$ time for each iteration. When $p = 1$ we simply refer to the algorithm as (basic) local search. We will ignore the convergence time. As we saw for MAX CUT, one can use standard tricks to make the algorithm run in polynomial time with a loss of a $(1 + o(1))$ -factor in the approximation bound guaranteed by the worst-case local optimum. Thus the main focus will be on the quality of the local optimum. The following is a very nice result.

Theorem 9.7 (Arya et al. [24]). *For any fixed $p \geq 1$ the p -swap local search heuristic has a tight worst-case approximation ratio of $(3 + 2/p)$ for k -Median. In particular the basic local search algorithm yields a 5-approximation.*

Here we give a proof/analysis of the preceding theorem for $p = 1$, following the simplified analysis presented in [135]. See also [255] and the notes from CMU. Given any set of centers S we define $\text{cost}(S) = \sum_{j \in \mathcal{D}} d(j, S)$ to be the sum of the distances of the clients to the centers. Let S be a local optimum and let S^* be some fixed optimum solution to the given k -Median instance. To compare $\text{cost}(S)$ with $\text{cost}(S^*)$ the key idea is to set up a *clever* set of potential swaps

between the centers in S and centers in S^* . That is, we consider a swap pair (r, f) where $r \in S$ and $f \in S^*$. Since S is a local optimum it must be the case that $\text{cost}(S - r + f) \leq \text{cost}(S)$. The analysis upper bounds the potential increase in the cost in some interesting fashion and sums up the resulting series of inequalities. This may seem magical, and indeed it is not obvious why the analysis proceeds in this fashion. The short answer is that the analysis ideas required a series of developments with the somewhat easier case of UCFL coming first.

We set up some notation. Let $\phi : \mathcal{D} \rightarrow S$ be the mapping of clients to facilities in S based on nearest distance. Similarly let $\phi^* : \mathcal{D} \rightarrow S^*$ be the mapping to facilities in the optimum solution S^* . Thus j connects to facility $\phi(j)$ in the local optimum and to $\phi^*(j)$ in the optimum solution. We also let $N(i)$ denote the set of all clients assigned to a facility $i \in S$ and let $N^*(i)$ denote the set of all clients assigned to a facility $i \in S^*$. Let $A_j = d(j, S)$ and $O_j = d(j, S^*)$ be the cost paid by j in local optimum and optimal solutions respectively. To reinforce the notation we express $\text{cost}(S)$ as follows.

$$\text{cost}(S) = \sum_{j \in \mathcal{D}} A_j = \sum_{i \in S} \sum_{j \in N(i)} d(j, i).$$

Similarly

$$\text{cost}(S^*) = \sum_{j \in \mathcal{D}} O_j = \sum_{i \in S^*} \sum_{j \in N^*(i)} d(j, i).$$

Setting up the swap pairs: We create a set of pairs $\mathcal{P}$ as follows. We will assume without loss of generality that $|S| = |S^*| = k$. For technical convenience we also assume $S \cap S^* = \emptyset$; we can always create dummy centers that are co-located to make this assumption. Consider the mapping $\rho : S^* \rightarrow S$ where each $i \in S^*$ is mapped to its *closest* center in S ; hence $\rho(i)$ for $i \in S^*$ is the closest center in S to it. Let R_1 be the set of centers in S that have exactly one center in S^* mapped to them. Let R_0 be the set of centers in S with no centers of S^* mapped to them. This means that for each $i \in S \setminus (R_0 \cup R_1)$ there are two or more centers mapped to them. Let $S_1^* \subseteq S^*$ be the centers mapped to R_1 . See Figure 9.2. By a simple averaging argument we have the following claim.

Claim 9.3.1. $2|R_0| \geq |S^* \setminus S_1^*|$.

We create a set of pairs $\mathcal{P}$ as follows. There will be exactly k pairs. For each $f^* \in S_1^*$ we add the pair (r, f^*) where $\rho(f^*) = r$. For each $f^* \in S^* \setminus S_1^*$ we add a pair (r, f^*) where r is any arbitrary center from R_0 — however we make sure that a center $r \in R_0$ is in at most two pairs in $\mathcal{P}$; this is possible because of Claim 9.3.1.

The pairs satisfy the following property.

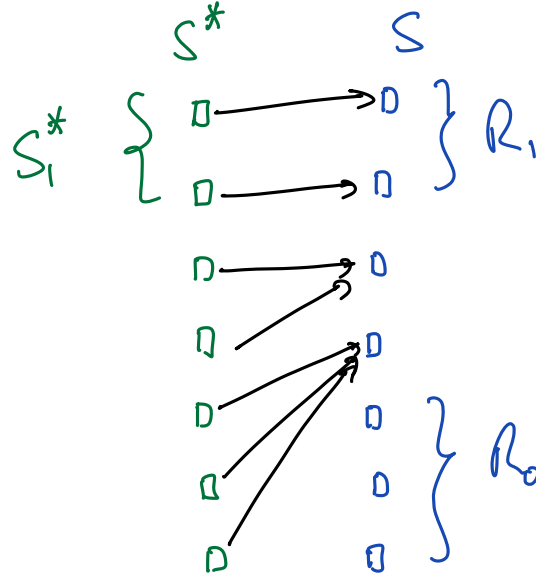


Figure 9.2: Mapping between S^* and S with each $f^* \in S^*$ mapped to its closest center in S .

Claim 9.3.2. *If $(r, f^*) \in \mathcal{P}$ then for any facility $\hat{f}^* \neq f^*$, $\rho(\hat{f}^*) \neq r$.*

The intuition for the pairs is as follows. If $\rho(f^*) = \{r\}$ then we are essentially forced to consider the pair (r, f^*) since r could be the only center near f^* with all other centers from S very far away. When considering the swap (r, f^*) we can move the clients connecting to r to f^* . On the other hand if $|\rho^{-1}(r)| > 1$ then r is close to several centers in S^* and may be serving many clients. Thus we do not consider such an r in the swap pairs.

The main technical claim about the swap pairs is the following.

Lemma 9.10. *Let (r, f^*) be a pair in $\mathcal{P}$. Then*

$$0 \leq \text{cost}(S + f^* - r) - \text{cost}(S) \leq \sum_{j \in N^*(f^*)} (O_j - A_j) + \sum_{j \in N(r)} 2O_j.$$

We defer the proof of the lemma for now and use it to show that $\text{cost}(S) \leq 5\text{cost}(S^*)$. We sum over all pairs $(r, f^*) \in \mathcal{P}$ and note that each $f^* \in S$ occurs in exactly one pair and each $r \in S$ occurs in at most two pairs. Note that $O_j - A_j$

can be a negative number while O_j is non-negative number.

$$\begin{aligned}
 0 &\leq \sum_{(r, f^*) \in \mathcal{P}} \left(\sum_{j \in N^*(f^*)} (O_j - A_j) + \sum_{j \in N(r)} 2O_j \right) \\
 &\leq \sum_{f^* \in S^*} \sum_{j \in N^*(f^*)} (O_j - A_j) + 2 \sum_{r \in S} \sum_{j \in N(r)} 2O_j \\
 &\leq \text{cost}(S^*) - \text{cost}(S) + 4\text{cost}(S^*).
 \end{aligned}$$

This implies the desired inequality.

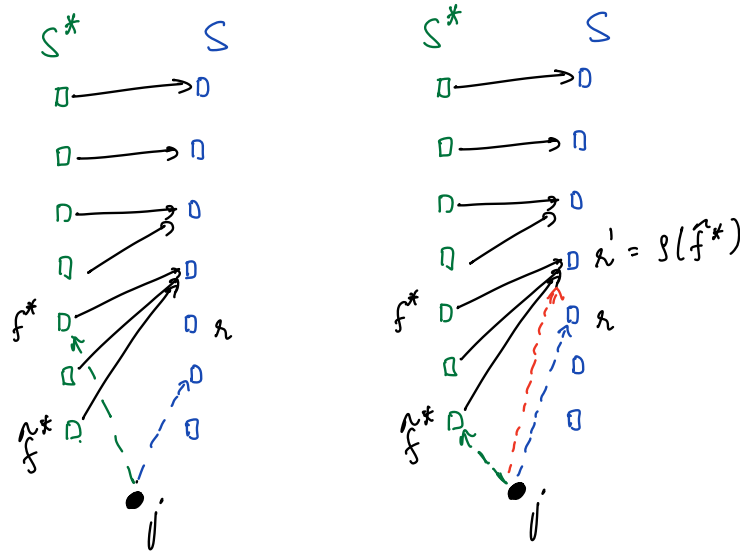


Figure 9.3: Two cases in proof of Lemma 9.10. Consider the swap pair (r, f^*) . In the figure on the left the client $j \in N^*(f^*)$ is assigned to f^* . In the figure on the right the client $j \in N(r) \setminus N^*(f^*)$ is assigned to $r' = \rho(\hat{f}^*)$.

Proof of Lemma 9.10. Since S is a local optimum swapping r with f^* cannot improve the cost and hence we obtain $\text{cost}(S + f^* - r) - \text{cost}(S) \geq 0$. We focus on the more interesting inequality. To bound the increase in cost by removing r and adding f^* to S , we consider a specific assignment of the clients. Any client $j \in N^*(f^*)$ is assigned to f^* (even if it is suboptimal to do so). See Figure 9.3. For such a client j the change in cost is $O_j - A_j$. Now consider any client $j \in N(r) \setminus N^*(f^*)$; since r is no longer available we need to assign it to another

facility. Which one? Let $\phi^*(j) = \hat{f}^*$ be the facility that j is assigned to in the optimum solution. Note that $\hat{f}^* \neq f^*$. We assign j to $r' = \rho(\hat{f}^*)$; from Claim 9.3.2, $\rho(\hat{f}^*) \neq r$ and hence $r' \in S - r + f^*$. See Figure 9.3. The change in the cost for such a client j is $d(j, r') - d(j, r)$. We bound it as follows

$$\begin{aligned}
d(j, r') - d(j, r) &\leq d(j, \hat{f}^*) + d(\hat{f}^*, r') - d(j, r) \quad (\text{via triangle inequality}) \\
&\leq d(j, \hat{f}^*) + d(\hat{f}^*, r) - d(j, r) \quad (\text{since } r' \text{ is closest to } \hat{f}^* \text{ in } S) \\
&\leq d(j, \hat{f}^*) + d(j, \hat{f}^*) \quad (\text{via triangle inequality}) \\
&= 2O_j.
\end{aligned}$$

Every other client is assigned to its existing center in S . Thus the total increase in the cost is obtained as

$$\sum_{j \in N^*(f^*)} (O_j - A_j) + \sum_{j \in N(r) \setminus N^*(f^*)} 2O_j \leq \sum_{j \in N^*(f^*)} (O_j - A_j) + \sum_{j \in N(r)} 2O_j.$$

■

See [24] for a description of the tight example. The example in the conference version of the paper is buggy.

9.4 k -Means

The k -Means problem is very popular in practice for a variety of reasons. In terms of center-based clustering the goal is to choose k centers $C = \{c_1, c_2, \dots, c_k\}$ to minimize $\sum_p d(p, C)^2$. In the discrete setting one can obtain constant factor approximation algorithms via several techniques that follow the approach of k -Median. We note that the squared distance does *not* satisfy triangle inequality, however, it satisfies a relaxed triangle inequality and this is sufficient to generalize certain LP rounding and local search techniques.

In practice the continuous version is popular for clustering applications. The input points P are in the Euclidean space $\mathbb{R}^d$ where d is typically large. Let X be the set of input points where each $x \in X$ is now a d -dimensional vector. The centers are now allowed to be in ambient space. This is called the Euclidean k -Means. Here the squared distance actually helps in a certain sense. For instance if $k = 1$ then we can see that the optimum center is simply obtained by $\frac{1}{|P|} \sum_{x \in X} x$ — in other words we take the “average”. One can see this by considering the problem of finding the center as an optimization problem: $\min_{y \in \mathbb{R}^d} \sum_{x \in X} \|x - y\|_2^2 = \min_{y \in \mathbb{R}^d} \sum_{x \in X} (x_i - y_i)^2$. It can be seen that we can optimize in each dimension separately and that the optimum in dimension

i , can be seen to be $y_i^* = \frac{1}{|X|}x_i$. Surprisingly, hardness results for Euclidean k -Means were only established in the last decade. NP-hardness even when $d = 2$ is established in [197], and APX-hardness for high dimensions was shown in [27].

9.5 Lloyd's algorithm, D^2 -sampling and k -Means ++

Lloyd's algorithm is a very well-known and widely used heuristic for the Euclidean k -Means problem. It can be viewed as a local search algorithm with an alternating optimization flavor. The algorithm starts with a set of centers $c_1, c_2, \dots, c_k$ which are typically chosen randomly in some fashion. The centers define clusters $C_1, C_2, \dots, C_k$ based on assigning each point to its nearest center. That is C_i is the set of all points in X that are closest to c_i (ties broken in some fashion). Once we have the clusters, via the observation above, one can find the best center for that cluster by taking the mean of the points in the cluster. That is, for each C_i we find a new center $c'_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$ (if C_i is empty we simply set $c'_i = c_i$). Thus we have a new set of centers and we repeat the process until convergence or some time limit. It is clear that the cost can only improve by recomputing the centers since we know the optimum center for $k = 1$ is obtained by using the average of the points.

LLOYDS- k -MEANS(X, k)

1. **Seeding:** Pick k centers $c_1, c_2, \dots, c_k$
2. repeat
 - A. Find clusters $C_1, C_2, \dots, C_k$ where
 $C_i = \{x \in X \mid c_i \text{ is closest center to } x\}$
 - B. cost = $\sum_{i=1}^k \sum_{x \in C_i} d(x, c_i)^2$.
 - C. For $i = 1$ to k do $c_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$
3. Until cost improvement is too small
4. Output clusters $C_1, C_2, \dots, C_k$

There are two issues with the algorithm. The first issue is that the algorithm can, in the worst-case, run for an exponential number of iterations. This issue is common for many local search algorithms and as we discussed, it can be

overcome by stopping when cost improvement is too small in a relative sense. The second issue is the more significant one. The algorithm can get stuck in a local optimum which can be arbitrarily bad when compared to the optimum solution. See figure below for a simple example.

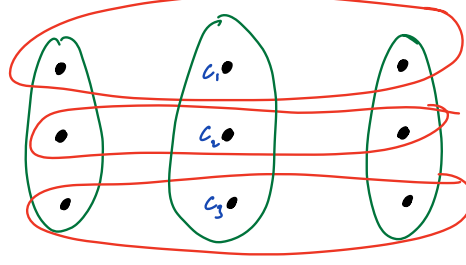


Figure 9.4: Example demonstrating that a local optimum for Lloyd's algorithm can be arbitrarily bad compared to the optimum clustering. The green clusters are the optimum ones and the red ones are the local optimum.

D^2 -sampling and k -Means ++: To overcome the bad local optima it is common to run the algorithm with random starting centers. Arthur and Vassilvitskii [249] suggested a specific random sampling scheme to initialize the centers that is closely related to independent work in [215]. This is called D^2 sampling.

D^2 -SAMPLING- k -MEANS ++(X, k)

1. $S = \{c_1\}$ where c_1 is chosen uniformly from X
2. for $i = 2$ to k do
 - A. Choose c_i randomly where $\mathbf{P}[c_i = x] \simeq d(x, S)^2$
 - B. $S \leftarrow S \cup \{c_i\}$
3. Output S

k -Means ++ is Lloyd's algorithm initialized with k centers obtained from D^2 sampling.

Theorem 9.8 ([249]). *Let S be the output of Lloyd's algorithm initialized with D^2 sampling. Then $\mathbf{E}[\text{cost}(S)] \leq 8(\ln k + 2) \text{OPT}$. Moreover there are examples showing that it is no better than $2 \ln k$ competitive.*

The analysis establishes that the seeding already creates a good approximation, so in a sense the local search is only refining the initial approximation. [7, 10] show that if one uses $O(k)$ centers, initialized according to D^2 sampling, then the local optimum will yield a constant factor approximation with constant probability; note that this is a bicriteria approximation where the number of centers is a constant factor more than k and the cost is being compared with respect to the optimum cost with k centers. The authors also show that there is a subset of k centers from the output of the algorithm that yields a constant factor approximation. One can then run a discrete optimization algorithm using the centers. Another interesting result based on D^2 -sampling ideas yields a PTAS but the running time is of the form $O(nd2^{\tilde{O}(k^2/\epsilon)})$ [157]. See [29] for a scalable version of k -Means ++.

9.6 Bibliographic Notes

Chapter 10

Introduction to Network Design

(Parts of this chapter are based on previous scribed lecture notes by Nitish Korula and Sungjin Im.)

Network Design is a broad topic that deals with finding a subgraph H of a given graph $G = (V, E)$ of minimum cost while satisfying certain requirements. G represents an existing network or a constraint over where one can build. The subgraph H is what we want to select/build. Many natural problems can be viewed this way. For instance the minimum spanning tree (MST) can be viewed as follows: given an undirected graph $G = (V, E)$ with edge costs $c : E \rightarrow \mathbb{R}_+$, find the cheapest connected spanning (spanning means that all vertices are included) subgraph of G . The fact that a minimal solution is a tree is clear, but the point is that the motivation does not explicitly mention the requirement that the output be a tree.

Connectivity problems are a large part of network design. As we already saw, MST is the most basic one and can be solved in polynomial time. The STEINER TREE problem is a generalization where we are given a subset S of terminals in an edge-weighted graph $G = (V, E)$ and the goal is to find a cheapest connected subgraph that contains all terminals. This is NP-Hard. Traveling Salesman Problem (TSP) and its variants can also be viewed as network design problems. Network design is heavily motivated by real-world problems in telecommunication networks and those problems combine aspects of connectivity and routing and in this context there are several problems related to buy-at-bulk network design, fixed-charge flow problems etc.

Graph theory plays an important role in most network algorithmic questions. The complexity and nature of the problems vary substantially based on whether the graph is *undirected* or *directed*. To illustrate this consider the DIRECTED STEINER TREE problem. Here $G = (V, E)$ is a directed graph with non-negative edge/arc weights, and we are given a root r and a set of terminals $S \subseteq V$. The goal is to

find a cheapest subgraph H of G such that r has a path to each terminal $t \in S$. Note that when $S = V$ the problem is the minimum-cost arborescence problem and is solvable in polynomial-time. One can see that DIRECTED STEINER TREE is a generalization of the (undirected) STEINER TREE problem. While STEINER TREE problem admits a constant factor approximation, it is easy to show that DIRECTED STEINER TREE is at least as hard as SET COVER. This immediately implies that it is hard to approximate to within an $\Omega(\log |S|)$ factor. In fact, substantial technical work has shown that it is in fact harder than SET COVER [140], while we only have a quasi-polynomial time algorithm that gives a poly-logarithmic approximation [62]. In even slightly more general settings, the directed graph problems get much harder when compared to their undirected graph counterparts. This phenomenon is generally true in many graph related problems and hence the literature mostly focuses on undirected graphs. We refer the reader to some surveys on network design [132, 181, 214].

This chapter will focus on two basic problems which are extensively studied and describe some simple approximation algorithms for them. Pointers are provided for sophisticated results including some very recent ones.

10.1 The Steiner Tree Problem

In the STEINER TREE problem, the input is a graph $G(V, E)$, together with a set of *terminals* $S \subseteq V$, and a cost $c(e)$ for each edge $e \in E$. The goal is to find a minimum-cost tree that connects all terminals, where the cost of a subgraph is the sum of the costs of its edges.

The STEINER TREE problem is **NP-Hard**, and also **APX-Hard** [41]. The latter means that there is a constant $\delta > 1$ such that it is **NP-Hard** to approximate the solution to within a ratio of less than δ ; it is currently known that it is hard to approximate the STEINER TREE problem to within a ratio of 95/94 [84].¹

Remark 10.1. If $|S| = 2$ (that is, there are only 2 terminals), an optimal Steiner Tree is simply a shortest path between these 2 terminals. If $S = V$ (that is, all vertices are terminals), an optimal solution is simply a minimum spanning tree of the input graph. In both these cases, the problem can be solved exactly in polynomial time.

Remark 10.2. There is a $c^k \text{poly}(n)$ -time algorithm where k is the number of terminals. Can you figure it out?

¹ Variants of the STEINER TREE problem, named after Jakob Steiner, have been studied by Fermat, Weber, and others for centuries. The front cover of the course textbook contains a reproduction of a letter from Gauss to Schumacher on a Steiner tree question.

Definition 10.1. Given a connected graph $G(V, E)$ with edge costs, the metric completion of G is a complete graph $H(V, E')$ such that for each $u, v \in V$, the cost of edge uv in H is the cost of the shortest path in G from u to v . The graph H with edge costs is a metric on V , because the edge costs satisfy the triangle inequality: $\forall u, v, w, \text{ cost}(uv) \leq \text{cost}(uw) + \text{cost}(wv)$.



Figure 10.1: On the left, a graph. On the right, its metric completion, with new edges and modified edge costs in red.

Observation 10.2. To solve the STEINER TREE problem on a graph G , it suffices to solve it on the metric completion of G .

We now look at two approximation algorithms for the STEINER TREE problem.

10.1.1 The MST Algorithm

The following algorithm, with an approximation ratio of $(2 - 2/|S|)$ is due to [244]:

STEINERMST($G(V, E), S \subseteq V$):
 Let $H(V, E') \leftarrow$ metric completion of G .
 Let $T \leftarrow$ MST of $H[S]$.
 Output T .

(Here, we use the notation $H[S]$ to denote the subgraph of H induced by the set of terminals S .)

The following lemma is central to the analysis of the algorithm STEINERMST.

Lemma 10.1. For any instance I of STEINER TREE, let H denote the metric completion of the graph, and S the set of terminals. There exists a spanning tree in $H[S]$ (the graph induced by terminals) of cost at most $2(1 - \frac{1}{|S|}) \text{OPT}$, where OPT is the cost of an optimal solution to instance I .

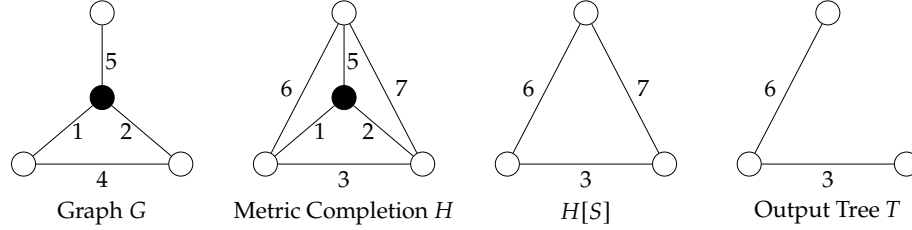
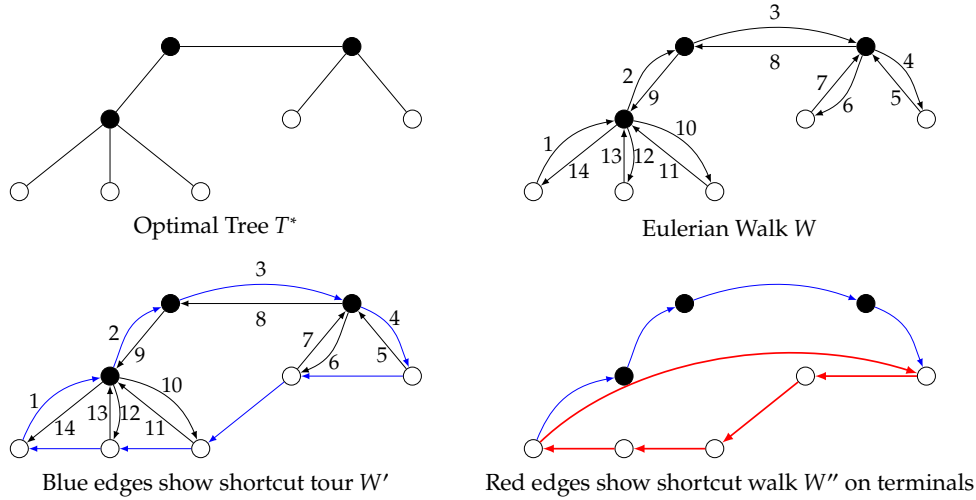


Figure 10.2: Illustrating the MST Heuristic for STEINER TREE

Before we prove the lemma, we note that if there exists *some spanning tree* in $H[S]$ of cost at most $2(1 - \frac{1}{|S|}) \text{OPT}$, the minimum spanning tree has at most this cost. Therefore, Lemma 10.1 implies that the algorithm STEINERMST is a $2(1 - \frac{1}{|S|})$ -approximation for the STEINER TREE problem.

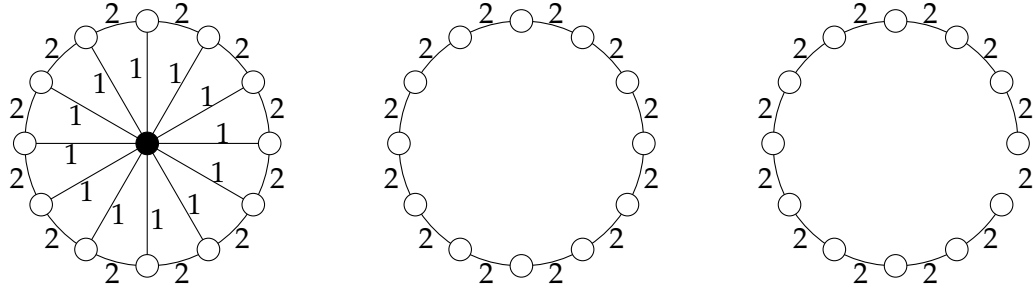
Proof. Proof of Lemma 10.1 Let T^* denote an optimal solution in H to the given instance, with cost $c(T^*)$. Double all the edges of T^* to obtain an Eulerian graph, and fix an Eulerian Tour W of this graph. See Fig 10.3. Now, shortcut edges of W to obtain a tour W' of the vertices in T^* in which each vertex is visited exactly once. Again, shortcut edges of W' to eliminate all non-terminals; this gives a walk W'' that visits each terminal exactly once.


 Figure 10.3: Doubling edges of T^* and shortcutting gives a low-cost spanning tree on terminals.

It is easy to see that $c(W'') \leq c(W') \leq c(W) = 2c(T^*)$, where the inequalities follow from the fact that by shortcutting, we can only decrease the length of the walk. (Recall that we are working in the metric completion H .) Now, delete the

heaviest edge of W'' to obtain a path through all the terminals in S , of cost at most $(1 - \frac{1}{|S|})c(W'')$. This path is a spanning tree of the terminals, and contains *only* terminals; therefore, there exists a spanning tree in $H[S]$ of cost at most $2(1 - \frac{1}{|S|})c(T^*)$. ■

A tight example: The following example (Fig. 4 below) shows that this analysis is tight; there are instances of STEINER TREE where the STEINERMST algorithm finds a tree of cost $2(1 - \frac{1}{|S|})\text{OPT}$. Here, each pair of terminals is connected by an edge of cost 2, and each terminal is connected to the central non-terminal by an edge of cost 1. The optimal tree is a star containing the central non-terminal, with edges to all the terminals; it has cost $|S|$. However, the only trees in $H[S]$ are formed by taking $|S| - 1$ edges of cost 2; they have cost $2(|S| - 1)$.



Graph G ; not all edges shown $H[S]$; not all edges shown. An MST of $H[S]$.

Figure 10.4: A tight example for the STEINERMST algorithm

10.1.2 The Greedy/Online Algorithm

We now describe another simple algorithm for the STEINER TREE problem, due to [154].

```

GREEDYSTEINER( $G(V, E), S \subseteq V$ ):
  Let  $\{s_1, s_2, \dots, s_{|S|}\}$  be an arbitrary ordering of the terminals.
  Let  $T \leftarrow \{s_1\}$ 
  For ( $i$  from 2 to  $|S|$ ):
    Let  $P_i$  be the shortest path in  $G$  from  $s_i$  to  $T$ .
    Add  $P_i$  to  $T$ .

```

GREEDYSTEINER is a $\lceil \log_2 |S| \rceil$ -approximation algorithm; here, we prove a slightly weaker result.

Theorem 10.3. *The algorithm GREEDYSTEINER has an approximation ratio of $2H_{|S|} \approx 2 \ln |S|$, where $H_i = \sum_{j=1}^i 1/j$ denotes the i 'th harmonic number.*

Note that this is an *online* algorithm; terminals are considered in an arbitrary order, and when a terminal is considered, it is immediately connected to the existing tree. Thus, even if the algorithm *could not see the entire input at once*, but instead terminals were revealed one at a time and the algorithm had to produce a Steiner tree at each stage, the algorithm GREEDYSTEINER outputs a tree of cost no more than $O(\log |S|)$ times the cost of the optimal tree.

To prove Theorem 10.3, we introduce some notation. Let $c(i)$ denote the cost of the path P_i used in the i th iteration to connect the terminal s_i to the already existing tree. Clearly, the total cost of the tree is $\sum_{i=1}^{|S|} c(i)$. Now, let $\{i_1, i_2, \dots, i_{|S|}\}$ be a permutation of $\{1, 2, \dots, |S|\}$ such that $c(i_1) \geq c(i_2) \geq \dots \geq c(i_{|S|})$. (That is, relabel the terminals in decreasing order of the cost paid to connect them to the tree that exists when they are considered by the algorithm.)

Claim 10.1.1. *For all j , the cost $c(i_j)$ is at most $2 \text{OPT}/j$, where OPT is the cost of an optimal solution to the given instance.*

Proof. Suppose by way of contradiction this were not true; since s_{i_j} is the terminal with j th highest cost of connection, there must be j terminals that each pay more than $2 \text{OPT}/j$ to connect to the tree that exists when they are considered. Let $S' = \{s_{i_1}, s_{i_2}, \dots, s_{i_j}\}$ denote this set of terminals.

We argue that no two terminals in $S' \cup \{s_1\}$ are within distance $2 \text{OPT}/j$ of each other. If some pair x, y were within this distance, one of these terminals (say y) must be considered later by the algorithm than the other. But then the cost of connecting y to the already existing tree (which includes x) must be at most $2 \text{OPT}/j$, and we have a contradiction.

Therefore, the minimum distance between any two terminals in $S' \cup \{s_1\}$ must be greater than $2 \text{OPT}/j$. Since there must be j edges in any MST of these terminals, an MST must have cost greater than 2OPT . But the MST of a subset of terminals cannot have cost more than 2OPT , exactly as argued in the proof of Lemma 10.1. Therefore, we obtain a contradiction. ■

Given this claim, it is easy to prove Theorem 10.3.

$$\sum_{i=1}^{|S|} c(i) = \sum_{j=1}^{|S|} c(i_j) \leq \sum_{j=1}^{|S|} \frac{2 \text{OPT}}{j} = 2 \text{OPT} \sum_{j=1}^{|S|} \frac{1}{j} = 2H_{|S|} \cdot \text{OPT}.$$

Question 10.2. Give an example of a graph and an ordering of terminals such that the output of the Greedy algorithm is $\Omega(\log |S|) \text{OPT}$.

Remark 10.3. We emphasize again that the analysis above holds for *every* ordering of the terminals. A natural variant might be to adaptively order the terminals so that in each iteration i , the algorithm picks the terminal s_i to be the one closest

to the already existing tree T built in the first i iterations. Do you see that this is equivalent to using the MST Heuristic with Prim's algorithm for MST? This illustrates the need to be careful in the design and analysis of heuristics.

10.1.3 LP Relaxation

A natural LP relaxation for the Steiner Tree problem is the following. For each edge $e \in E$ we have an indicator variable x_e to decide if we choose to include e in our solution. The chosen edges should ensure that no two terminals are separated. We write this via a constraint $\sum_{e \in \delta(A)} x_e \geq 1$ for any set $A \subset V$ such that A contains a terminal and $V \setminus A$ contains a terminal.

$$\begin{aligned} \min \quad & \sum_{e \in E} c_e x_e \\ \sum_{e \in \delta(A)} x_e \quad & \geq 1 \quad A \cap S \neq \emptyset, (V - A) \cap S \neq \emptyset \\ x_e \quad & \geq 0 \end{aligned}$$

Note that the preceding LP has an exponential number of constraints. However, there is a polynomial-time separation oracle. Given x it is feasible for the LP iff the s - t cut value is at least 1 between any two terminals $s, t \in S$ with edge capacities given by x . How good is this LP relaxation? We will see later that there is a $2(1 - 1/|S|)$ -approximation via this LP. Interestingly the LP has an integrality gap of $2(1 - 1/|S|)$ even if $S = V$ in which case we want to solve the MST problem! Despite the weakness of this cut based LP for these simple cases, we will see later that it generalizes nicely for higher connectivity problems and one can derive a 2-approximation even for those much more difficult problems.

10.1.4 Other Results on Steiner Trees

The 2-approximation algorithm using the MST Heuristic is not the best approximation algorithm for the STEINER TREE problem currently known. Some other results on this problem are listed below.

1. The first algorithm to obtain a ratio of better than 2 was due to Alexander Zelikovsky [260]; the approximation ratio of this algorithm was $11/6 \approx 1.83$. This was improved to $1 + \frac{\ln 3}{2} \approx 1.55$ [226] and is based on a local search based improvement starting with the MST heuristic, and follows the original approach of Zelikovsky.

2. Byrka *et al* gave an algorithm with an approximation ratio of $1.39 = \ln 4 + \epsilon$ [49] which is currently the best known for this problem. This was originally based on a combination of techniques and subsequently there is an LP based proof [121] that achieves the same approximation for the so-called Hypergraphic LP relaxation.
3. The *bidirected cut* LP relaxation for the STEINER TREE has been studied since the work of Rajagopalan and Vazirani [223]; for quasi-bipartite instances, Chakrabarty, Devanur, and Vazirani showed an upper bound of $4/3$ [56]. For general instances, it remained open for a long time whether its integrality gap is strictly smaller than 2. Byrka, Grandoni, and Traub recently proved an upper bound of 1.9988 [50], and Paschmanns and Traub improved this to 1.898 [217]. The best lower bound currently known is $6/5 = 1.2$ [251].
4. For many applications, the vertices can be modeled as points on the plane, where the distance between them is simply the Euclidean distance. The MST-based algorithm performs fairly well on such instances; it has an approximation ratio of $2/\sqrt{3} \approx 1.15$ [94]. An example which achieves this bound is three points at the corners of an equilateral triangle, say of side-length 1; the MST heuristic outputs a tree of cost 2 (two sides of the triangle) while the optimum solution is to connect the three points to a Steiner vertex which is the circumcenter of the triangle. One can do better still for instances in the plane (or in any Euclidean space of small-dimensions); for any $\epsilon > 0$, there is a $1 + \epsilon$ -approximation algorithm that runs in polynomial time [16]. Such an approximation scheme is also known for planar graphs [43] and more generally bounded-genus graphs.

10.2 The Traveling Salesperson Problem (TSP)

10.2.1 TSP in Undirected Graphs

In the Traveling Salesperson Problem (TSP), we are given an undirected graph $G = (V, E)$ and cost $c(e) > 0$ for each edge $e \in E$. Our goal is to find a Hamiltonian cycle with minimum cost. A cycle is said to be Hamiltonian if it visits every vertex in V exactly once.

TSP is known to be NP-Hard. Moreover, we cannot hope to find a good approximation algorithm for it unless $P = NP$. This is because if one can give a good approximation solution to TSP in polynomial time, then we can exactly solve the NP-Complete Hamiltonian cycle problem (HAM) in polynomial time,

which is not possible unless $P = NP$. Recall that HAM is a decision problem: given a graph $G = (V, E)$, does G have a Hamiltonian cycle?

Theorem 10.4 ([227]). *Let $\alpha : \mathbb{N} \rightarrow \mathbb{N}$ be a polynomial-time computable function. Unless $P = NP$ there is no polynomial-time algorithm that on every instance I of TSP outputs a solution of cost at most $\alpha(|I|) \cdot \text{OPT}(I)$.*

Proof. For the sake of contradiction, suppose we have an approximation algorithm $\mathcal{A}$ for TSP with an approximation ratio $\alpha(|I|)$. We show a contradiction by showing that using $\mathcal{A}$, we can exactly solve HAM in polynomial time. Let $G = (V, E)$ be the given instance of HAM. We create a new graph $H = (V, E')$ with cost $c(e)$ for each $e \in E'$ such that $c(e) = 1$ if $e \in E$, otherwise $c(e) = B$, where $B = n\alpha(n) + 2$ and $n = |V|$. Note that this is a polynomial-time reduction since α is a polynomial-time computable function.

We observe that if G has a Hamiltonian cycle, $\text{OPT} = n$, otherwise $\text{OPT} \geq n - 1 + B \geq n\alpha(n) + 1$. (Here, OPT denotes the cost of an optimal TSP solution in H .) Note that there is a “gap” between when G has a Hamiltonian cycle and when it does not. Thus, if $\mathcal{A}$ has an approximation ratio of $\alpha(n)$, we can tell whether G has a Hamiltonian cycle or not: Simply run $\mathcal{A}$ on the graph H ; if $\mathcal{A}$ returns a TSP tour in H of cost at most $\alpha(n)n$ output that G has a Hamiltonian cycle, otherwise output that G has no Hamiltonian cycle. We leave it as an exercise to formally verify that this would solve HAM in polynomial time. ■

Since we cannot even approximate the general TSP problem, we consider more tractable variants.

- *Metric-TSP*: In Metric-TSP, the instance is a complete graph $G = (V, E)$ with cost $c(e)$ on $e \in E$, where c satisfies the triangle inequality, i.e. $c(uw) \leq c(uv) + c(vw)$ for any $u, v, w \in V$.
- *TSP-R*: TSP with repetitions of vertices allowed. The input is a graph $G = (V, E)$ with non-negative edge costs as in TSP. Now we seek a minimum-cost *walk* that visits each vertex at least once and returns to the starting vertex.

Exercise 10.1. Show that an α -approximation for Metric-TSP implies an α -approximation for TSP-R and vice-versa.

We focus on Metric-TSP for the rest of this section. We first consider a natural greedy approach, the Nearest Neighbor Heuristic (NNH).

NEAREST NEIGHBOR HEURISTIC($G(V, E), c : E \rightarrow \mathcal{R}^+$):
 Start at an arbitrary vertex s ,
 While (there are unvisited vertices)
 From the current vertex u , go to the nearest unvisited vertex v .
 Return to s .

- Exercise 10.2.** 1. Prove that NNH is an $O(\log n)$ -approximation algorithm.
 (Hint: Think back to the proof of the $2H_{|S|}$ -approximation for the Greedy Steiner Tree Algorithm.)
2. NNH is not an $O(1)$ -approximation algorithm; can you find an example to show this? In fact one can show a lower bound of $\Omega(\log n)$ on the approximation-ratio achieved by NNH.

There are constant-factor approximation algorithms for TSP; we now consider an MST-based algorithm. See Fig 10.5.

TSP-MST($G(V, E), c : E \rightarrow \mathcal{R}^+$):
 Compute an MST T of G .
 Obtain an Eulerian graph $H = 2T$ by *doubling* edges of T .
 An Eulerian tour of $2T$ gives a tour in G .
 Obtain a Hamiltonian cycle by shortcutting the tour.

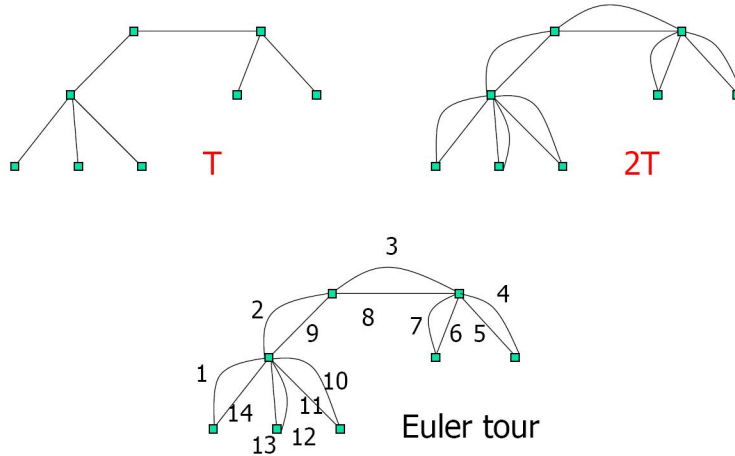


Figure 10.5: MST Based Heuristic

Theorem 10.5. *MST heuristic(TSP-MST) is a 2-approximation algorithm.*

Proof. We have $c(T) = \sum_{e \in E(T)} c(e) \leq \text{OPT}$, since we can get a spanning tree in G by removing any edge from the optimal Hamiltonian cycle, and T is a MST. Thus $c(H) = 2c(T) \leq 2\text{OPT}$. Also shortcutting only decreases the cost. ■

We observe that the loss of a factor 2 in the approximation ratio is due to doubling edges; we did this in order to obtain an Eulerian tour. But any graph in which all vertices have even degree is Eulerian, so one can still get an Eulerian tour by adding edges only between odd degree vertices in T . Christofides Heuristic [85] exploits this and improves the approximation ratio from 2 to $3/2$. See Fig 10.6 for a snapshot.

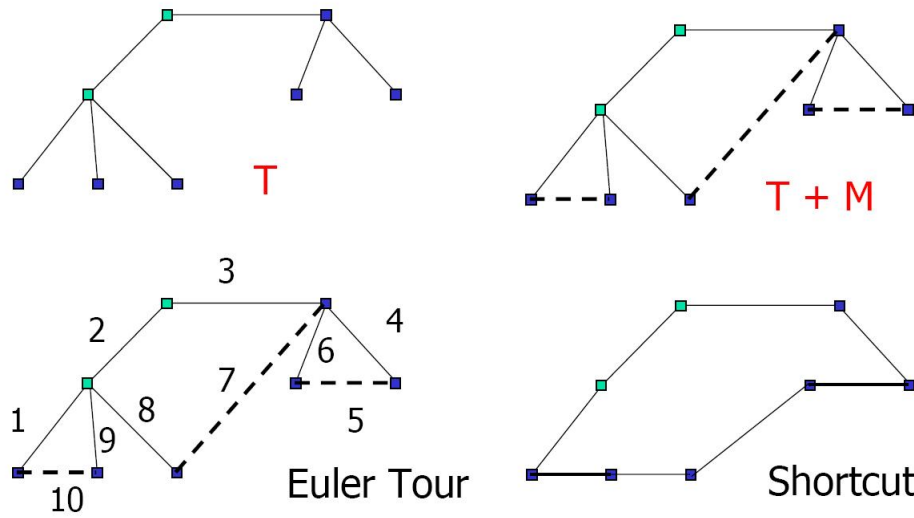


Figure 10.6: Christofides Heuristic

CHRISTOFIDES HEURISTIC($G(V, E), c : E \rightarrow \mathbb{R}^+$):
 Compute an MST T of G .
 Let S be the vertices of odd degree in T . (Note: $|S|$ is even)
 Find a minimum cost matching M on S in G .
 Add M to T to obtain an Eulerian graph H .
 Compute an Eulerian tour of H .
 Obtain a Hamiltonian cycle by shortcutting the tour.

Theorem 10.6. *Christofides Heuristic is a 1.5-approximation algorithm.*

Proof. The main part of the proof is to show that $c(M) \leq .5\text{OPT}$. Suppose that $c(M) \leq .5\text{OPT}$. Then, since the solution of Christofides Heuristic is

obtained by shortcutting the Eulerian tour on H , its cost is no more than $c(H) = c(T) + c(M) \leq 1.5 \text{OPT}$. (Refer to the proof of Lemma 10.5 for the fact $c(T) \leq \text{OPT}$.) Therefore we focus on proving that $c(M) \leq .5 \text{OPT}$.

Let F^* be an optimal tour in G of cost OPT ; since we have a metric-instance we can assume without loss of generality that F^* is a Hamiltonian cycle. We obtain a Hamiltonian cycle F_S^* in the graph $G[S]$ by short-cutting the portions of F^* that touch the vertices $V \setminus S$. By the metric-condition, $c(F_S^*) \leq c(F^*) = \text{OPT}$. Let $S = \{v_1, v_2, \dots, v_{|S|}\}$. Without loss of generality F_S^* visits the vertices of S in the order $v_1, v_2, \dots, v_{|S|}$. Recall that $|S|$ is even. Let $M_1 = \{v_1v_2, v_3v_4, \dots, v_{|S|-1}v_{|S|}\}$ and $M_2 = \{v_2v_3, v_4v_5, \dots, v_{|S|}v_1\}$. Note that both M_1 and M_2 are matchings, and $c(M_1) + c(M_2) = c(F_S^*) \leq \text{OPT}$. We can assume without loss of generality that $c(M_1) \leq c(M_2)$. Then we have $c(M_1) \leq .5 \text{OPT}$. Also we know that $c(M) \leq c(M_1)$, since M is a minimum cost matching on S in $G[S]$. Hence we have $c(M) \leq c(M_1) \leq .5 \text{OPT}$, which completes the proof. ■

10.2.2 LP Relaxation

We describe a well-known LP relaxation for TSP called the Subtour-Elimination LP and sometimes also called the Held-Karp LP relaxation although the formulation was first given by Dantzig, Fulkerson and Johnson [90]. The LP relaxation has a variable x_e for each edge $e \in E$. Note that the TSP solution is a Hamiltonian Cycle of least cost. A Hamiltonian cycle can be viewed as a *connected* subgraph of G with degree 2 at each vertex. Thus we write the degree constraints and also the cut constraints.

$$\begin{aligned} \min \quad & \sum_{e \in E} c_e x_e \\ \sum_{e \in \delta(v)} x_e &= 2 \quad v \in V \\ \sum_{e \in \delta(S)} x_e &\geq 2 \quad \emptyset \subsetneq S \subsetneq V \\ x_e &\in [0, 1] \quad e \in E \end{aligned}$$

The relaxation is not useful for a general graph since we saw that TSP is not approximable. To obtain a relaxation for Metric-TSP we apply the above to the metric completion of the graph G .

Another alternative is to consider the following LP which view the problem as finding a connected Eulerian multi-graph of the underlying graph G . In other words we are allowed to take an integer number of copies of each edge with the constraint that the degree of each vertex is even and the graph is connected. It

is not easy to write the even degree condition since we do not have an apriori bound. Instead one can write the following simpler LP, and interestingly one can show that its optimum value is the same as that of the preceding relaxation (when applied to the metric completion).

$$\begin{aligned} \min \quad & \sum_{e \in E} c_e x_e \\ \sum_{e \in \delta(S)} x_e \quad & \geq 2 \quad \emptyset \subsetneq S \subsetneq V \\ x_e \quad & \geq 0 \quad e \in E \end{aligned}$$

Wolsey showed that the $3/2$ -approximation of Christofides can be analyzed with respect to the LP above. Hence the integrality gap of the LP is at most $3/2$ for Metric-TSP. Is it better? There is a well-known example which shows that the gap is at least $4/3$. The $4/3$ conjecture states that the worst-case integrality gap is at most $4/3$. This has been an unsolved problem for many decades, and only recently was the $3/2$ barrier broken.

Remarks:

1. In practice, local search heuristics are widely used and they perform extremely well. A popular heuristic *2-Opt* is to swap pairs from xy, zw to xz, yw or xw, yz , if it improves the tour.
2. It was a major open problem to improve the approximation ratio of $\frac{3}{2}$ for Metric-TSP; it is conjectured that the Held-Karp LP relaxation [145] gives a ratio of $\frac{4}{3}$. In a breakthrough, Oveis-Gharan, Saberi and Singh [120] obtained a $3/2 - \delta$ approximation for some small but fixed $\delta > 0$ for the important special case where $c(e) = 1$ for each edge e (called GRAPHIC-TSP). The $3/2$ ratio was finally broken for the general case [169].

10.2.3 TSP in Directed Graphs

In this subsection, we consider TSP in directed graphs. As in undirected TSP, we need to relax the problem conditions to get any positive result. Again, allowing each vertex to be visited multiple times is equivalent to imposing the asymmetric triangle inequality $c(u, w) \leq c(u, v) + c(v, w)$ for all u, v, w . This is called the asymmetric TSP (ATSP) problem. We are given a directed graph $G = (V, A)$ with cost $c(a) > 0$ for each arc $a \in A$ and our goal is to find a closed walk visiting all vertices. Note that we are allowed to visit each vertex multiple times, as we are looking for a walk, not a cycle. For an example of a valid Hamiltonian walk, see Fig 10.7.

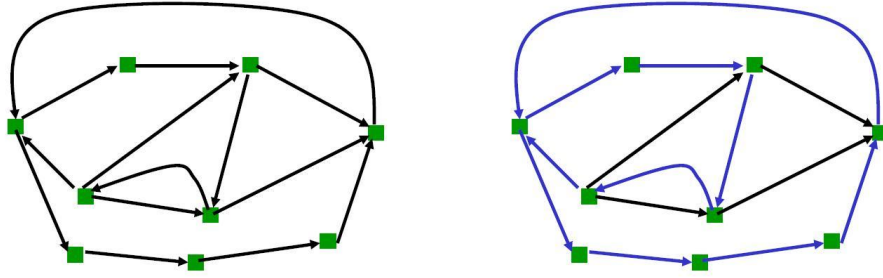


Figure 10.7: A directed graph and a valid Hamiltonian walk

The MST-based heuristic for the undirected case has no meaningful generalization to the directed setting. This is because costs on edges are not symmetric. Hence, we need another approach. The *Cycle Shrinking Algorithm* repeatedly finds a min-cost cycle cover and shrinks cycles, combining the cycle covers found. Recall that a cycle cover is a collection of disjoint cycles covering all vertices. It is known that finding a minimum-cost cycle cover can be done in polynomial time (see Homework 0). The Cycle Shrinking Algorithm achieves a $\log_2 n$ approximation ratio.

CYCLE SHRINKING ALGORITHM($G(V, A), c : A \rightarrow \mathcal{R}^+$):

Transform G so that it is complete and satisfies $c(u, v) + c(v, w) \geq c(u, w)$ for all u, v, w

If $|V| = 1$, output the trivial cycle consisting of the single node

Find a minimum cost cycle cover with cycles $C_1, \dots, C_k$

From each C_i pick an arbitrary proxy node v_i

Recursively solve the problem on $G[\{v_1, \dots, v_k\}]$ to obtain a solution C

$C' = C \cup C_1 \cup C_2 \dots C_k$ is an Eulerian graph.

Shortcut C' to obtain a cycle on V , and output that cycle.

For a snapshot of the Cycle Shrinking Algorithm, see Fig 10.8.

Lemma 10.3. *Let the cost of edges in G satisfy the asymmetric triangle inequality. Then for any $S \subseteq V$, the cost of an optimal TSP tour in $G[S]$ is at most the cost of an optimal TSP tour in G .*

Proof. Since G satisfies the triangle inequality, there is an optimal TSP tour in G that is a Hamiltonian cycle C . Given any $S \subseteq V$, the cycle C can be short-cut to produce another cycle C' that visits only S and whose cost is at most the cost of C . ■

Lemma 10.4. *The cost of a min-cost cycle-cover is at most the cost of an optimal TSP tour.*

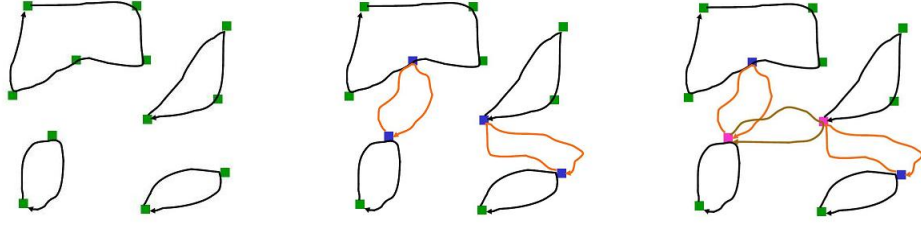


Figure 10.8: A snapshot of Cycle Shrinking Algorithm. To the left, a cycle cover C_1 . In the center, blue vertices indicate proxy nodes, and a cycle cover C_2 is found on the proxy nodes. To the right, pink vertices are new proxy nodes, and a cycle cover C_3 is found on the new proxy nodes.

Proof. An optimal TSP tour is a cycle cover. ■

Theorem 10.7. *The Cycle Shrinking Algorithm is a $\log_2 n$ -approximation for ATSP.*

Proof. We prove the above by induction on n the number of nodes in G . It is easy to see that the algorithm finds an optimal solution if $n \leq 2$. The main observation is that the number of cycles in the cycle-cover is at most $\lfloor n/2 \rfloor$; this follows from the fact that each cycle in the cover has to have at least 2 nodes and they are disjoint. Thus $k \leq \lfloor n/2 \rfloor$. Let $\text{OPT}(S)$ denote the cost of an optimal solution in $G[S]$. From Lemma 10.3 we have that $\text{OPT}(S) \leq \text{OPT}(V) = \text{OPT}$ for all $S \subseteq V$. The algorithm recurses on the proxy nodes $S = \{v_1, \dots, v_k\}$. Note that $|S| < n$, and by induction, the cost of the cycle C output by the recursive call is at most $(\log_2 |S|) \text{OPT}(S) \leq (\log_2 |S|) \text{OPT}$.

The algorithm outputs C' whose cost is at most the cost of C plus the cost of the cycle-cover computed in G . The cost of the cycle cover is at most OPT (Lemma 10.4). Hence the cost of C' is at most $(\log_2 |S|) \text{OPT} + \text{OPT} \leq (\log_2 n/2) \text{OPT} + \text{OPT} \leq (\log_2 n) \text{OPT}$; this finishes the inductive proof. ■

10.2.4 LP Relaxation

The LP relaxation for ATSP is given below. For each arc $e \in E$ we have a variable x_e . We view the problem as finding a connected Eulerian multi-graph in the support of G . That is, we can choose each edge e an integer number of times. We impose Eulerian constraint at each vertex by requiring the in-degree to be equal to the out-degree. We impose connectivity constraint by ensuring that at least one arc leaves each set of vertices S which is not V or $\emptyset$.

$$\begin{aligned}
& \min \quad \sum_{e \in A} c_e x_e \\
& \sum_{e \in \delta^+(v)} x_e - \sum_{e \in \delta^-(v)} x_e = 0 \quad v \in V \\
& \sum_{e \in \delta^+(S)} x_e \geq 1 \quad \emptyset \subsetneq S \subsetneq V \\
& x_e \geq 0 \quad e \in A
\end{aligned}$$

Remarks:

1. It has remained an open problem for more than 25 years whether there exists a constant factor approximation for ATSP. Asadpour *et al* [25] have obtained an $O(\log n / \log \log n)$ -approximation for ATSP using some very novel ideas and a well-known LP relaxation.
2. There is now an $O(1)$ -approximation for ATSP with initial breakthrough work by Svensson for a special case [240] and then followed up by Svensson, Tarnawski and Végé [242]. Traub and Vygen improved the guarantee to $22 + \epsilon$ [247], and a more recent preprint by Vygen, submitted on March 15, 2026, improves the ratio further to less than 15 [252]. The algorithms are highly non-trivial and are based on the LP relaxation.
3. For a broader treatment of both symmetric and asymmetric TSP, including many recent developments, see the book by Traub and Vygen published by Cambridge University Press in 2024 [248].

Chapter 11

Steiner Forest Problem

We discuss a primal-dual based 2-approximation for the STEINER FOREST problem¹. In the STEINER FOREST problem there is a graph $G = (V, E)$ where each edge $e \in E$ has a cost $c_e \in \mathbb{R}$. We are given k pairs of vertices $(s_1, t_1), (s_2, t_2), \dots, (s_k, t_k) \in V \times V$. The goal is to find the minimum cost set of edges F such that in the graph (V, F) the vertices s_i and t_i are in the same connected component for $1 \leq i \leq k$. We refer to the set $\{s_1, \dots, s_k, t_1, \dots, t_k\}$ as terminals. Notice that the graph (V, F) can contain multiple connected components.

One can see that STEINER FOREST generalizes the STEINER TREE problem. It is, however, much less easy to obtain a constant factor approximation for STEINER FOREST. A natural greedy heuristic, similar to that for the online STEINER TREE problem is the following. We order the pairs in some arbitrary fashion, say as 1 to k without loss of generality. We maintain a forest F of edges that we have already bought. When considering pair i we find a shortest path P from s_i to t_i in the graph in which we reduce the cost of each edge in F to 0 (since we already paid for them). We add the new edges in P to F . Although simple to describe, the algorithm is not straight forward to analyze. In fact the best bound we have on the performance of this algorithm is $O(\log^2 k)$ (note that the corresponding bound for STEINER TREE is $O(\log k)$) while the lower bound on its performance is only $\Omega(\log k)$. The analysis of the upper bound requires appealing to certain extremal results [28]. More recently, Bamas, Drygala, and Maggiori [32] improved the upper bound to $O(\log k \log \log k)$ for a broad class of instances. Closing the gap between the upper and lower bound on the analysis of this simplest of greedy algorithms is a very interesting open problem.

The first constant factor approximation for STEINER FOREST is due to the influential work of Agarwal, Klein and Ravi [8] who gave a primal-dual 2-approximation. This remained the best known ratio for roughly three decades,

¹Parts of this chapter are based on past scribed lecture notes by Ben Moseley from 2009.

until Ahmadi et al. gave a $(2 - \epsilon)$ -approximation in 2025 [9]. Subsequently, Gupta and Traub obtained a 1.994-approximation, to appear in STOC 2026 [136]. The primal-dual algorithm of [8] has since been generalized for a wide variety of network design problems via the work of Goemans and Williamson (see their survey [123]) and several others. STEINER FOREST has the advantage that one can visualize the algorithm and analysis more easily when compared to the more abstract settings that we will see shortly.

11.1 LP Relaxation

We now describe an integer programming formulation for the problem. In the **IP** we will have a variable x_e for each edge $e \in E$ such that x_e is 1 if and only if e is part of the solution. Let the set $\mathcal{S}$ be the collection of all sets $S \subset V$ such that $|S \cap \{s_i, t_i\}| = 1$ for some $1 \leq i \leq k$. For a set $S \subset V$ let $\delta(S)$ denote the set of edges crossing the cut $(S, V \setminus S)$. The **IP** can be written as the following.

$$\begin{aligned} \min \quad & \sum_{e \in E} c_e x_e \\ \text{such that} \quad & \sum_{e \in \delta(S)} x_e \geq 1 \quad \forall S \in \mathcal{S} \\ & x_e \in \{0, 1\} \quad \forall e \in E \end{aligned}$$

We can obtain an **LP**-relaxation by changing the constraint that $x_e \in \{0, 1\}$ to $x_e \geq 0$. Note that although the resulting LP has exponentially many constraints, it can be solved in polynomial time via the Ellipsoid method. Given a candidate solution x , for each pair (s_i, t_i) we compute a minimum s_i - t_i cut in the graph with edge capacities x_e ; if some cut has value less than 1, the corresponding set S gives a violated constraint. Thus we have an efficient separation oracle. One can also write a compact extended formulation by introducing, in addition to the x -variables, flow variables for each pair (s_i, t_i) and requiring that the edges selected by x support one unit of flow from s_i to t_i . The dual of the **LP**-relaxation can be written as the following.

$$\begin{aligned} \max \quad & \sum_{S \in \mathcal{S}} y_S \\ \text{such that} \quad & \sum_{S: e \in \delta(S)} y_S \leq c_e \quad \forall e \in E \\ & y_S \geq 0 \quad \forall S \in \mathcal{S} \end{aligned}$$

11.2 Warm-up: Primal-Dual for Steiner Tree

Before describing the primal-dual algorithm for STEINER FOREST, it is useful to see the simpler special case of STEINER TREE. Suppose the terminal set is $R \subseteq V$. The goal is to buy a minimum-cost set of edges that connects all terminals in R . Consider the following cut relaxation. Let C be the collection of all sets $S \subset V$ such that $S \cap R \neq \emptyset$ and $(V \setminus S) \cap R \neq \emptyset$. These are precisely the cuts that separate the terminal set.

$$\begin{aligned} \min \quad & \sum_{e \in E} c_e x_e \\ \text{such that} \quad & \sum_{e \in \delta(S)} x_e \geq 1 \quad \forall S \in C \\ & x_e \geq 0 \quad \forall e \in E \end{aligned}$$

The dual is

$$\begin{aligned} \max \quad & \sum_{S \in C} y_S \\ \text{such that} \quad & \sum_{S: e \in \delta(S)} y_S \leq c_e \quad \forall e \in E \\ & y_S \geq 0 \quad \forall S \in C. \end{aligned}$$

The primal-dual algorithm grows moats around the current components that contain terminals, until all terminals are in one component.

STEINERTREEPD:

$F \leftarrow \emptyset$

while not all terminals are in the same connected component of (V, F)

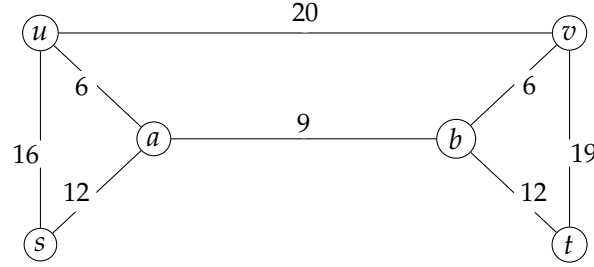
 Let $C_1, C_2, \dots, C_h$ be the connected components of (V, F) that contain a terminal

 Raise y_{C_i} for $1 \leq i \leq h$ uniformly until some edge e becomes tight

$F \leftarrow F + e$

Output $T = \{e \in F \mid F - e \text{ does not connect all terminals}\}$

Example 11.1. Consider the following graph with terminals $R = \{u, s, v, t\}$. The vertices a and b are nonterminals.



Initially the active components are the four singletons $\{u\}, \{s\}, \{v\}, \{t\}$. Raising their dual variables uniformly, the first tight edges are ua and vb , both after dual growth 6. Suppose the algorithm adds ua first. In the next iteration the edge vb is already tight, so it is added with no additional dual growth.

The active components are now $\{u, a\}, \{v, b\}, \{s\}, \{t\}$. The edge us currently feels dual $6 + 6 = 12$, coming from the original components $\{u\}$ and $\{s\}$. After raising all active duals by 2 more, us becomes tight and is added.

Next the active components are $\{u, a, s\}, \{v, b\}, \{t\}$. The edge bt already feels ten units of dual, so after one more unit of growth it becomes tight and is added. Finally the active components are $\{u, a, s\}$ and $\{v, b, t\}$. The edge uv feels seventeen units of dual, so after $3/2$ more units of growth it becomes tight and is added, connecting all terminals.

The algorithm has bought $\{ua, vb, us, bt, uv\}$, but the pruning step deletes ua : the remaining edges $\{us, uv, vb, bt\}$ still connect u, s, v, t . Thus the final tree uses the Steiner vertex b but not the Steiner vertex a .

This is the same template as the forest algorithm below, but the behavior of active components is simpler. While the algorithm is running, a component is active exactly when it contains at least one terminal. The algorithm stops when there is only one active component.

At the end of the algorithm, the constructed dual vector y is feasible for the dual LP, and the pruned edge set T is feasible for STEINER TREE. Moreover, every edge $e \in T$ has a tight dual constraint: $c_e = \sum_{S \in \mathcal{C}: e \in \delta(S)} y_S$.

Theorem 11.1. *The primal-dual algorithm above is a $2(1 - 1/|R|)$ -approximation for STEINER TREE, and hence a 2-approximation.*

Proof. Let A_i be the active components in iteration i , and let Δ_i be the amount of

dual growth in that iteration. Every edge in T has a tight dual constraint. Hence

$$\begin{aligned}
 c(T) &= \sum_{e \in T} c_e \\
 &= \sum_{e \in T} \sum_{S \in C: e \in \delta(S)} y_S \\
 &= \sum_{S \in C} y_S \deg_T(S) \\
 &= \sum_i \sum_{S \in A_i} \Delta_i \deg_T(S),
 \end{aligned}$$

where $\deg_T(S)$ denotes the number of edges of T crossing $\delta(S)$. The last equality uses the fact that y_S increases by Δ_i precisely in those iterations i in which $S \in A_i$.

We claim that, for every iteration i , $\sum_{S \in A_i} \deg_T(S) \leq 2|A_i| - 2$. To see this, contract the connected components of (V, F) at the start of iteration i and restrict attention to the edges of T . The component containing the active nodes is a tree. Every leaf of this tree must be an active node: otherwise the unique edge incident to an inactive leaf could be deleted without disconnecting the terminals, contradicting the pruning step. Applying the degree-counting bound to the active nodes gives $\sum_{S \in A_i} \deg_T(S) \leq 2|A_i| - 2$.

Therefore

$$c(T) \leq \sum_i \Delta_i (2|A_i| - 2) \leq 2 \left(1 - \frac{1}{|R|}\right) \sum_i \Delta_i |A_i| = 2 \left(1 - \frac{1}{|R|}\right) \sum_{S \in C} y_S \leq 2 \left(1 - \frac{1}{|R|}\right) \text{OPT},$$

where the second inequality uses $|A_i| \leq |R|$, and the last inequality follows by weak duality. ■

The STEINER FOREST problem can be seen as replacing the single requirement that all terminals be connected by many pairwise connectivity requirements. This creates a new phenomenon: a component can be inactive because all demands inside it are already satisfied, but later become part of an active component after it merges with another component. The algorithm below handles this by raising only the minimally violated components and by using a slightly more careful pruning argument.

11.3 Primal-Dual Algorithm for Steiner Forest

Before we continue, some definitions will be stated which will help to define our algorithm for the problem.

Definition 11.2. Given a set of edges $X \subseteq E$, a set $S \in \mathcal{S}$ is violated with respect to X if $\delta(S) \cap X = \emptyset$. In other words S is violated even with edge set X included.

Definition 11.3. Given a set of edges $X \subseteq E$, a set $S \in \mathcal{S}$ is minimally violated with respect to X if S is violated with respect to X and there is no $S' \subset S$ that is also violated with respect to X .

Next we show that any two minimally violated sets are disjoint.

Claim 11.3.1. $\forall X \subseteq E$ if S and S' are minimally violated sets then $S \cap S' = \emptyset$, i.e. S and S' are disjoint.

In fact we will prove the following claim which implies the preceding one.

Claim 11.3.2. Let $X \subseteq E$. The minimally violated sets with respect to X are the connected components of the graph (V, X) that are violated with respect to X .

Proof. Consider a minimal violated set S . We may assume the set S contains s_i but not t_i for some i . If S is not a connected component of (V, X) then there must be some connected component S' of $G[S]$ that contains s_i . But $\delta_X(S') = \emptyset$, and hence S' is violated; this contradicts the fact that S is minimal. Therefore, if a set S is a minimal violated set then it must be connected in the graph (V, X) .

Now suppose that S is a connected component of (V, X) ; it is easy to see that no proper subset of S can be violated since some edge will cross any such set. Thus, if S is a violated set then it is a minimal violated set.

Thus, the minimal violated sets with respect to X are the connected components of the graph (V, X) that are themselves violated sets. It follows that any two distinct minimal violated sets are disjoint. ■

The primal-dual algorithm for STEINER FOREST is described below.

```

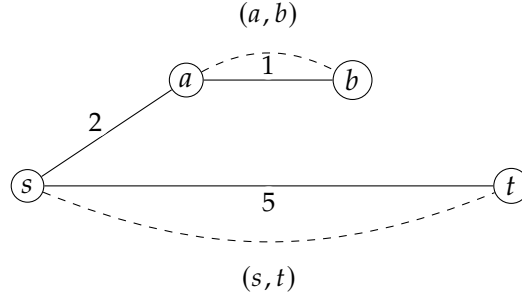
STEINERFOREST:
   $F \leftarrow \emptyset$ 
  while  $F$  is not feasible
    Let  $C_1, C_2, \dots, C_h$  be minimally violated sets with respect to  $F$ 
    Raise  $y_{C_i}$  for  $1 \leq i \leq h$  uniformly until some edge  $e$  becomes tight
     $F \leftarrow F + e$ 
     $x_e = 1$ 
  Output  $F' = \{e \in F \mid F - e \text{ is not feasible}\}$ 

```

The first thing to notice about the algorithm above is that it is closely related to our solution to the VERTEX COVER problem, however, there are two main differences. In the VERTEX COVER we raised the dual variables for all uncovered edges uniformly, however, in this algorithm we were more careful on which dual variables are raised. In this algorithm, we chose to only raise the variables

which correspond to the minimally violated sets. Unlike the case of STEINER TREE, in STEINER FOREST, there can be non-trivial connected components that are not violated and hence become inactive. A temporarily inactive component may become part of an active component later if an active component merges with it. The other main difference is that when we finally output the solution, we *prune* F to get F' . This is done for technical reasons, but the intuition is that we should include no edge in the solution which is not needed to obtain a feasible solution. To understand this algorithm, there is a non-trivial example in the textbook [250] that demonstrates the algorithm's finer points.

Example 11.2. The following small instance illustrates three phenomena at once: a component can become inactive, later become part of an active component again, and then get detached by pruning. Consider the graph with vertices s, a, b, t , edge costs $c(ab) = 1$, $c(sa) = 2$, and $c(st) = 5$, and demand pairs (a, b) and (s, t) .



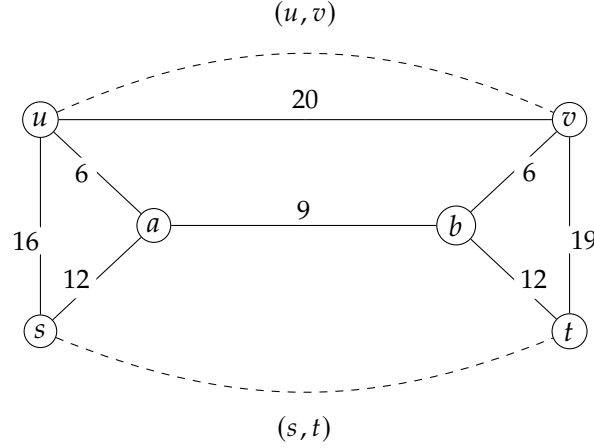
Initially the active components are the four singletons $\{s\}, \{a\}, \{b\}, \{t\}$. Raising their dual variables uniformly, the edge ab becomes tight first, at time $1/2$, so it is added. The component $\{a, b\}$ is now inactive: it already satisfies the pair (a, b) , and it does not separate the pair (s, t) .

The active components are now $\{s\}$ and $\{t\}$. The edge sa already feels one unit of dual, namely $y_{\{s\}} + y_{\{a\}} = 1$, so after one more unit of dual growth it becomes tight and is added. This merges $\{a, b\}$ into the component $\{s, a, b\}$, which is active again because it contains s but not t .

Finally, the only active components are $\{s, a, b\}$ and $\{t\}$. The edge st already feels three units of dual, coming from the previously raised sets $\{s\}$ and $\{t\}$, so after one more unit of growth it becomes tight and is added. The resulting forest is $\{ab, sa, st\}$.

In the pruning step, the edge sa is deleted: the remaining edges $\{ab, st\}$ still connect a to b and s to t . Thus the component $\{a, b\}$, which was inactive and later became part of the active component $\{s, a, b\}$, is detached again in the final pruned solution.

Example 11.3. We next give a more detailed example, adapted from [250], that illustrates simultaneous dual growth, tie-breaking, and pruning. Consider the following graph. The demand pairs are (u, v) and (s, t) .



The algorithm may return a solution of cost 54 on this instance, even though the optimum has cost 45. The point is that the primal-dual analysis only guarantees a factor-2 approximation, and not optimality. Let us trace the algorithm.

In the first iteration, the active sets are the four singleton components $\{u\}, \{v\}, \{s\}, \{t\}$. Raising their dual variables uniformly, the first tight edges are ua and vb , both when the active duals have been raised by 6. Suppose the algorithm adds ua .

In the second iteration, the active components are $\{u, a\}, \{v\}, \{s\}, \{t\}$. The edge vb is already tight, so no further dual growth is needed and the algorithm adds vb immediately.

In the third iteration, the active components are $\{u, a\}, \{v, b\}, \{s\}, \{t\}$. The edge us currently feels dual $6 + 6 = 12$, coming from the components containing u and s , respectively. Therefore after raising all active duals by 2 more, the edge us becomes tight and is added.

In the fourth iteration, the active components are $\{u, a, s\}, \{v, b\}, \{t\}$. Raising their duals by 1 makes the edge bt tight, so the algorithm adds bt .

In the fifth iteration, the active components are $\{u, a, s\}$ and $\{v, b, t\}$. Raising these two duals by 1 makes the edge uv tight, so it is added. At this point both demands are satisfied, and the current forest is $\{ua, vb, us, bt, uv\}$.

Finally, the pruning step removes the redundant edge ua , because after uv, us, vb , and bt have been bought, the edge ua is no longer needed to connect

either demand pair. Thus the output is the tree with edges $\{us, uv, vb, bt\}$, of total cost $16 + 20 + 6 + 12 = 54$.

On the other hand, the set of edges $\{ua, ab, bv, as, bt\}$ has total cost $6 + 9 + 6 + 12 + 12 = 45$ and is feasible, so the output above is not optimal. This example is useful because it shows several features of the algorithm at once: multiple edges can become tight together, tie-breaking can matter, and the pruning step is essential.

Lemma 11.1. *At the end of the algorithm, F' and $\mathbf{y}$ are primal and dual feasible solutions, respectively.*

Proof. In each iteration of the while loop, only the dual variables corresponding to connected components were raised. Therefore, no edge that is contained within the same component can become tight, and, therefore, F is acyclic. To see that none of the dual constraints are violated, observe that when a constraint becomes tight (that is, it holds with equality), the corresponding edge e is added to F . Subsequently, since e is contained in some connected component of F , no set S with $e \in \delta(S)$ ever has y_S raised. Therefore, the constraint for e cannot be violated, and so $\mathbf{y}$ is dual feasible.

As long as F is not feasible, the while loop will not terminate, and there are some minimal violated sets that can have their dual variables raised. Therefore, at the end of the algorithm F is feasible. Moreover, since F is acyclic (it is a forest), there is a unique s_i - t_i path in F for each $1 \leq i \leq k$. Thus, each edge on a s_i - t_i path is not redundant and is not deleted when pruning F to get F' . ■

Theorem 11.4. *The primal-dual algorithm for STEINER FOREST gives a $2(1 - 1/\tau)$ -approximation, where τ is the number of distinct terminals, and hence a 2-approximation.*

Proof. Let F' be the output from our algorithm. To prove this theorem, we want to compare $c(F')$ to the feasible dual constructed by the algorithm. First, we know that $c(F') = \sum_{e \in F'} c_e = \sum_{e \in F'} \sum_{S \in \mathcal{S}: e \in \delta(S)} y_S$ because every edge picked is tight. Let $\deg_{F'}(S)$ denote the number of edges of F' that cross the cut $(S, V \setminus S)$. It can be seen that $\sum_{e \in F'} \sum_{S \in \mathcal{S}: e \in \delta(S)} y_S = \sum_{S \in \mathcal{S}} y_S \deg_{F'}(S)$.

Let A_i contain the minimally violated sets in iteration i and let Δ_i denote the amount of dual growth in the i th iteration. Say that our algorithm runs for α iterations. We can then rewrite $\sum_{S \in \mathcal{S}} y_S \deg_{F'}(S)$ as the double summation $\sum_{i=1}^{\alpha} \sum_{S \in A_i} \Delta_i \deg_{F'}(S)$. In the next lemma it will be shown for any iteration i that $\sum_{S \in A_i} \deg_{F'}(S) \leq 2|A_i| - 2$. Knowing this we can prove the theorem. Since the minimally violated sets in A_i are disjoint and each contains at least one

terminal, we have $|A_i| \leq \tau$ for every i .

$$\begin{aligned}
 \sum_{S \in \mathcal{S}} y_S \deg_{F'}(S) &= \sum_{S \in \mathcal{S}} \sum_{i: S \in A_i} \Delta_i \deg_{F'}(S) \\
 &= \sum_{i=1}^{\alpha} \sum_{S \in A_i} \Delta_i \deg_{F'}(S) \leq \sum_{i=1}^{\alpha} \Delta_i \cdot (2|A_i| - 2) \\
 &\leq 2 \left(1 - \frac{1}{\tau}\right) \sum_{i=1}^{\alpha} \Delta_i |A_i| = 2 \left(1 - \frac{1}{\tau}\right) \sum_{S \in \mathcal{S}} y_S.
 \end{aligned}$$

The theorem follows by weak duality. $\blacksquare$

Now we show the lemma used in the previous theorem. It is in this lemma that we use the fact that we prune F to get F' .

We need a simple claim.

Lemma 11.2. *Let $T = (V, E)$ be a forest with at least one edge, and let $Z \subseteq V$ be a subset of the nodes such that every leaf of T is in Z . Then $\sum_{v \in Z} \deg_T(v) \leq 2|Z| - 2$ where $\deg_T(v)$ is the degree of v in T .*

Proof. We first prove it for a tree. We have $\sum_{v \in V} \deg_T(v) = 2|V| - 2$ since a tree has $|V| - 1$ edges. Every node $u \in V - Z$ has degree at least 2 since it is not a leaf. Thus $\sum_{u \in V-Z} \deg_T(u) \geq 2|V - Z|$, and hence

$$\sum_{v \in Z} \deg_T(v) \leq 2|V| - 2 - 2|V - Z| = 2|Z| - 2.$$

For a forest, apply the tree case to each non-trivial connected component and sum the resulting inequalities. This gives $\sum_{v \in Z} \deg_T(v) \leq 2|Z| - 2r$, where $r \geq 1$ is the number of non-trivial components of T , and hence the claimed bound. $\blacksquare$

Lemma 11.3. *For any iteration i of our algorithm, $\sum_{S \in A_i} \deg_{F'}(S) \leq 2|A_i| - 2$.*

Proof. Consider the graph (V, F') , and fix an iteration i . In this graph, contract each set S active in iteration i to a single node (call such a node an *active node*), and each inactive component to a single node. Let the resulting graph be denoted by H . We know that F is a forest and we have contracted connected subsets of vertices in F ; as $F' \subseteq F$, we conclude that H is also a forest.

Claim 11.3.3. *Every leaf of H is an active node.*

Proof. If not, consider a leaf node v of H which is an inactive node, and let $e \in F'$ be the edge incident to it. We claim that $F' - e$ is feasible, contradicting the minimality of F' . If deleting e made the solution infeasible, then some demand

pair (s_j, t_j) would be separated by the cut induced by this leaf: one of s_j, t_j would lie in the component corresponding to v , and the other would lie outside it. But then, at the start of iteration i , that component was a violated component and hence active, contradicting the choice of v . ■

The degree in H of an active node corresponding to violated set S is $\deg_{F'}(S)$. Now we apply Lemma 11.2. ■

Chapter 12

Primal Dual for Constrained Forest Problems

We previously saw a primal-dual based 2-approximation for the STEINER FOREST problem. The algorithm can be generalized to a much wider class of problems that involve finding a min-cost forest in an undirected edge-weighted graph that needs to satisfy some constraint. The resulting machinery is more abstract and requires more advanced tools. We start with some problems all of which are NP-hard.

Point-to-point connection problem: Given an edge-weighted graph $G = (V, E)$ and two disjoint sets $X = \{s_1, \dots, s_k\}$ and $Y = \{t_1, \dots, t_k\}$ of terminals find the min-cost forest in G such that each connected component contains the same number of terminals from X and Y .

Lower Capacitated Tree Problem: Given $G = (V, E)$, $c : E \rightarrow \mathbb{R}^+$ and a $k \in \mathbb{Z}^+$ find a set $E' \subseteq E$ of minimum cost such that every connected component in (V, E') has at least k vertices.

Connectivity Augmentation: Given an undirected k -edge connected graph $G = (V, E)$ and a set of edges $E^{aug} \subseteq V \times V - E$, find a set $E' \subseteq E^{aug}$ of minimum cost such that $G = (V, E \cup E')$ is $(k + 1)$ -edge connected.

Steiner Connectivity Augmentation: Let $G = (V, E)$ be an edge-weighted graph and let $(s_1, t_1), \dots, (s_h, t_h)$ be h pairs such that each pair is k -edge-connected in G . Given a set of edges $E^{aug} \subseteq V \times V - E$, find a set $E' \subseteq E^{aug}$ of minimum cost such that in $G' = (V, E \cup E')$ each pair (s_i, t_i) is $(k + 1)$ -edge connected.

Each of the preceding problems can be cast as a special case of the following abstract problem. Given an edge-weighted graph $G = (V, E)$ and a function

$f : 2^V \rightarrow \{0, 1\}$, find a min-cost subset of edges E' such that $|\delta_{E'}(S)| \geq f(S)$ for each $S \subseteq V$. We use the notation $\delta_F(S)$ to denote the edges from an edge set F that cross the set/cut S . Alternatively we want a min-cost subset of edges E' such that each set $S \in \mathcal{S}$ is crossed by an edge of E' where $\mathcal{S} = \{S \mid f(S) = 1\}$. It is easy to observe that a minimal solution to this abstract problem is a forest since any cut needs to be covered at most once. This formulation is too general since f may be completely arbitrary. The goal is to find a sufficiently general class that captures interesting problems while still being tractable. The advantage of $\{0, 1\}$ functions is precisely because the minimal solutions are forests. We will later consider integer valued functions.

Definition 12.1. Given a graph $G = (V, E)$ and an integer valued function $f : 2^V \rightarrow \mathbb{Z}$ we say that a subset of edges F is feasible for f or covers f iff $|\delta_F(S)| \geq f(S)$ for all $S \subseteq V$.

Remark 12.1. Even though it may seem natural to restrict attention to requirement functions that only have non-negative entries, we will see later that the flexibility of negative requirements is useful.

Given a network design problem Π in an undirected graph $G = (V, E)$ and an integer valued function $f : 2^V \rightarrow \mathbb{Z}_+$ we say that the *requirement* function of Π is f if covering f is equivalent to satisfying the constraints of Π .

12.1 Classes of Functions and Setup

Here we consider classes of requirement functions $f : 2^V \rightarrow \mathbb{Z}$ and their relationship. Even though we define more generally, for this chapter the focus will be on $\{0, 1\}$ functions.

Definition 12.2. f is maximal if for all disjoint A and B we have $f(A \cup B) \leq \max\{f(A), f(B)\}$.

Exercise 12.1. Prove that the requirement function of STEINER FOREST is maximal.

Definition 12.3. f is proper if it is symmetric, maximal and $f(V) = 0$.

Exercise 12.2. Prove that the requirement function of STEINER FOREST is proper.

Exercise 12.3. Prove that the requirement function of Steiner Connectivity Augmentation is proper.

Definition 12.4. f is downward monotone if $f(A) \leq f(B)$ for all $\emptyset \neq B \subset A$.

Exercise 12.4. Prove that the requirement function of the Lower Capacitated Tree problem is downward monotone.

A very general class is the one given below.

Definition 12.5. f is skew-supermodular (also called weakly-supermodular) if for all A and B one of the following conditions holds,

1. $f(A) + f(B) \leq f(A \cup B) + f(A \cap B)$
2. $f(A) + f(B) \leq f(A - B) + f(B - A)$

A specialization of skew-supermodular for $\{0, 1\}$ functions will be the focus of this chapter.

Definition 12.6. A $\{0, 1\}$ valued f is uncrossable if for all A and B such that $f(A) = f(B) = 1$, one of the following conditions holds,

1. $f(A \cup B) = 1$ and $f(A \cap B) = 1$
2. $f(A - B) = 1$ and $f(B - A) = 1$.

Claim 12.1.1. If f is downward monotone then it is skew-supermodular.

Proof. If $A - B = \emptyset$, then $A \subseteq B$. Since $A \cup B = B$ and $A \cap B = A$, the first skew-supermodular inequality holds with equality. The case $B - A = \emptyset$ is symmetric.

Otherwise, $A - B$ and $B - A$ are both nonempty. Since f is downward monotone, $A - B \subset A$ and $B - A \subset B$ imply:

$$\begin{aligned} f(A) &\leq f(A - B) \\ f(B) &\leq f(B - A) \end{aligned}$$

and hence the second condition of skew-supermodularity always holds. ■

Lemma 12.1. If f is $\{0, 1\}$ -valued and proper then it is skew-supermodular.

Proof. Consider two sets A, B . If $f(A) + f(B) \leq 1$, then by maximality applied to the disjoint union $A = (A - B) \cup (A \cap B)$, either $f(A - B) = 1$ or $f(A \cap B) = 1$ whenever $f(A) = 1$; the case $f(B) = 1$ is symmetric. Thus one of the two skew-supermodular inequalities holds.

It remains to consider the case $f(A) = f(B) = 1$. If $f(A \cap B) = 0$, then maximality applied to $A = (A - B) \cup (A \cap B)$ and $B = (B - A) \cup (A \cap B)$ implies $f(A - B) = f(B - A) = 1$. If $f(A \cup B) = 0$, then by symmetry $f(V - (A \cup B)) = 0$; applying maximality to $V - A = (B - A) \cup (V - (A \cup B))$ and $V - B = (A - B) \cup (V - (A \cup B))$ again implies $f(A - B) = f(B - A) = 1$. Therefore, if $f(A \cup B)$ and $f(A \cap B)$ are not both 1, then $f(A - B)$ and $f(B - A)$ are both 1. This proves one of the two required inequalities in all cases. ■

Definition 12.7. Let $G = (V, E)$ and $f : 2^V \rightarrow \mathbb{Z}$. For each $X \subseteq E$, the residual requirement function $f_X : 2^V \rightarrow \mathbb{Z}$ is defined as:

$$f_X(A) = f(A) - |\delta_X(A)|.$$

Exercise 12.5. Let $g : 2^V \rightarrow \mathbb{R}$ be a symmetric submodular function. Prove that g satisfies *posi-modularity*:

$$g(A) + g(B) \geq g(A - B) + g(B - A) \quad \forall A, B \subseteq V.$$

Lemma 12.2. If f is skew-supermodular then f_X is also skew-supermodular for any $X \subseteq E$.

Proof. The function $|\delta_X(\cdot)|$ is submodular. Hence

$$|\delta_X(A)| + |\delta_X(B)| \geq |\delta_X(A \cap B)| + |\delta_X(B \cup A)| \quad \forall A, B \subseteq V.$$

The function is also symmetric and hence also satisfies *posi-modularity*. Therefore,

$$|\delta_X(A)| + |\delta_X(B)| \geq |\delta_X(A - B)| + |\delta_X(B - A)| \quad \forall A, B \subseteq V.$$

Now consider the function f_X and let A, B be any two subsets of V . Suppose $f(A) + f(B) \leq f(A \cup B) + f(A \cap B)$. Then

$$\begin{aligned} f_X(A) + f_X(B) &= f(A) - |\delta_X(A)| + f(B) - |\delta_X(B)| \\ &\leq f(A \cup B) + f(A \cap B) - (|\delta_X(A)| + |\delta_X(B)|) \\ &\leq f(A \cup B) + f(A \cap B) - (|\delta_X(A \cap B)| + |\delta_X(B \cup A)|) \\ &= f_X(A \cup B) + f_X(A \cap B). \end{aligned}$$

Similarly, if $f(A) + f(B) \leq f(A - B) + f(B - A)$ we can use *posi-modularity* of $|\delta_X(\cdot)|$ to argue that $f_X(A) + f_X(B) \leq f_X(A - B) + f_X(B - A)$. We note that $|\delta_X|$ is both submodular *and* *posi-modular* which allows us to use the right inequality. ■

Remark 12.2. Allowing negative requirements in the definition of skew-supermodularity allows a clean proof when subtracting $|\delta_X(\cdot)|$.

In the case of $\{0, 1\}$ functions we make the following claim and leave the proof as an exercise which is similar to the one above.

Lemma 12.3. Let $f : 2^V \rightarrow \{0, 1\}$ be an uncrossable function and let $X \subseteq E$. Let $h_X : 2^V \rightarrow \{0, 1\}$ be the residual $\{0, 1\}$ function where $h_X(S) = 1$ iff $f(S) = 1$ and $|\delta_X(S)| = 0$. Then h_X is uncrossable.

Definition 12.8. Let f be a $\{0, 1\}$ requirement function over V and let $X \subseteq E$. A set S is violated with respect to X if $h_X(S) = 1$ (in other words S is not yet covered by the edge set X). A set S is a minimal violated set if there is no $S' \subsetneq S$ such that S' is violated.

Lemma 12.4. Let f be an uncrossable function and $X \subseteq E$, then the minimal violated sets of f with respect to X are disjoint. That is, if A, B are minimal violated sets then $A = B$ or $A \cap B = \emptyset$.

Proof. By the preceding lemma, the $\{0, 1\}$ residual function h_X is also uncrossable. Thus it suffices to consider minimal violated sets A, B with respect to a $\{0, 1\}$ uncrossable function and the empty edge set. Suppose the property does not hold. Then we can assume that A, B properly cross, that is, $A - B, A \cap B, B - A$ are all non-empty. We have $f(A) = f(B) = 1$ since both are violated. Since f is uncrossable, we have $f(A - B) = f(B - A) = 1$ or $f(A \cap B) = f(A \cup B) = 1$. In both cases we see that we violate minimality of A, B . ■

12.2 A Primal-Dual Algorithm for Covering Uncrossable Functions

In this section we consider the following problem. Given a graph $G = (V, E)$ with non-negative edge weights $c : E \rightarrow \mathbb{R}_+$ and an uncrossable function $f : 2^V \rightarrow \{0, 1\}$ find a min-cost set of edges $F \subseteq E$ such that $|\delta_F(S)| \geq f(S)$ for all S . An important computational issue is how f is specified. A natural model is to consider the value-oracle model where we have access to $f(S)$ via an oracle. Given $S \subseteq V$, the oracle returns $f(S)$. However this is not sufficient in the general context of uncrossable functions as we see from the following example. Fix some set A . Define the function f_A where $f_A(S) = 1$ iff $S = A$. It is easy to see that there is a feasible solution iff $\delta_E(A) \neq \emptyset$. How do we even verify that there is a feasible solution via a value oracle? In general it may take an exponential number of queries to find A . Hence we need more. We will assume that there is an oracle that given G and $X \subseteq E$ outputs the set of all minimal violated sets with respect to X . This will typically be easy to ensure for specific functions of interest. We note that for some special class of functions such as proper functions, a value oracle suffices to find the minimal violated sets.

We write the primal and dual LPs for covering f via edges of a given graph $G = (V, E)$.

$$\begin{array}{ll}
 \min & \sum_{e \in E} c_e x_e \\
 \text{such that} & \sum_{e \in \delta(S)} x_e \geq f(S) \quad \forall S \subseteq V \\
 & x_e \geq 0 \quad \forall e \in E
 \end{array}$$

$$\begin{array}{ll}
 \max & \sum_{S \subseteq V} f(S) y_S \\
 \text{such that} & \sum_{S: e \in \delta(S)} y_S \leq c_e \quad \forall e \in E \\
 & y_S \geq 0 \quad \forall S \subseteq V
 \end{array}$$

The primal-dual algorithm is similar to the one for STEINER FOREST with a growth phase and reverse-delete phase.

```

COVERUNCROSSABLEFUNC( $G = (V, E), f$ ):
     $F \leftarrow \emptyset$ 
    while  $F$  is not feasible
        Let  $C_1, C_2, \dots, C_\ell$  be minimal violated sets of  $f$  with respect to  $F$ 
        Raise  $y_{C_i}$  for  $1 \leq i \leq \ell$  uniformly until some edge  $e$  becomes tight
         $F \leftarrow F + e$ 
         $x_e = 1$ 
        Let  $e_1, e_2, \dots, e_t$  be the edges added to  $F$  in order
         $F' = F$ 
        for  $i = t$  down to 1 do
            if  $(F' - e_i)$  is feasible then  $F' = F' - e_i$ 
    Output  $F'$ 
    
```

Analysis: We can prove the following by induction on the iterations and omit the routine details.

Lemma 12.5. *The output of the algorithm, F' , is a feasible solution that covers f . Assuming oracle access to finding the minimal violated sets in each iteration, the algorithm can be implemented in polynomial time.*

The preceding lemma shows that the algorithm correctly outputs a feasible solution. The main part is to analyze the cost of the solution.

Theorem 12.9. *Let F' be the output of the algorithm. Then $c(F') \leq 2 \sum_S f(S) y_S$ and hence $c(F') \leq 2 \text{OPT}$.*

The dual solution y constructed by the algorithm is feasible: in each iteration we raise dual variables only until the first edge constraint becomes tight, and no dual constraint is ever exceeded.

The cost analysis is based on the following key structural lemma.

Lemma 12.6. *Let $G = (V, E)$ be a graph and let $f : 2^V \rightarrow \{0, 1\}$ be an uncrossable function and let C be the set of minimal violated sets of f with respect to $\emptyset$. Let $F \subseteq E$ be any minimal feasible solution that covers f . Then $\sum_{C \in \mathcal{C}} \deg_F(C) \leq 2|C|$ where $\deg_F(C) = |\delta(C) \cap F|$ is the number of edges of F crossing C .*

We defer the proof of the preceding lemma to the following subsection and finish the analysis of the 2-approximation. This is similar to the one for STEINER FOREST. Let t be the number of iterations of the algorithm. Let C_i be the minimal violated sets in iteration i of the algorithm and let Δ_i be the amount by which the duals grow in iteration i . We call any $C \in C_i$ *active* in iteration i . We have $\sum_{e \in F'} c_e = \sum_{e \in F'} \sum_{S: e \in \delta(S)} y_S$ since we add edges to F only when the dual constraint is tight and $F' \subseteq F$. We also observe that the duals are grown only for sets $S \subseteq V$ where $f(S) = 1$ and hence $\sum_S f(S) y_S = \sum_S y_S$ for the dual solution created by the algorithm.

$$\begin{aligned} c(F') &= \sum_{e \in F'} \sum_{S: e \in \delta(S)} y_S \\ &= \sum_S f(S) y_S \deg_{F'}(S) \\ &= \sum_{i=1}^t \Delta_i \sum_{S \in C_i} \deg_{F'}(S). \end{aligned}$$

To complete the analysis we need the following lemma which can be seen as a corollary of Lemma 12.6.

Lemma 12.7. *Consider any iteration i of the algorithm. Then $\sum_{C \in C_i} \deg_{F'}(C) \leq 2|C_i|$.*

Proof. Consider iteration i . Let $X = \{e_1, e_2, \dots, e_{i-1}\}$ be the set of edges added by the algorithm in the growth phase prior to iteration i . Thus C_i is the minimal violated sets with respect to X . Consider the graph $G' = (V, E \setminus X)$ and the $\{0, 1\}$ residual function h_X , where $h_X(S) = 1$ iff $f(S) = 1$ and $|\delta_X(S)| = 0$. We observe that $F'' = F' \setminus X$ is a minimal feasible solution to covering h_X in G' — this is because of the reverse delete process. Moreover we claim that $\deg_{F'}(C) = \deg_{F''}(C)$ for any $C \in C_i$ because no edge of X crosses any set in C_i . We can now apply Lemma 12.6 to the function h_X (which is uncrossable) and G'

and F'' to obtain:

$$\sum_{C \in C_i} \deg_{F'}(C) = \sum_{C \in C_i} \deg_{F''}(C) \leq 2|C_i|.$$

■

With the preceding lemma in place we have,

$$c(F') = \sum_{i=1}^t \Delta_i \sum_{S \in C_i} \deg_{F'}(S) \leq \sum_{i=1}^t \Delta_i \cdot 2|C_i| = 2 \sum_S f(S) y_S.$$

12.2.1 Proof of Lemma 12.6

Since F is a *minimal* feasible solution to cover f , for every $e \in F$ there must be a set S_e such that $f(S_e) = 1$ and $\delta_F(S_e) = \{e\}$; this is the reason we cannot remove e from F and maintain feasibility. We call such a set S_e a *witness set* for e . Clearly a witness set S_e is a violated set.

Claim 12.2.1. *Let S_e be a witness set for $e \in F$. Then for any minimal violated set C we have $C \subseteq S_e$ or $C \cap S_e = \emptyset$.*

Proof. If $S_e \subsetneq C$, then S_e itself contradicts the minimality of C . Otherwise, if C intersects S_e but is not contained in it, then C and S_e cross. Since f is uncrossable, either $C - S_e$ or $C \cap S_e$ is violated, again contradicting the minimality of C . ■

Note that there can be many witness sets for the same edge, however, the same set cannot be a witness set for two different edges.

Definition 12.10. *Given a minimal feasible solution F we call a family of sets $\mathcal{S}$ a witness family for F if there is a bijection $h : F \rightarrow \mathcal{S}$ such that $h(e)$ is a witness set for e .*

Given F we can construct a witness family by considering each edge $e \in F$ and picking an arbitrary witness set for e and adding it to the collection.

Definition 12.11. *A family $\mathcal{S}$ of finite sets is laminar if no two sets $A, B \in \mathcal{S}$ cross.*

We have seen that there is a witness family for F since it is minimal. We can obtain a special witness family starting with an arbitrary witness family.

Lemma 12.8. *There is a laminar witness family for F .*

Proof. The process is based on *uncrossing*. We start with an arbitrary witness family $\mathcal{S}$. If it is laminar we are done, otherwise let S_{e_1}, S_{e_2} be the witness sets for $e_1, e_2 \in F$ such that S_{e_1}, S_{e_2} cross.

Claim 12.2.2. *One of the following holds: (i) $(\mathcal{S} \setminus \{S_{e_1}, S_{e_2}\}) \cup \{S_{e_1} - S_{e_2}, S_{e_2} - S_{e_1}\}$ is a witness family for F (ii) $(\mathcal{S} \setminus \{S_{e_1}, S_{e_2}\}) \cup \{S_{e_1} \cap S_{e_2}, S_{e_1} \cup S_{e_2}\}$ is a witness family for F .*

The new sets don't cross each other but what about other sets in the family? One can argue that the number of crossings can only decrease. More formally, suppose we have three sets A, B, D and say D crosses p sets from A, B (p is one of $\{0, 1, 2\}$). If we uncross A, B (that is replace A, B by $A - B, B - A$ or $A \cap B, A \cup B$) then the number of sets that D crosses after uncrossing cannot increase. We leave this claim as an exercise. This means that repeated uncrossing of two crossing sets will eventually lead to a laminar family.

Now we prove the claim. S_{e_1} and S_{e_2} are violated sets which means that $f(S_{e_1}) = 1$ and $f(S_{e_2}) = 1$. Since f is uncrossable, say $f(S_{e_1} - S_{e_2}) = 1$ and $f(S_{e_2} - S_{e_1}) = 1$. The only edges from F that can cross $S_{e_1} - S_{e_2}$ and $S_{e_2} - S_{e_1}$ are e_1 and e_2 — if there is another edge e that crosses one of them then it would also cross one of S_{e_1} and S_{e_2} and hence they would not be witness sets. Since F is a feasible solution, $S_{e_1} - S_{e_2}$ and $S_{e_2} - S_{e_1}$ are both crossed by some edge from $Y = \{e_1, e_2\}$. We claim that each of them is crossed by exactly one of them. If this is the case then they are valid witness sets for the two edges e_1, e_2 . To see why exactly one of them crosses each set we use submodularity/posi-modularity:

$$2 \leq |\delta_Y(S_{e_1} - S_{e_2})| + |\delta_Y(S_{e_2} - S_{e_1})| \leq |\delta_Y(S_{e_1})| + |\delta_Y(S_{e_2})| = 2.$$

The first inequality holds since each is covered and the second inequality is because of posi-modularity of $|\delta_Y(\cdot)|$.

The other case when $f(S_{e_1} \cap S_{e_2}) = 1$ and $f(S_{e_1} \cup S_{e_2}) = 1$ can also be handled with a similar argument. In both cases the two new witness sets are assigned to distinct edges: a single edge cannot cross both new sets, since then it would cross both original witness sets and would have to be both e_1 and e_2 . ■

Let $\mathcal{S}$ be a laminar witness family for F . We create a rooted tree P from $\mathcal{S}$ as follows. Let $\widehat{\mathcal{S}} = \mathcal{S} \cup \{V\}$. For each set $S \in \widehat{\mathcal{S}}$ we add a vertex v_S . For any S, T such that T is the smallest set in $\widehat{\mathcal{S}}$ properly containing S we add the edge (v_S, v_T) which makes v_T the parent of v_S . Since we added V to the family we obtain a tree since every maximal set in the original family becomes a child of v_V . Note that P has $|F|$ edges and $|F| + 1$ nodes and each $e \in F$ corresponds to the edge (v_{S_e}, v_T) of P where v_T is the parent of v_{S_e} in P . We keep this bijection between F and edges of P in mind for later.

See Fig 12.1.

Consider any minimal violated set $C \in \mathcal{C}$. We observe that C cannot cross any set in $\mathcal{S}$ since it is a witness family. For each C we associate the minimal set $S \in \widehat{\mathcal{S}}$ such that $C \subseteq S$. We call v_S an active node of P if there is a $C \in \mathcal{C}$

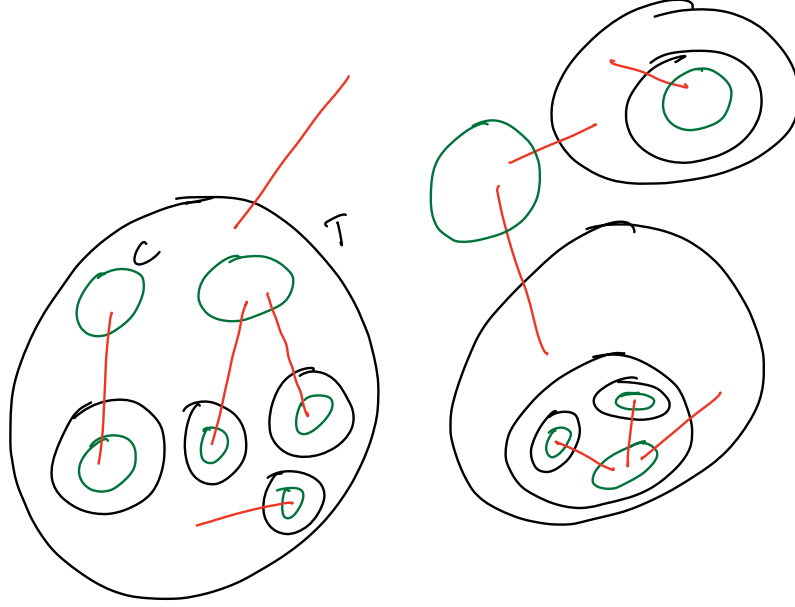


Figure 12.1: Laminar witness family for a minimal solution F shown as red edges. Sets in green are the minimal violated sets and the sets in black are the witness sets. The root set V is not drawn.

associated with it. Note that (i) not all nodes of P may be active (ii) every C is associated with exactly one active node of P (iii) multiple sets from C can be associated with the same active node of P .

Lemma 12.9. *Let P_a be the set of active nodes. Then $|P_a| \leq |C|$. And every leaf of P is an active node.*

Proof. Since each C is associated with an active node we obtain $|P_a| \leq |C|$. Since $\mathcal{S}$ is not empty, the root node of P cannot be a leaf. Suppose v_{S_e} is a leaf of P . By the construction of the tree P , S_e is a minimal set in $\mathcal{S}$. Since S_e is a violated set, it must contain a minimal violated set C . But then S_e is active because of C . ■

Lemma 12.10. *Let v_T be an active node in P . Let $C' \subseteq C$ be the set of all minimal violated sets associated with v_T . Then $\sum_{C \in C'} \deg_F(C) \leq \deg_P(v_T)$.*

Proof. Let $Y = \cup_{C \in C'} \delta_F(C)$ be the set of edges incident to the sets in C' . Consider any $C \in C'$. We have $C \subseteq T$, and C is disjoint from every child of T ; otherwise C

would be contained in that child, contradicting the choice of T as the minimal member of $\widehat{\mathcal{S}}$ containing C . Consider an edge $e \in \delta_F(C)$.

If e crosses T , then T is a witness set for e and we charge e to the parent edge of T in P . This case cannot occur when $T = V$. If e does not cross T , let S_e be the witness set for e . The set S_e must be contained in a child of T : if S_e contained C , then the minimality of T would imply that S_e is T or an ancestor of T , but then e , whose endpoints both lie in T , could not cross S_e ; hence S_e is disjoint from C , and laminarity places it inside some child of T . We charge e to that child edge of T in P .

Each incident edge of v_T in P is charged at most once, because each witness set is crossed by exactly one edge of F . Also, no edge of F can cross two sets $C_1, C_2 \in \mathcal{C}'$: such an edge has both endpoints in T , and its witness set would have to contain one of C_1, C_2 , forcing it to be T or an ancestor of T , a contradiction. Therefore $\sum_{C \in \mathcal{C}'} \deg_F(C)$ is at most the number of edges incident to v_T in P , which is $\deg_P(v_T)$. ■

Now we are ready to finish the proof. From Lemma 12.10 and the fact that each $C \in \mathcal{C}$ is associated to some active node, we have

$$\sum_{C \in \mathcal{C}} \deg_F(C) \leq \sum_{v \in P_a} \deg_P(v).$$

To bound $\sum_{v \in P_a} \deg_P(v)$ we had observed that in the tree P every leaf is an active node. Suppose the root is an active node. Then from Lemma 11.2, $\sum_{v \in P_a} \deg_P(v) \leq 2|P_a| - 2$. Suppose the root is inactive. Again from Lemma 11.2 where we consider $Z = P_a \cup \{v_V\}$, we have $\sum_{v \in P_a} \deg_P(v) + 1 \leq 2(|P_a| + 1) - 2 = 2|P_a|$. Hence $\sum_{v \in P_a} \deg_P(v) \leq 2|P_a| - 1$. Thus, in both cases we see that $\sum_{v \in P_a} \deg_P(v) \leq 2|P_a|$. Finally we note that $|P_a| \leq |C|$ and hence putting together we have

$$\sum_{C \in \mathcal{C}} \deg_F(C) \leq 2|P_a| \leq 2|C|$$

as desired.

Bibliographic Notes: The primal-dual algorithm for uncrossable functions is from the paper of Williamson, Goemans, Mihail and Vazirani [257]. See Williamson's thesis [256] for a detailed treatment of the proof. The survey by Goemans and Williamson [124] describes the many applications of the primal-dual method in network design.

Chapter 13

Survivable Network Design Problem

In this chapter we consider the SURVIVABLE NETWORK DESIGN PROBLEM problem. The input is an undirected graph $G = (V, E)$ with edge-weights $c : E \rightarrow \mathbb{R}_+$ and integer requirements $r(uv)$ for each pair of vertices uv . We wrote uv instead of (u, v) to indicate that the requirement function is for unordered pairs (alternatively, $r(u, v) = r(v, u)$ for all u, v). The goal is to find a min-cost subgraph $H = (V, F)$ of G such that the connectivity between u and v in H is at least $r(uv)$. We obtain two versions of the problem: EC-SNDP if the connectivity requirement is edge-connectivity and VC-SNDP for vertex connectivity. It turns out that EC-SNDP is much more tractable than VC-SNDP and we will focus on EC-SNDP.

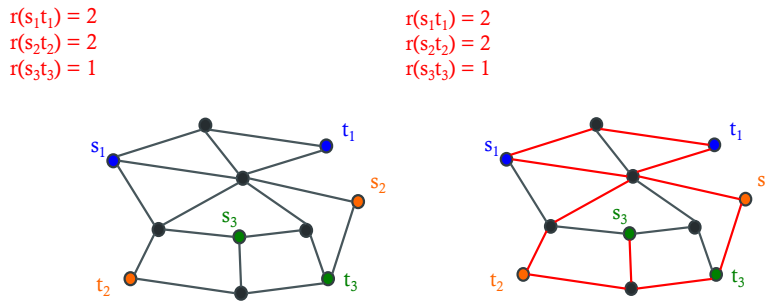


Figure 13.1: Example of EC-SNDP. Requirement only for three pairs. A feasible solution shown in the second figure as red edges. In this example the paths for each pair are also vertex disjoint even though the requirement is only for edge-disjointness.

For EC-SNDP there is a seminal work of Jain based on iterated rounding

that yields a 2-approximation as a special case of a more general problem. Prior to his work there was an augmentation based approach that yields $2k$ and $2H_k$ approximations where $k = \max_{uv} r(uv)$ is the maximum connectivity requirement. Despite being superseded by Jain's result in terms of the ratio, the augmentation approach is important for various reasons and we will discuss both.

We first consider the LP relaxation for the EC-SNDP. We do this by setting up the requirement function $f : 2^V \rightarrow \mathbb{Z}$ where we let $f(S) = \max_{u \in S, v \in V-S} r(uv)$. The goal is to find a min-cost subgraph H of G such that $\delta_H(S) \geq f(S)$.

Claim 13.0.1. *The requirement function f that captures EC-SNDP is proper and hence skew-supermodular.*

Proof. It is easy to see f is symmetric. Consider disjoint sets A, B . Suppose $f(A \cup B) = k$ which means that there is some $s \in A \cup B$ and $t \in V - (A \cup B)$ such that $r(st) = k$. If $s \in A$ then $f(A) \geq k$ and if $s \in B$ then $f(B) \geq k$. Therefore $\max\{f(A), f(B)\} \geq k = f(A \cup B)$ since $s \in A$ or $s \in B$. ■

13.1 Augmentation approach

The augmentation approach for EC-SNDP is based on iteratively increasing the connectivity of pairs from 1 to k where $k = \max_{uv} r(uv)$. In fact this works for any proper function $f : 2^V \rightarrow \mathbb{Z}$ and we will work in this generality rather than focus only on EC-SNDP.

Claim 13.1.1. *Let f be a proper function and let p be an integer. Then the truncated function $g_p : 2^V \rightarrow \mathbb{Z}$ defined as $g_p(S) = \min\{p, f(S)\}$ is proper.*

Proof. Exercise. ■

Lemma 13.1. *Let $G = (V, E)$ be a graph and let $f : 2^V \rightarrow \mathbb{Z}$ be a proper function and let $p \geq 0$ be a non-negative integer. Let $X \subseteq E$ be a set of edges such that $|\delta_X(S)| \geq g_p(S)$. Consider the function $h_{p+1} : 2^V \rightarrow \{0, 1\}$ where $h_{p+1}(S) = 1$ iff $f(S) \geq p + 1$ and $|\delta_X(S)| = p$. Then h_{p+1} is uncrossable and symmetric.*

Proof. Consider the function g_{p+1} which is proper and hence also skew-supermodular. For notational convenience we use h for h_{p+1} . Suppose $h(A) = h(B) = 1$. This implies $g_{p+1}(A) \geq p + 1$ and $g_{p+1}(B) \geq p + 1$ and $|\delta_X(A)| = |\delta_X(B)| = p$. g_{p+1} is skew-supermodular. First case is when $g_{p+1}(A) + g_{p+1}(B) \leq g_{p+1}(A \cup B) + g_{p+1}(A \cap B)$. This implies that $g_{p+1}(A \cup B) = g_{p+1}(A \cap B) = p + 1$. By submodularity of $|\delta_X|$ we have $|\delta_X(A)| + |\delta_X(B)| \geq |\delta_X(A \cap B)| + |\delta_X(A \cup B)|$ and by feasibility of X for g_p we have $|\delta_X(A \cap B)| = |\delta_X(A \cup B)| = p$. This implies that $h(A \cap B) = h(A \cup B) = 1$.

Similarly, if $g_{p+1}(A) + g_{p+1}(B) \leq g_{p+1}(A - B) + g_{p+1}(B - A)$ we can argue that $h(A - B) = h(B - A) = 1$ via posi-modularity of $|\delta_X|$. ■

Exercise 13.1. Let $G = (V, E)$ be a graph and let $f : 2^V \rightarrow \mathbb{Z}$ be a proper function. Suppose F is a feasible cover for g_p . Let h_{p+1} be the residual uncrossable function that arises from g_p as in the preceding lemma. Let $F' \subseteq E \setminus F$ be a feasible cover for h_{p+1} in the graph $G' = (V, E \setminus F)$. Then $F \cup F'$ is a feasible cover for g_{p+1} .

Lemma 13.2. Let f be the requirement function of an instance of EC-SNDP in $G = (V, E)$ and let p be an integer and let $X \subseteq E$ be a set of edges. There is a polynomial time algorithm to find the minimal violated sets of g_p with respect to X .

Proof. For each pair of nodes (s, t) with $r(st) \geq p$ find a source-minimal s - t mincut S in the graph $H = (V, X)$ and a sink-minimal mincut T via maxflow¹. Let S be the cut. If $|\delta_X(S)| < p$ then S is a violated set. We compute all such minimal cuts over all pairs of vertices and take the minimal sets in this collection. We leave it as an exercise to check that the minimal violated sets of g_p are the minimal sets in this collection and will be disjoint. ■

Corollary 13.1. Let f be the requirement function of an instance of EC-SNDP in $G = (V, E)$ and let p be an integer. Let X be a set of edges such that X is feasible to cover g_p . In the graph $G' = (V, E \setminus X)$ and for any $F \subseteq (E \setminus X)$ the minimal violated sets of h_{p+1} with respect to F can be computed in polynomial time.

Proof. The minimal violated sets of h_{p+1} with respect to F are the same as the minimal violated sets of g_{p+1} with respect to $X \cup F$. ■

¹The source minimal s - t mincut in a directed/undirected graph is unique via submodularity and can be found by computing s - t maxflow and finding the reachable set from s in the residual graph. Similarly sink minimal set.

Augmentation-Algorithm($G = (V, E), f$)

1. If E does not cover f output “infeasible”
2. $k = \max_S f(S)$ is the maximum requirement
3. $A \leftarrow \emptyset$
4. for $(p = 1$ to $k)$ do
 - A. $G' = (V, E \setminus A)$
 - B. Let g_p be the function defined as $g_p(S) = \min\{f(S), p\}$
 - C. Let h_p be the uncrossable function where $h_p(S) = 1$ iff $g_p(S) > |\delta_A(S)|$
 - D. Find $A' \subseteq E \setminus A$ that covers h_p in G'
 - E. $A \leftarrow A \cup A'$
5. Output A

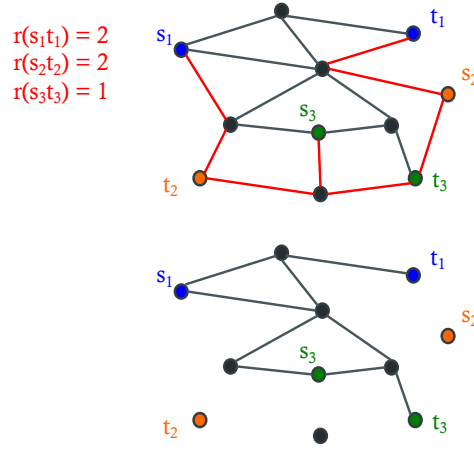


Figure 13.2: Example to illustrate the augmentation approach. Top picture shows a set of edges that connect all pairs with connectivity requirement at least 1. Second picture shows the residual graph in which one needs to solve the augmentation problem. Note that s_2 and t_2 are isolated vertices in the residual graph, however, the cuts induced by them are already satisfied by the edges chosen in the first iteration.

Theorem 13.2. *The augmentation algorithm yields a $2k$ -approximation for EC-SNDP where k is the maximum connectivity requirement.*

Proof. We sketch the proof. The algorithm has k iterations and in each iteration it uses a black box algorithm to cover an uncrossable function. We saw a primal-dual 2-approximation for this problem. We observe that if F^* is an optimum solution to the given instance then in each iteration $F^* \setminus A$ is a feasible solution to the covering problem in that iteration. Thus the cost paid by the algorithm in each iteration can be bound by $2c(F^*)$ and hence the total cost is at most $2k \text{ OPT}$. The preceding lemmas argue that the primal-dual algorithm can be implemented in polynomial time. ■

Remark 13.1. A different algorithm that is based on augmentation in *reverse* yields a $2H_k$ approximation where H_k is the k 'th harmonic number. We refer the reader to [123].

13.2 Iterated rounding based 2-approximation

In this section we describe the seminal result of Jain [155] who obtained a 2-approximation for EC-SNDP via iterated rounding. He proved a more general polyhedral result. Consider the problem of covering a skew-supermodular function $f : 2^V \rightarrow \mathbb{Z}$ by the edges of a graph $G = (V, E)$. The natural cut covering LP relaxation for the problem is given below.

$$\begin{aligned} \min \quad & \sum_{e \in E} c(e)x_e \\ & \sum_{e \in \delta(S)} x_e \geq f(S) \quad S \subset V \\ & x_e \in [0, 1] \quad e \in E \end{aligned}$$

Note that upper bound constraints $x_e \leq 1$ are necessary in the general setting when f is integer valued since we can only take one copy of an edge. The key structural theorem of Jain is the following.

Theorem 13.3. *Let x be a basic feasible solution to the LP relaxation. Then there is some edge $e \in E$ such that $x_e = 0$ or $x_e \geq 1/2$.*

With the above in place, and the observation that the residual function of a skew-supermodular function is again a skew-supermodular function, one obtains an iterative rounding algorithm.

Cover-Skew-Supermodular(G, f)

1. If E does not cover f output “infeasible”
2. $A \leftarrow \emptyset$, $g = f$
3. While A is not a feasible solution do
 - A. Find an optimum basic feasible solution x to cover g in $G' = (V, E \setminus A)$.
 - B. If there is some e such that $x_e = 0$ then $E \leftarrow E - \{e\}$
 - C. Else If there is some e such that $x_e \geq 1/2$ then
 1. $A = A \cup \{e\}$
 2. $g = f_A$ (recall $f_A(S) = f(S) - |\delta_A(S)|$)
4. Output A

Corollary 13.4. *The integrality gap of the cut LP is at most 2 for any skew-supermodular function f .*

Proof. We consider the iterative rounding algorithm and prove the result via induction on m the number of edges in G . The base case of $m = 0$ is trivial since the function has to be 0.

Let x^* be an optimum basic feasible solution to the LP relaxation. We have $\sum_{e \in E} c_e x_e^* \leq \text{OPT}$. We can assume without loss of generality that f is not trivial in the sense that $f(S) \geq 1$ for at least some set S , otherwise $x = 0$ is optimal and there is nothing to prove. By Theorem 13.3, there is an edge $\tilde{e} \in E$ such that $x_{\tilde{e}}^* = 0$ or $x_{\tilde{e}}^* \geq 1/2$. Let $E' = E \setminus \tilde{e}$ and $G' = (V, E')$. In the former case we can discard $\tilde{e}$ and the current LP solution restricted to E' is a feasible fractional solution and we obtain the desired result via induction since we have one less edge.

The more interesting case is when $x_{\tilde{e}}^* \geq 1/2$. The algorithm includes $\tilde{e}$ and recurses on G' and the residual function $g : 2^V \rightarrow \mathbb{Z}$ where $g(S) = f(S) - |\delta_{\tilde{e}}(S)|$. Note that g is skew-supermodular. We observe that $A' \subseteq E'$ is a feasible solution to cover g in G' iff $A' \cup \{\tilde{e}\}$ is a feasible solution to cover f in G . Furthermore, we also observe that the fractional solution x' obtained by restricting x^* to E' is a feasible fractional solution to the LP relaxation to cover g in G' . Thus, by induction, there is a solution $A' \subseteq E'$ such that $c(A') \leq 2 \sum_{e \in E'} c(e) x_e^*$. The algorithm outputs $A = A' \cup \{\tilde{e}\}$ which is feasible to cover f in G . We have

$$c(A) = c(A') + c(\tilde{e}) \leq c(A') + 2c(\tilde{e})x_{\tilde{e}}^* \leq 2 \sum_{e \in E'} c(e)x_e^* + 2c(\tilde{e})x_{\tilde{e}}^* = 2 \sum_{e \in E} c(e)x_e^*.$$

We used the fact that $x_{\tilde{e}}^* \geq 1/2$ to upper bound $c(\tilde{e})$ by $2c(\tilde{e})x_{\tilde{e}}^*$. ■

2-approximation for EC-SNDP : We had already seen that the requirement function for EC-SNDP is skew-supermodular. To apply Theorem 13.3 and obtain a 2-approximation for EC-SNDP we need to argue that the LP relaxation can be solved efficiently. We observe that the LP relaxation at the top level can be solved efficiently via maxflow. We need to check that in the graph G with edge capacities given by the fractional solution x the min-cut between every pair of vertices (s, t) is at least $r(s, t)$. Note that the algorithm is iterative. As we proceed the function $g = f_A$ where f is the original requirement function and the A is the set of edges already chosen.

Exercise 13.2. Prove that there is an efficient separation oracle for each step of the iterative rounding algorithm when f is the requirement function for a given EC-SNDP instance.

We now prove Theorem 13.3. The proof consists of two steps. The first step is a characterization of basic feasible solutions via laminar tight sets. The second step is a counting argument.

13.2.1 Basic feasible solutions and laminar family of tight sets

Let x be a basic feasible solution to the LP relaxation. We are done if there is any edge e such that $x_e = 0$ or $x_e = 1$. Hence the interesting case is when x is fully fractional, that is, $x_e \in (0, 1)$ for every $e \in E$.

Definition 13.5. A set $S \subseteq V$ is tight with respect to x if $x(\delta(S)) = f(S)$.

The LP relaxation is of the form $Ax \geq b, x \in [0, 1]^m$. We number the edges as $e_1, e_2, \dots, e_m$ arbitrarily. Note that each row of A corresponds to a set S and the non-zero entries in the row corresponding to S are precisely for edges in $\delta(S)$. For notational convenience we use χ_S to denote the m -dimensional row vector where $\chi_S(i) = 0$ if $e_i \notin \delta(S)$ and $\chi_S(i) = 1$ if $e_i \in \delta(S)$. By the rank lemma, if x is a basic feasible solution that is fully fractional, then there are m tight sets $S_1, S_2, \dots, S_m$ such that the vectors $\chi_{S_1}, \chi_{S_2}, \dots, \chi_{S_m}$ are linearly independent in $\mathbb{R}^m$. In other words x is the unique solution to the system $\chi_{S_i}^T x = f(S_i), 1 \leq i \leq m$. Note that for a given basic feasible solution x there can be many such bases. A key technical lemma is that one can choose a nice one.

Lemma 13.3. Let x be a basic feasible solution to the cut covering LP relaxation of a skew-supermodular function f where $x_e \in (0, 1)$ for all e . Then there is a laminar family $\mathcal{L}$ of tight sets $S_1, S_2, \dots, S_m$ such that x is the unique solution to the system $\chi_{S_i}^T x = f(S_i)$.

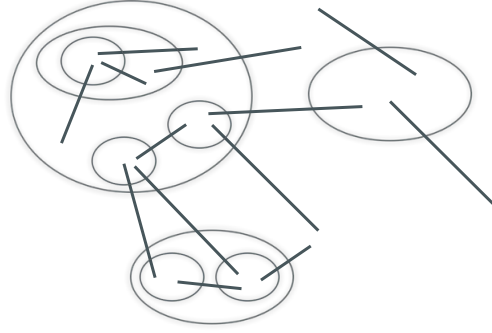


Figure 13.3: Laminar family of tight sets.

We need an auxiliary uncrossing lemma.

Lemma 13.4. *Suppose A and B are two tight sets with respect to x such that A, B cross. Then one of the following holds:*

- $A \cap B, A \cup B$ are tight and $\chi_A + \chi_B = \chi_{A \cup B} + \chi_{A \cap B}$.
- $A - B, B - A$ are tight and $\chi_A + \chi_B = \chi_{A - B} + \chi_{B - A}$.

Proof. Since f is skew-supermodular $f(A) + f(B) \leq f(A \cap B) + f(A \cup B)$ or $f(A) + f(B) \leq f(A - B) + f(B - A)$. We will consider the first case.

A, B are tight, hence $x(\delta(A)) = f(A)$ and $x(\delta(B)) = f(B)$. Moreover the function $h(S) = x(\delta(S))$ is submodular (recall that the cut capacity function in an undirected graph is symmetric submodular). Thus $x(\delta(A)) + x(\delta(B)) \geq x(\delta(A \cup B)) + x(\delta(A \cap B))$. We also have by feasibility of x that $x(\delta(A \cup B)) \geq f(A \cup B)$ and $x(\delta(A \cap B)) \geq f(A \cap B)$. Putting together we have

$$x(\delta(A)) + x(\delta(B)) = f(A) + f(B) \leq f(A \cap B) + f(A \cup B) \leq x(\delta(A \cup B)) + x(\delta(A \cap B)) \leq x(\delta(A)) + x(\delta(B)).$$

This implies that $x(\delta(A \cup B)) = f(A \cup B)$ and $x(\delta(A \cap B)) = f(A \cap B)$. Thus both $A \cap B$ and $A \cup B$ are tight. Moreover we observe that $x(\delta(A)) + x(\delta(B)) = x(\delta(A \cup B)) + x(\delta(A \cap B)) + 2x(E(A - B, B - A))$ where $E(A - B, B - A)$ is the set of edges between $A - B$ and $B - A$. From the above tightness we see that $x(\delta(A)) + x(\delta(B)) = x(\delta(A \cup B)) + x(\delta(A \cap B))$, and since x is fully fractional it means that $E(A - B, B - A) = \emptyset$. This implies that $\chi_A + \chi_B = \chi_{A \cup B} + \chi_{A \cap B}$ (why?).

The second case is similar where we use posi-modularity of the cut function. ■

Proof of Lemma 13.3. One natural way to proceed is as follows. We start with tight sets $\mathcal{S} = \{S_1, S_2, \dots, S_m\}$ such that x is characterized as the unique solution

of the equations implied by these sets. If the family $\{S_1, S_2, \dots, S_m\}$ is laminar we are done. Otherwise we pick some two crossing sets, say S_1, S_2 without loss of generality and uncross them using Lemma 13.4. We get a new family $\mathcal{S}'$ with m tight sets and the number of crossings in the new family goes down by at least one (as we saw in Lemma 12.8 previously). Suppose we replace S_1, S_2 by $S_1 \cap S_2, S_1 \cup S_2$. The technical issue is to argue linear independence of the vectors in the new family. This is where we need the property $\chi_{S_1} + \chi_{S_2} = \chi_{S_1 \cap S_2} + \chi_{S_1 \cup S_2}$. Although natural, the linear algebraic argument turns out to be a bit tedious.

Instead we use a slick argument. Let $\mathcal{L}$ be a *maximal* laminar family of x -tight sets such that the vectors $\chi_S, S \in \mathcal{L}$ are linearly independent. If $|\mathcal{L}| = m$ then we are done because we have m linearly independent vectors that together span $\mathbb{R}^m$. Suppose $|\mathcal{L}| < m$. Then there must be a tight set S such that χ_S is not spanned by the vectors in $\mathcal{L}$. Choose a tight set S that is not spanned and crosses the fewest number of sets from $\mathcal{L}$. Since $\mathcal{L}$ is maximal, there must be some set $T \in \mathcal{L}$ such that S, T cross (otherwise we can add S to $\mathcal{L}$). Here we use Lemma 13.4 and consider two cases. Suppose $S \cap T, S \cup T$ are tight. Note that $S \cap T, S \cup T$ cross fewer sets in $\mathcal{L}$ than S does. By the choice of S , it must be the case that both $S \cap T$ and $S \cup T$ are spanned by $\mathcal{L}$. However, we have $\chi_S + \chi_T = \chi_{S \cap T} + \chi_{S \cup T}$ which implies that χ_S is also spanned, a contradiction. The proof for the other case when $S - T$ and $T - S$ are tight is similar. Thus we have $|\mathcal{L}| = m$ and this is the desired family. ■

13.2.2 Counting argument

The second key ingredient in the proof is a counting argument that exploits Lemma 13.3. An easier counting argument shows that there is an edge with $x_e \geq 1/3$ in any basic feasible solution. The tight bound of $1/2$ is more delicate and Jain's original proof is perhaps a bit hard to understand (see [250]). The argument has been subsequently refined and a "fractional token" based analysis [210] was developed and this is the proof in [255]. The token based analysis is flexible and powerful in iterated rounding based algorithms. In an attempt to make the proof even more transparent, the author of these notes developed yet another proof in [80]. We describe that proof below.

The proof is via contradiction where we assume that $0 < x_e < \frac{1}{2}$ for each $e \in E$. We call the two nodes incident to an edge as the endpoints of the edges. We say that an endpoint u *belongs* to a set $S \in \mathcal{L}$ if S is the minimal set from $\mathcal{L}$ that contains u .

We consider the simplest setting where $\mathcal{L}$ is a collection of disjoint sets, in other words, all sets are maximal. In this case the counting argument is easy. Let $m = |E| = |\mathcal{L}|$. For each $S \in \mathcal{L}$, $f(S) \geq 1$ and $x(\delta(S)) = f(S)$. If we assume that $x_e < \frac{1}{2}$ for each e , we have $|\delta(S)| \geq 3$ which implies that each S contains at least

3 distinct endpoints. Thus, the m disjoint sets require a total of $3m$ endpoints. However the total number of endpoints is at most $2m$ since there are m edges, leading to a contradiction.

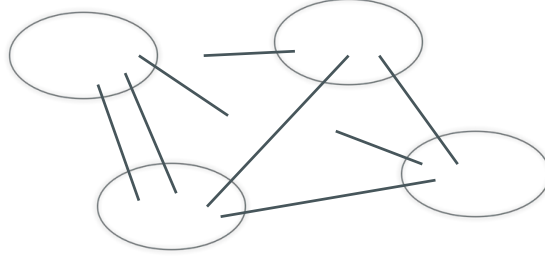


Figure 13.4: Easy case of counting argument.

Now we consider a second setting where the forest associated with $\mathcal{L}$ has k leaves and h internal nodes but each internal node has at least two children. In this case, following Jain, we can easily prove a weaker statement that $x_e \geq 1/3$ for some edge e . If not, then each leaf set S must have four edges leaving it and hence the total number of endpoints must be at least $4k$. However, if each internal node has at least two children, we have $h < k$ and since $h + k = m$ we have $k > m/2$. This implies that there must be at least $4k > 2m$ endpoints since the leaf sets are disjoint. But m edges can have at most $2m$ endpoints. Our assumption on each internal node having at least two children is obviously a restriction. So far we have not used the fact that the vectors $\chi_S, S \in \mathcal{L}$ are linearly independent. We can handle the general case to prove $x_e \geq 1/3$ by using the following lemma.

Lemma 13.5. *Suppose C is a unique child of S . Then there must be at least two endpoints in S that belong to S .*

Proof. If there is no endpoint that belongs to S then $\delta(S) = \delta(C)$ but then χ_S and χ_C are linearly dependent. Suppose there is exactly one endpoint that belongs to S and let it be the endpoint of edge e . But then $x(\delta(S)) = x(\delta(C)) + x_e$ or $x(\delta(S)) = x(\delta(C)) - x_e$. Both cases are not possible because $x(\delta(S)) = f(S)$ and $x(\delta(C)) = f(C)$ where $f(S)$ and $f(C)$ are positive integers while $x_e \in (0, 1)$. Thus there are at least two endpoints that belong to S . ■

Using the preceding lemma we prove that $x_e \geq 1/3$ for some edge e . Let k be the number of leaves in $\mathcal{L}$ and h be the number of internal nodes with at least two children and let ℓ be the number of internal nodes with exactly one child. We again have $h < k$ and we also have $k + h + \ell = m$. Each leaf has at

least four endpoints. Each internal node with exactly one child has at least two endpoints which means the total number of endpoints is at least $4k + 2\ell$. But $4k + 2\ell = 2k + 2k + 2\ell > 2k + 2h + 2\ell > 2m$ and there are only $2m$ endpoints for m edges. In other words, we can ignore the internal nodes with exactly one child since there are two endpoints in such a node/set and we can effectively charge one edge to such a node.

We now come to the more delicate argument to prove the tight bound that $x_e \geq \frac{1}{2}$ for some edge e . We describe an invariant that effectively reduces the argument to the case where we can assume that $\mathcal{L}$ is a collection of leaves. This is encapsulated in the lemma below which requires some notation. Let $\alpha(S)$ be the number of sets of $\mathcal{L}$ contained in S including S itself. Let $\beta(S)$ be the number of edges whose *both* endpoints lie inside S . Recall that $f(S)$ is the requirement of S .

Lemma 13.6. *For all $S \in \mathcal{L}$, $f(S) \geq \alpha(S) - \beta(S)$.*

Assuming that the lemma is true we can do an easy counting argument. Let $R_1, R_2, \dots, R_h$ be the maximal sets in $\mathcal{L}$ (the roots of the forest). Note that $\sum_{i=1}^h \alpha(R_i) = |\mathcal{L}| = m$. Applying the claim to each R_i and summing up,

$$\sum_{i=1}^h f(R_i) \geq \sum_{i=1}^h \alpha(R_i) - \sum_{i=1}^h \beta(R_i) \geq m - \sum_{i=1}^h \beta(R_i).$$

Note that $\sum_{i=1}^h f(R_i)$ is the total requirement of the maximal sets. And $m - \sum_{i=1}^h \beta(R_i)$ is the total number of edges that cross the sets $R_1, \dots, R_h$. Let E' be the set of edges crossing these maximal sets. Now we are back to the setting with h disjoint sets and E' edges with $\sum_{i=1}^h f(R_i) \geq |E'|$. This easily leads to a contradiction as before if we assume that $x_e < \frac{1}{2}$ for all $e \in E'$. Formally, each set R_i requires $> 2f(R_i)$ edges crossing it from E' and therefore R_i contains at least $2f(R_i) + 1$ endpoints of edges from E' . Since $R_1, \dots, R_h$ are disjoint the total number of endpoints is at least $2 \sum_i f(R_i) + h$ which is strictly more than $2|E'|$.

Proof of Lemma 13.6. Thus, it remains to prove the claim which we do by inductively starting at the leaves of the forest for $\mathcal{L}$.

Case 1: S is a leaf node. We have $f(S) \geq 1$ while $\alpha(S) = 1$ and $\beta(S) = 0$ which verifies the claim.

Case 2: S is an internal node with k children $C_1, C_2, \dots, C_k$. See Fig 13.5 for the different types of edges that are relevant. E_{cc} is the set of edges with endpoints in two different children of S . E_{cp} is the set of edges that cross exactly one child but do not cross S . E_{po} is the set of edges that cross S but do not cross any of the

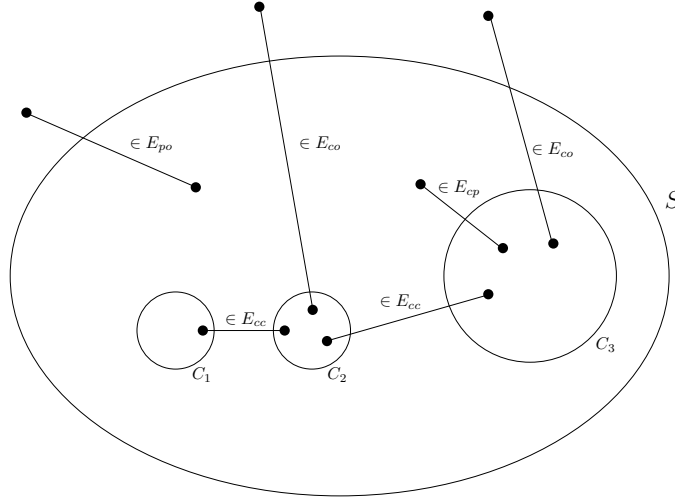


Figure 13.5: S is an internal node with several children. Different types of edges that play a role. p refers to parent set S , c refers to a child set and o refers to outside.

children. E_{co} is the set of edges that cross both a child and S . This notation is borrowed from [255].

Let $\gamma(S)$ be the number of edges whose both endpoints belong to S but not to any child of S . Note that $\gamma(S) = |E_{cc}| + |E_{cp}|$.

Then,

$$\begin{aligned}
 \beta(S) &= \gamma(S) + \sum_{i=1}^k \beta(C_i) \\
 &\geq \gamma(S) + \sum_{i=1}^k \alpha(C_i) - \left(\sum_{i=1}^k f(C_i) \right) \\
 &= \gamma(S) + \alpha(S) - 1 - \left(\sum_{i=1}^k f(C_i) \right)
 \end{aligned} \tag{13.1}$$

(13.1) follows by applying the inductive hypothesis to each child. From the preceding inequality, to prove that $\beta(S) \geq \alpha(S) - f(S)$ (the claim for S), it suffices to show the following inequality.

$$\gamma(S) \geq \left(\sum_{i=1}^k f(C_i) \right) - f(S) + 1. \tag{13.2}$$

The right hand side of the above inequality can be written as:

$$\left(\sum_{i=1}^k f(C_i) \right) - f(S) + 1 = \sum_{e \in E_{cc}} 2x_e + \sum_{e \in E_{cp}} x_e - \sum_{e \in E_{po}} x_e + 1. \quad (13.3)$$

We consider two subcases.

Case 2.1: $\gamma(S) = 0$. This implies that E_{cc} and E_{cp} are empty. Since $\chi(\delta(S))$ is linearly independent from $\chi(\delta(C_1)), \dots, \chi(\delta(C_k))$, we must have that E_{po} is not empty and hence $\sum_{e \in E_{po}} x_e > 0$. Therefore, in this case,

$$\left(\sum_{i=1}^k f(C_i) \right) - f(S) + 1 = \sum_{e \in E_{cc}} 2x_e + \sum_{e \in E_{cp}} x_e - \sum_{e \in E_{po}} x_e + 1 = - \sum_{e \in E_{po}} x_e + 1 < 1.$$

Since the left hand side is an integer, it follows that $\left(\sum_{i=1}^k f(C_i) \right) - f(S) + 1 \leq 0 = \gamma(S)$.

Case 2.2: $\gamma(S) \geq 1$. Recall that $\gamma(S) = |E_{cc}| + |E_{cp}|$.

$$\left(\sum_{i=1}^k f(C_i) \right) - f(S) + 1 = \sum_{e \in E_{cc}} 2x_e + \sum_{e \in E_{cp}} x_e - \sum_{e \in E_{po}} x_e + 1 \leq \sum_{e \in E_{cc}} 2x_e + \sum_{e \in E_{cp}} x_e + 1$$

By our assumption that $x_e < \frac{1}{2}$ for each e , we have $\sum_{e \in E_{cc}} 2x_e < |E_{cc}|$ if $|E_{cc}| > 0$, and similarly $\sum_{e \in E_{cp}} x_e < |E_{cp}|/2$ if $|E_{cp}| > 0$. Since $\gamma(S) = |E_{cc}| + |E_{cp}| \geq 1$ we conclude that

$$\sum_{e \in E_{cc}} 2x_e + \sum_{e \in E_{cp}} x_e < \gamma(S).$$

Putting together we have

$$\left(\sum_{i=1}^k f(C_i) \right) - f(S) + 1 \leq \sum_{e \in E_{cc}} 2x_e + \sum_{e \in E_{cp}} x_e + 1 < \gamma(S) + 1.$$

Since the term we are bounding is an integer, we get the desired inequality.

$$\left(\sum_{i=1}^k f(C_i) \right) - f(S) + 1 \leq \gamma(S).$$

■

Tightness of the analysis: The LP relaxation has an integrality gap of 2 even for the MST problem. Let G be the cycle on n vertices with all edge costs equal to 1. Then setting $x_e = 1/2$ on each edge is feasible and the cost is $n/2$ while the MST cost is $n - 1$. Note that the optimum fractional solution here is $1/2$ -integral. However, there are more involved examples (see Jain's paper or [250]) based on the Petersen graph where the optimum basic feasible solution is not half-integral while there are one or more edges with fractional value at least $1/2$. Jain's iterated rounding algorithm is an unusual algorithm in that the output of the algorithm may not have any discernible structure until it is completely done.

Running time: The strength of the iterated rounding approach is the remarkable approximation guarantees it delivers for various problems. The weakness is the high running time which is due to two reasons. First, one needs a basic feasible solution for the LP — this is typically much more expensive than finding an approximately good feasible solution. Second, the algorithm requires computing an LP solution many times. Finding faster algorithms with comparable approximation guarantees is an open research area.

Chapter 14

Introduction to Cut and Partitioning Problems

Graph cut and partitioning problems such as the well-known s - t minimum cut problem play a fundamental role in combinatorial optimization. Many natural cut problems that go beyond the s - t cut problem are NP-Hard and there has been extensive work on approximation algorithms and heuristics since they arise in many applications. In addition to algorithms, the structural results that capture approximate relationships between flows and cuts (called flow-cut gaps), and the connections to the theory of metric embeddings as well as graph theory have led to many beautiful and important results.

14.1 s - t mincut via LP Rounding and Maxflow-Mincut

Let $G = (V, E)$ be a directed graph with edge costs $c : E \rightarrow \mathbb{R}_+$. Let $s, t \in V$ be distinct vertices. The s - t mincut problem is to find the cheapest set of edges $E' \subseteq E$ such that there is no s - t path in $G - E'$. An s - t cut is often also defined as $\delta^+(S)$ for some $S \subset V$ where $s \in S, t \in V - S$. Suppose E' is an s - t cut. Let S be the set of nodes reachable from s in $G - E'$, then $\delta(S) \subseteq E'$ and moreover $\delta(S)$ is an s - t cut. Thus, it suffices to focus on such limited type of cuts, however in some more general settings it is useful to keep these notions separate.

It is well-known that s - t mincut can be computed efficiently via s - t maximumflow which also establishes the maxflow-mincut theorem. This is a fundamental theorem in combinatorial optimization with many direct and indirect applications.

Theorem 14.1. *Let $G = (V, E)$ be a directed graph with rational edge capacities $c : E \rightarrow \mathbb{Q}_+$ and let $s, t \in V$ be distinct vertices. Then the s - t maximum flow value in G*

is equal to the s - t minimum cut value and both can be computed in strongly polynomial time. Further, if c is integer valued then there exists an integer-valued maximum flow.

The proof of the preceding theorem is typically established via the augmenting path algorithm for computing a maximum flow. Here we take a different approach to finding an s - t cut via an LP relaxation whose dual can be seen as the maxflow LP.

Suppose we want to find an s - t mincut. We can write it as an integer program as follows. For each edge $e \in E$ we have a boolean variable $x_e \in \{0, 1\}$ to indicate whether we cut e . The constraint is that for any path $P \in \mathcal{P}_{s,t}$ (here $\mathcal{P}_{s,t}$ is the set of all s - t paths) we must choose at least one edge from P . This leads to the following IP.

$$\begin{aligned} \min \quad & \sum_{e \in E} c(e)x_e \\ \text{s.t.} \quad & \sum_{e \in P} x_e \geq 1 \quad P \in \mathcal{P}_{s,t} \\ & x_e \in \{0, 1\} \quad e \in E. \end{aligned}$$

The LP relaxation is obtained by changing $x_e \in \{0, 1\}$ to $x_e \geq 0$ since we can omit the constraints $x_e \leq 1$. We note that the LP has an exponential number of constraints, however, we have an efficient separation oracle since it corresponds to computing the shortest s - t path. The LP can be viewed as assigning lengths to the edges such that the shortest path between s and t according to the lengths is at least 1. This is a fractional relaxation of the cut.

Rounding the LP: We will prove that the LP relaxation can be rounded without any loss! The rounding algorithm is described below.

Theta-Rounding(G, s, t)

1. Solve LP to obtain fractional solution y
2. For each $v \in V$ let $d_y(s, v)$ be the shortest path distance from s to v according to edge lengths y_e .
3. Pick θ uniformly at random from $(0, 1)$
4. Output $E' = \delta^+(B(s, \theta))$ where $B(s, \theta) = \{v \mid d_y(s, v) \leq \theta\}$ is the ball of radius θ around s

It is easy to see that the algorithm outputs a valid s - t since $d_y(s, t) \geq 1$ by feasibility of the LP solution y and hence $t \notin B(s, \theta)$ for any $\theta < 1$.

Lemma 14.1. *Let $e = (u, v)$ be an edge. $\mathbf{P}[e \text{ is cut by algorithm}] \leq y(u, v)$.*

Proof. An edge $e = (u, v)$ is cut iff $d_y(s, u) \leq \theta < d_y(s, v)$. Hence the edge is not cut if $d_y(s, v) \leq d_y(s, u)$. If $d_y(s, v) > d_y(s, u)$ we have $d_y(s, v) - d_y(s, u) \leq y(u, v)$. Since θ is chosen uniformly at random from $(0, 1)$ the probability that θ lies in the interval $[d_y(s, u), d_y(s, v)]$ is at most $y(u, v)$. ■

Corollary 14.2. *The expected cost of the cut output by the algorithm is at most $\sum_e c(e)y_e$.*

The preceding corollary shows that there is an integral cut whose cost is at most that of the LP relaxation which implies that the LP relaxation yields an optimum solution. The algorithm can be easily derandomized by trying “all possible value of θ ”. What does this mean? Once we have y we compute the shortest path distances from s to each vertex v . We can think of these distances as producing a *line embedding* where we place s at 0 and each vertex v at $d_y(s, v)$. The only interesting choices for θ are given by the n values of $d_y(s, v)$ and one can try each of them and the corresponding cut and find the cheapest one. It is guaranteed to be at most $\sum_e c(e)y_e$.

What is the dual LP? We write it down below and you can verify that it is the path version of the maxflow!

$$\begin{aligned} \max_{P \in \mathcal{P}_{s,t}} z_P \\ \sum_{P: e \in P} z_P &\leq c(e) \quad e \in E \\ z_P &\geq 0 \quad P \in \mathcal{P}_{s,t}. \end{aligned}$$

Thus, we have seen a proof of the maxflow-mincut theorem via LP rounding of a relaxation for the s - t cut problem.

A compact LP via distance variables: The path based LP relaxation for the s - t mincut problem is natural and easy to formulate. We can also express shortest path constraints via distance variables. We first write a bigger LP than necessary via variables $d(u, v)$ for all ordered pairs of vertices (hence there are n^2 variables). We need triangle inequality constraints to enforce that $d(u, v)$ values respect shortest path distances.

$$\begin{aligned} \min_{(u,v) \in E} c(u, v) d(u, v) \\ d(u, v) + d(v, w) - d(u, w) &\geq 0 \quad u, v, w \in V \\ d(s, t) &\geq 1 \\ d(u, v) &\geq 0 \quad (u, v) \in V \times V \end{aligned}$$

Although the preceding LP is wasteful in some ways it is quite generic and can be used for many cut problems where we are interested in distances between multiple pairs of vertices.

Now we consider a more compact LP formulation. We have two types of variables, $x(u, v)$ for each edge $(u, v) \in E$ and d_v variables for each $v \in V$ to indicate distances from s .

$$\begin{aligned} \min_{(u,v) \in E} \quad & c(u, v)x(u, v) \\ & d_v \leq d_u + x(u, v) \quad (u, v) \in E \\ & d_t \geq 1 \\ & d_v \geq 0 \quad v \in V \\ & x(u, v) \geq 0 \quad (u, v) \in E \end{aligned}$$

Exercise 14.1. Write the dual of the above LP and see it as the standard edge-based flow formulation for s - t maximum flow.

14.2 A Catalog of Cut and Partitioning Problems

Here we list a few prominent problems.

Multiway Cut: Input is an undirected graph $G = (V, E)$ and edge weights $w : E \rightarrow \mathbb{R}_+$ and a set of k terminals $\{s_1, s_2, \dots, s_k\} \subseteq V$. The goal is to remove a minimum weight subset of edges such that there is no path from s_i to s_j for any $i \neq j$. This problem is NP-Hard even for $k = 3$. **Node-weighted Multiway Cut** is the generalization to the setting when G has node-weights instead of edge-weights. **Directed Multiway Cut** is the version where G is a directed graph and the goal is to remove a minimum weight set of edges so that there is no s_i - s_j path for any $i \neq j$. Note that the directed version generalizes the node-weighted undirected problem as well.

k -Cut: In k -Cut the input is a graph G and integer k . The goal is to remove a minimum weight set of edges such that the resulting graph has at least k non-trivial components. The problem is NP-Hard when k is part of the input but admits a polynomial time algorithm for any fixed k . A common generalization of k -Cut and Multiway Cut is the so-called **Steiner k -Cut**: here the input is a graph G , a set of terminals $S \subseteq V$ and an integer k where $k \leq |S|$. The goal is to remove a minimum-weight subset of edges such that there are at least k components that each contain a terminal.

Multicut: The input is an edge-weighted graph $G = (V, E)$ and k source-sink pairs $(s_1, t_1), (s_2, t_2), \dots, (s_k, t_k)$ and the goal is to remove a minimum-weight set of edges so that there is no path from s_i to t_i for all $i \in [k]$. Note that Multiway Cut is a special case. Multicut also naturally generalizes to the node-weighted setting and to directed graphs.

Sparsest Cut: We discuss the more general version first called the *Non-uniform Sparsest Cut*. The input is an edge-capacitated graph $G = (V, E)$ and k source-sink pairs $(s_1, t_1), \dots, (s_k, t_k)$ and associated non-negative scalar demand values $D_1, D_2, \dots, D_k$. The goal is to find a minimum-weight subset of edges $E' \subseteq E$ such the ratio $\frac{c(E')}{D(E')}$ is minimized where $c(E')$ is the total capacity of edges in E' and $D(E')$ is the total demand of pairs separated by removing E' . In undirected graphs one can see that this ratio is also minimized by a connected component and hence one can alternatively phrase the problem as $\min_{S \subseteq V} \frac{c(\delta(S))}{D(\delta(S))}$. In the *Uniform Sparsest Cut* problem we associate $D(u, v) = 1$ for every pair of vertices and hence one wants to find $\min_{S \subseteq V} \frac{c(\delta(S))}{|S||V-S|}$. This is closely related (to within a factor of 2) to the problem of finding the *expansion* of a graph where the objective is $\min_{S \subseteq V, |S| \leq |V|/2} \frac{c(\delta(S))}{|S|}$. Other closely related variants are to find the *conductance* and sparsest cut for *product* multicommodity flow instances where the demands are induced by vertex weights (that is, $D(u, v) = \pi(u)\pi(v)$ where $\pi : V \rightarrow \mathbb{R}_+$).

Sparsest Cut can be generalized to node-weighted settings and to directed graphs. In directed graphs there is a substantial difference when considering the non-uniform versus the uniform settings because the latter can be thought of a symmetric version. We will not detail the issues here.

Minimum Bisection and Balanced Partitioning: The input is an undirected edge-weighted graph G . In Minimum Bisection the goal is to partition V into V_1, V_2 where $\lfloor |V|/2 \rfloor \leq |V_1|, |V_2| \leq \lceil |V|/2 \rceil$ so as to minimize the weight of the edges crossing the partition. In Balanced Partition the sizes of the two parts can be approximately equal — there is a balance parameter $\alpha \in (0, 1/2]$ and the goal is to partition V into V_1, V_2 such that $\alpha|V| \leq |V_1|, |V_2| \leq (1 - \alpha)|V|$. These problems are partially motivated by parallel and distributed computation where a graph representing some computation is recursively decomposed into several pieces while minimizing the communication required between the pieces (captured by the edge weights). In this context partitioning into k given pieces to minimize the number of edges while each piece has size roughly $|V|/k$ is also considered as Balanced k -Partition.

Hypergraphs and Submodular functions: One can generalize several edge-weighted problems to node-weighted problems in a natural fashion but in some cases it is useful to consider other ways to model. In this context hypergraphs come in handy and have their own intrinsic appeal. Recall that a hypergraph

$H = (V, E)$ consists of a vertex set V and a set of hyperedges E where each $e \in E$ is a subset of vertices; thus $e \subseteq V$. The rank of a hypergraph is the maximum cardinality of a hyperedge, typically denote by r . Graphs are rank 2 hypergraphs. One can typically reduce a hypergraph cut problem to a node-weighted cut problem and vice-versa with some distinctions based on the specific problem at hand. Finally some of the problems can be naturally lifted to the setting where we consider an abstract submodular set function defined over the vertex set V ; that is we are given $f : 2^V \rightarrow \mathbb{R}_+$ rather than a graph or a hypergraph and the goal is to partition V where the cost of the partition is now measured with respect to f .

Chapter 15

Multiway Cut

MULTIWAY CUT problem is the following¹. Given a graph $G = (V, E)$ with edge weights $w : E \rightarrow \mathbb{R}_+$, and k terminal vertices $\{s_1, s_2, \dots, s_k\}$, remove a minimum weight set of edges such that there is no s_i - s_j path left for any $i \neq j$. We phrase it in this long-winded way since the definition then naturally generalizes to other settings. In the case of undirected graphs it is also useful to view the problem as a *partition* problem. Consider a feasible solution to the given instance; it consists of a set of edges whose removal leaves at least k components and no two terminals are in the same connected component. One can verify that in a minimal solution, if the original graph is connected, then we can assume that there will be exactly k components with each terminal contained in a separate one. Thus, an alternative view is to find a partition of V into k sets $V_1, V_2, \dots, V_k$ such that $s_i \in V_i$ for $1 \leq i \leq k$ and the goal is to minimize $\sum_i w(\delta(V_i))/2$ where the factor of 2 is because any edge crossing the partition is counted exactly twice in $\sum_i w(\delta(V_i))/2$. When $k = 2$ we can solve the problem in polynomial time since it is the same problem as finding an s_1 - s_2 minimum cut in G . The problem is known to be NP-Hard and APX-Hard even when $k = 3$ [89]. In this chapter we first consider a simple combinatorial heuristic that yields a $2(1 - 1/k)$ -approximation, followed by a distance based LP relaxation that also yields a $2(1 - 1/k)$ -approximation with a matching integrality gap. We then describe a geometric relaxation that yields the current best approximation ratio of 1.2965 [233] (the best known lower bound on the integrality gap for this relaxation is 1.20016 [40]). We will also discuss node-weighted and directed versions in the last section.

¹In the literature the problem is also referred to as the *Multiterminal Cut* problem.

15.1 Isolating Cut Heuristic

Let $S = \{s_1, s_2, \dots, s_k\}$ be the given set of terminals. A feasible solution to a given instance of MULTIWAY CUT separates each terminal s_i from the terminals $S - \{s_i\}$. One can find the cheapest cut separating s_i from $S - \{s_i\}$ via a minimum-cut computation (we shrink $S - \{s_i\}$ to a single vertex t and compute an s - t minimum cut). Taking the union of such cuts for each $i \in [k]$ clearly yields a feasible solution. It turns out that this simple heuristic is not too bad.

IsolatingCut($G = (V, E), S = \{s_1, \dots, s_k\}$)

1. for each $i \in [k]$ do
 - A. Let E_i be a minimum weight cut separating s_i from $S - \{s_i\}$ in G
2. Without loss of generality $w(E_1) \leq w(E_2) \leq \dots \leq w(E_k)$ (otherwise reindex)
3. Output $\cup_{i=1}^{k-1} E_i$

We leave the following as an easy exercise.

Lemma 15.1. *The algorithm outputs a feasible solution.*

Theorem 15.1. *The Isolating Cut heuristic yields a $2(1 - 1/k)$ -approximation for MULTIWAY CUT.*

Proof. Recall the partition view of the MULTIWAY CUT problem. Let E^* be an optimum and minimal solution and let $V_1, V_2, \dots, V_k$ be the components of $G - E^*$ such that $s_i \in V_i$ for $i \in [k]$. Each edge $e \in E^*$ crosses the partition and since it has exactly two end points, we see that $\sum_{i=1}^k w(\delta(V_i)) = 2w(E^*)$.

We claim that for each $i \in [k]$ $w(E_i) \leq w(\delta(V_i))$. This follows from the fact that $\delta(V_i)$ is a cut that separates s_i from $S - \{s_i\}$. Therefore $\sum_{i=1}^k w(E_i) \leq \sum_{i=1}^k w(\delta(V_i)) = 2w(E^*)$. Since $w(E_k) \geq w(E_i)$ for all $i \in [k]$, $\sum_{i=1}^{k-1} w(E_i) \leq (1 - 1/k) \sum_{i=1}^k w(E_i)$, and hence we have $\sum_{i=1}^{k-1} w(E_i) \leq 2(1 - 1/k)w(E^*) = 2(1 - 1/k) \text{OPT}$. ■

Exercise 15.1. The analysis of the algorithm is tight. Find an example to demonstrate this.

15.2 Distance based LP Relaxation

We describe an LP relaxation based on distance/length variables that generalizes the relaxation for s - t cut that we saw previously. For each edge e there is a

length variable x_e indicating whether e is cut or not. We require that the length of any path p connecting s_i and s_j , $i \neq j$, should be at least 1.

$$\begin{aligned} \min \quad & \sum_{e \in E} c_e x_e \\ \text{s.t.} \quad & \sum_{e \in p} x_e \geq 1 \quad p \in \mathcal{P}_{s_i, s_j}, i \neq j \\ & x_e \geq 0 \quad e \in E \end{aligned}$$

The preceding LP can be solved in polynomial time via the Ellipsoid method since the separation oracle is the shortest path problem. Alternatively, one can write a compact LP. We focus on rounding the LP and establishing its integrality gap.

BallCut($G = (V, E)$, $S \subseteq V$)

1. Solve distance based LP relaxation to obtain solution x
2. Let d_x be the metric induced on V by edge lengths x
3. Pick θ uniformly at random from $(0, 1/2)$
4. Output $\cup_{i=1}^k \delta(B_d(s_i, \theta))$ where $B(s_i, \theta)$ is the ball of radius θ around s_i

Lemma 15.2. *The algorithm outputs a feasible solution.*

Proof. Consider a terminal s_i . $B_d(s_i, \theta)$ does not contain any other terminal s_j , $j \neq i$ since $\theta < 1/2$ and $d(s_i, s_j) \geq 1$. Thus removing the edges $\delta(B_d(s_i, \theta))$ disconnects s_i from every other terminal. Since this holds for each s_i taking the union of the edges $\cup_i \delta(B_d(s_i, \theta))$ disconnects each terminal from all other terminals. ■

Consider the open balls of radius $1/2$ around the terminals, that is, $B_d(s_i, 1/2)$ for $i \in [k]$. We observe that they are disjoint for if $v \in B(s_i, 1/2) \cap B(s_j, 1/2)$ with $i \neq j$ then $d(s_i, s_j) < 1$ which would violate the LP constraint. Thus the algorithm is essentially running an s - t cut type algorithm in the disjoint balls but since we only have half the radius we lose a factor of 2. In fact, as we will see in a remark later, the analysis holds even if each s_i chose its own θ_i independently — the correlated choice is not exploited here — but a correlated choice is quite useful when considering other cut problems including DIRECTED MULTIWAY CUT later in the chapter.

Lemma 15.3. *Let $e = (u, v)$. $\mathbf{P}[e \text{ is cut}] \leq 2x_e$.*

Proof. There are several cases to consider but the main one is the one where both $u, v \in B_d(s_i, 1/2]$ for some s_i . Suppose this is the case. Then only s_i can cut the edge (u, v) . It is easy to see that $\mathbf{P}[e \text{ is cut}] = 2|d(s_i, u) - d(s_i, v)|$ since we pick θ uniformly at random from $(0, 1/2)$. But $|d(s_i, u) - d(s_i, v)| \leq x_e$ by triangle inequality and hence we obtain the desired claim.

Now we consider the case when $u \in B(s_i, 1/2)$ and $v \in B(s_j, 1/2)$ where $i \neq j$. Let $\alpha = 1/2 - d(s_i, u)$ and let $\beta = 1/2 - d(s_j, v)$. We observe that $\alpha + \beta \leq x_e$ for otherwise $d(s_i, u) + x_e + d(s_j, v) < 1$. We see that e is cut iff θ lies in the interval $[d(s_i, u), 1/2]$ or it lies in the interval $[d(s_j, v), 1/2]$. Thus e is cut with probability $2 \max(\alpha, \beta) \leq 2(\alpha + \beta) \leq 2x_e$.

There are two other cases. One is when both u, v are outside every half-ball. In this case the edge e is not cut. The other is when $u \in B_d(s_i, 1/2)$ for some i and v is not inside any ball. The analysis here is similar to the second case and we leave it as an exercise. ■

Thus the expected cost of the cut is at most $2 \sum_e c_e x_e \leq 2 \text{OPT}_{LP} \leq 2 \text{OPT}$. One can improve and obtain a $2(1 - 1/k)$ -approximation by saving on one terminal as we did in the preceding section.

Exercise 15.2. Modify the algorithm to obtain a $2(1 - 1/k)$ -approximation with respect to the LP relaxation.

Exercise 15.3. Consider a variant of the algorithm where each s_i picks an independent $\theta_i \in (0, 1/2)$; we output the cut $\cup_i \delta(B_d(s_i, \theta_i))$. Prove that this also yields a 2-approximation (can be improved to $2(1 - 1/k)$ -approximation).

Integrality gap: The analysis is tight as shown by the following integrality gap example. Consider a star with center r and k leaves $s_1, s_2, \dots, s_k$ which are the terminals. All edges have cost 1. Then it is easy to see that a feasible integral solution consists of removing $k - 1$ edges from the star. Hence $\text{OPT} = k - 1$. On the other hand, setting $x_e = 1/2$ for each edge is a feasible fractional solution of cost $k/2$. Hence the integrality gap is $2(1 - 1/k)$.

15.3 A Partitioning View and Geometric Relaxation

In this section we describe improved approximation algorithms for MULTIWAY CUT problem via a strengthened LP relaxation. It is useful to take another perspective on the LP that we saw in the previous section and write it with distance variables. For this purpose we can assume without loss of generality that G is a complete graph (we can add dummy edges of zero cost to achieve this). For each *unordered* pair uv of distinct nodes we have a variable d_{uv} ; the variable denotes the distance between u and v which is supposed to be 1 if u

and v are separated and 0 otherwise. The relaxation requires d to be a metric subject to the constraint that $d_{s_i s_j} = 1$ for each pair of terminals $s_i s_j$. Triangle inequality constraints ensure that d is a metric.

$$\begin{aligned} \min \sum_{uv} c_{uv} d_{uv} \\ d(s_i s_j) &= 1 \quad i \neq j \\ d_{uv} + d_{vw} &\geq d_{uw} \quad u, v, w \\ d_{uv} &\geq 0 \quad uv \end{aligned}$$

It is not hard to convince oneself that the two relaxations are equivalent in terms of the optimum cost; we note that the two relaxations are very different in the polyhedral sense. Syntactically it is sometimes more convenient to use variable $d_{u,v}$ for the ordered pair (u, v) and impose symmetry explicitly in the constraints $d_{u,v} = d_{v,u}$ and then let d_{uv} stand for the common variable.

We saw that the worst case integrality gap of the preceding relaxations is $2(1 - 1/k)$. Calinescu, Karloff and Rabani [53] were the first to obtain an approximation ratio better than 2 for MULTIWAYCUT, and they achieved this via novel LP relaxation that has since been extensively studied. Until very recently the current best upper bound on the integrality gap of this relaxation (henceforth called the CKR relaxation) was 1.2965 [233]. A very recent result obtains a 1.2787-approximation [45]. The best known lower bound on the relaxation is $1.20016 - \epsilon$ when $k \rightarrow \infty$ [40]. Moreover, it is known, that under the unique games conjecture (UGC) [199], for every fixed k , the integrality gap of the CKR relaxation is the worst-case hardness of the problem! We do not know the precise worst-case integrality gap of the CKR relaxation for $k \geq 4$. For $k = 3$ it is known to be $12/11$ [83, 168].

In this notes we describe the CKR relaxation and a rounding scheme that achieves a 1.5 approximation which was the bound initially shown in [53] (they show $(1.5 - 1/k)$). This analysis is given in the approximation text books. The main purpose of this lecture notes is to highlight a different analysis which is based on the work of Chekuri and Ene [69] on the Submodular Multiway Partition (SUBMP) problem which generalizes MULTIWAYCUT; they proved that the bound of 1.5 can be obtained for all symmetric functions. We believe that it gives a different perspective and also eliminates some of the intricacies in the standard analysis, at least as far as obtaining an approximation ratio of 1.5 is concerned. Improving this bound requires more intricate and messy analysis that takes the graph aspect into account.

15.3.1 CKR Relaxation

The CKR relaxation is typically described as a geometric relaxation and this is well-justified. However, there are two other “natural” ways to derive it and we will discuss them later. The basic observation that leads to the relaxation is the following. It is useful to view **MULTIWAYCUT** in undirected graphs as a *partition* problem rather than as an edge-removal problem. We can assume without loss of generality that G is connected. We observe that if $E' \subseteq E$ is a minimal feasible solution to disconnect the terminals, then $G - E'$ has exactly k connected components with exactly one terminal in each. Moreover, each edge in E' connects two of these components. Thus, we can reformulate **MULTIWAYCUT** in the following equivalent way. Find a partition $V_1, V_2, \dots, V_k$ of V such that $s_i \in V_i$ for $1 \leq i \leq k$, where the objective is to $\min \frac{1}{2} \sum_{i=1}^k c(\delta(V_i))$. Once we have this partition based formulation, it is natural to use variables $x(u, i)$ for each vertex u and index $i \in [k]$; the variable indicates whether u belongs to V_i . We then write the following constraints that model the requirement that each u belongs exactly to one piece of the partition and that $s_i \in V_i$.

$$\begin{aligned} \sum_{i=1}^k x(u, i) &= 1 & u \in V \\ x(s_i, i) &= 1 & i \in [k] \\ x(u, i) &\geq 0 & u \in V, i \in [k] \end{aligned}$$

How do we model the objective? Consider an edge uv and assume all the allocation variables are binary and satisfy the constraints. uv is cut iff there exists an i such that $x(u, i) \neq x(v, i)$. But if there is such an i then there will be an $i' \neq i$ such that $x(u, i') \neq x(v, i')$. Thus, one can see that uv is cut iff $\sum_{i=1}^k |x(u, i) - x(v, i)| = 2$. Hence the objective becomes

$$\min \sum_{uv \in E} c_{uv} \frac{1}{2} \sum_{i=1}^k |x(u, i) - x(v, i)|$$

The objective function involves the absolute value function and hence is not linear. Since we are minimizing, we can obtain an LP, with the help of additional variables $z_{uv,i}$ as follows:

$$\begin{aligned}
\min \sum_{uv \in E} c_{uv} \frac{1}{2} \sum_{i=1}^k z_{uv,i} \\
\sum_{i=1}^k x(u, i) &= 1 \quad u \in V \\
x(s_i, i) &= 1 \quad i \in [k] \\
z_{uv,i} &\geq x(u, i) - x(v, i) \quad uv \in E, i \in [k] \\
z_{uv,i} &\geq x(v, i) - x(u, i) \quad uv \in E, i \in [k] \\
x(u, i) &\geq 0 \quad u \in V, i \in [k] \\
z_{uv,i} &\geq 0 \quad uv \in E, i \in [k]
\end{aligned}$$

Geometric interpretation: Perhaps the most useful interpretation, as far as MULTIWAYCUT is considered, is the geometric interpretation. Consider the k unit basis vectors $\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_k$ in R^k (here $\mathbf{e}_i$ is the vector with 1 in the i 'th coordinate and 0 in all other coordinates). The convex hull of these k vectors creates the k -simplex and it is easy to see that for each $u \in V$, the vector $x_u = (x(u, 1), x(u, 2), \dots, x(u, k))$ lies in this convex hull. Note that the simplex lies on a hyperplane of R^k , and hence is not full-dimensional. The relaxation is embedding the terminals at the corners of the k -simplex, and mapping each node u to a point within the simplex. The distance between u and v is half the ℓ_1 distances between the vectors x_u and x_v . The advanced rounding algorithms for MULTIWAYCUT via the CKR relaxation make very strong use of the geometric view point, as well as the special structure enjoyed by the objective function for graphs, which decomposes in a nice fashion.

Interpretation as strengthening the distance relaxation: Another nice interpretation of the CKR relaxation (also from [53]) is as follows. We already discussed the basic distance LP and showed that it has a worst case integrality gap of $2(1 - 1/k)$. It is natural to consider *strengthening* the LP by adding inequalities that are *valid* for the integer program. Consider a node u . If we take the partition view then we note that u must belong to one of the components, which implies that it must have a distance 0 to the terminal whose component it belongs to, and distance 1 to the other $k - 1$ terminals. Thus the following inequalities are valid for the integer program:

$$\sum_{i=1}^k d(us_i) = k - 1 \quad u \in V$$

We can also add another set of valid inequalities as follows. Consider distinct nodes $u, v \in V$. Let $T \subset [k]$ correspond to a subset of terminals. Then we claim that

$$d(uv) \geq \sum_{i \in T} [d(us_i) - d(vs_i)]$$

is a valid constraint. We leave it as an exercise to check this inequality's validity when the distances are induced by partitions. Note that there are an exponential number of inequalities of the second type since we have to write it for each edge uv and each $T \subset [k]$. It is possible to separate over these inequalities in polynomial time (see [53] or think about it). Interestingly, if we add the preceding two sets of valid inequalities to the basic distance LP we get the CKR relaxation!

Interpretation via Lovász-extension of submodular functions: Recall that the objective function that came out naturally from the allocation variables $x(u, i)$ was the following.

$$\min \sum_{uv \in E} c_{uv} \frac{1}{2} \sum_{i=1}^k |x(u, i) - x(v, i)|$$

We had to add additional variables to model this objective as a linear function. However, one can observe that the function $\sum_{uv \in E} c_{uv} \frac{1}{2} \sum_{i=1}^k |x(u, i) - x(v, i)|$ is convex in the variables (in fact it is a piece-wise linear convex function). Thus, we can model our problem as a convex optimization problem with linear constraints. Why is the objective convex? Is there a more general phenomenon going on here? Indeed, that is the case. We can cast MULTIWAYCUT as a special case of the following more general problem. Let $f : 2^V \rightarrow \mathbb{R}_+$ be a submodular function and let $\{s_1, s_2, \dots, s_k\} \subseteq V$. In SUBMP the goal is to partition V into $V_1, V_2, \dots, V_k$ such that $s_i \in V_i$ for $1 \leq i \leq k$, and to minimize $\sum_i f(V_i)$. Note that MULTIWAYCUT is the special case of SUBMP where f is the cut function of a graph $G = (V, E)$. In fact the cut function is not only submodular but also symmetric, and hence MULTIWAYCUT is a special case of SYMSUBMP (Symmetric Submodular Multiway Partition) which is obtained by restricting f to be a symmetric submodular function. It is natural to write a relaxation for SUBMP via the same allocation variables $x(u, i)$ as we did for MULTIWAYCUT. The main difficulty is with the objective function f . How do we deal with a generic submodular function that is only available as a value oracle? It turns out that there are useful ways to *extend* a submodular set function f over a discrete ground set V to a continuous function g over the entire real cube $[0, 1]^{|V|}$. Lovász described a particular extension $\hat{f}$, named after him, that turns out to be convex and useful in the sense that it can be evaluated at any point in the real cube efficiently given f as a value oracle.

One can thus write a convex relaxation for SUBMP (and hence also SYMSUBMP) via the Lovász-extension. When specialized to MULTIWAYCUT this relaxation turns out to be the same as the CKR relaxation! We refer the reader to [69] for more details.

15.3.2 Rounding Algorithm and Analysis

In this section we show a rounding algorithm that gives a 1.5 approximation for MULTIWAYCUT. A slightly more careful analysis will yield a bound of $(1.5 - 1/k)$. As we already mentioned better ratios are known. The algorithm and analysis here is an adaptation of the algorithm and analysis from [69] for the SYMSUBMP problem. The original algorithm and analysis from [53] can be found in the approximation text books. We will briefly discuss the original algorithm and its connection to our algorithm later.

The informal description is as follows. It picks a random $\theta \in [0, 1)$ and for each terminal s_i we consider A_i to be ball of radius $(1 - \theta)$ around s_i . We observe that $A_i = \{v \mid x(u, i) \geq \theta\}$ because

$$d(s_i v) = \frac{1}{2} \sum_{j=1}^k |x(s_i, j) - x(u, j)| = 1 - x(u, i).$$

Note that the sets $A_1, A_2, \dots, A_k$ can overlap. We say that v is unallocated if $v \in V \setminus (\cup_i A_i)$. Since we are seeking a partition we need to fix the overalapping sets. We do this via an uncrossing operation. If $A_i \cap A_j \neq \emptyset$ we replace them with either $A_i - A_j, A_j$ or $A_i, A_j - A_i$. We will argue that this does not increase the cost. This way of understanding the algorithm is based on [69]. The algorithm from [53] avoids the overlap by picking a random permutation of the terminals, and A_i is defined to set of all vertices in the ball of radius $(1 - \theta)$ around s_i that have not already been allocated to a terminal that came earlier in the permutation. The random permutation is implicitly using the fact that sets can be uncrossed without increasing the cost.

The analysis hinges on understanding the expected cost of $\delta(A_i), i \in [k]$ and the expected cost of $\delta(U)$. The algorithm is formally described below.

RoundCKR(G, S)

1. Let $\bar{x}$ be an optimum solution to the CKR relaxation on G with terminals S
2. Pick θ uniformly at random from $[0, 1)$
3. For $i = 1$ to k let $A_i = \{v \mid x(v, i) \geq \theta\}$
4. Let $U = V \setminus (\cup_{i=1}^k A_i)$ be the unallocated nodes
// Uncross $A_1, A_2, \dots, A_k$ so that they are disjoint
5. While there exist $i \neq j$ such that $A_i \cap A_j \neq \emptyset$ do
 - A. If $c(\delta(A_i)) + c(\delta(A_j - A_i)) \leq c(\delta(A_i)) + c(\delta(A_j))$ then
 1. $A_j \leftarrow A_j - A_i$
 - B. Else
 1. $A_i \leftarrow A_i - A_j$
6. Output the partition $A_1 \cup U, A_2, \dots, A_k$

Let OPT be the cost of the CKR relaxation. For the sake of the analysis we let $A_1, A_2, \dots, A_k$ denote the *random* sets obtained in the first randomized step of the algorithm, and let $A'_1, \dots, A'_k$ denote the sets after the uncrossing step. We make some basic observations that are easy to verify. They show that the final partition is indeed a valid solution to the problem.

- For each i , $s_i \in A_i$, and $s_i \in A'_i$.
- $\cup_i A_i = \cup_i A'_i$.
- The uncrossing step terminates in $O(k^2)$ iterations. Why?

The cost analysis is based on the following facts that we will prove.

- The expected cost of $\frac{1}{2} \sum_{i=1}^k c(\delta(A_i))$ is at most OPT . That is

$$\mathbb{E} \left[\frac{1}{2} \sum_{i=1}^k c(\delta(A_i)) \right] \leq \text{OPT}.$$

- The expected cost of $\frac{1}{2} c(\delta(U))$ is at most $\text{OPT}/2$, that is,

$$\mathbb{E}[c(\delta(U))] \leq \text{OPT}.$$

- The uncrossing steps do not increase the cost. That is, $\sum_i c(\delta(A'_i)) \leq \sum_i c(\delta(A_i))$. This is a deterministic statement for any sets $A_1, \dots, A_k$.

Exercise 15.4. Argue, assuming the above properties, that the expected cost of the final partition is at most 1.5OPT .

We now prove the desired properties. The analysis will be on an edge by edge basis. Since the objective function is separable over the edges we can use linearity of expectation for the overall bounds. Recall that $z_{uv,i} = |x(u, i) - x(v, i)|$ and let $z_{uv} = \sum_i z_{uv,i}$ be the ℓ_1 distance between x_u and x_v .

Lemma 15.4. $\mathbf{P}[uv \in \delta(A_i)] = z_{uv,i}$.

Proof. It is easy to see that $uv \in \delta(A_i)$ iff θ is in the interval

$$[\min\{x(u, i), x(v, i)\}, \max\{x(u, i), x(v, i)\}]$$

and this happens with probability $|x(u, i) - x(v, i)|$. ■

The preceding lemma easily shows, via linearity of expectation, that $\mathbf{E}[c(\delta(A_i))] = \sum_{uv} c_{uv} z_{uv,i}$ and establishes the first property.

We now show that uncrossing can be done without increasing the cost. Here we rely on submodularity and symmetry of the cut function.

Lemma 15.5. Let $f : 2^V \rightarrow \mathbb{R}$ be a symmetric submodular function. For any two sets $A, B \subset V$ we have $f(A) + f(B) \geq f(A - B) + f(B)$ or $f(A) + f(B) \geq f(A) + f(B - A)$.

Proof. Submodularity and symmetry implies posi-modularity for f . For any A, B

$$f(A) + f(B) \geq f(A - B) + f(B - A).$$

Hence

$$2f(A) + 2f(B) \geq [f(A) + f(B - A)] + [f(A - B) + f(B)].$$

This implies the desired claim. ■

Exercise 15.5. Using the preceding lemma argue that $\sum_i c(\delta(A'_i)) \leq \sum_i c(\delta(A_i))$.

Now we consider the probability that $uv \in \delta(U)$. For this purpose, for each node v , we let α_v denote the quantity $\max_i x(v, i)$.

Lemma 15.6. $\mathbf{P}[uv \in \delta(U)] = |\alpha_u - \alpha_v|$.

Proof. Note that $v \in U$ iff $\theta > \alpha_v$; if $\theta \leq \alpha_v$ v will belong to some A_i . Thus $uv \in \delta(U)$ only if exactly one of u, v is in U . This happens only if θ is in the interval $[\min\{\alpha_u, \alpha_v\}, \max\{\alpha_u, \alpha_v\}]$. The lemma follows since θ is chosen uniformly from $[0, 1]$. ■

Now, all that remains is the following technical lemma, which is not quite intuitive at first glance but whose proof is not difficult.

Lemma 15.7. $|\alpha_u - \alpha_v| \leq \frac{1}{2} \sum_i |x(u, i) - x(v, i)|$.

Proof. Let $P = \sum_{i: x(u, i) \geq x(v, i)} [x(u, i) - x(v, i)]$. Since x_u and x_v are in the simplex we have $P = \sum_{i: x(u, i) \leq x(v, i)} [x(v, i) - x(u, i)]$, and therefore, $P = \frac{1}{2} \sum_i |x(u, i) - x(v, i)|$.

Without loss of generality assume that $\alpha_u \geq \alpha_v$. Let $\alpha_u = x(u, h)$ for some h . Then $\alpha_v \geq x(v, h)$. We have

$$\alpha_u - \alpha_v \leq x(u, h) - x(v, h) \leq \sum_{i: x(u, i) \geq x(v, i)} [x(u, i) - x(v, i)] = P = \frac{1}{2} \sum_i |x(u, i) - x(v, i)|.$$

■

From Lemma 15.6, and linearity of expectation, we obtain

$$\mathbb{E}[c(\delta(U))] \leq \frac{1}{2} \sum_{uv \in E} c_{uv} z_{uv}$$

and hence

$$\frac{1}{2} \mathbb{E}[c(\delta(U))] \leq \frac{1}{2} \text{OPT}$$

as desired.

15.3.3 The CKR Rounding

In this section we describe the rounding algorithm from [53]. The first step is to pick a θ randomly from $[0, 1)$ to take the balls of radius θ from each s_i . However we need to uncross them since they can overlap. We did it explicitly in the preceding algorithm. Instead the original algorithm does it *implicitly* via the use of a random permutation σ to uncross the balls.

CKR-Rounding(G, S)

1. Let $\bar{x}$ be an optimum solution to the CKR relaxation on G with terminals S
2. Pick θ uniformly at random from $[0, 1)$
3. Pick a random permutation σ of $\{1, 2, \dots, k\}$
4. For $i = 1$ to k let $A_{\sigma(i)} = B(s_{\sigma(i)}, \theta) \setminus (\cup_{j < i} A_{\sigma(j)})$
5. Let $U = V \setminus (\cup_{i=1}^k A_i)$ be the unallocated nodes
6. Output the partition $A_1 \cup U, A_2, \dots, A_k$

We leave the following as an exercise so that the reader can see that the random permutation is one simple and very efficient way to uncross to preserve the expected cost of uncrossing.

Lemma 15.8. *Suppose $f : 2^V \rightarrow \mathbb{R}$ is a posi-modular function and $A_1, A_2, \dots, A_k$ be subsets of V . Let σ be a random permutation of $\{1, 2, \dots, k\}$. Let $A'_i = A_i \setminus (\cup_{j:\sigma(j) < \sigma(i)} A_j)$. Then, $A'_1, \dots, A'_k$ are uncrossed and*

$$\mathbb{E} \left[\sum_{i=1}^k f(A'_i) \right] \leq \sum_{i=1}^k f(A_i).$$

Exercise 15.6. Modify the algorithm where $A_{\sigma(i)}$ is defined as in the algorithm but only for $1 \leq i \leq k-1$ and let $A_{\sigma(k)} = V \setminus \cup_{i=1}^{k-1} A_{\sigma(i)}$. In other words after the $k-1$ terminal grabs its vertices we assign the remaining vertices to the last terminal in the permutation. Prove that this yields a $(1.5 - 1/k)$ -approximation.

15.4 Node-weighted Multiway Cut

In NODE-WEIGHTED MULTIWAY CUT the graph $G = (V, E)$ is undirected with non-negative node weights $w : V \rightarrow \mathbb{R}_+$ and the goal is to find a minimum-weight set of non-terminal nodes whose removal separates all terminals from each other. It is easy to see that NODE-WEIGHTED MULTIWAY CUT generalizes MULTIWAY CUT. VERTEX COVER reduces to NODE-WEIGHTED MULTIWAY CUT in an approximation-preserving fashion [118], and hence, under UGC we do not expect any ratio better than 2. This also shows that NODE-WEIGHTED MULTIWAY CUT is strictly harder than MULTIWAY CUT in terms of approximation ratio. Garg, Vazirani, and Yannakakis [118] obtained a $2(1 - 1/k)$ -approximation for NODE-WEIGHTED MULTIWAY CUT via the natural LP relaxation. To do this they showed that there always exists a half-integral optimum solution to the natural LP relaxation and used that property to round. In [77] it was shown that there is a much simpler and natural rounding strategy which is similar to that for MULTIWAY CUT that we saw via the distance LP. We outline the ideas below and leave the details to the reader to work out.

LP relaxation. Assign a length $x_v \geq 0$ to each non-terminal node (terminals are fixed with $x_{s_i} = 0$). For any path p between terminals s_i and s_j , require

$$\sum_{v \in p} x_v \geq 1:$$

$$\begin{aligned} \min \quad & \sum_{v \in V \setminus S} w_v x_v \\ \text{s.t.} \quad & \sum_{v \in p} x_v \geq 1 \quad p \in \mathcal{P}_{ij}, i < j \\ & x_v = 0 \quad v \in S \\ & x_v \geq 0 \quad v \in V \end{aligned}$$

Define $d_x(u, v)$ as the shortest-path distance counting node weights at both endpoints, so $d_x(u, v) \geq x_u + x_v$. For a node u and radius r let $B(u, r) = \{v : d_x(u, v) \leq r\}$ and define the *boundary* $B^+(u, r) = \{v \notin B(u, r) : v \text{ has a neighbor in } B(u, r)\}$.

Lemma 15.9. $v \in B^+(u, r)$ if and only if $r < d_x(u, v) \leq r + x_v$. In particular, $v \in B^+(u, r)$ for $r < 1$ implies $x_v > 0$.

Rounding algorithm. The algorithm omits one terminal chosen uniformly at random and cuts the boundaries of half-radius balls around all remaining terminals.

RoundNodeMC(G, S, x)

1. Pick $\ell \in \{1, 2, \dots, k\}$ uniformly at random
2. Pick $\theta \in (0, 1/2)$ uniformly at random
3. Output $C = \bigcup_{i \neq \ell} B^+(s_i, \theta)$

Theorem 15.2 ([77]). RoundNodeMC always returns a feasible node multiway cut and its expected cost is at most $2(1 - 1/k) \sum_v w_v x_v$.

This is essentially the same as the BallCut algorithm for MULTIWAY CUT generalized to the node-weighted setting. We omit the formal proof and leave it for the reader to figure out the details. It is easier to prove a bound of 2 via the simpler BallCut algorithm.

15.5 Directed Multiway Cut

In DIRECTED MULTIWAY CUT the graph $G = (V, E)$ is directed with non-negative edge weights and the goal is to find a minimum-weight set of edges whose removal ensures there is no directed path from s_i to s_j for any $i \neq j$. Note that

DIRECTED MULTIWAY CUT with $k = 2$ requires simultaneously eliminating all paths from s_1 to s_2 and from s_2 to s_1 . This is also called the s - t Bicut problem, and surprisingly it is already NP-Hard [117].

There are approximation-preserving reductions which establish the following:

$$\text{MULTIWAY CUT} \leq \text{NODE-WEIGHTED MULTIWAY CUT} \leq \text{DIRECTED MULTIWAY CUT}.$$

Moreover, VERTEX COVER reduces to both NODE-WEIGHTED MULTIWAY CUT and DIRECTED MULTIWAY CUT in an approximation-preserving fashion [118], so both problems are provably harder to approximate than MULTIWAY CUT. Under the Unique Games Conjecture [176], this implies that 2-approximation is the best one can achieve. Naor and Zosin [212] obtained a 2-approximation for DIRECTED MULTIWAY CUT which was also the first constant factor approximation. Although their algorithms and analysis were short, they were somewhat mysterious. It is only in 2016 that a very simple rounding algorithm achieving the same bound was achieved in [77] and the analysis is also extremely simple. This algorithm also had the advantage that it can be applied to any feasible solution to the natural LP relaxation unlike the one in [212]. We describe it next.

LP relaxation. Let $\mathcal{P}_{ij}$ denote the set of all directed paths from s_i to s_j in G . The natural LP relaxation assigns a length $x_e \geq 0$ to each edge and requires every directed s_i - s_j path to have length at least 1:

$$\begin{aligned} \min \quad & \sum_{e \in E} w_e x_e \\ \text{s.t.} \quad & \sum_{e \in p} x_e \geq 1 \quad p \in \mathcal{P}_{ij}, i \neq j \\ & x_e \geq 0 \quad e \in E \end{aligned}$$

For any feasible x let $d_x(u, v)$ denote the shortest directed-path distance from u to v under edge lengths x , and let $B(v, r) = \{u : d_x(v, u) \leq r\}$ be the ball of radius r around v .

Rounding algorithm. The key insight is to reduce DIRECTED MULTIWAY CUT to a directed multicut instance via dummy terminals. For each terminal s_i introduce a new dummy node t_i and add zero-weight edges (t_i, s_j) for all $j \neq i$; see Figure 15.1. Setting $x(t_i, s_j) = 0$ extends the fractional solution feasibly: $d(t_i, s_j) = 0$ for $j \neq i$, while $d(t_i, s_i) \geq 1$ since any directed s_j -to- s_i path in the augmented graph still has length at least 1. The resulting instance requires separating each pair (t_i, s_i) , which the following ball-cutting algorithm handles.

RoundDirMC(G, S, x)

1. Add dummy nodes $t_1, \dots, t_k$; for each $i \neq j$ add edge (t_i, s_j) with $x(t_i, s_j) = 0$
2. Pick $\theta \in (0, 1)$ uniformly at random
3. Output $C = \bigcup_{i=1}^k \delta^+(B(t_i, \theta))$, where $\delta^+(A)$ denotes the edges leaving A

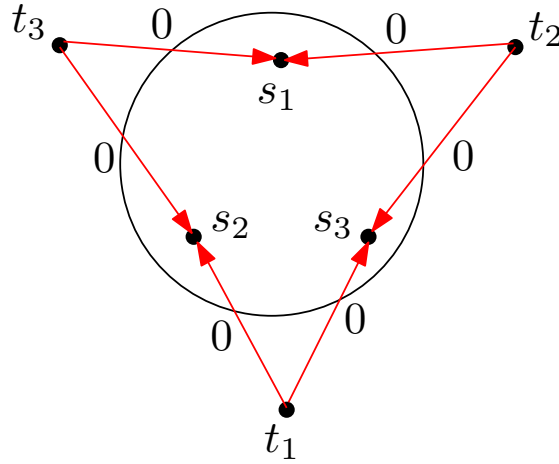


Figure 15.1: Addition of dummy vertices t_i (left) that reduce DIRECTED MULTIWAY CUT to a directed multicut instance separating pairs (t_i, s_i) for $i \in [k]$.

Theorem 15.3 ([77]). RoundDirMC *always returns a feasible multiway cut and its expected cost is at most $2 \sum_e w_e x_e$.*

Lemma 15.10. *For any $\theta \in (0, 1)$, C is a feasible multiway cut.*

Proof. Fix i and $\theta \in (0, 1)$. Since $d(t_i, s_j) = 0$ for all $j \neq i$, every such s_j lies in $B(t_i, \theta)$. Since $d(t_i, s_i) \geq 1 > \theta$, we have $s_i \notin B(t_i, \theta)$. Removing $\delta^+(B(t_i, \theta))$ therefore eliminates all directed paths from any s_j ($j \neq i$) to s_i . Taking the union over all i gives feasibility. ■

Lemma 15.11. *For any edge $e \in E$, $\mathbf{P}[e \in C] \leq 2x_e$.*

Proof. Let $e = (u, v)$. Rename terminals so $d(s_1, u) \leq d(s_2, u) \leq \dots \leq d(s_k, u)$. Then $d(t_1, u) = d(s_2, u)$ and $d(t_2, u) = \dots = d(t_k, u) = d(s_1, u)$. Edge e enters $\delta^+(B(t_i, \theta))$ iff $\theta \in I_i := [d(t_i, u), d(t_i, u) + x_e)$. Since $t_2, \dots, t_k$ are equidistant

from u , the intervals $I_2 = I_3 = \dots = I_k$ coincide. Hence $e \in C$ only if $\theta \in I_1 \cup I_2$, and since each interval has length at most x_e ,

$$\mathbf{P}[e \in C] \leq \mathbf{P}[\theta \in I_1] + \mathbf{P}[\theta \in I_2] \leq 2x_e.$$

■

By linearity of expectation Lemma 15.11 gives $\mathbb{E}[w(C)] \leq 2 \sum_e w_e x_e$, completing the proof of Theorem 15.3.

Remark 15.1. The dummy-terminal reduction is essential: the naive algorithm that cuts $\bigcup_i \delta^+(B(s_i, \theta))$ directly does not achieve a constant approximation ratio in general.

Integrality gap and hardness: The integrality gap of the LP is 2. Surprisingly this holds even for $k = 2$ in planar directed graphs [77], in other words for the s - t Bicut problem. Interestingly, under UGC, the hardness of 2 holds even for this case!

A notable structural difference from MULTIWAY CUT is that the LP relaxation for DIRECTED MULTIWAY CUT need not have half-integral optimum solutions, even for $k = 2$. In contrast, the LP for MULTIWAY CUT and NODE-WEIGHTED MULTIWAY CUT always has a half-integral optimum [118].

Bibliographic Notes

There is extensive work on MULTIWAY CUT problem after the CKR relaxation. Finding the right integrality gap is a fascinating open problem. The case of $k = 4$ is open. Although there has been substantial focus on the CKR relaxation for graphs there has been less attention to the generalization of the problem to the submodular setting. As we mentioned, Chekuri and Ene [69] obtained a 1.5-approximation for Symmetric Submodular Multiway Partition (SYM SUBMP) problem. They also obtained a 2-approximation for SUBMP problem. Ene, Vondrák and Yu [97] improved these bounds to $(1.5 - 1/k)$ and $2(1 - 1/k)$, and also simplified the analysis. Further, they proved hardness results: $2(1 - 1/k)$ for SUBMP and 1.26 for SYM SUBMP. These hardness results are of a different flavor and show that obtaining better than these ratios would require an exponential number of queries in the value oracle model. We refer the interested reader to [97]. Another interesting paper is that of Ene and Nguyen [96] who considered Hypergraph Partition which generalizes MULTIWAY CUT and is a special case of SYM SUBMP. Can one improve the 1.5-approximation for SYM SUBMP or its special cases including Hypergraph Partition?

Chapter 16

k -Cut and Steiner k -Cut

In the *minimum k -cut* problem (k -CUT) we are given an undirected graph $G = (V, E)$ with edge weights $w : E \rightarrow \mathbb{R}_+$ and a positive integer $k \geq 2$. The goal is to find a minimum weight set of edges $F \subseteq E$ such that $G - F$ has at least k connected components. When $k = 2$ this is the *global minimum cut* problem, which can be solved in polynomial time (and in near-linear time using randomization). The minimum k -cut problem can be solved in $O(n^{O(k)})$ time, and hence is polynomial-time solvable for any fixed k . However, the problem is NP-hard when k is part of the input. Note that unlike MULTIWAY CUT problem that we saw in the previous chapter there are no terminals specified in k -Cut.

We now describe the *Steiner k -cut* problem which generalizes k -CUT and MULTIWAY CUT. The input is an edge-weighted graph $G = (V, E)$, integer k , and a set of terminals $T = \{t_1, t_2, \dots, t_h\} \subseteq V$. The goal is to find a minimum weight set of edges F such that $G - F$ has at least k -connected components such that each of those components contains a terminal from T . When $h = k$ this is MULTIWAY CUT and when $T = V$ this is k -CUT.

We saw that MULTIWAY CUT admits a simple $2(1 - 1/k)$ -approximation and a 1.2787-approximation via the sophisticated CKR LP relaxation. In contrast, it is known that k -CUT does not admit a better than $(2 - \epsilon)$ -approximation under plausible hardness results [200]. In this chapter we describe two approaches, both achieving a $2(1 - 1/k)$ -approximation for Steiner k -cut. We describe an algorithm based on Gomory-Hu trees for k -CUT which can be generalized to Steiner k -cut, and a greedy splitting algorithm for the Steiner k -cut problem. These results can be seen as special cases of submodular partitioning problems that we will discuss later in bibliographic notes.

16.1 Gomory-Hu Trees

Let $G = (V, E)$ be an undirected graph with edge weights $w : E \rightarrow \mathbb{R}_+$ and let $n = |V|$. For a pair of vertices $a, b \in V$, let $\text{mc}(a, b)$ denote the value of a minimum a - b cut in G . There are $\binom{n}{2}$ pairs of vertices and hence potentially $\binom{n}{2}$ different minimum cut values. Remarkably, only $n - 1$ of these values are distinct. Moreover, the entire pairwise minimum cut structure is captured by a single weighted tree.

Theorem 16.1 (Gomory-Hu [126]). *For any undirected graph $G = (V, E)$ with edge weights $w : E \rightarrow \mathbb{R}_+$, there exists a weighted tree $\mathcal{T} = (V, E_{\mathcal{T}})$ on the same vertex set V with edge weights $u : E_{\mathcal{T}} \rightarrow \mathbb{R}_+$ such that:*

1. *For every pair $a, b \in V$, the value $\text{mc}(a, b)$ in G equals the minimum weight of any edge on the unique a - b path in $\mathcal{T}$.*
2. *For every edge $e = (p, q) \in E_{\mathcal{T}}$, the two components of $\mathcal{T} - e$ define a bipartition $(S, V \setminus S)$ of V that is a minimum p - q cut in G , and $u(e) = \text{mc}(p, q)$.*

The tree $\mathcal{T}$ is called a *Gomory-Hu tree* (or *cut tree*) of G . Note that $\mathcal{T}$ has the same vertex set as G but its edges need not correspond to edges of G ; it is a separate weighted tree encoding pairwise minimum cut information. A Gomory-Hu tree can be constructed using $n - 1$ maximum flow computations.

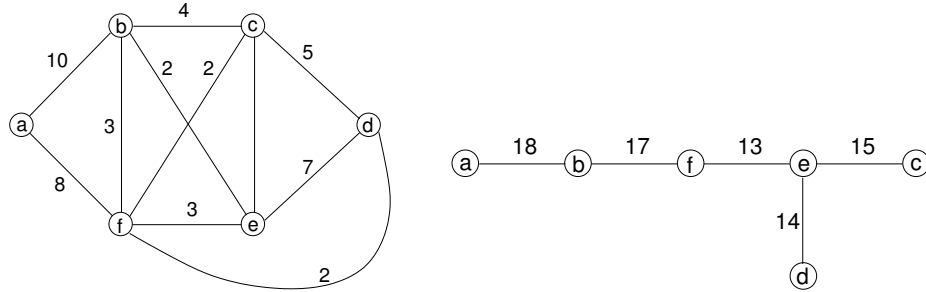


Figure 16.1: An edge-weighted graph and a Gomory-Hu tree for it (fig from [250]).

16.2 A Gomory-Hu Tree Based Algorithm for k -Cut

The Gomory-Hu tree gives a simple approximation algorithm for the k -Cut problem: remove the $k - 1$ minimum weight edges of the Gomory-Hu tree and use the resulting partition.

GomoryHuKCut($G = (V, E, w)$, k)

1. Compute the Gomory-Hu tree $\mathcal{T} = (V, E_{\mathcal{T}}, u)$ of G
2. Let $e_1, e_2, \dots, e_{k-1}$ be the $k-1$ minimum weight edges of $\mathcal{T}$ (breaking ties arbitrarily)
3. Remove $e_1, \dots, e_{k-1}$ from $\mathcal{T}$; let $V_1, V_2, \dots, V_k$ be the resulting connected components
4. Output the set of edges of G crossing the partition $(V_1, \dots, V_k)$

The partition $(V_1, \dots, V_k)$ is clearly a valid k -cut since removing $k-1$ edges from a tree on n vertices yields exactly k connected components. We now analyze the approximation ratio.

Theorem 16.2. *The GomoryHuKCut algorithm is a $2(1 - 1/k)$ -approximation for the minimum k -cut problem.*

Proof. Let OPT denote the value of an optimal k -cut and let $(V_1^*, V_2^*, \dots, V_k^*)$ be an optimal partition.

Step 1: Bounding the algorithm's cost. For each removed tree edge $e_j = (p_j, q_j)$, the two components of $\mathcal{T} - e_j$ give a bipartition $(S_j, V \setminus S_j)$ that is a minimum p_j - q_j cut in G of weight $u(e_j)$. An edge of G is in the output k -cut if and only if its endpoints lie in different components of $\mathcal{T} - \{e_1, \dots, e_{k-1}\}$. Since any such edge must cross at least one of the bipartitions $(S_j, V \setminus S_j)$, the set of output edges is contained in the union $\cup_{j=1}^{k-1} \delta_G(S_j)$, and therefore

$$w(\text{output}) \leq \sum_{j=1}^{k-1} u(e_j).$$

Step 2: Relating tree edges to OPT. For each part V_i^* of the optimal partition, $\delta_G(V_i^*)$ is the set of edges with exactly one endpoint in V_i^* . Since each edge in the optimal k -cut has its two endpoints in two different parts, it is counted exactly twice across the $\delta_G(V_i^*)$'s:

$$\sum_{i=1}^k w(\delta_G(V_i^*)) = 2 \text{OPT}.$$

We now identify k (not necessarily distinct) edges of $\mathcal{T}$, one for each part of the optimal partition. Construct a tree $\mathcal{T}'$ from $\mathcal{T}$ by contracting each V_i^* to a

single vertex v_i and removing any parallel edges. Since $\mathcal{T}$ is connected, $\mathcal{T}'$ is a connected graph on the k vertices $v_1, \dots, v_k$; we restrict to any spanning tree of this graph. Root $\mathcal{T}'$ at v_k and for each $i \in [k-1]$, let g_i be the tree edge in $\mathcal{T}$ corresponding to the edge from v_i towards its parent in $\mathcal{T}'$. Then $g_i = (a_i, b_i)$ where $a_i \in V_i^*$ and $b_i \notin V_i^*$.

By the Gomory-Hu tree property, $u(g_i) = \text{mc}(a_i, b_i)$. Since $\delta_G(V_i^*)$ separates a_i from b_i , we have $w(\delta_G(V_i^*)) \geq \text{mc}(a_i, b_i) = u(g_i)$. The edges $g_1, \dots, g_{k-1}$ are distinct edges of $\mathcal{T}$ (they correspond to distinct edges of the tree $\mathcal{T}'$), so

$$\sum_{j=1}^{k-1} u(e_j) \leq \sum_{i=1}^{k-1} u(g_i) \leq \sum_{i=1}^{k-1} w(\delta_G(V_i^*)),$$

where the first inequality uses the fact that $e_1, \dots, e_{k-1}$ are the $k-1$ lightest edges of $\mathcal{T}$.

Step 3: The drop-the-heaviest argument. Without loss of generality assume $w(\delta_G(V_1^*)) \leq \dots \leq w(\delta_G(V_k^*))$. Then

$$\sum_{j=1}^{k-1} u(e_j) \leq \sum_{i=1}^{k-1} w(\delta_G(V_i^*)) \leq \left(1 - \frac{1}{k}\right) \sum_{i=1}^k w(\delta_G(V_i^*)) = 2 \left(1 - \frac{1}{k}\right) \text{OPT},$$

where the second inequality uses the fact that dropping the largest of k non-negative terms reduces the sum by a factor of at most $(1 - 1/k)$. ■

Exercise 16.1. Describe and analyze a $2(1 - 1/k)$ -approximation for Steiner k -Cut via the Gomory-Hu tree.

16.3 Greedy Splitting for Steiner k -Cut

We now describe a greedy splitting algorithm for the Steiner k -cut problem. The idea is to build up a partition by splitting one component at a time, always choosing the cheapest available split, until each component contains at most one terminal.

GreedySplit($G = (V, E, w)$, $T = \{t_1, \dots, t_h\}$)

1. Initialize partition $\mathcal{P}_0 \leftarrow \{V\}$
2. For $i = 1, 2, \dots, k-1$:
 - A. For each component $C \in \mathcal{P}_{i-1}$ that contains at least two terminals, find the minimum weight cut of $G[C]$ that places at least one terminal on each side; let $f(C)$ be its value
 - B. Let C^* be the component minimizing $f(C)$
 - C. Split C^* into two parts along this cut to obtain $\mathcal{P}_i$
3. Output the edges of G crossing the final partition $\mathcal{P}_{k-1}$

At the start all h terminals are in a single component. Each split increases the number of components by one, and each resulting part contains at least one terminal. After $k-1$ splits there are k components, each containing at least one terminal, so the output is a feasible solution to the Steiner k -cut instance.

We use $w(\mathcal{P}_i)$ to denote the total weight of edges crossing the partition $\mathcal{P}_i$, so the algorithm's output cost is $w(\mathcal{P}_{k-1})$.

We say that an arbitrary partition $\mathcal{P} = \{V_1, V_2, \dots, V_j\}$ of V is *valid* if each V_i contains at least one terminal from T .

Theorem 16.3. *The GreedySplit algorithm is a $2(1 - 1/k)$ -approximation for the Steiner k -cut problem.*

The proof relies on the following inductive lemma.

Lemma 16.1. *For every $i \in [k-1]$ and every valid partition $\mathcal{P} = \{V_1, V_2, \dots, V_i\}$ into i parts, we have*

$$w(\mathcal{P}_{i-1}) \leq w(\delta(V_1)) + w(\delta(V_2)) + \dots + w(\delta(V_{i-1})).$$

Note that the right-hand side in the lemma sums over only $i-1$ of the i parts of $\mathcal{P}$.

Before proving the lemma, we show how it implies the theorem.

Proof of Theorem 16.3. Let $\{A_1, A_2, \dots, A_k\}$ be an optimal Steiner k -cut with cost OPT . Each edge in the optimal cut has endpoints in two different parts, so $\sum_{i=1}^k w(\delta(A_i)) = 2\text{OPT}$. Without loss of generality assume $w(\delta(A_1)) \leq w(\delta(A_2)) \leq \dots \leq w(\delta(A_k))$.

Applying Lemma 16.1 with $i = k$ and the valid partition $\{A_1, \dots, A_k\}$:

$$w(\mathcal{P}_{k-1}) \leq \sum_{j=1}^{k-1} w(\delta(A_j)) \leq \left(1 - \frac{1}{k}\right) \sum_{j=1}^k w(\delta(A_j)) = 2 \left(1 - \frac{1}{k}\right) \text{OPT},$$

where the second inequality uses $w(\delta(A_k)) \geq w(\delta(A_j))$ for all j , so dropping the heaviest term reduces the sum by a factor of at most $(1 - 1/k)$. ■

Proof of Lemma 16.1. We proceed by induction on i .

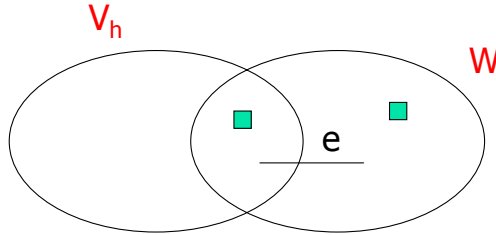
Base case ($i = 1$). $\mathcal{P} = \{V\}$ and $\mathcal{P}_0 = \{V\}$. Both sides equal zero (the empty sum on the right, and $w(\mathcal{P}_0) = 0$ since no edges are cut).

Inductive step. Assume the lemma holds for $i - 1$. Let $\mathcal{P} = \{V_1, V_2, \dots, V_i\}$ be an arbitrary valid partition into i parts.

Consider $\mathcal{P}_{i-2}$ (the partition produced by the algorithm after $i - 2$ splits). Since $\mathcal{P}_{i-2}$ has $i - 1$ components and $\mathcal{P}$ has i parts, by the pigeonhole principle there exists a component $W \in \mathcal{P}_{i-2}$ and two distinct indices h, l with $h < l \leq i$ such that $W \cap V_h$ and $W \cap V_l$ both contain a terminal.

Splitting W into $(W \cap V_h)$ and $(W \setminus V_h)$ is a valid option for the greedy algorithm when refining $\mathcal{P}_{i-2}$ into $\mathcal{P}_{i-1}$: both parts contain a terminal (the first by choice, and the second because it contains $W \cap V_l$ which has a terminal). Since the greedy algorithm chooses a cheapest split, $w(\mathcal{P}_{i-1}) - w(\mathcal{P}_{i-2})$ is at most the cost increase from splitting W into $(W \cap V_h)$ and $(W \setminus V_h)$.

Any new edge cut by this split of W must have one endpoint in $W \cap V_h$ and the other in $W \setminus V_h$. Since both endpoints are in W (an edge internal to W before the split), and the endpoints are in different parts of $\{V_h, W \setminus V_h\}$, every such edge belongs to $\delta(V_h)$.



Therefore:

$$w(\mathcal{P}_{i-1}) - w(\mathcal{P}_{i-2}) \leq w(\delta(V_h)).$$

Now consider the partition obtained by merging V_h and V_l :

$$\mathcal{P}' = \{V_1, \dots, V_{h-1}, V_{h+1}, \dots, V_{i-1}, V_h \cup V_l\}.$$

This is a valid partition into $i - 1$ parts (each original part retains its terminal, and $V_h \cup V_i$ contains the terminals from both V_h and V_i). Applying the induction hypothesis to $\mathcal{P}'$, we obtain

$$w(\mathcal{P}_{i-2}) \leq \sum_{\substack{j=1 \\ j \neq h}}^{i-1} w(\delta_{\mathcal{P}'}(V_j)),$$

where the sum on the right runs over the $i - 2$ parts of $\mathcal{P}'$ that are *not* $V_h \cup V_i$ (note that $V_h \cup V_i$ plays the role of the dropped part). Since $\mathcal{P}'$ is a coarsening of $\mathcal{P}$ (it merges two parts), for each $j \notin \{h, i\}$ we have $\delta_{\mathcal{P}'}(V_j) \subseteq \delta_{\mathcal{P}}(V_j)$, so $w(\delta_{\mathcal{P}'}(V_j)) \leq w(\delta(V_j))$. Hence:

$$w(\mathcal{P}_{i-2}) \leq \sum_{\substack{j=1 \\ j \neq h}}^{i-1} w(\delta(V_j)).$$

Adding the two inequalities above:

$$w(\mathcal{P}_{i-1}) \leq w(\delta(V_h)) + \sum_{\substack{j=1 \\ j \neq h}}^{i-1} w(\delta(V_j)) = \sum_{j=1}^{i-1} w(\delta(V_j)). \quad \blacksquare$$

Bibliographic Notes

The polynomial-time solvability of k -CUT for fixed k was initially due to a paper of Goldschmidt and Hochbaum [125]. There have been many results improving the run-time and generalizing to hypergraphs, submodular functions and so on; see [146] and references therein. The greedy splitting analysis for k -CUT was initially due to Saran and Vazirani [229] and the analysis was not so easy. The Gomory-Hu tree based algorithm for k -CUT is due to independent work of Kapoor and R. Ravi and is much simpler to analyze. The Steiner k -Cut problem was coined and considered in [70]; the paper gave a Gomory-Hu tree based algorithm as well as another algorithm based on an LP relaxation. The analysis of the greedy splitting algorithm for Steiner k -Cut is adapted from the paper of Zhao, Nagamochi and Ibaraki [261] who showed that it yields a $2(1 - 1/k)$ -approximation for any symmetric submodular function. The inductive claim and proof are short and elegant but maynot be that intuitive. One can also obtain the same ratio via a Gomory-Hu tree based algorithm since every symmetric submodular function admits such a tree. Yet another $2(1 - 1/k)$ -approximation can be obtained via

the principal sequence of partitions of a submodular function [61]; the same paper has other interesting results for monotone and posi-modular submodular functions. Zhao et al also considered general non-negative submodular functions and showed that one obtains a k -approximation. Chekuri and Li [75] showed that Hypograph k -Cut, which is a special case of the general submodular case, is hard to approximate to polynomial factor via hardness for the Densest k -Subgraph problem. Thus, the general submodular case is much harder. This is in contrast the version with terminals considered in the previous chapter which admits a $2(1 - 1/k)$ -approximation even for general non-negative submodular functions.

Chapter 17

Multicut

In the MULTICUT problem, we are given a graph $G = (V, E)$, a capacity function that assigns a capacity c_e to each edge e , and a set of pairs $(s_1, t_1), \dots, (s_k, t_k)$. The MULTICUT problem asks for a minimum capacity set of edges $F \subseteq E$ such that removing the edges in F disconnects s_i and t_i , for all i . Note that the MULTICUT problem generalizes the MULTIWAY CUT problem that we saw previously. We describe an $O(\log k)$ approximation algorithm for MULTICUT.

We start by describing an LP formulation for the problem. For each edge e , we have a variable d_e . We interpret each variable d_e as a distance label for the edge. Let $\mathcal{P}_{s_i, t_i}$ denote the set of all paths between s_i and t_i . We have the following LP for the problem:

$$\begin{aligned} \min \quad & \sum_{e \in E} c_e d_e \\ \text{s.t.} \quad & \sum_{e \in p} d_e \geq 1 \quad p \in \mathcal{P}_{s_i, t_i}, 1 \leq i \leq k \\ & d_e \geq 0 \quad e \in E \end{aligned}$$

The LP assigns distance labels to edges so that, on each path p between s_i and t_i , the distance labels of the edges on p sum up to at least one. Note that, even though the LP can have exponentially many constraints, we can solve the LP in polynomial time using the ellipsoid method and the following separation oracle. Given distance labels d_e , we set the length of each edge to d_e and, for each pair (s_i, t_i) , we compute the length of the shortest path between s_i and t_i and check whether it is at least one. If the shortest path between s_i and t_i has length smaller than one, we have a violated constraint. Conversely, if all shortest paths have length at least one, the distance labels define a feasible solution.

We also consider the dual of the previous LP. For each path p between any pair (s_i, t_i) we have a dual variable f_p . We interpret each variable f_p as the

amount of flow between s_i and t_i that is routed along the path p . We have the following dual LP:

$$\begin{aligned}
 \max \quad & \sum_{i=1}^k \sum_{p \in \mathcal{P}_{s_i, t_i}} f_p \\
 \text{s.t.} \quad & \sum_{p: e \in p} f_p \leq c_e & e \in E(G) \\
 & f_p \geq 0 & p \in \mathcal{P}_{s_1, t_1} \cup \dots \cup \mathcal{P}_{s_k, t_k}
 \end{aligned}$$

The dual is an LP formulation for the Maximum Throughput Multicommodity Flow problem. In the Maximum Throughput Multicommodity Flow problem, we have k different commodities. For each i , we want to route commodity i from the source s_i to the destination t_i . Each commodity must satisfy flow conservation at each vertex other than its source and its destination. Additionally, the total flow routed on each edge must not exceed the capacity of the edge. The goal is to maximize the sum of the commodities routed.

The dual LP tries to assign an amount of flow f_p to each path p so that the total flow on each edge is at most the capacity of the edge (the flow conservation constraints are automatically satisfied). Note that the endpoints of the path p determine which kind of commodity is routed along the path.

Exercise 17.1. Write the MULTICUT LP and its dual in a compact form with polynomially many constraints.

17.1 Upper Bound on the Integrality Gap

In this section, we will show that the integrality gap of the LP is $O(\log k)$ using a randomized rounding algorithm due to Calinescu, Karloff, and Rabani [51]. The first algorithm that achieved an $O(\log k)$ -approximation for MULTICUT is due to Garg, Vazirani, and Yannakakis [117] (see [250] and [255]), and it is based on the region growing technique introduced by Leighton and Rao [189]. The reason that we choose to present the randomized rounding algorithm is due to its future application for metric embeddings.

Let $B_d(v, r)$ denote the ball of radius r centered at the vertex v in the metric induced by the distance labels d_e .

CKR-RandomPartition:

Solve the LP to get the distance labels d_e
 Pick θ uniformly at random from $[0, 1/2)$
 Pick a random permutation σ on $\{1, 2, \dots, k\}$
 for $i = 1$ to k
 $V_{\sigma(i)} = B_d(s_{\sigma(i)}, \theta) \setminus \bigcup_{j < i} V_{\sigma(j)}$
 Output $\bigcup_{i=1}^k \delta(V_i)$

Lemma 17.1. *CKR-RandomPartition correctly outputs a feasible multicut for the given instance.*

Proof. Let F be the set of edges output by the algorithm. Suppose F is not a feasible multicut. Then there exists a pair of vertices (s_i, t_i) such that there is a path between s_i and t_i in $G - F$. Therefore there exists a j such that V_j contains s_i and t_i . Since $V_j \subseteq B_d(s_j, \theta)$, both s_i and t_i are contained in the ball of radius θ centered at s_j . Consequently, the distance between s_j and s_i is at most θ and the distance between s_j and t_i is at most θ . By the triangle inequality, the distance between s_i and t_i is at most 2θ . Since θ is smaller than $1/2$, it follows that the distance between s_i and t_i is smaller than one. This contradicts the fact that the distance labels d_e are a feasible solution for the LP. Therefore F is a multicut, as desired. ■

Lemma 17.2. *The probability that an edge e is cut is at most $2H_k d_e$, where H_k is the k -th harmonic number and d_e is the distance label of the edge e .*

Proof. Fix an edge $e = (u, v)$. Let:

$$L_i = \min\{d(s_i, u), d(s_i, v)\}$$

$$R_i = \max\{d(s_i, u), d(s_i, v)\}$$

We may assume without loss of generality that $L_1 \leq L_2 \leq \dots \leq L_k$ (by reindexing the pairs as needed). See Fig 17.1.

Let A_i be the event that the edge e is cut *first* by s_i . More precisely, A_i is the event that $|V_i \cap \{u, v\}| = 1$ and $|V_j \cap \{u, v\}| = 0$ for all j such that $\sigma(j) < \sigma(i)$. Note that $|V_i \cap \{u, v\}| = 1$ simply says that s_i cuts the edge e . If s_i is the first to cut the edge e , for all j that come before i in the permutation, neither u nor v can be in V_j (if only one of u and v is in V_j , s_j cuts the edge e ; if both u and v are in V_j , s_j cannot cut the edge e).

Note that the event that the edge e is cut is the union of the disjoint events $A_1, \dots, A_k$. Therefore we have:

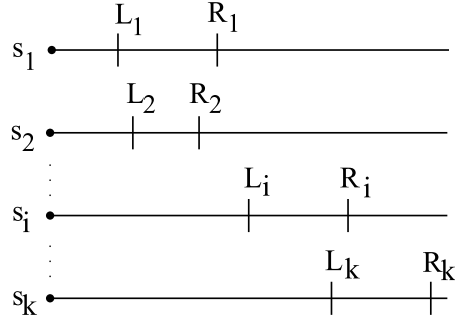
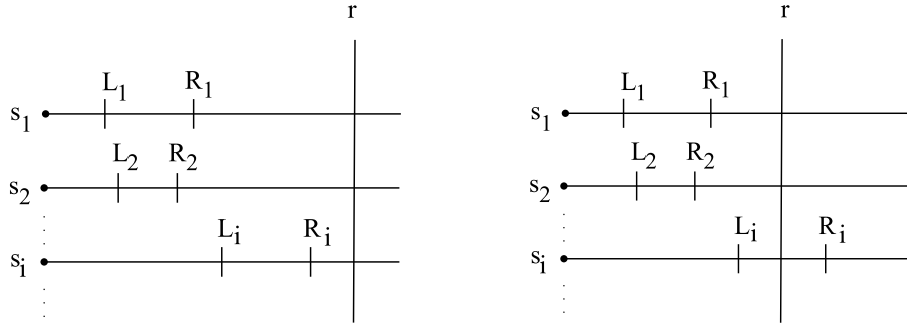


Figure 17.1: For a fixed edge $e = (u, v)$ we renumber the pairs such that $L_1 \leq L_2 \leq \dots \leq L_k$.

$$\mathbf{P}[e \text{ is cut}] = \sum_i \mathbf{P}[A_i].$$

Let us fix $r \in [0, 1/2)$ and consider $\mathbf{P}[A_i \mid \theta = r]$. Note that s_i cuts the edge e only if one of u, v is inside the ball of radius r centered at s_i and the other is outside the ball. Differently said, s_i cuts the edge only if $r \in [L_i, R_i)$:

$$\mathbf{P}[A_i \mid \theta = r] = 0 \quad \text{if } r \notin [L_i, R_i)$$



Now suppose that $r \in [L_i, R_i)$. Let us fix $j < i$ and suppose j comes before i in the permutation (that is, $\sigma(j) < \sigma(i)$). Recall that, since $j < i$, we have $L_j \leq L_i \leq r$. Therefore at least one of u, v is inside the ball of radius r centered at s_j . Consequently, s_i cannot be the first to cut the edge e . Therefore s_i is the first to cut the edge e only if $\sigma(i) < \sigma(j)$ for all $j < i$. See Fig 17.2. Since σ is a random permutation, i appears before j for all $j < i$ with probability $1/i$. Therefore we have:

$$\mathbf{P}[A_i \mid \theta = r] \leq \frac{1}{i} \quad \text{if } r \in [L_i, R_i)$$

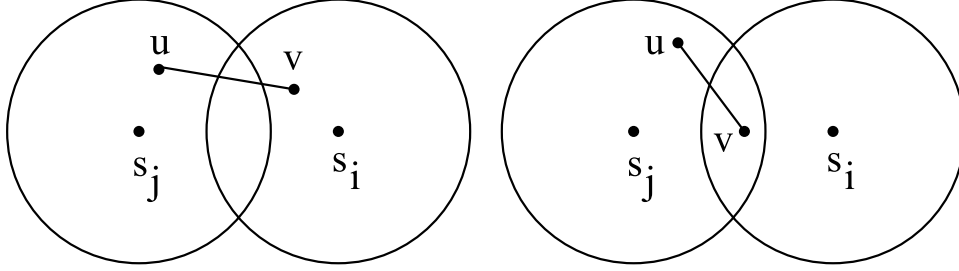


Figure 17.2: If $\sigma(j) < \sigma(i)$, s_i cannot be the first to cut the edge $e = (u, v)$. On the left s_j also cuts the edge. On the right s_j captures both end points and therefore s_i cannot cut it.

Since θ was selected uniformly at random from the interval $[0, 1/2)$, and independently from σ , we have:

$$\mathbf{P}[A_i] \leq \frac{1}{i} \mathbf{P}[\theta \in [L_i, R_i)] = \frac{2}{i} (R_i - L_i)$$

By the triangle inequality, $R_i \leq L_i + d_e$. Therefore:

$$\mathbf{P}[A_i] \leq \frac{2d_e}{i}$$

Consequently,

$$\mathbf{P}[e \text{ is cut}] = \sum_i \mathbf{P}[A_i] \leq 2H_k d_e.$$

■

Corollary 17.1. *The integrality gap of the MULTICUT LP is $O(\log k)$.*

Proof. Let F be the set of edges outputted by the Randomized Rounding algorithm. For each edge e , let χ_e be an indicator random variable equal to 1 if and only if the edge e is in F . As we have already seen,

$$\mathbf{E} \chi_e = \mathbf{P}[\chi_e = 1] \leq 2H_k d_e$$

Let $c(F)$ be a random variable equal to the total capacity of the edges in F . We have:

$$\mathbf{E} c(F) = \mathbf{E} \sum_e c_e \chi_e = \sum_e c_e \mathbf{P}[\chi_e] \leq 2H_k \sum_e c_e d_e = 2H_k \text{OPT}_{\text{LP}}$$

Consequently, there exists a set of edges F such that the total capacity of the edges in F is at most $2H_k \text{OPT}_{\text{LP}}$. Therefore $\text{OPT} \leq 2H_k \text{OPT}_{\text{LP}}$, as desired. ■

Corollary 17.2. *The algorithm achieves an $O(\log k)$ -approximation (in expectation) for the MULTICUT problem.*

Proof. As we have already seen,

$$\mathbf{E} c(F) \leq 2H_k \text{OPT}_{\text{LP}}$$

where F is the set of edges output by the algorithm and $c(F)$ is the total capacity of the edges in F . Since $\text{OPT}_{\text{LP}} \leq \text{OPT}$,

$$\mathbf{E} c(F) \leq 2H_k \text{OPT} = O(\log k) \text{OPT}$$

■

Remark 17.1. The expected cost analysis can be used to obtain an algorithm, via repetition, a randomized algorithm that outputs an $O(\log k)$ -approximation with high probability. The algorithm can also be derandomized but it is not straight forward. As we remarked there is an alternative deterministic $O(\log k)$ -approximation algorithm via region growing.

Flow-Cut Gap: Recall that when $k = 1$ we have the well-known maxflow-minicut theorem. The integrality gap of the standard LP for MULTICUT is the same as the relative gap between flow and cut when k is arbitrary. The upper bound on the integrality gap gives an upper bound on the gap.

Corollary 17.3. *We have:*

$$\max_{\text{m.c. flow } f} |f| \leq \min_{\text{multicut } C} |C| \leq O(\log k) \left(\max_{\text{m.c. flow } f} |f| \right)$$

where $|f|$ represents the value of the multicommodity flow f , and $|C|$ represents the capacity of the multicut C .

Proof. Let OPT_{LP} denote the total capacity of an optimal (fractional) solution for the MULTICUT LP. Let OPT_{dual} denote the flow value of an optimal solution for the dual LP. Since OPT_{LP} is a lower bound on the capacity of the minimum (integral) multicut, we have:

$$\max_{\text{m.c. flow } f} |f| = \text{OPT}_{\text{dual}} = \text{OPT}_{\text{LP}} \leq \min_{\text{multicut } C} |C|$$

As we have already seen, we have:

$$\min_{\text{multicut } C} |C| \leq 2H_k \text{OPT}_{\text{LP}} = 2H_k \text{OPT}_{\text{dual}} = 2H_k \left(\max_{\text{m.c. flow } f} |f| \right)$$

■

17.2 Lower Bound on the Integrality Gap

In this section, we will show that the integrality gap of the LP is $\Omega(\log k)$. That is, we will give a MULTICUT instance for which the LP gap is $\Omega(\log k)$. Let's start by looking at expander graphs and their properties.

17.2.1 Expander Graphs

Definition 17.4. A graph $G = (V, E)$ is an α -edge-expander if, for any subset S of at most $|V|/2$ vertices, the number of edges crossing the cut $(S, V \setminus S)$ is at least $\alpha|S|$.

Note that the complete graph K_n is a $(|V|/2)$ -edge-expander. However, the more interesting expander graphs are also sparse. Cycles and grids are examples of graphs that are very poor expanders.

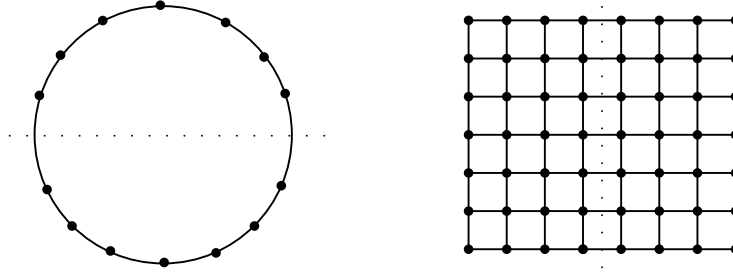


Figure 17.3: The top half of the cycle has $|V|/2$ vertices and only two edges crossing the cut. The left half of the grid has roughly $|V|/2$ vertices and only $\sqrt{|V|}$ edges crossing the cut.

Definition 17.5. A graph G is d -regular if every vertex in G has degree d .

Note that 2-regular graphs consist of a collection of edge disjoint cycles and therefore they have poor expansion. However, for any $d \geq 3$, there exist d -regular graphs that are very good expanders.

Theorem 17.6. *For every $d \geq 3$ there exists an infinite family of d -regular 1-edge-expanders.*

We will only need the following special case of the previous theorem.

Theorem 17.7. *There exists a universal constant $\alpha > 0$ and an integer n_0 such that, for all even integers $n \geq n_0$, there exists an n -vertex, 3-regular α -edge-expander.*

Proof Idea. The easiest way to prove this theorem is using the probabilistic method. The proof itself is beyond the scope of this lecture¹. The proof idea is the following.

Let's fix an even integer n . We will generate a 3-regular random graph G by selecting three random perfect matchings on the vertex set $\{1, 2, \dots, n\}$ (recall that a perfect matching is a set of edges such that every vertex is incident to exactly one of these edges). We select a random perfect matching as follows. We maintain a list of vertices that have not been matched so far. While there is at least one vertex that is not matched, we select a pair of distinct vertices u, v uniformly at random from all possible pairs of unmatched vertices. We add the edge (u, v) to our matching and we remove u and v from the list. We repeat this process three times (independently) to get three random matchings. The graph G will consist of the edges in these three matchings. Note that G is actually a 3-regular multigraph since it might have parallel edges (if the same edge is in at least two of the matchings). There are two properties of interest: (1) G is a simple graph and (2) G is an α -edge-expander for some constant $\alpha > 0$. If we can show that G has both properties with positive probability, it follows that there exists a 3-regular α -edge-expander (if no graph is a 3-regular α -edge-expander, the probability that our graph G has both properties is equal to 0).

It is not very hard to show that the probability that G does not have property (1) is small. To show that the probability that G does not have property (2) is small, for each set S with at most $n/2$ vertices, we estimate the expected number of edges that cross the cut $(S, V \setminus S)$ (e.g., we can easily show that $|\delta(S)| \geq |S|/2$). Using tail inequalities (e.g., Chernoff bounds), we can show that the probability that $|\delta(S)|$ differs significantly from its expectation is extremely small (i.e., small enough so that the sum – taken over all sets S – of these probabilities is also small) and we can use the union bound to get the desired result. ■

Note that explicit constructions of d -regular expanders are also known. Margulis [201] gave an infinite family of 8-regular expanders. There are many explicit construction by now and it is a very important topic of study — we refer the reader to the survey on expanders by Hoory, Linial and Wigderson

¹A more accurate statement is that the calculations are a bit involved and not terribly interesting for us.

[151]. The vertex set of a graph G_n in Margulis' construction is $\mathbb{Z}_n \times \mathbb{Z}_n$, where $\mathbb{Z}_n$ is the set of all integers mod n . The neighbors of a vertex (x, y) in G_n are $(x + y, y)$, $(x - y, y)$, $(x, y + x)$, $(x, y - x)$, $(x + y + 1, y)$, $(x - y + 1, y)$, $(x, y + x + 1)$, and $(x, y - x + 1)$ (all operations are mod n). Another example is the following infinite family of 3-regular expanders. For each prime p , we have a 3-regular graph G_p . The vertex set of G_p is $\mathbb{Z}_p$. The neighbors of a vertex x in G_p are $x + 1$, $x - 1$, and x^{-1} (as before, all operations are mod p ; x^{-1} is the inverse of x mod p , and we define the inverse of 0 to be 0)².

We conclude this section with the following observations (they will be very useful in showing the $\Omega(\log k)$ lower bound on the integrality gap of the LP).

Claim 17.2.1. *Let G be an n -vertex d -regular α -edge-expander, for some constants $d \geq 3$ and $\alpha > 0$. Then the diameter of G is $\Theta(\log n)$.*

Proof. For any two vertices u and v , let $\text{dist}(u, v)$ denote the length of a shortest path between u and v (the length of a path is the number of edges on the path). Let's fix a vertex s . Let L_i be the set of all vertices v such that $\text{dist}(s, v)$ is at most i . Now let's show that $(1 + \alpha/d)|L_{i-1}| \leq |L_i| \leq d|L_{i-1}|$. Clearly, $|L_1| = d$ (since s has degree d). Therefore we may assume that $i > 1$. Every vertex in L_i is in L_{i-1} or it has a neighbor in L_{i-1} . Therefore it suffices to bound $|L_i \setminus L_{i-1}|$.

Note that any vertex in L_{i-1} has at least one neighbor in L_{i-1} . Therefore the vertices in L_{i-1} have at most $(d - 1)|L_{i-1}|$ neighbors outside of L_{i-1} . Consequently, $|L_i| \leq d|L_{i-1}|$.

Now one of L_{i-1} , $V \setminus L_{i-1}$ has at most $|V|/2$ vertices. Let's assume without loss of generality that L_{i-1} has at most $|V|/2$ vertices (the other case is symmetric). Let $A = L_{i-1}$ and let B be the set of all vertices in $V \setminus L_{i-1}$ that have a neighbor in L_{i-1} (note that $|L_i| = |A| + |B|$). Let F be the set of all edges that cross the cut $(L_{i-1}, V \setminus L_{i-1})$. Now let's look at the bipartite graph $H = (A, B, F)$. Since G is an α -edge-expander, we have $|F| \geq \alpha|A|$. Moreover, $|F| = \sum_{v \in B} d_H(v)$, where $d_H(v)$ is the degree of v in H . Since $d_H(v)$ is at most d , we have $\alpha|A| \leq |F| \leq d|B|$. Therefore we have:

$$|L_i| = |A| + |B| \geq (1 + \alpha/d)|A| = (1 + \alpha/d)|L_{i-1}|$$

It follows by induction that $d(1 + \alpha/d)^{i-1} \leq |L_i| \leq d^i$. Therefore $\text{dist}(s, v)$ is $O(\log n)$ for all v and there exists a vertex v such that $\text{dist}(s, v)$ is $\Omega(\log n)$. Since this is true for any s , it follows that the diameter of G is $\Theta(\log n)$. ■

Claim 17.2.2. *Let G be an n -vertex 3-regular α -edge-expander and let $B(v, i)$ be the set of all vertices u such that there is a path between u and v with at most i edges. For any vertex v , $|B(v, \log_3 n/2)| \leq \sqrt{n}$.*

²Note that, unlike Margulis' construction, this construction is not very explicit since we don't know how to generate large primes deterministically.

Proof. Note that $B(v, \log_3 n/2)$ is the set of all vertices w such that $\text{dist}(v, w)$ is at most $\log_3 n/2$. As we have seen in the proof of the previous claim, we have $|B(v, \log_3 n/2)| \leq 3^{\log_3 n/2} = \sqrt{n}$. ■

17.2.2 The Multicut Instance

Let n_0, α be as in Theorem 7. Let $n \geq n_0$ and let G be an n -vertex 3-regular α -edge-expander. For each edge e in G , we set the capacity c_e to 1. Now let $X = \{(u, v) | u \notin B(v, \log_3 n/2)\}$. The pairs in X will be the pairs (s_i, t_i) that we want to disconnect. Let (G, X) be the resulting MULTICUT instance.

Claim 17.2.3. *There exists a feasible fractional solution for (G, X) of capacity $O(n/\log n)$.*

Proof. Let $d_e = 2/\log_3 n$, for all e . Note that, since G is 3-regular, G has $3n/2$ edges. Therefore the total capacity of the fractional solution is

$$\sum_e d_e = \frac{3n}{2} \cdot \frac{2}{\log_3 n} = \frac{3n}{\log_3 n}$$

Therefore we only need to show that the solution is feasible. Let (u, v) be a pair in X . Let's consider a path p between u and v . Since u is not in $B(v, \log_3 n/2)$, the path p has more than $\log_3 n/2$ edges (recall that $B(v, i)$ is the set of all vertices u such that there is a path between u and v with at most i edges). Consequently,

$$\sum_{e \in p} d_e > \frac{\log_3 n}{2} \cdot \frac{2}{\log_3 n} = 1$$

■

Claim 17.2.4. *Any integral solution for (G, X) has capacity $\Omega(n)$.*

Proof. Let F be an integral solution for (G, X) . Let $V_1, \dots, V_h$ be the connected components of $G - F$. Fix an i and let v be an arbitrary vertex in the connected component V_i . Note that, for any u in V_i , there is a path between v and u with at most $\log_3 n/2$ edges (if not, (u, v) is a pair in X which contradicts the fact that removing the edges in F disconnects every pair in X). Therefore V_i is contained in $B(v, \log_3 n/2)$. It follows from Claim 17.2.2 that $|V_i| \leq \sqrt{n}$. Since G is an α -edge-expander and $|V_i| \leq |V|/2$, we have $|\delta(V_i)| \geq \alpha|V_i|$, for all i . Consequently,

$$|F| = \frac{1}{2} \sum_{i=1}^h |\delta(V_i)| \geq \frac{\alpha}{2} \sum_{i=1}^h |V_i| = \frac{\alpha n}{2}$$

Therefore F has total capacity $\Omega(n)$ (recall that every edge has unit capacity). ■

Theorem 17.8. *The integrality gap of the MULTICUT LP is $\Omega(\log k)$.*

Proof. Note that $k = |X| = O(n^2)$. It follows from claims 10 and 11 that the LP has integrality gap $\Omega(\log n) = \Omega(\log k)$, as desired. ■

Bibliographic Notes

Multicut is closely related to the Sparsest Cut problem. Initial algorithms for Multicut were based on algorithms for Sparsest Cut. Garg, Vazirani and Yannakakis [117] then used Leighton and Rao's region growing argument (as well as their integrality gap example on expanders for the uniform sparsest cut problem) [189] to obtain a tight $O(\log k)$ bound on the integrality gap for Multicut. The randomized proof that we described is from the work of Calinescu, Karloff and Rabani [51] on the 0-extension problem; their algorithm and analysis eventually led to an optimal bound for approximating an arbitrary metric via random trees [101]. For planar graphs (and more generally any proper minor closed family of graphs) the integrality gap is $O(1)$, as shown by Klein, Plotkin and Rao [177] — the constant depends on the family. There have been several subsequent refinements of the precise dependence of the constant in the integrality gap — see [2]. The $O(\log k)$ bound extends to node-weighted case and the $O(1)$ approximation for planar graphs also extends to the node-weighted case. Multicut is APX-Hard even on trees and in general graphs. Assuming UGC, the problem is known to be hard to approximate to a super-constant factor [68]. For some special case of Multicut based on the structure of the demand graph one can obtain improved approximation ratios [76].

The directed graph version of Multicut turns out to be much more difficult. The flow-cut gap is known to be $\tilde{\Omega}(n^{1/7})$ and the problem is also known to be hard to approximate to almost polynomial factors; these negative results are due to Chuzhoy and Khanna [87]. The best known approximation ratio is $\min\{k, \tilde{O}(n^{11/23})\}$ [5]. Very recently Kawarabayashi and Sidiropoulos obtained a poly-logarithmic approximation for Directed Multicut if G is a planar directed graph [172]. There is a notion of symmetric demands in directed graphs and for that version of Multicut one can get a poly-logarithmic flow-cut gap and approximation; see [72, 179]. This is closely connected to the Feedback Arc Set problem in directed graphs [98, 231].

Chapter 18

Low Diameter Decomposition and Metric Tree Embeddings

18.1 Low Diameter Decompositions

Let $G = (V, E)$ be a graph with non-negative edge lengths $\ell : E \rightarrow \mathbb{R}_+$. The shortest path distances in G define a metric space (V, d) where $d(u, v)$ is the shortest path distance between u and v in G . There are several different ways to decompose a graph based on the application in question. In the context of metrics, a useful notion is to decompose or partition the graph into subgraphs/clusters of small radius/diameter. More precisely, given a graph $G = (V, E)$ we would like to partition V into clusters with vertex sets $V_1, V_2, \dots, V_h$ such that each V_i has diameter at most some given parameter δ . This is of course trivial by choosing the partition with each vertex in its own part. However, the goal in partitioning is to ensure that two vertices u, v that are close to each other are in the same cluster. It is clear that this is impossible to do this deterministically since the graph/metric space is connected. Consider the case of a line for instance. What we can hope for is a randomized partition that probabilistically ensures that any two points u, v are separated only with probability proportional to $d(u, v)/\delta$. The following is a formalization of this notion. We let $\mathcal{P}$ denote the set of all partitions of V . For a partition $P \in \mathcal{P}$ we let E_P be the set of edges/pairs that are separated by P .

Definition 18.1. Let $G = (V, E)$ be a graph with edge lengths $\ell : E \rightarrow \mathbb{R}_+$ inducing a distance $d : V \times V \rightarrow \mathbb{R}_+$ and let Δ be the diameter of the metric space (V, d) . For a given $\delta \in [0, \Delta]$, a low-diameter decomposition with cutting probability parameter α is a probability distribution $\mathcal{D}$ over $\mathcal{P}$ (partitions of V) such that

- For any partition $P = (V_1, V_2, \dots, V_h)$ in the support of the distribution, the

diameter of each (V_i, d) is at most δ .

- For any given u, v , the probability that u and v are in different parts is at most $\alpha \cdot \frac{d(u,v)}{\delta}$. That is $\mathbf{P}[uv \in E_P] \leq \alpha \cdot \frac{d(u,v)}{\delta}$

A low-diameter decomposition scheme with parameter α is a family of algorithms that given any $\delta \in [0, \Delta)$ generates a low-diameter decomposition with cutting/separation probability parameter at most α .

Strong vs weak diameter guarantee: A low-diameter decomposition is said to have the *strong diameter* guarantee if the diameter of each cluster V_i in the induced graphs $G[V_i]$ is at most δ . Note that the preceding definition does not require that because it is based on the metric closure (V, d) of the given graph. The standard definition is called the *weak diameter* guarantee. Some applications require the strong diameter guarantee. We will, by default, work with weak diameter guarantee and mention strong diameter guarantee when needed.

Padded decomposition: We can require more of a low-diameter decomposition. Given a point u let $B_d(u, r) = \{v \in V \mid d(u, v) \leq r\}$ be the ball of radius r around u . In padded decomposition we require that for each u , and for each $r \leq \delta$, the probability of $B(u, r)$ being contained in the same part is at least $e^{-\beta r}$.

Sparse covers: A sparse cover consists of several clusters $V_1, V_2, \dots, V_h$. Each cluster should have weak/strong diameter at most δ . For each u, v with distance at most δ , there must be some cluster V_i that contains both u and v . And no vertex u must be in more than some number s of clusters. The techniques underlying sparse covers and low-diameter decompositions are related though we will mostly work only with low-diameter decompositions.

Main results: The main questions regarding low-diameter decomposition is the smallest α that one can obtain. It turns out that for general metric spaces $\alpha = O(\log n)$ is a tight bound for both strong and weak diameter guarantee; this was formally shown by Bartal [38]. For planar graph metrics $\alpha = O(1)$ is achievable; weak diameter guarantee was first shown by Klein, Plotkin and Rao [178] and the strong diameter guarantee was more difficult and was shown later. These are important results that have many applications. There is substantial work on extending the planar graph results to graphs that exclude a fixed minor - we will not dwell into these more advanced issues in this course. See [111] for some recent work and pointers to literature on these topics.

18.2 CKR algorithm for weak diameter guarantee

Let (V, d) be a metric space with $|V| = n$. We had already seen the CKR-RandomPartition algorithm and analysis previously in the context of the Multi-

Cut algorithm. Implicit in the algorithm is a metric partitioning scheme. We modify the algorithm to ensure that the weak diameter of each cluster is at most a given parameter δ . We will assume that $V = \{v_1, \dots, v_n\}$.

CKR-RandomPartition:

Pick θ uniformly at random from $[0, \delta/2)$

Pick a random permutation σ of $\{1, 2, \dots, n\}$

for $i = 1$ to n

$$V_{\sigma(i)} = B_d(v_{\sigma(i)}, \theta) \setminus \bigcup_{j < i} V_{\sigma(j)}$$

Output the partition formed by $V_1, V_2, \dots, V_n$ (some of them will be empty)

The analysis is essentially the same as we did for Multicut so we simply state the lemmas and encourage the reader to see how to prove them via the previous analysis.

Lemma 18.1. *CKR-RandomPartition correctly outputs a partition of V into clusters, each of which has weak diameter at most δ .*

Lemma 18.2. *The probability that u and v are in different clusters is at most $2H_n \frac{d(u,v)}{\delta}$.*

18.2.1 A subtle modification in the algorithm and guarantee

We indicate a small but subtle change in the algorithm and the guarantee it provides which will be used later in an surprising way in an optimal algorithm for low-stretch spanning trees. The only change in the algorithm is in choosing the radius in the range $[\delta/4, \delta/2)$ rather than from $[0, \delta/2)$.

CKR-RandomPartition:

Pick θ uniformly at random from $[\delta/4, \delta/2)$

Pick a random permutation σ of $\{1, 2, \dots, n\}$

for $i = 1$ to n

$$V_{\sigma(i)} = B_d(v_{\sigma(i)}, \theta) \setminus \bigcup_{j < i} V_{\sigma(j)}$$

Output the partition formed by $V_1, V_2, \dots, V_n$ (some of them will be empty)

The guarantee about the radius/diameter remain the same.

Lemma 18.3. *CKR-RandomPartition correctly outputs a partition of V into clusters, each of which has weak diameter at most δ .*

The main difference is in the probability guarantee which is refinement of the previous bound.

Lemma 18.4. *The probability that u and v are in different clusters is at most $\frac{4d(u,v)}{\delta} \log\left(\frac{|B(u,\delta)|}{|B(u,\delta/8)|}\right)$.*

Proof. We sketch the proof assuming familiarity with the argument from the previous lecture for Multicut. Fix u, v and think of V as $v_1, v_2, \dots, v_n$. Let L_i, R_i be defined as before; distance of u, v from v_i with one of them at L_i and the other at R_i . We have $R_i - L_i \leq d(u, v)$. If $d(u, v) \geq \delta/8$ then the edge is going to get cut with constant probability and the bound is not giving anything interesting so we are primarily interested in $d(u, v) < \delta/8$. As before we assume that we have sorted the vertices that $L_1 \leq L_2 \leq \dots \leq L_n$.

We consider the event A_i which is that v_i is the *first* vertex to separate the pair u, v . We can argue as before that $\mathbf{P}[A_i] \leq \frac{1}{i} \mathbf{P}[\theta \in [L_i, R_i]]$ and this is at most $\frac{4}{i} \cdot \frac{d(u,v)}{\delta}$ since we are choosing the radius from $[\delta/4, \delta/2]$. This is because A_i happens only if i comes first among the first i vertices according the sorting. The new twist is that since we chose $\theta \in [\delta/4, \delta/2]$ and $d(u, v) < \delta/8$, no vertex $v_j \in B(u, \delta/8)$ can separate u, v because $L_j \leq d(v_i, u) < \delta/8$ and $R_j \leq L_j + d(u, v) \leq \delta/8 + \delta/8 \leq \delta/4$. Any such vertex will capture both u and v if they are not already separated. Similarly, if vertex $v_j \notin B(u, \delta)$ then it cannot cut the pair because $L_j \geq \delta - d(u, v) \geq \delta - \delta/8 \geq \delta/2$. Therefore, we can bound A , the probability of separating the pair (u, v) , as follows:

$$\begin{aligned} \mathbf{P}[A] &\leq \sum_{j \in B(u, \delta) \setminus B(u, \delta/8)} \mathbf{P}[A_j] \\ &\leq \frac{4d(u, v)}{\delta} \sum_{j > |B(u, \delta/8)|, j \leq |B(u, \delta)|} \frac{1}{j} \\ &\leq \frac{4d(u, v)}{\delta} \log \frac{|B(u, \delta)|}{|B(u, \delta/8)|} \end{aligned}$$

■

18.3 Randomized region growing for a strong diameter guarantee

We now describe an alternate scheme that also gives a strong diameter guarantee. This is by Bartal [38]. For this we need some properties of the geometric distribution.

Geometric distribution: Recall that $\text{Geom}(p)$ for a parameter $p \in (0, 1]$ is the number of independent Bernoulli trials of a coin with bias p to see the first heads (here p is the probability of the coin coming up heads). If $X \simeq \text{Geom}(p)$ then

$\mathbf{P}[X = k] = (1 - p)^{k-1}p$. The geometric distribution is a memory less discrete distribution (only one). That is, $\mathbf{P}[X > m + n \mid X > n] = \mathbf{P}[X > m]$; this is intuitive from the definition but can be verified formally. The corresponding continuous time distribution is the exponential distribution. The expectation of a random variable X distributed according to $\text{Geom}(p)$ is $1/p$ and the variances is $(1 - p)/p^2$.

Given parameter δ , we can shrink all edges with length at most δ/n^2 and discretize edge lengths to integer values. This does not affect the analysis by more than constant factors. Since we want strong diameter guarantee we will work with the underlying graph and not the metric completion. However, we will assume that for each edge $e = uv$ that $\ell(e)$ is equal to the shortest path distance between u and v in G .

RandomCarving: (G, δ)
 Let $p = \min\{1, \frac{4 \ln n}{\delta}\}$
 While $G \neq \emptyset$ do
 Let v be an arbitrary vertex
 $R_v = \text{Geom}(p)$.
 Let $C_v = B_G(v, R_v)$ be the ball of radius R_v in current graph
 Add C_v to clusters
 $G \leftarrow G - C_v$
 EndWhile
 Output the clusters found during the algorithm

Note that $G - C_v$ is the graph obtained by removing all the vertices in C_v . This may increase the shortest path distances in the remaining graph. We also remark that the choice of p depends on n , the original number of vertices in the graph and does not change while the graph is undergoing changes.

Lemma 18.5. *With probability at least $(1 - 1/n)$, the strong diameter of each cluster output by the algorithm is at most δ .*

Lemma 18.6. *For any $u \neq v$, probability that u and v are in different clusters is $O(\log n)d(uv)/\delta$.*

We refer the reader to the proofs in Gupta's lecture notes.

18.4 Deterministic region growing and duality

The first $O(\log k)$ -approximation algorithm for Multicut was due to Garg, Vazirani and Yannakakis [117] via a deterministic region growing argument. The basic argument is simple and shows up in several places (the use of the argument in bounding flow-cut gaps is credited to Leighton and Rao [189]). Our

goal here is to make a connection to low-diameter network decomposition that we have defined probabilistically. One of the consequence of the probabilistic statement is the following. Suppose we are given a graph $G = (V, E)$, edge lengths $\ell : E \rightarrow \mathbb{R}_+$ and edge weights $w : E \rightarrow \mathbb{R}_+$. Then G can be partitioned into clusters of diameter at most δ such that the total weight of edges between the clusters is $O(\log n)^{\frac{1}{\delta}} \sum_e w_e \ell_e$. To see this we simply apply the probabilistic low diameter decomposition to the metric induced by the edge lengths and use linearity of expectation. Recall that this is how we proved the approximation bound for Multicut where we obtained a slightly refined bound of $O(\log k)$. The deterministic region growing proof for Multicut can be found in the standard approximation textbooks [250, 255]. The proof is typically explained via a continuous time argument and can be somewhat mysterious at first glance. Moreover the proof is given only for Multicut and not for low-diameter decomposition though one can easily adapt it. For this reason we give a proof here while also giving some pedagogical perspective that may be helpful for some readers.

We will define the notion of volume. For a set of edges $A \subseteq E$ we let $\text{vol}(A) = \sum_{e \in A} w_e \ell_e$. For a set of vertices $S \subseteq V$ we overload notation and define $\text{vol}(S) = \text{vol}(E(S) \cup \delta(S))$ (that is, the volume counts all edges that have an end point in S). Fix a vertex v and let R be the radius from v , that is $R = \max_u d(v, u)$ where d is based on the shortest path distances induced by ℓ . When considering the s - t mincut rounding we argued that if we picked a random $r \in (0, R)$ then the $E[w(\delta(S_r))] \leq \text{vol}(V)$ where S_r is the set of vertices in the ball of radius r around v ; technically we had $R = 1$ but the same proof holds by scaling by R . Note that we are bounding the weight of the cut around S_r by the *total volume* of the graph.

Instead we want a lemma that bounds the weight of the cut only with respect to the volume of S_r . This *local guarantee* will allow the algorithm to be used repeatedly to cut the graph into many clusters while still being able to charge to the total volume but now with an approximation factor.

Theorem 18.2. *Let $G = (V, E)$ be a graph with edge lengths $\ell : E \rightarrow \mathbb{R}_+$ and edge weights $w : E \rightarrow \mathbb{R}_+$. Then G can be partitioned into clusters $C_1, C_2, \dots, C_h$ of strong diameter at most δ such that $\sum_{i=1}^h \sum_{e \in \delta(C_i)} w_e \leq O(\log n)^{\frac{1}{\delta}} \sum_{e \in E} w_e \ell_e$.*

Remark 18.1. In the context of Multicut we were interested in $\delta = 1/2$ and $\Delta(G) = 1$. The $1/\delta$ term is necessary when considering smaller diameter clusters.

Note that we are trying to remove as few edges as possible while making the diameter small. First, we will remove all edges e with $\ell_e > \frac{\delta}{c \log n}$ for some sufficiently large constant c and the total weight of these edges is clearly $O(\log n)^{\frac{1}{\delta}} \sum_{e \in E} w_e \ell_e$. Thus, we can assume that $\ell_e < \frac{\delta}{c \log n}$. This is a simple

preprocessing step to simplify the analysis. We will work with the remaining graph and overload notation and assume that it is G .

Fix a vertex v and consider $r_i = i \cdot \delta / (c \log n)$ for $i = 0, 1, 2, \dots$, and let $S_i = B(v, r_i)$ be the set of vertices in the ball of radius r_i . We have $S_0 = \{v\}$ and $S_{i+1} \supset S_i$ for $i \geq 0$.

The claim below follows from the fact that edge lengths were assumed to be at most $\delta / (c \log n)$ and $r_{i+1} = r_i + \delta / (c \log n)$.

Claim 18.4.1. *If $e = uv \in \delta(S_i)$ then $u, v \in S_{i+1}$.*

For each i we define $\text{vol}'(S_i) = \text{vol}(S_i) + \text{vol}(V)/n$. Note that this means that $\text{vol}'(S_0) = \text{vol}(V)/n$. This is a technical scaling trick.

Claim 18.4.2. *There is some $0 \leq i < 1 + \lceil \log n \rceil$ such $\text{vol}'(S_{i+1}) \leq 2 \text{vol}'(S_i)$.*

Proof. Suppose not. Then $\text{vol}'(S_j) \geq 2^j \text{vol}'(S_0)$ for $j = 1 + \lceil \log n \rceil$ but that would imply that $\text{vol}'(S_j) > 2n \cdot \text{vol}'(S_0) = 2 \text{vol}(V)$ which is a contradiction. ■

Lemma 18.7. *There is a radius $r \in [r_i, r_{i+1})$ such that $w(\delta(S_r)) \leq O(\frac{\log n}{\delta})(\text{vol}(S_r) + \text{vol}(V)/n)$.*

Proof. Pick r at random from $[r_i, r_{i+1})$ and consider S_r . The only edges that can be cut are those whose end points are in S_{i+1} and hence they are counted in $\text{vol}'(S_{i+1})$. Let e be such an edge. We have $\mathbf{P}[e \in \delta(S_r)] \leq \frac{\ell(e)}{r_{i+1} - r_i} \leq \frac{c \log n}{\delta} \ell(e)$. Thus the expected cost of $w(S_r)$ is at most

$$\begin{aligned} \frac{c \log n}{\delta} \text{vol}'(S_{i+1}) &\leq 2 \frac{c \log n}{\delta} \text{vol}'(S_i) \\ &\leq 2 \frac{c \log n}{\delta} (\text{vol}(S_i) + \text{vol}(V)/n) \\ &\leq 2 \frac{c \log n}{\delta} (\text{vol}(S_r) + \text{vol}(V)/n). \end{aligned}$$

■

Remark 18.2. The analysis applies to directed graphs where we obtain a guarantee about $w(\delta^+(S_r))$. Note that S_r has the property that every vertex is reachable from v within a distance of $r \leq \delta/2$, however, removing $\delta^+(S_r)$ may still leave incoming arcs.

We can find the radius r deterministically by trying all possible values of r in $[r_i, r_{i+1})$ since there are at most n interesting values when S_r changes. Thus, we have shown that there is a radius $r \leq (1 + 2 \log n) \delta / (c \log n)$ such that $w(\delta(B(v, r))) \leq 2 \frac{c \log n}{\delta} (\text{vol}(B(v, r)) + \text{vol}(V)/n)$. Note that the radius of S_r is at

most $\delta/2$ if we choose c large enough. Thus the strong diameter of S_r is at most δ . This is the first cluster. We remove $B(v, r)$ from G and repeat the process with a remaining vertex to find the next cluster. The process stops when there is no more vertices left in the graph. Let $v_1, v_2, \dots, v_h$ be the centers of the vertices chosen in the clustering process and let $C_1, C_2, \dots, C_h$ be the clusters found. The construction ensures that the strong diameter of each cluster is at most δ . For each C_i we have seen that $w(\delta(C_i)) \leq O(\frac{\log n}{\delta})(\text{vol}(C_i) + \text{vol}(V)/n)$. Note that an edge e contributes to the volume of at most two clusters. Thus,

$$\sum_{i=1}^h w(\delta(C_i)) \leq O\left(\frac{\log n}{\delta}\right) \text{vol}(V).$$

18.4.1 Using LP Duality to Obtain an LDD

We will now show how one can use Theorem 18.2 to obtain a probabilistic low diameter decomposition as we initially defined it. For this we rely on a useful trick that is used often but also not known as widely as it should be. Recall that an LDD is a distributions over partitions P such that clusters in P have diameter at most δ . Given a graph $G = (V, E)$ with edge lengths $\ell : E \rightarrow \mathbb{R}_+$ we define $\mathcal{P}$ to be the set of all partitions of V which are *valid* — that is, $P \in \mathcal{P}$ if each cluster of P has diameter at most δ . This is an exponentially large set and we are just defining it implicitly. Let E_P be the set of edges crossing the partition P (in other words cut by P). Any probability distribution $\mathcal{D}$ of $\mathcal{P}$ can be defined by numbers $y_P, P \in \mathcal{P}$ such that $\sum_P y_P = 1$ and $\bar{y} \geq 0$. The probability that an edge $e = uv$ is cut by the distribution y is $\sum_{P: e \in E_P} y_P$. Let this be denoted by $\alpha_{y,e}$. We want to find the distribution y to minimize $\alpha_{y,e} \ell(e)/\delta$ since we are interested in the scaled probability. We can write a large implicit LP to find the best y .

$$\begin{array}{ll} \min & \alpha \\ \text{s.t} & \\ & \sum_{P \in \mathcal{P}} y_P = 1 \\ & y_P \geq 0 \quad P \in \mathcal{P} \\ & \sum_{P: e \in E_P} y_P \leq \alpha \ell(e) \quad e \in E \end{array}$$

We write the dual LP

$$\begin{aligned}
 & \max \beta \\
 & \text{s.t} \\
 & \sum_{e \in E} \ell(e) x_e = 1 \\
 & \beta \geq \sum_{e \in E_P} x_e \quad P \in \mathcal{P} \\
 & x_e \geq 0 \quad e \in E
 \end{aligned}$$

It may be a bit hard to interpret the dual at first glance but it is worth rewriting it as the following non-linear problem:

$$\beta = \max_{x \in \mathbb{R}_+^m} \min_{P \in \mathcal{P}} \frac{\sum_{e \in E_P} x_e}{\sum_{e \in E} \ell(e) x_e}$$

Exercise 18.1. Verify that the two optimization problems are the same.

We claim that the following upper bound on β .

Claim 18.4.3. $\beta = O(\frac{\log n}{\delta})$.

Proof. Suppose x^* is an optimum solution to the dual. Think of x_e^* as the weight w_e in the setting of Theorem 18.2. Then we know that there is a partition $P \in \mathcal{P}$ such that $\sum_{e \in E_P} x_e^* \leq O(\frac{\log n}{\delta}) \sum_{e \in E} x_e^* \ell(e)$. Hence $\beta = O(\frac{\log n}{\delta})$. ■

Thus we have obtained an existential proof that a good probabilistic LDD exists. But this seems not very useful since we want an efficient algorithm to sample from this distribution. It turns out that one can use an algorithmic version of Theorem 18.2 to obtain an efficient algorithm for creating and sampling the desired distribution. This is based on the multiplicative weight update (MWU) method. We will not discuss the technical details of this but refer the reader to old papers of the instructor with his co-authors which employed this methodology to obtain deterministic algorithms based on tree embeddings [63, 64].

18.5 LDDs and Probabilistic Embedding of Metrics into Dominating Tree Metrics

Using tree representations of graphs is a powerful tool in algorithm design. Here we will be interested in representing the distances in an undirected graph

via distances in a tree. Let $G = (V, E)$ be a graph with non-negative edge length $\ell : E \rightarrow \mathbb{R}_+$. This naturally defines a metric (V, d) via shortest path distances.

Can we approximate distances in G by a spanning tree $T = (V, E_T)$ of G ? Clearly $d_T(u, v) \geq d_G(u, v)$ for every pair u, v where $d_T(u, v)$ is the distance in the tree and $d_G(u, v)$ is the distance in the graph. An extreme example is the cycle C on n vertices with unit edge lengths. A spanning tree corresponds to removing one edge (u, v) from C and in the resulting tree T , $d_T(u, v) = n - 1$ while $d_G(u, v) = 1$.

Exercise 18.2. Prove that for every weighted graph G there is a spanning tree T such that $d_T(u, v) \leq (n - 1)d_G(u, v)$.

Alon et al [13], motivated by applications of spanning tree based metric approximations, noticed that if we are allowed to pick a probability distribution over spanning trees then the expected distance for any pair of vertices can be much better than the above worst-case example shows. For instance, in the cycle, if we pick a spanning tree at random from the n trees of the cycle then for any given pair (u, v) the *expected* distance is only 2! This kind of guarantee suffices for many applications. [13] showed that for any weighted graph $G = (V, E)$ there is a distribution $\mathcal{D}$ over spanning trees of G such that for any u, v ,

$$E[d_T(u, v)] \leq \exp(\sqrt{\log n \log \log n}) \cdot d_G(u, v)$$

We think of this as probabilistic approximation of a graph metric by tree metrics. This can be viewed also as a metric *embedding* result. Note that the distances only increase so we are interested in the worst-case guarantee of how much the expected distance for any pair increases? In keeping with metric embedding terminology this quantity is often called *distortion* or *stretch*.

Alon et al proved a lower bound of $\Omega(\log n)$ on the distortion required for probabilistic tree approximation and conjectured that this is the tight bound. It turned out to be quite difficult to obtain even a poly-logarithmic bound. It was finally shown by Elkin et al. [95], and currently the best known bound is $O(\log n \log \log n)$ [1]. Removing the $\log \log n$ factor is an interesting and non-trivial open problem (or proving a stronger lower bound than $\Omega(\log n)$).

Bartal [38] overcame the difficulty of spanning tree embeddings by considering metric embeddings. What does this mean? Instead of starting with the topology of the graph G we work with the metric completion (V, d) and view it as a complete graph on V . Any spanning tree of the complete graph is now allowed. Note that such a tree may not correspond to a spanning tree of the original graph G ; for instance consider the cycle C and its metric completion. As Bartal noted, it can have additional vertices.

Definition 18.3. A tree (V_T, E_T) with edge lengths $\ell_T : E \rightarrow \mathbb{R}_+$ is a dominating tree metric for a finite metric (V, d) if $V \subseteq V_T$ and for all $u, v \in V$, $d_T(u, v) \geq d(u, v)$ where d_T is the metric induced on V_T by the edge lengths of T .

Bartal considered approximating metrics probabilistically by dominating tree metrics.

Definition 18.4. Let (V, d) be a metric space. A probabilistic approximation of (V, d) by dominating tree metrics is a probability distribution $\mathcal{D}$ over a collection of trees $T_1, T_2, \dots, T_h$ if each T_i is a dominating tree metric for (V, d) . The distribution has stretch/distortion α if for all $u, v \in V$,

$$E_{\mathcal{D}}[d_T(u, v)] \leq \alpha \cdot d(u, v).$$

Bartal developed LDDs and showed that one can use them to efficiently sample from a distribution that has stretch $O(\log^2 n)$ [38]. He subsequently improved the stretch bound to $O(\log n \log \log n)$ [37] and this was finally improved to the optimal $O(\log n)$ bound by Fakcharaenphol, Talwar and Rao [100] using the CKR-RandomPartition procedure. The construction of the trees from LDDs is quite simple in retrospect and we do the basic construction and then show how the subtle properties of the CKR procedure yields the optimal bound.

18.5.1 Low stretch trees via LDDs

Bartal's construction is based a recursive use of LDDs. The high-level idea is quite simple. Let (V, d) be a metric space with diameter Δ . We use a LDD with parameter $\delta = \Delta/2$ to randomly partition V into clusters $V_1, V_2, \dots, V_h$ of diameter at most $\Delta/2$. We then recursively find a tree for each of the V_i , say with root r_i . We create a new dummy root r and connect each r_i to r with an edge of length Δ . The base case is when $|V| = 1$ when we simply return a tree with the root as the lone vertex. That's it! We can easily prove that this yields a stretch of $O(\log n \log \Delta)$ (here we assume that the minimum distance is at least 1) which depends on the aspect ratio but this can be easily altered to get stretch $O(\log^2 n)$. To be more formal, especially for the next section, we use the notion of levels and use $\delta = \Delta/2^i$ at level i to make the analysis cleaner.

```

TreeEmbed:((V, d), D)
  if |V| = 1 then
    return tree with single vertex  $v \in V$  as root
  Create tree  $T$  with root  $r$ 
  Use LDD with on  $(V, d)$  with  $\delta = D/2$  to obtain clusters  $V_1, V_2, \dots, V_h$ 
  For  $j = 1$  to  $h$ 
    Let  $T_j$  be output of TreeEmbed(( $V_j, d$ ),  $D/2$ ) with root  $r_j$ 
    Connect  $T_j$  to  $T$  by adding edge  $(r, r_j)$  of length  $D$ 
  return  $T$  with root  $r$ 
    
```

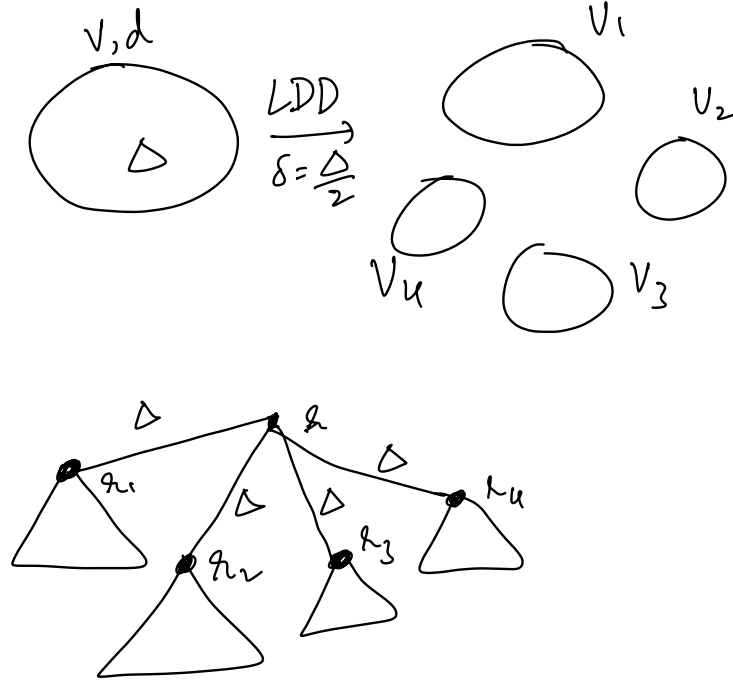


Figure 18.1: Illustration of recursive tree construction

We will assume that the minimum non-zero distance in the metric space is 1 by scaling. Let Δ be the diameter of the metric space with this assumption.

Theorem 18.5. *Let Δ be the diameter of (V, d) . The algorithm **TreeEmbed** $((V, d), \Delta)$ outputs a random dominating tree metric $T = (V_T, E_T, \ell_T)$ such that for each $u, v \in V$, $E[d_T(u, v)] \leq O(\alpha \log \Delta)d(u, v)$ where α is the cutting probability of the LDD procedure.*

The proof is by induction. The base case is easy since the tree has a singleton and there is nothing to prove. It is easy to see that if we start with $D = \Delta$ then at depth i of the recursion the parameter is $\Delta/2^{i-1}$ and it is an upper bound on the diameter of the metric space in that recursive call.

We omit the proof of the following easy claim.

Claim 18.5.1. *The length of the root to leaf path of a tree created at level i of the recursion is at most $\sum_{j \geq i} \Delta/2^{j-1} \leq 2\Delta/2^{i-1}$.*

The following should be easy to see from the construction.

Claim 18.5.2. *The output of the algorithm is a dominating metric for (V, d) .*

Fix a pair of vertices u, v with distance $d(u, v)$.

Claim 18.5.3. *Suppose u and v are first separated at level i of the recursion. Then $d_T(u, v) \leq 4\Delta/2^{i-1}$.*

Proof. Follows from the previous claim. ■

The pair u, v are separated in the first level of the recursion due to the LDD with probability $\alpha d(u, v)/(\Delta/2) \leq 2\alpha d(u, v)/\Delta$ in which case their distance in the tree is at most 4Δ . Otherwise they are in the same cluster and we can apply induction. Note that u and v are definitely separated by level t where t is the smallest integer such that $\Delta/2^{t+1} < d(u, v)$. The depth of the recursion is at most $1 + \lceil \log \Delta \rceil \leq 2 \log \Delta$.

It is easy to unroll the induction and use the preceding claim to obtain:

$$E[d_T(u, v)] \leq \sum_{i=0}^{t+1} 2\alpha \frac{d(u, v)}{(\Delta/2^{i-1})} \cdot (4 \frac{\Delta}{2^{i-1}}) \leq O(\alpha \log \Delta) d(u, v)$$

since the depth of the recursion is $O(\log \Delta)$.

Remark 18.3. The tree may require depth $\log \Delta$ to provide a good approximation. Note that in general $\log \Delta$ can be as large as n and moreover one cannot expect the depth of the tree (in terms of number of edges) to be less than $\log \Delta$ if one wants reasonable distortion. Consider the metric induced by a path with n edges and edge lengths 2^i , $i = 1$ to n . In such cases the dependence of the stretch on $\log \Delta$ is undesirable. One can alter the algorithm slightly to make the stretch bound $O(\log^2 n)$ as follows. In applying the LDD with parameter δ , we ensure that any pair u, v such that $d(u, v) \leq \delta/n^2$ is not cut during the procedure. We can do this by contracting all such pairs without changing the diameter of the resulting metric space too much. This will ensure that in the tree construction process a pair u, v participates in only $O(\log n)$ levels, and hence the expected stretch can be bounded by $O(\alpha \log n)$.

Hierarchically well-separated trees (HSTs): The reader may have noticed that the trees constructed by the algorithm have an additional strong property: the edge lengths at each level are the same and the length from the root to the leaf go down by a factor of 2 at each level (we can adjust this factor which will of course impact the stretch factor correspondingly). A tree metric with such a property is called hierarchically well-separated tree metric and this additional property can be exploited in algorithms and comes for free in the construction.

18.5.2 An optimal construction

[100] showed that if one used the LDD based on the CKR-RandomPartition in the TreeEmbed algorithm then the expected stretch is $O(\log n)$ which is optimal! We recall the guarantee of the LDD which is refined one.

Lemma 18.8. *The probability that u and v are in different clusters is at most $\frac{4d(u,v)}{\delta} \log\left(\frac{|B(u,\delta)|}{|B(u,\delta/8)|}\right)$.*

Thus, the LDD guarantee is no longer uniform but depends on the scale δ . We plug this refined bound into the previous expression for the stretch.

$$\begin{aligned} E[d_T(u, v)] &\leq \sum_{i=0}^{t+1} 2 \log\left(\frac{|B(u, \Delta/2^i)|}{|B(u, \Delta/2^{i+3})|}\right) \frac{d(u, v)}{(\Delta/2^{i-1})} \cdot \left(4 \frac{\Delta}{2^{i-1}}\right) \\ &\leq 8d(u, v)(\log |B(u, \Delta)| + \log |B(u, \Delta/2)| + \log |B(u, \Delta/4)|) \\ &\leq O(\log n)d(u, v). \end{aligned}$$

Lower bounds: How can we prove that the $O(\log n)$ bound for LDDs and tree embeddings are near optimal (modulo precise constant factors)? One can use existence of low-girth graphs which are closely tied to expanders in a direct fashion — see Gupta’s notes. Another way is via indirection. In the previous lecture we saw that the integrality gap of the LP relaxation for Multicut is $\Omega(\log k)$. Note that we prove an upper bound of $O(\log k)$ essentially via an LDD. In fact if α is the factor for LDD then we get an $O(\alpha)$ approximation for Multicut via the LP. Thus, one sees that $\alpha = \Omega(\log n)$ for general metrics (in the integrality gap example for Multicut $k = \Omega(n^2)$). One can use similar approach to prove lower bound for tree embeddings.

Fast Algorithms for LDDs and Tree Embeddings: We focused on the quality of LDDs and tree embeddings but not so much on the running times. It is easy to see that the algorithms themselves can be implemented in polynomial-time. The main computation is about shortest paths and if one computes all-pairs-shortest paths (APSP) then the algorithms are pretty simple. However APSP is slow; takes $O(mn)$ time. It is possible compute LDDs and metric tree embeddings in close to linear time on a weighted graph with m edges. This involves computing approximate shortest paths and several tricks and sometimes we give up on the quality of the approximation by logarithmic factors. See [42] for an $O(m \log n)$ time algorithm for finding an $O(\log n)$ stretch (in expectation) tree.

Chapter 19

Sparsest Cut

SPARSEST CUT is a fundamental problem in graph algorithms with many applications and connections. There are several variants that are considered in the literature and they are closely related but it is useful to have proper terminology and understand the similarities and differences.

NON-UNIFORM SPARSEST CUT: We consider the general one first. The input is a graph $G = (V, E)$ with non-negative edge capacities $c : E \rightarrow \mathbb{R}_+$ and a set of pairs $(s_1, t_1), \dots, (s_k, t_k)$ along with non-negative demand values $D_1, D_2, \dots, D_k$. When considering undirected graphs the demand pairs are unordered — by this we mean that we do not distinguish (s_1, t_1) from (t_1, s_1) . One can also think of the demand values as “weights” but the demand terminology makes more sense when considering the dual flow problem. Given a set/cut $S \subseteq V$ the *sparsity* of the cut S is defined as the ratio $\frac{c(\delta(S))}{(\sum_{i: |S \cap \{s_i, t_i\}|=1} D_i)}$. The numerator is the capacity of the cut and the denominator is the total demand of the pairs separated by S . The goal is to find the cut S with minimum sparsity. In other words we are trying to find the best “bang per buck” cut: how much capacity do we need to remove per amount of demand separated? It is sometime convenient to consider G as the supply graph and the demands as forming a demand graph $H = (V, F)$ where F represents the pairs and we associate $D : F \rightarrow \mathbb{R}_+$ to represent the demand value (alternatively we can also consider multigraphs). With this representation of the demand pairs, the sparsity of cut S is simply $\frac{c(\delta_G(S))}{D(\delta_H(S))}$ note that $\delta_G(S)$ represents the supply edges crossing S and $\delta_H(S)$ represents the demand edges crossing the cut S .

Remark 19.1. One can define a cut as removing a set of edges. This may lead to more than two components. In the case of sparsest cut in undirected graphs it suffices to restrict attention to cuts of the form $\delta(S)$ for some $S \subseteq V$. It is a useful exercise to see why there is always a sparsest cut of that form for any given

instance. This is not necessarily true for directed graphs or even in undirected graphs with node-weights or in hypergraphs.

UNIFORM SPARSEST CUT: Very often when people say SPARSEST CUT they mean the uniform version. This is the version in which $D(u, v) = 1$ for each unordered pair of vertices (u, v) . For these demands a cut S is $\frac{c(\delta_G(S))}{|S||V \setminus S|}$. Alternatively the demand graph H is a complete graph with unit demand values on each edge. A slight generalization of UNIFORM SPARSEST CUT is obtained by considering demands induced by weights on vertices (the dual flow instances are called PRODUCT MULTICOMMODITY FLOW instances). There is a weight function $\pi : V \rightarrow \mathbb{R}_+$ on the vertices and demand $D(u, v)$ for pair (u, v) is set to be $\pi(u)\pi(v)$. Note that if $\pi(u) = 1$ for all u then we obtain UNIFORM SPARSEST CUT. If $\pi(u) \in \{0, 1\}$ for all u then we are focusing our attention on sparsity with respect to the set $V' = \{v \mid \pi(v) = 1\}$ since the vertices with $\pi(u) = 0$ play no role. This may seem unnatural at first but it is closely connected to expansion and conductance as we will see below.

EXPANSION: The expansion of a multi-graph $G = (V, E)$ is defined as $\min_{S: |S| \leq |V|/2} \frac{|\delta(S)|}{|S|}$. Recall that G is an α -expander if the expansion of G is at least α . A random 3-regular graph is α -expander with $\alpha = \Omega(1)$ with high probability. Thus, to find an α -expander one can obtain an efficient randomized algorithm by picking a random graph and then verifying its expansion. However, checking expansion is coNP-Hard. EXPANSION is closely related to UNIFORM SPARSEST CUT. Note that when $|S| \leq |V|/2$ we have

$$\frac{1}{|V|} \frac{|\delta(S)|}{|S|} \leq \frac{|\delta(S)|}{|S||V \setminus S|} \leq \frac{2}{|V|} \frac{|\delta(S)|}{|S|}.$$

Thus EXPANSION and UNIFORM SPARSEST CUT are within a factor of 2 of each other. Sometimes it is useful to consider expansion with vertex weights $w : V \rightarrow \mathbb{R}_+$. Here the expansion is defined as $\min_{S: w(S) \leq w(V)/2} \frac{|\delta(S)|}{w(S)}$. This corresponds to product multicommodity flow instances where $\pi(v) = w(v)$. The term CONDUCTANCE is used to denote the quantity $\frac{|\delta(S)|}{\text{vol}(S)}$ where $\text{vol}(S) = \sum_{v \in S} \deg(v)$ (here vol is short for volume). When a graph is regular the definition of expansion and conductance are the same but not in the general setting. Note that we can capture conductance by setting weights on vertices where $w(v) = \deg(v)$. Conductance arises naturally when considering random walks in graphs. The stationary distribution of a random walk in an undirected graph (we will assume the graph is connected and not bipartite) is the vector $\bar{p}$ where the probability of being in vertex v is $p(v) = \deg(v)/(2m)$. How fast does the random walk converge to the stationary distribution? This is governed by the conductance.

Some key applications: UNIFORM SPARSEST CUT is fundamentally interesting because it helps us directly and indirectly solve the BALANCED SEPARATOR problem. In the latter problem we want to partition $G = (V, E)$ into two pieces $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ where $|V_1|$ and $|V_2|$ are roughly the same size so that we minimize the number of edges between V_1 and V_2 . One can repeatedly use a sparse cut routine to get a balanced separator. The other key application is to certify expansion of a graph. Expander graphs and relatives arise in many applications and knowing whether a graph is expanding or not is very useful.

19.1 LP Relaxation and Maximum Concurrent Flow

How do we write an LP relaxation for SPARSEST CUT? This is less obvious than it is for MULTICUT and other cut problems where we have explicit terminal pairs that we wish to separate. We consider NON-UNIFORM SPARSEST CUT. First we develop an integer program. We have two sets of variables. For each pair (s_i, t_i) we have a variable y_i to indicate whether we want to separate the pair i . For each edge we have a variable x_e to indicate whether e is cut. If we decide to separate pair i then for every path between s_i and t_i we should cut at least one edge on the path — this is similar to relaxations we have seen before. We let $\mathcal{P}_{s_i, t_i}$ be the set of all s_i - t_i paths. The following captures the problem:

$$\begin{aligned} \min \quad & \frac{\sum_{e \in E} c_e x_e}{\sum_{i=1}^k D_i y_i} \\ & \sum_{e \in p} x_e \geq y_i \quad p \in \mathcal{P}_{s_i, t_i}, i \in [k] \\ & x_e \in \{0, 1\} \quad e \in E \\ & y_i \in \{0, 1\} \quad i \in [k] \end{aligned}$$

Note, however, that the objective is a ratio and not linear. It is a standard trick to obtain an LP relaxation wherein we normalize the denominator in the ratio to 1 and relax the variables to be real-valued. Thus we obtain the following LP relaxation.

$$\begin{aligned}
\min \quad & \sum_{e \in E} c_e x_e \\
\sum_{i=1}^k D_i y_i &= 1 \\
\sum_{e \in p} x_e &\geq y_i \quad p \in \mathcal{P}_{s_i, t_i}, i \in [k] \\
x_e &\geq 0 \quad e \in E \\
y_i &\geq 0 \quad i \in [k]
\end{aligned}$$

Exercise 19.1. Show that the LP is indeed a relaxation for the SPARSEST CUT problem. Formally, given an integer feasible solution with sparsity λ find a feasible solution to the relaxation such that its value is no more than λ .

Now we consider the dual LP. For each path $p \in \cup_i \mathcal{P}_{s_i, t_i}$ there is a non-negative variable y_p which is the amount of “flow” sent on path p . There is a variable λ that we will interpret later.

$$\begin{aligned}
\max \quad & \lambda \\
\sum_{p \in \mathcal{P}_{s_i, t_i}} y_p &\geq \lambda D_i \quad i \in [k] \\
\sum_{i=1}^k \sum_{p \in \mathcal{P}_{s_i, t_i}, e \in p} y_p &\leq c_e \quad e \in E \\
y_p &\geq 0 \quad p \in \mathcal{P}_{s_i, t_i}, i \in [k]
\end{aligned}$$

The dual LP is a multicommodity flow. It solves the MAXIMUM CONCURRENT MULTICOMMODITY FLOW problem for the given instance. It finds the largest value of λ such that there is a feasible multicommodity flow for the given pairs in which the flow routed for pair (s_i, t_i) is at least λD_i . It is called *concurrent flow* since we need to route all demand pairs to the same factor which is in contrast to the dual of Multicut which corresponds to the maximum throughput multicommodity flow (in which some pairs may have zero flow while others have a lot of flow).

Exercise 19.2. Suppose we have a cut S with sparsity $c(\delta(S))/(\sum_{i: |S \cap \{s_i, t_i\}|=1} D_i)$. Why is the maximum concurrent flow at most the sparsity of S ?

Note that the LP can be solved via the Ellipsoid method. One can also write a compact LP via distance variables which will help us later to focus on constraining the metric in other ways.

$$\begin{aligned} \min \quad & \sum_{uv \in E} c(uv)d(uv) \\ \text{subject to} \quad & \sum_{i=1}^k D_i d(s_i t_i) = 1 \\ & d \text{ is a metric on } V \end{aligned}$$

Flow-cut gap: The flow-cut gap in this context is the following equivalent way of thinking about the problem. Consider a multicommodity flow instance on G with demand pairs $(s_1, t_1), \dots, (s_k, t_k)$ and demand values $D_1, \dots, D_k$. Suppose G satisfies the *cut-condition*, that is, for every $S \subseteq V$ the capacity $c(\delta(S))$ is at least the demand separated by S . Can we route all the demand pairs? This is true when $k = 1$ but is not true in general even for $k = 3$ in undirected graphs. The question is the maximum value of λ such that we can route λD_i for every pair i ? The worst-case integrality gap of the preceding LP relaxation for SPARSEST CUT is precisely the flow-cut gap. One can ask about the flow-cut gap for all graphs, a specific class of graphs, for a specific class of demand graphs, a specific class of supply and demand graphs, and so on.

In these notes we will establish that the flow-cut gap in general undirected graphs is at most $O(\log k)$. And there are instances where the gap is $\Omega(\log k)$ which are uniform instances — in fact the same expander based example we saw for MULTICUT shows that the gap is $\Omega(\log n)$ even for UNIFORM SPARSEST CUT. It is conjectured that the gap is $O(1)$ for planar graphs but the best upper bound we have is $O(\sqrt{\log n})$. Resolving the flow-cut gap in planar graphs is a major open problem.

Exercise 19.3. Use the expander construction that we saw for MULTICUT to show that the flow-cut gap for UNIFORM SPARSEST CUT can be $\Omega(\log n)$.

Remark 19.2. Approximating the SPARSEST CUT problem is not the same as establishing flow-cut gaps. One can obtain improved approximations for SPARSEST CUT via stronger relaxations than the natural LP. Indeed the best approximation ratio for SPARSEST CUT is $O(\sqrt{\log n})$ via an SDP relaxation.

19.2 Rounding the LP via Connection to Multicut

There are close connections between SPARSEST CUT and MULTICUT. By repeatedly using SPARSEST CUT routine and SET COVER style analysis prove the following.

Exercise 19.4. Suppose there is an $\alpha(k, n)$ -approximation for NON-UNIFORM SPARSEST CUT. Prove that this implies an $O(\alpha(k, n) \ln k)$ -approximation for MULTICUT.

Can we prove some form a converse? That is, can we use an approximation algorithm for MULTICUT to obtain an approximation algorithm for SPARSEST CUT? Note that if someone told us the pairs to separate in an optimum solution to the SPARSEST CUT instance then we can use an (approximation) algorithm for MULTICUT to separate those pairs. Here we show that one can use information from the LP solution to figure out which pairs to separate. We sketch the argument and focus our attention on the simpler case when $D_i = 1$ for all $i \in [k]$. We give this argument even though it does not lead to the optimum ratio, for historical interest, as well as to illustrate a useful high-level technique that has found applications in other settings.

Identifying the pairs to separate from LP solution: Suppose we solve the LP and obtain a feasible solution (x, y) . y_i indicates the extent to which pair i is separated. Suppose we have an ideal situation where $y_i \in \{0, p\}$ for every i . Let $A = \{i \mid y_i = p\}$. We have $|A| = 1/p$ since $\sum_i y_i = 1$. Then it is intuitively clear that the LP is separating the pairs in A . We can then solve the Multicut problem for the pairs in A and consider the ratio of the cost of the cut to $|A|$. How do we argue about this algorithm? We do the following. Consider a fractional assignment $x' : E \rightarrow \mathbb{R}_+$ where $x'_e = \min\{1, x_e/p\}$; in other words we scale each x_e by $1/p$. Note that $y_i = d_x(s_i, t_i)$. Since we scaled up each x_e by $1/p$ it is not hard to see that $d_{x'}(s_i, t_i) \geq 1$; in other words x' is a feasible solution to the Multicut instance on G for the pairs in A . The fractional cost of x' is $\sum_e c_e x'_e \leq \sum_e c_e x_e / p$. Thus, by the algorithm for Multicut in the previous chapter, we can find a feasible Multicut $E' \subseteq E$ that separates all pairs in A and $c(E') = O(\log k) \sum_e c_e x_e / p$. What is the sparsity of this cut? It is $c(E')/|A|$ which is $O(\log k) \sum_e x_e$. Thus the sparsity of the cut is $O(\log k)\lambda$ where λ is the value of the LP relaxation.

Now we consider the general setting. Recall that $\sum_i y_i = 1$. We partition the pairs into groups that have similar y_i values. For $j \geq 0$, let $A_j = \{i \mid y_i \in (1/2^{j+1}, 1/2^j]\}$. Thus all pairs in A_j have a y_i value that are within a factor of 2 of each other.

Claim 19.2.1. *There exists a $j \leq \log_2 k$ such that $\sum_{i \in A_j} y_i \geq \frac{1}{2(1+\log_2 k)} \geq \frac{1}{4 \log k}$.*

Proof. Consider any i such that $i \in A_j$ where $j > \log_2 k$. By definition we have $y_i \leq 1/2^j$. Since there are only k pairs, $\sum_{j > \log_2 k} \sum_{i \in A_j} y_i \leq k/2^j \leq 1/2$. Thus $\sum_{j \leq \log_2 k} \sum_{i \in A_j} y_i \geq 1/2$ and therefore, there must be a $j \leq \log_2 k$ such that $\sum_{i \in A_j} y_i \geq \frac{1}{2(1+\log_2 k)}$ (there are only so many groups). ■

Consider the A_j with $\sum_{i \in A_j} y_i \geq \frac{1}{4 \log_2 k}$. For each $i \in A_j$ we have $1/2^{j+1} \leq y_i \leq 1/2^j$. Therefore $|A_j| \geq \min\{1, 2^j/4 \log_2 k\}$. The algorithm now separates the pairs in A_j via an algorithm for Multicut.

Claim 19.2.2. *Consider the fractional solution $x' : E \rightarrow [0, 1]$ where $x'_e = \min\{1, 2^{j+1} x_e\}$. Then $d_{x'}(s_i, t_i) \geq 1$ for all $i \in A_j$. Thus x' is a feasible fractional solution to the Multicut LP for separating the pairs in A_j .*

Via the rounding algorithm in the preceding chapter we have there is a set $E' \subseteq E$ such that E' is a feasible multicut for the pairs in A_j and $c(E') = O(\log k) 2^{j+1} \sum_e c_e x_e$. The sparsity of this cut is $c(E')/|A_j| = O(\log^2 k) \sum_e c_e x_e$. Thus we obtained an $O(\log^2 k)$ -approximation for SPARSEST CUT when $D_i = 1$ for each pair.

Remark 19.3. When demands are not 1 (or identical) the preceding argument yields an $O(\log k \log D)$ approximation where $D = \sum_i D_i$ with the normalization that $D_i \geq 1$ for all i .

19.3 Rounding via ℓ_1 embeddings

Leighton and Rao [189], in their seminal work, obtained an $O(\log n)$ approximation and flow-cut gap for UNIFORM SPARSEST CUT and showed many applications. They also showed a lower bound of $\Omega(\log n)$ on the flow-cut gap for UNIFORM SPARSEST CUT via expanders which was also an important connection. This led to $O(\log^2 n)$ -approximation for SPARSEST CUT and it was an open problem to obtain a tight conjectured bound of $O(\log n)$. The optimal rounding of the LP relaxation turns out to go via metric embedding theory and this connection was discovered by Linial, London and Rabinovich [194] and Aumann and Rabani [26]. We need some basics in metric embeddings to point out the connection and rounding. Even though the metric embedding machinery is powerful, it can seem like magic. The more basic ideas for UNIFORM SPARSEST CUT based on region growing is useful to know. One can find the details in [189] or in a more accessible form in the book of Williamson and Shmoys [255].

19.3.1 A digression through trees

It is instructive to consider the simple setting when G is a tree $T = (V, E)$. In this case it is easy to find the sparsest cut. For each edge $e \in T$ we can associate a cut S_e which is one side of the two components in $T - e$. The capacity of the cut $\delta(S_e)$, by definition, is c_e . Let $D(e) = \sum_{i: |S_e \cap \{s_i, t_i\}|=1} D_i$ be the demand separated by e . The sparsity of the cut S_e is simply c_e/D_e . Finding sparsest cut in a tree is easy from the following exercise.

Exercise 19.5. The sparsest cut in a tree is given by $\arg \min_e c_e/D_e$.

A more interesting exercise is to prove that the LP relaxation gives an optimum solution on a tree.

Lemma 19.1. *Let (x, y) be a feasible solution to the LP with objective value λ . If G is a tree T then there is an edge $e \in T$ such that $c_e/D_e \leq \lambda$.*

Proof. We have $\lambda = \frac{\sum_e c_e x_e}{\sum_i D_i d_x(s_i, t_i)}$ where $d_x(s_i, t_i)$ is the shortest path distance between s_i and t_i . There is a unique path P_{s_i, t_i} from s_i to t_i in a tree so $d_x(s_i, t_i) = \sum_{e \in P_{s_i, t_i}} x_e$. Thus,

$$\begin{aligned} \lambda &= \frac{\sum_e c_e x_e}{\sum_i D_i d_x(s_i, t_i)} \\ &= \frac{\sum_e c_e x_e}{\sum_i D_i \sum_{e \in P_{s_i, t_i}} x_e} \\ &= \frac{\sum_e c_e x_e}{\sum_e D_e x_e} \\ &\geq \min_e \frac{c_e}{D_e}. \end{aligned}$$

In the last inequality we are using the simple fact that $\frac{a_1+a_2+\dots+a_n}{b_1+b_2+\dots+b_n} \geq \min_i \frac{a_i}{b_i}$ for positive a 's and b 's. ■

What made the proof work for trees? Is there a more general phenomenon than the fact that trees are pretty simple structures? It turns out that the key fact is that shortest path distances induced by a tree are ℓ_1 metrics or equivalently cut metrics.

19.3.2 Cut metrics, line metrics, and ℓ_1 metrics

Let (V, d) be a finite metric space. We will be interested in two special types of metrics.

Definition 19.1. *Let V be a finite set and let $S \subseteq V$. The metric d_S associated with the cut S is the following: $d_S(u, v) = 1$ if $|S \cap \{u, v\}| = 1$ and $d_S(u, v) = 0$ otherwise.*

Definition 19.2. *Let (V, d) be a finite metric space. The metric d is a cut metric if there is a set $S \subset V$ such that $d = d_S$. d is in the cut cone (or in the cone of cut metrics) if there exist non-negative scalars $y_S, S \subset V$ such that $d(u, v) = \sum_{S \subset V} y_S d_S(u, v)$ for all $u, v \in V$.*

Definition 19.3. *Let (V, d) be a finite metric space. The metric d is a line metric if there is a mapping $f : V \rightarrow \mathbb{R}$ (the real line) such that $d(u, v) = |f(u) - f(v)|$ for all $u, v \in V$.*

Definition 19.4. Let (V, d) be a finite metric space. The metric d is an ℓ_1 metric¹ if there is some integer d and a mapping $f : V \rightarrow \mathbb{R}^d$ (Euclidean space in d dimensions) such that $d(u, v) = |f(u) - f(v)|_1$ (the ℓ_1 distance) for all $u, v \in V$.

Claim 19.3.1. A metric (V, d) is an ℓ_1 metric iff it is a non-negative combination of line metrics (in the cone of line metrics).

Proof Sketch. If d is an ℓ_1 metric then each dimension corresponds to a line metric and since the ℓ_1 metric is separable over the dimensions it is a non-negative combination of line metric. Conversely, any non-negative combination of line metrics can be made into an ℓ_1 metric where each line metric becomes a separate dimension (scalar multiplication of a line metric is also a line metric). ■

Lemma 19.2. d is an ℓ_1 metric iff d is in the cut cone.

Proof. Consider the metric d_S . It is easy to see that it is a simple line metric. Map all vertices in S to 0 and all vertices in $V - S$ to 1. If d is in the cut cone then it is a non-negative combination of the cut metrics, and hence it is a non-negative combination of line metrics, and hence an ℓ_1 metric.

To prove the converse, it suffices to argue that any line metric is in the cut cone. Let $V = \{v_1, v_2, \dots, v_n\}$ and let d be a line metric on V . Without loss of generality assume that the coordinates of the points corresponding to the line metric d are $x_1 \leq x_2 \leq \dots \leq x_n$ on the real line. For $1 \leq i < n$ let $S_i = \{v_1, v_2, \dots, v_i\}$. It is not hard to verify that $\sum_{i=1}^{n-1} |x_{i+1} - x_i| d_{S_i} = d$. ■

19.3.3 Brief introduction to metric embeddings

Let (V, d) be a finite metric space. Note that any finite metric space can be viewed as one that is derived from the shortest path metric induced on a graph with some non-negative edge lengths. If $G = (V, E)$ is a simple graph and $\ell : E \rightarrow \mathbb{R}_+$ are some edge-lengths, the metric induced on V depends both on the “topology” of G as well as the lengths. Finite metrics can encode graph structure and hence can be diverse. When trying to round we may want to work with simpler metric spaces. One way to do this is to *embed* a given metric space (V, d) into a simpler *host* metric space (V', d') . An embedding is simply a mapping of V to V' . Even though we may be interested in finite metric spaces, the host metric space can be continuous/infinite such as the Euclidean space in some dimension d . Embedding typically *distorts* the distances and thus one wants to find embeddings with small distortion. We will focus on relative notion of

¹We define ℓ_1 metric with respect to finite dimensional embeddings. Technically we can allow infinite dimensional embeddings but they are not needed for finite metrics. Moreover, it is algorithmically more useful to confine attention to finite dimensional embeddings.

distortion; additive notions are also explored in the literature although they are very restrictive due to lack of scale invariance.

Definition 19.5. *An embedding of a finite metric space (V, d) to a host metric space (V', d') is a mapping $f : V \rightarrow V'$. The embedding is an isometric embedding if $d(u, v) = d'(f(u), f(v))$ for all $u, v \in V$. An embedding is a contraction if $d'(f(u), f(v)) \leq d(u, v)$ for all $u, v \in V$. An embedding is non-contracting if $d'(f(u), f(v)) \geq d(u, v)$ for all $u, v \in V$.*

Definition 19.6. *Let (V, d) and (V', d') be two metric spaces and let $f : V \rightarrow V'$ be an embedding. The distortion of f is $\max_{u, v \in V, u \neq v} \max\{\frac{d(u, v)}{d'(f(u), f(v))}, \frac{d'(f(u), f(v))}{d(u, v)}\}$.*

Of particular importance are embeddings of finite metric spaces into Euclidean space $\mathbb{R}^d$ where the distance in the host space is measured under a norm. Examples include $\ell_1, \ell_2, \ell_\infty$. An embedding of a finite metric space (V, d) into $\mathbb{R}^d$ means that we map each v to a point $(x_1, x_2, \dots, x_d)$ and the distance between say x, y is measured as $\|x - y\|$ for some norm of interest.

The dimension d is also important in various applications but in some settings like with SPARSEST CUT the dimension is not important.

Theorem 19.7 (Bourgain). *Any n -point finite metric space can be embedded into ℓ_1 with distortion $O(\log n)$. Moreover the embedding can be chosen to be a contraction, can be constructed in randomized polynomial time, and maps points into $\mathbb{R}^d$ where $d = O(\log^2 n)$. There is also an ℓ_2 version of the theorem with the same asymptotic distortion.*

In fact one can obtain a refined theorem that is useful for SPARSEST CUT.

Theorem 19.8 (Bourgain). *Let (V, d) be n -point finite metric and let $S \subseteq V$ with $|S| = k$. Then there is a randomized polynomial time algorithm to compute an embedding $f : V \rightarrow \mathbb{R}^{O(\log^2 n)}$ such that (i) the embedding is a contraction (that is, $\|f(u) - f(v)\|_1 \leq d(u, v)$ for all $u, v \in V$ and (ii) for every $u, v \in S$, $\|f(u) - f(v)\|_1 \geq \frac{c}{\log k} d(u, v)$ for some universal constant c .*

Remark 19.4. Theorem 19.8 is stronger than what is needed for the sparsest-cut application. To obtain an $O(\log k)$ -approximation it would suffice to have a *single line embedding* $f : V \rightarrow \mathbb{R}$ that preserves, up to an $O(\log k)$ factor, the weighted sum of the k terminal-pair distances $\sum_{i=1}^k D_i d(s_i, t_i)$. We return to this point in Exercise 19.10.

19.3.4 Utilizing the ℓ_1 embedding

We saw that the integrality gap of the LP is 1 on trees since the shortest path metric on trees is in the cut cone (equivalently ℓ_1 -embeddable). More generally

one can prove that if the shortest path metric on a graph G embeds into ℓ_1 with distortion α then the integrality gap of the LP for SPARSEST CUT is at most α . This will imply an $O(\log n)$ -integrality gap via Bourgain's theorem since any n point finite metric embeds in to ℓ_1 with distortion $O(\log n)$.

Theorem 19.9. *Let $G = (V, E)$ be a graph. Suppose any finite metric induced by edge lengths on E can be embedded into ℓ_1 with distortion α . Then the integrality gap of the LP for SPARSEST CUT is at most α for any instance on G .*

Proof. Let (x, y) be a feasible fractional solution and let d be the metric induced by edge lengths given by x . Let λ be the value of the solution and recall that $\lambda = \frac{\sum_{uv \in E} c(uv)d(uv)}{\sum_{i=1}^k D_i d(s_i, t_i)}$.

Since d can be embedded into ℓ_1 with distortion at most α and any ℓ_1 metric is in the cut-cone, it implies that there are scalars $z_S, S \subset V$ such that for all u, v

$$\frac{1}{\alpha} \sum_{S \subset V} y_S d_S(u, v) \leq d(u, v) \leq \sum_{S \subset V} y_S d_S(u, v).$$

Here we assumed without loss of generality that the embedding is a contraction. For a set $S \subset V$ we use $\text{Dem}(\delta(S)) = \sum_{i: |S \cap \{s_i, t_i\}|=1} D_i$ to denote the total demand crossing the cut S .

$$\begin{aligned} \lambda &= \frac{\sum_{uv \in E} c(uv)d(uv)}{\sum_{i=1}^k D_i d(s_i, t_i)} \\ &\geq \frac{1}{\alpha} \frac{\sum_{uv \in E} c(uv) \sum_{S \subset V} z_S d_S(uv)}{\sum_{i=1}^k D_i \sum_{S \subset V} d_S(s_i, t_i)} \quad (\text{using embedding of } d \text{ with distortion } \alpha) \\ &= \frac{1}{\alpha} \frac{\sum_{S \subset V} z_S c(\delta(S))}{\sum_{S \subset V} z_S \text{Dem}(\delta(S))} \\ &\geq \frac{1}{\alpha} \min_{S \subset V} \frac{c(\delta(S))}{\text{Dem}(\delta(S))}. \end{aligned}$$

Thus there is a cut whose sparsity is at most $\alpha \cdot \lambda$. ■

Polynomial-time algorithm: How do we find a sparse cut? The preceding proof used the embedding of d into the cut-cone. The proof shows that one of the cuts with $z_S > 0$ has sparsity at most $\alpha \cdot \lambda$. Recall the proof that a metric is in the cut-cone iff it is ℓ_1 -embeddable. That argument shows the following. Suppose we have an ℓ_1 embedding into d -dimensions. Each dimension corresponds to a line-embedding. Each line embedding is in the cut-cone with only $n - 1$ cuts used to express it. Thus, given an ℓ_1 embedding into d dimensions with distortion α we only need to try $d(n - 1)$ cuts and one of them will be guaranteed to have sparsity at most $\alpha \cdot \lambda$.

Via Theorem 19.8 we can obtain an $O(\log k)$ randomized approximation and the algorithm is described below.

SparseCutviaEmbedding

1. Solve LP relaxation to obtain (x, y) and metric d_x on V
2. Use Theorem 19.8 obtain map $f : V \rightarrow \mathbb{R}^d$ where $d = O(\log^2 n)$
3. For $i = 1$ to d do
 - A. Let $v_{j_1}, v_{j_2}, \dots, v_{j_n}$ be the sorting of V according to dimension i
 - B. For $h = 1$ to $n - 1$ let $S_{i,h} = \{v_{j_1}, v_{j_2}, \dots, v_{j_h}\}$
4. Among all cuts $S_{i,h}$ with $1 \leq i \leq d$ and $1 \leq h \leq n - 1$ output the one with the smallest sparsity.

Exercise 19.6. Show that the described algorithm is a randomized $O(\log k)$ -approximation algorithm for SPARSEST CUT. You may either use the refined guarantee in Theorem 19.8, or alternatively use the weighted line-embedding statement from Exercise 19.10 together with the proof outline in Theorem 19.9.

19.3.5 Line embeddings, ℓ_1 embeddings and tree embeddings

Bourgain's theorem shows that any finite metric space on n points embeds into ℓ_1 with distortion $O(\log n)$. This can also be derived via probabilistic tree embeddings because every tree metric embeds into ℓ_1 isometrically (that is, without any distortion or with distortion 1).

Exercise 19.7. Prove formally that if a finite metric (V, d) can be probabilistically approximated via dominating tree metrics with distortion α then it can be embedded into ℓ_1 with distortion α . Is the resulting map an expansion or a contraction?

Remark 19.5. One can also obtain the $O(\log k)$ bound for SPARSEST CUT via tree embeddings, not only via Bourgain's ℓ_1 embedding. Gupta, Nagarajan, and Ravi [133] prove a strengthened FRT-style tree embedding theorem tailored to a designated terminal set $W \subseteq V$: the sampled tree metric is non-contracting on pairs in W , while the expected stretch on all pairs is $O(\log |W|)$. If we take W to be the set of endpoints that appear in the demand pairs, then $|W| \leq 2k$, and combining this with the fact that the flow-cut gap on trees is 1 yields an alternative $O(\log k)$ bound for SPARSEST CUT.

For general metrics, tree embeddings provide a more constrained space while yielding the same worst-case distortion. However, one can ask if ℓ_1 embeddings provide better distortion for concrete graph classes. Indeed this is the case. For instance consider a ring network (a cycle with capacities). One can prove that tree embeddings require a distortion 2. However the ring metric can be isometrically embedded into ℓ_1 . Thus, the flow-cut gap on ring networks is 1 which is not obvious.

Exercise 19.8. Prove that the metric on a ring network embeds into ℓ_1 isometrically.

Planar graphs are an important class of graphs for a number of reasons, both from a theoretical and applications point of view. They also pave the way to the class of proper minor closed families of graphs. There is a famous conjecture that the flow-cut gap in planar graphs is $O(1)$ [134]. Interestingly, for tree embeddings there is a lower bound of $\Omega(\log n)$ even on the special case of planar graphs called series parallel graphs. Thus, tree embeddings are not powerful enough to prove the conjecture. Rao proved that the flow-cut gap is $O(\sqrt{\log n})$ via ℓ_1 embeddings, thus separating the general graph case from planar graph case. For series-parallel graphs we know that the flow-cut gap is a tight bound of 2 and establishing this tight bound took a fair amount of work. For UNIFORM SPARSEST CUT, [178] showed that the flow-cut gap in planar graphs is $O(1)$. One can show a tight connection between embeddability into ℓ_1 and flow-cut gap [134].

We saw that ℓ_1 embeddings are a non-negative combination of line embeddings. A particular type of line-embedding is called a Frechet embedding. Let (V, d) be a metric space and let $S \subseteq V$. Then we can define a mapping $f : V \rightarrow \mathbb{R}$ to the real line where $f(v) = d(S, v)$, that is the distance of v from S . Note that all vertices in S get mapped to 0. And the mapping is a contraction. Many results in embeddings into ℓ_p spaces are based on using Frechet embeddings in various clever and often highly non-trivial ways. Bourgain's embedding is, in particular, based on picking many random sets and combining the resulting Frechet embeddings. Line embeddings are simple and have substantial power on their own. Rabinovich [219] defined the following notion of *average* distortion for line embeddings.

Definition 19.10. Let (V, d) be a metric space and let $w : V \times V \rightarrow \mathbb{R}_+$ be a non-negative symmetric weight/demand function on the vertices. We say that a line embedding $f : V \rightarrow \mathbb{R}$ has average weighted distortion α for some $\alpha \geq 1$ if

$$\sum_{uv} w(uv) |f(u) - f(v)| \geq \frac{1}{\alpha} \sum_{uv} w(uv) d(uv).$$

Note that a contraction implies $\sum_{uv} w(uv)|f(u) - f(v)| \leq \sum_{uv} w(uv)d(uv)$ so we are mainly interested in lower bounding the average distances.

Bourgain's result in fact implies the following.

Theorem 19.11. *For any n point metric space (V, d) and any weight function $w : V \times V \rightarrow \mathbb{R}_+$ there is a line embedding with average weighted distortion $O(\log n)$.*

Remark 19.6. In the sparsest-cut application the weight function is supported only on the demand pairs. Thus if there are only k demand pairs, one expects a bound depending on k rather than on n . Indeed, by adapting Bourgain's proof one can obtain a line embedding whose weighted distortion for these k prescribed pairs is $O(\log k)$; see Exercise 19.10.

One can use line embeddings with average distortion to prove flow-cut gap. You should think of the weight $w(uv)$ as the demand $D(uv)$ for pair uv .

Exercise 19.9. Prove that the flow-cut gap for SPARSEST CUT is $O(\log n)$ using Theorem 19.11.

For product multicommodity flow the demand $D(uv)$ is of the form $\pi(u)\pi(v)$. Rabinovich [219] showed that existing results such as [178] can be interpreted as providing line embeddings with low average distortion for these demands/weights. Line embeddings turn out to be very relevant when considering node capacities and directed graphs (among other generalizations) where ℓ_1 embedding do not quite work. They are also relevant in spectral methods and SDP methods. We refer the reader to [104] for the relevance of line embeddings in node-capacitated settings and their utility beyond node-capacities [71].

19.4 A Proof Sketch of Bourgain's Theorem

We now sketch Bourgain's $O(\log n)$ -distortion embedding theorem in the ℓ_1 form that is most useful for SPARSEST CUT. The proof is based on *Fréchet embeddings*: if $S \subseteq V$, define

$$f_S(v) = d(v, S) := \min_{x \in S} d(v, x).$$

This is a line embedding $f_S : V \rightarrow \mathbb{R}$.

Lemma 19.3. *For every $S \subseteq V$, the map f_S is a contraction:*

$$|f_S(u) - f_S(v)| \leq d(u, v) \quad \text{for all } u, v \in V.$$

Proof. Let $x \in S$ be such that $f_S(v) = d(v, x)$. Then

$$f_S(u) \leq d(u, x) \leq d(u, v) + d(v, x) = d(u, v) + f_S(v).$$

Reversing the roles of u and v gives $f_S(v) \leq d(u, v) + f_S(u)$, and hence $|f_S(u) - f_S(v)| \leq d(u, v)$. ■

The proof of Bourgain's theorem constructs many such contractions at different *scales*. Let $h = \lceil \log_2 n \rceil$. For each $t = 1, \dots, h+1$ we choose a random subset $S_t \subseteq V$ by including each vertex independently with probability 2^{-t} . Then we define

$$F(v) = \frac{1}{h} (f_{S_1}(v), f_{S_2}(v), \dots, f_{S_{h+1}}(v)) \in \mathbb{R}^{h+1}.$$

Since each coordinate is a contraction, F is also a contraction in ℓ_1 :

$$\|F(u) - F(v)\|_1 \leq d(u, v).$$

The nontrivial part is to show that every pair u, v is separated by a constant fraction of its distance in expectation.

Choosing the relevant scales. Fix a pair $u, v \in V$, and let $L = d(u, v)$. For $t \geq 0$, define ρ_t to be the smallest radius r such that both balls $B(u, r)$ and $B(v, r)$ contain at least 2^t points; let $\rho_0 = 0$. Let ℓ be the largest index with $\rho_\ell < L/2$. Thus the interval $[0, L/2]$ is partitioned into the increments

$$\rho_1 - \rho_0, \rho_2 - \rho_1, \dots, \rho_\ell - \rho_{\ell-1}, L/2 - \rho_\ell.$$

How a single scale contributes. The random set S_t is tailored to detect the t -th increment. For $1 \leq t \leq \ell$, one of the two open balls $B^\circ(u, \rho_t)$ and $B^\circ(v, \rho_t)$ must contain fewer than 2^t points; assume without loss of generality that $A = B^\circ(u, \rho_t)$ does. Let $B = B(v, \rho_{t-1})$. By the choice of ℓ , the sets A and B are disjoint, $|A| < 2^t$, and $|B| \geq 2^{t-1}$. If $S_t \cap A = \emptyset$ but $S_t \cap B \neq \emptyset$, then every point of S_t is at distance at least ρ_t from u , while some point of S_t is within distance ρ_{t-1} of v . Hence

$$f_{S_t}(u) - f_{S_t}(v) \geq \rho_t - \rho_{t-1}.$$

The figure below is only schematic: the sets A and B are metric balls defined through the distance function d , and the proof uses only that they are disjoint and that points in A are closer to u than points in B are to v .

Because each vertex is included with probability 2^{-t} , the event $S_t \cap A = \emptyset$ has probability at least $(1 - 2^{-t})^{|A|} \geq 1/2$, while the event $S_t \cap B \neq \emptyset$ has probability at least $1 - (1 - 2^{-t})^{2^{t-1}} \geq 1 - e^{-1/2}$. Since A and B are disjoint, these events are independent, and therefore

$$\mathbf{P}[S_t \cap A = \emptyset \text{ and } S_t \cap B \neq \emptyset] \geq c_0$$

for some universal constant $c_0 > 0$. It follows that

$$\mathbb{E}[|f_{S_t}(u) - f_{S_t}(v)|] \geq c_0(\rho_t - \rho_{t-1}).$$

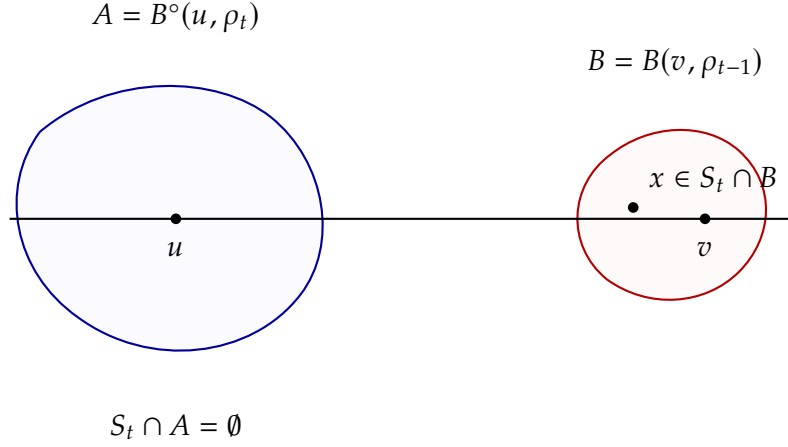


Figure 19.1: A schematic view of the single-scale argument in Bourgain's proof. The picture is not to scale. If the random set misses A but hits B , then the distance-to-set map separates u and v by at least $\rho_t - \rho_{t-1}$.

The final scale. The same argument at the last scale $t = \ell + 1$ yields

$$\mathbb{E}[|f_{S_{\ell+1}}(u) - f_{S_{\ell+1}}(v)|] \geq c_0(L/2 - \rho_\ell).$$

Summing the scales. Summing over t and telescoping gives

$$\mathbb{E}[\|F(u) - F(v)\|_1] = \frac{1}{h} \sum_{t=1}^{\ell+1} \mathbb{E}[|f_{S_t}(u) - f_{S_t}(v)|] \geq \frac{c_0}{h} \left(\sum_{t=1}^{\ell} (\rho_t - \rho_{t-1}) + L/2 - \rho_\ell \right) = \Omega\left(\frac{L}{\log n}\right).$$

Thus for every fixed pair u, v ,

$$\Omega\left(\frac{d(u, v)}{\log n}\right) \leq \mathbb{E}[\|F(u) - F(v)\|_1] \leq d(u, v).$$

Making the guarantee hold for all pairs. To obtain one embedding that works simultaneously for all pairs, we repeat the whole construction independently $\Theta(\log n)$ times and concatenate the resulting vectors. By a Chernoff bound and a union bound over all $\binom{n}{2}$ pairs, with high probability we obtain a single contraction $F : V \rightarrow \mathbb{R}^{O(\log^2 n)}$ satisfying

$$\|F(u) - F(v)\|_1 \geq \Omega\left(\frac{d(u, v)}{\log n}\right) \quad \text{for all } u, v \in V.$$

This proves Theorem 19.7.

The line-embedding corollary. The preceding proof also yields Theorem 19.11. The key point is that the expectation lower bound was obtained by averaging over individual Fréchet embeddings to the line. More precisely, let $\mathcal{F}$ be the distribution on line embeddings obtained by first choosing a scale $t \in \{1, \dots, h+1\}$ uniformly at random and then sampling S_t as above; the sampled embedding is $f(v) = d(v, S_t)$. The pairwise analysis above shows that for every $u, v \in V$,

$$\mathbb{E}_{f \sim \mathcal{F}}[|f(u) - f(v)|] \geq \Omega\left(\frac{d(u, v)}{\log n}\right),$$

and every such f is a contraction.

Proof of Theorem 19.11. Let $W = \sum_{uv} w(uv)d(u, v)$. Taking expectation over $f \sim \mathcal{F}$ and using linearity of expectation,

$$\mathbb{E}_{f \sim \mathcal{F}} \left[\sum_{uv} w(uv) |f(u) - f(v)| \right] = \sum_{uv} w(uv) \mathbb{E}_{f \sim \mathcal{F}}[|f(u) - f(v)|] \geq \Omega\left(\frac{W}{\log n}\right).$$

Therefore there exists some realization of the random set S_t for which the associated line embedding $f(v) = d(v, S_t)$ satisfies

$$\sum_{uv} w(uv) |f(u) - f(v)| \geq \Omega\left(\frac{1}{\log n}\right) \sum_{uv} w(uv) d(u, v).$$

Since f is a contraction, this is exactly a line embedding with average weighted distortion $O(\log n)$. ■

Exercise 19.10. Let (V, d) be a finite metric space and let $(s_1, t_1), \dots, (s_k, t_k)$ be k prescribed pairs with nonnegative weights $\eta_1, \dots, \eta_k$. Show, by adapting Bourgain's proof, that there is a line embedding $f : V \rightarrow \mathbb{R}$ such that f is a contraction and

$$\sum_{i=1}^k \eta_i |f(s_i) - f(t_i)| \geq \Omega\left(\frac{1}{\log k}\right) \sum_{i=1}^k \eta_i d(s_i, t_i).$$

19.5 SDP Relaxation

Can we obtain a better approximation than $O(\log k)$ for sparsest cut? Using semi-definite programming based relaxation, Arora, Rao and Vazirani [21] obtained an $O(\sqrt{\log n})$ -approximation for UNIFORM SPARSEST CUT (and more generally product instances). Building on this work, Arora, Lee and Naor [18, 19] obtained an $O(\sqrt{\log n} \log \log n)$ -approximation for SPARSEST CUT. Currently these are

the best approximation algorithms known for these problems. There was a conjecture that the SDP based relaxation would yield an $O(1)$ -approximation but it was shown that the integrality gap is essentially close to $\Omega(\sqrt{\log n})$; this required substantial mathematical work.

19.6 Spectral Relaxation for Conductance

As we said expander graphs are very important. However, certifying expansion is co-NP hard. In several applications it is important to obtain constant-degree expanders with constant expansion. The approximation algorithms we saw are not useful in this regime. It turns out that there is a very different method based on spectral graph theory that helps in this regime. For an undirected graph on n vertices one can associate a symmetric matrix called the Laplacian $\mathcal{L}_G$. The matrix has the following entries. The diagonal entry for i is $\deg(v_i)$. We have $\mathcal{L}_G[i, j] = \mathcal{L}_G[j, i] = -1$. Since this matrix is symmetric all its eigen values are real. More over, this matrix is also positive semidefinite (PSD) and hence all its eigen values are non-negative. Let $0 = \lambda_1 \leq \lambda_2 \leq \dots \lambda_n$ be its eigen values (it is not hard to verify that 0 is an eigen value with eigen vector being the all 1's vector). A well-known and famous result in spectral graph theory is the following Cheeger's inequality on the conductance of G .

$$\frac{\lambda_2}{2} \leq \phi(G) \leq \sqrt{2\lambda_2}$$

Thus λ_2 provides a constant factor approximation for the conductance when the conductance is a constant! Note that expansion and conductance are related by the maximum degree and hence when the degree is small and constant one can use λ_2 to certify expansion. Due to its importance for certifying expansion/conductance, some use λ_2 as the definition of expansion since it is computable and also helps in construction of expanders.

19.7 Node capacities and Directed Graphs

We typically discuss s - t cuts and other problems such as SPARSEST CUT cut with edge capacities. One can define cut problems with node capacities; for instance Menger's theorem on node disjoint paths is an example. In the s - t case it is typical to reduce the node-capacitated problems to edge capacitated case when the graph is *directed*. However this reduction does not apply when the graph is *undirected*. When dealing with more complicated problems involving multicommodity flows and cuts, the directed graph problems often become

harder than their undirected graph counterparts. And thus one has to consider node-capacitated undirected graph problems separately.

In the context of UNIFORM SPARSEST CUT, node capacitated cuts are quite important since they are closely related to the notion of *treewidth* of a graph. It is a graph parameter and tree decompositions associated with treewidth play a fundamental role in graph structure theory and many algorithmic applications. Treewidth is NP-hard to compute but an α approximation for UNIFORM SPARSEST CUT implies an $O(\alpha)$ -approximation for treewidth. We think of node separator as a set of vertices S such that $G - S$ has (at least) two non-trivial components. Sometimes one uses the terminology (A, S, B) for disjoint sets A, B, S with S as the separation if there is no edge between A and B (all paths between A and B go via S). The cost/weight of the cut/separation is now the cost/weight of the separator set S . When considering sparse cuts and the like there is a bit of an added complication because S itself is a set of vertices which may have demands associated with them. Handling this requires some care and proper definitions. The existing algorithms for UNIFORM SPARSEST CUT can be generalized with additional technical work (one needs to work with line embeddings rather than ℓ_1 embeddings) to handle node capacities. This leads to an $O(\sqrt{\log n})$ -approximation for UNIFORM SPARSEST CUT and a corresponding algorithm for treewidth. See [104].

SPARSEST CUT in directed graphs is more difficult. The natural definition, motivated by obstructions to routing, is the following. We are given a directed edge-capacitated graph $G = (V, E)$ and a set of k ordered demand pairs $(s_1, t_1), \dots, (s_k, t_k)$ each with a non-negative demand D_i . We think of a cut as a set of edges $E' \subseteq E$. Removing E' disconnects some pairs and the sparsity is simply $c(E')/\text{dem}(E')$ where $\text{dem}(E')$ is the demand separated by E' . Unlike undirected graphs minimal E' cuts do not induce bipartitions of the vertex set. It turns out that the flow-cut gap (for Multicut also) can be as large as k [228], and also $\Omega(n^{11/27})$ [87]. Moreover, [87] prove hardness of approximation close to polynomial factors. However, there is an important special case of *symmetric* demands that is quite relevant in several problems. Here the demands are *unordered*, that is, each demand pair is $s_i t_i$ and we say that a pair is separated by an edge set E' if they are not in the same strongly connected component. Note that this means that we are asking that at least one of the pairs (s_i, t_i) or (t_i, s_i) to be separated. It turns out that Multicut and SPARSEST CUT and UNIFORM SPARSEST CUT in this formulation have poly-logarithmic approximations and flow-cut gaps. These problems have also connected to the problem of removing edges/nodes to delete all cycles (called Feedback problems). We refer the reader to [99] on flow-cut gaps and to [6, 66] for SDP based algorithms.

Finally, since it is connected to the work of the instructor, there is a notion of polymatroid networks which connect network flows to submodular functions.

This comes from classical work in combinatorial optimization and one can generalize the notions of multicommodity flows and cuts. Several ideas from regular flows and cuts can be ported over with useful applications and consequences. See [71] and some recent work on submodular hypergraphs [82].

Bibliographic Notes

The highly influential paper of Leighton and Rao [189] obtained an $O(\log n)$ -approximation and flow-cut gap for UNIFORM SPARSEST CUT and introduced the region growing argument as well as the lower bound via expanders (an important influence is the paper of Sharokhi and Matula [232]). [189] demonstrated many applications of the divide and conquer approach. There is a large literature on SPARSEST CUT and related problems and we only touched upon a small part. An outstanding open problem is whether the flow-cut gap for NON-UNIFORM SPARSEST CUT in planar graphs is $O(1)$ (this is called the GNRS conjecture [134] in the more general context of minor-free graphs); Rao, building on ideas from [177], showed that the gap is $O(\sqrt{\log n})$ [224]. No super-constant lower bound is known for planar graphs. The theory of metric embeddings has been a fruitful bridge between TCS and mathematics and there are several surveys and connections from both perspectives. The argument via MULTICUT is attributed to Nabil Kahale — see the chapter by Shmoys on approximation algorithms for cut problems [235].

Chapter 20

Semidefinite Programming and Approximation

Semidefinite programming (SDP) is a powerful tool in the design of approximation algorithms. Many of the approximation algorithms we have seen were based on LP relaxations. LP is a special case of convex programming. Since convex programs can also be solved efficiently (modulo some technicalities with precision), it is natural to wonder whether one can use convex programs in approximation algorithms. It is not straightforward to use convexity generically since we also need some clean way to model discrete optimization problems via an appropriate convex program and then be able to round the resulting fractional solution. SDP is a special case of convex programming and they turn out to have a good sweet spot. Nevertheless, the first breakthrough via SDP in approximation was only in 1994 via the celebrated 0.878-approximation for MAX-CUT by Goemans and Williamson [122]. This was subsequently generalized to yield an optimum approximation for all Max-CSP problems [222]. Other well-known approximation results based on SDP include algorithms for graph coloring due to Karger, Motwani and Sudan [167] (see [33] for a recent paper) and the $O(\sqrt{\log n})$ -approximation for uniform sparsest cut due to Arora, Rao and Vazirani [22]. The power of SDP in classical combinatorial optimization was pointed out by Lovász in his famous work on the weak perfect graph theorem [195] where he introduced the Lovász theta function of a graph.

This chapter describes some necessary background for SDP-based approximation. We begin with background on convex programming and positive semidefinite matrices, establish the equivalence between SDPs and vector programs, and then present several applications: the GW Max-Cut algorithm, Max-CSPs, graph coloring, metric embeddings, balanced graph partitioning, and finally the connection between SDPs and sum-of-squares certificates for

polynomial optimization.

20.1 Convex Programming and Semidefinite Matrices

Convex Programs: A *convex program* has the form

$$\min f(x), \quad x \in S \subseteq \mathbb{R}^n,$$

where $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is a convex function and S is a convex set. Two ingredients suffice for efficient optimization:

- a first-order oracle for f : given x , output $f(x)$ and a subgradient $g \in \partial f(x)$; and
- a *separation oracle* for S : given x , either certify $x \in S$ or output a hyperplane separating x from S .

When both oracles run in polynomial time, the ellipsoid method solves the convex program to within additive error ε in time polynomial in the input size and $\log(1/\varepsilon)$. Linear programs are the canonical example. Beyond LPs, the most important special case for combinatorial optimization is semidefinite programming.

Quadratic objectives and matrices: A quadratic function on n variables is of the following form:

$$f(x) = \sum_{i=1}^n a_i x_i^2 + \sum_{1 \leq i < j \leq n} a_{\{i,j\}} x_i x_j + \sum_{i=1}^n b_i x_i + c$$

In order to write it in matrix form we rely on the standard quadratic form $x^T A x$ where x is a n -dimensional vector and A is a $n \times n$ matrix. We note that

$$x^T A x = \sum_{i=1}^n \sum_{j=1}^n A_{i,j} x_i x_j = \sum_{i=1}^n A_{i,i} x_i^2 + \sum_{1 \leq i < j \leq n} (A_{i,j} + A_{j,i}) x_i x_j$$

Thus, if we have a quadratic function $f(x)$ we can rewrite it as

$$f(x) = x^T A x + b^T x + c$$

where A is a *symmetric* matrix such that $A_{i,i} = a_i$ and $A_{i,j} = A_{j,i} = \frac{1}{2}a_{\{i,j\}}$. Thus symmetric matrices naturally arise in considering quadratic functions. Which quadratic functions are convex? Note that the convexity depends only on the term $x^T A x$ and hence it is the same as asking the following: when is the function $x^T A x$ convex for a symmetric matrix A ? It turns out that the answer is when A is positive semidefinite (PSD) matrices. We will define them formally and then revisit convexity aspects.

Eigenvalues and spectral theorem for symmetric matrices: Recall that for a $n \times n$ real-valued matrix A , a vector x is an eigenvector if $Ax = \lambda x$ for some real number λ . We say that λ is an eigenvalue. Recall that the eigenvalues of a matrix are the roots of the univariate polynomial $\det(A - \lambda I) = 0$. A matrix A may not have (real) eigenvalues. However, symmetric matrices have all real eigenvalues. The spectral theorem is a very important result in linear algebra with many applications in applied mathematics.

Theorem 20.1 (Spectral Theorem for Real Symmetric Matrices). *Let A be a $n \times n$ real symmetric matrix. Then*

- *All eigenvalues of A are real.*
- *There is an orthonormal basis of $\mathbb{R}^n$ consisting of eigenvectors of A*
- *$A = VDV^T$ where D is a diagonal matrix with eigenvalues of A and V is an orthogonal matrix consisting of eigenvectors of A .*

Positive Semidefinite (PSD) Matrices: A real symmetric $n \times n$ matrix A is *positive semidefinite* (psd), written $A \geq 0$, if any of the following equivalent conditions holds:

1. $x^T Ax \geq 0$ for all $x \in \mathbb{R}^n$;
2. all eigenvalues of A are real and non-negative;
3. $A = W^T W$ for some real matrix W (Cholesky decomposition).

If the inequality in (1) is strict for all $x \neq 0$, we say A is *positive definite* and write $A > 0$.

Theorem 20.2. *For a real symmetric $n \times n$ matrix A , conditions (1), (2), and (3) are equivalent.*

Proof. We show $(1) \Rightarrow (2) \Rightarrow (3) \Rightarrow (1)$.

$(1) \Rightarrow (2)$: Since A is real and symmetric, by the spectral theorem all eigenvalues of A are real. Let λ be any eigenvalue with eigenvector $v \neq 0$, so $Av = \lambda v$. Then $v^T Av = \lambda \|v\|^2$. By (1), $v^T Av \geq 0$, and since $\|v\|^2 > 0$ we conclude $\lambda \geq 0$.

$(2) \Rightarrow (3)$: By the spectral theorem, $A = UDU^T$ where U is orthogonal and $D = \text{diag}(\lambda_1, \dots, \lambda_n)$ with each $\lambda_i \geq 0$. Let $D^{1/2} = \text{diag}(\sqrt{\lambda_1}, \dots, \sqrt{\lambda_n})$. Set $W = D^{1/2}U^T$. Then

$$W^T W = UD^{1/2}D^{1/2}U^T = UDU^T = A.$$

(3) $\Rightarrow$ (1): If $A = W^T W$ then for any $x \in \mathbb{R}^n$,

$$x^T A x = x^T W^T W x = \|Wx\|^2 \geq 0.$$

■

PSD Matrices and Ellipsoids: We can interpret a $n \times n$ matrix A as a linear operator from $\mathbb{R}^n$ to $\mathbb{R}^n$. If A is psd then $x^T A x \geq 0$. Letting $Ax = u$ we see that $x^T u \geq 0$ is equivalent to the vectors u and x forming an acute angle; thus A does not rotate x by more than 90 degrees. PSD matrices arise naturally when defining ellipsoids in high dimensions. An ellipsoid is defined as a linear transformation of the unit ball. Note that the unit ball is the set $\{x \in \mathbb{R}^n \mid \|x\|_2 \leq 1\}$. Thus an ellipsoid is $\{Bx + c \mid \|x\|_2 \leq 1\}$ where c is the translation of center, and B is a non-singular matrix — we ask for non-singularity so that the ellipsoid is full-dimensional, otherwise we have to work with the smaller dimension. If we ignore the translation aspect then an origin centered full-dimensional ellipsoid is simply $\{Bx \mid \|x\|_2 \leq 1\}$ for some non-singular matrix. We note that this set is the same as $\{y \mid y^T A y \leq 1\}$ where $A = (B^{-1})^T B^{-1}$; hence A is psd. Note that B may not be psd. Conversely the set $\{y \mid y^T A y \leq 1\}$ for a positive definite matrix A defines an ellipsoid (do you see why?). Thus an ellipsoid centered at c can be defined alternatively as $\{y \mid (y - c)^T Q^{-1} (y - c) \leq 1\}$ where Q is a positive definite matrix. The reason to use the inverse is because Q defines the shape of the ellipsoid: the eigenvectors of Q are the principal axes of the ellipsoid and the scaling in each dimension is given by the eigenvalues of Q .

Convexity and PSD Matrices: We discuss two connections between psd matrices and convexity. The first is about the set of all $n \times n$ psd matrices. We interpret these matrices as vectors in $\mathbb{R}^{n^2}$. Technically we can work with $\mathbb{R}^{\binom{n}{2}}$ since we are interested in symmetric matrices but it is convenient to work in the higher dimensions.

Lemma 20.1. *The set $\mathcal{S}_n^+ = \{A \in \mathbb{R}^{n \times n} : A \text{ symmetric}, A \geq 0\}$ is a convex cone.*

Proof. If $A, B \geq 0$ and $\alpha, \beta \geq 0$, then for any x , $x^T(\alpha A + \beta B)x = \alpha x^T A x + \beta x^T B x \geq 0$. ■

The cone $\mathcal{S}_n^+$ is a convex subset of $\mathbb{R}^{n^2}$ when we view matrices as vectors.

Another connection to convexity of functions is given by the following fact.

Lemma 20.2. *Let $f : D \rightarrow \mathbb{R}$ be smooth convex function over a domain $D \subseteq \mathbb{R}^n$. Then f is convex if and only if its Hessian $\nabla^2 f(x) \geq 0$ for all $x \in D$.*

Proof sketch. Convexity requires $f(y) \geq f(x) + (y - x)^T \nabla f(x)$ for all x, y . It suffices to check convexity for all points in their infinitesimal neighborhood

(one needs an argument for this but can be shown). Suppose y is in a small neighborhood of x . A second-order Taylor expansion gives $f(y) \approx f(x) + (y - x)^T \nabla f(x) + \frac{1}{2}(y - x)^T \nabla^2 f(x)(y - x)$, so nonnegativity of the Hessian quadratic form is necessary and sufficient to ensure that $f(y) \geq f(x) + (y - x)^T \nabla f(x)$. ■

Corollary 20.3. *A quadratic function $x^T A x$ is convex over $\mathbb{R}^n$ iff A is psd.*

Proof. If $f(x) = x^T A x$ and A is symmetric, then $\nabla^2 f(x) = 2A$. By the preceding lemma, f is convex iff $\nabla^2 f(x)$ is psd for all x , which is equivalent to $A \geq 0$. ■

The ellipsoid viewpoint is also suggestive. If $A \geq 0$, then the sublevel sets $\{x \mid x^T A x \leq \lambda\}$ are ellipsoids (or degenerate ellipsoids) and hence convex. If A has a negative eigenvalue with eigenvector v , then along the line $x = tv$ we have $x^T A x = t^2 v^T A v$, which is a concave quadratic in t since $v^T A v < 0$. Thus $x^T A x$ cannot be convex.

20.2 Semidefinite Programming (SDP)

For matrices $A, B \in \mathbb{R}^{n \times n}$, define the inner product $A \cdot B = \sum_{i,j} A_{ij} B_{ij} = \text{tr}(A^T B)$; in other words we are treating A and B as vectors in n^2 dimensions. An SDP in standard form maximizes a linear function of a psd matrix subject to linear constraints:

$$\begin{aligned} \max \quad & C \cdot Y \\ \text{s.t.} \quad & D_i \cdot Y = d_i, \quad i = 1, \dots, k \\ & Y \geq 0, \quad Y \in \mathbb{R}^{n \times n}. \end{aligned}$$

The objective and constraints are linear in the entries of Y ; the only nonlinearity is the psd constraint $Y \geq 0$, which restricts Y to a convex set $\mathcal{S}_n^+$. Hence an SDP is a special case of convex programming.

We can also allow a minimization objective and inequality constraints $D_i \cdot Y \geq d_i$.

Solvability of SDP via a separation oracle. We can rewrite the SDP as a linear program but with an infinite number of linear constraints.

$$\begin{aligned} \max \quad & C \cdot Y \\ \text{s.t.} \quad & D_i \cdot Y = d_i, \quad i = 1, \dots, k \\ & v^T Y v \geq 0 \quad v \in \mathbb{R}^n \\ & y_{i,j} = y_{j,i} \quad 1 \leq i < j \leq n \\ & Y \in M_n \end{aligned}$$

Here M_n is the set of all $n \times n$ matrices, indicating that the space has dimension n^2 . This reformulation allows us to see that the original SDP has an efficient

separation oracle so that we can apply the ellipsoid method. Equivalently, we wish to exhibit a separation oracle for the convex set $\mathcal{S}_n^+$. Given a matrix A , first check symmetry; if not symmetric, the violated constraint $y_{ij} = y_{ji}$ is a separating hyperplane. Then compute the eigenvalue decomposition of A : if all eigenvalues are non-negative then $A \geq 0$. Otherwise let v be an eigenvector with eigenvalue $\lambda < 0$; then $v^T A v = \lambda < 0$ and the separating hyperplane is $v^T Y v \geq 0$. This oracle runs in polynomial time, so SDPs can be solved in polynomial time to arbitrary precision.

20.2.1 SDP and Vector Programming

Why are SDPs useful in modeling discrete optimization problems? One important reason is because they can be recast as solving vector programming relaxations of quadratic optimization problems. We formalize this below.

An SDP solution $Y \geq 0$ can always be written as $Y = W^T W$ via the Cholesky decomposition. Let $v_1, v_2, \dots, v_n$ denote the columns of W ; then $Y_{ij} = v_i \cdot v_j$ with the standard inner product.

Claim 20.2.1. *The solution to any SDP with $Y \in M_n$ is a set of n vectors $v_1, v_2, \dots, v_n \in \mathbb{R}^n$ such that $Y_{ij} = v_i \cdot v_j$.*

This observation motivates the *vector programming* viewpoint: instead of optimizing over a psd matrix, we optimize directly over a collection of vectors $v_1, \dots, v_n \in \mathbb{R}^n$ with an objective and constraints that are linear in the inner products $v_i \cdot v_j$.

Example 20.1. The following is a valid vector program:

$$\max \quad (v_1 \cdot v_2 + v_2 \cdot v_3 - 5v_1 \cdot v_5) \quad (20.1)$$

$$\text{s.t.} \quad v_1 \cdot v_2 - v_2 \cdot v_3 = 5 \quad (20.2)$$

$$v_1, v_2, v_3 \in \mathbb{R}^3 \quad (20.3)$$

All constraints are linear in inner products, so this is an SDP.

Claim 20.2.2. *Every vector program is an SDP.*

Proof. We introduce a variable Y_{ij} for each dot product $v_i \cdot v_j$ and then the objective and constraints become linear in these variables. Then we add the constraint that $Y \geq 0$. ■

The vector programming perspective is useful because it is often more natural to describe combinatorial problems as programs over *vectors* than as matrix programs. We will use both formulations interchangeably.

20.3 SDP for Max-Cut: The GW Algorithm

In the MAX-CUT problem we are given an edge-weighted graph $G = (V, E)$ with weights $w : E \rightarrow \mathbb{R}_{\geq 0}$. A *cut* is a partition of V into two sets $(S, V \setminus S)$. The weight of the cut is $w(\delta(S)) = \sum_{ij \in E, i \in S, j \notin S} w_{ij}$. The goal is to find the maximum-weight cut. Another view of the problem is to find the largest weight bipartite subgraph of a given graph.

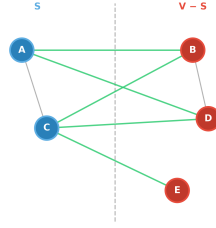


Figure 20.1: An example of a max-cut: the red and blue vertices form the two sides of the partition, and the bold edges crossing the cut are the edges contributing to the cut weight.

MAX-CUT is NP-hard; it is one of Karp's original 21 NP-complete problems. In contrast, MIN-CUT is solvable in polynomial time via max-flow. We have seen that some simple algorithms achieve a $\frac{1}{2}$ -approximation: pick S by including each vertex independently with probability $\frac{1}{2}$, or run a local search that moves one vertex at a time into or out of S to improve the cut. One can also develop several LP-based $1/2$ -approximation algorithms. Despite the simplicity of the problem it was unclear whether one can obtain a better than $1/2$ -approximation. Goemans and Williamson, in a major breakthrough, showed in 1994 that an SDP relaxation yields a substantially better approximation ratio of $\alpha \approx 0.878$.

Quadratic Integer Program and Vector Relaxation: Assume $V = \{1, 2, \dots, n\}$. Assign each vertex i a variable $y_i \in \{-1, +1\}$, where $y_i = -1$ means $i \in S$ and $y_i = +1$ means $i \in V \setminus S$. An edge (i, j) is cut if and only if $y_i \neq y_j$, which happens exactly when $y_i y_j = -1$, so its contribution to the cut is $w_{ij}(1 - y_i y_j)/2$. This gives the following quadratic integer program:

$$\begin{aligned} \max \quad & \sum_{ij \in E} w_{ij} (1 - y_i y_j) / 2 \\ \text{s.t.} \quad & y_i^2 = 1, \quad i \in V. \end{aligned}$$

This is NP-hard: solving it exactly is equivalent to MAX-CUT.

To obtain a tractable relaxation, we interpret each scalar y_i as a one-dimensional vector and then *relax* by allowing v_i to be a unit vector in $\mathbb{R}^n$. The inner product $y_i y_j$ becomes $v_i \cdot v_j$, and the constraint $y_i^2 = 1$ becomes $\|v_i\|^2 = 1$. This yields the vector program:

$$\begin{aligned} \max \quad & \sum_{ij \in E} w_{ij} (1 - v_i \cdot v_j) / 2 \\ \text{s.t.} \quad & v_i \cdot v_i = 1, \quad i \in V \\ & v_i \in \mathbb{R}^n, \quad i \in V. \end{aligned}$$

Since this is a relaxation of the original IP (the ± 1 vectors form a feasible subset), the optimum value OPT_v satisfies $\text{OPT}_v \geq \text{OPT}$. Being a vector program, it can be solved as an SDP in polynomial time.

20.3.1 Rounding: The Random Hyperplane Algorithm

Let $v_1^*, \dots, v_n^*$ be an optimal solution to the vector program with value OPT_v . Each v_i^* is a unit vector in $\mathbb{R}^n$. We produce a cut via the following randomized rounding procedure.

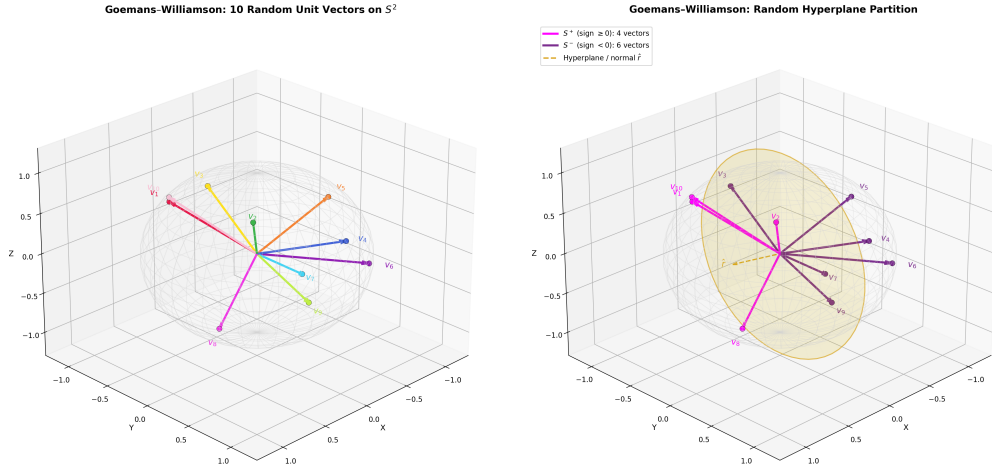


Figure 20.2: Left: the unit vectors $v_1^*, \dots, v_n^*$ (one per vertex) on the unit sphere after solving the SDP. Right: a random hyperplane through the origin divides the vectors into two groups, defining the cut $(S, V \setminus S)$.

GW-RandomHyperplane:

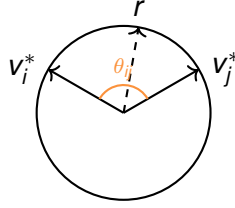
Solve the SDP to obtain optimal unit vectors $v_1^*, \dots, v_n^*$.
 Choose a random unit vector $r \in \mathbb{R}^n$ uniformly from the unit sphere.
 Set $S = \{i \in V : r \cdot v_i^* \geq 0\}$.
 Output the cut $(S, V \setminus S)$.

The intuition is that a random hyperplane through the origin (with normal r) partitions the unit sphere into two hemispheres; the cut assigns vertex i to the hemisphere containing v_i^* .

Analysis: Let θ_{ij} be the angle between the vectors v_i^* and v_j^* . Note that since the vectors are unit vectors we have $v_i^* \cdot v_j^* = \cos \theta_{ij}$. From the definition of the SDP objective:

$$\text{OPT}_v = \sum_{ij \in E} w_{ij} \frac{1}{2} (1 - \cos \theta_{ij})$$

Lemma 20.3. For any edge (i, j) , if $\theta_{ij} \in [0, \pi]$ is the angle between v_i^* and v_j^* , then the edge (i, j) is cut by the random hyperplane algorithm with probability θ_{ij}/π .



Proof. Edge (i, j) is cut if and only if the random hyperplane separates v_i^* and v_j^* . We can focus on the two-dimensional plane spanned by these two vectors. Then, by symmetry, the probability of this event equals the fraction of hyperplanes through the origin (equivalently, the fraction of unit normal vectors r) that lie in the “wedge” between the two vectors. This fraction is exactly θ_{ij}/π . ■

The contribution of edge (i, j) to the SDP objective OPT_v is

$$w_{ij} (1 - v_i^* \cdot v_j^*)/2 = w_{ij} (1 - \cos \theta_{ij})/2.$$

By linearity of expectation, the expected weight of the cut produced by the algorithm is

$$\mathbb{E}[w(\delta(S))] = \sum_{ij \in E} w_{ij} \frac{\theta_{ij}}{\pi}.$$

To bound the approximation ratio we need to compare θ_{ij}/π with $(1 - \cos \theta_{ij})/2$ for every $\theta_{ij} \in [0, \pi]$.

Lemma 20.4 (Goemans–Williamson, 1994). *For all $\theta \in [0, \pi]$,*

$$\frac{\theta}{\pi} \geq \alpha \cdot \frac{1 - \cos \theta}{2}, \quad \text{where } \alpha = \min_{\theta \in (0, \pi]} \frac{2\theta}{\pi(1 - \cos \theta)} \approx 0.87856.$$

The minimum in Lemma 20.4 is achieved at $\theta^* \approx 2.331$ radians; it can be verified numerically.

Theorem 20.4 (Goemans–Williamson, 1994). *The random hyperplane algorithm is a 0.878-approximation for MAX-CUT.*

Proof. By Lemma 20.4, for every edge (i, j) ,

$$w_{ij} \frac{\theta_{ij}}{\pi} \geq \alpha \cdot w_{ij} \frac{1 - \cos \theta_{ij}}{2}.$$

Summing over all edges and using linearity of expectation:

$$\mathbb{E}[w(\delta(S))] = \sum_{ij \in E} w_{ij} \frac{\theta_{ij}}{\pi} \geq \alpha \cdot \text{OPT}_v \geq \alpha \cdot \text{OPT}.$$

■

Generating a random unit vector: To implement the rounding algorithm one needs to generate a random unit vector in n dimensions. This is not obvious. Here is one trick to accomplish this. Consider the standard 1-d Normal distribution $N(0, 1)$ with mean 0 and standard deviation 1, having density function $\frac{1}{\sqrt{2\pi}} e^{-x^2/2}$.

1. Generate a vector $\bar{z} = (X_1, X_2, \dots, X_n)$, where each X_i is an independent random variable drawn from $N(0, 1)$.
2. The joint density is:

$$f_{\bar{z}}(x_1, \dots, x_d) = \frac{1}{(\sqrt{2\pi})^d} e^{-(x_1^2 + \dots + x_n^2)/2}.$$

3. Notice the density is the same for all points $(x_1, \dots, x_n)$ with the same length, i.e., it depends only on $\|\bar{z}\|$. Therefore the distribution is centrally symmetric, and the direction of $\bar{z}$ is uniformly distributed.
4. Output the unit vector $\bar{r} = \frac{\bar{z}}{\|\bar{z}\|}$.

How does one generate a random number from the standard normal distribution? This is not completely obvious either. There is a way to use two random numbers uniformly distributed in $(0, 1)$ to generate two random numbers distributed according to the Normal distribution via the Box-Muller transform. See https://en.wikipedia.org/wiki/Box%E2%80%93Muller_transform. There are precision issues involved in these computations but the algorithm and analysis are robust.

Integrality gap and hardness: What is the integrality gap of the SDP? Consider the cycle C_5 with unit edge weights. It is easy to see that the optimum value is 4. However the SDP can assign vectors to the five vertices such that its value is 4.5225. If $C = 1, 2, 3, 4, 5$ then consider the feasible SDP solution which maps the vertices to the corners of a regular pentagon inscribed inside a unit circle in the order 1, 4, 2, 5, 3. The angle between the vectors of adjacent vertices on the cycle is 144 degrees. Thus shows a gap of 0.8845.

It turns out that the integrality gap of the SDP is exactly α ! This is a non-trivial result [107] and requires a sophisticated construction. Moreover, under the Unique Games Conjecture (UGC) of Khot [174], no polynomial-time algorithm can approximate MAX-CUT with ratio better than α ; this is a surprising and deep result [175] that connects worst-case integrality gaps of SDPs to hardness. In other words, the GW algorithm is essentially optimal among efficient algorithms under UGC.

Remark 20.1. The algorithm outputs a good cut *in expectation*. One can repeat the rounding a polynomial number of times and keeps the best solution found, obtaining a cut of expected quality $\geq 0.878 \cdot \text{OPT}$ with high probability. A deterministic algorithm achieving the same ratio is also known but technically more involved [198].

20.4 SDP for Constraint Satisfaction Problems

The MAX-CUT SDP is the first important example of a much more general phenomenon: SDPs can capture the geometry of assignments to constraint satisfaction problems. We illustrate this first for MAX-2-SAT and then describe a generic SDP for arbitrary MAX-CSPs.

20.4.1 Warm-Up: MAX-2-SAT

In MAX-2-SAT we are given Boolean variables $x_1, x_2, \dots, x_n$ and weighted clauses, each of which is a disjunction of at most two literals. The goal is to maximize the total weight of satisfied clauses. If every clause has exactly two literals, then a uniformly random assignment satisfies each clause with probability $3/4$. The

standard definition also allows unit clauses; the SDP relaxation below extends directly to that setting, with unit clauses contributing a simpler bilinear term involving only one variable and the truth variable.

It is convenient to first write the problem as a quadratic program using sign variables. Introduce variables $y_0, y_1, y_2, \dots, y_n \in \{-1, +1\}$ where y_0 is a special variable representing “truth” and y_i represents the value of x_i . We interpret x_i as true if $y_i = y_0$ and false if $y_i = -y_0$. For a literal ℓ define

$$\tilde{y}_\ell = \begin{cases} y_i & \text{if } \ell = x_i, \\ -y_i & \text{if } \ell = \neg x_i. \end{cases}$$

Then ℓ is true iff $\tilde{y}_\ell = y_0$ and false iff $\tilde{y}_\ell = -y_0$. Hence the indicator that ℓ is false is

$$\frac{1 - \tilde{y}_\ell y_0}{2}.$$

For a unit clause $C = (\ell)$ of weight w_C , the satisfaction indicator is simply

$$\mathbf{1}[C \text{ is satisfied}] = \frac{1 + \tilde{y}_\ell y_0}{2}.$$

For a 2-clause $C = (\ell_1 \vee \ell_2)$ of weight w_C , the clause is unsatisfied iff both literals are false. Therefore, for an integral assignment,

$$\begin{aligned} \mathbf{1}[C \text{ is satisfied}] &= 1 - \frac{1 - \tilde{y}_{\ell_1} y_0}{2} \cdot \frac{1 - \tilde{y}_{\ell_2} y_0}{2} \\ &= 1 - \frac{1 - \tilde{y}_{\ell_1} y_0 - \tilde{y}_{\ell_2} y_0 + \tilde{y}_{\ell_1} \tilde{y}_{\ell_2} y_0^2}{4} \\ &= \frac{3 + \tilde{y}_{\ell_1} y_0 + \tilde{y}_{\ell_2} y_0 - \tilde{y}_{\ell_1} \tilde{y}_{\ell_2}}{4}, \end{aligned}$$

where the last step uses $y_0^2 = 1$. Let C_1 and C_2 denote the unit clauses and the 2-clauses respectively. Then MAX-2-SAT can be written as the quadratic integer program

$$\begin{aligned} \max \quad & \sum_{C=(\ell) \in C_1} w_C \frac{1 + \tilde{y}_\ell y_0}{2} + \sum_{C=(\ell_1 \vee \ell_2) \in C_2} w_C \frac{3 + \tilde{y}_{\ell_1} y_0 + \tilde{y}_{\ell_2} y_0 - \tilde{y}_{\ell_1} \tilde{y}_{\ell_2}}{4} \\ \text{s.t.} \quad & y_i^2 = 1 \quad i = 0, 1, \dots, n. \end{aligned}$$

We now relax this quadratic program in the same way as for MAX-CUT. Replace each sign variable y_i by a unit vector v_i , and replace each product $y_i y_j$ by the inner product $v_i \cdot v_j$. For a literal ℓ define

$$\tilde{v}_\ell = \begin{cases} v_i & \text{if } \ell = x_i, \\ -v_i & \text{if } \ell = \neg x_i. \end{cases}$$

This yields the vector relaxation

$$\begin{aligned} \max \quad & \sum_{C=(\ell) \in C_1} w_C \frac{1 + \tilde{v}_\ell \cdot v_0}{2} + \sum_{C=(\ell_1 \vee \ell_2) \in C_2} w_C \frac{3 + \tilde{v}_{\ell_1} \cdot v_0 + \tilde{v}_{\ell_2} \cdot v_0 - \tilde{v}_{\ell_1} \cdot \tilde{v}_{\ell_2}}{4} \\ \text{s.t.} \quad & \|v_i\|^2 = 1 \quad i = 0, 1, \dots, n. \end{aligned}$$

This is a vector program and hence an SDP. An integral assignment is recovered by taking all vectors to lie on a common line and setting $v_i = y_i v_0$ for each i ; therefore the SDP optimum upper bounds the optimum 2-SAT value.

The role of the SDP is the same as for MAX-CUT: it replaces a discrete sign choice by a geometric configuration of unit vectors. The rounding is more delicate than a plain random hyperplane because the objective depends not only on whether two vectors are separated but also on how each literal correlates with the truth vector v_0 . Standard roundings bias the hyperplane according to these correlations and yield better guarantees than the naive random assignment when clauses of size 2 are present; see [250, 255] for textbook treatments.

Hyperplane-2SAT:

Solve the SDP to obtain optimal unit vectors $v_0^*, v_1^*, \dots, v_n^*$.
 Choose a random unit vector $r \in \mathbb{R}^n$ uniformly from the unit sphere.
 For each variable i , set $x_i = \text{true}$ iff $r \cdot v_i^*$ and $r \cdot v_0^*$ have the same sign.
 Output the resulting truth assignment.

Claim 20.4.1. *Let C be a clause of weight w_C and let the partition induced by HYPERPLANE-2SAT on the relevant vectors be denoted by $(S, \bar{S})$.*

1. *If $C = (\ell)$ is a unit clause, define a graph H_C with one edge of weight w_C between the two vertices v_0^* and $u_\ell := -\tilde{v}_\ell^*$. Then*
 - (a) *the SDP contribution of C equals the SDP objective value of this edge in the MAX-CUT relaxation, namely $w_C(1 - u_\ell \cdot v_0^*)/2$;*
 - (b) *the clause C is satisfied iff this edge is cut by $(S, \bar{S})$.*
2. *If $C = (\ell_1 \vee \ell_2)$ is a 2-clause, define a triangle H_C on the three vertices $v_0^*, u_1 := -\tilde{v}_{\ell_1}^*, u_2 := -\tilde{v}_{\ell_2}^*$, where each edge has weight $w_C/2$. Then*
 - (a) *the SDP contribution of C equals the SDP objective value of this triangle in the MAX-CUT relaxation;*
 - (b) *the clause C is satisfied iff the cut induced by $(S, \bar{S})$ has weight exactly w_C in H_C .*

Proof. For a unit clause $C = (\ell)$ we have

$$w_C \frac{1 + \tilde{v}_\ell^* \cdot v_0^*}{2} = w_C \frac{1 - u_\ell \cdot v_0^*}{2},$$

which proves the SDP identity. Moreover, C is satisfied iff $\tilde{v}_\ell^*$ and v_0^* lie on the same side of the random hyperplane, which is equivalent to u_ℓ and v_0^* lying on opposite sides. Hence the edge is cut iff the clause is satisfied.

For a 2-clause $C = (\ell_1 \vee \ell_2)$, the SDP value of the triangle H_C is

$$\begin{aligned} \frac{w_C}{2} \left(\frac{1 - u_1 \cdot v_0^*}{2} + \frac{1 - u_2 \cdot v_0^*}{2} + \frac{1 - u_1 \cdot u_2}{2} \right) &= \frac{w_C}{2} \left(\frac{1 + \tilde{v}_{\ell_1}^* \cdot v_0^*}{2} + \frac{1 + \tilde{v}_{\ell_2}^* \cdot v_0^*}{2} + \frac{1 - \tilde{v}_{\ell_1}^* \cdot \tilde{v}_{\ell_2}^*}{2} \right) \\ &= w_C \frac{3 + \tilde{v}_{\ell_1}^* \cdot v_0^* + \tilde{v}_{\ell_2}^* \cdot v_0^* - \tilde{v}_{\ell_1}^* \cdot \tilde{v}_{\ell_2}^*}{4}, \end{aligned}$$

which is exactly the SDP contribution of C .

To prove the second part, note that C is unsatisfied iff both literals are false, that is, iff both $\tilde{v}_{\ell_1}^*, \tilde{v}_{\ell_2}^*$ lie on the side of the hyperplane opposite from v_0^* . Equivalently, u_1, u_2, v_0^* all lie on the same side of the hyperplane. Thus C is satisfied iff the cut of the triangle H_C is non-trivial. Any non-trivial cut of a triangle cuts exactly two edges, and since each edge has weight $w_C/2$, the cut weight is exactly w_C . ■

Theorem 20.5. *HYPERPLANE-2SAT is an α -approximation for MAX-2-SAT, where α is the Goemans–Williamson constant from Lemma 20.4.*

Proof. Consider the gadget H_C from Claim 20.4.1 for each clause C . The vectors from the MAX-2-SAT SDP give a feasible MAX-CUT SDP solution for each gadget. By the Goemans–Williamson analysis, the expected cut weight produced by the random hyperplane rounding on H_C is at least α times the SDP value of H_C . By Claim 20.4.1, this implies that the expected weight contributed by clause C under HYPERPLANE-2SAT is at least α times the SDP contribution of C .

Summing over all clauses and using linearity of expectation, the expected weight of the assignment produced by HYPERPLANE-2SAT is at least α times the SDP optimum, and hence at least α times the true optimum. ■

The preceding theorem shows that the plain random hyperplane rounding already matches the Goemans–Williamson guarantee for MAX-CUT. One can also prove the same α bound by a direct geometric analysis of unit clauses and 2-clauses. The gadget proof above is not strictly necessary, but it makes a useful conceptual connection between MAX-2-SAT and MAX-CUT.

To do better, one uses a problem-specific rounding that first modifies the vectors before the hyperplane step. Concretely, for each variable i one replaces

v_i^* by a rotated vector v_i' lying in the plane spanned by v_i^* and v_0^* , where the new angle with v_0^* is chosen as a careful function of $v_i^* \cdot v_0^*$. One then applies the random hyperplane rounding to the rotated vectors. With an appropriate choice of the rotation function this yields an approximation ratio of $\beta \approx 0.94016567$ [192]. This constant is known to be optimal under the Unique Games Conjecture [46].

20.4.2 A Generic SDP for MAX-CSPs

It is helpful to first discuss Boolean CSPs. Let $\Gamma = \{P_1, \dots, P_m\}$ be a finite family of predicates, where each predicate P has some arity $k(P)$ and maps $\{0, 1\}^{k(P)} \rightarrow \{0, 1\}$. An instance of MAX-CSP(Γ) has Boolean variables $x_1, \dots, x_n$ and weighted constraints C . Each constraint chooses a predicate $P_C \in \Gamma$ and a tuple of variables $S_C = (i_1, \dots, i_t)$ of the appropriate length, and contributes

$$w_C \cdot P_C(x_{i_1}, \dots, x_{i_t})$$

to the objective. The goal is to maximize the total weight of satisfied constraints.

The family Γ specifies which *types* of local constraints are allowed. Several familiar problems fit this template.

- **MAX-CUT:** use the single binary predicate $\text{XOR}(x, y)$ (equivalently, $\text{NEQ}(x, y)$). If $x_i \in \{0, 1\}$ indicates on which side of the cut vertex i lies, then an edge (i, j) is cut iff $x_i \oplus x_j = 1$.
- **MAX-2-SAT:** if we focus on 2-clauses, then Γ consists of the four binary predicates corresponding to $(x \vee y)$, $(x \vee \neg y)$, $(\neg x \vee y)$, and $(\neg x \vee \neg y)$. If one also allows unit clauses, then one adds the two unary predicates x and $\neg x$.
- **MAX-3-SAT:** here the arity is 3, and the family consists of the eight predicates obtained from $(\ell_1 \vee \ell_2 \vee \ell_3)$ by independently negating each literal.

For specific Boolean problems such as MAX-CUT or MAX-2-SAT, it is often useful to write a quadratic program first and then relax the products $y_i y_j$ to vector inner products, as we did earlier. However, that viewpoint is tied to the special algebra of a particular predicate family. For a *generic* CSP, especially when the family contains predicates of different arities, the cleaner starting point is a local-distribution relaxation.

There is nevertheless a direct algebraic formulation. For each variable x_i and value $a \in \{0, 1\}$, introduce an indicator variable $x_{i,a} \in \{0, 1\}$ with $x_{i,0} + x_{i,1} = 1$.

Then a constraint C on scope $S_C = (i_1, \dots, i_t)$ contributes

$$\sum_{\alpha \in \{0,1\}^{S_C}} P_C(\alpha) \prod_{r=1}^t x_{i_r, \alpha(i_r)}$$

to the objective. Thus a generic Boolean CSP can be viewed as a *polynomial optimization problem* whose degree is the arity of the constraints. The local-distribution LP can be interpreted as a linearization of this polynomial program: the variables $\mu_{i,a}$ stand for the degree-1 indicators $x_{i,a}$, while the variables $\lambda_{C,\alpha}$ stand for the monomials $\prod_{r=1}^t x_{i_r, \alpha(i_r)}$. The consistency constraints are exactly the identities these monomials satisfy under an integral assignment. The SDP then uses the same local variables, but replaces nonnegative scalars by vectors and enforces the same local consistency through inner-product constraints; in this sense it is a lifted, geometric strengthening of the LP relaxation.

For MAX-CUT, this polynomial viewpoint is very concrete. For each vertex i we have indicator variables $x_{i,0}, x_{i,1}$ with $x_{i,0} + x_{i,1} = 1$. An edge ij is cut iff either $x_{i,0} = 1, x_{j,1} = 1$ or $x_{i,1} = 1, x_{j,0} = 1$. Hence the cut value is

$$\sum_{ij \in E} w_{ij} (x_{i,0}x_{j,1} + x_{i,1}x_{j,0}).$$

This is a quadratic program because the predicate XOR has arity 2. For a ternary predicate, such as in MAX-3-SAT, the analogous expression would involve degree-3 monomials. Thus MAX-CUT and MAX-2-SAT are special cases where the generic polynomial program happens to be quadratic.

The key point is that each constraint depends on only a *constant* number of variables. Thus, rather than trying to optimize over a full distribution on all 2^n assignments, we keep track only of local distributions. This gives a natural LP relaxation.

Local-Distribution LP: For each variable x_i and value $a \in \{0,1\}$ introduce a variable $\mu_{i,a}$, intended to represent the marginal probability that $x_i = a$. For each constraint C and each local assignment $\alpha \in \{0,1\}^{S_C}$ introduce a variable $\lambda_{C,\alpha}$, intended to represent the local probability that the variables in S_C take the

values prescribed by α . The LP is:

$$\begin{aligned}
 \max \quad & \sum_C w_C \sum_{\alpha \in \{0,1\}^{S_C}} P_C(\alpha) \lambda_{C,\alpha} \\
 \text{s.t.} \quad & \mu_{i,0} + \mu_{i,1} = 1 \quad i = 1, \dots, n, \\
 & \sum_{\alpha \in \{0,1\}^{S_C}} \lambda_{C,\alpha} = 1 \quad \text{for every constraint } C, \\
 & \sum_{\alpha \in \{0,1\}^{S_C} : \alpha(i)=a} \lambda_{C,\alpha} = \mu_{i,a} \quad \text{for every } C, i \in S_C, a \in \{0,1\}, \\
 & \mu_{i,a} \geq 0 \quad i = 1, \dots, n, a \in \{0,1\}, \\
 & \lambda_{C,\alpha} \geq 0 \quad \text{for every } C, \alpha \in \{0,1\}^{S_C}.
 \end{aligned}$$

The constraints say exactly what they should:

- $\mu_{i,0}, \mu_{i,1}$ form a distribution on the value of x_i ;
- for each constraint C , the variables $\{\lambda_{C,\alpha}\}_\alpha$ form a distribution on assignments to its scope;
- these local distributions are *consistent*: if x_i is in the scope of C , then the marginal of the local distribution for C on coordinate i must equal the one-variable marginal $\mu_{i,a}$.

Why is this an LP relaxation? If $x_1, \dots, x_n$ is an integral assignment, we set

$$\mu_{i,a} = \begin{cases} 1 & \text{if } x_i = a, \\ 0 & \text{otherwise,} \end{cases} \quad \lambda_{C,\alpha} = \begin{cases} 1 & \text{if } \alpha \text{ is the restriction of the assignment to } S_C, \\ 0 & \text{otherwise.} \end{cases}$$

Then the LP objective is exactly the total weight of satisfied constraints. Thus the LP optimum upper bounds the true optimum.

Example: Max-Cut via local distributions. Let $G = (V, E)$ be an edge-weighted graph. For each vertex i we have variables $\mu_{i,0}, \mu_{i,1}$, and for each edge $e = ij$ we have four local variables $\lambda_{ij,00}, \lambda_{ij,01}, \lambda_{ij,10}, \lambda_{ij,11}$. Since the predicate is XOR, the edge contributes weight when its endpoints receive different values. The LP becomes

$$\begin{aligned}
 \max \quad & \sum_{ij \in E} w_{ij} (\lambda_{ij,01} + \lambda_{ij,10}) \\
 \text{s.t.} \quad & \mu_{i,0} + \mu_{i,1} = 1 \quad i \in V, \\
 & \lambda_{ij,00} + \lambda_{ij,01} + \lambda_{ij,10} + \lambda_{ij,11} = 1 \quad ij \in E, \\
 & \lambda_{ij,00} + \lambda_{ij,01} = \mu_{i,0}, \quad \lambda_{ij,10} + \lambda_{ij,11} = \mu_{i,1} \quad ij \in E, \\
 & \lambda_{ij,00} + \lambda_{ij,10} = \mu_{j,0}, \quad \lambda_{ij,01} + \lambda_{ij,11} = \mu_{j,1} \quad ij \in E, \\
 & \mu_{i,a}, \lambda_{ij,ab} \geq 0.
 \end{aligned}$$

The objective counts the probability that an edge is cut, and the constraints say that the edge distribution must agree with the one-vertex marginals at both endpoints. This is exactly the same local consistency principle as in the general LP, specialized to arity 2 and the XOR predicate.

SDP Lift: The SDP uses the same local-distribution viewpoint but encodes the probabilities by orthogonal vectors. In particular, orthogonality expresses mutual exclusion of different values or different local assignments. Introduce:

- a root vector u with $\|u\|^2 = 1$;
- for every variable i and value $a \in \{0, 1\}$, a vector $v_{i,a}$;
- for every constraint C and local assignment $\alpha \in \{0, 1\}^{S_C}$, a vector $z_{C,\alpha}$.

The intended meaning is:

- $\|v_{i,a}\|^2$ is the marginal probability that $x_i = a$;
- $\|z_{C,\alpha}\|^2$ is the local probability that the variables in S_C take the joint values prescribed by α .

To make this a genuine vector program, we write all constraints in terms of inner products.

The resulting SDP is:

$$\begin{aligned}
 \max \quad & \sum_C w_C \sum_{\alpha \in \{0,1\}^{S_C}} P_C(\alpha) \|z_{C,\alpha}\|^2 \\
 \text{s.t.} \quad & \|u\|^2 = 1, \\
 & v_{i,0} \cdot v_{i,1} = 0 \quad i = 1, \dots, n, \\
 & u \cdot v_{i,a} = \|v_{i,a}\|^2 \quad i = 1, \dots, n, a \in \{0, 1\}, \\
 & \|v_{i,0}\|^2 + \|v_{i,1}\|^2 = 1 \quad i = 1, \dots, n, \\
 & z_{C,\alpha} \cdot z_{C,\beta} = 0 \quad \text{for every } C, \alpha \neq \beta, \\
 & u \cdot z_{C,\alpha} = \|z_{C,\alpha}\|^2 \quad \text{for every } C, \alpha \in \{0, 1\}^{S_C}, \\
 & \sum_{\alpha \in \{0,1\}^{S_C}} \|z_{C,\alpha}\|^2 = 1 \quad \text{for every constraint } C, \\
 & \sum_{\alpha \in \{0,1\}^{S_C}: \alpha(i)=a} \|z_{C,\alpha}\|^2 = \|v_{i,a}\|^2 \quad \text{for every } C, i \in S_C, a \in \{0, 1\}, \\
 & v_{i,a} \cdot z_{C,\alpha} = 0 \quad \text{if } i \in S_C \text{ and } \alpha(i) \neq a, \\
 & v_{i,\alpha(i)} \cdot z_{C,\alpha} = \|z_{C,\alpha}\|^2 \quad \text{if } i \in S_C.
 \end{aligned}$$

All constraints are linear in the Gram matrix entries, so this is an SDP. The interpretation is parallel to the LP:

- $\{v_{i,0}, v_{i,1}\}$ acts like a distribution on the value of x_i ;
- $\{z_{C,\alpha}\}_\alpha$ acts like a local distribution on assignments to the scope of C ;
- the local distributions are consistent with the one-variable marginals.

The consistency constraints have two parts. First,

$$\sum_{\alpha: \alpha(i)=a} \|z_{C,\alpha}\|^2 = \|v_{i,a}\|^2$$

says that the marginal of the local distribution for C on variable i agrees with the one-variable marginal. Second, if $\alpha(i) = a$, then the vector $z_{C,\alpha}$ is allowed to overlap only with $v_{i,a}$ and is forced to be orthogonal to the wrong value $v_{i,1-a}$.

The constraints involving u and the norm sums play the role of normalization. Intuitively they say that the vectors $\{v_{i,0}, v_{i,1}\}$ partition the unit vector u , and similarly the vectors $\{z_{C,\alpha}\}_\alpha$ partition u for each constraint C . This is the vector-program analogue of saying that the underlying probabilities sum to 1.

This should be compared with the preceding MAX-2-SAT relaxation. There we exploited the special structure of the Boolean domain and encoded the two values of each variable by a single sign vector. For a generic Boolean CSP, the more symmetric and more useful view is to keep one object for each possible value and one object for each local assignment to a constraint scope.

Why is this an SDP relaxation? Given an integral assignment $x_1, \dots, x_n \in \{0, 1\}$, fix any unit vector u and set

$$v_{i,a} = \begin{cases} u & \text{if } x_i = a, \\ 0 & \text{otherwise,} \end{cases} \quad z_{C,\alpha} = \begin{cases} u & \text{if } \alpha \text{ is the restriction of the assignment to } S_C, \\ 0 & \text{otherwise.} \end{cases}$$

Then all constraints are satisfied, and the objective value is exactly the total weight of satisfied constraints. Thus the SDP optimum is at least the true optimum.

Example: Max-Cut via the SDP lift. For MAX-CUT, in addition to the vectors $v_{i,0}, v_{i,1}$ for each vertex i , we have four local vectors $z_{ij,00}, z_{ij,01}, z_{ij,10}, z_{ij,11}$ for

each edge ij . The SDP specializes to

$$\begin{aligned}
 \max \quad & \sum_{ij \in E} w_{ij} (\|z_{ij,01}\|^2 + \|z_{ij,10}\|^2) \\
 \text{s.t.} \quad & \|u\|^2 = 1, \\
 & v_{i,0} \cdot v_{i,1} = 0 \quad i \in V, \\
 & u \cdot v_{i,a} = \|v_{i,a}\|^2 \quad i \in V, a \in \{0,1\}, \\
 & \|v_{i,0}\|^2 + \|v_{i,1}\|^2 = 1 \quad i \in V, \\
 & z_{ij,ab} \cdot z_{ij,a'b'} = 0 \quad ij \in E, (a,b) \neq (a',b'), \\
 & u \cdot z_{ij,ab} = \|z_{ij,ab}\|^2 \quad ij \in E, a,b \in \{0,1\}, \\
 & \sum_{a,b \in \{0,1\}} \|z_{ij,ab}\|^2 = 1 \quad ij \in E, \\
 & \|z_{ij,00}\|^2 + \|z_{ij,01}\|^2 = \|v_{i,0}\|^2 \quad ij \in E, \\
 & \|z_{ij,10}\|^2 + \|z_{ij,11}\|^2 = \|v_{i,1}\|^2 \quad ij \in E, \\
 & \|z_{ij,00}\|^2 + \|z_{ij,10}\|^2 = \|v_{j,0}\|^2 \quad ij \in E, \\
 & \|z_{ij,01}\|^2 + \|z_{ij,11}\|^2 = \|v_{j,1}\|^2 \quad ij \in E, \\
 & v_{i,a} \cdot z_{ij,a'b} = 0 \quad ij \in E, a \neq a', \\
 & v_{j,b} \cdot z_{ij,ab'} = 0 \quad ij \in E, b \neq b', \\
 & v_{i,a} \cdot z_{ij,ab} = \|z_{ij,ab}\|^2 \quad ij \in E, a,b \in \{0,1\}, \\
 & v_{j,b} \cdot z_{ij,ab} = \|z_{ij,ab}\|^2 \quad ij \in E, a,b \in \{0,1\}.
 \end{aligned}$$

Here the objective counts the probability mass on the two edge assignments $(0,1)$ and $(1,0)$ that correspond to being cut. This is the SDP analogue of the preceding LP. It is more cumbersome than the Goemans–Williamson vector program because it keeps explicit local assignment vectors for every edge, but it illustrates the generic CSP paradigm very clearly.

Finite-Domain Generalization: The same template extends immediately to any finite domain $[q] = \{1, 2, \dots, q\}$: one introduces vectors $v_{i,a}$ for each $a \in [q]$ and vectors $z_{C,\alpha}$ for each local assignment $\alpha \in [q]^{S_C}$. The only change is notational.

Let us write the finite-domain version explicitly. Let Λ be a finite constraint language over domain $[q]$. An instance of $\text{MAX-CSP}(\Lambda)$ has variables $x_1, \dots, x_n$ taking values in $[q]$, and weighted constraints C , where each constraint C has a scope $S_C = (i_1, \dots, i_t)$ and a predicate $P_C : [q]^t \rightarrow \{0,1\}$. The objective is to

maximize

$$\sum_C w_C P_C(x_{i_1}, \dots, x_{i_t}).$$

The corresponding finite-domain SDP is obtained by replacing $\{0, 1\}$ by $[q]$ throughout:

$$\begin{aligned} \max \quad & \sum_C w_C \sum_{\alpha \in [q]^{S_C}} P_C(\alpha) \|z_{C,\alpha}\|^2 \\ \text{s.t.} \quad & \|u\|^2 = 1, \\ & v_{i,a} \cdot v_{i,b} = 0 \quad i = 1, \dots, n, a \neq b, \\ & u \cdot v_{i,a} = \|v_{i,a}\|^2 \quad i = 1, \dots, n, a \in [q], \\ & \sum_{a \in [q]} \|v_{i,a}\|^2 = 1 \quad i = 1, \dots, n, \\ & z_{C,\alpha} \cdot z_{C,\beta} = 0 \quad \text{for every } C, \alpha \neq \beta, \\ & u \cdot z_{C,\alpha} = \|z_{C,\alpha}\|^2 \quad \text{for every } C, \alpha \in [q]^{S_C}, \\ & \sum_{\alpha \in [q]^{S_C}} \|z_{C,\alpha}\|^2 = 1 \quad \text{for every constraint } C, \\ & \sum_{\alpha \in [q]^{S_C} : \alpha(i)=a} \|z_{C,\alpha}\|^2 = \|v_{i,a}\|^2 \quad \text{for every } C, i \in S_C, a \in [q], \\ & v_{i,a} \cdot z_{C,\alpha} = 0 \quad \text{if } i \in S_C \text{ and } \alpha(i) \neq a, \\ & v_{i,\alpha(i)} \cdot z_{C,\alpha} = \|z_{C,\alpha}\|^2 \quad \text{if } i \in S_C. \end{aligned}$$

This formulation is deliberately generic. Different problems admit additional valid constraints, stronger hierarchies, or more specialized geometric interpretations, but the important point is that the same local-distribution template applies to every finite-domain MAX-CSP. One then seeks a rounding scheme that converts these consistent local distributions into a single global assignment while preserving as much of the objective value as possible.

20.4.3 Raghavendra's Theorem

The SDP above is meant to illustrate the local-distribution viewpoint. Raghavendra's theorem is stated for a stronger canonical SDP relaxation (often called the *basic SDP*) built from the same local-consistency idea. The surprise is not merely that one can write a generic SDP, but that this canonical relaxation captures the optimal approximation threshold for every MAX-CSP, assuming the Unique Games Conjecture.

Theorem 20.6 (Raghavendra [222]). *Fix any finite MAX-CSP(Λ) over a finite domain, and let α_Λ denote the best approximation ratio achievable by rounding the canonical basic SDP relaxation for Λ . Then for every $\epsilon > 0$ there is a polynomial-time algorithm that achieves approximation ratio at least $\alpha_\Lambda - \epsilon$. Moreover, assuming the Unique Games Conjecture, no polynomial-time algorithm can achieve approximation ratio better than $\alpha_\Lambda + \epsilon$.*

In other words, under UGC the optimum approximation ratio for every finite-domain MAX-CSP is exactly the integrality-gap threshold of a canonical SDP relaxation. The Goemans–Williamson theorem for MAX-CUT is the most famous special case, while MAX-2-SAT, MAX-DICUT, MAX- k -CUT, and many other problems fall under the same umbrella.

20.5 Graph Coloring via SDP

A proper k -coloring of a graph $G = (V, E)$ assigns each vertex a color from $\{1, \dots, k\}$ so that no two adjacent vertices share a color. The *chromatic number* $\chi(G)$ is the minimum k for which a proper k -coloring exists. Deciding whether $\chi(G) \leq 3$ is NP-complete [116]. General graph coloring is extremely hard to approximate: it is NP-hard to color a graph using fewer than $n^{1-\epsilon}$ colors for any fixed $\epsilon > 0$ [262].

This hardness does not preclude interesting algorithms for graphs with small chromatic number. We focus on the case $\chi(G) = 3$ and ask: how few colors does an efficient algorithm need to properly color a 3-colorable graph? This is a *promise problem* since we cannot efficiently verify 3-colorability. SDP-based approximation algorithms for this setting were initiated by Karger, Motwani, and Sudan [167].

20.5.1 A Simple $O(\sqrt{n})$ -Coloring

A key observation is that if $\chi(G) = 3$, then for any vertex u the subgraph $G[N(u)]$ induced by the neighbors of u is 2-colorable (bipartite), since the neighbors of u receive only two of the three colors. We can identify a bipartite decomposition of $G[N(u)]$ in polynomial time.

This leads to a simple algorithm parameterized by a threshold δ ; it is essentially Wigderson’s neighborhood-coloring idea [254].

SimpleColoring:

```

 $G' \leftarrow G$ 
while  $(\Delta(G') > \delta)$  do
    let  $v$  be a vertex of degree  $\Delta(G')$  in  $G'$ 
    2-color  $G'[N(v)]$  using 2 fresh colors; assign  $v$  a fresh color
    remove  $v$  and  $N(v)$  from  $G'$ 
endwhile
color  $G'$  with  $\delta + 1$  fresh colors (greedy)
    
```

In each iteration of the while loop at least $\delta + 1$ vertices are removed, so the loop runs at most $t \leq n/(\delta + 1)$ times. Each iteration uses at most 3 fresh colors (2 for $N(v)$, 1 for v), so the total number of colors is at most $3t + (\delta + 1) \leq 3n/(\delta + 1) + (\delta + 1)$. Choosing $\delta = \Theta(\sqrt{n})$ minimizes this to $O(\sqrt{n})$.

Theorem 20.7. *There is a polynomial-time algorithm that properly colors any 3-colorable graph using $O(\sqrt{n})$ colors [254].*

20.5.2 Vector Chromatic Number

To do better than $O(\sqrt{n})$ we use an SDP relaxation of coloring called the *vector chromatic number* [167]. It is closely related to the Lovász theta function and other classical SDPs for graph coloring [130]. It is convenient to first optimize the edge-correlation parameter

$$z^*(G) = \max \left\{ z : \begin{array}{ll} v_i \cdot v_j \leq -z & (i, j) \in E, \\ v_i \cdot v_i = 1 & i \in V, \\ v_i \in \mathbb{R}^n & i \in V \end{array} \right\}.$$

Since all constraints are linear in the inner products, this is an SDP. The *vector chromatic number* is then defined by

$$\chi_v(G) := 1 + \frac{1}{z^*(G)}.$$

Thus larger feasible values of z correspond to better colorings; if adjacent vertices can be forced to make angle at least 120° , then $z \geq 1/2$ and hence $\chi_v(G) \leq 3$.

Lemma 20.5. *If G is k -colorable, then $z^*(G) \geq 1/(k - 1)$, and hence $\chi_v(G) \leq k$. In particular, every 3-colorable graph has $z^*(G) \geq 1/2$ and therefore $\chi_v(G) \leq 3$.*

Proof. Given a proper k -coloring with color classes $V_1, \dots, V_k$, we assign to the k color classes the k vertices of a regular $(k - 1)$ -simplex centered at the origin. Concretely, assign the same k -dimensional vector to every vertex in color class V_ℓ :

$$v_i = (\beta, \dots, \beta, \alpha, \beta, \dots, \beta) \quad \text{with } \alpha \text{ in position } \ell,$$

where $\alpha = -\sqrt{(k-1)/k}$ and $\beta = 1/\sqrt{k(k-1)}$. These are unit vectors with the same pairwise inner product $-1/(k-1)$, which is exactly the defining geometry of a regular simplex. Thus $v_i \cdot v_i = 1$ for all i and $v_i \cdot v_j = -1/(k-1)$ whenever vertices i, j receive different colors, in particular for all edges (i, j) . Hence $z = 1/(k-1)$ is feasible, so $z^*(G) \geq 1/(k-1)$ and $\chi_v(G) \leq 1 + (k-1) = k$. ■

Equivalently, among k unit vectors the largest possible common pairwise angle is achieved by a regular $(k-1)$ -simplex, whose vertices have inner product $-1/(k-1)$. For $k = 3$, this is just the familiar configuration of three unit vectors in the plane spaced 120° apart; adjacent vertices then have inner product $\cos(120^\circ) = -1/2$.

20.5.3 SDP-Based Coloring: $O(\Delta^a \log n)$ Colors

Let $v_1, \dots, v_n$ be a feasible vector coloring for a 3-colorable graph G with $v_i \cdot v_j \leq -1/2$ on every edge (i, j) . Such a solution exists by Lemma 20.5. Then the angle θ_{ij} between the vectors of any edge (i, j) satisfies $\theta_{ij} \geq 2\pi/3$ (i.e., at least 120°).

VecColoring: (parameter $t = 2 + \lceil \log_3 \Delta \rceil$)
 compute vector coloring $v_1, \dots, v_n$ via SDP
while $V \neq \emptyset$ **do**
 choose t random unit vectors $r_1, \dots, r_t \in \mathbb{R}^n$
 assign each vertex i color $(\text{sgn}(v_i \cdot r_1), \dots, \text{sgn}(v_i \cdot r_t))$
 let B = vertices incident to a monochromatic edge
 permanently assign colors to $V \setminus B$; set $V \leftarrow B$
endwhile

We analyze the probability that a given edge (i, j) is monochromatic (assigned the same color string).

Lemma 20.6. *For each edge (i, j) , $\mathbf{P}[\text{color}(i) = \text{color}(j)] \leq (1/3)^t$.*

Proof. Vertex i and j receive the same color in one coordinate if and only if the random hyperplane r_ℓ does not separate v_i and v_j . Since $\theta_{ij} \geq 2\pi/3$,

$$\mathbf{P}[\text{sgn}(r \cdot v_i) = \text{sgn}(r \cdot v_j)] = 1 - \frac{\theta_{ij}}{\pi} \leq 1 - \frac{2}{3} = \frac{1}{3}.$$

The t random vectors are independent, so the edge is monochromatic with probability at most $(1/3)^t$. ■

Setting $t = 2 + \log_3 \Delta$ gives $(1/3)^t \leq 1/(9\Delta)$. The expected number of monochromatic edges is at most $m/(9\Delta) \leq n/18$. Each monochromatic edge

causes at most 2 vertices to be added to B , so $\mathbb{E}[|B|] \leq n/9$. By Markov's inequality, with probability at least $\frac{1}{2}$, $|B| \leq 2n/9$. Each iteration thus colors at least a constant fraction of the remaining vertices.

In each iteration we use $2^t \leq 4 \cdot \Delta^a$ fresh colors, where $a = \log 2 / \log 3 \approx 0.631$. After $O(\log n)$ iterations all vertices are colored.

Theorem 20.8 (Karger–Motwani–Sudan, 1998). *There is a polynomial-time randomized algorithm that colors any 3-colorable graph using $O(\Delta^a \log n)$ colors, where $a = \log_3 2 \approx 0.631$ [167].*

Plugging this into the **SimpleColoring** algorithm: when $\Delta(G') > \delta$ we use the SDP-based coloring on $N(v)$, using $O(\delta^a \log n)$ colors per iteration. Optimizing δ gives an $O(n^{0.387})$ -coloring. A further refinement of the SDP rounding (using a smarter projection step) improves the per-iteration cost to $O(\Delta^{1/3}(\log \Delta)^{1/2} \log n)$ colors, yielding:

Theorem 20.9 (Karger–Motwani–Sudan, 1998). *There is a polynomial-time randomized algorithm that colors any 3-colorable graph using $O(n^{1/4} \log^{1/2} n)$ colors [167].*

The approach extends to k -colorable graphs: one can color them with $O(\Delta^{1-2/k}(\log \Delta)^{1/2} \log n)$ colors, or $O(n^{1-3/(k+1)} \log^{1/2} n)$ colors overall.

It is worth noting that the vector chromatic number itself can have a large integrality gap. Feige, Langberg, and Schechtman showed that for every positive integer k there are graphs with $\chi_v(G) \leq k$ but with no independent set significantly larger than $n/\Delta^{1-2/k}$, and hence with chromatic number at least on the order of $\Delta^{1-2/k}$ [105]. In particular, the SDP is not a good approximation to the true chromatic number in general; they also show that the vector chromatic number does not approximate the chromatic number within a factor better than $n/\text{polylog } n$ [105].

20.5.4 The Lovász Theta Function

Another important SDP relaxation for graph coloring is the *Lovász theta function*. It is usually defined for a graph $H = (V, E)$ via an *orthonormal representation* [130, 196]: this is an assignment of unit vectors u_i to the vertices such that

$$u_i \cdot u_j = 0 \quad \text{whenever } ij \notin E(H).$$

Thus nonadjacent vertices must receive orthogonal vectors. Given in addition a unit vector c , called a *handle*, define

$$\vartheta(H) := \min \left\{ \max_{i \in V} \frac{1}{(c \cdot u_i)^2} : \begin{array}{l} \|c\|^2 = 1, \\ \|u_i\|^2 = 1 \quad i \in V, \\ u_i \cdot u_j = 0 \quad ij \notin E(H) \end{array} \right\}.$$

This is an SDP: the objective can be written by introducing a variable t and minimizing t subject to $(c \cdot u_i)^2 \geq 1/t$ for all i , or equivalently $c \cdot u_i \geq 1/\sqrt{t}$ after choosing the signs of the u_i so that $c \cdot u_i \geq 0$.

It is useful to view $\vartheta(H)$ as a relaxation of coloring the complement graph $\overline{H}$. A proper k -coloring of $\overline{H}$ assigns each vertex one of k colors so that adjacent vertices get different colors. We can encode such a coloring by assigning to each color ℓ the basis vector $e_\ell \in \mathbb{R}^k$, and assigning each vertex the basis vector of its color. Then adjacent vertices of $\overline{H}$ receive orthogonal vectors, which means exactly that nonadjacent vertices of H receive orthogonal vectors. Thus a genuine coloring gives a very special orthonormal representation in which all vectors are chosen from a finite set of basis directions. The theta function relaxes this discrete condition by allowing each vertex to choose *any* unit vector, subject only to the orthogonality constraints. The handle vector c measures how concentrated these vectors are around a common direction: in an integral k -coloring one can take $c = (1, \dots, 1)/\sqrt{k}$, in which case $(c \cdot u_i)^2 = 1/k$ for every vertex, and the objective value is exactly k . Thus $\vartheta(H)$ can be seen as a continuous, SDP-based analogue of the coloring number $\chi(\overline{H})$.

The theta function gives a remarkable bridge between independence and coloring.

Theorem 20.10 (Lovász sandwich theorem). *For every graph H ,*

$$\alpha(H) \leq \vartheta(H) \leq \chi(\overline{H}).$$

Proof. Let S be any independent set in H . By the definition of an orthonormal representation, the vectors $\{u_i : i \in S\}$ are pairwise orthogonal, hence orthonormal. If $t = \max_i 1/(c \cdot u_i)^2$, then $(c \cdot u_i)^2 \geq 1/t$ for every i , so by Bessel's inequality,

$$1 = \|c\|^2 \geq \sum_{i \in S} (c \cdot u_i)^2 \geq \frac{|S|}{t}.$$

Hence $t \geq |S|$. Since this holds for every feasible solution and every independent set, $\vartheta(H) \geq \alpha(H)$.

For the upper bound, suppose $\overline{H}$ has a proper k -coloring with color classes $V_1, \dots, V_k$. Assign to each vertex in V_ℓ the basis vector $e_\ell \in \mathbb{R}^k$, and let $c = (1, 1, \dots, 1)/\sqrt{k}$. If $ij \notin E(H)$ then ij is an edge of $\overline{H}$, so i and j receive different colors and therefore orthogonal basis vectors. Thus this is a valid orthonormal representation of H . Moreover, for every vertex i ,

$$(c \cdot u_i)^2 = \frac{1}{k},$$

so the objective value is k . Therefore $\vartheta(H) \leq k = \chi(\overline{H})$. ■

Applying the theorem to $H = \overline{G}$ yields

$$\omega(G) \leq \vartheta(\overline{G}) \leq \chi(G).$$

Thus $\vartheta(\overline{G})$ is another efficiently computable lower bound on the chromatic number. It is closely related to the vector chromatic number, but the formulations are conceptually different: vector coloring constrains the angles on edges of G , while $\vartheta(\overline{G})$ uses orthogonality on edges of G together with a global handle vector.

The connection to perfect graphs is especially striking. Recall that a graph G is *perfect* if every induced subgraph H satisfies $\chi(H) = \omega(H)$. Since the sandwich theorem gives $\omega(H) \leq \vartheta(\overline{H}) \leq \chi(H)$ for every induced subgraph H , it follows that if G is perfect then

$$\vartheta(\overline{H}) = \omega(H) = \chi(H) \quad \text{for every induced subgraph } H.$$

Thus on perfect graphs the theta function is exact: the SDP relaxation has no gap. This exactness was one of the key reasons the theta function became central in perfect graph theory, and it played a major role in Lovász's proof of the weak perfect graph theorem that G is perfect if and only if $\overline{G}$ is perfect [195, 234].

For the approximation setting, however, this bound is less directly useful. If G is 3-colorable then certainly $\vartheta(\overline{G}) \leq \chi(G) \leq 3$, but this only gives a necessary condition for 3-colorability. It does not by itself yield a rounding procedure that converts a theta solution into a good coloring. By contrast, the vector chromatic number relaxation is tailored to the rounding algorithms above: the constraint that adjacent vertices make large angles is exactly what allows repeated random hyperplane cuts to produce partial colorings.

Example: the five-cycle. The graph C_5 is self-complementary, so the sandwich theorem gives

$$2 = \alpha(C_5) \leq \vartheta(C_5) \leq \chi(C_5) = 3.$$

Lovász showed that in fact

$$\vartheta(C_5) = \sqrt{5}.$$

This is already nontrivial: the SDP relaxation returns the exact value $\sqrt{5}$, which lies strictly between the combinatorial quantities 2 and 3. This example was one of the first indications that SDP can capture delicate graph structure not visible to simpler LP-based relaxations [130, 196].

20.6 Metric Embeddings via SDP

A *metric space* is a pair (V, d) where $d : V \times V \rightarrow \mathbb{R}_{\geq 0}$ is symmetric, satisfies the triangle inequality, and has $d(u, v) = 0$ iff $u = v$. A *semi-metric* allows distinct

points to have distance 0.

Given a finite metric (V, d) , we often wish to embed it into a simpler, more structured target metric (W, ρ) via a map $f : V \rightarrow W$ that approximately preserves distances. The *distortion* of f is the smallest $c \geq 1$ such that for some scale $\alpha > 0$:

$$\alpha \cdot d(u, v) \leq \rho(f(u), f(v)) \leq \alpha \cdot c \cdot d(u, v) \quad \forall u \neq v.$$

Distortion 1 means f is isometric (perfectly preserving distances).

Two natural target metrics arise in the context of vector programs:

- **Euclidean (ℓ_2):** $\rho(i, j) = \|v_i - v_j\|_2$, for vectors $v_i \in \mathbb{R}^k$.
- **Squared Euclidean (negative-type):** $\rho(i, j) = \|v_i - v_j\|_2^2$.

Minimizing the distortion into either of these targets can be formulated as an SDP.

20.6.1 Finding the Optimal ℓ_2 Embedding

Suppose we are given a finite metric (V, d) with $|V| = n$. We seek to find the least distortion embedding of this metric into ℓ_2 . It suffices to consider ℓ_2 space with n dimensions. We can find the optimum distortion via SDP as follows. Let $V = \{1, 2, \dots, n\}$. The problem requires us to find n vectors $v_1, \dots, v_n \in \mathbb{R}^n$ such that the Euclidean distances between these vectors minimize the distortion. Since we can scale the vectors we can assume without loss of generality that we are seeking an embedding that does not contract any pair of points in (V, d) . Thus, we want $\|v_i - v_j\|_2 \geq d(i, j)$ for all $i \neq j$. Subject to this constraint we want to $\min \max_{i \neq j} \|v_i - v_j\|_2 / d(i, j)$. To write this as an SDP we consider the decision problem where we wish to check if there is an embedding with distortion at most c where $c \geq 1$ is a given number. We see that this is a vector program below.

$$\begin{aligned} \|v_i - v_j\|_2^2 &\geq d(i, j)^2 & i \neq j \\ \|v_i - v_j\|_2^2 &\leq c^2 d(i, j)^2 & i \neq j \\ v_i &\in \mathbb{R}^n & i \in [n] \end{aligned}$$

Expanding $\|v_i - v_j\|_2^2 = v_i \cdot v_i - 2v_i \cdot v_j + v_j \cdot v_j$, all constraints become linear in the inner products. This is a vector program, hence an SDP. Note that the ℓ_2 triangle inequality holds automatically for any vectors, so no additional constraints are needed. To find the smallest c one can do binary search.

The following fundamental results bound the distortion of ℓ_2 embeddings.

Theorem 20.11 (Bourgain, 1985). *Every n -point metric embeds into ℓ_2 with distortion $O(\log n)$. Moreover, there exist metrics that require distortion $\Omega(\log n)$.*

Lemma 20.7 (Johnson–Lindenstrauss, 1984). *Every n -point subset of ℓ_2 (in any dimension) can be embedded into $\mathbb{R}^d$, for $d = O(\varepsilon^{-2} \log n)$, with distortion at most $1 + \varepsilon$.*

20.6.2 Embedding into Negative-type Metric

Negative-type metrics are formally defined as follows.

Definition 20.12. *A metric space (V, d) is of negative-type if for all finite set of points $x_1, x_2, \dots, x_n \in V$ and all real numbers $c_1, c_2, \dots, c_n$ such that $\sum_i c_i = 0$ the following inequality is true:*

$$\sum_{i=1}^n \sum_{j=1}^n c_i c_j d(x_i, x_j) \leq 0.$$

We will not discuss the motivation for the preceding abstract but there is a nice geometric characterization via Schoenberg’s theorem. We state it for finite metric spaces though it holds in the general case as well.

Theorem 20.13. *A finite metric (V, d) is of negative-type iff there exists a mapping $f : V \rightarrow \mathbb{R}^d$ such that $d(u, v) = \|f(u) - f(v)\|_2^2$ for all $u, v \in V$.*

For the squared Euclidean distance $d(i, j) = \|v_i - v_j\|_2^2$, the triangle inequality does *not* hold automatically. For example, three collinear points at 0, 1, 2 give $\|0 - 2\|^2 = 4 > 1 + 1 = \|0 - 1\|^2 + \|1 - 2\|^2$.

Exercise 20.1. Prove that if (V, d) is induced by square distances of vectors in Euclidean space then d satisfies the abstract negative-type condition.

Lemma 20.8. *Vectors $v_1, \dots, v_n$ satisfy the triangle inequality for squared Euclidean distances, i.e. $\|v_i - v_k\|^2 \leq \|v_i - v_j\|^2 + \|v_j - v_k\|^2$ for all i, j, k , if and only if every three vectors form an acute or right triangle.*

Proof. Exercise. ■

Lemma 20.9. *Suppose (V, d) is an ℓ_1 metric. Then (V, d) is of negative type.*

Proof. We sketch the proof. If (V, d_1) and (V, d_2) are of negative-type then their non-negative sum is also of negative-type. This is easy to see by the abstract characterization. We have seen that (V, d) is an ℓ_1 metric iff it is in the cut cone. It is easy to verify that a cut metric is of negative-type by seeing it via the Euclidean characterization. Putting together these two facts yields the desired lemma. ■

SDP to check whether metric is negative-type: Suppose we are given a finite metric (V, d) and want to check whether it is of negative-type. Schoenberg's theorem gives a way to recast this as checking whether there are vectors $v_1, v_2, \dots, v_n$ (assuming $|V| = [n]$) such that $d(i, j) = \|v_i - v_j\|_2^2$ for all $i, j \in V$. We can write as a vector program. The same idea allows us to check whether a given metric (V, d) has a distortion c embedding into negative-type.

$$\begin{aligned} \|v_i - v_j\|_2^2 &\geq d(i, j) & i \neq j \\ \|v_i - v_j\|_2^2 &\leq cd(i, j) & i \neq j \\ \|v_i - v_j\|_2^2 + \|v_j - v_k\|_2^2 &\geq \|v_i - v_k\|_2^2 & i, j, k \\ v_i &\in \mathbb{R}^n & i \in [n] \end{aligned}$$

Note the similarities and differences from the previous vector program on finding the best embedding into ℓ_2 . We need to explicitly write the triangle inequality constraints. Since the vector program is linear in c we can make it a variable instead of doing binary search.

$$\begin{aligned} \min c \\ \|v_i - v_j\|_2^2 &\geq d(i, j) & i \neq j \\ \|v_i - v_j\|_2^2 &\leq cd(i, j) & i \neq j \\ \|v_i - v_j\|_2^2 + \|v_j - v_k\|_2^2 &\geq \|v_i - v_k\|_2^2 & i, j, k \\ v_i &\in \mathbb{R}^n & i \in [n] \end{aligned}$$

Exercise 20.2. Argue formally why the preceding optimization problem can be written as an SDP.

20.6.3 Application to Sparsest Cut

Recall the NON-UNIFORM SPARSEST CUT problem. The input is a graph $G = (V, E)$ with non-negative edge capacities $c : E \rightarrow \mathbb{R}_+$ and a set of pairs $(s_1, t_1), \dots, (s_k, t_k)$ along with non-negative demand values $D_1, D_2, \dots, D_k$. Given a set/cut $S \subseteq V$ the *sparsity* of the cut S is defined as the ratio $\frac{c(\delta(S))}{(\sum_{i: |S \cap \{s_i, t_i\}|=1} D_i)}$. In the preceding chapter we wrote an LP relaxation as follows.

$$\begin{aligned}
 \min \quad & \sum_{uv \in E} c(uv) d(uv) \\
 \sum_{i=1}^k D_i d(s_i t_i) &= 1 \\
 d \text{ is a metric on } V
 \end{aligned}$$

If we let d be the set of all metrics we get a natural relaxation which requires us to write the triangle inequalities $d(u, v) + d(v, w) \geq d(u, w)$ for all triples. The dual of this LP relaxation, as we saw, the maximum concurrent flow LP. We rounded the LP using the fact that any n -point metric embeds into ℓ_1 with $O(\log n)$ -distortion. We also saw that if d is an ℓ_1 metric then there is no loss in the rounding. Note that an optimum integer solution to the sparsest cut problem corresponds to a cut $(S, V - S)$ which embeds into ℓ_1 since all cut metrics are ℓ_1 metrics. Thus, if we can force d to be an ℓ_1 metric, we will get an exact algorithm for NON-UNIFORM SPARSEST CUT but this is not feasible because the problem is NP-Hard. However, we can try negative-type metrics since, as we saw, ℓ_1 is a negative-type metric. As we saw in the preceding subsection, we can write an SDP relaxation to enforce that d is a negative-type metric by mapping the vertices to vectors in Euclidean space and enforcing triangle inequalities. To make it clear we write the explicit relaxation below.

We change the notation since we will also be working vectors. We let $V = [n]$ and use $d(i, j)$ for distance between i and j . We let $P \subseteq \binom{V}{2}$ be set of demand pairs.

$$\begin{aligned}
 \min \quad & \sum_{ij \in E} c(ij) d(ij) \\
 \sum_{ij \in P} Dem(ij) d(ij) &= 1 \\
 d(ij) &= \|v_i - v_j\|_2^2 \quad i, j \in V \\
 \|v_i - v_j\|_2^2 + \|v_j - v_k\|_2^2 &\geq \|v_i - v_k\|_2^2 \quad i, j, k \in V \\
 v_i &\in \mathbb{R}^n \quad i \in [n]
 \end{aligned}$$

The preceding is an SDP that can be solved in polynomial-time to arbitrarily good precision. The main question is whether its integrality gap of the SDP is better. This boils to the following question. What is the best distortion for embedding a negative-type metric into ℓ_1 ? Arora, Rao and Vazirani, in a

breakthrough [22], obtained an $O(\sqrt{\log n})$ approximation for UNIFORM SPARSEST CUT problem using this relaxation and developed powerful technical tools along the way. Subsequently, Chawla, Gupta and Ræcke [67], and then Arora, Lee, Naor [17, 19] showed a general result upper bounding the distortion to embed an n -point negative-type metric embeds into ℓ_1 .

Theorem 20.14 (Arora-Lee-Naor). *Every n -point negative-type metric embeds into ℓ_1 with distortion $O(\sqrt{\log n} \log \log n)$. This yields a polynomial-time $O(\sqrt{\log n} \log \log n)$ -approximation for NON-UNIFORM SPARSEST CUT.*

There is a lower bound of $\Omega(\sqrt{\log n})$ on the distortion required so the theorem is almost-tight.

20.7 Sum-of-Squares and Polynomial Optimization

SDP has a deep connection to polynomial optimization through the theory of sum-of-squares (SOS) decompositions. We motivate it below.

Examples of univariate polynomials

- $f(x) = 2x^2 + 3x - 10$
- $f(x) = x^3 + 10x^2 + x - 100$
- $f(x) = 10x^4 + x^3 - 53x^2$

Examples of multivariate polynomials

- $f(x) = x^3y^2 + x^2y^1 - 3x^2y^2 + 1$ (degree is 5)
- $f(x) = x^4y^2 + x^2y^4 - 3x^2y^2 + 1$ (degree is 6)

Consider the unconstrained optimization problem

$$\min_{x \in \mathbb{R}^n} p(x_1, x_2, \dots, x_n).$$

Claim 20.7.1. *If the degree of p is odd, the minimum is $-\infty$.*

Hence the interesting case is even-degree polynomials. Can we find the minimum efficiently? Note that polynomials are neither convex nor concave in general and can have many local minima. See figures. Finding a global minimum is non-trivial, especially for multivariate polynomials. We consider the decision version of the minimization problem phrased in a particularly nice way.

Question 20.10. Given $p(x_1, \dots, x_n)$ of even degree, is $p(\mathbf{x}) \geq 0$ for all $\mathbf{x} \in \mathbb{R}^n$?

Suppose we can answer the above. Then finding the minimum of p is finding

$$\max \lambda \text{ such that } p(x_1, \dots, x_n) - \lambda \geq 0$$

We can ask two questions about the preceding problem.

- Computational. Can we solve it efficiently?
- Structural/mathematical: Is there a nice and easy to understand reason that a polynomial is always non-negative?

Definition 20.15. A polynomial $p(x_1, \dots, x_n)$ is a sum of squares (SOS) if there exist polynomials $q_1, \dots, q_k$ such that $p(\mathbf{x}) = \sum_{i=1}^k q_i(\mathbf{x})^2$.

Clearly every SOS polynomial is non-negative. For univariate polynomials of any even degree, the converse also holds.

Theorem 20.16. A univariate polynomial $p(x)$ of even degree is non-negative for all $x \in \mathbb{R}$ if and only if it is SOS.

Proof sketch. Factor p over $\mathbb{C}$. Since p has real coefficients, complex roots appear in conjugate pairs $(x - \alpha)(x - \bar{\alpha}) = (x - a)^2 + b^2$, which is a sum of squares. Non-negativity forces all real roots to have even multiplicity, so they contribute squares as well. Hence p is a product of non-negative quadratics and therefore a sum of squares. ■

For multivariate polynomials the situation is more subtle. Hilbert proved in 1888 that non-negativity does not imply SOS in general.

Theorem 20.17 (Hilbert, 1888). Non-negative polynomials are SOS in exactly three cases:

1. univariate polynomials of any even degree ($n = 1$);
2. quadratic polynomials in any number of variables ($\deg = 2$);
3. bivariate quartic polynomials ($n = 2, \deg = 4$).

In all other cases there exist non-negative polynomials that are not SOS.

The first explicit counterexample was given by Motzkin in 1967. Consider the following polynomial that is named after him.

$$M(x, y) = x^4 y^2 + x^2 y^4 - 3x^2 y^2 + 1.$$

Applying the AM-GM inequality to the three terms $x^4 y^2$, $x^2 y^4$, and 1 gives

$$\frac{x^4 y^2 + x^2 y^4 + 1}{3} \geq (x^4 y^2 \cdot x^2 y^4 \cdot 1)^{1/3} = x^2 y^2$$

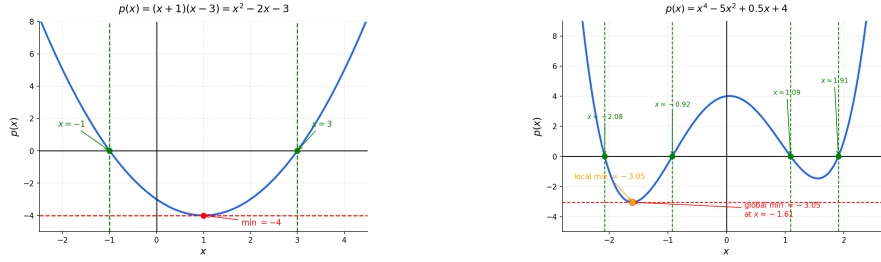


Figure 20.3: Examples of even-degree univariate polynomials. A non-negative univariate polynomial always admits an SOS decomposition.

and hence $M(x, y) \geq 0$ everywhere. However, one can show via SDP that M is not SOS. In fact, $M - \gamma$ is not SOS for any constant γ .

It turns out that checking whether a polynomial is non-negative is strongly NP-Hard even for degree 4.

Theorem 20.18. *Given a polynomial p of degree 4, it is strongly NP-hard to decide if it is positive definite, that is, if $p(x) > 0$ for all $x \in \mathbb{R}^n$.*

Despite the preceding negative result one can obtain interesting results by considering SOS polynomials and using hierarchies of relaxations to approach the global minimization problem.

20.7.1 Checking SOS via SDP

It turns out that one can check whether p is an SOS polynomial via SDP. Since the notation is a bit messy we will first illustrate this in the simpler setting of univariate polynomials.

Let $p(x)$ be a univariate polynomial of degree $2d$.

$$p(x) = p_0 + p_1x + p_2x^2 + \dots + p_{2d}x^{2d}$$

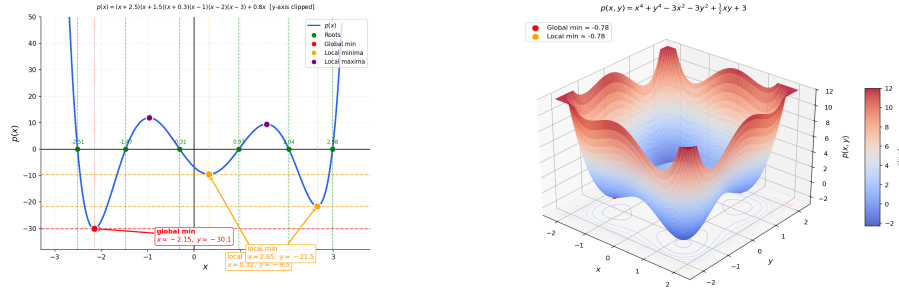


Figure 20.4: Examples of multivariate even-degree polynomials. Unlike the univariate case, non-negative multivariate polynomials need not be SOS.

We can view p as a vector of size $2d + 1$. Let $z(x) = [1 \ x \ x^2 \ \dots \ x^d]^T$ be a vector of monomials in $(d + 1)$ dimensions.

Theorem 20.19. $p(x)$ is SOS iff there exists a psd matrix $Q \in M_{d+1}$ such that $z(x)^T Q z(x) = p(x)$.

We will see the proof in the more general setting of multivariate case but one direction is easy. Suppose there exists $Q = WW^T$ such that $z(x)^T WW^T z(x) = p(x)$ then $p(x) = \sum_k (v_k^T z(x))^2$ where $v_1, v_2, \dots, v_{d+1}$ are the column vectors of W which implies that $p(x)$ is an SOS polynomial. The converse direction is not too hard either.

We can see the above in action in the setting of a degree 4 polynomial. $p(x) = p_0 + p_1x + p_2x^2 + p_3x^3 + p_4x^4$. We use monomial vector $z = [1, x, x^2]^T$. We expand $p(x) = z^T Q z$ for an unknown 3×3 symmetric matrix Q :

$$z^T Q z = Q_{11} \cdot 1 + 2Q_{12} \cdot x + (Q_{22} + 2Q_{13}) \cdot x^2 + 2Q_{23} \cdot x^3 + Q_{33} \cdot x^4$$

We then match each monomial coefficient with that of p :

$$Q_{11} = p_0, 2Q_{12} = p_1, (Q_{22} + 2Q_{13}) = p_2, 2Q_{23} = p_3, Q_{33} = p_4$$

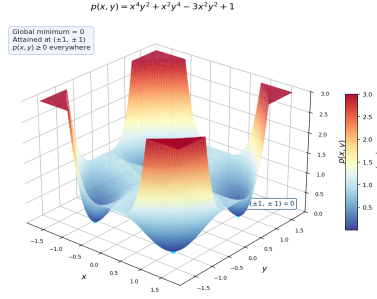


Figure 20.5: The surface of the Motzkin polynomial $M(x, y) = x^4y^2 + x^2y^4 - 3x^2y^2 + 1$. It is non-negative everywhere (touching zero at $(x, y) = (\pm 1, \pm 1)$ and at the origin) yet cannot be written as a sum of squares of polynomials.

Via SDP we find any $Q \geq 0$ satisfying the 5 equations above. If it is feasible then we can write $Q = LL^T$ and then $p(x) = \sum_k (L_k^T z(x))^2$.

Say $p(x) = x^4 - 2x^3 + 3x^2 - 2x + 1$. We can solve the SDP with the constraints as given above and see that

$$(1 - x + x^2)^2 = 1 - 2x + 3x^2 - 2x^3 + x^4.$$

Multivariate case: The preceding approach works for the multivariate case as well. Let $z(x)$ be the column vector of all monomials in $x_1, \dots, x_n$ of degree at most d ; its dimension is $s = \binom{n+d}{d}$.

Theorem 20.20. *A polynomial $p(x)$ of degree $2d$ is SOS if and only if there exists a psd matrix $Q \in \mathbb{R}^{s \times s}$ such that $p(x) = z(x)^T Q z(x)$.*

Proof. ($\Leftarrow$) Write $Q = W^T W$ and let $w_1, \dots, w_s$ be the rows of W . Then $z^T Q z = \sum_i (w_i \cdot z)^2$, and each $q_i(x) = w_i \cdot z(x)$ is a polynomial of degree $\leq d$, giving an SOS decomposition of p .

($\Rightarrow$) Suppose $p = \sum_i q_i^2$. Write each $q_i(x) = c_i^T z(x)$ for a coefficient vector c_i . Set $Q_i = c_i c_i^T \geq 0$. Then $p(x) = \sum_i z(x)^T Q_i z(x) = z(x)^T (\sum_i Q_i) z(x)$ and $Q = \sum_i Q_i \geq 0$. ■

Checking whether p is SOS reduces to an SDP feasibility problem: expand $z(x)^T Q z(x)$ and match each monomial coefficient with the corresponding coefficient in p . Each such equality is a linear constraint on the entries of Q . The problem is:

$$\begin{array}{ll} \text{Find} & Q \in \mathbb{R}^{s \times s} \\ \text{s.t.} & \sum_{\alpha+\beta=\gamma} Q_{\alpha\beta} = p_\gamma \quad \forall \text{ monomials } \gamma, \\ & Q \geq 0. \end{array}$$

If feasible, the Cholesky decomposition of Q yields the SOS representation. Infeasibility certifies that p is not SOS.

Example. For the Motzkin polynomial $M(x, y)$ of degree 6, the monomial vector z for degree ≤ 3 in two variables has $s = \binom{5}{3} = 10$ entries, so Q would be 10×10 . The linear system arising from matching coefficients, subject to $Q \geq 0$, turns out to be infeasible. This confirms M is not SOS.

Remark 20.2. Note that the running time to check whether $p(x)$ is an SOS depends polynomially on $n^{O(d)}$. Since a degree d polynomial in n variables has representation size at least n (each variable needs to occur at least once), the running time is polynomial for any fixed d , nevertheless the complexity grows quite rapidly.

20.7.2 Hilbert's 17th Problem

As we saw Hilbert showed that $p(x)$ can be non-negative without being an SOS polynomial. He asked a related question.

Question 20.11. Hilbert's 17th Problem (1900) If $p(x_1, \dots, x_n) \geq 0$ for all $x \in \mathbb{R}^n$, can p be written as a *sum of squares of rational functions*?

$$p(x) = \sum_k \left(\frac{q_k(x)}{r_k(x)} \right)^2$$

His question was answered affirmatively by Artin.

Theorem 20.21 (Artin, 1927). *Every nonnegative polynomial $p \in \mathbb{R}[x_1, \dots, x_n]$ is a sum of squares of rational functions.*

Artin's proof was non-constructive. Finding an explicit representation and doing so algorithmically required additional ideas and SDP plays a key role in this.

Recall the Motzkin polynomial.

$$M(x, y) = x^4y^2 + x^2y^4 - 3x^2y^2 + 1$$

We saw that $M(x, y) \geq 0$ but it is not SOS. However, one can show that $M(x, y) \cdot (x^2 + y^2 + 1)$ is SOS. This proves that $M(x, y) \geq 0$ since $(x^2 + y^2 + 1)$ is SOS and gives an explicit ratio-of-SOS representation consistent with Artin's theorem.

How did we know to try $(x^2 + y^2 + 1)$? This is not obvious. Artin's theorem says that one can prove non-negativity of a polynomial by using rational functions but does not explicitly bound the degree of the denominators nor does it provide a systematic way to search for them. Subsequent work has developed various theorems and methods to make this search constructive and algorithmic.

20.7.3 Polynomial Optimization with Constraints: The Lasserre Hierarchy

Consider the constrained polynomial optimization problem

$$p^* = \min_{x \in K} p(x), \quad K = \{x \in \mathbb{R}^n : g_i(x) \geq 0, i = 1, \dots, m\},$$

where p and $g_1, \dots, g_m$ are polynomials. The feasible set K is a *semialgebraic set*. Many important settings are semialgebraic: the box $[-1, 1]^n$ (via $g_i = 1 - x_i^2$), the unit ball ($g = 1 - \|x\|^2$), polytopes (linear g_i), and combinatorial problems with binary variables ($x_i^2 = 1$).

Polynomial optimization is NP-hard in general: it subsumes integer programming and quadratic programming over combinatorial constraints. The strategy is not to solve exactly but to compute provable lower bounds via SOS-based algebraic certificates.

A value $\lambda \leq p^*$ is a valid lower bound if and only if $p(x) - \lambda \geq 0$ on K . Checking this condition is NP-hard, but we can *certify* it efficiently by finding an SOS decomposition of the form

$$p(x) - \lambda = \sigma_0(x) + \sum_{i=1}^m \sigma_i(x) g_i(x),$$

where each σ_i is SOS. This is valid because on K each term $\sigma_i g_i \geq 0$ (since $\sigma_i \geq 0$ everywhere and $g_i \geq 0$ on K), so the right-hand side is non-negative on K , confirming $\lambda \leq p^*$. Maximizing λ subject to the existence of such a certificate is an SDP.

Level- r SDP (Lasserre Hierarchy). Fix a level $r \geq 1$ (the half-degree), large enough that $2r \geq \deg p$. Write $d_i = \lceil \deg g_i / 2 \rceil$. Then we represent

$$\sigma_0(x) = z_r(x)^T Q_0 z_r(x), \quad \sigma_i(x) = z_{r-d_i}(x)^T Q_i z_{r-d_i}(x) \quad (i \geq 1),$$

where each $Q_i \geq 0$. This ensures that $\deg(\sigma_i g_i) \leq 2r$ for every i . Thus Q_0 has size $\binom{n+r}{r} \times \binom{n+r}{r}$, while Q_i has size $\binom{n+r-d_i}{r-d_i} \times \binom{n+r-d_i}{r-d_i}$ for $i \geq 1$. After expanding and matching monomial coefficients with $p - \lambda$, one obtains linear constraints in λ and the entries of $Q_0, Q_1, \dots, Q_m$:

$$\begin{aligned} \max \quad & \lambda \\ \text{s.t.} \quad & \text{coefficient equations: } p - \lambda = \sigma_0 + \sum_i \sigma_i g_i, \\ & Q_0, Q_1, \dots, Q_m \geq 0. \end{aligned}$$

This is an SDP that can be solved in polynomial time (in the matrix size, which is $n^{O(r)}$).

Theorem 20.22 (Putinar, 1993). *Suppose K satisfies the Archimedean condition (which holds whenever K is compact and this is certified by the constraints). Then every polynomial $p > 0$ on K admits an SOS-multiplier representation $p = \sigma_0 + \sum_i \sigma_i g_i$ with σ_i SOS.*

Theorem 20.23 (Lasserre, 2001). *The optimal values λ_r of the level- r SDPs form a non-decreasing sequence converging to p^* :*

$$\lambda_1 \leq \lambda_2 \leq \dots \leq p^*.$$

Under the Archimedean condition, $\lambda_r \rightarrow p^$ as $r \rightarrow \infty$ (with finite convergence when $p > 0$ on K).*

The Lasserre hierarchy (also known as the SOS hierarchy) is a systematic sequence of increasingly tight SDP relaxations that converges to the true polynomial optimum. The tradeoff is that the SDP size grows as $n^{O(r)}$, so higher levels are more expensive but provide tighter bounds. In practice the hierarchy often converges at level $r = 1$ or $r = 2$.

Exercise 20.3. Verify that the quadratic program for MAX-CUT—maximizing $\sum_{i,j \in E} (1 - y_i y_j)/2$ over $y_i^2 = 1$ —can be cast as a level-1 Lasserre relaxation and that this relaxation coincides with the Goemans–Williamson SDP.

Exercise 20.4. Show that for the Motzkin polynomial $M(x, y) = x^4 y^2 + x^2 y^4 - 3x^2 y^2 + 1$, the polynomial $M \cdot (x^2 + y^2 + 1)$ is SOS by finding an explicit SOS decomposition.

20.7.4 Dual Viewpoint: Moments and Pseudo-Distributions

It is useful to reinterpret the Lasserre hierarchy from the dual point of view. Instead of thinking of the relaxation as searching for an SOS certificate proving that $\lambda \leq p^*$, we can think of it as optimizing over a relaxed notion of probability distributions on the feasible set K .

Exact distributional formulation. Suppose first that μ is an actual probability distribution supported on K . Then

$$\mathbb{E}_\mu[p(x)] \geq p^*$$

when p^* is the minimum of p over K , and in fact the minimum is attained by a point mass distribution $\mu = \delta_{x^*}$ at an optimal point $x^* \in K$. Thus we may write the original problem as

$$p^* = \min_{\mu \text{ supported on } K} \mathbb{E}_\mu[p(x)].$$

This formulation is exact but infinite-dimensional: a general distribution has too many degrees of freedom to optimize over directly.

Truncated moments. The key idea is to remember only the low-degree moments of the distribution. For each monomial $x^\alpha = x_1^{\alpha_1} \cdots x_n^{\alpha_n}$ of degree at most $2r$, define

$$y_\alpha := \mathbb{E}_\mu[x^\alpha].$$

These numbers are the *moments* of μ . Since expectation is linear, the objective depends linearly on them:

$$\mathbb{E}_\mu[p(x)] = \sum_{\alpha} p_{\alpha} y_{\alpha}.$$

Thus the moments play the role of SDP variables. When $\mu = \delta_{x^*}$, we simply have $y_{\alpha} = (x^*)^{\alpha}$.

Why psd matrices appear. If the moments come from a genuine distribution, then for every polynomial q of degree at most r ,

$$\mathbb{E}_\mu[q(x)^2] \geq 0.$$

Write $q(x) = c^T z_r(x)$ in the monomial basis. Then

$$\mathbb{E}_\mu[q(x)^2] = \mathbb{E}_\mu[z_r(x)^T c c^T z_r(x)] = c^T M_r(y) c,$$

where $M_r(y)$ is the *moment matrix* indexed by monomials up to degree r , with entries

$$M_r(y)_{\alpha, \beta} = y_{\alpha + \beta}.$$

Hence any true distribution must satisfy $M_r(y) \geq 0$. Similarly, if μ is supported on $K = \{x : g_i(x) \geq 0\}$, then for every q of appropriate degree,

$$\mathbb{E}_\mu[g_i(x)q(x)^2] \geq 0,$$

which yields the psd *localizing matrices* associated with the constraints g_i .

Pseudo-distributions. The level- r Lasserre relaxation keeps only moments up to degree $2r$ and uses the numbers

$$y_\alpha \quad (|\alpha| \leq 2r)$$

as the SDP variables. That is, for every monomial x^α up to degree $2r$ we introduce one scalar variable y_α , intended to represent $\mathbb{E}[x^\alpha]$. The SDP objective is simply

$$\min \sum_{\alpha} p_{\alpha} y_{\alpha},$$

and the constraints are:

- $y_0 = 1$,
- the moment matrix $M_r(y)$ is psd,
- for each constraint polynomial g_i , we form the matrix $M_{r-d_i}(g_i y)$ indexed by monomials of degree at most $r - d_i$, where $d_i = \lceil \deg g_i / 2 \rceil$, and require this matrix to be psd. Concretely, if $q(x) = c^T z_{r-d_i}(x)$ then

$$c^T M_{r-d_i}(g_i y) c = \tilde{\mathbb{E}}[g_i(x)q(x)^2] \geq 0,$$

so this matrix constraint is exactly the condition that the pseudo-expectation sees $g_i(x)$ as nonnegative when multiplied by any square of appropriate degree.

Thus the SDP is really optimizing over a truncated moment vector y . However, not every feasible truncated moment vector y comes from an actual probability distribution. For this reason it is better to think of y as defining a *degree- $2r$ pseudo-expectation operator* $\tilde{\mathbb{E}}$, or equivalently a *pseudo-distribution*, by setting

$$\tilde{\mathbb{E}}[x^\alpha] := y_\alpha$$

and then extending linearly to all polynomials of degree at most $2r$. It behaves like expectation on all such polynomials:

- $\tilde{\mathbb{E}}[1] = 1$,
- $\tilde{\mathbb{E}}[q(x)^2] \geq 0$ for $\deg q \leq r$,
- $\tilde{\mathbb{E}}[g_i(x)q(x)^2] \geq 0$ whenever degrees fit.

But there may be no genuine probability measure behind it. Thus the hierarchy optimizes over a larger space than the space of actual distributions, and is therefore a relaxation.

Example: Boolean variables. Many optimization problems in approximation algorithms can be written as 0/1 integer programs

$$\min\{c^T x : Ax \leq b, x \in \{0, 1\}^n\}.$$

This is a special case of polynomial optimization in which the constraint polynomials are linear inequalities together with the Boolean identities $x_i^2 - x_i = 0$. Thus the general Lasserre hierarchy specializes directly: one introduces moments

$$y_S := \tilde{\mathbb{E}} \left[\prod_{i \in S} x_i \right]$$

for $|S| \leq 2r$, with the interpretation that they represent relaxed joint moments of the Boolean variables. Because the identities $x_i^2 = x_i$ allow every polynomial to be multilinearized, the moment variables are naturally indexed by subsets $S \subseteq [n]$ rather than arbitrary multi-indices. The localizing matrices are also simpler than in the general polynomial case, since the only inequality constraints are the linear forms $b_\ell - a^\ell x \geq 0$. Thus the Lasserre hierarchy for 0/1 integer programs may be viewed as the Boolean specialization of the general moment/SOS framework. If one prefers variables in $\{-1, 1\}$ instead, then the Boolean identities become $x_i^2 = 1$, and the same viewpoint applies after this change of coordinates.

Duality with SOS certificates. The primal SOS view and the dual moment view are two sides of the same SDP. The primal asks for an SOS proof that $p(x) - \lambda \geq 0$ on K ; the dual asks for a pseudo-distribution on K minimizing the relaxed expectation of p . Thus:

- SOS certificates are efficiently checkable *proofs* of lower bounds;
- pseudo-distributions are relaxed *witnesses* that mimic feasible random solutions.

To make this precise, let $y = \{y_\alpha\}_{|\alpha| \leq 2r}$ be a truncated moment sequence. The associated *moment functional* L_y is the linear map on polynomials of degree at most $2r$ defined by

$$L_y(x^\alpha) := y_\alpha$$

for each monomial x^α , and then extended linearly. Thus if

$$h(x) = \sum_{\alpha} h_{\alpha} x^{\alpha},$$

then

$$L_y(h) = \sum_{\alpha} h_{\alpha} y_{\alpha}.$$

If y comes from a genuine probability distribution μ , then $L_y(h) = \mathbb{E}_\mu[h(x)]$; in the pseudo-distribution setting, this is exactly the pseudo-expectation operator $\tilde{\mathbb{E}}[h]$.

The connection becomes very concrete once we pair an SOS certificate with this moment functional. If

$$\sigma_0(x) = z_r(x)^T Q_0 z_r(x), \quad \sigma_i(x) = z_{r-d_i}(x)^T Q_i z_{r-d_i}(x)$$

with $Q_i \geq 0$, then for any degree- $2r$ pseudo-expectation $\tilde{\mathbb{E}} = L_y$ we have

$$\tilde{\mathbb{E}}[\sigma_0] = L_y(\sigma_0) = \text{tr}(Q_0 M_r(y)), \quad \tilde{\mathbb{E}}[\sigma_i g_i] = L_y(\sigma_i g_i) = \text{tr}(Q_i M_{r-d_i}(g_i y)).$$

Indeed, if $Q_0 = \sum_j a_j a_j^T$ then

$$\sigma_0(x) = \sum_j (a_j^T z_r(x))^2,$$

and therefore

$$\tilde{\mathbb{E}}[\sigma_0] = \sum_j \tilde{\mathbb{E}}[(a_j^T z_r(x))^2] \geq 0$$

because $M_r(y) \geq 0$. The same argument applies to each $\sigma_i g_i$ using the localizing matrix. Thus if the primal SOS certificate proves

$$p(x) - \lambda = \sigma_0(x) + \sum_{i=1}^m \sigma_i(x) g_i(x),$$

then every feasible pseudo-distribution satisfies

$$\tilde{\mathbb{E}}[p] - \lambda = \tilde{\mathbb{E}}[\sigma_0] + \sum_{i=1}^m \tilde{\mathbb{E}}[\sigma_i g_i] \geq 0.$$

Equivalently,

$$\lambda \leq \tilde{\mathbb{E}}[p].$$

This is exactly the weak duality relation between the primal SOS SDP and the dual moment SDP.

One can read this identity in two complementary ways. From the primal side, the matrices Q_i encode a formal algebraic proof that $p(x) - \lambda$ is nonnegative on K . From the dual side, the moments y define a linear functional that tests this proof against every low-degree square. If the pseudo-distribution were an actual distribution supported on K , the inequality above would simply say that every valid proof of a lower bound remains valid after taking expectation. The

moment relaxation keeps exactly the part of this reasoning that only depends on low-degree moments.

This also clarifies why the two SDPs have the shapes they do. In the primal, coefficient matching is the natural way to enforce the polynomial identity $p - \lambda = \sigma_0 + \sum_i \sigma_i g_i$. In the dual, each such coefficient equation gets a dual variable y_α , interpreted as the moment of the monomial x^α . The psd constraints on the Gram matrices Q_i become psd constraints on the moment and localizing matrices. Thus “Gram matrix” and “moment matrix” are dual linear algebraic avatars of the same low-degree polynomial space.

What about strong duality? In general, weak duality always holds, but strong duality need not hold automatically for an arbitrary SDP. Standard conic duality tells us that there is no duality gap, and that the optimum is attained on the opposite side, if a Slater-type condition holds: one of the two SDPs must be *strictly feasible*. For the moment relaxation, this means there is a truncated moment vector y with

$$M_r(y) > 0, \quad M_{r-d_i}(g_i y) > 0 \text{ for all } i,$$

while for the SOS side it means there are positive definite Gram matrices satisfying the coefficient equations.

In polynomial optimization this strict feasibility is not automatic. If the feasible set K has nonempty interior, then one often gets it on the moment side by taking a genuine probability distribution supported inside an open subset of K on which all g_i are strictly positive; this makes every nonzero low-degree square have strictly positive expectation, yielding positive definite moment and localizing matrices. Under such a condition, the SOS SDP and the moment SDP have the same optimum value. On the other hand, if K lies in a lower dimensional variety, or if one imposes equality constraints such as $x_i^2 = 1$, strict feasibility may fail in the naive formulation, and then one should only assert weak duality unless one has taken care of this degeneracy by an equivalent reformulation.

Thus the safe interpretation is:

- weak duality always holds;
- strong duality holds under a Slater condition, which in many natural compact semialgebraic settings is satisfied after a suitable formulation;
- for degenerate algebraic feasible sets, one has to be more careful.

Conceptually, the primal is trying to prove that no feasible point can have value below λ , while the dual is trying to imitate a feasible point, or more generally a feasible distribution, whose expected value is as small as possible. When strong

duality holds and the relaxation is tight, these two viewpoints meet at the true optimum.

This dual viewpoint is often very helpful in understanding both the power and the limitations of the Lasserre hierarchy; see also [186].

20.7.5 Sherali–Adams on the Boolean Hypercube

It is instructive to compare the Lasserre hierarchy with the Sherali–Adams (SA) hierarchy, which is an LP hierarchy based on a very similar philosophy. Classical Sherali–Adams is most naturally viewed as a hierarchy for Boolean integer programs with linear constraints, and we restrict ourselves to that setting here. Thus we work with Boolean variables $x_1, \dots, x_n \in \{0, 1\}$.

Suppose we have a 0/1 optimization problem

$$\max c^T x \quad \text{s.t. } Ax \leq b, \quad x_i \in \{0, 1\} \text{ for all } i.$$

If μ is an actual probability distribution on feasible Boolean solutions, then for every pair of disjoint sets $S, T \subseteq [n]$ the quantity

$$y_{S,T} := \mathbb{E}_\mu \left[\prod_{i \in S} x_i \prod_{j \in T} (1 - x_j) \right]$$

is nonnegative. In fact, it is exactly the probability that all variables in S are equal to 1 and all variables in T are equal to 0. These are the local probabilities that SA keeps track of.

The level- r Sherali–Adams relaxation introduces variables $y_{S,T}$ for all disjoint S, T with $|S| + |T| \leq r$. The objective is the relaxed expectation

$$\tilde{\mathbb{E}}[c^T x] = \sum_{i=1}^n c_i \tilde{\mathbb{E}}[x_i].$$

Formally, the relaxation is the LP

$$\begin{aligned} \max \quad & \sum_{i=1}^n c_i y_{\{i\}, \emptyset} \\ \text{s.t.} \quad & y_{\emptyset, \emptyset} = 1, \\ & y_{S,T} \geq 0 \quad \text{for all disjoint } S, T \text{ with } |S| + |T| \leq r, \\ & y_{S,T} = y_{S \cup \{i\}, T} + y_{S, T \cup \{i\}} \\ & \quad \text{for all disjoint } S, T \text{ with } |S| + |T| \leq r - 1 \text{ and } i \notin S \cup T, \\ & \tilde{\mathbb{E}} \left[(b_\ell - a^\ell x) \prod_{i \in S} x_i \prod_{j \in T} (1 - x_j) \right] \geq 0 \\ & \quad \text{for every row } \ell \text{ of } Ax \leq b \text{ and all disjoint } S, T \text{ with } |S| + |T| \leq r - 1. \end{aligned}$$

The splitting identities encode local consistency: once i is not yet specified, the event (S, T) is the disjoint union of the two cases $x_i = 1$ and $x_i = 0$. The lifted constraint inequalities come from the fact that if $a^\ell x \leq b_\ell$ is valid for every feasible Boolean assignment, then

$$(b_\ell - a^\ell x) \prod_{i \in S} x_i \prod_{j \in T} (1 - x_j) \geq 0$$

for every disjoint S, T , and hence its expectation must also be nonnegative. Thus SA optimizes over a collection of locally consistent marginals that look like probabilities on all subsets of at most r variables.

This is closely related to the pseudo-distribution viewpoint for Lasserre. Both hierarchies replace a true distribution by a truncated collection of low-degree moments. The key difference is in what nonnegativity conditions they enforce. Sherali–Adams only uses nonnegativity statements obtained by taking an original constraint, rewriting it as a linear inequality of the form $b_\ell - a^\ell x \geq 0$, and multiplying it by Boolean indicator monomials $\prod_{i \in S} x_i \prod_{j \in T} (1 - x_j)$. Because these test functions are so special, the resulting conditions are all linear. Lasserre, on the other hand, tests nonnegativity against *all* low-degree squares $q(x)^2$ and, more generally, $g_i(x)q(x)^2$, which yields the psd moment and localizing matrix constraints.

In this sense, Sherali–Adams is the LP analogue of the moment method: it captures local consistency of small Boolean events. Lasserre is a strictly stronger SDP strengthening that captures positivity of all low-degree polynomial test functions. One can think of SA as asking that “all small local views look like valid distributions,” while Lasserre asks for the stronger condition that “all low-degree polynomials behave as if expectation were taken under a genuine distribution.”

20.7.6 Boolean CSPs as a Common Framework

Boolean CSPs provide a particularly clean setting in which to see both Sherali–Adams and Lasserre arise from the same basic idea. Suppose we have Boolean variables $x_1, \dots, x_n \in \{0, 1\}$ and an objective of the form

$$\max \sum_{C \in \mathcal{C}} w_C P_C(x_{S_C}),$$

where each constraint C depends only on a small set S_C of variables, has weight $w_C \geq 0$, and $P_C : \{0, 1\}^{S_C} \rightarrow \{0, 1\}$ indicates whether the local assignment satisfies the predicate of C . Equivalently, we seek a Boolean assignment maximizing the total weight of satisfied constraints. In contrast to the general constrained

polynomial optimization setup discussed earlier, there are no additional feasibility constraints here beyond the requirement that the variables lie in $\{0, 1\}^n$; the predicates only define the objective being optimized.

The common relaxation idea is to replace a genuine distribution on Boolean assignments by a truncated collection of local statistics. In the Sherali–Adams view, these statistics are local probabilities of partial assignments. For each small set of variables one keeps track of quantities such as

$$\mathbf{P}[x_i = 1 \text{ for } i \in S, x_j = 0 \text{ for } j \in T],$$

and imposes linear constraints saying that these local probabilities are nonnegative and marginalize consistently. The contribution of a constraint C to the objective is then just its expected satisfaction under the corresponding local distribution. If every predicate has arity at most k , then one needs at least level $r \geq k$ in order for the level- r relaxation to represent the objective exactly, since the value of P_C depends on the joint distribution of all variables in S_C . Higher levels allow the relaxation to correlate several nearby constraints by tracking larger local neighborhoods.

The Lasserre viewpoint starts from the same Boolean CSP, but expresses it as a polynomial optimization problem over the Boolean cube. In the 0/1 setting one enforces the constraints $x_i \in \{0, 1\}$ via the polynomial identities $x_i^2 = x_i$, and each predicate P_C can be represented by a multilinear polynomial in the variables of S_C . One then introduces moments or pseudo-expectations of low-degree monomials and optimizes the relaxed objective

$$\sum_{C \in \mathcal{C}} w_C \tilde{\mathbb{E}}[P_C(x_{S_C})]$$

subject to the moment and localizing matrix constraints. These psd constraints enforce not just consistency of local marginals but positivity of $\tilde{\mathbb{E}}[q(x)^2]$ and $\tilde{\mathbb{E}}[g(x)q(x)^2]$ for all low-degree test polynomials q . Here too the arity matters: if each predicate has arity at most k , then one wants at least level $r \geq k$, since otherwise the relaxation does not contain all degree- k moments needed to evaluate the predicate polynomials exactly.

Thus Boolean CSPs form a natural common framework for both hierarchies. Sherali–Adams is the LP relaxation obtained by keeping only linear local-probability consistency conditions, while Lasserre is the stronger SDP relaxation obtained by imposing psd constraints on low-degree moments. Presenting the two hierarchies in this common Boolean CSP setting helps isolate their conceptual difference: SA reasons about small local events, whereas Lasserre reasons about all low-degree polynomials.

Bibliographic Notes

The Goemans–Williamson 0.878-approximation for MAX-CUT appeared in [122]; it was the first use of SDP to achieve a polynomial-time approximation ratio substantially better than $1/2$ for a fundamental NP-hard problem. The matching hardness result under the Unique Games Conjecture is due to Khot, Kindler, Mossel, and O’Donnell [175].

The SDP-based graph coloring results— $O(\sqrt{n})$ coloring via neighborhood 2-coloring, $O(\Delta^a \log n)$ coloring via the vector chromatic number, and the $O(n^{1/4} \text{polylog})$ algorithm—are from Karger, Motwani, and Sudan [167]. The vector chromatic number and the Lovász theta function, closely related SDPs for graph problems, were introduced earlier by Grötschel, Lovász, and Schrijver [130].

Bourgain’s $O(\log n)$ embedding theorem appeared in [44]; the Johnson–Lindenstrauss lemma is from [163]. The $O(\sqrt{\log n})$ -approximation for sparsest cut is due to Arora, Rao, and Vazirani [22]; see also the Fulkerson Prize citation.

Hilbert’s 1888 theorem on non-negative polynomials appears in his paper on ternary quartic forms; Artin’s positive solution to Hilbert’s 17th Problem is from [23]. The Motzkin polynomial was published in [209]. The SOS–SDP connection and the Lasserre SDP hierarchy were developed by Lasserre [184] and, independently, by Parrilo [216]. See the survey of Laurent [186] for a comprehensive treatment of SOS hierarchies and their applications. For background on the relation between Sherali–Adams and Lasserre for 0/1 problems, see also Laurent [186]. In the CSP setting, Sherali–Adams is especially useful for languages where local consistency captures the global optimum; see, for example, Thapper–Živný [245] on valued CSPs of bounded width. For approximation of hard Boolean max-CSPs, Sherali–Adams often serves more as a benchmark for the limitations of LP relaxations: see Chan–Lee–Raghavendra–Steurer [58] and the strong Sherali–Adams gap constructions of Georgiou–Magen–Tulsiani [119]. By contrast, the strongest positive approximation results for Boolean CSPs come from the SDP side: the basic examples are the Goemans–Williamson SDP for MAX-CUT and MAX-2-SAT [122], Raghavendra’s theorem gives the canonical SDP viewpoint for all CSPs [222], and Barak–Raghavendra–Steurer [36] show how higher rounds of the Lasserre hierarchy can be rounded effectively for 2-CSPs and UNIQUE GAMES.

Chapter 21

Hardness of Approximation

The preceding chapters have emphasized algorithmic techniques for proving upper bounds on approximation ratios for several classes of NP-Hard optimization problems. In this chapter we briefly look at the complementary question. How do we prove lower bounds on the approximation ratio that can be achieved in polynomial-time? Clearly, such lower bounds can only be shown under the assumption that $P \neq NP$. Proving hardness of approximation bounds is a big topic and this study has led to many deep results with the PCP theorem being the most notable one. We will not be able to go over these results in any level of detail. The goal in this chapter is to give a flavor of hardness results by considering three techniques.

1. direct reductions that create an objective-value gap;
2. amplification, where a small gap is used via reductions to obtain a larger gap; and
3. the PCP theorem, which is the source of many hardness results.

Throughout this chapter, an *inapproximability result* is a statement of the following form. If a polynomial-time algorithm with a specified approximation guarantee existed, then we could decide an NP-complete language in polynomial time. Thus the approximation algorithm would imply $P = NP$.

We use the same convention as in the rest of the notes: for a maximization problem, an α -approximation has value at least $\alpha \text{ OPT}$ with $0 < \alpha \leq 1$; for a minimization problem, a ρ -approximation has value at most $\rho \text{ OPT}$ with $\rho \geq 1$.

21.1 Direct Gap Reductions

The simplest hardness proofs for approximation are ordinary many-one reductions that convert a decision problem into an optimization problem where the objective value exhibits a gap between the yes and no instances of the decision problem. The reduction in Theorem 1.3 of Chapter 1 for k -CENTER is a simple and nice example. The reduction starts from the decision version of DOMINATING SET: given an undirected graph $G = (V, E)$ and an integer k , is there a dominating set in G of size at most k ? It creates a metric k -CENTER instance whose optimum is 1 in the yes case and at least 2 in the no case. The reduction is extremely simple: we convert (G, k) into (H, k) where H is the metric closure of G and k is the number of centers allowed. It is easy to see that if G has a dominating set of size $\leq k$ then the optimum value for the k -center instance on H is at most 1. If there is no dominating set of size k in G then any k -center solution in H must have value at least 2. A $(2 - \epsilon)$ -approximation would distinguish these two cases, which is impossible unless $P = NP$ [153]. We saw that k -Center admits a 2-approximation and hence we obtain a tight approximability threshold for this basic problem in an easy fashion. We also mentioned the k -Supplier problem in the clustering chapter where one can show a tight approximation threshold of 3, and the hardness is again via a simple direct reduction.

The general pattern is worth spelling out. Let Π be a minimization problem. Suppose that from an instance I of an NP-hard decision problem L (think of the problem as a language recognition problem) we can construct, in polynomial time, an instance $f(I)$ of Π such that

$$I \in L \Rightarrow \text{OPT}(f(I)) \leq A, \quad I \notin L \Rightarrow \text{OPT}(f(I)) \geq B,$$

where $B > A$. Then no polynomial-time approximation algorithm with ratio strictly smaller than B/A can exist unless $P = NP$. Indeed, such an algorithm would output a solution of value less than B in the yes case, while every feasible solution has value at least B in the no case.

For maximization problems the inequalities are reversed. If a reduction creates instances with

$$I \in L \Rightarrow \text{OPT}(f(I)) \geq B, \quad I \notin L \Rightarrow \text{OPT}(f(I)) \leq A,$$

where $B > A$, then no algorithm can always output a solution of value more than A/B times optimum, unless $P = NP$.

Two points are important. First, the proof only uses the algorithm as a decision procedure; we compare the value of the algorithm's output with a threshold.

Ideally we want the hardness to be robust in the sense that $\text{OPT}(f(I))$ to grow with the $|I|$. So technically we want the ratio $\text{OPT}(f(I))$ in the no and

yes cases to be larger than some ratio $\alpha > 1$ (for minimization problems). For example, in the k -Center problem we can easily scale the lengths of the edges and the reduction is robust. However, in some other examples we cannot do that. Recall the reduction from 2-Partition to Bin Packing. In the yes case we can pack using 2 bins and in the no case we need at least 3 bins and this shows that we cannot get better than a $(3/2 - \epsilon)$ -approximation. However, the reduction is not multiplicatively robust since we know the existence of an asymptotic PTAS.

21.2 Maximum Directed Edge-Disjoint Paths

We next give a second direct reduction, this time for a maximization problem. In the MAXIMUM DIRECTED EDGE-DISJOINT PATHS problem, the input is a directed graph $G = (V, A)$ and terminal pairs $(s_1, t_1), \dots, (s_k, t_k)$. The goal is to route as many pairs as possible by directed paths, subject to the paths being mutually edge-disjoint.

Theorem 21.1 (Fortune, Hopcroft, and Wyllie [114]). *Given a directed graph G and two terminal pairs (s_1, t_1) and (s_2, t_2) , it is NP-complete to decide whether there are two mutually edge-disjoint directed paths P_1 and P_2 where P_1 connects s_1 to t_1 and P_2 connects s_2 to t_2 .*

Theorem 21.2. *Unless $P = NP$, for every fixed $\epsilon > 0$ there is no $(1/2 + \epsilon)$ -approximation for MAXIMUM DIRECTED EDGE-DISJOINT PATHS.*

Proof. Take an instance of the two-pair directed disjoint paths problem and view it as an instance of the maximization problem with two demand pairs. We can assume without loss of generality that each of the given two pairs can be routed individually, otherwise the problem is easy. If the two-pair instance is a yes instance, then the optimum value is 2. If it is a no instance, then the optimum value is at 1.

A polynomial-time algorithm that always returns more than $(1/2 + \epsilon)$ times optimum would, on yes instances, return a solution of value strictly larger than 1, and hence of value 2. On no instances no solution of value 2 exists. Thus the approximation algorithm would decide the NP-complete two-pair problem. ■

21.2.1 Gap Amplification for Directed EDP

The preceding direct reduction gives only a factor-2 gap. A simple and beautiful amplification due to Guruswami, Khanna, Rajaraman, Shepherd, and Yannakakis [137] turns the same two-pair hardness into a near- $\sqrt{n}$ inapproximability result for MAX-EDP, where n is the number of vertices in the final directed graph. The

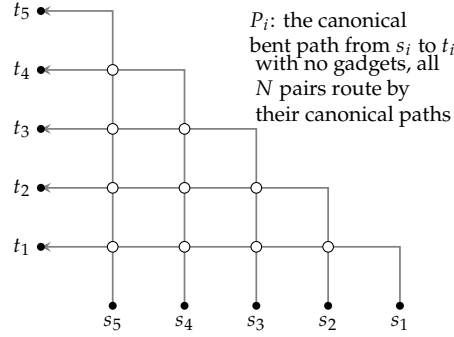


Figure 21.1: The skeleton for $N = 5$. There is one demand pair (s_i, t_i) for each canonical path P_i . The intersections are potential conflicts between two canonical paths; they are not additional demand pairs.

construction is a grid of copies of the hard two-pair instance; it is sparse, so the number of arcs is of the same order as the number of vertices.

Let H be a two-pair directed EDP instance with terminal pairs (x_1, y_1) and (x_2, y_2) . Choose an integer N . We construct a new directed EDP instance G_N with N demand pairs $(s_1, t_1), \dots, (s_N, t_N)$ as follows.

The skeleton. The reduction starts with a skeleton graph which is directed half-grid as shown in Figure 21.1 for $N = 5$. We observe the following. First, all the N pairs can be routed in the skeleton. Second, let Q_i be any path connecting s_i to t_i and Q_j be any path connecting s_j to t_j in the skeleton. If $i \neq j$ then Q_i and Q_j must intersect and “cross” at a vertex of the grid. We call P_i the canonical path for pair i which connects s_i to t_i with only one bend.

If we leave the skeleton in Figure 21.1 alone and simply regard the crossings as ordinary vertices with enough local arcs, all N pairs can clearly be routed: route each demand (s_i, t_i) along its canonical path P_i . The reduction works by replacing each crossing in that figure with a copy of the hard two-pair instance H .

Replacing crossings by gadgets. For every pair $1 \leq i < j \leq N$, put a fresh copy H^{ij} of H at the crossing of P_i and P_j . The copy H^{ij} has its own terminals

$$x_1^{ij}, y_1^{ij}, x_2^{ij}, y_2^{ij}.$$

For each demand i , add private connector arcs so that any s_i - t_i path is forced to pass through the copies involving i in a fixed order: in copy H^{ij} with $i < j$ it must enter at x_1^{ij} and leave at y_1^{ij} , while in copy H^{ji} with $j < i$ it must enter at x_2^{ji} and leave at y_2^{ji} . The connector arcs are private to this demand; the only places

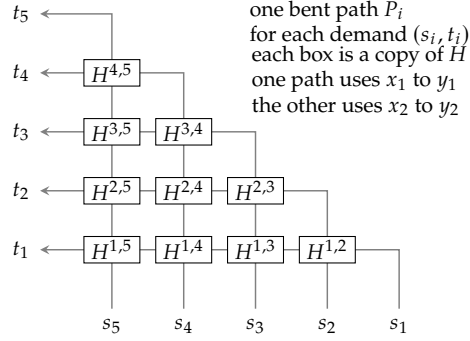


Figure 21.2: Schematic of the Guruswami–Khanna–Rajaraman–Shepherd–Yannakakis amplification for $N = 5$. There are N demand pairs, one for each bent route P_i from s_i to t_i , and a copy H^{ij} at the intersection of routes P_i and P_j . Private connector arcs, which concatenate the gadgets into the actual s_i – t_i routes, are suppressed.

where two canonical demands can interact are their common crossing copies H^{ij} . Figure 21.2 suppresses these private connectors, but each gray bent route should be read as a directed corridor through the corresponding sequence of boxes.

Thus the construction has N demand pairs and $\binom{N}{2}$ copies of the two-pair gadget.

The intuition is now simple. If H is a yes instance, every gadget can locally pass the two canonical paths through edge-disjointly, so all N canonical paths in Figure 21.2 survive. If H is a no instance, then no two demands can both be routed. The reason is not planarity by itself, but the forced ordering of the corridors: any two routed demands must pass through a common crossing box, and inside that box they would give a forbidden two-pair routing of a copy of H .

Lemma 21.1. *If the original two-pair instance H is a yes instance, then all N demands in G_N can be routed edge-disjointly. If H is a no instance, then at most one demand in G_N can be routed.*

Proof. Suppose first that H has edge-disjoint paths Q_1 from x_1 to y_1 and Q_2 from x_2 to y_2 . In every copy H^{ij} , route demand i through the copy of Q_1 and demand j through the copy of Q_2 . Concatenating these local routings with the private connector arcs gives a routing of all N demands. The only shared gadgets are the copies H^{ij} , and inside each such copy the two paths used are edge-disjoint.

Conversely, suppose H is a no instance and two demands are both routed in G_N . Say they are demands i and j , with $i < j$. Let Q_i and Q_j be their paths. Suppose we shrink each of the H -gadget graphs to a single node. Then we will

obtain paths Q'_i and Q'_j in the skeleton graph that connect the pairs and are edge-disjoint. These paths, by our earlier observation, have to cross at some internal vertex v of the skeleton. This implies that Q_i and Q_j have to traverse the gadget graph H at v and this is only possible if H is a yes instances — this is an informal proof sketch and formalizing it requires a bit more work but does not offer much more insight. ■

Theorem 21.3 (Guruswami et al. [137]). *Let n be the number of vertices. For every fixed $\epsilon > 0$, unless $P = NP$, directed MAX-EDP has no polynomial-time $n^{-1/2+\epsilon}$ -approximation.*

Proof. Let q be the size of the two-pair instance H . The construction above uses $\binom{N}{2}$ copies of H and $O(N^2)$ private connector arcs. Thus the number of vertices in G_N is

$$n = O(N^2 q).$$

The number of arcs is also $O(N^2 q)$; this is the sparsity point in the Guruswami et al. construction. We state the hardness in terms of n . In the yes case $\text{OPT}(G_N) = N$, while in the no case $\text{OPT}(G_N) \leq 1$.

Choose $N = q^c$ for a constant c to be fixed. Then $n = O(q^{2c+1})$. For any fixed $\epsilon > 0$, by choosing c large enough we have

$$q^c > C q^{(2c+1)(1/2-\epsilon)}$$

for the constant C hidden in the $O(\cdot)$ notation and all sufficiently large q . Equivalently, in the yes case $N > n^{1/2-\epsilon}$.

Therefore an $n^{-1/2+\epsilon}$ -approximation algorithm would route more than one demand in the yes case, since it would return a solution of value at least

$$\text{OPT}(G_N) \cdot n^{-1/2+\epsilon} > 1.$$

In the no case it could route at most one demand. This would decide the NP-complete two-pair directed EDP problem. ■

Undirected graphs. There is an important contrast between the exact and approximate versions of the undirected problem. If the input has k demand pairs and the question is whether all k pairs can be routed by edge-disjoint paths in an undirected graph, then the problem is fixed parameter tractable in k ; in particular, it is polynomial-time solvable for each fixed k . This follows from the Robertson–Seymour algorithm for the fixed-pair disjoint paths problem [225]; the edge-disjoint version can be reduced to the vertex-disjoint version by standard transformations.

This does not imply good approximability when k is part of the input. For MAX-EDP in undirected graphs, the strongest hardness results are much more

recent than the directed reduction above. Chuzhoy, Kim, and Nimavat [88] proved that undirected MAX-EDP is hard to approximate within a reciprocal factor $2^{\Omega(\sqrt{\log n})}$, unless all problems in NP have quasi-polynomial-time algorithms, that is, unless

$$\text{NP} \subseteq \text{DTIME}(n^{O(\log n)}).$$

In the approximation-ratio convention used in these notes, this says that there is no polynomial-time $2^{-\Omega(\sqrt{\log n})}$ -approximation under the same assumption. Their result holds even for subcubic planar graphs with all source vertices on the boundary of one face.

21.3 Gap Amplification for MAX-CLIQUE

Graph products give a clean way to amplify a multiplicative gap for some problems — this can be viewed as a self-reduction. We describe the idea for MAX-CLIQUE. The same discussion also applies to MAX-INDEPENDENT-SET in general graphs: an independent set in G is exactly a clique in the complement graph $\overline{G}$, and hence

$$\alpha(G) = \omega(\overline{G}).$$

Thus MAX-CLIQUE and MAX-INDEPENDENT-SET are equivalent for approximation on unrestricted graphs.

We first define the product for two graphs. Let $G = (V, E)$ and $H = (W, F)$ be undirected graphs. The *strong product* $G \boxtimes H$ has vertex set $V \times W$. Two distinct vertices (u, x) and (v, y) are adjacent in $G \boxtimes H$ if

$$u = v \text{ or } uv \in E, \quad \text{and} \quad x = y \text{ or } xy \in F.$$

Thus adjacency has to hold coordinate by coordinate, where equality is allowed in a coordinate (at least one coordinate has to differ since they are distinct vertices). Figure 21.3 shows the product $K_2 \boxtimes P_3$.

For one graph G , define the t -fold strong power by $G^{\boxtimes 1} = G$ and

$$G^{\boxtimes(t+1)} = G^{\boxtimes t} \boxtimes G.$$

Equivalently, the vertices of $G^{\boxtimes t}$ are t -tuples from V^t , and two distinct tuples $x = (x_1, \dots, x_t)$ and $y = (y_1, \dots, y_t)$ are adjacent if, for every coordinate i , either $x_i = y_i$ or $x_i y_i \in E$.

Lemma 21.2. *For every graph G and integer $t \geq 1$,*

$$\omega(G^{\boxtimes t}) = \omega(G)^t.$$

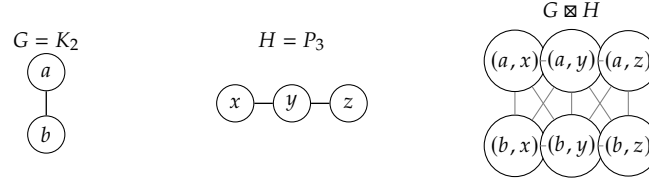


Figure 21.3: The strong product $K_2 \boxtimes P_3$. Horizontal edges come from P_3 , vertical edges come from K_2 , and diagonal edges come from moving in both coordinates at once.

Proof. If C is a clique in G , then C^t is a clique in $G^{\boxtimes t}$, so $\omega(G^{\boxtimes t}) \geq \omega(G)^t$.

Conversely, let Q be a clique in $G^{\boxtimes t}$. For each coordinate i , let Q_i be the set of vertices of G that appear as the i th coordinate of some tuple in Q . The definition of the strong product implies that each Q_i is a clique in G : if two different vertices u, v appear in coordinate i among tuples in Q , the corresponding tuples are adjacent, and hence $uv \in E$. Therefore $|Q_i| \leq \omega(G)$ for every i . Since $Q \subseteq Q_1 \times \cdots \times Q_t$, we have $|Q| \leq \omega(G)^t$. ■

Suppose a reduction produces graphs for which it is **NP**-hard to distinguish

$$\omega(G) \geq a \quad \text{from} \quad \omega(G) \leq b,$$

where $a > b$. Applying the strong power gives a polynomial-time reduction for each fixed t that distinguishes

$$\omega(G^{\boxtimes t}) \geq a^t \quad \text{from} \quad \omega(G^{\boxtimes t}) \leq b^t.$$

Thus a constant-factor gap can be raised to an arbitrarily large constant-factor gap by choosing t to be a sufficiently large constant. The price is that the number of vertices grows from n to n^t .

One can push this deterministic powering further if one is willing to make a stronger complexity assumption. If we choose $t = c \log n$, then the output graph has

$$n^t = 2^{O((\log n)^2)}$$

vertices, so the reduction is quasi-polynomial rather than polynomial. The gap, however, becomes

$$(a/b)^t = n^{\Omega(1)}$$

for a suitable constant c . Thus graph powering alone gives polynomial-factor hardness for **MAX-CLIQUE** under the assumption that **NP** is not contained in quasi-polynomial time, for example

$$\mathbf{NP} \not\subseteq \text{DTIME}(n^{O(\log n)}).$$

This simple powering argument should be compared with the much stronger known hardness for MAX-CLIQUE: for every fixed $\epsilon > 0$, no polynomial-time algorithm can achieve an $n^{-1+\epsilon}$ -approximation for MAX-CLIQUE unless $P = NP$ [144, 262]. Proving such a near-polynomial gap requires PCP machinery and additional amplification ideas. The graph-powering argument above is the clean algebraic core that turns a base gap into a larger one.

21.3.1 Randomized graph products

The deterministic strong product has one drawback: the graph size grows from n to n^t . Randomized graph products reduce this blow-up by sampling from the product graph. This idea was introduced in the early hardness work on MAX-CLIQUE and later derandomized by Alon, Feige, Wigderson, and Zuckerman [12].

Here is the basic construction. Let $G = (V, E)$ be a graph on n vertices, and choose parameters t and M with $M \leq n^t$. Form the full strong power $G^{\boxtimes t}$, whose vertex set is V^t , but keep only a random sample. Let S be a uniformly random M -element subset of V^t , and define

$$R = G^{\boxtimes t}[S],$$

the subgraph induced by S . We call R the sampled product. If $X = (x_1, \dots, x_t)$ and $Y = (y_1, \dots, y_t)$ are sampled tuples, then X and Y are adjacent in R iff, for every coordinate i , either $x_i = y_i$ or $x_i y_i \in E$.

The sampled product is useful when the original reduction gives a promise gap in which the optimum clique size is a constant fraction of the number of vertices:

$$\omega(G) \geq \alpha n \quad \text{versus} \quad \omega(G) \leq \beta n,$$

where $0 < \beta < \alpha \leq 1$ are constants. We use “with high probability” to mean probability tending to one as $n \rightarrow \infty$. The two cases behave as follows.

Lemma 21.3 (Completeness of the sampled product). *If $\omega(G) \geq \alpha n$, then with high probability*

$$\omega(R) \geq \frac{1}{2} \alpha^t M.$$

Proof idea. Let C be a clique in G of size at least αn . Then C^t is a clique in $G^{\boxtimes t}$ of size at least $\alpha^t n^t$. The random set S keeps about an M/n^t fraction of the vertices of C^t , so

$$\mathbb{E}[|S \cap C^t|] = M \cdot \frac{|C^t|}{n^t} \geq \alpha^t M.$$

One can use concentration bounds for sampling without replacement to show that the probability that we will get less than $1/2$ the expected value is small. ■

Lemma 21.4 (Witnesses for product cliques). *If Q is a clique in $G^{\boxtimes t}$, then there are cliques $C_1, \dots, C_t$ in G such that*

$$Q \subseteq C_1 \times C_2 \times \dots \times C_t.$$

In particular, if $\omega(G) \leq \beta n$, then every clique in $G^{\boxtimes t}$ is contained in a product set of size at most $(\beta n)^t$.

Proof. Let C_i be the set of vertices that appear in the i th coordinate of tuples in Q . If $u, v \in C_i$, then they are the i th coordinates of two tuples in Q . Since those tuples are adjacent in the strong product, either $u = v$ or $uv \in E(G)$. Thus C_i is a clique in G , and clearly $Q \subseteq C_1 \times \dots \times C_t$. ■

Lemma 21.5 (Soundness of the sampled product). *Fix constants $0 < \beta < \tau < 1$. There are constants $c_0, c_1 > 0$, depending on β and τ , such that the following holds. Let $t = c_0 \log n$ and $M = n^{c_1}$. If $\omega(G) \leq \beta n$, then with high probability*

$$\omega(R) \leq \tau^t M.$$

Proof idea. By the witness lemma, any clique in R is contained in $S \cap (C_1 \times \dots \times C_t)$ for some choice of cliques $C_1, \dots, C_t$ in G . In the no case, every such product set has density at most β^t inside V^t . For any fixed product set, a concentration bound can be used to show that the probability that the random set S contains more than $\tau^t M$ of its vertices, where $\tau > \beta$, is exponentially small in $\tau^t M$. There are at most 2^{tn} possible witnesses $C_1, \dots, C_t$. Taking $t = c_0 \log n$ and then choosing c_1 sufficiently large makes the union bound over these witnesses go to zero while keeping $M = n^{c_1}$ polynomial. ■

Choosing τ with $\beta < \tau < \alpha$, the two lemmas give a formal gap amplification. In the yes case,

$$\omega(R) \geq \frac{1}{2} \alpha^t M,$$

while in the no case,

$$\omega(R) \leq \tau^t M.$$

Thus the gap is at least

$$\frac{\alpha^t M / 2}{\tau^t M} = \frac{1}{2} \left(\frac{\alpha}{\tau} \right)^t.$$

Since $t = \Theta(\log n)$ and $M = n^{c_1}$, this is a polynomial gap in the size M of the sampled graph:

$$\left(\frac{\alpha}{\tau} \right)^t = M^\gamma$$

for some constant $\gamma > 0$. The randomized product is therefore more powerful than the full strong power for reductions: it keeps a polynomial-size random

induced subgraph of the product while preserving many vertices from the completeness clique and suppressing large soundness cliques with high probability. The derandomized graph product replaces the fully random subset S by a structured expander-walk sample space, removing the need for randomness in the reduction.

Graph products also play a role in hardness for graph coloring, but the details are more delicate because the completeness case must preserve small colorability; see the derandomized product ideas of Alon, Feige, Wigderson, and Zuckerman [12] and later coloring hardness work for precise statements.

21.4 APX-Hardness of MAX-3-SAT from PCP Theorem

We now turn to the source of constant-factor hardness for many optimization problems. The class APX consists of optimization problems that admit a polynomial-time constant-factor approximation. A problem is APX-hard if every problem in APX has an approximation-preserving reduction to it. In particular, if an APX-hard problem has a PTAS, then every problem in APX has a PTAS; for the problems here this would imply $P = NP$. The gap reductions below show the concrete consequence we need: a PTAS for MAX-3-SAT would imply $P = NP$.

The PCP theorem is best thought of as saying that witnesses for **NP** statements can be rewritten in a special proof format. The verifier does not read the whole proof. Instead it tosses a logarithmic number of random bits, looks at only constantly many locations of this rewritten proof, and accepts or rejects according to a constant-size local test.

Let $PCP(O(\log n), O(1))$ denote the class of languages L with the following kind of verifier. There are constants c and q and a polynomial-time verifier V such that, on input x , the verifier receives a proof string y , uses a random string r of length $c \log |x|$, queries at most q bits of y , where $q = O(1)$, and then accepts or rejects. The verifier has completeness 1 and soundness error $< 1/2$:

1. if $x \in L$, then there is a proof y that makes V accept with probability 1 over the choice of r ;
2. if $x \notin L$, then every proof y makes V accept with probability $< 1/2$ over the choice of r .

Theorem 21.4 (PCP theorem [20]). $NP = PCP(O(\log n), O(1))$.

We will only use the theorem for $L = \text{SAT}$. Thus there is a verifier for SAT with constants c and q as above. This is much stronger than the elementary verifier that chooses one random clause and checks the three variables in that clause: that verifier has error roughly $1 - 1/m$ on an unsatisfiable formula with

m clauses. The PCP theorem gives a different proof format in which the error is bounded away from 1, even though the verifier still reads only constantly many proof bits.

We first turn this verifier into a constraint satisfaction problem. For a fixed constant q , an instance of Boolean MAX- q -CSP has Boolean variables and constraints, where each constraint is an arbitrary Boolean predicate on at most q variables. The objective is to maximize the number of satisfied constraints.

Let Φ be a SAT formula of length n . The PCP theorem gives us a particular polynomial-time verifier V for SAT; it is part of the reduction, not an object we search for. On input Φ , the verifier uses $c \log n$ random bits. Hence there are only n^c possible random strings, and the reduction can enumerate all of them in polynomial time. We may also assume, after increasing the constant c if necessary, that the relevant proof has length at most n^c . Indeed, over all random strings the verifier makes at most qn^c queries, so only polynomially many proof positions can ever be seen; all other positions are irrelevant and the queried positions can be relabeled.

Fix one random string r . Once Φ and r are fixed, the verifier decides which proof locations it will inspect, at most q of them, and it accepts or rejects as a Boolean function of the bits in those locations. Call this accepting predicate f_r . The CSP variables are the proof bits $y_1, \dots, y_N$, where $N \leq n^c$. For each random string r , add the constraint f_r on the proof bits queried by $V(\Phi, \cdot; r)$. Thus the CSP is asking whether there is a proof string y of length at most n^c such that the verifier accepts Φ for the corresponding random strings. More precisely, the value of the CSP assignment y is exactly the number of random strings r on which V accepts when it is given proof oracle y .

This gives a Boolean MAX- q -CSP instance with n^c constraints. The key point is that $q = O(1)$: the arity of every constraint is bounded by an absolute constant independent of Φ .

Lemma 21.6. *There is a constant $\eta > 0$ and a polynomial-time reduction from SAT to Boolean MAX- q -CSP, for some constant q , that maps a formula Φ to a CSP instance I_Φ such that: if Φ is satisfiable, then all constraints of I_Φ can be satisfied; if Φ is not satisfiable, then every assignment satisfies at most a $(1 - \eta)$ fraction of the constraints.*

Proof. Use the construction above. If Φ is satisfiable, completeness of the PCP verifier gives a proof string that makes V accept for every random string r . The corresponding assignment to the CSP variables satisfies every constraint f_r .

If Φ is not satisfiable, soundness says that every proof string makes V accept with probability less than $1/2$ over the random string r . Therefore every assignment to the CSP variables satisfies fewer than half of the constraints. The lemma follows with, for instance, $\eta = 1/2$. ■

We now convert this fixed-arity CSP gap to a MAX-3-SAT gap. Consider one CSP constraint f on at most q variables. Since q is constant, we may write down the whole truth table of f . For every assignment on which f evaluates to 0, add one clause that forbids exactly that assignment. The conjunction of these clauses is a CNF formula ψ_f equivalent to f , and it has at most 2^q clauses, each of width at most q .

If a CSP assignment satisfies f , then it satisfies all clauses of ψ_f . If it does not satisfy f , then it violates at least one clause of ψ_f . Thus replacing every CSP constraint by its CNF formula preserves perfect completeness, and in the soundness case each violated CSP constraint creates at least one violated CNF clause.

The clauses in ψ_f may have width larger than 3, but their width is at most the constant q . Replace each such clause by the standard chain of 3-clauses using auxiliary variables. A satisfied long clause can be extended, by choosing the auxiliary variables appropriately, to satisfy all clauses in the chain; an unsatisfied long clause leaves at least one chain clause unsatisfied no matter how the auxiliary variables are chosen. Clauses of width one or two can be padded in the usual way, or simply allowed if one uses the convention that 3-CNF means width at most three. Since q is constant, each original CSP constraint produces at most a constant number, say B , of 3-clauses.

Combining the two steps, a satisfiable SAT instance produces a satisfiable 3-CNF formula. If the original SAT instance is unsatisfiable, then at least an η fraction of the CSP constraints are violated by every assignment, and hence at least an η/B fraction of the resulting 3-CNF clauses are violated by every assignment.

Corollary 21.5. *There is a constant $\delta > 0$ such that it is NP-hard to distinguish satisfiable 3-CNF formulas from 3-CNF formulas in which at most a $(1 - \delta)$ fraction of clauses can be satisfied. Consequently, unless $P = NP$, MAX-3-SAT has no PTAS.*

Standard approximation-preserving reductions strengthen this statement to the usual terminology: MAX-3-SAT is APX-hard. This conclusion is much weaker than the sharp inapproximability threshold known for MAX-3-SAT, but it captures the main conceptual point: the PCP theorem turns ordinary NP-hardness into a constant multiplicative gap.

21.4.1 From MAX-3-SAT to MAX-INDEPENDENT-SET

We next use the standard reduction from 3-CNF satisfiability to INDEPENDENT-SET, due to Karp [171], and observe that it preserves the number of clauses that can be satisfied. Let Φ be a 3-CNF formula with m clauses. For simplicity assume

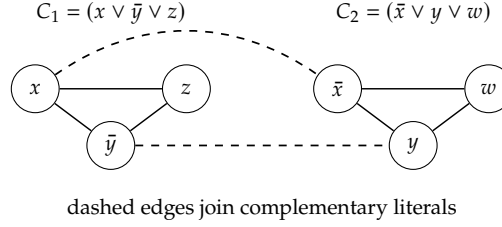


Figure 21.4: The standard reduction from 3-CNF formulas to INDEPENDENT-SET: each clause becomes a triangle, and complementary literals in different clauses are joined by consistency edges.

that each clause has exactly three literals; shorter clauses can be padded, or handled by using a smaller clique.

Build the graph H_Φ as follows. For each literal occurrence in each clause, create one vertex. For each clause, make its three vertices pairwise adjacent; equivalently, create a triangle for each clause. These edges ensure that an independent set chooses at most one literal from any clause. Finally, add consistency edges between vertices from different clauses when they correspond to complementary literals, say x and $\bar{x}$. Figure 21.4 shows the construction on a two-clause example.

Lemma 21.7. *There is a constant $\delta > 0$ and a polynomial-time reduction from SAT to MAX-INDEPENDENT-SET that maps a SAT instance I to a graph H and an integer M such that, if I is satisfiable, then $\alpha(H) = M$, while if I is not satisfiable, then $\alpha(H) \leq (1 - \delta)M$. Moreover, $|V(H)| = 3M$.*

Proof. Apply the PCP-based reduction from SAT to MAX-3-SAT from the preceding subsection. It produces a 3-CNF formula Φ with M clauses such that, in the YES case, all M clauses of Φ can be satisfied, while in the NO case, at most $(1 - \delta)M$ clauses can be satisfied.

Now build the graph H_Φ described above and illustrated in Figure 21.4. An independent set in H_Φ chooses at most one literal from each clause, and the consistency edges ensure that it never chooses both x and $\bar{x}$. Hence an independent set of size s can be extended to a truth assignment satisfying at least the s clauses from which those literals were chosen: set the chosen literals to true and assign all remaining variables arbitrarily. Conversely, any assignment satisfying s clauses gives an independent set of size s , by choosing one true literal from each satisfied clause. Thus

$$\alpha(H_\Phi) = \text{OPT}(\Phi),$$

where $\text{OPT}(\Phi)$ is the maximum number of satisfiable clauses. The claimed gap follows, and H_Φ has exactly three vertices for each clause of Φ . ■

Equivalently, by taking complements, the same reduction gives a constant multiplicative gap for MAX-CLIQUE: the clique number is M in the YES case and at most $(1 - \delta)M$ in the NO case. The randomized graph products from the previous section can then amplify this constant gap while keeping the graph size polynomial, yielding hardness for some polynomially small approximation ratio for MAX-CLIQUE, and hence also for MAX-INDEPENDENT-SET on general graphs. Equivalently, in the reciprocal terminology often used for clique hardness, this rules out approximation within a polynomial factor.

Bibliography

- [1] Ittai Abraham, Yair Bartal, and Ofer Neiman. “Nearly tight low stretch spanning trees”. In: *2008 49th Annual IEEE Symposium on Foundations of Computer Science*. IEEE. 2008, pp. 781–790.
- [2] Ittai Abraham, Cyril Gavoille, Anupam Gupta, Ofer Neiman, and Kunal Talwar. “Cops, robbers, and threatening skeletons: Padded decomposition for minor-free graphs”. In: *SIAM Journal on Computing* 48.3 (2019), pp. 1120–1145.
- [3] Anna Adamaszek, Parinya Chalermsook, Alina Ene, and Andreas Wiese. “Submodular unsplittable flow on trees”. In: *International Conference on Integer Programming and Combinatorial Optimization*. Springer. 2016, pp. 337–349.
- [4] Anna Adamaszek, Sariel Har-Peled, and Andreas Wiese. “Approximation schemes for independent set and sparse subsets of polygons”. In: *Journal of the ACM (JACM)* 66.4 (2019), pp. 1–40.
- [5] Amit Agarwal, Noga Alon, and Moses S Charikar. “Improved approximation for directed cut problems”. In: *Proceedings of the thirty-ninth annual ACM symposium on Theory of computing*. 2007, pp. 671–680.
- [6] Amit Agarwal, Moses Charikar, Konstantin Makarychev, and Yury Makarychev. “ $O(\sqrt{\log n})$ approximation algorithms for Min UnCut, Min 2CNF Deletion, and directed cut problems”. In: *Proceedings of the thirty-seventh annual ACM symposium on Theory of computing*. 2005, pp. 573–581.
- [7] Ankit Aggarwal, Amit Deshpande, and Ravi Kannan. “Adaptive sampling for k-means clustering”. In: *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques*. Springer, 2009, pp. 15–28.
- [8] Ajit Agrawal, Philip Klein, and Ramamoorthi Ravi. “When trees collide: An approximation algorithm for the generalized Steiner problem on networks”. In: *SIAM journal on Computing* 24.3 (1995), pp. 440–456.

- [9] Ali Ahmadi, Iman Gholami, MohammadTaghi Hajiaghayi, Peyman Jabbarzade, and Mohammad Mahdavi. "Breaking a Long-Standing Barrier: $2 - \epsilon$ Approximation for Steiner Forest". In: *66th IEEE Annual Symposium on Foundations of Computer Science (FOCS 2025)*. IEEE, 2025, pp. 373–444. doi: 10.1109/FOCS63196.2025.00023.
- [10] Nir Ailon, Ragesh Jaiswal, and Claire Monteleoni. "Streaming k-means approximation." In: *NIPS*. Vol. 22. 2009, pp. 10–18.
- [11] Karhan Akcoglu, James Aspnes, Bhaskar DasGupta, and Ming-Yang Kao. "Opportunity cost algorithms for combinatorial auctions". In: *Computational Methods in Decision-Making, Economics and Finance*. Springer, 2002, pp. 455–479.
- [12] Noga Alon, Uriel Feige, Avi Wigderson, and David Zuckerman. "Derandomized graph products". In: *Computational Complexity 5.1* (1995), pp. 60–75. doi: 10.1007/BF01277956.
- [13] Noga Alon, Richard M Karp, David Peleg, and Douglas West. "A graph-theoretic game and its application to the k-server problem". In: *SIAM Journal on Computing* 24.1 (1995), pp. 78–100.
- [14] Matthew Andrews, Julia Chuzhoy, Venkatesan Guruswami, Sanjeev Khanna, Kunal Talwar, and Lisa Zhang. "Inapproximability of edge-disjoint paths and low congestion routing on undirected graphs". In: *Combinatorica* 30.5 (2010), pp. 485–520.
- [15] Kenneth Appel, Wolfgang Haken, et al. "Every planar map is four colorable. Part I: Discharging". In: *Illinois Journal of Mathematics* 21.3 (1977), pp. 429–490.
- [16] Sanjeev Arora. "Polynomial time approximation schemes for Euclidean traveling salesman and other geometric problems". In: *Journal of the ACM (JACM)* 45.5 (1998), pp. 753–782.
- [17] Sanjeev Arora, James R Lee, and Assaf Naor. "Euclidean distortion and the sparsest cut". In: *Proceedings of the thirty-seventh annual ACM symposium on Theory of computing*. 2005, pp. 553–562.
- [18] Sanjeev Arora, James R Lee, and Assaf Naor. "Euclidean distortion and the sparsest cut". In: *Journal of the AMS* 21.1 (2008), pp. 1–21.
- [19] Sanjeev Arora, James R Lee, and Assaf Naor. "Fréchet embeddings of negative type metrics". In: *Discrete & Computational Geometry* 38.4 (2007), pp. 726–739.

- [20] Sanjeev Arora, Carsten Lund, Rajeev Motwani, Madhu Sudan, and Mario Szegedy. "Proof verification and the hardness of approximation problems". In: *Journal of the ACM* 45.3 (1998), pp. 501–555. doi: 10.1145/278298.278306.
- [21] Sanjeev Arora, Satish Rao, and Umesh Vazirani. "Expander flows, geometric embeddings and graph partitioning". In: *Journal of the ACM (JACM)* 56.2 (2009), pp. 1–37.
- [22] Sanjeev Arora, Satish Rao, and Umesh Vazirani. "Expander flows, geometric embeddings and graph partitioning". In: *Journal of the ACM* 56.2 (2009), pp. 1–37.
- [23] Emil Artin. "Über die Zerlegung definiter Funktionen in Quadrate". In: *Abhandlungen aus dem Mathematischen Seminar der Universität Hamburg* 5 (1927), pp. 100–115.
- [24] Vijay Arya, Naveen Garg, Rohit Khandekar, Adam Meyerson, Kamesh Munagala, and Vinayaka Pandit. "Local search heuristics for k-median and facility location problems". In: *SIAM Journal on computing* 33.3 (2004), pp. 544–562.
- [25] Arash Asadpour, Michel X Goemans, Aleksander Mądry, Shayan Oveis Gharan, and Amin Saberi. "An $O(\log n / \log \log n)$ -approximation algorithm for the asymmetric traveling salesman problem". In: *Operations Research* 65.4 (2017). Preliminary version in Proc. of ACM-SIAM SODA, 2010., pp. 1043–1061.
- [26] Yonatan Aumann and Yuval Rabani. "An $O(\log k)$ approximate min-cut max-flow theorem and approximation algorithm". In: *SIAM Journal on Computing* 27.1 (1998), pp. 291–301.
- [27] Pranjal Awasthi, Moses Charikar, Ravishankar Krishnaswamy, and Ali Kemal Sinop. "The Hardness of Approximation of Euclidean k-Means". In: *31st International Symposium on Computational Geometry (SoCG 2015)*. Ed. by Lars Arge and János Pach. Vol. 34. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2015, pp. 754–767. ISBN: 978-3-939897-83-5. doi: 10.4230/LIPIcs.SOCG.2015.754. URL: <http://drops.dagstuhl.de/opus/volltexte/2015/5117>.
- [28] Baruch Awerbuch, Yossi Azar, and Yair Bartal. "On-line generalized Steiner problem". In: *Theoretical Computer Science* 324.2-3 (2004), pp. 313–324.

- [29] Bahman Bahmani, Benjamin Moseley, Andrea Vattani, Ravi Kumar, and Sergei Vassilvitskii. “Scalable K-Means++”. In: *Proc. VLDB Endow.* 5.7 (Mar. 2012), pp. 622–633. ISSN: 2150-8097. DOI: 10.14778/2180912.2180915. URL: <https://doi.org/10.14778/2180912.2180915>.
- [30] Tanvi Bajpai, Deeparnab Chakrabarty, Chandra Chekuri, and Maryam Negahbani. “Revisiting Priority k -Center: Fairness and Outliers”. In: *arXiv preprint arXiv:2103.03337* (2021).
- [31] Brenda S Baker. “Approximation algorithms for NP-complete problems on planar graphs”. In: *Journal of the ACM (JACM)* 41.1 (1994), pp. 153–180.
- [32] Étienne Bamas, Marina Drygala, and Andreas Maggiori. “An Improved Analysis of Greedy for Online Steiner Forest”. In: *Proceedings of the 2022 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*. SIAM, 2022, pp. 3202–3229. DOI: 10.1137/1.9781611977073.125.
- [33] Nikhil Bansal, Neng Huang, and Euiwoong Lee. “Improved SDP-Based Algorithm for Coloring 3-Colorable Graphs”. In: *arXiv preprint arXiv:2602.05904* (2026).
- [34] Nikhil Bansal, Nitish Korula, Viswanath Nagarajan, and Aravind Srinivasan. “Solving packing integer programs via randomized rounding with alterations”. In: *Theory of Computing* 8.1 (2012), pp. 533–565.
- [35] Reuven Bar-Yehuda and Shimon Even. “A linear-time approximation algorithm for the weighted vertex cover problem”. In: *Journal of Algorithms* 2.2 (1981), pp. 198–203.
- [36] Boaz Barak, Prasad Raghavendra, and David Steurer. “Rounding semidefinite programming hierarchies via global correlation”. In: *Proceedings of the 52nd Annual IEEE Symposium on Foundations of Computer Science (FOCS 2011)*. IEEE, 2011, pp. 472–481. DOI: 10.1109/FOCS.2011.95.
- [37] Yair Bartal. “On approximating arbitrary metrics by tree metrics”. In: *Proceedings of the thirtieth annual ACM symposium on Theory of computing*. 1998, pp. 161–168.
- [38] Yair Bartal. “Probabilistic approximation of metric spaces and its algorithmic applications”. In: *Proceedings of 37th Conference on Foundations of Computer Science*. IEEE, 1996, pp. 184–193.
- [39] Yair Bartal, Moses Charikar, and Danny Raz. “Approximating Min-Sum k -Clustering in Metric Spaces”. In: *Proceedings of the Thirty-Third Annual ACM Symposium on Theory of Computing*. STOC ’01. Hersonissos, Greece: Association for Computing Machinery, 2001, pp. 11–20. ISBN: 1581133499. DOI: 10.1145/380752.380754. URL: <https://doi.org/10.1145/380752.380754>.

- [40] Kristóf Bérczi, Karthekeyan Chandrasekaran, Tamás Király, and Vivek Madan. “Improving the integrality gap for multiway cut”. In: *Mathematical Programming* 183.1 (2020), pp. 171–193.
- [41] Marshall Bern and Paul Plassmann. “The Steiner problem with edge lengths 1 and 2”. In: *Information Processing Letters* 32.4 (1989), pp. 171–176.
- [42] Guy E Blelloch, Yan Gu, and Yihan Sun. “Efficient construction of probabilistic tree embeddings”. In: *arXiv preprint arXiv:1605.04651* (2016).
- [43] Glencora Borradaile, Philip Klein, and Claire Mathieu. “An $O(n \log n)$ approximation scheme for Steiner tree in planar graphs”. In: *ACM Transactions on Algorithms (TALG)* 5.3 (2009), pp. 1–31.
- [44] Jean Bourgain. “On Lipschitz embedding of finite metric spaces in Hilbert space”. In: *Israel Journal of Mathematics* 52.1–2 (1985), pp. 46–52.
- [45] Joshua Brakensiek, Neng Huang, Aaron Potechin, and Uri Zwick. “Improved Approximation Algorithms for Multiway Cut by Large Mixtures of New and Old Rounding Schemes”. In: *Proceedings of the 58th Annual ACM Symposium on Theory of Computing (STOC 2026)*. Salt Lake City, UT, USA: ACM, 2026.
- [46] Joshua Brakensiek, Neng Huang, and Uri Zwick. “Tight Approximability of MAX 2-SAT and Relatives, under UGC”. In: *Proceedings of the 35th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2024)*. SIAM, 2024, pp. 1328–1344. doi: 10.1137/1.9781611977912.53.
- [47] Simon Bruggmann and Rico Zenklusen. “Submodular maximization through the lens of linear programming”. In: *Mathematics of Operations Research* 44.4 (2019), pp. 1221–1244.
- [48] Niv Buchbinder and Moran Feldman. “Submodular functions maximization problems”. In: *Handbook of Approximation Algorithms and Metaheuristics, Second Edition*. Chapman and Hall/CRC, 2018, pp. 753–788.
- [49] Jarosław Byrka, Fabrizio Grandoni, Thomas Rothvoß, and Laura Sanita. “An improved LP-based approximation for Steiner tree”. In: *Proceedings of the forty-second ACM symposium on Theory of computing*. 2010, pp. 583–592.
- [50] Jarosław Byrka, Fabrizio Grandoni, and Vera Traub. “The Bidirected Cut Relaxation for Steiner Tree has Integrality Gap Smaller than 2”. In: *2024 IEEE 65th Annual Symposium on Foundations of Computer Science (FOCS)*. 2024, pp. 730–753. doi: 10.1109/FOCS61266.2024.00052.
- [51] G. Calinescu, H. Karloff, and Y. Rabani. “Approximation algorithms for the 0-extension problem”. In: *Proceedings of the twelfth annual ACM-SIAM symposium on Discrete algorithms*. Society for Industrial and Applied Mathematics Philadelphia, PA, USA. 2001, pp. 8–16.

- [52] Gruia Calinescu, Chandra Chekuri, Martin Pal, and Jan Vondrák. “Maximizing a monotone submodular function subject to a matroid constraint”. In: *SIAM Journal on Computing* 40.6 (2011), pp. 1740–1766.
- [53] Gruia Călinescu, Howard J. Karloff, and Yuval Rabani. “An Improved Approximation Algorithm for Multiway Cut”. In: *Proceedings of the 30th Annual ACM Symposium on Theory of Computing (STOC)*. Journal version in *Journal of Computer and System Sciences*, 60(3):564–574, 2000. 1998, pp. 48–52.
- [54] Robert Carr and Santosh Vempala. “Randomized metarounding”. In: *Proceedings of the thirty-second annual ACM symposium on Theory of computing*. 2000, pp. 58–62.
- [55] Amit Chakrabarti, Chandra Chekuri, Anupam Gupta, and Amit Kumar. “Approximation algorithms for the unsplittable flow problem”. In: *Algorithmica* 47.1 (2007), pp. 53–78.
- [56] Deeparnab Chakrabarty, Nikhil R. Devanur, and Vijay V. Vazirani. “New geometry-inspired relaxations and algorithms for the metric Steiner tree problem”. In: *Integer Programming and Combinatorial Optimization*. 2008, pp. 344–358. doi: 10.1007/978-3-540-68891-4_24.
- [57] Deeparnab Chakrabarty and Maryam Negahbani. “Generalized center problems with outliers”. In: *ACM Transactions on Algorithms (TALG)* 15.3 (2019), pp. 1–14.
- [58] Siu On Chan, James R. Lee, Prasad Raghavendra, and David Steurer. “Approximate constraint satisfaction requires large LP relaxations”. In: *Journal of the ACM* 63.4 (2016), 34:1–34:22. doi: 10.1145/2811255.
- [59] Timothy M Chan. “Approximation schemes for 0-1 knapsack”. In: *1st Symposium on Simplicity in Algorithms (SOSA 2018)*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik. 2018.
- [60] Ashok K Chandra, Daniel S. Hirschberg, and Chak-Kuen Wong. “Approximate algorithms for some generalized knapsack problems”. In: *Theoretical Computer Science* 3.3 (1976), pp. 293–304.
- [61] Karthekeyan Chandrasekaran and Weihang Wang. “Approximating Submodular-Partition via Principal Partition Sequence”. In: *SIAM Journal on Discrete Mathematics* 38.4 (2024), pp. 3198–3219.
- [62] Moses Charikar, Chandra Chekuri, To-Yat Cheung, Zuo Dai, Ashish Goel, Sudipto Guha, and Ming Li. “Approximation algorithms for directed Steiner problems”. In: *Journal of Algorithms* 33.1 (1999), pp. 73–91.

- [63] Moses Charikar, Chandra Chekuri, Ashish Goel, and Sudipto Guha. "Rounding via trees: deterministic approximation algorithms for group Steiner trees and k-median". In: *Proceedings of the thirtieth annual ACM symposium on Theory of computing*. 1998, pp. 114–123.
- [64] Moses Charikar, Chandra Chekuri, Ashish Goel, Sudipto Guha, and Serge Plotkin. "Approximating a finite metric by a small number of tree metrics". In: *Proceedings 39th Annual Symposium on Foundations of Computer Science (Cat. No. 98CB36280)*. IEEE. 1998, pp. 379–388.
- [65] Moses Charikar, Sudipto Guha, Éva Tardos, and David B Shmoys. "A constant-factor approximation algorithm for the k-median problem". In: *Journal of Computer and System Sciences* 65.1 (2002), pp. 129–149.
- [66] Moses Charikar, Konstantin Makarychev, and Yury Makarychev. "Directed metrics and directed graph partitioning problems". In: *SODA*. Vol. 6. 2006, pp. 51–60.
- [67] Shuchi Chawla, Anupam Gupta, and Harald Räcke. "Embeddings of negative-type metrics and an improved approximation to generalized sparsest cut". In: *ACM Transactions on Algorithms (TALG)* 4.2 (2008), pp. 1–18.
- [68] Shuchi Chawla, Robert Krauthgamer, Ravi Kumar, Yuval Rabani, and D Sivakumar. "On the hardness of approximating multicut and sparsest-cut". In: *computational complexity* 15.2 (2006), pp. 94–114.
- [69] Chandra Chekuri and Alina Ene. "Approximation Algorithms for Submodular Multiway Partition". In: *Proceedings of the 52nd Annual IEEE Symposium on Foundations of Computer Science (FOCS)*. 2011, pp. 807–816.
- [70] Chandra Chekuri, Sudipto Guha, and Joseph Naor. "The Steiner k-cut problem". In: *SIAM Journal on Discrete Mathematics* 20.1 (2006), pp. 261–271.
- [71] Chandra Chekuri, Sreeram Kannan, Adnan Raja, and Pramod Viswanath. "Multicommodity flows and cuts in polymatroidal networks". In: *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference*. 2012, pp. 399–408.
- [72] Chandra Chekuri, Sreeram Kannan, Adnan Raja, and Pramod Viswanath. "Multicommodity flows and cuts in polymatroidal networks". In: *SIAM Journal on Computing* 44.4 (2015), pp. 912–943.
- [73] Chandra Chekuri and Sanjeev Khanna. "A polynomial time approximation scheme for the multiple knapsack problem". In: *SIAM Journal on Computing* 35.3 (2005), pp. 713–728.

- [74] Chandra Chekuri, Sanjeev Khanna, and F. Bruce Shepherd. "The all-or-nothing multicommodity flow problem". In: *SIAM Journal on Computing* 42.4 (2013), pp. 1467–1493.
- [75] Chandra Chekuri and Shi Li. "On the hardness of approximating the k-way hypergraph cut problem". In: *Theory of Computing* 16.1 (2020).
- [76] Chandra Chekuri and Vivek Madan. "Approximating multicut and the demand graph". In: *Proceedings of the Twenty-Eighth Annual ACM-SIAM Symposium on Discrete Algorithms*. SIAM. 2017, pp. 855–874.
- [77] Chandra Chekuri and Vivek Madan. "Simple and Fast Rounding Algorithms for Directed and Node-weighted Multiway Cut". In: *Proceedings of the 27th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*. 2016, pp. 797–808.
- [78] Chandra Chekuri and Kent Quanrud. "Independent sets in elimination graphs with a submodular objective". In: *arXiv preprint arXiv:2307.02022* (2023).
- [79] Chandra Chekuri and Kent Quanrud. "On approximating (sparse) covering integer programs". In: *Proceedings of the Thirtieth Annual ACM-SIAM Symposium on Discrete Algorithms*. SIAM. 2019, pp. 1596–1615.
- [80] Chandra Chekuri and Thapanapong Rukkanchanunt. "A note on iterated rounding for the Survivable Network Design Problem". In: *1st Symposium on Simplicity in Algorithms (SOSA 2018)*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik. 2018.
- [81] Chandra Chekuri, Jan Vondrák, and Rico Zenklusen. "Submodular function maximization via the multilinear relaxation and contention resolution schemes". In: *SIAM Journal on Computing* 43.6 (2014), pp. 1831–1879.
- [82] Antares Chen, Lorenzo Orecchia, and Erasmo Tani. "Submodular Hypergraph Partitioning: Metric Relaxations and Fast Algorithms via an Improved Cut-Matching Game". In: *arXiv preprint arXiv:2301.08920* (2023).
- [83] Kevin KH Cheung, William H Cunningham, and Lawrence Tang. "Optimal 3-terminal cuts and linear programming". In: *Mathematical Programming* 106.1 (2006), pp. 1–23.
- [84] Miroslav Chlebik and Janka Chlebíková. "Approximation hardness of the Steiner tree problem on graphs". In: *Scandinavian Workshop on Algorithm Theory*. Springer. 2002, pp. 170–179.
- [85] Nicos Christofides. *Worst-case analysis of a new heuristic for the traveling salesman problem*. Tech. rep. Carnegie-Mellon Univ Pittsburgh Pa Management Sciences Research Group, 1976.

- [86] Julia Chuzhoy, Venkatesan Guruswami, Sanjeev Khanna, and Kunal Talwar. "Hardness of Routing with Congestion in Directed Graphs". In: *Proceedings of the Thirty-Ninth Annual ACM Symposium on Theory of Computing*. STOC '07. San Diego, California, USA: Association for Computing Machinery, 2007, pp. 165–178. ISBN: 9781595936318. DOI: 10.1145/1250790.1250816. URL: <https://doi.org/10.1145/1250790.1250816>.
- [87] Julia Chuzhoy and Sanjeev Khanna. "Polynomial flow-cut gaps and hardness of directed cut problems". In: *Journal of the ACM (JACM)* 56.2 (2009), pp. 1–28.
- [88] Julia Chuzhoy, David H. K. Kim, and Rachit Nimavat. "New hardness results for routing on disjoint paths". In: *SIAM Journal on Computing* 51.2 (2022), STOC17–189–STOC17–237. DOI: 10.1137/17M1146580.
- [89] Elias Dahlhaus, David S. Johnson, Christos H. Papadimitriou, Paul D. Seymour, and Mihalis Yannakakis. "The complexity of multiterminal cuts". In: *SIAM Journal on Computing* 23.4 (1994), pp. 864–894.
- [90] George Dantzig, Ray Fulkerson, and Selmer Johnson. "Solution of a large-scale traveling-salesman problem". In: *Journal of the operations research society of America* 2.4 (1954), pp. 393–410.
- [91] W Fernandez De La Vega and George S. Lueker. "Bin packing can be solved within $(1 + \varepsilon)$ in linear time". In: *Combinatorica* 1.4 (1981), pp. 349–355.
- [92] Yefim Dinitz, Naveen Garg, and Michel X Goemans. "On the single-source unsplittable flow problem". In: *Combinatorica* 19.1 (1999), pp. 17–41.
- [93] Irit Dinur and Samuel Safra. "On the hardness of approximating minimum vertex cover". In: *Annals of mathematics* (2005), pp. 439–485.
- [94] D-Z Du and Frank K. Hwang. "A proof of the Gilbert-Pollak conjecture on the Steiner ratio". In: *Algorithmica* 7.1 (1992), pp. 121–135.
- [95] Michael Elkin, Yuval Emek, Daniel A Spielman, and Shang-Hua Teng. "Lower-stretch spanning trees". In: *Proceedings of the thirty-seventh annual ACM symposium on Theory of computing*. 2005, pp. 494–503.
- [96] Alina Ene and Huy L Nguyễn. "From Graph to Hypergraph Multiway Partition: Is the Single Threshold the Only Route?" In: *European Symposium on Algorithms*. Springer. 2014, pp. 382–393.
- [97] Alina Ene, Jan Vondrák, and Yi Wu. "Local distribution and the symmetry gap: Approximability of multiway partitioning problems". In: *Proceedings of the Twenty-Fourth Annual ACM-SIAM Symposium on Discrete Algorithms*. SIAM. 2013, pp. 306–325.

- [98] Guy Even, Joseph Seffi Naor, Satish Rao, and Baruch Schieber. "Divide-and-conquer approximation algorithms via spreading metrics". In: *Journal of the ACM (JACM)* 47.4 (2000), pp. 585–616.
- [99] Guy Even, Joseph Seffi Naor, Satish Rao, and Baruch Schieber. "Divide-and-conquer approximation algorithms via spreading metrics". In: *Journal of the ACM (JACM)* 47.4 (2000), pp. 585–616.
- [100] Jittat Fakcharoenphol, Satish Rao, and Kunal Talwar. "A tight bound on approximating arbitrary metrics by tree metrics". In: *Proceedings of the thirty-fifth annual ACM symposium on Theory of computing*. 2003, pp. 448–455.
- [101] Jittat Fakcharoenphol, Satish Rao, and Kunal Talwar. "A tight bound on approximating arbitrary metrics by tree metrics". In: *Journal of Computer and System Sciences* 69.3 (2004), pp. 485–497.
- [102] Tomás Feder and Daniel Greene. "Optimal algorithms for approximate clustering". In: *Proceedings of the twentieth annual ACM symposium on Theory of computing*. 1988, pp. 434–444.
- [103] Uriel Feige. "A threshold of $\ln n$ for approximating set cover". In: *Journal of the ACM (JACM)* 45.4 (1998), pp. 634–652.
- [104] Uriel Feige, MohammadTaghi Hajiaghayi, and James R Lee. "Improved approximation algorithms for minimum-weight vertex separators". In: *Proceedings of the thirty-seventh annual ACM symposium on Theory of computing*. 2005, pp. 563–572.
- [105] Uriel Feige, Michael Langberg, and Gideon Schechtman. "Graphs with Tiny Vector Chromatic Numbers and Huge Chromatic Numbers". In: *SIAM Journal on Computing* 33.6 (2004), pp. 1338–1368. doi: 10.1137/S0097539703431391.
- [106] Uriel Feige, Vahab S Mirrokni, and Jan Vondrak. "Maximizing non-monotone submodular functions". In: *SIAM Journal on Computing* 40.4 (2011), pp. 1133–1153.
- [107] Uriel Feige and Gideon Schechtman. "On the optimality of the random hyperplane rounding technique for MAX CUT". In: *Random Structures & Algorithms* 20.3 (2002), pp. 403–440. doi: <https://doi.org/10.1002/rsa.10036>. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/rsa.10036>. URL: <https://onlinelibrary.wiley.com/doi/abs/10.1002/rsa.10036>.

- [108] Uriel Feige and Jan Vondrak. "Approximation algorithms for allocation problems: Improving the factor of $1-1/e$ ". In: *2006 47th Annual IEEE Symposium on Foundations of Computer Science (FOCS'06)*. IEEE. 2006, pp. 667–676.
- [109] Moran Feldman, Joseph Seffi Naor, Roy Schwartz, and Justin Ward. "Improved approximations for k-exchange systems". In: *European Symposium on Algorithms*. Springer. 2011, pp. 784–798.
- [110] Moran Feldman, Joseph Seffi Naor, Roy Schwartz, and Justin Ward. "Improved approximations for k-exchange systems". In: *European Symposium on Algorithms*. Springer. 2011, pp. 784–798.
- [111] Arnold Filtser. *On Sparse Covers of Minor Free Graphs, Low Dimensional Metric Embeddings, and other applications*. 2024. arXiv: 2401.14060 [cs.DS]. URL: <https://arxiv.org/abs/2401.14060>.
- [112] Marshall L Fisher, George L Nemhauser, and Laurence A Wolsey. "An analysis of approximations for maximizing submodular set functions II". In: *Polyhedral combinatorics* (1978), pp. 73–87.
- [113] Lisa Fleischer, Michel X Goemans, Vahab S Mirrokni, and Maxim Sviridenko. "Tight approximation algorithms for maximum separable assignment problems". In: *Mathematics of Operations Research* 36.3 (2011). Preliminary version in Proc. of ACM-SIAM SODA 2006., pp. 416–431.
- [114] Steven Fortune, John Hopcroft, and James Wyllie. "The directed subgraph homeomorphism problem". In: *Theoretical Computer Science* 10.2 (1980), pp. 111–121. doi: 10.1016/0304-3975(80)90009-2.
- [115] Waldo Gálvez, Arindam Khan, Mathieu Mari, Tobias Mömke, Madhusudhan Reddy, and Andreas Wiese. "A $(2+\epsilon)$ -Approximation Algorithm for Maximum Independent Set of Rectangles". In: *arXiv preprint arXiv:2106.00623* (2021).
- [116] Michael R Garey and David S Johnson. *Computers and intractability*. Vol. 174. Freeman, San Francisco, 1979.
- [117] N. Garg, V.V. Vazirani, and M. Yannakakis. "Approximate max-flow min-(multi) cut theorems and their applications". In: *Proceedings of the twenty-fifth annual ACM symposium on Theory of computing*. ACM New York, NY, USA. 1993, pp. 698–707.
- [118] Naveen Garg, Vijay V. Vazirani, and Mihalis Yannakakis. "Multiway cuts in node weighted graphs". In: *Journal of Algorithms* 50.1 (2004), pp. 49–61.

- [119] Konstantinos Georgiou, Avner Magen, and Madhur Tulsiani. "Optimal Sherali-Adams gaps from pairwise independence". In: *Proceedings of the 12th International Workshop on Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques (APPROX-RANDOM 2009)*. Springer, 2009, pp. 125–139. doi: 10.1007/978-3-642-03685-9_10.
- [120] Shayan Oveis Gharan, Amin Saberi, and Mohit Singh. "A randomized rounding approach to the traveling salesman problem". In: *2011 IEEE 52nd Annual Symposium on Foundations of Computer Science*. IEEE, 2011, pp. 550–559.
- [121] Michel X Goemans, Neil Olver, Thomas Rothvoß, and Rico Zenklusen. "Matroids and integrality gaps for hypergraphic steiner tree relaxations". In: *Proceedings of the forty-fourth annual ACM symposium on Theory of computing*. 2012, pp. 1161–1176.
- [122] Michel X Goemans and David P Williamson. "Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming". In: *Journal of the ACM* 42.6 (1995), pp. 1115–1145.
- [123] Michel X Goemans and David P Williamson. "The primal-dual method for approximation algorithms and its application to network design problems". In: *Approximation algorithms for NP-hard problems* (1997), pp. 144–191.
- [124] Michel X Goemans and David P Williamson. "The primal-dual method for approximation algorithms and its application to network design problems". In: *Approximation algorithms for NP-hard problems* (1997), pp. 144–191.
- [125] Olivier Goldschmidt and Dorit S Hochbaum. "A polynomial algorithm for the k-cut problem for fixed k". In: *Mathematics of operations research* 19.1 (1994), pp. 24–37.
- [126] Ralph E. Gomory and Tien Chung Hu. "Multi-terminal network flows". In: *Journal of the Society for Industrial and Applied Mathematics* 9.4 (1961), pp. 551–570.
- [127] Ronald L Graham. "Bounds for certain multiprocessing anomalies". In: *Bell system technical journal* 45.9 (1966), pp. 1563–1581.
- [128] Ronald L. Graham. "Bounds on multiprocessing timing anomalies". In: *SIAM journal on Applied Mathematics* 17.2 (1969), pp. 416–429.
- [129] Fabrizio Grandoni, Tobias Mömke, Andreas Wiese, and Hang Zhou. "A $(5/3 + \epsilon)$ -approximation for unsplittable flow on a path: placing small tasks into boxes". In: *Proceedings of the 50th Annual ACM SIGACT Symposium on Theory of Computing*. 2018, pp. 607–619.

- [130] Martin Grötschel, László Lovász, and Alexander Schrijver. *Geometric Algorithms and Combinatorial Optimization*. Vol. 2. Algorithms and Combinatorics. Springer-Verlag, 1988.
- [131] Sudipto Guha and Samir Khuller. “Greedy strikes back: Improved facility location algorithms”. In: *Journal of algorithms* 31.1 (1999), pp. 228–248.
- [132] Anupam Gupta and Jochen Könemann. “Approximation algorithms for network design: A survey”. In: *Surveys in Operations Research and Management Science* 16.1 (2011), pp. 3–20.
- [133] Anupam Gupta, Viswanath Nagarajan, and R. Ravi. “An improved approximation algorithm for requirement cut”. In: *Operations Research Letters* 38.4 (2010), pp. 322–325. doi: 10.1016/j.orl.2010.02.009.
- [134] Anupam Gupta, Ilan Newman, Yuri Rabinovich, and Alistair Sinclair. “Cuts, trees and ℓ_1 -embeddings of graphs”. In: *Combinatorica* 24.2 (2004), pp. 233–269.
- [135] Anupam Gupta and Kanat Tangwongsan. *Simpler Analyses of Local Search Algorithms for Facility Location*. 2008. arXiv: 0809.2554 [cs.DS].
- [136] Anupam Gupta and Vera Traub. “Steiner Forest: A Simplified Better-Than-2 Approximation”. In: *Proceedings of the 58th ACM Symposium on Theory of Computing (STOC 2026)*. To appear. 2026. doi: 10.48550/arXiv.2511.18460.
- [137] Venkatesan Guruswami, Sanjeev Khanna, Rajmohan Rajaraman, F. Bruce Shepherd, and Mihalis Yannakakis. “Near-optimal hardness results and approximation algorithms for edge-disjoint paths and related problems”. In: *Proceedings of the Thirty-First Annual ACM Symposium on Theory of Computing*. 1999, pp. 19–28. doi: 10.1145/301250.301262.
- [138] Bernhard Haeupler, Barna Saha, and Aravind Srinivasan. “New constructive aspects of the Lovász local lemma”. In: *Journal of the ACM (JACM)* 58.6 (2011), pp. 1–28.
- [139] Magnús M. Halldórsson and Tigran Tonoyan. “Computing inductive vertex orderings”. In: *Information Processing Letters* 172 (2021), p. 106159. ISSN: 0020-0190. doi: <https://doi.org/10.1016/j.ipl.2021.106159>. URL: <https://www.sciencedirect.com/science/article/pii/S0020019021000740>.
- [140] Eran Halperin and Robert Krauthgamer. “Polylogarithmic inapproximability”. In: *Proceedings of the thirty-fifth annual ACM symposium on Theory of computing*. 2003, pp. 585–594.
- [141] Sariel Har-Peled and Manor Mendel. “Fast construction of nets in low-dimensional metrics and their applications”. In: *SIAM Journal on Computing* 35.5 (2006), pp. 1148–1184.

- [142] Sarel Har-Peled and Benjamin Raichel. “Net and prune: A linear time algorithm for euclidean distance problems”. In: *Journal of the ACM (JACM)* 62.6 (2015), pp. 1–35.
- [143] Sarel Har-Peled. *Concentration of Random Variables — Chernoff’s Inequality*. Available at https://sarielhp.org/teach/13/b_574_rand_alg/lec/07_chernoff.pdf.
- [144] Johan Hastad. “Clique is hard to approximate within $n^{1-\epsilon}$ ”. In: *Proceedings of 37th Conference on Foundations of Computer Science*. IEEE. 1996, pp. 627–636.
- [145] Michael Held and Richard M Karp. “The traveling-salesman problem and minimum spanning trees”. In: *Operations Research* 18.6 (1970), pp. 1138–1162.
- [146] Tsuyoshi Hirayama, Yuhao Liu, Kazuhisa Makino, Ke Shi, and Chao Xu. “A polynomial time algorithm for finding a minimum 4-partition of a submodular function”. In: *Mathematical Programming* 207.1 (2024), pp. 717–732.
- [147] Rebecca Hoberg and Thomas Rothvoss. “A logarithmic additive integrality gap for bin packing”. In: *Proceedings of the Twenty-Eighth Annual ACM-SIAM Symposium on Discrete Algorithms*. SIAM. 2017, pp. 2616–2625.
- [148] Dorit S Hochbaum and David B Shmoys. “A polynomial approximation scheme for scheduling on uniform processors: Using the dual approximation approach”. In: *SIAM journal on computing* 17.3 (1988), pp. 539–551.
- [149] Dorit S Hochbaum and David B Shmoys. “A unified approach to approximation algorithms for bottleneck problems”. In: *Journal of the ACM (JACM)* 33.3 (1986), pp. 533–550.
- [150] Ian Holyer. “The NP-completeness of edge-coloring”. In: *SIAM Journal on computing* 10.4 (1981), pp. 718–720.
- [151] Shlomo Hoory, Nathan Linial, and Avi Wigderson. “Expander graphs and their applications”. In: *Bulletin of the American Mathematical Society* 43.4 (2006), pp. 439–561.
- [152] Ellis Horowitz and Sartaj Sahni. “Exact and approximate algorithms for scheduling nonidentical processors”. In: *Journal of the ACM (JACM)* 23.2 (1976), pp. 317–327.
- [153] Wen-Lian Hsu and George L Nemhauser. “Easy and hard bottleneck location problems”. In: *Discrete Applied Mathematics* 1.3 (1979), pp. 209–215.

- [154] Makoto Imase and Bernard M Waxman. "Dynamic Steiner tree problem". In: *SIAM Journal on Discrete Mathematics* 4.3 (1991), pp. 369–384.
- [155] Kamal Jain. "A factor 2 approximation algorithm for the generalized Steiner network problem". In: *Combinatorica* 21.1 (2001), pp. 39–60.
- [156] Kamal Jain and Vijay V Vazirani. "Approximation algorithms for metric facility location and k-median problems using the primal-dual schema and Lagrangian relaxation". In: *Journal of the ACM (JACM)* 48.2 (2001), pp. 274–296.
- [157] Ragesh Jaiswal, Amit Kumar, and Sandeep Sen. "A simple D 2-sampling based PTAS for k-means and other clustering problems". In: *Algorithmica* 70.1 (2014), pp. 22–46.
- [158] Klaus Jansen. "An EPTAS for scheduling jobs on uniform processors: using an MILP relaxation with a constant number of integral variables". In: *SIAM Journal on Discrete Mathematics* 24.2 (2010), pp. 457–485.
- [159] Klaus Jansen. "Parameterized approximation scheme for the multiple knapsack problem". In: *SIAM Journal on Computing* 39.4 (2010), pp. 1392–1412.
- [160] Klaus Jansen and Lars Rohwedder. "A quasi-polynomial approximation for the restricted assignment problem". In: *International Conference on Integer Programming and Combinatorial Optimization*. Springer. 2017, pp. 305–316.
- [161] Klaus Jansen and Lars Rohwedder. "On the configuration-lp of the restricted assignment problem". In: *Proceedings of the Twenty-Eighth Annual ACM-SIAM Symposium on Discrete Algorithms*. SIAM. 2017, pp. 2670–2678.
- [162] David S Johnson. "Approximation algorithms for combinatorial problems". In: *Journal of computer and system sciences* 9.3 (1974), pp. 256–278.
- [163] William B. Johnson and Joram Lindenstrauss. "Extensions of Lipschitz mappings into a Hilbert space". In: *Conference in Modern Analysis and Probability*. Vol. 26. Contemporary Mathematics. American Mathematical Society, 1984, pp. 189–206.
- [164] EG Co man Jr, MR Garey, and DS Johnson. "Approximation algorithms for bin packing: A survey". In: *Approximation algorithms for NP-hard problems* (1996), pp. 46–93.
- [165] Frank Kammer and Torsten Tholey. "Approximation algorithms for intersection graphs". In: *Algorithmica* 68.2 (2014), pp. 312–336.
- [166] George Karakostas. "A better approximation ratio for the vertex cover problem". In: *ACM Transactions on Algorithms (TALG)* 5.4 (2009), p. 41.

- [167] David Karger, Rajeev Motwani, and Madhu Sudan. "Approximate graph coloring by semidefinite programming". In: *Journal of the ACM (JACM)* 45.2 (1998), pp. 246–265. DOI: 10.1145/274787.274791.
- [168] David R Karger, Philip Klein, Cliff Stein, Mikkel Thorup, and Neal E Young. "Rounding algorithms for a geometric embedding of minimum multiway cut". In: *Proceedings of the thirty-first annual ACM symposium on Theory of Computing*. 1999, pp. 668–678.
- [169] Anna R Karlin, Nathan Klein, and Shayan Oveis Gharan. "A (slightly) improved approximation algorithm for metric TSP". In: *Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing*. 2021, pp. 32–45.
- [170] Narendra Karmarkar and Richard M Karp. "An efficient approximation scheme for the one-dimensional bin-packing problem". In: *23rd Annual Symposium on Foundations of Computer Science (sfcs 1982)*. IEEE. 1982, pp. 312–320.
- [171] Richard M. Karp. "Reducibility among combinatorial problems". In: *Complexity of Computer Computations*. Ed. by Raymond E. Miller, James W. Thatcher, and Jean D. Bohlinger. Plenum Press, 1972, pp. 85–103.
- [172] Ken-ichi Kawarabayashi and Anastasios Sidiropoulos. "Embeddings of Planar Quasimetrics into Directed ℓ_1 and Polylogarithmic Approximation for Directed Sparsest-Cut". In: *Proceedings of IEEE FOCS (2021)*. To appear. 2021.
- [173] H. Kellerer, H.K.U.P.D. Pisinger, U. Pferschy, and D. Pisinger. *Knapsack Problems*. Springer Nature Book Archives Millennium. Springer, 2004. ISBN: 9783540402862. URL: <https://books.google.com/books?id=u5DB7gck08YC>.
- [174] Subhash Khot. "On the power of unique 2-prover 1-round games". In: *Proceedings of the 34th Annual ACM Symposium on Theory of Computing (STOC)*. 2002, pp. 767–775.
- [175] Subhash Khot, Guy Kindler, Elchanan Mossel, and Ryan O'Donnell. "Optimal Inapproximability Results for MAX-CUT and Other 2-Variable CSPs?" In: *SIAM Journal on Computing* 37.1 (2007), pp. 319–357. DOI: 10.1137/S0097539705447372. eprint: <https://doi.org/10.1137/S0097539705447372>. URL: <https://doi.org/10.1137/S0097539705447372>.
- [176] Subhash Khot and Oded Regev. "Vertex cover might be hard to approximate to within $2 - \epsilon$ ". In: *Journal of Computer and System Sciences* 74.3 (2008), pp. 335–349.

- [177] Philip Klein, Serge A Plotkin, and Satish Rao. "Excluded minors, network decomposition, and multicommodity flow". In: *Proceedings of the twenty-fifth annual ACM symposium on Theory of computing*. 1993, pp. 682–690.
- [178] Philip Klein, Serge A Plotkin, and Satish Rao. "Excluded minors, network decomposition, and multicommodity flow". In: *Proceedings of the twenty-fifth annual ACM symposium on Theory of computing*. 1993, pp. 682–690.
- [179] Philip N Klein, Serge A Plotkin, Satish Rao, and Eva Tardos. "Approximation algorithms for Steiner and directed multicuts". In: *Journal of Algorithms* 22.2 (1997), pp. 241–269.
- [180] Stavros G Kolliopoulos and Neal E Young. "Approximation algorithms for covering/packing integer programs". In: *Journal of Computer and System Sciences* 71.4 (2005), pp. 495–505.
- [181] Guy Kortsarz and Zeev Nutov. "Approximating minimum cost connectivity problems". In: *Parameterized complexity and approximation algorithms*. Ed. by Erik D. Demaine, MohammadTaghi Hajiaghayi, and Dániel Marx. Dagstuhl Seminar Proceedings 09511. Dagstuhl, Germany: Schloss Dagstuhl - Leibniz-Zentrum fuer Informatik, Germany, 2010. URL: <http://drops.dagstuhl.de/opus/volltexte/2010/2497>.
- [182] Madhukar R Korupolu, C Greg Plaxton, and Rajmohan Rajaraman. "Analysis of a local search heuristic for facility location problems". In: *Journal of algorithms* 37.1 (2000), pp. 146–188.
- [183] Ravishankar Krishnaswamy, Amit Kumar, Viswanath Nagarajan, Yogish Sabharwal, and Barna Saha. "The matroid median problem". In: *Proceedings of the twenty-second annual ACM-SIAM symposium on Discrete Algorithms*. SIAM. 2011, pp. 1117–1130.
- [184] Jean B. Lasserre. "Global optimization with polynomials and the problem of moments". In: *SIAM Journal on Optimization* 11.3 (2001), pp. 796–817.
- [185] Lap Chi Lau, Ramamoorthi Ravi, and Mohit Singh. *Iterative methods in combinatorial optimization*. Vol. 46. Cambridge University Press, 2011.
- [186] Monique Laurent. "Sums of squares, moment matrices and optimization over polynomials". In: *Emerging Applications of Algebraic Geometry, IMA Volumes in Mathematics and its Applications* 149 (2009), pp. 157–270.
- [187] Eugene L Lawler. "Fast approximation algorithms for knapsack problems". In: *Mathematics of Operations Research* 4.4 (1979), pp. 339–356.
- [188] Jon Lee, Maxim Sviridenko, and Jan Vondrák. "Matroid matching: the power of local search". In: *SIAM Journal on Computing* 42.1 (2013), pp. 357–379.

- [189] T. Leighton and S. Rao. "Multicommodity max-flow min-cut theorems and their use in designing approximation algorithms". In: *Journal of the ACM (JACM)* 46.6 (1999). Conference version is from 1988., pp. 787–832.
- [190] Tom Leighton, Satish Rao, and Aravind Srinivasan. "Multicommodity flow and circuit switching". In: *Proceedings of the Thirty-First Hawaii International Conference on System Sciences*. Vol. 7. IEEE. 1998, pp. 459–465.
- [191] Jan Karel Lenstra, David B Shmoys, and Éva Tardos. "Approximation algorithms for scheduling unrelated parallel machines". In: *Mathematical programming* 46.1 (1990), pp. 259–271.
- [192] Michael Lewin, Dror Livnat, and Uri Zwick. "Improved Rounding Techniques for the MAX 2-SAT and MAX DI-CUT Problems". In: *Proceedings of the 9th International IPCO Conference*. Springer, 2002, pp. 67–82.
- [193] Shi Li. "A 1.488 approximation algorithm for the uncapacitated facility location problem". In: *Information and Computation* 222 (2013), pp. 45–58.
- [194] Nathan Linial, Eran London, and Yuri Rabinovich. "The geometry of graphs and some of its algorithmic applications". In: *Combinatorica* 15.2 (1995), pp. 215–245.
- [195] László Lovász. "A characterization of perfect graphs". In: *Journal of Combinatorial Theory, Series B* 13.2 (1972), pp. 95–98.
- [196] László Lovász. "On the Shannon capacity of a graph". In: *IEEE Transactions on Information Theory* 25.1 (1979), pp. 1–7.
- [197] Meena Mahajan, Prajakta Nimbhorkar, and Kasturi Varadarajan. "The planar k-means problem is NP-hard". In: *Theoretical Computer Science* 442 (2012), pp. 13–21.
- [198] S. Mahajan and H. Ramesh. "Derandomizing semidefinite programming based approximation algorithms". In: *Proceedings of IEEE 36th Annual Foundations of Computer Science*. 1995, pp. 162–169. doi: 10.1109/SFCS.1995.492473.
- [199] Rajsekar Manokaran, Joseph Naor, Prasad Raghavendra, and Roy Schwartz. "SDP Gaps and UGC-Hardness for Multiway Cut, 0-Extension, and Metric Labeling". In: *Proceedings of the 40th Annual ACM Symposium on Theory of Computing (STOC)*. 2008, pp. 11–20.
- [200] Pasin Manurangsi. "Inapproximability of maximum biclique problems, minimum k-cut and densest at-least-k-subgraph from the small set expansion hypothesis". In: *Algorithms* 11.1 (2018), p. 10.
- [201] G.A. Margulis. "Explicit constructions of expanders". In: *Problemy Peredaci Informacii* 9.4 (1973), pp. 71–80.

- [202] Julián Mestre. “Greedy in approximation algorithms”. In: *European Symposium on Algorithms*. Springer, 2006, pp. 528–539.
- [203] Joseph Mitchell. “Approximating maximum independent set for rectangles in the plane”. In: *SIAM Journal on Computing* (2024), FOCS21–299.
- [204] Michael Mitzenmacher and Eli Upfal. *Probability and computing: Randomization and probabilistic techniques in algorithms and data analysis*. Cambridge university press, 2017.
- [205] Sarah Morell and Martin Skutella. “Single source unsplittable flows with arc-wise lower and upper bounds”. In: *Mathematical Programming* (2021), pp. 1–20.
- [206] Robin A Moser and Gábor Tardos. “A constructive proof of the general Lovász local lemma”. In: *Journal of the ACM (JACM)* 57.2 (2010), pp. 1–15.
- [207] Dana Moshkovitz. “The Projection Games Conjecture and the NP-Hardness of $\ln n$ -Approximating Set-Cover”. In: *Theory of Computing* 11.1 (2015), pp. 221–235.
- [208] Rajeev Motwani and Prabhakar Raghavan. *Randomized algorithms*. Cambridge university press, 1995.
- [209] Theodore S. Motzkin. “The arithmetic-geometric inequality”. In: *Inequalities (Proc. Sympos. Wright-Patterson Air Force Base, Ohio, 1965)*, Academic Press, New York, pp. 205–224. 1967.
- [210] Viswanath Nagarajan, R Ravi, and Mohit Singh. “Simpler analysis of LP extreme points for traveling salesman and survivable network design problems”. In: *Operations Research Letters* 38.3 (2010), pp. 156–160.
- [211] Viswanath Nagarajan, Baruch Schieber, and Hadas Shachnai. “The Euclidean k -supplier problem”. In: *Mathematics of Operations Research* 45.1 (2020), pp. 1–14.
- [212] Joseph Naor and Leonard Zosin. “A 2-approximation algorithm for the directed multiway cut problem”. In: *SIAM Journal on Computing* 31.2 (2001), pp. 477–482.
- [213] George L Nemhauser, Laurence A Wolsey, and Marshall L Fisher. “An analysis of approximations for maximizing submodular set functions—I”. In: *Mathematical Programming* 14.1 (1978), pp. 265–294.
- [214] Zeev Nutov. “Node-connectivity survivable network problems”. In: *Handbook of Approximation Algorithms and Metaheuristics*. Chapman and Hall/CRC, 2018, pp. 233–253.

- [215] Rafail Ostrovsky, Yuval Rabani, Leonard J Schulman, and Chaitanya Swamy. "The effectiveness of Lloyd-type methods for the k-means problem". In: *Journal of the ACM (JACM)* 59.6 (2013), pp. 1–22.
- [216] Pablo A. Parrilo. "Structured Semidefinite Programs and Semialgebraic Geometry Methods in Robustness and Optimization". See also: Semidefinite programming relaxations for semialgebraic problems, *Mathematical Programming*, 96(2):293–320, 2003. PhD thesis. California Institute of Technology, 2000.
- [217] Paul Paschmanns and Vera Traub. "The Bidirected Cut Relaxation for Steiner Tree: Better Integrality Gap Bounds and the Limits of Moat Growing". In: *arXiv preprint arXiv:2602.19879* (2026). doi: 10.48550/arXiv.2602.19879.
- [218] Ján Plesník. "A heuristic for the p-center problems in graphs". In: *Discrete Applied Mathematics* 17.3 (1987), pp. 263–268.
- [219] Yuri Rabinovich. "On average distortion of embedding metrics into the line". In: *Discrete & Computational Geometry* 39.4 (2008), pp. 720–733.
- [220] Prabhakar Raghavan. "Probabilistic construction of deterministic algorithms: approximating packing integer programs". In: *Journal of Computer and System Sciences* 37.2 (1988), pp. 130–143.
- [221] Prabhakar Raghavan and Clark D Tompson. "Randomized rounding: a technique for provably good algorithms and algorithmic proofs". In: *Combinatorica* 7.4 (1987), pp. 365–374.
- [222] Prasad Raghavendra. "Optimal algorithms and inapproximability results for every CSP?" In: *Proceedings of the 40th Annual ACM Symposium on Theory of Computing (STOC)*. 2008, pp. 245–254.
- [223] Sridhar Rajagopalan and Vijay V. Vazirani. "On the bidirected cut relaxation for the metric Steiner tree problem". In: *Proceedings of the Tenth Annual ACM-SIAM Symposium on Discrete Algorithms*. 1999. doi: 10.1145/314500.314909.
- [224] Satish Rao. "Small distortion and volume preserving embeddings for planar and Euclidean metrics". In: *Proceedings of the fifteenth annual symposium on Computational geometry*. 1999, pp. 300–306.
- [225] Neil Robertson and Paul D. Seymour. "Graph Minors. XIII. The disjoint paths problem". In: *Journal of Combinatorial Theory, Series B* 63.1 (1995), pp. 65–110. doi: 10.1006/jctb.1995.1006.
- [226] Gabriel Robins and Alexander Zelikovsky. "Tighter bounds for graph Steiner tree approximation". In: *SIAM Journal on Discrete Mathematics* 19.1 (2005), pp. 122–134.

- [227] Sartaj Sahni and Teofilo Gonzalez. "P-complete approximation problems". In: *Journal of the ACM (JACM)* 23.3 (1976), pp. 555–565.
- [228] Michael Saks*, Alex Samorodnitsky, and Leonid Zosin. "A lower bound on the integrality gap for minimum multicut in directed networks". In: *Combinatorica* 24.3 (2004), pp. 525–530.
- [229] Huzur Saran and Vijay V Vazirani. "Finding k cuts within twice the optimal". In: *SIAM Journal on Computing* 24.1 (1995), pp. 101–108.
- [230] Alexander Schrijver. *Combinatorial optimization: polyhedra and efficiency*. Vol. 24. Springer Science & Business Media, 2003.
- [231] Paul D. Seymour. "Packing directed circuits fractionally". In: *Combinatorica* 15.2 (1995), pp. 281–288.
- [232] Farhad Shahrokhi and David W Matula. "The maximum concurrent flow problem". In: *Journal of the ACM (JACM)* 37.2 (1990), pp. 318–334.
- [233] Ankit Sharma and Jan Vondrák. "Multiway cut, pairwise realizable distributions, and descending thresholds". In: *Proceedings of the forty-sixth annual ACM symposium on theory of computing*. 2014, pp. 724–733.
- [234] F. Bruce Shepherd. "The theta body and imperfection". In: *Perfect Graphs*. Ed. by Jorge L. Ramírez-Alfonsín and Bruce A. Reed. Wiley, 2001, pp. 261–291.
- [235] David B Shmoys. "Cut problems and their application to divide-and-conquer". In: *Approximation algorithms for NP-hard problems* (1997), pp. 192–235.
- [236] David B Shmoys and Éva Tardos. "An approximation algorithm for the generalized assignment problem". In: *Mathematical programming* 62.1 (1993), pp. 461–474.
- [237] Martin Skutella. "A note on the ring loading problem". In: *SIAM Journal on Discrete Mathematics* 30.1 (2016), pp. 327–342.
- [238] Aravind Srinivasan. "An extension of the Lovász Local Lemma, and its applications to integer programming". In: *SIAM Journal on Computing* 36.3 (2006), pp. 609–634.
- [239] Larry Stockmeyer. "Planar 3-colorability is Polynomial Complete". In: *SIGACT News* 5.3 (July 1973), pp. 19–25. ISSN: 0163-5700. DOI: 10.1145/1008293.1008294. URL: <http://doi.acm.org/10.1145/1008293.1008294>.
- [240] Ola Svensson. "Approximating ATSP by relaxing connectivity". In: *2015 IEEE 56th Annual Symposium on Foundations of Computer Science*. IEEE. 2015, pp. 1–19.

- [241] Ola Svensson. “Santa claus schedules jobs on unrelated machines”. In: *SIAM Journal on Computing* 41.5 (2012), pp. 1318–1341.
- [242] Ola Svensson, Jakub Tarnawski, and László A Végh. “A constant-factor approximation algorithm for the asymmetric traveling salesman problem”. In: *Journal of the ACM (JACM)* 67.6 (2020), pp. 1–53.
- [243] Chaitanya Swamy. “Improved approximation algorithms for matroid and knapsack median problems and applications”. In: *ACM Transactions on Algorithms (TALG)* 12.4 (2016), pp. 1–22.
- [244] Hiromitsu Takahashi and A Matsuyama. “An approximate solution for the Steiner problem in graphs”. In: *Math. Jap.* 24.6 (1980), pp. 573–577.
- [245] Johan Thapper and Stanislav Živný. “The power of Sherali–Adams relaxations for general-valued CSPs”. In: *SIAM Journal on Computing* 46.4 (2017), pp. 1241–1279. DOI: 10.1137/16M1079245.
- [246] Robin Thomas. “An update on the four-color theorem”. In: *Notices of the AMS* 45.7 (1998), pp. 848–859.
- [247] Vera Traub and Jens Vygen. “An improved approximation algorithm for the asymmetric traveling salesman problem”. In: *SIAM Journal on Computing* 51.1 (2022), pp. 139–173. DOI: 10.1137/20M1339313.
- [248] Vera Traub and Jens Vygen. *Approximation Algorithms for Traveling Salesman Problems*. Cambridge University Press, 2024. ISBN: 9781009445412. DOI: 10.1017/9781009445436.
- [249] Sergei Vassilvitskii and David Arthur. “k-means++: The advantages of careful seeding”. In: *Proceedings of the eighteenth annual ACM-SIAM symposium on Discrete algorithms*. 2006, pp. 1027–1035.
- [250] Vijay V Vazirani. *Approximation algorithms*. Springer Science & Business Media, 2013.
- [251] Robert Vicari. “Simplex based Steiner tree instances yield large integrality gaps for the bidirected cut relaxation”. In: *arXiv preprint arXiv:2002.07912* (2020).
- [252] Jens Vygen. “Better approximation guarantee for Asymmetric TSP”. In: *arXiv preprint arXiv:2603.14334* (2026). DOI: 10.48550/arXiv.2603.14334.
- [253] Douglas Brent West et al. *Introduction to graph theory*. Vol. 2. Prentice hall Upper Saddle River, 2001.
- [254] Avi Wigderson. “Improving the performance guarantee for approximate graph coloring”. In: *Journal of the ACM* 30.4 (1983), pp. 729–735. DOI: 10.1145/2157.322385.

- [255] David P Williamson and David B Shmoys. *The design of approximation algorithms*. Cambridge university press, 2011.
- [256] David P. Williamson. “On the design of approximation algorithms for a class of graphs problems”. PhD thesis. Cambridge, MA: MIT, Sept. 1993.
- [257] David P. Williamson, Michel X. Goemans, Milena Mihail, and Vijay V. Vazirani. “A Primal-Dual Approximation Algorithm for Generalized Steiner Network Problems”. In: *Combinatorica* 15 (1995), pp. 435–454. doi: <http://dx.doi.org/10.1007/BF01299747>.
- [258] Laurence A Wolsey. “An analysis of the greedy algorithm for the submodular set covering problem”. In: *Combinatorica* 2.4 (1982), pp. 385–393.
- [259] Yuli Ye and Allan Borodin. “Elimination graphs”. In: *ACM Transactions on Algorithms (TALG)* 8.2 (2012), pp. 1–23.
- [260] Alexander Z Zelikovsky. “An 11/6-approximation algorithm for the network Steiner problem”. In: *Algorithmica* 9.5 (1993), pp. 463–470.
- [261] Liang Zhao, Hiroshi Nagamochi, and Toshihide Ibaraki. “Greedy splitting algorithms for approximating multiway partition problems”. In: *Mathematical Programming* 102.1 (2005), pp. 167–183.
- [262] David Zuckerman. “Linear Degree Extractors and the Inapproximability of Max Clique and Chromatic Number”. In: *Theory of Computing* 3.1 (2007), pp. 103–128. doi: [10.4086/toc.2007.v003a006](https://doi.org/10.4086/toc.2007.v003a006).

Appendix A

Basic Feasible Solutions to LPs and the Rank Lemma

We discuss the rank lemma about vertex solutions for linear programs. Recall that a *polyhedron* in $\mathbb{R}^n$ is defined as the intersection of finite collection of half spaces. Without loss of generality we can assume that it is defined by a system of inequalities of the form $Ax \leq b$ where A is a $m \times n$ matrix and b is a $m \times 1$ vector. A polyhedron P is bounded if P is contained in finite radius ball around the origin. A *polytope* in $\mathbb{R}^n$ is defined as the convex hull of a finite collection of points. A fundamental theorem about linear programming states that any bounded polyhedron is a polytope. If the polyhedron is not bounded then it can be expressed as the Minkowski sum of a polytope and a cone.

A bounded polyhedron P in $\mathbb{R}^n$ defined by a system $Ax \leq b$ must necessarily have $m \geq n$. A point $p \in P$ is a basic feasible solution or a vertex solution of the system if it is the unique solution to a system $A'y = b'$ where A' is a sub-matrix of A with n inequalities and the rank of A' is equal to n . The inequalities in A' are said to be *tight* for y . Note that there may be many other inequalities in $Ax \leq b$ that are tight at y and in general there may be many different rank n sub-matrices that give rise to the same basic feasible solution y .

Lemma A.1. *Suppose y is a basic feasible solution of a system $Ax \leq b, \ell \leq x \leq u$ where A is a $m \times n$ matrix and ℓ and u are vectors defining lower and upper bounds on the variables $x \in \mathbb{R}^n$. Let $S = \{i : \ell_i < y_i < u_i\}$ be the set of indices of “fractional” variables in y . Then $|S| \leq \text{rank}(A) \leq m$. In particular the number of fractional variables in y is at most the number of “non-trivial” constraints (those that are defined by A).*

An extension of the previous lemma is often useful when the system defining the polyhedron has equality constraints.

Corollary A.1. *Suppose y is a basic feasible solution of a system $Ax \leq b, Cx = d, \ell \leq x \leq u$ where A is a $m \times n$ matrix, C is a $m' \times n$ matrix, and ℓ and u are vectors defining lower and upper bounds on the variables $x \in \mathbb{R}^n$. Let $S = \{i : \ell_i < y_i < u_i\}$ be the set of indices of “fractional” variables in y . Then $|S| \leq \text{rank}(A, C) \leq m + m'$.*

A special case of the preceding corollary is called the rank lemma in [185].

The lemmas are a simple consequence of the definition of basic feasible solution. We will focus on the proof of Lemma A.1. It is interesting only when $\text{rank}(A)$ or m is smaller than n , otherwise the claim is trivial. Before we prove it formally we observe some simple corollaries. Suppose we have a system $Ax \leq b, x \geq 0$ where $m < n$. Then the number of non-zero variables in a basic feasible solution is at most m . Similarly if the system is $Ax \leq b, x \in [0, 1]^n$ then the number of non-integer variables in y is at most m . For example in the knapsack LP we have $m = 1$ and hence in any basic feasible solution there can only be one fractional variable.

Now for the proof. We consider the system $Ax \leq b, -x \leq -\ell, x \leq u$ as a single system $Cx \leq d$ which has $m + 2n$ inequalities. Since y is a basic feasible solution to this system, from the definition, it is the unique solution of sub-system $C'x = d'$ where C' is a $n \times n$ full-rank matrix. How many rows of C' can come from A ? At most $\text{rank}(A) \leq m$ rows. It means that the rest of the rows of C' are from the other set of inequalities $-x \leq \ell$ or $x \leq u$. There are at least $n - \text{rank}(A)$ such rows which are tight at y . Thus $n - \text{rank}(A)$ variables in y are tight at lower or upper bounds and hence there can only be $\text{rank}(A)$ fractional variables in y .

See [185] for iterated rounding based methodology for exact and approximation algorithms. The whole methodology relies on properties of basic feasible solutions to LP relaxations of combinatorial optimization problems.

A.0.1 Some Examples

We give some examples to illustrate the utility of the rank lemma in the context of LP relaxations that arise in approximation algorithms.

Knapsack: The natural LP relaxation for this of the form $\max \sum_{i=1}^n w_i x_i$ subject to $x \in [0, 1]^n, \sum_{i=1}^n s_i x_i \leq 1$ consisting of a single non-trivial constraint. A basic feasible solution has at most 1 fractional variable. See Chapter 3.

Packing Integer Programs (PIPs): The LP relaxation is of the form $\max wx$ subject to $Ax \leq b, x \in [0, 1]^n$ where A is a $m \times n$ non-negative matrix. See Chapter 4. When $m = 1$ we have the Knapsack problem and hence the general problem is some-times referred to as the m -dimensional Knapsack problem, especially when m is a fixed constant. A basic feasible solution for the LP has at most m fractional variables. When m is a fixed constant one can exploit this after guessing the big items to obtain a PTAS.

Generalized Assignment: See Chapter 6.

A.0.2 Connection to Caratheodary's Theorem

Suppose we have n points $P = \{p_1, p_2, \dots, p_n\}$ in d -dimensional Euclidean space $\mathbb{R}^d$. A point $p \in \mathbb{R}^d$ is in the convex hull of P iff p is a convex combination of points in P . Formally, this means that there exist scalars $\lambda_1, \dots, \lambda_n \geq 0$ such that $\sum_i \lambda_i = 1$ and $p = \sum_i \lambda_i p_i$ (note that this is a vector sum). Caratheodary's theorem states that if p is in the convex hull of P then there is subset $P' \subseteq P$ such that p is in the convex hull of P' and $|P'| \leq d + 1$. One can prove Caratheodary's theorem directly but it is helpful to see it also as a consequence of the rank lemma. Consider the system of inequalities $\lambda_i \geq 0, 1 \leq i \leq n, \sum_i \lambda_i = 1, \sum_i \lambda_i p_i = p$ where the system $\sum_i \lambda_i p_i = p$ consists of d equalities. This system of inequalities in the variables $\lambda_1, \dots, \lambda_n$ is feasible by assumption (since p is in the convex hull of P). If we take any basic feasible solution of this system of inequalities we see that at most $d + 1$ of them are non-zero by the rank lemma.

One implication of Caratheodary's theorem in the context of combinatorial optimization is the following. Suppose we have a polytope P which is an LP relaxation of some combinatorial problem and let $x \in P$ be any feasible point. Then x can be written as a convex combination of at most $n + 1$ vertices of P where n is number of variables. Moreover, via the Ellipsoid method, one can find such a convex combination efficiently as long as one can optimize over P efficiently. As an example suppose $G = (V, E)$ is a graph and P is the spanning tree polytope of G (the vertices are the characteristic vectors of spanning trees) which is $\mathbb{R}^m$ ($m = |E|$). Then any fractional spanning tree $x \in P$ can be decomposed into at most $m + 1$ spanning trees.

In the context of approximation P is typically a relaxation of some hard combinatorial optimization problem. In such a case the vertices of P do not correspond to structures we are interested in. For example we can consider the minimum Steiner tree problem in a graph $G = (V, E)$ with terminal set $S \subseteq V$. There are several LP relaxations but perhaps the simplest one is the cut relaxation which has an integrality gap of 2. In such a case a feasible point $x \in P$ cannot be decomposed into convex combination of Steiner trees. However it can be shown that $2x$ dominates a convex combination of Steiner trees and such a convex combination can be found efficiently. It requires more technical work to precisely formalize this and we refer the reader to the work of Carr and Vempala [54] — you can also find a few applications of such decompositions in the same paper and it is a simple yet powerful tool to keep in mind.

Appendix B

Probabilistic Inequalities

The course will rely heavily on probabilistic methods. We will mostly rely on discrete probability spaces. We will keep the discussion high-level where possible and use certain results in a black-box fashion.

Let Ω be a finite set. A probability measure p assigns a non-negative number $p(\omega)$ for each $\omega \in \Omega$ such that $\sum_{\omega \in \Omega} p(\omega) = 1$. The tuple (Ω, p) defines a discrete probability space; an event in this space is any subset $A \subseteq \Omega$ and the probability of an event is simply $p(A) = \sum_{\omega \in A} p(\omega)$. When Ω is a continuous space such as the interval $[0, 1]$ things get trickier and we need to talk about measure spaces σ -algebras over Ω ; we can only assign probability to certain subsets of Ω . We will not go into details since we will not need any formal machinery for what we do in this course.

An important definition is that of a *random variable*. We will focus only on real-valued random variables in this course. A random variable X in a probability space is a function $X : \Omega \rightarrow \mathbb{R}$. In the discrete setting the *expectation* of X , denoted by $\mathbf{E}[X]$, is defined as $\sum_{\omega \in \Omega} p(\omega)X(\omega)$. For continuous spaces $\mathbf{E}[X] = \int X(\omega)dp(\omega)$ with appropriate definition of the integral. The variance of X , denoted by $\text{Var}[X]$ or as σ_X^2 , is defined as $\mathbf{E}[(X - \mathbf{E}[X])^2]$. The standard deviation is σ_X , the square root of the variance.

Theorem B.1 (Markov's Inequality). *Let X be a non-negative random variable such that $\mathbf{E}_X$ is finite. Then for any $t > 0$, $\mathbf{P}[X \geq t] \leq \mathbf{E}[X]/t$.*

Proof. The proof is in some sense obvious, especially in the discrete case. Here is a sketch. Define a new random variable Y where $Y(\omega) = X(\omega)$ if $X(\omega) < t$ and $Y(\omega) = t$ if $X(\omega) \geq t$. Y is non-negative and $Y \leq X$ point-wise and hence

$\mathbf{E}[Y] \leq \mathbf{E}[X]$. We also see that:

$$\begin{aligned} \mathbf{E}[X] \geq \mathbf{E}[Y] &= \sum_{\omega: X(\omega) < t} X(\omega)p(\omega) + \sum_{\omega: X(\omega) \geq t} tp(\omega) \\ &\geq t \sum_{\omega: X(\omega) \geq t} p(\omega) \quad (\text{since } X \text{ is non-negative}) \\ &\geq t \mathbf{P}[X \geq t]. \end{aligned}$$

The continuous case follows by replacing sums by integrals. $\blacksquare$

Markov's inequality is tight under the assumption. It is useful to construct an example. The more information we have about a random variable the better we can bound its deviation from the expectation.

Theorem B.2 (Chebyshev's Inequality). *Let X be a random variable with $\mathbf{E}[X]$ and $\text{Var}[X]$ are finite. Then $\mathbf{P}[|X| \geq t] \leq \mathbf{E}[X^2]/t^2$ and $\mathbf{P}[|X - \mathbf{E}[X]| \geq t\sigma_X] \leq 1/t^2$.*

Proof. Consider the non-negative random variable $Y = X^2$. $\mathbf{P}[|X| \geq t] = \mathbf{P}[Y \geq t^2]$ and we apply Markov's inequality to the latter. The second inequality is similar by considering $Y = (X - \mathbf{E}[X])^2$. $\blacksquare$

Chernoff-Hoeffding Bounds: We will use several times various forms of the Chernoff-Hoeffding bounds that apply to a random variable that is a finite sum of bounded and *independent* random variables. There are several versions of these bounds. First we state a general bound that is applicable to non-negative random variables and is dimension-free in that it depends only the expectation rather than the number of variables.

Theorem B.3 (Chernoff-Hoeffding). *Let $X_1, X_2, \dots, X_n$ be independent binary random variables and let $a_1, a_2, \dots, a_n$ be coefficients in $[0, 1]$. Let $X = \sum_i a_i X_i$. Then*

- For any $\mu \geq \mathbf{E}[X]$ and any $\delta > 0$, $\mathbf{P}[X > (1 + \delta)\mu] \leq \left(\frac{e^\delta}{(1+\delta)^{(1+\delta)}}\right)^\mu$.
- For any $\mu \leq \mathbf{E}[X]$ and any $\delta > 0$, $\mathbf{P}[X < (1 - \delta)\mu] \leq e^{-\mu\delta^2/2}$.

The following corollary bounds the deviation from the mean in both directions.

Corollary B.4. *Under the conditions of Theorem B.3, the following hold:*

- If $\delta > 2e - 1$, $\mathbf{P}[X \geq (1 + \delta)\mu] \leq 2^{-(1+\delta)\mu}$.
- For any U there is a constant $c(U)$ such that for $0 < \delta < U$, $\mathbf{P}[X \geq (1 + \delta)\mu] \leq e^{-c(U)\delta^2\mu}$. In particular, combining with the lower tail bound,

$$\mathbf{P}[|X - \mu| \geq \delta\mu] \leq 2e^{-c(U)\delta^2\mu}.$$

We refer the reader to the standard books on randomized algorithms [208] and [204] for the derivation of the above bounds.

If we are interested only in the upper tail we also have the following bounds which show the dependence of μ on n to obtain an inverse polynomial probability.

Corollary B.5. *Under the conditions of Theorem B.3, there is a universal constant α such that for any $\mu \geq \max\{1, \mathbf{E}[X]\}$, and sufficiently large n and for $c \geq 1$, $\mathbf{P}[X > \frac{\alpha c \ln n}{\ln \ln n} \cdot \mu] \leq 1/n^c$. Similarly, there is a constant α such that for any $\epsilon > 0$, $\mathbf{P}[X \geq (1 + \epsilon)\mu + \alpha c \log n / \epsilon] \leq 1/n^c$.*

Remark B.1. If the X_i are in the range $[0, b]$ for some b not equal to 1 one can scale them appropriately and then use the standard bounds.

Some times we need to deal with random variables that are in the range $[-1, 1]$. Consider the setting where $X = \sum_i X_i$ where for each i , $X_i \in [-1, 1]$ and $\mathbf{E}[X_i] = 0$, and the X_i are independent. In this case $\mathbf{E}[X] = 0$ and we can no longer expect a dimension-free bound. Suppose each X_i is 1 with probability $1/2$ and -1 with probability $1/2$. Then $X = \sum_i X_i$ corresponds to a 1-dimensional random walk and even though the expected value is 0 the standard deviation of X is $\Theta(\sqrt{n})$. One can show that $\mathbf{P}[|X| \geq t\sqrt{n}] \leq 2e^{-t^2/2}$. For these settings we can use the following bounds.

Theorem B.6. *Let $X_1, X_2, \dots, X_n$ be independent random variables such that for each i , $X_i \in [a_i, b_i]$. Let $X = \sum_i X_i$ and let $\mu = \mathbf{E}[X]$. Then*

$$\mathbf{P}[|X - \mu| \geq t] \leq 2e^{-\frac{2t^2}{\sum_{i=1}^n (b_i - a_i)^2}}.$$

In particular if $b_i - a_i \leq 1$ for all i then

$$\mathbf{P}[|X - \mu| \geq t] \leq 2e^{-\frac{2t^2}{n}}.$$

Note that $\text{Var}[X] = \sum_i \text{Var}[X_i]$. One can show a bound based of the following form

$$\mathbf{P}[|X - \mu| \geq t] \leq 2e^{-\frac{t^2}{2(\sigma_X^2 + Mt/3)}}$$

where $|X_i| \leq M$ for all i .

Remark B.2. Compare the Chebyshev bound to the Chernoff-Hoeffding bounds for the same variance.

Sariel Har-Peled maintains a cheat sheet of Chernoff bounds and also has an interesting derivation. See his notes [143].

Statistical Estimators, Reducing Variance and Boosting: Randomized algorithms compute a function f of the input. In many cases they produce an unbiased estimator, via a random variable X , for the function value. That is, the algorithm will have the property that the $E[X]$ is the desired value. Note that the randomness is internal to the algorithm and not part of the input (we can also consider randomness in the input). Having an estimator is not often useful. We will also typically try to evaluate $\text{Var}[X]$ and then we can use Chebyshev's inequality. One way to reduce the variance of the estimate is to run the algorithm in parallel (with separate random bits) and get estimators $X_1, X_2, \dots, X_h$ and use $X = \frac{1}{h} \sum_i X_i$ as the final estimator. Note that $\text{Var}[X] = \frac{1}{h} \sum_i \text{Var}[X_i]$ since the X_i are independent. Thus the variance has been reduced by a factor of h . A different approach is to use the *median* value of $X_1, X_2, \dots, X_h$ as the final estimator. We can then use Chernoff-Hoeffding bounds to get a much better dependence on h . In fact both approaches can be combined.